Package 'jjstat'

October 21, 2026

Type Package

Title Statistical Analysis Tools for Writing Papers

Version 0.1.9

Author Park Joonghee

Maintainer The package maintainer <shoutjoy@hanmail.net>

Description More about what it does (maybe more than one line)
Use four spaces when indenting paragraphs within the Description.

License GPL-3

Encoding UTF-8

LazyData true

Imports dplyr ,
knitr ,
ggplot2 (>= 3.4.4),
magrittr (>= 2.0.3),
Hmisc,
ggrepel

Suggests broom (>= 1.0.5),
httpuv (>= 1.6.13),
rvest (>= 1.0.3),
tibble (>= 3.2.1),
kableExtra (>= 1.3.4),
testthat (>= 3.0.0),
psych,
gridExtra

Depends tidyverse,
lavaan,
broom,
plspm,
R (>= 2.10),

Config/roxygen2/version 8.1.1

R topics documented:

accent_table 12
add_bind_stat 13
add_col_ratio 14

add_cor . . . 15
add_id . . . 16
add_normality . . . 17
add_normality_kline . . . 18
add_partial_sum . . . 18
add_pca_scores . . . 19
add_prop . . . 21
add_ratio . . . 22
add_ratio_df . . . 23
add_rep_id . . . 24
add_rows . . . 25
add_rows_freq . . . 26
add_row_sum . . . 27
add_sig . . . 28
add_significance_symbols . . . 29
add_sig_star . . . 30
add_stat . . . 31
add_sum . . . 32
add_t_sig . . . 33
all_na_zero . . . 35
all_na_zero1 . . . 35
alpha_apa . . . 36
alpha_table . . . 36
anova_apa . . . 37
aov_apa . . . 38
aov_df . . . 39
aov_df2 . . . 41
aov_df_emmeans . . . 42
aov_post . . . 44
aov_post2 . . . 45
aov_rename . . . 46
aov_table . . . 47
apa_aov_df . . . 49
apa_aov_df_res . . . 50
aroian_sobel_test_general . . . 51
arrange_by_mind . . . 52
as_trt . . . 53
as_trt2 . . . 54
auto_coding . . . 55
ave_candidates_plot . . . 57
ave_raise_candidates . . . 59
bartlett_test . . . 60
binary_recode . . . 61
binary_recode_sep . . . 62
bind_alpha_table . . . 63
bind_compare_data . . . 64
bind_lme_model . . . 65
bind_mysummaryBy . . . 65
box_cox . . . 67
breslowdata_test . . . 68
calculate_chi_sig . . . 69
cal_vif . . . 70

centering . . . 71
cfa2 . . . 71
cfa_ml . . . 73
change_node_name . . . 74
Char2num . . . 77
Char2num_fct . . . 78
chisq.multcomp . . . 79
chisqTest . . . 79
chisq_apa . . . 80
chisq_contrast . . . 81
chisq_gof_posthoc . . . 82
chisq_gof_posthoc_apa . . . 83
chisq_multcomp2 . . . 84
chisq_pivot_table . . . 85
chisq_posthoc . . . 86
chisq_retest . . . 87
chisq_table_sum . . . 89
chisq_test . . . 90
chisq_test_By1 . . . 91
chisq_test_By2 . . . 92
chisq_test_By3 . . . 93
chisq_test_cell . . . 94
chisq_test_cor . . . 95
chisq_test_kge . . . 97
cmh_test . . . 99
col2row . . . 100
collapse_path . . . 101
collapse_path2 . . . 102
colnames_by_dict . . . 104
colnames_change . . . 105
colnames_change_df . . . 106
colnames_to_row . . . 107
col_capital . . . 107
col_to_colnames . . . 108
col_to_rownames . . . 109
combine_data . . . 110
combine_rows . . . 111
comma . . . 112
compare_bar . . . 113
compare_bind_data . . . 115
confint_mark_sig . . . 115
confint_plot_data . . . 116
confint_plot_lm . . . 117
confint_plot_sem . . . 118
contrast_msdn . . . 119
convert_min_max_scaler . . . 121
convert_scale_df . . . 121
Cor2cov . . . 122
count_ne . . . 123
Cov2cor . . . 124
cov_lm_sem . . . 125
cov_simdata . . . 127

cramers_v . . . 129
cronbach_alpha . . . 129
cronbach_from_cor . . . 130
cr_ave_from_lambda . . . 131
cr_candidates_plot . . . 132
cr_raise_candidates . . . 133
cut_print . . . 134
cut_print_all . . . 135
dall . . . 136
datapaste . . . 137
datapaste_text . . . 138
DescribeBy . . . 139
DE_effect . . . 140
diagram . . . 142
diagram2 . . . 143
diagram_model . . . 145
Diff_effect . . . 148
e . . . 149
EAI_index . . . 150
edit_layout . . . 152
effect1 . . . 152
factor_layout . . . 154
factor_scores . . . 155
fa_print . . . 156
fei_Chisq_test . . . 157
fei_dv_res . . . 158
filter_rows . . . 159
find_paths . . . 160
find_paths_cal . . . 161
format_col . . . 163
format_number . . . 163
Freq_rowcounter . . . 164
Freq_table . . . 165
Freq_table_apa . . . 167
Freq_table_colnames . . . 168
Freq_table_multi . . . 169
fs . . . 170
g . . . 171
general_Sobel_test . . . 171
GetCov . . . 172
goodman_sobel_test_general . . . 174
gtl . . . 175
gtl_df . . . 176
hist2 . . . 177
histogram . . . 177
HTMT . . . 178
htmt_mlsem . . . 179
htmt_sem . . . 180
ICC . . . 181
icc_batch . . . 182
icc_lme4 . . . 182
IE_effect . . . 184

indicator_validity_ml . . . 185
interlist . . . 185
interplot . . . 186
interplot_data . . . 189
interpretation_de . . . 190
interpretation_ie . . . 191
interpretation_ie_CI . . . 192
item . . . 193
item_CR_AVE . . . 193
item_parceling . . . 194
jjstat_plot_path . . . 197
jjstat_SEM . . . 199
jtl . . . 201
k . . . 201
kakaoi . . . 202
kge_ca_analysis . . . 202
kge_chisq_table . . . 203
kge_weigth . . . 205
kge_weigth_add_ca . . . 206
kge_weigth_ko . . . 207
KURT . . . 208
k_df . . . 209
LavaanPlot . . . 210
lav_change_node_name . . . 212
lav_de_hypo . . . 214
lav_edge_value . . . 215
lav_extract_imlist . . . 218
lav_extract_ind . . . 220
lav_extract_ind_paths . . . 222
lav_extract_int . . . 224
lav_extract_mm . . . 225
lav_extract_mm2 . . . 227
lav_extract_path . . . 228
lav_extract_sm . . . 229
lav_factor_layout . . . 231
lav_htmt . . . 232
lav_ind_hypo . . . 233
lav_interact_model . . . 234
lav_latentProd . . . 236
lav_matrix . . . 237
lav_modelfit . . . 239
lav_new_model . . . 240
lav_Pattern . . . 241
lav_remodel . . . 242
lav_return_blocks . . . 244
lav_return_data . . . 245
lav_return_model . . . 246
lav_rotate_resid . . . 247
lav_semPaths . . . 248
lav_semPaths2 . . . 251
layout_mat . . . 255
lcaGraphData . . . 256

lca_class_add . . . 257
lca_class_freq . . . 258
lca_class_plot . . . 258
lca_freq_plot . . . 259
lca_gamma . . . 260
lca_gof . . . 260
lca_GraphData . . . 261
lca_result . . . 262
lme_report . . . 263
long_df . . . 264
lowerdf . . . 265
lowerMat . . . 265
lowerMat_na . . . 266
lpa . . . 266
lpa_BIC_plot . . . 268
lpa_bind_classdata . . . 269
lpa_bind_classdata_tidyLPA . . . 270
lpa_create_profiles . . . 271
lpa_create_profiles2 . . . 272
lpa_explore_modelfit . . . 273
lpa_gmm . . . 274
lpa_plot . . . 275
lpa_profile_plot . . . 277
lpa_profile_plot2 . . . 279
lpa_profile_plot_summary . . . 279
LRT_apa . . . 280
LRT_apa2 . . . 281
make_c_text . . . 282
make_df_text . . . 282
make_label_list . . . 283
make_layout . . . 284
make_list_text . . . 285
make_mat_text . . . 286
make_mat_text2 . . . 288
make_mat_text_col . . . 289
make_text_df . . . 290
markdown_table . . . 293
mat_text_arrange . . . 295
mcnemar_test . . . 296
md . . . 297
meanCentering . . . 299
meanCentering_df . . . 300
mean_apa . . . 300
mean_bar . . . 301
med_effect . . . 302
metacorMat . . . 302
min_max_scaler . . . 306
missCheck . . . 306
missfillin . . . 307
missfillinFor . . . 308
mlsem_fit_indices . . . 309
mlsem_fornell_larcker . . . 311

mlsem_htmt . . . 311
mlsem_modelfit_paper . . . 312
mlsem_reliability . . . 314
modelfit_paper . . . 315
Modindices . . . 316
ModSEM . . . 317
move_col . . . 321
move_cols . . . 322
move_mat . . . 323
move_matpos . . . 324
move_mat_change . . . 325
move_mat_cut . . . 326
move_mat_nc . . . 327
move_mat_ori . . . 327
move_mat_substr . . . 328
move_row . . . 329
move_rows . . . 330
mplus_est_df . . . 332
mplus_fit_df . . . 334
mr . . . 335
mrs . . . 336
msdn_contrast . . . 338
msdn_ttest . . . 339
mutate_col . . . 341
mydes . . . 342
mysummary . . . 343
mysummaryBy . . . 344
mysummary_apa . . . 345
na2zero . . . 346
na_remover . . . 347
nice_aov_df . . . 348
nice_table . . . 349
nice_Var . . . 350
Normality_test . . . 351
obs_exp_table . . . 351
oneway_aov . . . 352
pall . . . 352
path . . . 354
patternGraph . . . 354
patternGraph2 . . . 356
patternGraph_obs_exp . . . 357
patternGraph_obs_exp_ko . . . 359
pattern_detector . . . 361
pcfa . . . 362
pcfa_porp_plot . . . 363
pdata . . . 364
perform_chi_square_test . . . 364
pivot_table . . . 365
plot_lpa . . . 366
plot_pie_chart . . . 368
plspm_add_blocks . . . 369
plspm_add_inter_paths . . . 370

plspm_apa_boot_paths . . . 371
plspm_apa_grp . . . 371
plspm_apa_ind . . . 372
plspm_apa_paths . . . 373
plspm_auto_blocks . . . 374
plspm_auto_genData . . . 375
plspm_blocks . . . 376
plspm_blocks2lav . . . 379
plspm_blocks_match . . . 380
plspm_boot_effect . . . 381
plspm_boot_factor_layout . . . 382
plspm_boot_factor_point_to . . . 384
plspm_boot_ind_factor . . . 385
plspm_boot_ind_order . . . 385
plspm_boot_ind_push . . . 386
plspm_boot_paths_sig . . . 387
plspm_boot_paths_sig_vec . . . 387
plspm_boot_summary . . . 388
plspm_cfa . . . 388
plspm_change_nodeLabels . . . 390
plspm_col2lav . . . 391
plspm_cor . . . 392
plspm_CRAVE . . . 393
plspm_crossloadings . . . 394
plspm_crossloadings_plot . . . 395
plspm_curve . . . 396
plspm_diagram_model . . . 397
plspm_dULS_modelfit . . . 401
plspm_edges_coefs . . . 402
plspm_edges_loadings . . . 404
plspm_edges_values . . . 405
plspm_edge_position . . . 408
plspm_edge_values . . . 409
plspm_effect . . . 409
plspm_effectbar . . . 410
plspm_effect_bar . . . 411
plspm_extract_blocks . . . 412
plspm_f2 . . . 413
plspm_f2_plot . . . 414
plspm_factor_layout . . . 415
plspm_fl . . . 416
plspm_grp_comp . . . 418
plspm_grp_compare_bar . . . 418
plspm_grp_comp_apa . . . 420
plspm_grp_summary . . . 421
plspm_htmt . . . 422
plspm_htmt2 . . . 423
plspm_indicator_factor . . . 425
plspm_indicator_order . . . 426
plspm_indicator_test . . . 426
plspm_indicator_test_multi . . . 427
plspm_ind_effect . . . 429

plspm_ind_effect_boot . . . 430
plspm_ind_effect_sig . . . 432
plspm_ind_effect_test . . . 433
plspm_ind_factor . . . 435
plspm_ind_order . . . 436
plspm_ind_order2 . . . 437
plspm_ind_push . . . 437
plspm_ind_spread . . . 438
plspm_innerplot . . . 439
plspm_inner_model_sig . . . 442
plspm_interaction . . . 443
plspm_lav2blocks . . . 446
plspm_lav2paths . . . 447
plspm_layout_mat . . . 448
plspm_loadings . . . 449
plspm_loadings_plot . . . 450
plspm_loadings_values . . . 452
plspm_loading_lav . . . 453
plspm_make_layout_mat . . . 454
plspm_make_prod . . . 455
plspm_mat_lav . . . 456
plspm_modelfit . . . 459
plspm_model_lav . . . 459
plspm_model_plot . . . 460
plspm_multigroups . . . 463
plspm_paths . . . 466
plspm_paths2lav . . . 468
plspm_paths_coefs_plot . . . 469
plspm_paths_sig . . . 472
plspm_paths_sig_plot . . . 473
plspm_path_coefs_plot . . . 477
plspm_permutation_test . . . 480
plspm_permutation_test_multi . . . 482
plspm_prod . . . 482
plspm_rotate_resid . . . 484
plspm_sem . . . 485
plspm_semPaths . . . 488
plspm_semPaths2 . . . 491
plspm_semPaths_lav . . . 493
plspm_semptools . . . 494
pls_modelfit_apa . . . 497
poolcor_comment . . . 498
posthoc_duncan . . . 500
posthoc_plot . . . 501
PRE . . . 502
Pull . . . 502
Pull_to_text . . . 503
p_mark_sig . . . 504
p_sig_cal . . . 505
p_to_001 . . . 506
p_value_cal . . . 507
read_text . . . 507

recode_sep_vec . . . 508
reg_apa . . . 509
remove_all_attr . . . 509
remove_attr . . . 510
remove_blank . . . 510
remove_cols . . . 511
remove_filter . . . 512
remove_rows . . . 513
rename_by_dict . . . 514
reorder_row . . . 515
Replace . . . 516
replace_dataframe . . . 517
replace_df . . . 517
replace_df2 . . . 518
replace_df_rep . . . 519
replace_in_dataframe . . . 520
replace_in_df . . . 521
Replace_na . . . 521
report_aov . . . 522
report_aov_ko . . . 523
report_lme . . . 523
report_manova . . . 524
report_stat . . . 526
report_ttest . . . 527
report_vartest . . . 528
revcode . . . 528
rev_coding . . . 530
rev_coding_extract_only . . . 531
rev_coding_map . . . 532
rev_coding_plot_var . . . 532
rev_coding_rep . . . 533
rev_coding_restore_original . . . 535
rev_coding_validate_scale . . . 536
Rload . . . 536
Round . . . 537
round2 . . . 538
round_col . . . 539
row2col . . . 539
row2cols . . . 540
rowdata2col . . . 541
rownames2colnames . . . 541
rows_to_colnames . . . 542
row_filter . . . 543
Rsave . . . 544
Rsave_xlsx . . . 544
save_xlsx . . . 545
select_format . . . 546
SEM . . . 546
semPaths2 . . . 550
sem_apa . . . 552
sem_apa_ie . . . 553
sem_apa_ie_CI . . . 554

sem_APA_path_table . . . 555
sem_count_df . . . 557
sem_effect . . . 558
sem_effect_ci . . . 560
sem_effect_diff . . . 561
sem_effect_ie . . . 563
sem_effect_ind . . . 564
sem_effect_te . . . 565
sem_fit . . . 567
sem_lav_hypo . . . 568
sem_lav_ind_hypo . . . 569
sem_model . . . 570
sem_plot . . . 573
shapiro_test_all . . . 575
shapiro_test_df . . . 575
simdata_cov . . . 576
simdata_sem . . . 578
simple_bar . . . 579
SKEW . . . 580
Sobel_test . . . 580
sobel_test_extend . . . 582
sobel_test_general . . . 583
sobel_test_serial . . . 585
split_korean_word . . . 587
split_korean_word_check . . . 587
split_kw_df . . . 588
split_kw_df_match . . . 589
split_word . . . 590
str_cut . . . 590
summary.jjstat_sem . . . 591
summary_bartlett . . . 592
summary_fa . . . 593
summary_kmo . . . 594
summary_sem . . . 594
table_apa . . . 596
table_df . . . 596
table_df_bar . . . 597
table_df_bar2 . . . 598
table_df_colnames . . . 600
table_df_multi . . . 600
table_freq . . . 601
table_reorder . . . 603
table_sum . . . 604
text2df . . . 605
TE_effect . . . 607
to_long . . . 609
to_table . . . 610
to_wide . . . 611
ttest_df . . . 613
t_sep . . . 615
t_test_report . . . 616
unCombine . . . 617

uncombine_data 618
unCount 620
Unite 621
unite_ci 622
Unite_col 623
Unite_confint 624
unite_rows 626
validate_residual_cov 627
validate_variance 627
wald_test 628
wald_test2 629
wald_test_df 630
wald_test_gender 631
wald_test_general 632
wald_test_general_df 634
wald_test_general_mat 635
wald_test_general_str 636
wald_test_lm 637
web 638
webr 639
white_se 640
wide_df 640
write_text 643
ztest 644
ztest_df 644

`accent_table` *aaccent_table: table() analytics as data frame, mosaicplot, usign add_ratio*

Description

aaccent_table: table() analytics as data frame, mosaicplot, usign add_ratio

Usage

```
accent_table(
  data,
  cols = "a1",
  rows = "성조",
  title = "",
  trans = TRUE,
  cex = 1.3,
  color = TRUE,
  ylab = "onset",
  xlab = "accent",
  sub = NULL,
  plot = FALSE,
  type = "res",
  raw = FALSE
)
```

Arguments

data	table data
cols	column names in data
rows	row names in data
title	table main
trans	transpose
cex	graph text size
color	graph color 1:2, mosaicplot
ylab	ylab
xlab	xlab
sub	subtitle
plot	output option
type	res, res_df, ratio, g, all
raw	raw data, or table data

Value

matrix type data.frame

Examples

```
## Not run:
mtcars %>% select(am, vs) %>% table() %>% accent_table("am","vs")

mtcars %>% select(am, vs) %>% table() %>% accent_table("am","vs", plot=TRUE)


mtcars %>% select(am, vs) %>% table() %>% accent_table("am","vs", plot=TRUE, title = "Mtcars")


## End(Not run)
```

---

add_bind_stat *Functions that item parceling generate multiple pieces of data,*

Description

Functions that item parceling generate multiple pieces of data,

Usage

```
add_bind_stat(data, ..., fun = mean, type = "res")
```

Arguments

data	data.fram
...	colname, cols
fun	fun mean sd
type	data, res –data.fram , res, cols_check

Value

data.frame

Examples

```
## Not run:

#Common item parceling methods (group by average )
mtcars %>% mutate(
Mean_col1 = (mpg + cyl)/2 ,
Mean_col2 = (disp + hp)/2 ,
Mean_col3 = (drat + wt )/2 ,
Mean_col4 = (qsec + vs + am)/3 )

#Methods for item parceling using functions
add_bind_stat(mtcars,
c("Mean_col1","mpg", "cyl"),
c("Mean_col2", "disp", "hp"),
c("Mean_col3", "drat", "wt"),
c("Mean_col4", "qsec", "vs", "am") )



add_bind_stat(mtcars,
   c("Mean_col1", 1:2),
   c("Mean_col2", 3:4),
   c("Mean_col3", 5:6),
   c("Mean_col4", 7:9))
#'  ##col_check
#'
add_bind_stat(mtcars,
              c("Mean_col1", "mpg:cyl"),
              c("Mean_col2", "disp:hp"),
              c("Mean_col3", "drat:wt"),
              c("Mean_col4", "qsec:am"))

## End(Not run)
```

---

add_col_ratio 그룹합에비율을만들어주는함수

Description

그룹합에비율을만들어주는함수

Usage

```
add_col_ratio(data, col, n_col)
```

Arguments

data	data
col	기준이되는열
n_col	빈도를나타내는열

Value

df

Examples

```
## Not run:
#'
# 사용 예제
library(dplyr)

# 주어진 데이터프레임
data <- data.frame(
  Q17 = c("", "고1 남학생", "고1 여학생", "고2 남학생", "고2 여학생",
          "고3 남학생", "고3 여학생", "중1 남학생", "중1 여학생",
          "중2 남학생", "중2 여학생", "중3 남학생", "중3 여학생",
          "초1 남학생", "초1 여학생", "초2 남학생", "초2 여학생",
          "초3 남학생", "초3 여학생", "초4 남학생", "초4 여학생",
          "초5 남학생", "초5 여학생", "초6 남학생", "초6 여학생"),
  Q16 = c("", rep("", 24)),
  n = c(1130, 413, 184, 329, 163, 216, 72, 354, 323, 370, 308,
        195, 169, 7, 4, 25, 23, 60, 53, 174, 162, 237, 216, 307, 293)
)

# 함수 적용: 열 이름으로 지정
result1 <- add_col_ratio(data, col = "Q17", n_col = "n")

# 함수 적용: 숫자 인덱스로 지정
result2 <- add_col_ratio(data, col = 1, n_col = 3)

# 결과 확인
print(result1)
print(result2)



## End(Not run)
```

---

add_cor *correlation analysis*

Description

correlation analysis

Usage

```
add_cor(selectdata, type = "res_all", digits = 3, plot = TRUE, md = FALSE)
```

Arguments

selectdata	seledted daa
type	res_all, res, p, r, res_p, res_md, res_lower, g(chart.Correlation)
digits	default 3
plot	graph performanceAnalytis::chart.Correlation
md	markdown table output

Value

table and picture

Examples

```
## Not run:

mtcars%>%select(mpg, disp, wt, hp, qsec)%>%add_cor()

## End(Not run)
```

---

add_id *Add row id*

Description

Add row id

Usage

```
add_id(data, sel = "names", each = TRUE)
```

Arguments

data	data.frame
sel	select column
each	each =TRUE then 111222....FALSE 123123

Value

add column data

Examples

```
## Not run:

mtcars %>% mutate(MPG= which(colnames(mtcars) =="mpg"))%>%add_id()
mtcars %>% mutate(MPG= which(colnames(mtcars) =="mpg"))%>%add_id("cyl")
mtcars[1:3, 1:3] %>% add_id()
mtcars[1:3, 1:3] %>% add_id("cyl")
mtcars[1:4, 1:4] %>% to_long() %>% add_id()
mtcars[1:4, 1:4] %>% to_long() %>% add_id(each=FALSE)




## End(Not run)
```

`add_normality` *Add skewness and kurtosis normality checks to descriptive statistics results*

Description

Add skewness and kurtosis normality checks to descriptive statistics results

Usage

```
add_normality(data, skew = "Skew", kurt = "Kurt")
```

Arguments

`data`	mysummary data
`skew`	select skew
`kurt`	select kurt

Value

normalitydata

Examples

```
## Not run:

mysummary(mtcars)
mysummary(mtcars)%>%add_normality()



## End(Not run)
```

add_normality_kline *Add Normality Kline*

Description

This function evaluates normality for each variable in the dataset based on skewness and kurtosis.

Usage

```
add_normality_kline(df, skew = 3, kurt = 8)
```

Arguments

df	A data frame containing statistical summary including Skew and Kurt columns.
skew	Threshold for skewness. Default is 3.0.
kurt	Threshold for kurtosis. Default is 8.0.

Value

A data frame with added columns 'skew_sig' and 'kurt_sig' indicating the normality evaluation.

Examples

```
## Not run:
data <- data.frame(
  var = c("SUX", "FLOW", "PU", "PEOU", "ATT", "USE"),
  N = c(895, 895, 895, 895, 895, 895),
  MEAN = runif(6, 2, 3),
  SD = runif(6, 0.9, 1.1),
  MIN = rep(1, 6),
  MAX = rep(5, 6),
  Skew = c(0.74, 0.23, 0.81, 0.75, 0.69, 0.53),
  Kurt = c(0.16, -0.31, 0.37, 0.06, 0.02, -0.42)
)
add_normality_kline(data)

## End(Not run)
```

add_partial_sum *add_partial_sum*

Description

add_partial_sum

Usage

```
add_partial_sum(df, col, freq_col)
```

Arguments

df	df
col	select col
freq_col	select count frequency col

Value

df

Examples

```
## Not run:
#'
# 함수 사용 예시
df <- tibble::tribble(
  ~학교급, ~term, ~Freq, ~`prop(%)`,
  "-", "합계", 3110, 100,
  "고등학교", "고1", 341, 11.0,
  "고등학교", "고2", 322, 10.4,
  "고등학교", "고3", 192, 6.17,
  "중학교", "중1", 591, 19.0,
  "중학교", "중2", 527, 16.9,
  "중학교", "중3", 290, 9.32,
  "초등학교", "초1", 8, 0.257,
  "초등학교", "초2", 48, 1.54,
  "초등학교", "초3", 47, 1.51,
  "초등학교", "초4", 191, 6.14,
  "초등학교", "초5", 207, 6.66,
  "초등학교", "초6", 346, 11.1
)

# 함수 실행
result_df <- add_partial_sum(df, col = "학교급", freq_col = "Freq")
print(result_df)

## End(Not run)
```

---

add_pca_scores *add_pca_scores*

Description

add_pca_scores

Usage

```
add_pca_scores(data, ..., method = "nipals", adjust = NULL)
```

Arguments

data	data
...	select variable
method	niplas, prcomp
adjust	Options to change the sign of prcomp c(F, T ...)

Value

add data

Examples

```
## Not run:
data(offense)

nfl_path = matrix(
  c(0, 0, 0, 0, 0,
    0, 0, 0, 0, 0,
    0, 0, 0, 0, 0,
    0, 1, 1, 0, 0,
    1, 0, 0, 1, 0),
  nrow = 5, ncol = 5, byrow = TRUE )

rownames(nfl_path) <- c("Speical", "Rushing", "Passing", "Offense", "Scoring")
colnames(nfl_path) <- c("Speical", "Rushing", "Passing", "Offense", "Scoring")
nfl_path
Offense = offense %>% add_pca_scores(rush1 = c(1:3), pass1 = c(4:6))

Offense =  add_pca_scores(offense,
               rush1 = c("YardsRushAtt", "RushYards", "RushFirstDown"),
               pass1 = c("YardsPassComp", "PassYards", "PassFirstDown"))


nfl_blocks2 = list(7:8, 1:3, 4:6,  18:19, 9:11)

plspm_blocks_match(data=Offense, blocks=nfl_blocks2,
                   paths=nfl_path)

plspm_blocks_match(data=Offense, blocks=nfl_blocks2,
                   paths=nfl_path)%>%make_list_text()

nfl_blocks2 = list(
  Speical = c("FieldGoals", "OtherTDs"),
  Rushing = c("YardsRushAtt", "RushYards", "RushFirstDown"),
  Passing = c("YardsPassComp", "PassYards", "PassFirstDown"),
  Offense = c("rush1", "pass1"),
  Scoring = c("PointsGame", "OffensTD", "TDGame")
)
nfl_blocks2
nfl_mode2 =c("B","A","A","A","A")
nfl_pls2 = plspm(Data=Offense, nfl_path, nfl_blocks2, modes = nfl_mode2)
nfl_pls2%>%summary()

nfl_pls_boot2 = plspm_sem(Data=Offense, nfl_path, nfl_blocks2, modes = nfl_mode)
```

```
x11()
nfl_pls_boot2 %>%plspm_path_coefs_plot()
nfl_pls_boot2 %>%plspm_semPaths2()

#'
#'

# 함수 테스트
offense <- data.frame(
  YardsRushAtt = rnorm(10),
  RushYards = rnorm(10),
  RushFirstDown = rnorm(10),
  YardsPassComp = rnorm(10),
  PassYards = rnorm(10),
  PassFirstDown = rnorm(10)
)

# 테스트 케이스 1
offense1 <- add_pca_scores(offense,
                           rush1 = c("YardsRushAtt", "RushYards", "RushFirstDown"),
                           pass1 = c("YardsPassComp", "PassYards", "PassFirstDown"))

str(offense1)

# 테스트 케이스 2
offense2 <- add_pca_scores(offense, method = "prcomp",
                           rush1 = c("YardsRushAtt", "RushYards", "RushFirstDown"),
                           pass1 = c("YardsPassComp", "PassYards", "PassFirstDown"))

str(offense2)

# 테스트 케이스 3 (adjust 옵션 포함)
offense3 <- add_pca_scores(offense, method = "prcomp", adjust = c(F, T),
                           rush1 = c("YardsRushAtt", "RushYards", "RushFirstDown"),
                           pass1 = c("YardsPassComp", "PassYards", "PassFirstDown"))

str(offense3)
#'
#'


## End(Not run)
```

`add_prop` *Add Proportions to Frequency Data*

Description

This function adds a column to a data frame representing the proportion of each frequency value, either as a percentage or a simple ratio.

Usage

```
add_prop(df, freq = "Freq", prop = TRUE)
```

Arguments

df	A data frame containing frequency data.
freq	A column name or index representing the frequency column. Defaults to "Freq". If a numeric value is provided, it is converted to the corresponding column name.
prop	Logical, whether to return proportions as percentages (TRUE, default) or ratios (FALSE).

Value

A data frame with an additional column for proportions. If 'prop = TRUE', the column is named 'prop(If 'prop = FALSE', the column is named 'prop'.

Examples

```
## Not run:
# Example data
df <- data.frame(Category = c("A", "B", "C"), Freq = c(30, 50, 20))

# Adding proportions as percentages
add_prop(df, freq = "Freq", prop = TRUE)

# Adding proportions as simple ratios
add_prop(df, freq = "Freq", prop = FALSE)

## End(Not run)
```

---

add_ratio *Convert matrix data to a ratio by using the sum of columns to get a ratio*

Description

Convert matrix data to a ratio by using the sum of columns to get a ratio

Usage

```
add_ratio(data, type = "res2", digits = 3)
```

Arguments

data	matrix
type	res2: *100, res1
digits	defalut 3

Value

ratio tabl e

Examples

```
## Not run:


mat = matrix(c(36, 67, 11,
          0, 35, 44,
         41, 108, 1,
          56, 87, 54),
           nrow = 4, byrow = TRUE,
 dimnames = list(c("공명음", "마찰음",
                   "유기음_경음", "평파열음_평파찰음"),
                    c("H", "H(H)", "L")))

## step1
  mat %>% add_ratio()
##conbind
 mat %>% add_ratio_df()
##add margins
 mat %>% add_sum()


## End(Not run)
```

`add_ratio_df` *Combine matrices and their ratios*

Description

Combine matrices and their ratios

Usage

```
add_ratio_df(res)
```

Arguments

`res`	matrix

Value

combine table /matrix

Examples

```
## Not run:

#' #'mat = matrix(c(36, 67, 11,
          0, 35, 44,
         41, 108, 1,
          56, 87, 54),
           nrow = 4, byrow = TRUE,
```

```
   dimnames = list(c("공명음", "마찰음",
     "유기음_경음", "평파열음_평파찰음"),  c("H", "H(H)", "L")))

## step1
  mat %>% add_ratio()
##conbind
 mat %>% add_ratio_df()
##add margins
 mat %>% add_sum()

## End(Not run)
```

`add_rep_id` *Create an ordered variable for overlapping groups*

Description

Create an ordered variable for overlapping groups

Usage

```
add_rep_id(data, grp)
```

Arguments

`data`	data.frame
`grp`	select colname or number

Value

create data

Examples

```
## Not run:
#'
data <- data.frame(
  weight = c(4.17, 5.58, 5.18, 6.11, 4.50, 4.61, 5.17, 4.53, 5.33, 5.14,
             4.81, 4.17, 4.41, 3.59, 5.87, 3.83, 6.03, 4.89, 4.32, 4.69,
             6.31, 5.12, 5.54, 5.50, 5.37, 5.29, 4.92, 6.15, 5.80, 5.26),
  group = c(rep("ctrl", 10), rep("trt1", 10), rep("trt2", 10))
)
data

#generate id from Colname
add_rep_id(data, "group")

#generate id from Col number
add_rep_id(data, 2)

#data spread wide
```

```
data%>%add_rep_id("group")%>%as_trt("id") %>%wide_df("group", "weight")


data%>%
  add_rep_id("group")%>%
  as_trt("id") %>%
  wide_df("group", "weight")


## End(Not run)
```

`add_rows` *Insert data into the required act in the dataframe*

Description

Insert data into the required act in the dataframe

Usage

```
add_rows(df, ..., row = NULL, char = FALSE)
```

Arguments

`df`	data.frame
`...`	data, c(100, 200), c(200, 300),
`row`	row position
`char`	transpose character column

Value

dagta

Examples

```
## Not run:
#'
#'


df <- data.frame(A = 1:5, B = 6:10)
df
# 2번째 행부터 새 데이터(벡터 c(1,2,3,4)와 c(2,3,4,5))를 삽입

add_rows(df, c(100, 200), c(200, 300), row = 2)

df <- data.frame(
  측정변수 = c("진로목표(L01)", "준비여부(L02)", "직업결정(L03)", "부모추천(L04)", "학업성적(L05)"),
  Class.1 = c(0.2178, 0.3533, 0.1552, 0.2708, 0.1605),
  Class.2 = c(0.9888, 0.9769, 0.9910, 1.0000, 0.9804),
  Class.3 = c(0.9122, 0.8704, 0.9667, 0.0000, 0.6761)
```

```
)

# 1행에 새 데이터를 삽입
new_df <- add_rows(df, c("class name", "불명확 진로인식형", "적극적 진로준비형", "조건분 진로지향형")

print(new_df)


df <- data.frame(
  측정변수 = c("진로목표(L01)", "준비여부(L02)", "직업결정(L03)", "부모추천(L04)", "학업성적(L05)"),
  Class.1 = c(0.2178, 0.3533, 0.1552, 0.2708, 0.1605),
  Class.2 = c(0.9888, 0.9769, 0.9910, 1.0000, 0.9804),
  Class.3 = c(0.9122, 0.8704, 0.9667, 0.0000, 0.6761)
)

# 첫 번째 행에 새 데이터를 삽입, char = TRUE로 문자형 변환
new_df_1 <- add_rows(df, c("class name", "불명확 진로인식형", "적극적 진로준비형", "조건분 진로지향형

print(new_df_1)

# 마지막 행에 새 데이터를 삽입, char = FALSE로 기존 타입 유지
new_df_2 <- add_rows(df, c("class name", "불명확 진로인식형", "적극적 진로준비형", "조건분 진로지향형

print(new_df_2)



## End(Not run)
```

`add_rows_freq` *Functions that generate rows with a frequency*

Description

Functions that generate rows with a frequency

Usage

```
add_rows_freq(data, type = "res", sel = ncol(data), fix = NULL)
```

Arguments

`data`	data
`type`	res, all
`sel`	frequency column
`fix`	final select column

Value

data

Examples

```
## Not run:
dff = data.frame(
  trt = c("r","s","z"),
  con = c("f","m","l"),
  num = c(10,0,2),
  fre = c(4,0,5) )
dff
dff %>%add_rows_freq()
dff %>%add_rows_freq(sel=4)
dff %>%add_rows_freq(sel="fre")
dff %>%add_rows_freq(sel=3)
dff %>%add_rows_freq(sel="num")
dff %>%add_rows_freq(fix=1)
dff %>%add_rows_freq(type= "res", fix=1:2)
dff %>%add_rows_freq(type= "res", fix="con")
dff %>%add_rows_freq(type= "all")
dff %>%add_rows_freq(type= "real")
dff %>%add_rows_freq(type= "zero")
dff %>%add_rows_freq(type= "zero", fix="con")


# Making frequency data into real data
dff %>%to_long()%>% add_rows_freq()
 dff_real =  dff %>%to_long()%>% add_rows_freq()
 dff_real


 freq_data =  mtcars %>% select(am, vs) %>% table()
 freq_data%>% data.frame()
 freq_data%>% data.frame() %>% add_rows_freq(type ="zero")
 freq_data%>% data.frame() %>% add_rows_freq(type ="zero") %>%table()



## End(Not run)
```

---

add_row_sum *Add Row Sum to Data Frame*

Description

This function calculates the sum of specified numeric columns and appends a summary row to the input data frame. The summary row includes totals for ‘freq_col‘ and ‘prop_col‘.

Usage

```
add_row_sum(df, freq_col = 2, prop_col = 3)
```

Arguments

df	A data frame containing the data to be summed.
freq_col	Column name or index for the frequency column. Default is 2.
prop_col	Column name or index for the proportion column. Default is 3.

Value

A data frame with an added summary row containing the sums.

Examples

```
## Not run:
data <- data.frame(
  Group = c("A", "B", "C"),
  Freq = c(10, 20, 30),
  `prop(%)` = c(25.0, 50.0, 25.0)
)
add_row_sum(data, freq_col = "Freq", prop_col = "prop(%)")

## End(Not run)
```

---

add_sig *mutate data generate*

Description

mutate data generate

Usage

```
add_sig(var)
```

Arguments

p.value	pvalue

Value

in data

Examples

```
## Not run:

aov_table(data = mtcars, dv_var = "mpg",
iv_vars = c("cyl", "gear", "carb"), grp_mean = T,
unite_F=TRUE) %>%
 mutate(STAR= add_sig(p_value))


## End(Not run)
```

`add_significance_symbols` *Functions that output statistical significance for matrices and dataframes Simple and exact expressions*

Description

Functions that output statistical significance for matrices and dataframes Simple and exact expressions

Usage

```
add_significance_symbols(p_value_table, simple = FALSE)
```

Arguments

`p_value_table`	A matrix of p-values, or other numerical matrix
`simple`	Whether to represent the * mark simply or precisely with ***.

Value

Matrices marked with a *.

Examples

```
## Not run:

#' p_value <- matrix(c(0, 0.00001, 0.00000,
                   0, 0.00000, 0.01497,
                   0, 0.00013, 0.00000,
                   0, 0.00000, 0.00102),
                 nrow = 4, byrow = TRUE,
                 dimnames = list(c("공명음", "마찰음", "유기음_경음", "평파열음_평파찰음"),
                                 c("H", "H(H)", "L")))
    # Add significance symbols
    # p_value

    add_significance_symbols(p_value)

##                   H     H(H)  L
##공명음             "***" "***" "***"
##마찰음             "***" "***" "*"
##유기음_경음        "***" "***" "***"
##평파열음_평파찰음 "***" "***" "**"




## End(Not run)
```

`add_sig_star` *Functions that output statistical significance for matrices and dataframes Simple and exact expressions*

Description

Functions that output statistical significance for matrices and dataframes Simple and exact expressions

Usage

```
add_sig_star(p_value_table, simple = FALSE)
```

Arguments

`p_value_table`	A matrix of p-values, or other numerical matrix
`simple`	Whether to represent the * mark simply or precisely with ***.

Value

Matrices marked with a *.

Examples

```
## Not run:

#' p_value <- matrix(c(0, 0.00001, 0.00000,
                   0, 0.00000, 0.01497,
                   0, 0.00013, 0.00000,
                   0, 0.00000, 0.00102),
                 nrow = 4, byrow = TRUE,
                 dimnames = list(c("공명음", "마찰음", "유기음_경음", "평파열음_평파찰음"),
                                 c("H", "H(H)", "L")))
    # Add significance symbols
    # p_value

    add_significance_symbols(p_value)

##                   H     H(H)  L
##공명음             "***" "***" "***"
##마찰음             "***" "***" "*"
##유기음_경음        "***" "***" "***"
##평파열음_평파찰음 "***" "***" "**"




## End(Not run)
```

add_stat 데이터에서변수를택하면함수에맞는새로운열을추가하는함수

Description

데이터에서변수를택하면함수에맞는새로운열을추가하는함수

Usage

```
add_stat(
  data,
  ...,
  fun = mean,
  col_name = NULL,
  na.rm = TRUE,
  margin = 1,
  type = "data"
)
```

Arguments

data	data.frame
...	cols
fun	mean, ad, ...
col_name	New variable
na.rm	na.rm=TRUE
margin	margin 1, 2
type	data, col 2type

Value

data.fram or vector

Examples

```
## Not run:

df = data.frame(A = c(1, 2, 3, 4, 5), B = c(6, 7, NA, 9, 10), C = c(11, 12, 13, 14, 15) )

df%>% add_stat( "AC_MEAN","A", "C", fun = mean)
df%>% add_stat( "AC_MEAN","A", "C", fun = mean, type="col")
df %>%add_stat( "A", "C", type="col")
df %>%add_stat( "A", "C")
df %>%add_stat( "A", "C", fun = mean, "AC_MEAN")

df %>%add_stat( "A", "C", "AC_MEAN",fun = sd)

df %>%add_stat(col_name = "AC_MEAN","A", "C", fun = mean)
df %>%add_stat("A", "C", fun = mean, col_name = "AC_MEAN")
df %>%add_stat("A", "C", fun = mean)
```

```
df %>%add_stat(c("A", "C"))
df %>%add_stat("ROWs", c("A", "C"))

mtcars %>% add_stat("MEAN", "am","vs")
mtcars %>% add_stat("am","vs")
mtcars %>% add_stat("MEAN", c("am","vs"))
mtcars %>% add_stat("MEAN", c("mpg","qsec"))
mtcars %>% add_stat("MEAN", c("mpg","qsec"), col_name="ACME")
df


## End(Not run)
```

`add_sum` *Functions to sum margins after adding ratios*

Description

Functions to sum margins after adding ratios

Usage

```
add_sum(
  dataset,
  v1 = "a1",
  v2 = "성조",
  raw = FALSE,
  add_ratio = TRUE,
  plot = FALSE,
  color = TRUE,
  ylab = "onset",
  xlab = "speaker",
  cex = 1.2,
  title = NULL,
  type = "df"
)
```

Arguments

`dataset`	data(rwa=T)or table
`v1`	col
`v2`	row
`raw`	TRUE dataset. not table
`add_ratio`	using add_ration function
`plot`	mosicplot
`color`	color 1:4, etc
`ylab`	yalb
`xlab`	xlab

cex	size
title	title
type	all, df, res

Value

matrix

Examples

```
#'
## Not run:

#' #'mat = matrix(c(36, 67, 11,
          0, 35, 44,
         41, 108, 1,
          56, 87, 54),
           nrow = 4, byrow = TRUE.
   dimnames = list(c("공명음", "마찰음",
     "유기음_경음", "평파열음_평파찰음"),  c("H", "H(H)", "L")))
## step1
  mat %>% add_ratio()
##conbind
 mat %>% add_ratio_df()

##add margins
 mat %>% add_sum()

 #                          H H(H)+H(H)_1         L SUM
 #공명음              26(11.5%)   35(19.4%) 36(28.1%)  97
 #마찰음              49(21.6%)   62(34.4%) 35(27.3%) 146
 #유기음_경음        108(47.6%)   82(45.6%)  11(8.6%) 201
 #평파열음_평파찰음  44(19.4%)     1(0.6%) 46(35.9%)  91
 #SUM                      227         180       128 535




## End(Not run)
```

---

add_t_sig *add_t_sig*

Description

add_t_sig

Usage

```
add_t_sig(
  data,
  est,
  se,
  col = ncol(data) + 1,
  unite = FALSE,
  digits = 3,
  ns = " ns"
)
```

Arguments

data	data
col	position t value
unite	unite sig star +t
digits	3
ns	ns is ns sig

Value

data

Examples

```
## Not run:
#'
New_df = data.frame(rownames = c('IMAG -> EXPE', 'IMAG -> QUAL', 'IMAG -> VAL', 'IMAG -> SAT'),
  Original = c(0.578959128452789, 0.491156739303251, 0.393411171344413, 0.490971137879492),
 Mean.Boot = c(0.584251828061525, 0.495822941940513, 0.400070801457516, 0.5022633230577),
 Std.Error = c(0.0518709126012371, 0.0510037451930923, 0.0528454527326974, 0.0681551639743143),
 perc.025 = c(0.477709787839589, 0.391876781940297, 0.308196019466015, 0.369649328752085),
  perc.975 = c(0.664779330969341, 0.58560953201285, 0.495717252789497, 0.614490783702303)
)
#Auto-create a separate T without creating a SIG and combine it with the SIG to move it
New_df %>% add_t_sig(est=3, se=4, col=4, unite=T)


# Move only sigs after creating a t
New_df %>% add_t_sig(est=3, se=4, col=4, unite=F)

# 열을 결합
New_df %>% add_t_sig(est=3, se=4, col=4, unite=F) %>%Unite(2,4)

New_df %>% add_t_sig(est=3, se=4, col=4, unite=F)%>%Round(3) %>%Unite(2,4)


## End(Not run)
```

`all_na_zero` *all_na_zero*

Description

all_na_zero

Usage

```
all_na_zero(df, imp = 0)
```

Arguments

`df`	data.frame
`imp`	implement default = 0

Value

NA to 0

`all_na_zero1` *NA to zero*

Description

NA to zero

Usage

```
all_na_zero1(data, fun = "extend")
```

Arguments

`data`	data.frame
`fun`	'extend', 'base' are equal method

Value

Na data are changed zero

alpha_apa *alpha_apa report*

Description

alpha_apa report

Usage

```
alpha_apa(data)
```

Arguments

data	분석결과

Value

apa 보고서

Examples

```
## Not run:

#'
# 진로동기 신뢰도 분석
jjstat::bind_alpha_table(jutr2%>% select(A01:A15),
                         c(1:5), c(6:10), c(11:15)) %>%nice_Var()%>% nice_table()%>%
  replace_df("a","진로정체감") %>%
  replace_df("b","진로통찰력") %>%
  replace_df("c","진로탄력성") %>%
  alpha_apa()


## End(Not run)
```

alpha_table *Function to tabulate reliability*

Description

Function to tabulate reliability

Usage

```
alpha_table(
  alpha_data_res,
  title = "",
  show = "data",
  format = "markdown",
  digits = 3,
  check.keys = TRUE,
  variable = "."
)
```

Arguments

alpha_data_res	alpha result
title	your title
show	'data', 'markdown'
format	markdown, html
digits	default 3
check.keys	check.keys=TRUE
variable	. output

Examples

```
## Not run:

stat_onl %>% select(S_Review, S_Add_learn, S_Feedback) %>%
   jjstat::alpha_table(show="data", variable = "A" )


bind_rows(
 stat_onl %>% select(S_Review, S_Add_learn, S_Feedback) %>%
   jjstat::alpha_table(show="data", variable = "A" ),
 stat_onl %>% select(SE_Place, SE_Time) %>%
   jjstat::alpha_table(show="data", variable = "B"),
 stat_onl %>% select(On_Satisfy, On_Joy, On_Easy, On_Engage) %>%
   jjstat::alpha_table(show="data", variable = "C"),
 stat_onl %>% select(upgrade,satisfy,fatigue_inverse) %>%
   jjstat::alpha_table(show="data", variable = "D")
) |> markdown_table(digits=3, font_size = 12,
                    caption= "total Cronbach alpha")

## End(Not run)
```

---

anova_apa *Report aov data (format APA)*

Description

Report aov data (format APA)

Usage

```
anova_apa(
  aov_post_data,
  ns = 7,
  np = 6,
  ndf = 4,
  nt = 5,
```

```
  digits = 3,
  md = FALSE,
  posthoc_type = "The Multi Least Significant Difference(MLSD)"
)
```

Arguments

aov_post_data	aov_post data
ns	ncol sig column 7
np	ncol p value column 6
ndf	ncol df column 4
nt	ncol t value column 5
digits	digits 3
md	md = TRUE is markdown outpput

Value

report data

Examples

```
## Not run:
aov(mpg ~ cyl , data=Mtcars)%>% aov_post() %>% anova_apa()
# This is error
aov(mpg ~ cyl , data=Mtcars)%>%aov_post(type="contrast")%>% anova_apa()

# aov(mpg ~ cyl , data=Mtcars)%>%aov_post(type="all")

aov(mpg ~ cyl , data=Mtcars)%>%aov_post(type="all")%>% anova_apa()
aov(mpg ~ cyl , data=Mtcars)%>%aov_post(type="res")%>% anova_apa()


## End(Not run)
```

---

aov_apa *Report aov data (format APA)*

Description

Report aov data (format APA)

Usage

```
aov_apa(
  aov_post_data,
  ns = 7,
  np = 6,
  ndf = 4,
  nt = 5,
  digits = 3,
```

```
  md = FALSE,
  posthoc_type = "The Multi Least Significant Difference(MLSD)"
)
```

Arguments

aov_post_data	aov_post data
ns	ncol sig column 7
np	ncol p value column 6
ndf	ncol df column 4
nt	ncol t value column 5
digits	digits 3
md	md = TRUE is markdown outpput

Value

report data

Examples

```
## Not run:
aov(mpg ~ cyl , data=Mtcars)%>%aov_post()%>% aov_apa()
# This is error
aov(mpg ~ cyl , data=Mtcars)%>%aov_post(type="contrast")%>% aov_apa()

# aov(mpg ~ cyl , data=Mtcars)%>%aov_post(type="all")

aov(mpg ~ cyl , data=Mtcars)%>%aov_post(type="all")%>% aov_apa()
aov(mpg ~ cyl , data=Mtcars)%>%aov_post(type="res")%>% aov_apa()


## End(Not run)
```

---

aov_df *#' Generate an AOV results table for multiple independent variables*

Description

#' Generate an AOV results table for multiple independent variables

Usage

```
aov_df(
  data,
  dv_var = NULL,
  iv_vars = NULL,
  grp_mean = TRUE,
  unite_p = FALSE,
  unite_F = FALSE,
```

```
  digits = 2,
  posthoc = "lsd",
  p.adj = "bonferroni",
  sig = FALSE
)
```

Arguments

data	data.frame
dv_var	dv
iv_vars	c(iv1, iv2....)
grp_mean	'grooup mean remove
unite_p	pvalue unite
unite_F	Fvalue unite
posthoc	'tukey, scheffe, duncan, lsd
p.adj	p.adj=c("none","holm","hommel", "hochberg", "bonferroni", "BH", "BY", "fdr") only lsd test
sig	'sig = T' is add star

Value

table

Examples

```
## Not run:
aov_table(data = mtcars, dv_var = "mpg",
iv_vars = c("cyl", "gear", "carb"))

aov_table(data = mtcars, dv_var = "mpg",
iv_vars = c("cyl", "gear", "carb"),
           mean = TRUE)

aov_table(data = mtcars, dv_var = "mpg",
iv_vars = c("cyl", "gear", "carb"), grp_mean = FALSE)
aov_table(data = mtcars, dv_var = "mpg",
iv_vars = c("cyl", "gear", "carb"), grp_mean = FALSE) %>%


aov_table(data = mtcars, dv_var = "mpg",
iv_vars = c("cyl", "gear", "carb"),
 grp_mean = TRUE)

aov_table(data = mtcars, dv_var = "mpg",
 iv_vars = c("cyl", "gear", "carb"),
 grp_mean = TRUE, unite_F = TRUE)

aov_table(data = mtcars, dv_var = "mpg",
iv_vars = c("cyl", "gear", "carb"),
 grp_mean = TRUE, unite_F = TRUE) %>%
  markdown_table()
```

```
# park516d데이터 이용
park516d %>%
  select(1:6,teaching, academyType) %>%  #주요변수 선택
  bind_add_stat(c("RV_Teah",1:6)) %>% # 평균을 이용하여 항목묶음
  aov_df(dv_var="RV_Teah",
         iv_vars =c("teaching", "academyType"), sig = T)



## End(Not run)
```

`aov_df2` *Perform ANOVA with emmeans and extract group letters and p-values*

Description

This function performs ANOVA for each independent variable (IV) and extracts the estimated marginal means (emmeans), post-hoc comparison group letters, and p-values, based on the specified multiple comparisons adjustment method.

Usage

```
aov_df2(
  data,
  dv_var,
  iv_vars,
  adjust = "none",
  alpha = 0.05,
  digits = 2,
  show_group_letter = TRUE,
  adjust_only = TRUE
)
```

Arguments

`data`	A data.frame containing the variables.
`dv_var`	The name of the dependent variable (character).
`iv_vars`	A character vector of independent variable names.
`adjust`	Adjustment method for pairwise comparisons. One of: ”none”, ”tukey”, ”bonferroni”, ”holm”, ”sidak”, ”fdr”, ”scheffe”.
`alpha`	Significance level (default = 0.05).
`digits`	Number of digits to round numeric results (default = 2).
`show_group_letter`	Logical. Whether to show group letters (default = TRUE).
`adjust_only`	Logical. If TRUE and adjust != ”none”, only adjusted p-values (adj_p) are shown (default = TRUE).

Details

Adjustment Methods - ‘"none"‘: No adjustment for multiple comparisons (raw p-values). Can inflate Type I error. - ‘"tukey"‘: Tukey's Honest Significant Difference method. Simultaneously compares all pairwise mean differences. - ‘"bonferroni"‘: Conservative method; adjusts p-values by multiplying by the number of comparisons. - ‘"holm"‘: Sequential Bonferroni procedure; less conservative and more powerful alternative. - ‘"sidak"‘: Adjustment assuming independent comparisons. - ‘"fdr"‘: Controls the False Discovery Rate (Benjamini-Hochberg procedure). - ‘"scheffe"‘: Not supported in ‘emmeans‘; typically used for general contrasts. - ‘"duncan"‘, ‘"lsd"‘: Not supported in ‘emmeans‘; internally treated as ‘"none"‘.

Value

A tibble with ANOVA results per IV level, group letters, and p-values.

Examples

```
## Not run:
# Basic usage
aov_df_emmeans(data = neulbom_p2_stat2,
               dv_var = "relation",
               iv_vars = c("gender", "age", "child"))

# Using Bonferroni adjustment
aov_df_emmeans(data = neulbom_p2_stat2,
               dv_var = "relation",
               iv_vars = c("gender", "age", "child"),
               adjust = "bonferroni", adjust_only = FALSE)

# Showing only adjusted p-values
aov_df_emmeans(data = neulbom_p2_stat2,
               dv_var = "relation",
               iv_vars = c("gender", "age", "child"),
               adjust = "holm", adjust_only = TRUE)

# Tukey fallback warning
aov_df_emmeans(data = neulbom_p2_stat2,
               dv_var = "relation",
               iv_vars = c("gender", "age", "child"),
               adjust = "tukey")

# Unsupported methods fallback
aov_df_emmeans(data = neulbom_p2_stat2,
               dv_var = "relation",
               iv_vars = c("gender", "age", "child"),
               adjust = "duncan")

## End(Not run)
```

`aov_df_emmeans` *Perform ANOVA with emmeans and extract group letters and p-values*

Description

This function performs ANOVA for each independent variable (IV) and extracts the estimated marginal means (emmeans), post-hoc comparison group letters, and p-values, based on the specified multiple comparisons adjustment method.

Usage

```
aov_df_emmeans(
  data,
  dv_var,
  iv_vars,
  adjust = "none",
  alpha = 0.05,
  digits = 2,
  show_group_letter = TRUE,
  adjust_only = TRUE
)
```

Arguments

`data`	A data.frame containing the variables.
`dv_var`	The name of the dependent variable (character).
`iv_vars`	A character vector of independent variable names.
`adjust`	Adjustment method for pairwise comparisons. One of: ”none”, ”tukey”, ”bonferroni”, ”holm”, ”sidak”, ”fdr”, ”scheffe”.
`alpha`	Significance level (default = 0.05).
`digits`	Number of digits to round numeric results (default = 2).
`show_group_letter`	Logical. Whether to show group letters (default = TRUE).
`adjust_only`	Logical. If TRUE and adjust != ”none”, only adjusted p-values (adj_p) are shown (default = TRUE).

Details

Adjustment Methods - ‘”none”‘: No adjustment for multiple comparisons (raw p-values). Can inflate Type I error. - ‘”tukey”‘: Tukey’s Honest Significant Difference method. Simultaneously compares all pairwise mean differences. - ‘”bonferroni”‘: Conservative method; adjusts p-values by multiplying by the number of comparisons. - ‘”holm”‘: Sequential Bonferroni procedure; less conservative and more powerful alternative. - ‘”sidak”‘: Adjustment assuming independent comparisons. - ‘”fdr”‘: Controls the False Discovery Rate (Benjamini-Hochberg procedure). - ‘”scheffe”‘: Not supported in ‘emmeans‘; typically used for general contrasts. - ‘”duncan”‘, ‘”lsd”‘: Not supported in ‘emmeans‘; internally treated as ‘”none”‘.

Value

A tibble with ANOVA results per IV level, group letters, and p-values.

Examples

```
## Not run:
# Basic usage
aov_df_emmeans(data = neulbom_p2_stat2,
               dv_var = "relation",
               iv_vars = c("gender", "age", "child"))

# Using Bonferroni adjustment
aov_df_emmeans(data = neulbom_p2_stat2,
               dv_var = "relation",
               iv_vars = c("gender", "age", "child"),
               adjust = "bonferroni", adjust_only = FALSE)

# Showing only adjusted p-values
aov_df_emmeans(data = neulbom_p2_stat2,
               dv_var = "relation",
               iv_vars = c("gender", "age", "child"),
               adjust = "holm", adjust_only = TRUE)

# Tukey fallback warning
aov_df_emmeans(data = neulbom_p2_stat2,
               dv_var = "relation",
               iv_vars = c("gender", "age", "child"),
               adjust = "tukey")

# Unsupported methods fallback
aov_df_emmeans(data = neulbom_p2_stat2,
               dv_var = "relation",
               iv_vars = c("gender", "age", "child"),
               adjust = "duncan")

## End(Not run)
```

`aov_post` *ANOVA posthoc and plot, Contrast data*

Description

ANOVA posthoc and plot, Contrast data

Usage

```
aov_post(aov_data, type = "df", adjust = "none", lsd = FALSE)
```

Arguments

`aov_data`	aov data
`type`	type is all, res, contras, df, raq(plotdata), g(plot), aov, anova
`adjust`	adjust =”none”, bonferroni, sidak etc
`lsd`	plosthoc LSD

Value

contrast and g

Examples

```
## Not run:
Mtcars <- mtcars%>%as_trt("cyl")
aov(mpg ~ cyl , data=Mtcars)%>% aov_post()
aov(mpg ~ cyl , data=Mtcars)%>% aov_post(type="all")
aov(mpg ~ cyl , data=Mtcars)%>% aov_post(type="res")
aov(mpg ~ cyl , data=Mtcars)%>% aov_post(type="df")
aov(mpg ~ cyl , data=Mtcars)%>% aov_post(type="call")
aov(mpg ~ cyl , data=Mtcars)%>% aov_post(type="formula")
aov(mpg ~ cyl , data=Mtcars)%>% aov_post(type="g")
aov(mpg ~ cyl , data=Mtcars)%>% aov_post(type="contrast")
aov(mpg ~ cyl , data=Mtcars)%>% aov_post(type="aov")
aov(mpg ~ cyl , data=Mtcars)%>% aov_post(type="anova")
aov(mpg ~ cyl , data=Mtcars)%>% aov_post(type="posthoc", lsd=TRUE)

#report
aov(mpg ~ cyl , data=Mtcars)%>%aov_post()%>% anova_apa()

## End(Not run)
```

---

aov_post2 *ANOVA posthoc and plot, Contrast data*

Description

Post-hoc analysis and plotting for ANOVA results. Supports various posthoc methods including LSD, Duncan, Scheffe, Tukey HSD, SNK, FDR and Bonferroni correction.

Usage

```
aov_post2(
  aov_data,
  type = "df",
  adjust = "none",
  posthoc = TRUE,
  posthoc_method = "LSD"
)
```

Arguments

aov_data	An object returned by `aov()`; ANOVA model object.
type	Output type. One of: `"all"`, `"res"`, `"contrast"`, `"df"`, `"raw"`, `"g"`, `"aov"`, `"anova"`, `"call"`, `"formula"`, `"posthoc"`.
adjust	Method for p-value adjustment for emmeans contrast. One of `"none"`, `"bonferroni"`, `"fdr"`, etc. Only used in contrast.
posthoc	Logical; if `TRUE`, performs posthoc test using agricolae.
posthoc_method	Posthoc method: one of `"LSD"`, `"BONFERRONI"`, `"FDR"`, `"SCHEFFE"`, `"DUNCAN"`, `"TUKEY"`, `"SNK"`.

Value

Depending on `type`, returns a list or specific object.

Examples

```
## Not run:
library(agricolae)
library(dplyr)
library(emmeans)
library(broom)
library(ggplot2)

# Example 1: mtcars dataset (mpg ~ cyl)
mtcars$cyl <- as.factor(mtcars$cyl)
fit1 <- aov(mpg ~ cyl, data = mtcars)
aov_post(fit1, type = "df", posthoc = TRUE, posthoc_method = "LSD")
aov_post(fit1, type = "df", posthoc = TRUE, posthoc_method = "BONFERRONI")
aov_post(fit1, type = "df", posthoc = TRUE, posthoc_method = "FDR")
aov_post(fit1, type = "df", posthoc = TRUE, posthoc_method = "SCHEFFE")
aov_post(fit1, type = "df", posthoc = TRUE, posthoc_method = "DUNCAN")
aov_post(fit1, type = "df", posthoc = TRUE, posthoc_method = "TUKEY")
aov_post(fit1, type = "df", posthoc = TRUE, posthoc_method = "SNK")

# Example 2: ToothGrowth dataset (len ~ dose)
ToothGrowth$dose <- as.factor(ToothGrowth$dose)
fit2 <- aov(len ~ dose, data = ToothGrowth)
aov_post(fit2, type = "df", posthoc = TRUE, posthoc_method = "DUNCAN")

# Example 3: iris dataset (Sepal.Length ~ Species)
fit3 <- aov(Sepal.Length ~ Species, data = iris)
aov_post(fit3, type = "df", posthoc = TRUE, posthoc_method = "SCHEFFE")

## End(Not run)
```

`aov_rename` *aov_rename*

Description

aov_rename

aov_rename

Usage

```
aov_rename(data)

aov_rename(data)
```

Arguments

`data`	data

Value

result

result

`aov_table` *Generate an AOV results table for multiple independent variables*

Description

Generate an AOV results table for multiple independent variables

Generate an AOV results table for multiple independent variables

Usage

```
aov_table(
  data,
  dv_var = NULL,
  iv_vars = NULL,
  grp_mean = TRUE,
  unite_p = FALSE,
  unite_F = FALSE,
  digits = 2,
  posthoc = "lsd",
  p.adj = "bonferroni",
  sig = FALSE
)

aov_table(
  data,
  dv_var = NULL,
  iv_vars = NULL,
  grp_mean = TRUE,
  unite_p = FALSE,
  unite_F = FALSE,
  digits = 2,
  posthoc = "lsd",
  p.adj = "bonferroni",
  sig = FALSE
)
```

Arguments

`data`	data.frame
`dv_var`	dv
`iv_vars`	c(iv1, iv2....)
`grp_mean`	'grooup mean remove
`unite_p`	pvalue unite
`unite_F`	Fvalue unite

posthoc	'tukey, scheffe, duncan, lsd
p.adj	p.adj=c("none","holm","hommel", "hochberg", "bonferroni", "BH", "BY", "fdr") only lsd test
sig	'sig = T' is add star

Value

table

table

Examples

```
## Not run:
mtcars <- mtcars%>%as_trt("cyl", "gear", "carb")
aov_table(data = mtcars, dv_var = "mpg", iv_vars = c("cyl", "gear", "carb"))

aov_table(data = mtcars, dv_var = "mpg",
iv_vars = c("cyl", "gear", "carb"), grp_mean = FALSE)
aov_table(data = mtcars, dv_var = "mpg",
iv_vars = c("cyl", "gear", "carb"), grp_mean = FALSE) %>%
aov_table_apaOP

aov_table(data = mtcars, dv_var = "mpg",
 iv_vars = c("cyl", "gear", "carb"), grp_mean = TRUE)

aov_table(data = mtcars, dv_var = "mpg",
 iv_vars = c("cyl", "gear", "carb"), grp_mean = TRUE, unite_F = TRUE)

aov_table(data = mtcars, dv_var = "mpg",
 iv_vars = c("cyl", "gear", "carb"),
  grp_mean = TRUE, unite_F = TRUE) %>% aov_rename()
  markdown_table()

## End(Not run)
```

```
## Not run:
mtcars <- mtcars%>%as_trt("cyl", "gear", "carb")
aov_table(data = mtcars, dv_var = "mpg", iv_vars = c("cyl", "gear", "carb"))

aov_table(data = mtcars, dv_var = "mpg",
iv_vars = c("cyl", "gear", "carb"), grp_mean = FALSE)
aov_table(data = mtcars, dv_var = "mpg",
iv_vars = c("cyl", "gear", "carb"), grp_mean = FALSE) %>%
aov_table_apaOP

aov_table(data = mtcars, dv_var = "mpg",
 iv_vars = c("cyl", "gear", "carb"), grp_mean = TRUE)

aov_table(data = mtcars, dv_var = "mpg",
 iv_vars = c("cyl", "gear", "carb"), grp_mean = TRUE, unite_F = TRUE)

aov_table(data = mtcars, dv_var = "mpg",
```

```
 iv_vars = c("cyl", "gear", "carb"),
  grp_mean = TRUE, unite_F = TRUE) %>% aov_rename()
  markdown_table()

## End(Not run)
```

`apa_aov_df` *total anova report*

Description

total anova report

Usage

```
apa_aov_df(data, show = TRUE, dv_var = NULL, iv_vars = NULL)
```

Arguments

`data`	aoa_df(grp_mean=F)
`show`	show data

Value

report

Examples

```
## Not run:
aov_table(data = mtcars, dv_var = "mpg",
iv_vars = c("cyl", "gear", "carb"),
grp_mean = FALSE)%>% apa_aov_df()

apa_aov_df(data = mtcars, dv_var = "mpg",
 iv_vars = c("cyl", "gear", "carb"))

## End(Not run)
```

apa_aov_df_res *apa_aov_df_res, aov_df* 의분석결과

Description

apa_aov_df_res, aov_df 의분석결과

Usage

```
apa_aov_df_res(data, dv_col = 2, iv_col = 1, show = TRUE, digits = 3)
```

Arguments

data	data
dv_col	dv
iv_col	iv
show	TRUE, data result
digits	3

Value

결과해석 text

Examples

```
## Not run:
#'
# 테스트 데이터로 실행
data <- data.frame(
  독립변수 = c("LCA_class", "", "", "LCA_class",
  "", "", "LCA_class", "", "", "LCA_class", "", ""),
  종속변수 = c("진로동기", "진로동기", "진로동기",
  "진로태도성숙", "진로태도성숙", "진로태도성숙",
  "진로결정자기효능감", "진로결정자기효능감", "진로결정자기효능감",
  "진로준비행동", "진로준비행동", "진로준비행동"),
  df1 = c(2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2),
  df2 = c(511, 511, 511, 511, 511, 511,
   511, 511, 511, 511, 511, 511),
  F = c(132.64, NA, NA, 107.45, NA, NA,
   138.02, NA, NA, 106.1, NA, NA),
  p = c("4.00665784818538e-47", NA, NA,
   "1.11197564656087e-39", NA, NA,
   "1.18550277068589e-48", NA, NA,
   "2.8829281865653e-39", NA, NA)
)

# show=TRUE로 결과 출력
apa_results <- apa_aov_df_res(data, dv_col = 2, iv_col = 1, show = TRUE)


## End(Not run)
```

`aroian_sobel_test_general` *Aroian Approximation Sobel Test generalized function*

Description

Aroian Approximation Sobel Test generalized function

Usage

```
aroian_sobel_test_general(
  a,
  SE,
  roburst = FALSE,
  n = NULL,
  k = 2,
  digits = 5,
  conf = 0.95
)
```

Arguments

a	est
SE	se
roburst	roburst T whide correction
n	sample size
k	number of regressin coefs, default 2
digits	5
conf	0.90, 0.95, 0.99, 0.999

Value

result

Examples

```
## Not run:
#'
# Example usage
a <- 0.5  # Example coefficient a
b <- 0.3  # Example coefficient b
sa <- 0.1 # Standard error of a
sb <- 0.05 # Standard error of b

Sobel_test(a, b, sa, sb)
Sobel_test(a, b, sa, sb, "all")
Sobel_test(a, b, sa, sb, "Sobel")

Sobel_test(a, b, sa, sb,"Aroian")
```

```
Sobel_test(a, b, sa, sb,"Goodman")

#'
# 예제 데이터
a <- c(0.5, 0.3, 0.4)
SE <- c(0.1, 0.1, 0.1)

# 함수 실행 (기본 95% 신뢰구간)
aroian_sobel_test_general(a, SE)
print(result_95)

# 함수 실행 (90% 신뢰구간)
result_90 <- aroian_sobel_test_general(a, SE, conf=0.90)
print(result_90)

# 함수 실행 (99% 신뢰구간)
result_99 <- aroian_sobel_test_general(a, SE, conf=0.99)
print(result_99)

# 함수 실행 (99.9% 신뢰구간)
result_999 <- aroian_sobel_test_general(a, SE, conf=0.999)
print(result_999)


aroian_sobel_test_general(a = c(0.2, 0.3, 0.4, 0.5),
                          SE = c(0.05, 0.04, 0.03, 0.02))%>%unite_ci()


## End(Not run)
```

`arrange_by_mind` *Arrange dataframe by user-defined priority on rhs column*

Description

This function arranges a data frame according to a user-defined priority of values in the `rhs` column. If no priority is supplied, the data frame is sorted alphabetically by `rhs`.

Usage

```
arrange_by_mind(df, priority = NULL)
```

Arguments

`df`	A data frame containing at least a column named `rhs`.
`priority`	An optional character vector specifying the preferred order of values in the `rhs` column. If `NULL`, rows are sorted alphabetically by `rhs`.

Details

When `priority` is provided, values of `rhs` that appear in `priority` are ordered first according to that order. Remaining values are placed afterward and sorted alphabetically. If `priority` is `NULL`, the function simply sorts the data frame alphabetically by `rhs`.

Value

A data frame sorted by user-defined priority and/or alphabetically by `rhs`.

Examples

```
## Not run:
df <- data.frame(
  lhs = c("intention", "access", "efficacy"),
  rhs = c("barrier", "experience", "grade"),
  est = c(0.2, 0.4, -0.1)
)

# Example 1: Alphabetical ordering only
arrange_by_mind(df)

# Example 2: User-defined priority
arrange_by_mind(df,
  priority = c("experience", "grade")
)

# Example 3: Partial priority specification
arrange_by_mind(df,
  priority = "experience"
)

## End(Not run)
```

---

as_trt *Treat selected variables as factors*

Description

Treat selected variables as factors

Usage

```
as_trt(data, ..., fun = factor)
```

Arguments

`data`	data.frame
`...`	variables
`fun`	funtion default as.factor

Value

data treatment

Examples

```
## Not run:
 str(mrcars)

 mtcars1 = mtcars %>% sel_as_trt("cyl", "am","vs", "gear", "carb")
 str(mtcars1)

 #col number
 mtcars%>%as_trt(c(2,8:11))%>%str()
#'
# 데이터 프레임 생성
df <- data.frame(matrix(1:500, ncol=100))
colnames(df) <- paste0("P", sprintf("%02d", 1:100))

# 열 번호 범위로 변환
as_trt(df, 81:91)%>%str()
as_trt(df, "P81","P91")%>%str()
as_trt(df, 81, 92)%>%str()
#'



## End(Not run)
```

---

as_trt2 *Treat selected variables as factors*

Description

Treat selected variables as factors

Usage

```
as_trt2(data, ..., fun = factor)
```

Arguments

data	data.frame
...	variables
fun	funtion default as.factor

Value

data treatment

Examples

```
## Not run:
 str(mrcars)

 mtcars1 = mtcars %>% as_trt("cyl", "am","vs", "gear", "carb")
 str(mtcars1)
```

```
 #col number
 mtcars%>%as_trt(c(2,8:11))%>%str()
#'
# 데이터 프레임 생성
df <- data.frame(matrix(1:500, ncol=100))
colnames(df) <- paste0("P", sprintf("%02d", 1:100))

# 열 번호 범위로 변환
as_trt(df, 81:91)%>%str()
as_trt(df, "P81:P91")%>%str()
as_trt(df, "P81","P91")%>%str()
as_trt(df, 81, 92)%>%str()
#'



## End(Not run)
```

`auto_coding` *Automated Likert scale conversion functions*

Description

Automated Likert scale conversion functions

Usage

```
auto_coding(
  data,
  ...,
  range = 1:ncol(data),
  custom = NULL,
  rev = FALSE,
  data.fram = FALSE,
  start_number = 1,
  default = c("전혀 그렇지 않다", "그렇지 않다", "보통이다", "그렇다",
    "매우 그렇다")
)
```

Arguments

`data`	data.frame, vector
`...`	coding levels
`range`	data col range#'
`start_number`	start_number = 1
`defaultc(" 전혀`	그렇지않다", " 그렇지않다"," 보통이다", " 그렇다", " 매우그렇다")

Value

transforamtion data

Examples

```
## Not run:
#'

# test
jut1_custom <- c("그렇다", "매우 그렇다", "보통이다",
                 "그렇지 않다", "전혀 그렇지 않다")
jut1_custom
jut1_custom%>%rev()


jut1_custom <- c("그렇다", "매우 그렇다", "보통이다", "그렇지 않다", "전혀 그렇지 않다")

auto_coding(jut1_custom, custom=jut1_custom)
auto_coding(jut1_custom, custom=jut1_custom, rev=TRUE)


# Example data
vec1 <- c("그렇다", "그렇다", "매우 그렇다", "그렇다", "매우 그렇다", "매우 그렇다",
          "매우 그렇다", "그렇다", "그렇다", "그렇지 않다", "그렇다", "그렇다",
          "매우 그렇다", "그렇다", "그렇다", "그렇다", "그렇다", "매우 그렇다",
          "전혀 그렇지 않다", "보통이다", "그렇다", "매우 그렇다", "매우 그렇다",
          "보통이다", "그렇다", "매우 그렇다", "그렇다", "그렇다", "매우 그렇다",
          "매우 그렇다", "그렇다", "그렇다", "그렇다", "그렇다", "보통이다",
          "그렇다", "그렇다", "그렇다", "매우 그렇다", "보통이다", "매우 그렇다",
          "매우 그렇다", "그렇다", "그렇다", "그렇지 않다", "그렇다", "그렇다",
          "그렇다", "보통이다", "그렇다", "그렇다", "보통이다", "그렇지 않다",
          "그렇지 않다", "매우 그렇다", "보통이다", "그렇다", "매우 그렇다",
          "그렇다", "매우 그렇다", "그렇다", "그렇다", "그렇다", "그렇다",
          "그렇다", "매우 그렇다", "그렇다", "보통이다", "그렇다", "그렇다",
          "그렇다", "매우 그렇다", "보통이다", "그렇다", "그렇다", "그렇지 않다",
          "그렇지 않다", "그렇지 않다", "그렇지 않다", "매우 그렇다", "매우 그렇다",
          "그렇지 않다", "그렇다", "전혀 그렇지 않다", "그렇다", "그렇다",
          "그렇다", "보통이다", "그렇다", "그렇지 않다", "그렇다", "보통이다",
          "매우 그렇다")

auto_coding(vec1, custom=jut1_custom, rev=FALSE)
auto_coding(vec1, custom=jut1_custom, rev=TRUE)



# Example data
data1 <- data.frame(
  V1 = c("그렇다", "그렇다", "매우 그렇다", "그렇다", "매우 그렇다",
         "매우 그렇다", "매우 그렇다", "그렇다", "그렇다", "그렇지 않다",
         "그렇다", "그렇다", "매우 그렇다", "그렇다", "그렇다",
         "그렇다", "그렇다", "매우 그렇다",  "전혀 그렇지 않다", "보통이다",
         "그렇다", "매우 그렇다", "매우 그렇다", "보통이다", "그렇다",
         "매우 그렇다", "그렇다", "그렇다", "매우 그렇다",  "매우 그렇다",
         "그렇다", "그렇다", "그렇다", "그렇다", "보통이다",
         "그렇다", "그렇다", "그렇다", "매우 그렇다", "보통이다",
         "매우 그렇다", "매우 그렇다", "그렇다", "그렇다", "그렇지 않다",
         "그렇다", "그렇다",
         "그렇다", "보통이다", "그렇다", "그렇다", "보통이다", "그렇지 않다",
         "그렇지 않다", "매우 그렇다", "보통이다", "그렇다", "매우 그렇다",
```

```
          "그렇다", "매우 그렇다", "그렇다", "그렇다", "그렇다", "그렇다",
          "그렇다", "매우 그렇다", "그렇다", "보통이다", "그렇다", "그렇다",
          "그렇다", "매우 그렇다", "보통이다", "그렇다", "그렇다", "그렇지 않다",
          "그렇지 않다", "그렇지 않다", "그렇지 않다", "매우 그렇다", "매우 그렇다",
          "그렇지 않다", "그렇다", "전혀 그렇지 않다", "그렇다", "그렇다",
          "그렇다", "보통이다", "그렇다", "그렇지 않다", "그렇다", "보통이다",
          "매우 그렇다", "보통이다")
)

auto_coding(data1)
data1

# Example data
data2 <- data.frame(
  V1 = c("그렇다", "그렇다", "매우 그렇다", "그렇다", "매우 그렇다",
         "매우 그렇다", "매우 그렇다", "그렇다", "그렇다", "그렇지 않다",
         "그렇다", "그렇다", "매우 그렇다", "그렇다", "그렇다",
         "그렇다", "그렇다", "매우 그렇다",  "전혀 그렇지 않다", "보통이다",
         "그렇다", "매우 그렇다", "매우 그렇다", "보통이다", "그렇다",
         "매우 그렇다", "그렇다", "그렇다", "매우 그렇다",  "매우 그렇다",
         "그렇다", "그렇다", "그렇다", "그렇다", "보통이다",
         "그렇다", "그렇다", "그렇다", "매우 그렇다", "보통이다",
         "매우 그렇다", "매우 그렇다", "그렇다", "그렇다", "그렇지 않다",
         "그렇다", "그렇다"),
  v2=c( "그렇다", "보통이다", "그렇다", "그렇다", "보통이다", "그렇지 않다",
        "그렇지 않다", "매우 그렇다", "보통이다", "그렇다", "매우 그렇다",
        "그렇다", "매우 그렇다", "그렇다", "그렇다", "그렇다", "그렇다",
        "그렇다", "매우 그렇다", "그렇다", "보통이다", "그렇다", "그렇다",
        "그렇다", "매우 그렇다", "보통이다", "그렇다", "그렇다", "그렇지 않다",
        "그렇지 않다", "그렇지 않다", "그렇지 않다", "매우 그렇다", "매우 그렇다",
        "그렇지 않다", "그렇다", "전혀 그렇지 않다", "그렇다", "그렇다",
        "그렇다", "보통이다", "그렇다", "그렇지 않다", "그렇다", "보통이다",
        "매우 그렇다", "보통이다")
)
data2
auto_coding(data2)
# jutr1 %>% auto_coding( range=91:95, custom=c("아니다.","그렇다."),start_number=0)%>%dall("head")
# jutr1 %>% auto_coding( range="L01:L05", custom=c("아니다.","그렇다."),start_number=0)%>% head(30)

## End(Not run)
```

`ave_candidates_plot` *ave_candidates_plot:* 문항제거전후 *AVE* 비교시각화

Description

ave_candidates_plot: 문항제거전후 AVE 비교시각화

Usage

```
ave_candidates_plot(
  df,
  latent = NULL,
  range_focus = TRUE,
```

```
  font_size = 4,
  padding = 0.04,
  v_line_show = TRUE
)
```

Arguments

df	lavaan 객체또는 ave_raise_candidates() 결과
latent	특정잠재변수만시각화 (선택)
range_focus	TRUE 면 AVE 변화중심으로 x 축범위조정
font_size	텍스트크기
padding	범위조절값 (기본 0.04)
v_line_show	각문항의 ave_current 위치에수직선추가여부

Value

ggplot2 객체

Examples

```
## Not run:
library(lavaan)
library(ggplot2)
library(dplyr)

# 예시 데이터: HolzingerSwineford1939
model <- '
  visual  =~ x1 + x2 + x3
  textual =~ x4 + x5 + x6
  speed   =~ x7 + x8 + x9
'
fit <- cfa(model, data = lavaan::HolzingerSwineford1939)

# 1. lavaan 객체 입력 시 자동 처리
ave_candidates_plot(fit)

# 2. 특정 잠재변수만 보기
ave_candidates_plot(fit, latent = "visual")

# 3. AVE 변화량이 두드러진 경우에만 확대해서 보기
ave_candidates_plot(fit, range_focus = TRUE)

# 4. AVE 변화 전후 수직선 제거
ave_candidates_plot(fit, v_line_show = FALSE)

# 5. font 크기 조절
ave_candidates_plot(fit, font_size = 3)

# 6. ave_raise_candidates() 결과를 직접 전달
result <- ave_raise_candidates(fit, print_result = FALSE)
ave_candidates_plot(result)

## End(Not run)
```

`ave_raise_candidates` *ave_raise_candidates:* 잠재변수별 *AVE* 향상후보문항탐색

Description

lavaan 모형결과를기반으로각잠재변수별로문항제거시 AVE(Average Variance Extracted) 의변화량을계산합니다. 모든잠재변수를자동으로분석하거나, 특정잠재변수만선택할수있습니다.

Usage

```
ave_raise_candidates(fit, latent = NULL, cutoff = -Inf, print_result = TRUE)
```

Arguments

`fit`	lavaan::cfa 또는 lavaan::sem 결과객체
`latent`	분석대상잠재변수이름 (문자열벡터). NULL 이면모델내모든잠재변수를자동분석합니다.
`cutoff`	AVE 변화량 (delta) 의최소기준. 기본값은 -Inf.
`print_result`	논리값. TRUE 이면잠재변수별요약메시지를출력합니다.

Value

tibble

latent 잠재변수이름

Item 문항이름

lambda 표준화요인적재량

ave_current 현재 AVE

ave_if_dropped 문항제거시 AVE

delta AVE 변화량 (제거후 - 제거전)

Examples

```
## Not run:
library(lavaan)
library(dplyr)
library(purrr)
library(magrittr)

#----------------------------------------
# 1. CFA 모델 적합
#----------------------------------------
model <- '
  Visual  =~ x1 + x2 + x3
  Textual =~ x4 + x5 + x6
  Speed   =~ x7 + x8 + x9
'
fit <- cfa(model, data = HolzingerSwineford1939, std.lv = TRUE)

#----------------------------------------
```

```
# 2. 모든 잠재변수 자동 분석
#----------------------------------------
ave_raise_candidates(fit)

#----------------------------------------
# 3. 특정 잠재변수만 분석
#----------------------------------------
ave_raise_candidates(fit, latent = "Visual")

#----------------------------------------
# 4. AVE가 0.02 이상 증가하는 경우만 필터링
#----------------------------------------
ave_raise_candidates(fit, cutoff = 0.02)

#----------------------------------------
# 5. 여러 잠재변수 선택 분석
#----------------------------------------
ave_raise_candidates(fit, latent = c("Visual", "Textual"))

## End(Not run)
```

`bartlett_test` *bartlett_test*

Description

bartlett_test

Usage

```
bartlett_test(
  data,
  formula = NULL,
  Mean = "Mean",
  sd = "SD",
  n = "N",
  var = "variable"
)
```

Arguments

`data`	data.frame or mean. sd. n
`formula`	dv ~ iv
`Mean`	mean selection
`sd`	sd selection
`n`	n selection
`var`	variable

Value

result

Examples

```
## Not run:

set.seed(20240418)
my_data <- data.frame(
  value = c(group1 <- rnorm(20, mean = 10, sd = 2),
            group2 <- rnorm(20, mean = 12, sd = 2),
            group3 <- rnorm(20, mean = 15, sd = 2)),
  group = rep(c("Group1", "Group2", "Group3"), each = 20)
)
my_data %>%mysummaryBy(value ~ group)%>%select(1:4) %>%bartlett_test()

data = data.frame(row.names = c("1", "2", "3"),
                  var = c('Group1', 'Group2', 'Group3'),
                  Mean = c('10.0782306287367', '11.5824600813951', '14.764623393546'),
                  SD = c('1.88743751774179', '1.9951475292551', '2.07862332265816'),
                  N = c('20', '20', '20'))
data
bartlett_test(data)

# original test
bartlett.test(value ~ group, data=my_data)
#now test
bartlett_test(value ~ group, data=my_data)


#origin
bartlett.test(mpg~cyl, data=mtcars)
#aggregated data
mtcars%>%mysummaryBy(mpg~cyl) %>%bartlett_test()
# Using function
bartlett_test(mpg~cyl, data=mtcars)
#'

## End(Not run)
```

`binary_recode` *binary_recode*

Description

binary_recode

Usage

```
binary_recode(data, variable, recode_list, new_name = NULL, tail = "_grp")
```

Arguments

`data`	data
`variable`	original variable
`recode_list`	list recode variable

new_name	new name
tail	tails _grp

Value

data

Examples

```
## Not run:
#'
# 예제 데이터
data <- data.frame(
  학력 = factor(c("고졸", "대학졸", "석박사졸", "고졸", "석박사졸")),
  grade = 1:5
)
data
binary_recode(data, grade, list(미숙자 = 1:3, 숙달자 = 4:5))
binary_recode(data, grade, list(미숙자 = c(1,2,3), 숙달자 = 4:5))
binary_recode(data, "grade", list(미숙자 = 1:3, 숙달자= 4:5), new_name="수준")

binary_recode(data, 학력, list(저학년= 1:2, 고학년= 3))
binary_recode(data, "학력", list(저학년= 1:2, 고학년= 3))
binary_recode(data, "학력", list(저학년= c("고졸","대학졸"), 고학년= 3))
binary_recode(data, 학력, list(저학년= c("고졸","대학졸"), 고학년= 3))
binary_recode(data, 학력, list(저학년= c("고졸","대학졸"), 고학년= "석박사졸"))


## End(Not run)
```

---

binary_recode_sep *Multi-recoding binary variables*

Description

Multi-recoding binary variables

Usage

```
binary_recode_sep(data, ..., new_names = NULL, tail = "_grp")
```

Arguments

data	data
...	recode list
new_names	newnaem list
tail	tail _grp

Value

data

Examples

```
## Not run:



# 예제 데이터
data <- data.frame(
  학력 = factor(c("고졸", "대학졸", "석박사졸", "고졸", "석박사졸")),
  grade = 1:5
)
data


# 함수 호출
binary_recode_sep(  data,
                    "학력", list(저학년 = 1:2, 고학년 = 3),
                    "grade", list(저학년 = 1:2, 고학년 = 3:5),
                    new_names = c("학력_grp", "grade_grp")
)




## End(Not run)
```

---

bind_alpha_table *Alpha Table Combination*

Description

Alpha Table Combination

Usage

```
bind_alpha_table(data, ..., check.keys = TRUE)
```

Arguments

`data`	Data frame containing the variables
`...`	Variable names to be included in the alpha table, column name and number. Variable names to be included in the alpha table
`check.keys`	check.keys=TRUE

Value

alpha table

Examples

```
## Not run:
## select variable
bind_alpha_table(data = stat_onl, c("S_Review", "S_Add_learn", "S_Feedback"))

#error
bind_alpha_table(data = stat_onl, c(S_Review, S_Add_learn, S_Feedback))

#select col number
bind_alpha_table(data = stat_onl, c(9:11))

# mutilple data
bind_alpha_table( stat_onl,
                  c("S_Review", "S_Add_learn", "S_Feedback"),
                  c("SE_Place", "SE_Time"),
                  c("On_Satisfy", "On_Joy", "On_Easy", "On_Engage"),
                  c("upgrade","satisfy","fatigue_inverse") )


#mixed multiple
bind_alpha_table( stat_onl,
                  c("S_Review", "S_Add_learn", "S_Feedback"),
                  c("SE_Place", "SE_Time"),
                  c("On_Satisfy", "On_Joy", "On_Easy", "On_Engage"),
                  c(5:6, 21) )




## End(Not run)
```

---

bind_compare_data *Table-building functions that pairwise compare two pieces of data*

Description

Table-building functions that pairwise compare two pieces of data

Usage

```
bind_compare_data(dff1, dff2, digits = 3, pair = TRUE)
```

Arguments

dff1	data1
dff2	data2
digits	rounding
pair	pari option default TRUE

Value

table

`bind_lme_model` *Tbind_lme_model*

Description

Tbind_lme_model

Usage

```
bind_lme_model(
  ...,
  model_name = paste("모형", 1:length(model)),
  title = "모형의 비교"
)
```

Arguments

`...`	input
`model_name`	model name
`title`	model title

Value

table

Examples

```
## Not run:
data(birthwt)
str(birthwt)
fitm <- lm(bwt ~ age+lwt+factor(race)+smoke+ptl+ht+ui, data=birthwt)
fitm2 <- lm(bwt ~ lwt+factor(race)+smoke+ht+ui, data=birthwt)
bind_lme_model(fitm, fitm2)


## End(Not run)
```

`bind_mysummaryBy` *bind_mysummaryBy is group mean of mean*

Description

bind_mysummaryBy is group mean of mean

Usage

```
bind_mysummaryBy(data, ..., unite = FALSE)
```

Arguments

data	data.frame
...	fomula data ...
unite	... bind persional data and group data TRUE

Value

Use group data obtained from group descriptive statistics to obtain group-level descriptive statistics (means, standard deviations, etc.) and to analyze a mixed model

Examples

```
## Not run:
##
bind_mysummaryBy(mtcars, mpg ~ am)

##compare
mysummaryBy(data=mtcars, mpg ~ am)

group_by(mtcars, am) %>%  summarise(mpg = mean(mpg))

 ## A tibble: 2 × 2
 ##    am   mpg
 ##<dbl> <dbl>
 ## 1     0  17.1
 ##2     1  24.4




bind_mysummaryBy(mtcars, mpg ~ am, mpg ~ vs)


## grp   dv        N  MEAN    SD   MIN   MAX
##  <chr> <chr> <int> <dbl> <dbl> <dbl> <dbl>
##1 am    mpg       2  20.8  5.12  17.1  24.4
##2 vs    mpg       2  20.6  5.61  16.6  24.6
#

bind_mysummaryBy(mtcars, mpg ~ am, mpg ~ vs, mpg ~cyl)

# A tibble: 3 × 9
# grp   dv        N  MEAN    SD   MIN   MAX      Skew  Kurt
# <chr> <chr> <int> <dbl> <dbl> <dbl> <dbl>     <dbl> <dbl>
#   1 am    mpg       2  20.8  5.12  17.1  24.4 -5.28e-16 -2.75
# 2 vs    mpg       2  20.6  5.61  16.6  24.6  0        -2.75
# 3 cyl   mpg       3  20.5  5.82  15.1  26.7  1.28e- 1 -2.33


jjstat::bind_mysummaryBy(mtcars, mpg ~ am, mpg ~ vs, mpg ~ cyl, unite=TRUE)

# A tibble: 3 × 8
# var         N  MEAN    SD   MIN   MAX      Skew  Kurt
# <chr>   <int> <dbl> <dbl> <dbl> <dbl>     <dbl> <dbl>
#   1 am_mpg      2  20.8  5.12  17.1  24.4 -5.28e-16 -2.75
# 2 vs_mpg      2  20.6  5.61  16.6  24.6  0        -2.75
```

```
# 3 cyl_mpg     3  20.5  5.82  15.1  26.7  1.28e- 1 -2.33




## End(Not run)
```

---

box_cox *Apply Box-Cox Style Transformations to a Data Frame*

Description

This function applies various mathematical transformations to all numeric columns in a data frame.

Usage

```
box_cox(df, use = "log")
```

Arguments

df	A data frame with numeric columns to be transformed.
use	A character string specifying the transformation to apply. Options are: • "log": Logarithmic transformation ($\log(x + 1)$) • "sqrt": Square root transformation ($\sqrt{x}$) • "inverse": Inverse transformation ($1/(x + 1)$) • "square": Square transformation (x^2) • "cube_root": Cube root transformation ($x^{1/3}$) • "exp": Exponential transformation (e^x)

Value

A data frame with transformed numeric columns.

Examples

```
## Not run:
# Example: Apply transformations to a data frame
df <- data.frame(
  col1 = c(1, 2, 3),
  col2 = c(4, 5, 6)
)

# Logarithmic transformation
box_cox(df, use = "log")

# Square root transformation
box_cox(df, use = "sqrt")

# Cube root transformation
box_cox(df, use = "cube_root")

## End(Not run)
```

`breslowdata_test` *Breslow-Day Test for Homogeneity of the Odds Ratios*

Description

Breslow-Day Test for Homogeneity of the Odds Ratios

Usage

```
breslowdata_test(data, OR = NA, correct = FALSE, tidy = TRUE)
```

Arguments

`data`	2x2xk table
`OR`	the odds ratio to be tested against. If left undefined (default) the Mantel-Haenszel estimate will be used.
`correct`	If TRUE, the Breslow-Day test with Tarone's adjustment is computed, which subtracts an adjustment factor to make the resulting statistic asymptotically chi-square.
`tidy`	broom::tidy

Details

For the Breslow-Day test to be valid, the sample size should be relatively large in each stratum, and at least 80 Alternatively, it might be better to cast the entire inference problem into the setting of a logistic regression model. Here, the underlying question of the Breslow-Day test can be answered by investigating whether an interaction term with the strata variable is necessary (e.g. using a likelihood ratio test using the anova function).

Value

result

Examples

```
## Not run:
#'
migraine <- xtabs(freq ~ .,
                  cbind(expand.grid(treatment=c("active", "placebo"),
                                    response =c("better", "same"),
                                    gender   =c("female", "male")),
                        freq=c(16, 5, 11, 20, 12, 7, 16, 19))
)
# get rid of gender
tab <- xtabs(Freq ~ treatment + response, migraine)
DescTools::Desc(tab)
# only the women
female <- migraine[,, 1]
DescTools::Desc(female)
# .. and the men
male <- migraine[,, 2]
DescTools::Desc(male)
```

```
breslowdata_test(migraine)
breslowdata_test(migraine, correct = TRUE)


## End(Not run)
```

`calculate_chi_sig` *Create oserved/expected tapes after chisq calculation and give them individual significance*

Description

Create oserved/expected tapes after chisq calculation and give them individual significance

Usage

```
calculate_chi_sig(observed, type = "data", digits = 3, exact = TRUE)
```

Arguments

`observed`	contigency table
`type`	data, Res(all), chisq_test, cramers v, observed_over_expected_ratios,cell_p_values, p_sig, data, data2
`digits`	digits= 3
`exact`	yYates's continuity correction

Value

matrix, data.frame

Examples

```
## Not run:
# # calculate_chi_sig(data)
# Example data
matrix(c(36, 67, 11,
          0, 35, 44,
         41, 108, 1,
          56, 87, 54),
           nrow = 4, byrow = TRUE,
   dimnames = list(c("공명음", "마찰음",
     "유기음_경음", "평파열음_평파찰음"),  c("H", "H(H)", "L"))) %>%
     calculate_chi_sig("p_sig")


     matrix(c(36, 67, 11,
              40, 35, 44,
              41, 108, 1,
              56, 87, 54), nrow = 4, byrow = TRUE,
                     dimnames = list(c("공명음", "마찰음",
                                       "유기음_경음", "평파열음_평파찰음"),
                                     c("H", "H(H)", "L"))) %>%
```

```
                calculate_chi_sig(simple = TRUE)


  calculate_chi_sig(matrix(c(36, 67, 11,
                     40, 35, 44,
                  41, 108, 1,
              56, 87, 54), nrow = 4, byrow = TRUE,
               dimnames = list(c("공명음", "마찰음",
              "유기음_경음", "평파열음_평파찰음"),
              c("H", "H(H)", "L")))   )


  # calculate_chi_sig(data)

## End(Not run)
```

`cal_vif` *calculation vif*

Description

calculation vif

Usage

```
cal_vif(model)
```

Arguments

`model`	lm model data

Value

vif

Examples

```
## Not run:
#'
# Create the linear model
model <- lm(hp ~ wt + disp + drat, data = mtcars)

# Calculate VIF using the custom function
cal_vif(model)

lm(mpg ~ wt + disp + drat + hp , data = mtcars) %>%cal_vif()



## End(Not run)
```

`centering` *centering each vector*

Description

centering each vector

Usage

```
centering(variable, na.rm = TRUE)
```

Arguments

`variable`	vector variable in data
`na.rm`	missing data chedk

`cfa2` *Create CFA result summary by lavaan sem estimated result By Park Joonghee Ph.D*

Description

Create CFA result summary by lavaan sem estimated result By Park Joonghee Ph.D

Usage

```
cfa2(
  x,
  format = "markdown",
  htmt_cut = 0.9,
  htmt2 = FALSE,
  cut = 0.7,
  angle = 90,
  cex = 11,
  hjust = 0.9,
  val.size = 4,
  dis.sort = TRUE,
  rename = FALSE,
  var_name = NA,
  digits = 3,
  type = "all"
)
```

Arguments

`x`	lavaan result
`format`	knitr markdown decision
`htmt_cut`	htmt_cut is default =1. if you want roburst option htmt_cut =0.9
`htmt2`	TRUE using htmt2

cut	loading criteran
angle	tesxt angle
cex	text size
hjust	on text value gap
val.size	text size
dis.sort	item sort
rename	item validiation new name apply
var_name	new text name default NULL
digits	value rounding
dataset	this is import name that dataset column name
model	lavaan model need indices validation name
res	result type

Examples

```
## Not run:
library(lavaan)
data(PoliticalDemocracy)

model1 <- "
  ind60 =~ x1 + x2 + x3
  dem60 =~ y1 + y2 + y3 + y4
  dem65 =~ y5 + y6 + y7 + y8
  dem60 ~ ind60
  dem65 ~ ind60 + dem60
"

fit1 <- sem(model1, data = PoliticalDemocracy)

# 기본 사용법: 전체 결과 출력
cfa2(fit1)

# 특정 결과(res)만 출력 예시
cfa2(fit1, res = "modelfit")        # 모델 적합도
cfa2(fit1, res = "loadings")        # 요인적재량 테이블
cfa2(fit1, res = "alpha")           # 신뢰도(Cronbach alpha)
cfa2(fit1, res = "CR_AVE")          # 수렴타당도(CR, AVE)
cfa2(fit1, res = "Discriminant")    # 판별타당도 (Fornell-Larcker 기준)
cfa2(fit1, res = "htmt", dataset = PoliticalDemocracy, model = model1)  # HTMT 지표
cfa2(fit1, res = "str_cor")         # 잠재변수 간 경로 유의성

# 시각화 결과: 요인적재량 바그래프 출력
cfa2(fit1, res = "loadings_bar")

# 사용자 정의 변수명 적용(rename + var_name)
new_names <- c("Indicator1", "Indicator2", "Indicator3",
               "Democracy1", "Democracy2", "Democracy3", "Democracy4",
               "Democracy5", "Democracy6", "Democracy7", "Democracy8")

# 그래프의 색을 바꾸고 싶은 경우
cfa2(fit, type = "loadings_bar") +
scale_fill_grey()
# rename = TRUE일 경우에만 var_name이 반영됨
```

```
cfa2(fit1, rename = TRUE, var_name = new_names)

# rename = FALSE일 경우 var_name은 무시됨
cfa2(fit1, rename = FALSE, var_name = new_names)

# 다양한 출력 포맷 지정
cfa2(fit1, format = "markdown")
cfa2(fit1, format = "html") %>% kableExtra::kable_minimal(full_width = FALSE)

# 모델 간 비교
model2 <- "
  ind60 =~ x1 + x2 + x3
  dem60 =~ y1 + y2 + y3 + y4
  dem65 =~ y5 + y6 + y7 + y8
  dem60 ~ ind60
  dem65 ~ ind60 + dem60
"
fit2 <- sem(model2, data = PoliticalDemocracy)
CompareFit_diff(fit1, fit2)

# AVE 직접 확인
AVE(fit1)
AVE(fit2)

# 통합 요약 테이블 출력 (CR, AVE 포함)
library(dplyr)
full_join(
  cfa2(fit1, type = "item") %>% dplyr::select(-Accept, -p),
  cfa2(fit1, type = "Convergent") %>% row2col("Latent") %>%
    dplyr::select(-Cronbach) %>% Round(3),
  by = "Latent"
) %>% nice_table()

## End(Not run)
```

`cfa_ml` *Multilevel CFA summary wrapper*

Description

다층구조방정식 (CFA/SEM) 모형의 within / between 수준결과를한번에요약하여제공하는래퍼함수이다.

Usage

```
cfa_ml(
  fit,
  level = c("within", "between"),
  type = c("all", "model_fit", "indicator_validity", "CR_AVE", "item_CR_AVE", "fornell",
    "htmt"),
  digits = 3
)
```

Arguments

`fit`	lavaan multilevel SEM object
`level`	character. "within" 또는"between"
`type`	character. 출력제어옵션

- "all" 모든결과를 list 로반환
- "model_fit" 모형적합도
- "indicator_validity" 요인부하량및지표타당도
- "CR_AVE" 잠재변수수준 CR / AVE
- "item_CR_AVE" 문항수준 CR / AVE
- "fornell"Fornell–Larcker 판별타당도
- "htmt"HTMT 판별타당도

`digits`	numeric. 소수점자리수

Value

data.frame 또는 list

Examples

```
## Not run:
res_all <- cfa_ml(fit_ml, level = "within", type = "all")
res_fit <- cfa_ml(fit_ml, level = "between", type = "model_fit")

## End(Not run)
```

`change_node_name` *Change node names in semPlot / qgraph object*

Description

semPlot::semPaths() 로생성된구조방정식시각화객체에서노드이름을사용자가지정한 (원래이름, 변경라벨) 쌍에따라일괄적으로변경한다. 본함수는 pipe(

Usage

```
change_node_name(plot_obj, var_named)
```

Arguments

`plot_obj`	semPlot 또는 qgraph 객체
`var_named`	character vector. c(old1, new1, old2, new2, ...)

Value

노드명칭이변경된 semPlot 객체

Examples

```
## Not run:

#################################
# 1. SCCT 구조방정식 모형 정의
#################################

model_scct_indirect <- '

REFF =~ efficacy2 + efficacy3 + efficacy4
OEXP =~ Outcome1 + Outcome2 + Outcome3 + Outcome4 + Outcome5 + Outcome8
INTR =~ Interest1 + Interest2 + Interest3 + Interest4

REFF ~ e1*grade + e2*experience0
OEXP ~ a1*REFF + d1*grade + d2*experience0
INTR ~ b1*REFF + b2*OEXP

intention ~ c1*INTR + c2*OEXP + c3*REFF

grade ~~ experience0

# 간접효과
ind_grade_REFF        := e1 * c3
ind_grade_REFF_INTR   := e1 * b1 * c1
ind_grade_REFF_OEXP   := e1 * a1 * c2
ind_grade_REFF_OEXP_INTR := e1 * a1 * b2 * c1

ind_exp_REFF          := e2 * c3
ind_exp_REFF_INTR     := e2 * b1 * c1
ind_exp_REFF_OEXP     := e2 * a1 * c2
ind_exp_REFF_OEXP_INTR := e2 * a1 * b2 * c1
'

#################################
# 2. 모형 추정
#################################

library(lavaan)
fit_scct_indirect <- sem(
  model = model_scct_indirect,
  data  = shindata,
  se = "bootstrap",
  bootstrap = 2000
)

#################################
# 3. 노드 명칭 매핑 벡터
#################################

var_named1 <- c(
  "REFF", "연구효능감",
  "efficacy2","연구효능2",
  "efficacy3","연구효능3",
  "efficacy4","연구효능4",

  "INTR", "진로흥미",
```

```
  "Interest1","진로흥미1",
  "Interest2","진로흥미2",
  "Interest3","진로흥미3",
  "Interest4","진로흥미4",

  "OEXP", "결과기대",
  "Outcome1","결과기대1",
  "Outcome2","결과기대2",
  "Outcome3","결과기대3",
  "Outcome4","결과기대4",
  "Outcome5","결과기대5",
  "Outcome8","결과기대8",

  "grade","학년",
  "experience0","연구경험",
  "intention","진로의지"
)

################################
# 4. SCCT 전용 layout 행렬
################################

layout_scct <- matrix(
  c(
    NA, "efficacy2", "efficacy3", "efficacy4", NA,
        "Interest1", "Interest2", "Interest3", "Interest4", NA,

    "experience0", NA, "REFF", NA, NA,
        NA, "INTR", NA, NA, NA,

    "grade", NA, "OEXP", NA, NA,
        NA, NA, "intention", NA, NA,

    NA, "Outcome1", "Outcome2", "Outcome3", "Outcome4",
        "Outcome5", "Outcome8", NA, NA, NA
  ),
  nrow = 4,
  byrow = TRUE
)

################################
# 5. 구조방정식 그림 + 노드명 변경
################################

library(semPlot)
library(semptools)

semPlot::semPaths(
  fit_scct_indirect,
  layout        = layout_scct,
  whatLabels    = "std",
  style         = "ram",
  residuals     = FALSE,
  intercepts    = FALSE,
  nCharNodes    = 0,
  sizeLat       = 9,
  sizeMan       = 9,
```

```
  edge.label.cex = 0.85,
  edge.label.position = 0.65
) %>%
  semptools::mark_sig(fit_scct_indirect) %>%
  change_node_name(var_named1) %>%  ##### 이것을 이용하여 이름 변경
  plot()


## End(Not run)
```

`Char2num` *Functions to convert character to numeric*

Description

Functions to convert character to numeric

Usage

```
Char2num(data, iter = NULL, char_col = 1, last_col = FALSE)
```

Arguments

`data`	data.frame
`iter`	Select the range of variables to convert
`char_col`	Specify which columns should be kept as char col

Value

data.frame

Examples

```
## Not run:

## Replacing a character variable with numeric
mysummary(sats, "sux1") %>%
    mutate(across(everything(), ~ format(., digits = 4))) %>%
    tibble() %>% Char2num()


## Set remove=TRUE if you only want to process the selected variable and output only that.
  mysummary(sats, "sux1") %>%
  mutate(across(everything(), ~ format(., digits = 5))) %>%
    tibble() %>%Char2num(auto=FALSE, iter= 4:ncol(.), remove= TRUE)

##
mysummary(sats, "sux1") %>%
  mutate(across(everything(), ~ format(., digits = 5))) %>%
    tibble() %>%Char2num(auto=FALSE, iter= 4:ncol(.), remove= FALSE)
## same
  mysummary(sats, "sux1") %>%
     mutate(across(everything(), ~ format(., digits = 5))) %>%
```

```
    tibble() %>%Char2num(auto=FALSE, iter= 4:ncol(.))


#새로운 데이터
reg_text <- "var B se b t
성취감 -.03 .09 -.03 -.33
인정감 .24 .09 .20 2.50
책임감 .30 .08 .23 3.56***
발전성 -.08 .06 -.10 -1.29
감독 .06 .05 .08 1.35
보수 -.14 .07 -.18 -2.01*
대인관계 .06 .06 .06 .89
작업환경 .33 .09 .34 3.82***
안정성 .17 .05 .26 3.42**
성별 -.05 .09 -.03 -.52
연령 -.20 .07 -.22 -3.01**
학력 .05 .07 .04 .70
소득 .00 .04 .01 .12
종사기간 .17 .06 .23 2.71**
기관규모 -.10 .03 -.20 -3.36**"

# 함수 호출
text2df(reg_text) %>% tibble()
text2df(reg_text)%>% Char2num() %>% tibble()
text2df(reg_text)%>% Char2num(iter=2:4) %>% tibble()

## End(Not run)
```

`Char2num_fct` *fact varaible to num in data.frame*

Description

fact varaible to num in data.frame

Usage

```
Char2num_fct(df, col = 1)
```

Arguments

`df`	df
`col`	select col

Value

df

`chisq.multcomp` *RVAideMemoire::chisq.multcomp, Pairwise comparisons after a chi-squared goodness-of-fit test*

Description

RVAideMemoire::chisq.multcomp, Pairwise comparisons after a chi-squared goodness-of-fit test

Usage

```
chisq.multcomp(x, p.method = "fdr")
```

Arguments

`x`	numeric vector (counts).
`p.method`	method for p-values correction. See help of p.adjust.

Value

result

Examples

```
## Not run:
#'
counts <- c(49,30,63,59)
chisq.test(counts)
chisq.multcomp(counts)
#comapre
chisq_multcomp(counts)


## End(Not run)
```

`chisqTest` *chisq_test 2way*

Description

chisq_test 2way

Usage

```
chisqTest(Obs, correct = FALSE, digits = 3, cat = TRUE)
```

Arguments

Obs	observation
correct	Yates' continuity correction
digits	3
cat	chisq.test form

Value

chisq

Examples

```
## Not run:

table(mtcars$am, mtcars$vs) %>% chisq_test()


table(mtcars$am, mtcars$vs) %>% chisq_test(correct=F)

## compare original chisq.test
table(mtcars$am, mtcars$vs) %>% chisq.test()


##Pearson's Chi-squared test with Yates' continuity correction
table(mtcars$am, mtcars$vs) %>% chisq_test(correct=T)

## End(Not run)
```

---

chisq_apa 카이제곱 *apa* 보고

Description

카이제곱 apa 보고

Usage

```
chisq_apa(data, chi_type = NULL, show = TRUE)
```

Arguments

data	chisq.test result data
chi_type	" 적합성검정", " 독립성검정", " 동질성검정"
show	TRUE cat output

Value

data result

Examples

```
## Not run:
#'
dfa= matrix(c(22, 26, 55,13, 31, 33,13, 16, 38), ncol=3, byrow=TRUE)
colnames(dfa)=c("가","나","다")
rownames(dfa)=c("A","B","C")
dfa
dfat= t(dfa)
dfat%>%chisq.test()%>%chisq_apa()

## End(Not run)
```

`chisq_contrast` *Chisq test Contrast test*

Description

Chisq test Contrast test

Usage

```
chisq_contrast(
  data,
  ...,
  test = chisq.test,
  p.adjust = "bonferroni",
  adj = FALSE,
  name = FALSE
)
```

Arguments

`data`	data
`...`	contrast
`test`	chisq.test, fisher.test
`p.adjust`	c("bonferroni","fdr","BH","BY","holm","hochberg","hommel")
`adj`	cadjust p value TRUE
`name`	rowname, when pairwise

Value

data

Examples

```
## Not run:
#'
d <- matrix(c(584, 24, 1721, 56, 2400, 90, 8164, 289),
            nrow = 4, byrow = TRUE,
```

```
            dimnames = list(c("kangnam", "Moksdong", "nowon", "dongtan"), c("Yes", "No")))

d%>%chisq.test()
chisq_contrast(d)
chisq_contrast(d, c(1,2,3,4))


chisq_contrast(d, c(1,1,2,3),c(2,1,2,2),c(2,2,1,2),c(2,2,2,1))
# fisher.test
chisq_contrast(d, c(1,2,2,2),c(2,1,2,2),c(2,2,1,2),c(2,2,2,1), test=fisher.test)
# fisher.exact test
chisq_contrast(d, c(1,2,2,3),c(2,1,2,2),c(2,2,1,2),c(2,2,2,1), test=fisher.test)

chisq_contrast(d, c(1,2,2,2),c(2,1,1,2),c(2,2,1,2),c(2,2,2,1), test=fisher.test)
chisq_contrast(d, c(1,2,2,2),c(2,3,1,2),c(2,2,1,2),c(2,2,2,1), test=fisher.test)
chisq_contrast(d, c(1,2,2,2),c(2,3,1,2),c(2,2,1,3),c(2,2,2,1), test=fisher.test)
chisq_contrast(d, c(1,2,2,2),c(2,3,1,2),c(2,2,1,3),c(3,4,2,1), test=fisher.test)


#other data
data_text = "Tobacco_Dependence (0,5] (5,10] (10,15] (15,20] (20,98]
No_Nicotine_Dependence 130 210 43 114 20
Nicotine_Dependence 119 267 91 254 67"

#method2
data_text%>% text2df(header=T) %>% make_df_text()
nt
nt = data_text%>% text2df(header=T)%>% jjstat::col2row()
nt %>%tibble()
nt=Char2num(nt,1:5)
nt%>%t() %>%as.data.frame()%>%
  chisq_contrast(c(1,2,2,2,2),
                 c(2,1,2,2,2),
                 c(2,2,1,2,2),
                 c(2,3,4,1,2))

## End(Not run)
```

---

chisq_gof_posthoc *chisq_gof_posthoc*

Description

chisq_gof_posthoc

Usage

```
chisq_gof_posthoc(vector, type = "res", method = "bonferroni", names = TRUE)
```

Arguments

vector	data

`type`	all, res, p, chisq
`method`	p.adust fdr, bonferroni, ...
`names`	TRUE using names

Value

data

Examples

```
## Not run:
#'

X <- c(49, 30, 63, 59)

chisq_gof_posthoc(X, "res")
chisq_gof_posthoc(X,"all")$gof_test
chisq_gof_posthoc(X,"all")$p
chisq_gof_posthoc(X,"all")$chisq
chisq_gof_posthoc(X, typ="all")
chisq_gof_posthoc(X, typ="res")
chisq_gof_posthoc(X, type="p")
chisq_gof_posthoc(X, type="chisq")

chisq_gof_posthoc(X, method="bonferroni")

X1 <- c(4, 3, 6, 19)
chisq_gof_posthoc(X1)
chisq_gof_posthoc(X1, typ="all")
chisq_gof_posthoc(X1, type="p")
chisq_gof_posthoc(X1, type="chisq")
chisq_gof_posthoc(X1, method="bonferroni")


chisq_gof_posthoc(X, method="bonferroni")

chisq_gof_posthoc(c(49, 30, 63, 59, 40, 60))

#' #'

## End(Not run)
```

`chisq_gof_posthoc_apa` *chisq_gof_posthoc_apa*

Description

chisq_gof_posthoc_apa

Usage

```
chisq_gof_posthoc_apa(df, adj = FALSE)
```

Arguments

df	df
adj	adj

Value

df

---

chisq_multcomp2 *Pairwise comparisons after a chi-squared goodness-of-fit test*

Description

Pairwise comparisons after a chi-squared goodness-of-fit test

Usage

```
chisq_multcomp2(x, p.method = "fdr", ko = TRUE, sep = " / ")
```

Arguments

x	numeric vector (counts).
p.method	method for p-values correction. See help of p.adjust
ko	TRUE variable names is changed to korean language
sep	sep =" / " default

Value

result

Functions

- `chisq_multcomp2()`: pairwise comparisons after a global chi-squared goodness-of-fit test.

Examples

```
## Not run:
#'
counts <- c(49,30,63,59)
chisq.test(counts)
chisq_multcomp2(counts)
# compare
chisq.multcomp2(counts)


## End(Not run)
```

`chisq_pivot_table` *pivot*

Description

pivot

Usage

```
chisq_pivot_table(
  data,
  formula,
  names_from = NULL,
  unite = FALSE,
  col = 1:2,
  remove = TRUE,
  change_col = TRUE
)
```

Arguments

`data`	data
`formula`	formul
`names_from`	column
`unite`	1,2 colum unite
`col`	합쳐질컬럼
`remove`	기존것지우기
`change_col`	열의위치바꾸기

Value

데이터

Examples

```
## Not run:

#' kge_bind3a %>%
filter(type == "고유어1")%>%
  pivot_table(~ area + age + 성조, unite=TRUE)
#'
kge_bind3a %>%
  filter(type == "고유어1")%>%
  pivot_table(~  age + area+ 성조, unite=FALSE)

#고유어1 공명음  지역별 연령
kge_bind3a %>%
  filter(type == "고유어1" & a1=="공명음")%>%
  pivot_table(~ age + area + 성조, unite=T)  #%>% md()
```

```
kge_bind3a %>%
  filter(type == "고유어1")%>%
  pivot_table(~  age +  성조, unite=FALSE)

#고유어1 공명음  지역별 연령
kge_bind3a %>%
  filter(type == "고유어1" & a1=="공명음")%>%
  pivot_table(~  area + 성조, unite=F)  #%>% md()



## End(Not run)
#'
```

`chisq_posthoc` *chisq post hoc*

Description

chisq post hoc

Usage

```
chisq_posthoc(
  tbl,
  test = fisher.test,
  Rows = TRUE,
  control = c("bonferroni", "fdr", "BH", "BY", "holm", "hochberg", "hommel"),
  digits = 4,
  ...
)
```

Arguments

`tbl`	tavke
`test`	fusger.test, chisq.test
`Rows`	rows , FLASE = cols
`control`	p adjust"bonferroni","fdr","BH","BY","holm","hochberg","hommel"
`digits`	4
`...`	...

Value

result

Examples

```
## Not run:
as.table(cbind("T"=c(30,40,60,120),"S"= c(50,70,60,80)))%>%
  chisq_posthoc(Rows  = TRUE)

as.table(cbind("T"=c(30,40,60,120),"S"= c(50,70,60,80)))%>%
  chisq_posthoc(Rows  = FALSE)

as.table(cbind("T"=c(30,40,60,120),"S"= c(50,70,60,80)))%>%
  chisq_posthoc(test="chisq.test", Rows  = TRUE)

## End(Not run)
```

---

chisq_retest *Functions to check the chi-square test of a paper*

Description

Functions to check the chi-square test of a paper

Usage

```
chisq_retest(
  input,
  ncol = 2,
  colname = NULL,
  rowname = NULL,
  type = "all",
  simulate.p.value = FALSE,
  digits = 3,
  correct = TRUE,
  B = 2000
)
```

Arguments

input	vectro
ncol	matrix col
colname	input
rowname	input
type	all, chisq
simulate.p.value	default FaLSE fisher exact test
digits	round 3
correct	default is Pearson′s Chi-squared test with Yates′continuity correction (If you have something with a small sample count, you should use it.), FALSE Pearson's Chi-squared test
B	B is bootstrap default 2000

Value

chisquare data

Examples

```
## Not run:
## Mina Choi. (2007). Analysis of learning effects
# according to online and offline learning environments
#  in the same class by the same instructor.
#  Educational Information Media Research, 13(4>  5-23.
## jjpaper
## https://blog.naver.com/shoutjoy/223005233887

chisq_retest(c(111, 33, 64, 58), ncol=2 )
## 논문에 나온 값이 잘못됨 :chisq = 17.793
## this chisq = 16.7

chisq_retest(c(111, 33, 64, 58), ncol=2, type="chisq" )

chisq_retest(c(111, 33, 64, 58), ncol=2,
  colname = c("experience", "Non-experience"),
  rowname =c("E-Leanring","FaceToFace"))

chisq_retest(c(4,10, 78, 46, 6, 0,5,79,36,2),
  ncol=2, simulate.p.value=T )

chisq_retest(c(4,10, 78, 46, 6, 0,5,90,36,2),
  ncol=2 ,simulate.p.value=T)

chisq_retest(c(4,10, 78, 46, 6, 0,5,79,36,2), ncol=2,
  colname = c("experience", "Non-experience"),
  rowname = c("maxlow","low","mid","high","maxhigh") )


##  Comparison of gender learning methods
# --> An invalid value was calculated.
## chisq = 5.238, p =0.015
# But recalculate chisq = 4.69, p = 0.0303
chisq_retest(c(84, 60, 54, 68), ncol=2 ,
             colname = c("experience", "Non-experience"),
                         rowname = c("male", "female"))

## Comparing Learner Characteristics by Grade Level, Mina Choi (2007)

chisq_retest(c(18,38, 40, 48, 52,35,19,16), ncol=2 ,
colname = c("experience", "Non-experience"),
rowname = c("1학년","2학년","3학년","4학년")


##If the value entered is not a vector value, but a value calculated with table(),
# it is automatically calculated with the ....
table(mtcars$am, mtcars$vs) %>% Chisq_retest()


## End(Not run)
```

`chisq_table_sum` *chisq table resize*

Description

chisq table resize

Usage

```
chisq_table_sum(data, ..., trans = FALSE, sep = "_")
```

Arguments

`data`	mat
`...`	con 1,1,2,2
`trans`	matr transpose
`sep`	sep rowname combine ”-”

Value

mat

Examples

```
## Not run:
#'
# Test data
data <- data.frame(T = c(30, 40, 60, 120),
                   S = c(50, 70, 60, 80))
rownames(data) <- LETTERS[1:4]
data <- as.matrix(data)

# Example usage
print(chisq_table_sum(data, 1, 1, 2, 2))

chisq_table_sum(data, 1, 1, 2, 2)
chisq_table_sum(data, con = c(1, 1, 2, 2))
chisq_table_sum(data, la = c(1, 1, 2, 2))


## End(Not run)
```

chisq_test *chisq_test*

Description

chisq_test

Usage

```
chisq_test(
  x,
  y = NULL,
  correct = TRUE,
  p = rep(1/length(x), length(x)),
  rescale.p = FALSE,
  simulate.p.value = FALSE,
  B = 2000
)
```

Arguments

x	a numeric vector or matrix. x and y can also both be factors.
y	a numeric vector; ignored if x is a matrix. If x is a factor, y should be a factor of the same length.
correct	a logical indicating whether to apply continuity correction when computing the test statistic for 2 by 2 tables: one half is subtracted from all O−E differences ; however, the correction will not be bigger than the differences themselves. No correction is done if simulate.p.value = TRUE.
p	a vector of probabilities of the same length as x. An error is given if any entry of p is negative.
rescale.p	a logical scalar; if TRUE then p is rescaled (if necessary) to sum to 1. If rescale.p is FALSE, and p does not sum to 1, an error is given.
simulate.p.value	a logical indicating whether to compute p-values by Monte Carlo simulation
B	an integer specifying the number of replicates used in the Monte Carlo test.

Value

A list with class ”htest” containing the following components:

Examples

```
## Not run:

## From Agresti(2007) p.39
M <- as.table(rbind(c(762, 327, 468), c(484, 239, 477)))
dimnames(M) <- list(gender = c("F", "M"),
party = c("Democrat","Independent", "Republican"))
```

```
(Xsq <- chisq.test(M))  # Prints test summary
Xsq$observed   # observed counts (same as M)
Xsq$expected   # expected counts under the nul
Xsq$residuals  # Pearson residuals
Xsq$stdres     # standardized residuals


x <- matrix(c(12, 5, 7, 7), ncol = 2)
chisq_test(x)$p.value           # 0.4233
chisq_test(x, simulate.p.value = TRUE, B = 10000)$p.value

## End(Not run)
```

`chisq_test_By1` *one sample chisq test by group*

Description

one sample chisq test by group

Usage

```
chisq_test_By1(dataset, sel = NULL, correct = FALSE, simulate.p.value = FALSE)
```

Arguments

`dataset`	data.frame
`sel`	select column group
`correct`	chisq correct
`simulate.p.value`	simulate.p.value

Value

Chi-square results by group

Examples

```
## Not run:
mtcars%>%chisq_test_By1("cyl")

mtcars%>%chisq_test_By1("am")
iris%>%chisq_test_By1("Species")

## End(Not run)
```

`chisq_test_By2` *twosample chisq.test by group*

Description

twosample chisq.test by group

Usage

```
chisq_test_By2(
  data,
  v1,
  v2,
  sel,
  correct = TRUE,
  simulate.p.value = FALSE,
  warning = FALSE,
  type = "all"
)
```

Arguments

`data`	data,frame
`v1`	first variable
`v2`	second variable
`sel`	group
`correct`	correct
`simulate.p.value`	simulate.p.value
`warning`	warning
`type`	”all”, ”res”

Value

chisq result

Examples

```
## Not run:

mtcars%>%  chisq_test_By2("am", "vs", "cyl")

mtcars%>% chisq_test_By2("am", "vs", sel= "cyl")

Mtcars <- mtcars  # 데이터를 원본과 겹치지 않도록 복사
Mtcars$am = factor(Mtcars$am, levels=c(0,1), labels= c("automatic","manual" ))
Mtcars$vs = factor(Mtcars$vs, levels=c(0,1), labels= c("V-shaped","straight" ))
Mtcars$cyl = factor(Mtcars$cyl, levels=c(4,6,8),
             labels= c("cyl-4","cyl-6","cyl-8"))

chisq_test_By2(data = Mtcars, v1 = "am", v2 = "vs", sel = "cyl")
```

```
## End(Not run)
```

`chisq_test_By3` 그룹별카이제곱분석

Description

그룹별카이제곱분석

Usage

```
chisq_test_By3(data, grp, formula, raw = FALSE)
```

Arguments

`data`	data
`grp`	group variable
`formula`	~ vq +v2
`raw`	warning

Value

table

Examples

```
## Not run:
#'
chisq_test_By3(mtcars, cyl, ~ am+vs )
chisq_test_By3(mtcars, cyl, ~ am+vs , raw=T)


## End(Not run)
```

`chisq_test_cell` *Chi-square analysis by cell*

Description

Chi-square analysis by cell

Usage

```
chisq_test_cell(data, type = "res", trans = FALSE, plot = TRUE, digits = 3)
```

Arguments

`data`	table
`type`	res, alll()default), chisq_result , observed expected, residuals , chisq_cell , stdres, chisq_cell_sig , cramersv , OE_ratio, OE_sig , graph
`trans`	plot transpose
`plot`	TRUE
`digits`	digits= 3

Value

chisq

Examples

```
## Not run:
#'


M <- as.table(rbind(c(762, 327, 468), c(484, 239, 477)))
dimnames(M) <- list(gender = c("F", "M"),
                    party = c("Democrat","Independent", "Republican"))
M %>% chisq_test_cell()

#notsig
examtest  = matrix(c(7,14,13,6), nrow=2,
                   dimnames= list(c("사전검사","사후검사"),
                                  c("합격","불합격")))
examtest
fisher.test(examtest)

examtest %>% chisq_test_cell()

## End(Not run)
```

`chisq_test_cor` *kge_chisq_table chisq table observed/Expected table-*

Description

kge_chisq_table chisq table observed/Expected table-

Usage

```
chisq_test_cor(
  dataset,
  v1 = "var1",
  v2 = "var2",
  title = "Table",
  type = "result",
  digits = 3,
  yadd = 0.4,
  Ncol = NULL,
  trans = FALSE,
  simple = FALSE,
  ko = TRUE,
  simulate.p.value = FALSE,
  correct = FALSE,
  size_bartext = 5,
  strip_size = 16,
  axis_size = 16,
  text_size = 13,
  xlab = "관측기대비율",
  ylab = "var2",
  cramer = "adjust"
)
```

Arguments

`dataset`	data.frame
`v1`	col
`v2`	row
`title`	title
`type`	res2 default
`digits`	3
`yadd`	up yadd
`Ncol`	grid
`trans`	transpose
`simple`	* style
`ko`	graph korean
`simulate.p.value`	exact
`correct`	exactt

size_bartext	graph text
strip_size	strip
axis_size	axis_size
text_size	text_size
xlab	xlab
ylab	ylab
cramer	cramer cor

Value

multiple data

Examples

```
## Not run:
kge_chisq_table(mtcars,"am","cyl")


#chisq test result
vote_chi <- Chisq_retest(c(183, 35, 824, 50), ncol=2,
                         colname = c("Did not vote", "Voted"),
                          rowname =c("All other ages","Youngest"))

vote_chi


# generate matrix
voteMat = matrix(c(183, 35, 824, 50), ncol=2)
colnames(voteMat) = c("Did not vote", "Voted")
rownames(voteMat) = c("All other ages","Youngest")

voteMat%>% data.frame() %>%
  long_df("voted", rowname="youth")%>% #long format
  add_rows_freq() %>%     # add rows
  kge_chisq_table("voted","youth")    #chisq test


#신진영(2013)
shin2013_mat =matrix(c(14,1,23,3,4,0,9,0,13,18,89,8,11,40,46,12,3,4,22,0), ncol= 5)
# colnames(shin2013_mat)=c("지능지수가높다","학업성취도가높다","창의적사고를한다","특정분야에서뛰어난
colnames(shin2013_mat)=c("지능지수","학업성취도","창의적사고","특정분야","모든분야")
rownames(shin2013_mat)=c("고졸","초대졸","대졸","대학원이상")
shin2013_mat

shin2013_mat %>% data.frame() %>%
  rownames_to_column("학력") %>%
  to_long(names_to = "영재성",cols=2:6) %>%
  unCount() %>%
  chisq_test_cor(v1="영재성", v2="학력")


shin2013_mat%>%  data.frame() %>%
  long_df("학력","영재성") %>%
  unCount() %>%
```

```
  chisq_test_cor("학력","영재성")


## End(Not run)
```

`chisq_test_kge` *kge_chisq_table chisq table observed/Expected table-*

Description

kge_chisq_table chisq table observed/Expected table-

Usage

```
chisq_test_kge(
  dataset,
  v1 = "var1",
  v2 = "var2",
  title = "Table",
  type = "result",
  digits = 3,
  yadd = 0.4,
  Ncol = NULL,
  trans = FALSE,
  simple = FALSE,
  ko = TRUE,
  simulate.p.value = FALSE,
  correct = FALSE,
  size_bartext = 5,
  strip_size = 16,
  axis_size = 16,
  text_size = 13,
  xlab = "관측기대비율",
  ylab = "var2",
  cramer = "adjust"
)
```

Arguments

`dataset`	data.frame
`v1`	col
`v2`	row
`title`	title
`type`	res2 default
`digits`	3
`yadd`	up yadd
`Ncol`	grid
`trans`	transpose
`simple`	* style

ko	graph korean
simulate.p.value	exact
correct	exactt
size_bartext	graph text
strip_size	strip
axis_size	axis_size
text_size	text_size
xlab	xlab
ylab	ylab
cramer	cramer cor

Value

multiple data

Examples

```
## Not run:
kge_chisq_table(mtcars,"am","cyl")


#chisq test result
vote_chi <- Chisq_retest(c(183, 35, 824, 50), ncol=2,
                         colname = c("Did not vote", "Voted"),
                          rowname =c("All other ages","Youngest"))

vote_chi


# generate matrix
voteMat = matrix(c(183, 35, 824, 50), ncol=2)
colnames(voteMat) = c("Did not vote", "Voted")
rownames(voteMat) = c("All other ages","Youngest")

voteMat%>% data.frame() %>%
  long_df("voted", rowname="youth")%>% #long format
  add_rows_freq() %>%     # add rows
  kge_chisq_table("voted","youth")    #chisq test


#신진영(2013)
shin2013_mat =matrix(c(14,1,23,3,4,0,9,0,13,18,89,8,11,40,46,12,3,4,22,0), ncol= 5)
# colnames(shin2013_mat)=c("지능지수가높다","학업성취도가높다","창의적사고를한다","특정분야에서뛰어난
colnames(shin2013_mat)=c("지능지수","학업성취도","창의적사고","특정분야","모든분야")
rownames(shin2013_mat)=c("고졸","초대졸","대졸","대학원이상")
shin2013_mat

shin2013_mat %>% data.frame() %>%
  rownames_to_column("학력") %>%
  to_long(names_to = "영재성",cols=2:6) %>%
  unCount() %>%
```

```
  chisq_test_kge(v1="영재성", v2="학력")


shin2013_mat%>%  data.frame() %>%
  long_df("학력","영재성") %>%
  unCount() %>%
  chisq_test_kge("학력","영재성")


## End(Not run)
```

`cmh_test` *cochran_mantel_haenszel_test*

Description

cochran_mantel_haenszel_test

Usage

```
cmh_test(
  x,
  y = NULL,
  z = NULL,
  alternative = c("two.sided", "less", "greater"),
  correct = TRUE,
  exact = FALSE,
  conf.level = 0.95
)
```

Arguments

`x`	array
`y`	NULL
`z`	NULL
`correct`	TRUE
`exact`	FALSE
`conf.level`	0.95
`alternativec("two.sided",`	”less”, ”greater”)

Value

result

Examples

```
## Not run:

CAD <-array(c(8, 4, 10, 11, 21, 6, 9, 9), dim = c(2, 2, 2),
            dimnames = list(
              EKG = c(">=0.1 ST Dep", "< 0.1 ST Dep"),
              Response = c("Case", "Control"),
              Penicillin.Level = c("Female", "Male")))
CAD

#Example: Coronary Artery Disease
mantelhaen.test(CAD,correct=FALSE)

cmh_test(CAD,correct=FALSE)%>%tidy()


## End(Not run)
```

---

col2row *column to rownames col2row*

Description

column to rownames col2row

Usage

```
col2row(data, col = 1)
```

Arguments

data	data.frame
col	Columns that should be rows

Value

data

Examples

```
## Not run:
#'
data_text = "Tobacco_Dependence (0,5] (5,10] (10,15] (15,20] (20,98]
No_Nicotine_Dependence 130 210 43 114 20
Nicotine_Dependence 119 267 91 254 67"
#method1
# nt = data_text%>% text2df(header=T)%>%
#         tibble::column_to_rownames("Tobacco_Dependence")
#method2
nt = data_text%>% text2df(header=T)%>% jjstat::col2row()
nt=Char2num(nt,1:5)
nt
```

```
nt %>%tibble()

#NEW data
New = data.frame(
  rownames= c('No_Nicotine_Dependence', 'Nicotine_Dependence'),

  "A" = c(130, 119),
  "B" = c(210, 267),
  "C" = c(43, 91),
  "D" = c(114, 254),
  "E" = c(20, 67)
);New
New%>%col2row()
New%>%col2row(col="B")
#mtcars
data_mtcars=mtcars%>%tibble::rownames_to_column()
data_mtcars%>%col2row()




## End(Not run)
```

`collapse_path` *Extract only the corresponding paths from the data with the given material and the given paths*

Description

Extract only the corresponding paths from the data with the given material and the given paths

Usage

```
collapse_path(data, path_string, est = "Original")
```

Arguments

`data`	boot
`path_string`	path IMAG -> EXPE -> VAL
`est`	Original, Mean.Boot

Value

paths

Examples

```
## Not run:
#'

# Example usage
sdata <- data.frame(
  paths = c("IMAG -> EXPE", "IMAG -> SAT", "IMAG -> LOY",
   "EXPE -> QUAL", "EXPE -> VAL", "EXPE -> SAT",
            "QUAL -> VAL", "QUAL -> SAT",
            "VAL -> SAT", "SAT -> LOY"),
  Original = c(0.578959128, 0.200724200, 0.275149576,
   0.848344408, 0.105477650, -0.002753995,
               0.676655529, 0.122144566, 0.589330650, 0.495479322),
  Mean.Boot = c(0.58232765, 0.21209094, 0.27486564,
  0.84793899, 0.10992051, -0.01008222,
                0.67780527, 0.15132543, 0.56312717, 0.50757755),
  Std.Error = c(0.04358293, 0.06056988, 0.07729000,
  0.01864573, 0.06968826, 0.06507903,
                0.07760527, 0.08440236, 0.08723293, 0.07856694),
  perc.025 = c(0.497644719, 0.107561558, 0.140641502,
   0.815794494, -0.025566471, -0.147788188,
               0.519959683, -0.008629516, 0.392664443, 0.351774130),
  perc.975 = c(0.6646198, 0.3275312, 0.4161131,
   0.8828875, 0.2545785, 0.1086856,
               0.8166486, 0.2991045, 0.7012494, 0.6556333)
)
collapse_path(sdata, path_string= "IMAG -> EXPE -> VAL -> SAT -> LOY")
## paths  Original  Std.Error
## 1 IMAG -> EXPE 0.5789591 0.04358293
## 2  EXPE -> VAL 0.1054776 0.06968826
## 3   VAL -> SAT 0.5893307 0.08723293
## 4   SAT -> LOY 0.4954793 0.07856694


#활용방법
collapse_path(sdata,
path_string= "IMAG -> EXPE -> VAL -> SAT -> LOY")%>%
find_paths()%>%
find_paths_cal()

#'
## End(Not run)
```

`collapse_path2` 소벡테스트에입력될값으로산정하여 *list* 로출력

Description

소벡테스트에입력될값으로산정하여 list 로출력

Usage

```
collapse_path2(data, path_string, show = FALSE, est = "Original")
```

Arguments

data	boot
path_string	paths
show	message
est	Original, Mean.Boot

Value

data

Examples

```
## Not run:
#'

# Example usage
sdata <- data.frame(
  paths = c("IMAG -> EXPE", "IMAG -> SAT", "IMAG -> LOY",
   "EXPE -> QUAL", "EXPE -> VAL", "EXPE -> SAT",
            "QUAL -> VAL", "QUAL -> SAT",
            "VAL -> SAT", "SAT -> LOY"),
  Original = c(0.578959128, 0.200724200, 0.275149576,
   0.848344408, 0.105477650, -0.002753995,
              0.676655529, 0.122144566, 0.589330650, 0.495479322),
  Mean.Boot = c(0.58232765, 0.21209094, 0.27486564,
  0.84793899, 0.10992051, -0.01008222,
               0.67780527, 0.15132543, 0.56312717, 0.50757755),
  Std.Error = c(0.04358293, 0.06056988, 0.07729000,
  0.01864573, 0.06968826, 0.06507903,
               0.07760527, 0.08440236, 0.08723293, 0.07856694),
  perc.025 = c(0.497644719, 0.107561558, 0.140641502,
   0.815794494, -0.025566471, -0.147788188,
              0.519959683, -0.008629516, 0.392664443, 0.351774130),
  perc.975 = c(0.6646198, 0.3275312, 0.4161131,
   0.8828875, 0.2545785, 0.1086856,
              0.8166486, 0.2991045, 0.7012494, 0.6556333)
)
#check 1
collapse_path2(sdata, path_string= "IMAG -> EXPE -> VAL -> SAT -> LOY")

#check 2
collapse_path2(sdata, path_string= "IMAG -> EXPE -> VAL -> SAT -> LOY", show=TRUE)
#'
# TEST

collapse_path2(sdata, path_string= "IMAG -> EXPE -> VAL -> SAT -> LOY")
collapse_path2(sdata, path_string= "IMAG -> EXPE -> VAL -> SAT -> LOY", show=TRUE)
collapse_path2(satpls_boot, path_string= "IMAG -> EXPE -> VAL -> SAT -> LOY", show=TRUE)

# 소벨테스트
```

```
sobel_test_extend(coefficients= c("IMAG -> EXPE" = 0.578959128,
                                  "EXPE -> SAT" = -0.002753995,
                                  "SAT -> LOY" = 0.495479322),
                  se_values = c(se1 = 0.04358293,
                                se2 = 0.06507903,
                                se3 = 0.0785669))


Resdata  =   collapse_path2(sdata, path_string= "IMAG -> EXPE -> VAL -> SAT -> LOY")

sobel_test_extend(coefficients= Resdata$coefficients, Resdata$se_values)


sobel_test_extend(
  coefficients= collapse_path2(sdata,
              path_string= "IMAG -> EXPE -> VAL -> SAT -> LOY")$coefficients,
  se_values =  collapse_path2(sdata,
        path_string= "IMAG -> EXPE -> VAL -> SAT -> LOY")$se_values)


sobel_test_extend(coefficients= collapse_path2(sdata,
         path_string=  "IMAG -> EXPE -> SAT -> LOY")$coefficients,
         se_values =  collapse_path2(sdata, path_string= "IMAG -> EXPE -> SAT -> LOY")$
                   se_values, show=FALSE)


## End(Not run)
```

`colnames_by_dict` *colnames_by_dict: Rename only column names using a dictionary*

Description

This function renames the column names of a data frame using a dictionary vector.

Usage

```
colnames_by_dict(df, dict)
```

Arguments

`df`	A data.frame or tibble whose column names you want to rename.
`dict`	A character vector of key-value pairs. It can be either: - Named vector: c("old1" = "new1", "old2" = "new2") - Unnamed pairwise vector: c("old1", "new1", "old2", "new2")

Value

A data.frame with renamed column names.

Examples

```
var_named <- c(
  "access","연구접근성",
  "barrier","진로장벽",
  "efficacy","연구효능감",
  "interest","진로흥미",
  "outcome","결과기대"
)

df <- data.frame(access = 1:2, barrier = 3:4, outcome = 5:6)
colnames_by_dict(df, var_named)
```

`colnames_change` *colnames_change: Change column names partially or entirely, with optional preservation of original names*

Description

colnames_change: Change column names partially or entirely, with optional preservation of original names

Usage

```
colnames_change(df, ...)
```

Arguments

`df`	A data frame. The target data frame whose column names need to be changed.
`...`	Character vector of new column names. Use an empty string (‘""‘) to keep the original column name for specific positions.

Value

A data frame with updated column names. Columns corresponding to ‘""‘ in the input retain their original names.

Examples

```
## Not run:
# Example with partial column renaming using mtcars
data(mtcars)
head(mtcars)

# Rename selected columns
mtcars_updated <- mtcars %>%
  colnames_change("Miles_per_Gallon", "Cylinders", "Displacement", "", "Weight")

head(mtcars_updated)

## End(Not run)
```

`colnames_change_df` *colnames_change_df: Change column names of a data frame within a specified range*

Description

colnames_change_df: Change column names of a data frame within a specified range

Usage

```
colnames_change_df(df, range = 1:ncol(df), impute = NULL)
```

Arguments

`df`	A data frame whose column names need to be changed.
`range`	A vector or range indicating the columns to change. It can be numeric (e.g., 1:27) or character specifying column names (e.g., "disp:vs").
`impute`	A character vector of new column names to assign to the specified range.

Value

The data frame with updated column names.

Examples

```
# Example 1: Change the first 11 columns of mtcars
new_colnames <- c(
  paste0("feature_", 1:6),
  paste0("metric_", 1:5)
)
updated_mtcars <- colnames_change_df(
  df = mtcars,
  range = 1:11,
  impute = new_colnames
)

# Example 2: Change a subset of columns in mtcars by name range
new_colnames <- c(
  paste0("feature_", 1:6)   # Replace columns "disp" to "vs"
)
updated_mtcars <- colnames_change_df(
  df = mtcars,
  range = "disp:vs",
  impute = new_colnames
)
```

`colnames_to_row` *colnames_to_row: Insert column names as a row in a data frame*

Description

colnames_to_row: Insert column names as a row in a data frame

Usage

```
colnames_to_row(df, row = 1)
```

Arguments

`df`	A data frame whose column names will be inserted as a row.
`row`	An integer specifying the row number where column names should be inserted. Default is 1.

Value

A data frame with column names inserted as a row at the specified position.

Examples

```
## Not run:
df <- data.frame(a = 1:3, b = 4:6, c = 7:9)
colnames_to_row(df)
colnames_to_row(df, row = 2)
mtcars
mtcars %>% nrow()
mtcars %>% colnames_to_row(1)
mtcars %>% colnames_to_row(3)
mtcars %>% colnames_to_row(32)

## End(Not run)
```

`col_capital` *col_capital: Convert letters in a selected column to upper or lower case*

Description

col_capital: Convert letters in a selected column to upper or lower case

Usage

```
col_capital(df, col, direction = TRUE)
```

Arguments

df	A data.frame. The input dataset.
col	A column name (character) or index (numeric) indicating which column to transform.
direction	Logical. If TRUE (default), convert to upper case. If FALSE, convert to lower case.

Value

A data.frame with the selected column transformed to upper or lower case.

Examples

```
## Not run:
df <- data.frame(
  구분 = c("chisq", "df", "pvalue"),
  criterian = c("Chisq", "df", "p > .05"),
  Value = c("1732.243", "269", "0")
)

# Use column name
col_capital(df, "criterian", TRUE)

# Use column index
col_capital(df, 2, FALSE)

## End(Not run)
```

---

col_to_colnames *one select column to colnames*

Description

one select column to colnames

Usage

```
col_to_colnames(df, col = 1, remove = TRUE)
```

Arguments

df	data frame
col	column to use as new column names (default = 1)
remove	logical. If TRUE, removes the selected column (default = TRUE).

Value

data frame with updated column names

Examples

```
## Not run:
# Example usage
df <- data.frame(
  요인 = c("에듀테크", "학습몰입", "지각된편리성", "지각된유용성", "태도", "지속사용"),
  SUX = c(NA, "0.948*", "0.933*", "0.947*", "0.902*", "0.865*"),
  FLOW = c(NA, NA, "0.951*", "0.967*", "0.929*", "0.876*"),
  PEOU = c(NA, NA, NA, "0.992*", "0.972*", "0.940*"),
  PU = c(NA, NA, NA, NA, "0.956*", "0.927*"),
  ATT = c(NA, NA, NA, NA, NA, "0.972*")
)

# col_to_colnames 실행
new_df <- col_to_colnames(df, col = 1, remove = FALSE)
print(new_df)

new_df2 <- col_to_colnames(df, col = 1, remove = TRUE)
print(new_df2)

## End(Not run)
```

---

col_to_rownames *column to rownames , col_to_rownames*

Description

column to rownames , col_to_rownames

Usage

```
col_to_rownames(data, col = 1)
```

Arguments

data	data.frame
col	Columns that should be rows

Value

data

Examples

```
## Not run:
#'
data_text = "Tobacco_Dependence (0,5] (5,10] (10,15] (15,20] (20,98]
No_Nicotine_Dependence 130 210 43 114 20
Nicotine_Dependence 119 267 91 254 67"
#method1
# nt = data_text%>% text2df(header=T)%>%
#         tibble::column_to_rownames("Tobacco_Dependence")
#method2
```

```
nt = data_text%>% text2df(header=T)%>% jjstat::col2row()
nt=Char2num(nt,1:5)
nt
nt %>%tibble()

#NEW data
New = data.frame(
  rownames= c('No_Nicotine_Dependence', 'Nicotine_Dependence'),

  "A" = c(130, 119),
  "B" = c(210, 267),
  "C" = c(43, 91),
  "D" = c(114, 254),
  "E" = c(20, 67)
);New
New%>%col_to_rownames()

#mtcars
data_mtcars=mtcars%>%tibble::rownames_to_column()
data_mtcars%>%col_to_rownames()




## End(Not run)
```

`combine_data` *A function that combines two matrices of data, for example, to combine observation/expectation tables and p-values.*

Description

A function that combines two matrices of data, for example, to combine observation/expectation tables and p-values.

Usage

```
combine_data(mat_observed_expected, mat_p_value, left = " ", right = "")
```

Arguments

`mat_observed_expected`	matrix observed/expected
`mat_p_value`	matirx pvalue
`left`	”(” etc input
`right`	”)”

Value

matrix

Examples

```
 ## Not run:

# Example data
observed_expected <- matrix(c(1.0587162, 1.1477347, 0.50877193,
                              1.1269248, 0.5743712, 1.94957983,
                              0.9163776, 1.4060606, 0.03515152,
                              0.9530237, 0.8624314, 1.44531611),
                            nrow = 4, byrow = TRUE,
         dimnames = list(c("공명음", "마찰음", "유기음_경음", "평파열음_평파찰음"),
                         c("H", "H(H)", "L")))

p_value <- matrix(c(0, 0.00001, 0.00000,
                    0, 0.00000, 0.01497,
                    0, 0.00013, 0.00000,
                    0, 0.00000, 0.00102),
                  nrow = 4, byrow = TRUE,
       dimnames = list(c("공명음", "마찰음", "유기음_경음", "평파열음_평파찰음"),
                       c("H", "H(H)", "L")))

sig_sym = add_significance_symbols(p_value)

# Combine the data
combined_data1 <- combine_data(observed_expected, p_value)
 print(combined_data1)
combined_data2 <- combine_data(observed_expected, sig_sym)
 print(combined_data2)

#iris cor
iris_cor = iris %>% select(-Species) %>%psych::corr.test()
iris_cor$r
iris_cor$p

combine_data(iris_cor$r%>%round(2),
             iris_cor$p%>%round(5),
             left="(", right=")")
 sig = add_significance_symbols(iris_cor$p)
 sig

 combine_data( round(iris_cor$r, 3) , sig)

 combine_data( round(iris_cor$r, 3) , sig) %>%
 lowerMat(diag = 1, fill="")

## End(Not run)
```

`combine_rows` 중복된데이터를합쳐주는함수

Description

중복된데이터를합쳐주는함수

Usage

```
combine_rows(df, rows = NULL, name_col = 1, value_col = 2, prop_col = 3)
```

Arguments

df	df
rows	rows =c(1,2,3) 지정된행을합침
name_col	이름열숫자로입력가능
value_col	빈도열
prop_col	비율열

Value

df

Examples

```
## Not run:
#'
# 예제 데이터
library(dplyr)

df <- data.frame(
  term = c("지평선중", "한국게임과학고", "가천초", "감곡초", "감곡초",
           "강호항공고교", "강호항공고", "개야도초", "고", "고부초",
           "고산고", "고산고"),
  Freq = c(1, 1, 1, 2, 1, 1, 1, 1, 4, 1, 1, 2),
  prop.. = c(0.1237624, 0.1237624, 0.1237624, 0.2475248, 0.1237624,
             0.1237624, 0.1237624, 0.1237624, 0.4950495, 0.1237624,
             0.1237624, 0.2475248)
)

# 함수 호출: 디폴트 값 사용 (열 번호)
result <- combine_rows(df)

# 결과 출력
print(result)

# 함수 호출: 열 이름 사용
result_named <- combine_rows(df, name_col = "term", value_col = "Freq", score_col = "prop..")
print(result_named)

#'

## End(Not run)
```

---

comma *number comma*

Description

number comma

Usage

```
comma(x, digits = 2, big_mark = ",")
```

Arguments

`x`	vector
`digits`	digits 2
`big_mark`	"," is default

Value

include comma

Examples

```
## Not run:
comma(3452345)
comma(.12358124331)

## End(Not run)
```

`compare_bar` *plspm_compare_bar*

Description

plspm_compare_bar

Usage

```
compare_bar(
  data,
  x = NULL,
  y = NULL,
  fill = NULL,
  size = 3,
  size_x = 12,
  angle = 90,
  position = "dodge",
  xlab = "",
  ylab = "Values",
  text = TRUE,
  digits = 3
)
```

Arguments

data	arragne data
x	paths
y	values
fill	group
size	bargrahp text size
size_x	textsize
angle	xangle
position	dodge, stack
xlab	default ""
ylab	values
text	hide bartext

Value

graph

Examples

```
## Not run:
#'
# 예시 데이터
REs <- tibble::tribble(
  ~paths, ~grp, ~values,
  "자기효능감->진로동기", "group.남자", 0.902,
  "자기효능감->진로동기", "group.여자", 0.866,
  "자기효능감->진로태도", "group.남자", -0.639,
  "자기효능감->진로태도", "group.여자", -0.672,
  "자기효능감->진로준비", "group.남자", 0.149,
  "자기효능감->진로준비", "group.여자", 0.487,
  "진로동기->진로준비", "group.남자", 0.63,
  "진로동기->진로준비", "group.여자", 0.432,
  "진로태도->진로준비", "group.남자", -0.153,
  "진로태도->진로준비", "group.여자", 0.059
)

# # 함수 호출 예시
REs %>% compare_bar()
REs %>% compare_bar(text=FALSE)




## End(Not run)
```

compare_bind_data *Table-building functions that pairwise compare two pieces of data*

Description

Table-building functions that pairwise compare two pieces of data

Usage

```
compare_bind_data(dff1, dff2, digits = 3, pair = TRUE)
```

Arguments

dff1	data1
dff2	data2
digits	rounding
pair	pari option default TRUE

Value

table

confint_mark_sig *Confint mark significant*

Description

Confint mark significant

Usage

```
confint_mark_sig(data, unite = FALSE, digits = 3)
```

Arguments

data	data.frame Confidence interval significance test: When the confidence interval is in the last column. Significance test when the last two columns of data are confidence intervals
unite	TRUE is combine CI and sig
digits	default 3

Examples

```
## Not run:

lm(mpg ~ hp + wt, data=mtcars)%>%
 confint()%>%as.data.frame()%>%
  rownames_to_column()%>%
  confint_mark_sig()

## combind data unite=TRUE
  lm(mpg ~ hp + wt, data=mtcars)%>%
     confint()%>%
     as.data.frame()%>%
     rownames_to_column()%>%
     confint_mark_sig(unite=TRUE, digits=2)




## End(Not run)
```

---

confint_plot_data *Plot Confidence Intervals from a Summary Data Frame*

Description

Draws horizontal error bars using data with estimate and confidence interval columns.

Usage

```
confint_plot_data(
  data,
  var_named = NULL,
  size_axis = 14,
  size_text = 4,
  linewidth = 2
)
```

Arguments

`data`	A data.frame or tibble containing path analysis results with columns: 'Path', 'est', 'ci.lower', 'ci.upper'. Optionally 'sig'.
`var_named`	Optional named vector to rename 'Path' labels.
`size_axis`	Font size for axis text. Default is 14.
`size_text`	Font size for estimate labels. Default is 4.
`linewidth`	Line width of confidence interval bars. Default is 2.

Value

A ggplot object showing the confidence intervals.

Examples

```
## Not run:
df <- sem_effect_ie(fit_le)
confint_plot_data(df)

# With renaming
named_vec <- c("IE.경험_효능감_진로의지" = "경험→효능감→진로의지")
confint_plot_data(df, var_named = named_vec)

## End(Not run)
```

---

confint_plot_lm *Plot Confidence Intervals for Linear Model or Data Frame*

Description

Plot Confidence Intervals for Linear Model or Data Frame

Usage

```
confint_plot_lm(
  lm_data,
  input = c("lm", "data.frame"),
  size_text = 12,
  size_title = 14,
  color = "steelblue",
  linewidth = 1,
  alpha = 1,
  intercept = FALSE,
  errorbar_height = 0.3,
  x.title = "Estimate",
  y.title = "Item",
  type = "g"
)
```

Arguments

lm_data	An object of class 'lm' (linear model) or a data.frame containing results with estimates and confidence intervals.
input	Character, one of 'lm' or 'data.frame'. Determines how lm_data is handled.
size_text	Numeric, size of text in the plot (default = 12).
size_title	Numeric, size of axis titles in the plot (default = 14).
color	Character, color of error bars (default = "steelblue").
linewidth	Numeric, thickness of error bars (default = 1).
alpha	Numeric, transparency of error bars (default = 1).
intercept	Logical, whether to include the intercept in the plot (default = FALSE).
errorbar_height	Numeric, height of error bar ends (default = 0.3).

x.title	Character, label for the x-axis (default = "Estimate").
y.title	Character, label for the y-axis (default = "Item").
type	Character, output type ('g' for plot, 'res' for results, 'df' for tidy dataframe).

Value

A ggplot object, results list, or tidy dataframe based on 'type'.

Examples

```
## Not run:
# Example with lm object
model <- lm(mpg ~ wt + hp, data = mtcars)
confint_plot_lm(model, input = "lm", errorbar_height = 0.1, size_title = 16)

# Example with data.frame
df <- data.frame(term = c("wt", "hp"), estimate = c(-5.344, -0.018),
                 conf.low = c(-7.344, -0.032), conf.high = c(-3.344, -0.004))
confint_plot_lm(df, input = "data.frame", errorbar_height = 0.3,
                x.title = "Beta Coefficients", y.title = "Predictors")

## End(Not run)
```

---

confint_plot_sem *Plot Confidence Intervals for SEM Paths using lavaan*

Description

Create a confidence interval plot of structural paths from a lavaan SEM object.

Usage

```
confint_plot_sem(
  lavaan_obj,
  type = "g",
  effect = "~",
  var_named = NULL,
  size_axis = 14,
  size_text = 4,
  linewidth = 2
)
```

Arguments

lavaan_obj	A fitted 'lavaan' object (e.g., from 'sem()' or 'cfa()').
type	Type of output: '"g"' for the ggplot object, '"res"' for the result table, '"all"' for both.
effect	Operator type to filter by, default is '"~"' for regression paths. Use '"=~"' for latent loadings.
var_named	Named vector for renaming variables using 'replace_df_rep()'. Default is 'NULL'.

`size_axis`	Numeric. Font size for axis labels in the plot. Default is 14.
`size_text`	Numeric. Size of text showing estimates in the plot. Default is 4.
`linewidth`	Numeric. Width of confidence interval error bars. Default is 2.

Value

A ggplot object, a data frame, or a list depending on the 'type' argument.

Examples

```
## Not run:
library(lavaan)
model <- '
  visual  =~ x1 + x2 + x3
  textual =~ x4 + x5 + x6
  speed   =~ x7 + x8 + x9
  speed ~ visual + textual
'
fit <- sem(model, data = HolzingerSwineford1939)
named_vec <- c("visual" = "시각", "textual" = "언어", "speed" = "속도")
confint_plot_sem(fit, type = "g", effect = "~", var_named = named_vec)

## End(Not run)
```

`contrast_msdn` *Contrast test with mean, standard deviation, and number of samples*

Description

Contrast test with mean, standard deviation, and number of samples

Usage

```
contrast_msdn(M, sd, n, ..., digits = 3, type = "res", meanadd = TRUE)
```

Arguments

`M`	means M = c(26.66, 19.74, 15.10)
`sd`	sd sd = c(4.51, 1.45, 2.56)
`n`	sample n = c(11, 7, 14)
`...`	contrast c(1, -1, 0), c(1, 0, -1), c(0, 1, -1) ....
`digits`	default 3
`type`	res, res_ci
`meanadd`	include means

Value

conrast result

Examples

```
## Not run:


contrast_msdn(M = c(26.66, 19.74, 15.10),
              sd = c(4.51, 1.45, 2.56),
              n = c(11, 7, 14),
              c(1, -1, 0) )

contrast_msdn(M = c(26.66, 19.74, 15.10),
              sd = c(4.51, 1.45, 2.56),
              n = c(11, 7, 14),
              c(1, -1, 0) ,
              meanadd = FALSE)

contrast_msdn(M = c(26.66, 19.74, 15.10),
              sd = c(4.51, 1.45, 2.56),
              n = c(11, 7, 14),
              c(1, -1, 0) ,
             meanadd = FALSE,
              type="res_ci")


contrast_msdn(M = c(26.66, 19.74, 15.10),
              sd = c(4.51, 1.45, 2.56),
             n = c(11, 7, 14),
              c(1, -1,  0),
              c(1,  0, -1),
              c(0,  1, -1)
)
contrast_msdn(M = c(26.66, 19.74, 15.10),
              sd = c(4.51, 1.45, 2.56),
              n = c(11, 7, 14),
              contr1 = c(1, -1,  0),
              contr2= c(1,  0, -1),
              contr3= c(0,  1, -1),
              meanadd=F
)
contrast_msdn(M = c(26.66, 19.74, 15.10),
              sd = c(4.51, 1.45, 2.56),
              n = c(11, 7, 14),
             contr1 = c(1, -1,  0),
             contr2= c(1,  0, -1),
              contr3= c(0,  1, -1),
              meanadd=FALSE, type="res_ci"
)
#error
contrast_msdn(M = c(26.66, 19.74, 15.10),
              sd = c(4.51, 1.45, 2.56),
             n = c(11, 7, 14),
              contr = list(c(1, -1,  0),
                           c(1,  0, -1),
                          c(0,  1, -1)
              ))
```

```
## End(Not run)
```

`convert_min_max_scaler` 비율척도로변환하는함수정의

Description

비율척도로변환하는함수정의

Usage

```
convert_min_max_scaler(data, columns = NULL)
```

Arguments

`data`	data
`columns`	colum all

Value

df

Examples

```
## Not run:
#'
 사용 예
data_frame: 데이터프레임 객체
변환할 열 이름을 명시적으로 지정하거나 NULL로 모든 열을 변환
scaled_data <- min_max_scaler(data_frame, columns = c("V1_1", "V2_1", "V3_1"))
print(scaled_data)

## End(Not run)
```

`convert_scale_df` 범위를변환하는데이터프레임전용함수정의

Description

범위를변환하는데이터프레임전용함수정의

Usage

```
convert_scale_df(
  df,
  from_min = -4,
  from_max = 4,
  to_min = 1,
  to_max = 9,
  select_col = everything(),
  remove = FALSE,
  post_fix = "_re"
)
```

Arguments

df	df
from_min	from_min
from_max	from_max
to_min	tomin
to_max	to max
select_col	col
remove	FALSE 유지
post_fix	_re

Value

data

Examples

```
## Not run:

convert_scale_df(jen01, from_min = -4, from_max = 4, to_min = 1, to_max = 9,
                 select_col = contains("Q6_"), remove = FALSE, post_fix = "_re") %>%
                 str()

# 변환된 데이터 구조 확인
str(jen01_transformed)

## End(Not run)
```

Cor2cov *cor2cov*

Description

cor2cov

Usage

```
Cor2cov(cor_matrix, sds = rep(1, ncol(cor_matrix)))
```

Arguments

`cor_matrix`	cor
`sds`	sds =c(...), default sds 1

Value

cov

Examples

```
## Not run:
# correlation matrix
mtcars[1:6] %>%cor()

#change cor to cov
mtcars[1:6] %>%cor() %>% cor2cov(sds = mtcars[1:6] %>% sapply(sd))

#real cov matrix
mtcars[1:6] %>%cov()


## End(Not run)
```

`count_ne` *0* 이아닌요소의개수를세는함수 *count_nonzero_elements*

Description

0 이아닌요소의개수를세는함수 count_nonzero_elements

Usage

```
count_ne(matrix)
```

Arguments

`matrix`	matrix

Value

count

Examples

```
## Not run:
#'

#  행렬
matrix2 <- matrix(c(0, 1, 2, 0, 0, 0,
```

```
                    0, 0, 0, 0, 3, 4), nrow = 6, byrow = TRUE)
matrix2
count_ne(matrix2)

# 주어진 theta 행렬

theta <- matrix(c(5, 0, 0, 0, 0, 0,
                  0, 6, 0, 0, 0, 0,
                  0, 0, 7, 0, 0, 0,
                  0, 0, 0, 8, 0, 0,
                  0, 0, 0, 0, 9, 0,
                  0, 0, 0, 0, 0, 10), nrow = 6, byrow = TRUE)
theta
count_ne(theta)

## End(Not run)
```

`Cov2cor` *cov to cor*

Description

cov to cor

Usage

```
Cov2cor(cov_matrix)
```

Arguments

`cov_matrix`	cov matrix

Value

cor mat

Examples

```
## Not run:
cov(mtcars[,c("mpg","hp")])%>%cov2cor()

## End(Not run)
```

`cov_lm_sem` *Regression analysis with covariance matrix, for paper checking*

Description

Regression analysis with covariance matrix, for paper checking

Usage

```
cov_lm_sem(
  formula = NULL,
  cov,
  n,
  sds = NULL,
  dv,
  ivs,
  method = "none",
  md = FALSE,
  lm = TRUE
)
```

Arguments

`formula`	regression formula
`cov`	cov matrix if your data cor, then you must sds
`n`	sample
`dv`	dv
`ivs`	iv variales
`method`	c("holm", "hochberg", "hommel", "bonferroni", "BH", "BY")
`md`	markdown default FALSE
`lm`	lm option TREW, FALSE sem

Value

result table, plot, confint sig plot

Examples

```
## Not run:
#' #method1
cov_lm_sem(formula="mpg ~ cyl+disp+hp+drat+wt" ,
n = 32, cov = cov(mtcars[1:6]))
# method 2
cov_lm_sem(dv = "mpg", ivs = c("cyl", "disp", "hp", "drat", "wt"),
n = 32, cov = cov(mtcars[1:6]))
# method 2 cor matrix
cov_lm_sem(formula="mpg ~ cyl+disp+hp+drat+wt" ,n = 32,
cov = cor(mtcars[1:6]),
           sds = mtcars[1:6]%>%sapply(sd))
#est
```

```
cov_lm(formula="mpg ~ cyl+disp+hp+drat+wt; qsec ~ mpg +vs;" ,
       n = 32, cov = cov(mtcars[1:8]),lm=FALSE)
#std
cov_lm(formula="mpg ~ cyl+disp+hp+drat+wt; qsec ~ mpg +vs;" ,
       n = 32, cov = cor(mtcars[1:8]), sds = sapply(mtcars[1:8], sd),
       lm=FALSE)

###################
#'
Songkj2024 = lav_matrix_lower2full(
  c(
    1,
    .38,   1,
    .49, .54, 1,
    .58, .33, .45, 1,
    .35, .17, .35, .60, 1,
    .22, .11, .20, .30, .28, 1,
    .46, .25, .46, .61, .67, .31, 1,
    .52, .29, .37, .57, .47, .31, .55, 1,
    -.09, -.03,-.02, -.20,-.05, .03, .00, .01, 1,
    .23, .10, .13, .07, -.03, .06, -.06, .40,-.18, 1   )
)

Songkj2024

colnames(Songkj2024)=c("학습애정열정","학습기회개방성",
                       "솔선수범독립성","효율학습자자아개념",
                       "학업적효능감","사회적효능감",
                       "자기조절효능감","경력개발역량",
                       "성별","연령")
rownames(Songkj2024)=c("학습애정열정","학습기회개방성",
                       "솔선수범독립성","효율학습자자아개념",
                       "학업적효능감","사회적효능감",
                       "자기조절효능감","경력개발역량",
                       "성별","연령")

#기술통계 데이터
songki2024_stat=  data.frame(
  var = c("학습애정열정","학습기회개방성",
          "솔선수범독립성","효율학습자자아개념",
          "학업적효능감","사회적효능감","자기조절효능감","경력개발역량",
          "성별","연령"),
  mean = c(4.14, 3.86, 4.14, 3.86, 3.61, 3.44, 3.95, 3.89, 0.65, 1.70),
  sd =   c(.52, .72, .70, .59, .64, .82, .54, .60, 0.48, 0.82)

);songki2024_stat

Songkj2024_cov = cor2cov(Songkj2024, sds= songki2024_stat$sd)
# Songkj2024_cov
# original
Songkj2024_model_1="
학업적효능감 ~ 학습애정열정 + 학습기회개방성 +
솔선수범독립성 + 효율학습자자아개념+ 성별 + 연령
"
Songkj2024_sem1 = sem( Songkj2024_model_1,
                       sample.cov = Songkj2024 ,
                       sample.nobs =  389  )
```

```
#cor, std
cov_lm(formula="학업적효능감 ~ 학습애정열정 +
학습기회개방성 + 솔선수범독립성 + 효율학습자자아개념 +
성별 + 연령" ,n = 389, cov = Songkj2024)
# cov
cov_lm(formula="학업적효능감 ~ 학습애정열정 +
학습기회개방성 + 솔선수범독립성 + 효율학습자자아개념 +
성별 + 연령" ,n = 389, cov = Songkj2024, sds = songki2024_stat$sd)
# cov
cov_lm(formula="학업적효능감 ~ 학습애정열정 +
학습기회개방성 + 솔선수범독립성 +
효율학습자자아개념 + 성별 + 연령" ,n = 389, cov = Songkj2024_cov)

## End(Not run)
```

`cov_simdata` *Regression with covariance, generate structural equation data restoration*

Description

Regression with covariance, generate structural equation data restoration

Usage

```
cov_simdata(
  covmat = NULL,
  n = 300,
  dv_mean = 0,
  iv_means = rep(0, ncol(covmat) - 1),
  means = NULL,
  cormat = NULL,
  sds = rep(1, ncol(cormat)),
  seed = NULL
)
```

Arguments

`covmat`	cov matrix
`n`	sample size default = 300
`dv_mean`	dv mean
`iv_means`	iv means
`means`	total means default null
`cormat`	correlation matrix
`sds`	sds is need cor to cov
`seed`	set seed today Sys.Data

Value

sim data

Examples

```
## Not run:
#'
#simdata mean setting 0
#set.seed(todaty)
cov(mtcars[,c("mpg","hp","wt")]) %>%cov_simdata(n=32)
cov(mtcars[,c("mpg","hp","wt")]) %>%cov_simdata(n=32, means=c(0,0,0))

cov(mtcars[,c("mpg","hp","wt")]) %>%
  cov_simdata(n=32, means=c(20.09, 146.69 ,3.22))

cov(mtcars[,c("mpg","hp","wt")]) %>%
  cov_simdata(n=32, dv_mean=20.09, iv_means=c(146.69 ,3.22))

#simdata
cov(mtcars[,c("mpg","hp","wt")]) %>%
  cov_simdata(n=32) %>%
  lm(formula = mpg ~ hp + wt)%>%summary()

#revised mean data Closer to real-world data
cov(mtcars[,c("mpg","hp","wt")]) %>%
  cov_simdata(n=32, dv_mean=20.09, iv_means=c(146.69 ,3.22)) %>%
  lm(formula = mpg ~ hp + wt)%>%summary()


#real data
lm(formula = mpg ~ hp +wt,data=mtcars)%>%summary()



# define sample statistics
x <- MASS::Cars93[,c("EngineSize", "Horsepower", "RPM")]
covx <- cov(x)
covx <- matrix(c(1.076122, 39.776999, -339.163745,
                 39.776999, 2743.079, 1146.634,
                 -339.163745, 1146.634, 356088.7097), nrow = 3,
               dimnames = list(c("EngineSize", "Horsepower", "RPM"),
                               c("EngineSize", "Horsepower", "RPM")))
covx
# 함수 실행
sim_data <- cov_simdata(covmat = covx, n = 100)
head(sim_data)
# check
sim_data <- cov_simdata(covmat = covx, n = 100)
lm( EngineSize~ Horsepower+RPM, data = sim_data) %>% summary()
#real check
lm( EngineSize ~ Horsepower +RPM , data=x)%>% summary()



#'

## End(Not run)
```

cramers_v *Tcramers_v*

Description

Tcramers_v

Usage

```
cramers_v(
  data,
  type = "cramer",
  digits = 3,
  correct = FALSE,
  simulate.p.value = FALSE
)
```

Arguments

data	data table
type	cramer
digits	3

Value

cramer correlation

cronbach_alpha *Model-based Cronbach's Alpha from Correlation Matrix*

Description

This function computes Cronbach's alpha using a correlation matrix. The correlation matrix can be directly provided or extracted from a fitted lavaan object (e.g., CFA or SEM results).

Usage

```
cronbach_alpha(x, type = c("ov", "lv"), digits = 3)
```

Arguments

x	A correlation matrix or a fitted lavaan object.
type	Character. If x is a lavaan object, choose which correlation matrix to use: **"ov"** Observed variable correlation matrix (default). **"lv"** Latent variable correlation matrix.
digits	Integer. Number of digits to round the result.

Details

The formula used is: alpha = k over (k - 1) (1 - trace(C) over sum(C))

where k is the number of items and C is a correlation matrix.

This alpha should be interpreted as a model-implied internal consistency rather than a raw data-based reliability coefficient.

Value

A numeric value of Cronbach's alpha.

Examples

```
## Example 1: Using a correlation matrix
R <- matrix(c(
  1.00, 0.45, 0.38,
  0.45, 1.00, 0.41,
  0.38, 0.41, 1.00
), nrow = 3, byrow = TRUE)
cronbach_alpha_model(R)

## Example 2: Using a CFA lavaan object
## Not run:
library(lavaan)

model <- '
  F =~ x1 + x2 + x3
'
fit <- cfa(model, data = mydata)

cronbach_alpha_model(fit)

## End(Not run)
```

---

cronbach_from_cor *Cronbach's Alpha from Model-Implied Correlation Matrix*

Description

This function computes Cronbach's alpha using a correlation matrix. It is intended for use with model-implied correlation matrices (e.g., from CFA or SEM), not raw data.

Usage

```
cronbach_from_cor(C)
```

Arguments

C	A correlation matrix.

Details

The formula used is: alpha = k over (k - 1) (1 - sum(diag(C)) over sum(C))

where k is the number of items and C is a correlation matrix.

This coefficient represents a model-based internal consistency estimate.

Value

A numeric value of Cronbach's alpha.

Examples

```
## Example 1: Simple correlation matrix
R <- matrix(c(
  1.00, 0.40, 0.35,
  0.40, 1.00, 0.42,
  0.35, 0.42, 1.00
), nrow = 3, byrow = TRUE)

cronbach_from_cor(R)

## Example 2: Model-implied correlation matrix
## Not run:
library(lavaan)

fit <- cfa(model, data = dat)
C <- lavInspect(fit, "cor.ov")
cronbach_from_cor(C)

## End(Not run)
```

---

cr_ave_from_lambda *Compute Composite Reliability (CR) and AVE from standardized loadings*

Description

표준화요인부하량 (lambda) 을이용하여합성신뢰도 (CR) 와평균분산추출지수 (AVE) 를계산한다.

Usage

```
cr_ave_from_lambda(lambda, na.rm = TRUE, check = TRUE)
```

Arguments

`lambda`	Numeric vector. 표준화요인부하량 (standardized factor loadings).
`na.rm`	Logical. 결측값제거여부. 기본값은 TRUE.
`check`	Logical. TRUE 일경우입력값의범위 (|lambda| 1) 를점검하고경고를출력한다.

Value

A named numeric vector with elements:

- CR: Composite Reliability
- AVE: Average Variance Extracted

References

Fornell, C., & Larcker, D. F. (1981). Evaluating structural equation models with unobservable variables and measurement error. Journal of Marketing Research, 18(1), 39–50.

Hair, J. F., Hult, G. T. M., Ringle, C. M., & Sarstedt, M. (2019). A Primer on Partial Least Squares Structural Equation Modeling (PLS-SEM).

Examples

```
lambda <- c(0.72, 0.81, 0.76)
cr_ave_from_lambda(lambda)

lambda_bad <- c(1.05, 0.80)
cr_ave_from_lambda(lambda_bad, check = TRUE)
```

---

cr_candidates_plot *cr_candidates_plot:* 문항제거전후 *CR* 비교시각화

Description

cr_candidates_plot: 문항제거전후 CR 비교시각화

Usage

```
cr_candidates_plot(
  df,
  latent = NULL,
  range_focus = TRUE,
  font_size = 4,
  padding = 0.04,
  v_line_show = TRUE
)
```

Arguments

df	lavaan 객체또는 cr_raise_candidates() 결과
latent	특정잠재변수만시각화 (선택)
range_focus	TRUE 면 CR 변화중심으로 x 축범위조정
font_size	텍스트크기
padding	범위조절값 (기본 0.04)
v_line_show	각문항의 cr_current 위치에수직선추가여부

Value

ggplot2 객체

Examples

```
## Not run:
library(lavaan)

model <- '
  Visual  =~ x1 + x2 + x3
  Textual =~ x4 + x5 + x6
  Speed   =~ x7 + x8 + x9
'

fit <- cfa(model, data = HolzingerSwineford1939, std.lv = TRUE)

# CR 비교 시각화 (전체)
cr_candidates_plot(fit)

# 특정 잠재변수만 시각화
cr_candidates_plot(fit, latent = "Speed")

## End(Not run)
```

---

cr_raise_candidates *cr_raise_candidates:* 잠재변수별 *CR* 향상후보문항탐색

Description

lavaan 모형결과를기반으로각잠재변수별로문항제거시 CR 변화량을계산합니다. 모든잠재변수를자동탐색하거나, 특정잠재변수를지정할수있습니다.

Usage

```
cr_raise_candidates(fit, latent = NULL, cutoff = -Inf, print_result = TRUE)
```

Arguments

`fit`	lavaan::cfa 또는 sem 결과객체
`latent`	분석대상잠재변수이름 (문자열). NULL 이면전체잠재변수자동분석.
`cutoff`	CR 변화량 (delta) 의최소기준. 기본값은 -Inf.
`print_result`	논리값. TRUE 면변수별요약메시지출력.

Value

tibble

Examples

```
## Not run:
## 예제: lavaan 내장 데이터와 CFA 모델 사용
library(lavaan)

# 모델 정의
model <- '
  Visual  =~ x1 + x2 + x3
  Textual =~ x4 + x5 + x6
  Speed   =~ x7 + x8 + x9
'

# CFA 적합
fit <- cfa(model, data = HolzingerSwineford1939, std.lv = TRUE)

# CR 향상 후보 탐색 (전체 변수)
cr_raise_candidates(fit)

# 특정 잠재변수만 분석
cr_raise_candidates(fit, latent = "Visual", cutoff = 0.01)

## End(Not run)
```

---

cut_print *Functions that cutoff a given value*

Description

Functions that cutoff a given value

Usage

```
cut_print(
  data,
  cut = 0,
  digits = 3,
  rm_row = FALSE,
  rm_cut = FALSE,
  star = FALSE,
  each = NULL
)
```

Arguments

data	data.frame
cut	cutoff default 0
digits	round 3
rm_row	remove row
rm_cut	remove cutoff rows
star	over cut marked star pasted
each	each col using cut

Value

data

Examples

```
## Not run:
#'
mtcars%>% cut_print(cut = 280)
mtcars%>% cut_print(cut = 280, rm_row = T)
mtcars%>% cut_print(cut = 280, rm_row = T,rm_cut=T)

#'
# 예제 데이터
data <- data.frame(
  name = c("C_S1", "C_S2", "C_S3", "C_S4", "C_S5", "A_M1", "A_M2",
           "A_M3", "B_A2", "B_A3", "D_P1", "D_P2", "D_P3"),
  block = c("자기효능감", "자기효능감", "자기효능감", "자기효능감",
            "자기효능감", "진로동기", "진로동기", "진로동기",
            "진로태도", "진로태도", "진로준비", "진로준비", "진로준비"),
  자기효능감 = c(0.88334202, 0.82571836, 0.79357537, 0.78069220,
             0.63356092, 0.79936329, 0.75998517, 0.66930167, 0.08384875,
             0.67950207, 0.75093254, 0.70621491, 0.33677926),
  진로동기 = c(0.819634450, 0.756922172, 0.732994108, 0.617655739,
            0.507011139, 0.859130054, 0.851836536, 0.815780379, 0.006420011,
            0.699152545, 0.769048149, 0.688524060, 0.336711208),
  진로태도 = c(0.6497353, 0.6010873, 0.5346407, 0.5128086,
            0.3223576, 0.6769730, 0.6030949, 0.4615263, 0.1463340,
            0.9964532, 0.5030203, 0.6625537, 0.4041093),
  진로준비 = c(0.76921767, 0.67147592, 0.58862568, 0.63738845,
            0.50527519, 0.69871013, 0.68835534, 0.66810321, 0.03263257,
            0.69001534, 0.86708694, 0.84986139, 0.55547251)
)

# 각 열이 numeric 변수인지 확인
numeric_cols <- sapply(data, is.numeric)
print(numeric_cols)

# 함수 실행 예시
result <- cut_print(data, cut = 0.01, digits = 2, rm_row = FALSE,
                    rm_cut = TRUE, star = FALSE, each = c(0.78, 0.8, 0.8, 0.8))
print(result)

#'

## End(Not run)
```

---

cut_print_all *overall cut*

Description

overall cut

Usage

```
cut_print_all(data, cut = 0, imp = "", digits = 3)
```

Arguments

data	data
cut	cut =0.5
imp	”
digits	3

Value

data

---

dall *Quick View Utility for data.frame or tibble*

Description

This function provides a convenient way to preview or fully display a data.frame or tibble object in a pipe-friendly workflow. By default, it returns the first `n` rows of the data.

Usage

```
dall(data, n = 10, type = NULL)
```

Arguments

data	A data.frame or tibble object.
n	Integer. Number of rows to display when previewing. Default is 10.
type	Character. One of `NULL`, `"head"`, or `"all"`. If NULL (default), the first `n` rows are returned. If `"all"`, the full data.frame is returned. If `"head"`, the first `n` rows are returned explicitly.

Details

This function is designed as a lightweight alternative to `View()`, optimized for console-based inspection and pipe workflows. The default behavior (type = NULL) prioritizes safe data previewing.

Value

A data.frame.

Examples

```
## Example 1: Basic preview (default n = 10)
dall(mtcars)

## Example 2: Preview first 5 rows
dall(mtcars, n = 5)

## Example 3: Explicit head option
dall(mtcars, type = "head", n = 3)

## Example 4: Return full data
dall(mtcars, type = "all")

## Example 5: Pipe-friendly usage
library(dplyr)
mtcars %>%
  dplyr::select(mpg, cyl, hp) %>%
  dall(n = 6)
```

`datapaste` *exceldata to dataframe*

Description

exceldata to dataframe

Usage

```
datapaste(type = "matrix", x = "clipboard", header = TRUE)
```

Arguments

`type`	”clipboard” to table data. dataframe, tibble
`x`	”clipboard”
`header`	header=FALSE

Value

paste data.frame

Examples

```
#first you must copy data (Ctrl+C)
## then excute below ctrl+Enter
df = datapaste()
df = datapaste("tibble")
df = datapaste("table")
df = datapaste("data.frame")


datapaste("datapasta")
New = data.frame(row.names = c("1", "2", "3", "4", "5", "6"),
```

```
응답자 = c("응답1", "응답2", "응답3", "응답4", "응답5", "응답6"),
role_v_inst1 = c(5, 5, 5, 5, 4, 5),
role_v_inst2 = c(5, 5, 5, 5, 5, 5),
role_v_inst3 = c(4, 4, 5, 5, 4, 4),
role_v_inst4 = c(5, 4, 5, 5, 3, 4))
```

`datapaste_text` *data.frame to input text*

Description

data.frame to input text

Usage

```
datapaste_text(data, rownames = TRUE)
```

Arguments

`data`	data.frame
`rownames`	rownames=TRUE, FALSE -> nothing rownames

Value

text

Examples

```
## Not run:

# 데이터 프레임 생성
cho2013_cor <- data.frame(
  필요성 = c(1.000, 0.320, 0.266, 0.506, 0.429, 0.401),
  수업 = c(0.320, 1.000, 0.647, 0.463, 0.548, 0.554),
  교사 = c(0.266, 0.647, 1.000, 0.383, 0.491, 0.545),
  계획 = c(0.506, 0.463, 0.383, 1.000, 0.531, 0.491),
  영향 = c(0.429, 0.548, 0.491, 0.531, 1.000, 0.743),
  만족도 = c(0.401, 0.554, 0.545, 0.491, 0.743, 1.000)
)
cho2013_cor %>% datapaste_text()
#'#정성령(2014)
Jungsy2014 = "
수강료 1
강사 .165** 1
수업 .303** .144** 1
개인관리 .494** .069 .284** 1
인지도 .425** .176** .407** .424** 1
이상적영향력 .210** -.056 .212** .286** .222** 1
영감적동기화 .190** -.133** .211** .290** .230** .659** 1
지적자극 .300** -.136** .159** .402** .238** .533** .517** 1
개별적 배려 .202** -.153** .150** .250** .291** .684** .618** .530** 1
```

```
"
lav_matrix(Jungsy2014)

Jungsy2014_cor = lav_matrix(Jungsy2014)

Jungsy2014_cor %>% make_df_text()

## End(Not run)
```

`DescribeBy` *DescribeBy statistics by group*

Description

DescribeBy statistics by group

Usage

```
DescribeBy(
  data,
  formula,
  add_var = NULL,
  stat = FALSE,
  agg = FALSE,
  gm = FALSE,
  digits = 2
)
```

Arguments

`data`	data.frame
`formula`	general formula
`add_var`	one sample name, Used to obtain statistics for one variable
`stat`	TRUE t.test and aov(), thes stat ="t.test", or 'aov'
`agg`	TRUE, mnay to many variable is showed. This functuion that makes aggregate() work full
`gm`	default FALSE, TRUE Use group data obtained from group descriptive statistics to obtain group-level descriptive statistics (means, standard deviations, etc.) and to analyze a mixed model
`digits`	Troundings

Value

Mea, SD, N, min, max, skew, kurt

Examples

```
## Not run:
## Used to obtain statistics for one variable
DescribeBy(mpg ~ 1, data = mtcars)
DescribeBy(mpg ~ 1, data = mtcars, "mpg")
DescribeBy(mpg ~ 1, data = mtcars, add_var = "mpg")

##The group variable is 2 or greater
DescribeBy(mpg ~ vs, data = mtcars)
DescribeBy(mpg ~ vs+am, data = mtcars)
DescribeBy(mpg ~ vs+am+cyl, data = mtcars)

##statistic data output
DescribeBy(mpg ~ 1, data = mtcars, "mpg", stat="t.test")
DescribeBy(mpg ~ vs, data = mtcars, stat="t.test")
DescribeBy(mpg ~ vs+am, data = mtcars, stat="aov")
DescribeBy(mpg ~ vs*cyl, data = mtcars, stat="aov")

##new group mean by group
#' group_by(mtcars, am) %>%
 summarise(mpg = mean(mpg)) %>%
  summarise(mean(mpg),sd(mpg),length(mpg), min(mpg),max(mpg))

### A tibble: 1 × 5
###`mean(mpg)` `sd(mpg)` `length(mpg)` `min(mpg)` `max(mpg)`
###<dbl>     <dbl>          <int>      <dbl>      <dbl>
###  1        20.8      5.12             2       17.1       24.4

DescribeBy(mpg ~ am , data= mtcars) %>%
    rename(mpg = Mean) %>%   #change name
      DescribeBy(formula = mpg ~ 1)  # mean of mpg

## End(Not run)
```

`DE_effect` *SEM direct effect result*

Description

SEM direct effect result

Usage

```
DE_effect(model_sem, effect = "~", effect2 = NULL)
```

Arguments

`model_sem`	lavaan object
`effect`	select op
`effect2`	select op 2nd option

Value

repot

Examples

```
## Not run:

 #library(lavaan) # only needed once per session

models1 <- '
  # measurement model
    ind60 =~ x1 + x2 + x3
    dem60 =~ y1 + y2 + y3 + y4
    dem65 =~ y5 + y6 + y7 + y8
    # regressions
    dem60 ~ ind60
    dem65 ~ ind60 + dem60
      # residual correlations
    y1 ~~ y5
    y2 ~~ y4 + y6
    y3 ~~ y7
    y4 ~~ y8
    y6 ~~ y8
'
models2 ='
  # measurement model
    ind60 =~ x1 + x2 + x3
    dem60 =~ y1 + y2 + y3 + y4
    dem65 =~ y5 + y6 + y7 + y8
     # regressions
    dem60 ~ a1*ind60
    dem65 ~ a2*ind60 + a3*dem60
 #NEw
   ie := a1*a3
   te := a2 +a1*a3
'
fits1 <- sem(models1, data=PoliticalDemocracy)
fits2 <- sem(models2, data=PoliticalDemocracy)
summary(fits1, standardized=TRUE)
summary(fits2, standardized=TRUE)

parameterEstimates(fits1, ci=F, stand=T)
parameterEstimates(fits2, ci=F, stand=T)
DE_effect(fits1)
DE_effect(fits1, effect="=~")
DE_effect(fits1, effect="=~", effect2="~")
DE_effect(fits1, effect="~~")
DE_effect(fits1, effect=":=")

DE_effect(fits2)
DE_effect(fits2, effect="=~")
DE_effect(fits2, effect="~~")
DE_effect(fits2, effect=":=")
DE_effect(fits2, effect=":=", effect2="~")
 DE_effect(fits)%>% interpretation_de()
 # same method
 DE_effect(fits)%>% report_sem_de()
```

```
## End(Not run)
```

`diagram` *diagram sem plot*

Description

diagram sem plot

Usage

```
diagram(
  lav_obj,
  what = "est",
  rotation = 4,
  nDigits = 2,
  residScale = 18,
  sig = TRUE,
  edge.label.cex = 1.5
)
```

Arguments

`lav_obj`	lavaan
`what`	est, std, model
`rotation`	4
`nDigits`	3
`residScale`	18
`sig`	FALSE
`edge.label.cex`	1

Value

plot

Examples

```
## Not run:
example(cfa)
fit %>% diagram()


## End(Not run)
```

`diagram2` *diagram sem plot 2*

Description

diagram sem plot 2

Usage

```
diagram2(
  lav_obj,
  whatLabels = "est",
  rotation = 2,
  nDigits = 2,
  residScale = 18,
  mar = c(2, 8, 3, 8),
  edge.label.cex = 1,
  edge.label.position = 0.5,
  sizeMan = 6,
  sizeMan2 = 4,
  sizeLat = 8,
  sizeLat2 = 6,
  border.width = 2,
  edge.width = 1.5,
  style = "lisrel",
  sig = TRUE,
  asize = 1.5,
  shapeLat = "circle",
  residuals = FALSE,
  curve = 1,
  layout = "tree",
  exoVar = TRUE,
  exoCov = TRUE,
  structual = FALSE,
  nodeLabels = NULL,
  groups = FALSE
)
```

Arguments

`lav_obj`	lavaan
`whatLabels`	est, std, model(sig=FALSE)
`rotation`	rotation= 4
`nDigits`	3
`residScale`	residScale=18
`mar`	c(2,8,3,10)
`edge.label.cex`	edge.label.cex= 1
`edge.label.position`	edge.label.position= 0.5

sizeMan	sizeMan = 8,
sizeMan2	sizeMan2 = 4,
sizeLat	sizeLat = 10,
sizeLat2	sizeLat2 = 6
border.width	border.width 2
edge.width	edge.width 1.5
style	style=”lisrel”
sig	sig=TRUE
asize	asize = 1.5
shapeLat	shapeLat =”circle”,ellipse
residuals	residuals=TRUE
curve	curve=1
layout	”tree”
exoVar	exoVar=TRUE,
exoCov	exoCov=TRUE,
structual	structual=FALSE
nodeLabels	nodeLabels=NULL
groups	true color

Value

plot

Examples

```
## Not run:

example(sem)
fit%>% diagram2()
#'
fit %>%diagram2(mar=c(3,8,3,6), sizeLat = 6, sizeMan = 6,
                edge.label.cex = 0.7)

#model
fit %>%diagram2("model", sig=F)

fit %>%diagram2("model", sig=F, residuals=F, curve=2)

fit %>%diagram2("model", sig=F, residuals=F, curve=2, rotation = 4)

#making method layout
make_layout(nc=10)
make_layout(nc=10, nr=6)
make_layout(3,6)

lay1 = make_layout(6)
lay1

edit_layout(lay1)
```

```
lay1 %>%edit_layout()
lay2 = lay1 %>%edit_layout()
lay2


## End(Not run)
```

`diagram_model` *Drawing Structural Equation Models Using the lavaan Syntax (park JH PhD)*

Description

Drawing Structural Equation Models Using the lavaan Syntax (park JH PhD)

Usage

```
diagram_model(
  model,
  residuals = FALSE,
  whatLabels = "model",
  rotation = 2,
  residScale = 12,
  layout = "tree2",
  edge.label.cex = 1,
  edge.label.position = 0.5,
  edge.color = "black",
  sizeMan = 6,
  sizeMan2 = 3,
  sizeLat = 8,
  sizeLat2 = 4,
  style = "lisrel",
  sig = FALSE,
  exoVar = FALSE,
  exoCov = TRUE,
  curve = 1.5,
  asize = 2,
  mar = c(3, 3, 3, 3),
  nDigits = 3,
  shapeLat = "circle",
  shapeMan = "rectangle",
  shapeInt = "triangle",
  sample.nobs = 100,
  border.width = 2,
  edge.width = 1.5,
  Groups = FALSE,
  group_match = "lat",
  growth = FALSE,
  structural = FALSE,
  edgeLabels = NULL,
```

```
  nodeLabels = NULL,
  fixedStyle = 1,
  color = list(lat = "white", man = "white"),
  type = "plot"
)
```

Arguments

`model`	lavaan syntax
`residuals`	residauls TRUE
`whatLabels`	”model
`rotation`	rotation=2
`residScale`	residScale=12
`layout`	layout=”tree”
`edge.label.cex`	1
`edge.label.position`	0.5
`edge.color`	edge.color= black
`sizeMan`	8
`sizeMan2`	4
`sizeLat`	10
`sizeLat2`	6
`style`	lisrel
`sig`	FALSE
`exoVar`	FALSE
`exoCov`	TRUE
`curve`	1.5
`asize`	1.5
`mar`	c(2,8,3,10)
`nDigits`	2
`shapeLat`	”circle, ellipse
`shapeMan`	retangle
`shapeInt`	triangle ,
`sample.nobs`	sample.nobs = 100
`border.width`	border.width=1
`edge.width`	edge.width=1.5
`Groups`	FALSE
`group_match`	”FALSE”lat#’
`growth`	growth= FALSE, TRUE model.type =”grouwth”
`structural`	structural = FALSE
`edgeLabels`	edgeLabels=NULL
`nodeLabels`	nodeLabels=NULL
`fixedStyle`	fixedStyle=1 default rbase linetype
`color`	color
`type`	type =”plot, and res is lavaan simdata result

Value

plot

Examples

```
## Not run:
#'
model_test <- '
  # latent variable definitions
     ind60 =~ x1 + x2 + x3
     dem60 =~ y1 + a*y2 + b*y3 + c*y4
     dem65 =~ y5 + a*y6 + b*y7 + c*y8

  # regressions
    dem60 ~ ind60
    dem65 ~ ind60 + dem60

  # residual correlations
    # y1 ~~ y5
    # y2 ~~ y4 + y6
    # y3 ~~ y7
    # y4 ~~ y8
    # y6 ~~ y8
'

HSmodel <- ' visual  =~ x1 + x2 + x3
             textual =~ x4 + x5 + x6
             speed   =~ x7 + x8 + x9 '


m1a <- "
y1 =~ x1 + x2 +x5
y2 =~ x3 + x4 +x6
y1 ~~ y2"

diagram_model(model= HSmodel, Goups=F,  residuals=F)

diagram_model(model= model_test, style="ram", groups=F)
diagram_model(model= model_test, residuals=T)
diagram_model(model= m1a, residuals=T)


#plspm model
#model 1
model1 = "
EXPE ~ IMAG
QUAL ~ EXPE
VAL ~  QUAL+ EXPE
SAT ~ IMAG + EXPE + QUAL + VAL
LOY ~ SAT + IMAG
"
sat_path = plspm_lav2path(model1,
                          fct_order=c("IMAG", "EXPE","QUAL", "VAL", "SAT", "LOY" ))
sat_path

blocks_model ="
IMAG =~ imag1 + imag2 + imag3 + imag4 + imag5
```

```
EXPE =~ expe1 + expe2 + expe3 + expe4 + expe5
QUAL =~ qual1 + qual2 + qual3 + qual4 + qual5
VAL =~ val1 + val2 + val3 + val4
SAT =~ sat1 + sat2 + sat3 + sat4
LOY =~ loy1 + loy2 + loy3 + loy4
"
blocks_model
plspm_lav2blocks(blocks_model)

#model 그리기
paste(model1, blocks_model)%>%

  diagram_model(rotation = 1, sizeLat = 6, sizeMan = 4, sizeMan2=3,
                mar=c(2,2,2,2))

#'
#'
#'

## End(Not run)
```

`Diff_effect` *Diff effect*

Description

Diff effect

Usage

```
Diff_effect(
  model_sem,
  effect1 = "Diff",
  effect2 = "diff",
  label = FALSE,
  ci = FALSE
)
```

Arguments

`model_sem`	lavaan objgect
`effect1`	”Diff”
`effect2`	”diff”
`label`	label
`ci`	confidence interval

Value

parameterEstimates()

Examples

```
## Not run:
#library(lavaan) # only needed once per session

models1 <- '
  # measurement model
    ind60 =~ x1 + x2 + x3
    dem60 =~ y1 + y2 + y3 + y4
    dem65 =~ y5 + y6 + y7 + y8
    # regressions
    dem60 ~ ind60
    dem65 ~ ind60 + dem60
      # residual correlations
    y1 ~~ y5
    y2 ~~ y4 + y6
    y3 ~~ y7
    y4 ~~ y8
    y6 ~~ y8

models2 ='
  # measurement model
    ind60 =~ x1 + x2 + x3
    dem60 =~ y1 + y2 + y3 + y4
    dem65 =~ y5 + y6 + y7 + y8
     # regressions
    dem60 ~ a1*ind60
    dem65 ~ a2*ind60 + a3*dem60
 #NEw
   ie := a1*a3
   te := a2 +a1*a3
   Diff := a1 - a3




fits1 <- sem(models1, data=PoliticalDemocracy)
fits2 <- sem(models2, data=PoliticalDemocracy)
summary(fits1, standardized=TRUE)
summary(fits2, standardized=TRUE)
fits2 %>% Diff_effect()

## End(Not run)
```

---

e *Google translaste english to koean*

Description

Google translaste english to koean

Usage

```
e(
```

```
  text = "hello let's start translate english to korean.",
  slang = "en",
  elang = "ko",
  show = "translate"
)
```

Arguments

text	Input text
slang	english
elang	korean
show	'translate', 'browser', 'all' select 3 opttions

Examples

```
#eexamples code
## Not run:

e("Translating English to Korean is the default option.")

## End(Not run)
```

EAI_index — *EAI_index: Compute and visualize the Eduteck Acceptance Index (EAI) for given profiles*

Description

EAI_index: Compute and visualize the Eduteck Acceptance Index (EAI) for given profiles

Usage

```
EAI_index(
  df,
  profile = NULL,
  caption = "",
  sel_col = "Class",
  level_start = 9,
  jump = 5,
  adj = 0,
  size_label = 5,
  size_text = 5,
  color_line = "gray20",
  color_label = "gray20",
  lpos = 2.35,
  rpos = 2.65,
  print = TRUE,
  web = FALSE
)
```

Arguments

df	Data frame containing the necessary columns for EAI calculation.
profile	A specific profile (e.g., a category from a column) to filter the data. Default is NULL, meaning all data will be used.
caption	Optional caption for the plot title. Default is an empty string.
sel_col	The column name in the data frame used for selecting profiles. Default is "Class".
level_start	Starting level for categorization in the plot. Default is 9.
jump	Increment to calculate subsequent levels. Default is 5.
adj	Adjustment factor for level calculations. Default is 0.
size_label	Font size for level annotations. Default is 4.
size_text	Font size for score labels in the plot. Default is 5.
color_line	Color for the dashed lines in the plot. Default is "gray20".
color_label	Color for the level annotations. Default is "gray20".
lpos	Horizontal position for left-side annotations. Default is 2.35.
rpos	Horizontal position for right-side annotations. Default is 2.65.
print	Logical indicating whether to print the summary table. Default is TRUE.
web	Logical indicating whether to print web the summary table. Default is FALSE.

Value

A ggplot object visualizing the EAI index along with calculated levels and annotations.

Examples

```
## Not run:
set.seed(123)
df <- data.frame(
  Class = rep(c("A", "B"), each = 10),
  EAI = rnorm(20, 15, 5),
  PU_sum = rnorm(20, 20, 4),
  PEOU_sum = rnorm(20, 18, 3),
  ATT_sum = rnorm(20, 17, 4),
  USE_sum = rnorm(20, 19, 5),
  EAI_check = rnorm(20, 16, 2)
)

g <- EAI_index(df, profile = "A", caption = "EAI Analysis", sel_col = "Class")
print(g)

## End(Not run)
```

`edit_layout` *Edit an Existing Layout Matrix*

Description

Edit an Existing Layout Matrix

Usage

```
edit_layout(data)
```

Arguments

`data`	A matrix or data frame to be edited interactively.

Value

A matrix or data frame representing the edited layout.

Examples

```
## Not run:
layout <- matrix(1:9, ncol = 3)
edited_layout <- edit_layout(layout)
print(edited_layout)

## End(Not run)
```

`effect1` *effec1 is the path coefficient of the structural equation.*

Description

effec1 is the path coefficient of the structural equation.

Usage

```
effect1(
  x,
  format = "markdown",
  op1 = "~",
  op2 = ":=",
  what = "std",
  digits = 3,
  layout = "tree2",
  curve = 1,
  mar = c(8, 8, 8, 8),
  residScale = 15,
  sizeLat = 12,
  sizeLat2 = 6,
  sizeMan = 8,
```

```
  sizeMan2 = 4,
  edge.label.cex = 0.9,
  label.cex = 1.5,
  asize = 1.4,
  name_change = FALSE,
  label_list = NULL,
  type = "res"
)
```

Arguments

x	lavaan result
format	'markdown', 'html'
op1	first'~'
op2	first ':=' newparameters
what	default 'std', Can be changeed to 'est'
digits	round 3
layout	'tree2'
curve	curve =1
mar	margins default c(8,8,8,8)
residScale	length of residuals. default 15
sizeLat	Horizontal length of latent variable
sizeLat2	Vertical length of latent variable
sizeMan	horizontal length of the item
sizeMan2	Vertical length of the item
edge.label.cex	edge size
label.cex	label size
asize	arrow size
name_change	When set to default, the variable name of the data is used, and when set to TRUE, a new variable name can be entered. Just enter them all in label_list using the c() function.
label_list	label list
type	type output

Examples

```
## Not run:
## change label_list
abel_listnode.name =list(
  list(node="f인정욕구", to="인정신념"),
  list(node="f파국화",  to="불안신념1"),
  list(node="f과잉불안", to="불안신념2"),
  list(node="f무력감",  to="의존신념"),
  list(node="f개인완벽", to="해결신념"),
  list(node="IRB",  to="비합리적신념"),
  list(node="PREACT",  to="진로준비행동"),
```

```
  list(node="DECISION",  to="진로결정수준"),
  list(node="f정보수집",  to="정보수집"),
  list(node="f직업체험",  to="직업체험"),
  list(node="f진로탐색",  to="정보탐색"),
  list(node="f구체성",  to="구체성"),
  list(node="f확신성",  to="확신성"),
  list(node="f적합성",  to="적합성"))

## End(Not run)
```

`factor_layout` *Extracting the layout matrix of a plspm model*

Description

Extracting the layout matrix of a plspm model

Usage

```
factor_layout(plspm_model)
```

Arguments

`plspm_model`	sempaths

Value

mat

Examples

```
## Not run:
#'
data(offense)
offense %>%str()

nfl_path = matrix(
  c(0, 0, 0, 0, 0,
    0, 0, 0, 0, 0,
    0, 0, 0, 0, 0,
    0, 1, 1, 0, 0,
    1, 0, 0, 1, 0),
  nrow = 5, ncol = 5, byrow = TRUE )

rownames(nfl_path) <- c("Speical", "Rushing", "Passing", "Offense", "Scoring")
colnames(nfl_path) <- c("Speical", "Rushing", "Passing", "Offense", "Scoring")
nfl_path

nfl_blocks = list(7:8, 1:3, 4:6,1:6, 9:11)

plspm_blocks_match(data=offense, blocks=nfl_blocks,
                   paths=nfl_path)
```

```
nfl_mode =c("B","A","A","A","A")

nfl_pls = plspm(Data=offense, nfl_path, nfl_blocks, modes = nfl_mode)
nfl_pls%>%summary()

#########
nfl_pls_boot = plspm_sem(Data=offense, nfl_path, nfl_blocks, modes = nfl_mode)

#Layouts extracted from mockups
plspm_semPaths2(nfl_pls_boot,sizeLat = 6)%>%
  plspm_factor_layout()

#Make it input material
plspm_semPaths2(nfl_pls_boot,sizeLat =6)%>%
  plspm_factor_layout() %>%layout_mat(text=T)

#Organizing materials for input
plspm_semPaths2(nfl_pls_boot,sizeLat =6)%>%
  plspm_factor_layout() %>%layout_mat()%>%
  mat_text_arrange()

## End(Not run)
```

`factor_scores` *Add factor score calculation functions and data*

Description

Add factor score calculation functions and data

Usage

```
factor_scores(
  data,
  range = 1:ncol(data),
  nfactor = NULL,
  prefix = "",
  rotate = "varimax"
)
```

Arguments

`data`	data
`range`	range default 1:ncol(data)
`nfactor`	number of factor
`prefix`	data prefic
`rotate`	varimax

Value

data

Examples

```
## Not run:
#'
# 활용방법 1
factor_scores(mtcars,
              range = c("disp", "hp", "drat", "wt", "qsec"),
              nfactor = 2, prefix="a")

mtcars %>% select("disp", "hp", "drat", "wt", "qsec") %>% factor_scores(nfactor=2)

# 활용방법 2
factor_scores(mtcars, range = c(3:7), nfactor = 2)

factor_scores(mtcars, c("disp", "hp", "drat", "wt", "qsec"), nfactor = 2)
factor_scores(mtcars, c(3, 4, 5, 6, 7), nfactor = 2)

factor_scores(mtcars, range = "disp:qsec", nfactor = 2)


mtcars %>% select("disp", "hp", "drat", "wt", "qsec") %>%
  factor_scores(c("disp","hp"),nfactor=2, prefix="a") %>%
  factor_scores(c("drat", "wt", "qsec"),nfactor=3, prefix="b") %>%head()


## End(Not run)
```

`fa_print` *fa_print: A custom function to format and display Factor Analysis (FA) results*

Description

fa_print: A custom function to format and display Factor Analysis (FA) results

Usage

```
fa_print(
  data,
  cut = 0.4,
  sort = TRUE,
  abs = TRUE,
  desc = TRUE,
  star = FALSE,
  justify = "right"
)
```

Arguments

`data`	A data frame containing the FA results to format.
`cut`	Numeric. The threshold for suppressing small factor loadings. Defaults to 0.4.
`sort`	Logical. If TRUE, sorts the factor loadings. Defaults to TRUE.
`abs`	Logical. If TRUE, applies the absolute value to the factor loadings for threshold comparison. Defaults to TRUE.
`desc`	Logical. If TRUE, sorts in descending order. Defaults to TRUE.
`star`	Logical. If TRUE, marks factor loadings exceeding the cut-off with "*". Defaults to FALSE.
`justify`	Character. Specifies justification ("left", "center", or "right") for formatted values. Defaults to "right".

Value

A data frame with formatted and optionally sorted FA results, keeping "Eigenvalues", "Proportion Var", and "Cumulative Var" unaffected by filtering and sorting.

Examples

```
## Not run:
# Example usage with a FA result
library(psych)
results <- psych::fa(mtcars, nfactors = 2, rotate = "promax")
summary <- summary_fa(results)
formatted_results <- fa_print(summary, cut = 0.3, star = TRUE)
print(formatted_results)

## End(Not run)
```

`fei_Chisq_test` *fei* 분석을위한카이제곱함수

Description

fei 분석을위한카이제곱함수

Usage

```
fei_Chisq_test(tabledf, col = "빈도", name = "구분", ns = FALSE, sep = " / ")
```

Arguments

`tabledf`	빈도분석 Freq_table 데이터
`col`	" 빈도"
`name`	구분
`ns`	FALSE 생성한것사용,TRUE RVAideMemoire 로행렬로나타남
`sep`	sep="/"

Value

df

Examples

```
## Not run:
#'
eduteck_s$Q31%>% na_remover() %>% Freq_table(prop=TRUE) %>%
  fei_Chisq_test()

## End(Not run)
```

`fei_dv_res` *T 종속변수의빈도를세는함수 (에듀테크연구보고용)*

Description

T 종속변수의빈도를세는함수 (에듀테크연구보고용)

Usage

```
fei_dv_res(
  df,
  title = "",
  x_size = 20,
  label_text = 5,
  x_range = 100,
  type = "res"
)
```

Arguments

df	df
title	변수명
x_size	20 사이즈로조절
label_text	막대그래프끄트
x_range	범위조절
type	res all, freq, console, md, gg, graph na_Count, cleandata

Value

df

Examples

```
## Not run:

eduteck_s$Q14_5 %>% fei_dv_res("자기주도 공부력 향상")



## End(Not run)
```

`filter_rows` *filter and, or function*

Description

filter and, or function

Usage

```
filter_rows(data, op = "or", ...)
```

Arguments

`data`	data
`op`	operatror or, and
`...`	condition

Value

data

Examples

```
## Not run:
#'
mtcars %>% filter_rows(cyl = 6)
mtcars %>% filter_rows(vs = 1)
mtcars %>% filter_rows(cyl = 6, vs = 1, op ="and")
mtcars %>% filter_rows(op="and", cyl = 6, vs = 1)
mtcars %>% filter_rows("and", cyl = 6, vs = 1 )

mtcars %>% filter_rows("or", cyl = 6, vs = 1 )
mtcars %>% filter_rows(op= "or", cyl = 6, vs = 1 )
mtcars %>% filter_rows(cyl = 6, vs = 1, op ="or")

mtcars %>% filter_rows(cyl = 6, vs = 1, gear = 4, op = "and")
mtcars %>% filter_rows(cyl = 6, vs = 1, gear = 4, op = "or")
mtcars %>% filter_rows(cyl = 6, vs = 1, gear = 4, op = "or")

mtcars %>% filter_rows(mpg > 25) #error
mtcars %>% filter(mpg > 25)

## End(Not run)
```

`find_paths` *Resolver knows the path*

Description

Resolver knows the path

Usage

```
find_paths(data, type = "paths", est = "Original", allpaths = FALSE)
```

Arguments

`data`	boot data
`type`	es, all, est, ind, paths(default)
`est`	Original
`allpaths`	allpaths TRUE all, FALSE is ind
`start`	IMAG

Value

result text

Examples

```
## Not run:
#'
# 예제 데이터
satpls_boot <- list(boot = list(paths = data.frame(
  Original = c(0.578959128, 0.200724200, 0.275149576,
  0.848344408, 0.105477650, -0.002753995,
  0.676655529, 0.122144566, 0.589330650, 0.495479322),
  Mean.Boot = c(0.57466537, 0.20926883,
  0.28748175, 0.84907549, 0.09629728,
  -0.02304841, 0.68789406, 0.13300281,
  0.59112195, 0.48611617),
  Std.Error = c(0.04941293, 0.05440409,
   0.07095248, 0.02022514, 0.05731729,
   0.06209752, 0.06286102, 0.08889983,
   0.08591013, 0.07920673),
  perc.025 = c(0.46120153, 0.09494506,
  0.14255079, 0.80126496, -0.02555946,
  -0.15182035, 0.57655200, -0.01836564,
   0.41297885, 0.34267859),
  perc.975 = c(0.65528114, 0.30696127,
   0.41537966, 0.88210331, 0.19678402,
   0.09426639, 0.81986383, 0.32270662,
    0.75274086, 0.63147051),
  row.names = c("IMAG -> EXPE",
  "IMAG -> SAT",
   "IMAG -> LOY", "EXPE -> QUAL",
   "EXPE -> VAL", "EXPE -> SAT",
```

```
   "QUAL -> VAL", "QUAL -> SAT",
   "VAL -> SAT", "SAT -> LOY")
)))

# 함수 실행
find_paths_all(satpls_boot$boot$paths)
find_paths_all(satpls_boot$boot$paths,"all")

## End(Not run)
```

`find_paths_cal` *real paths calculation est*

Description

real paths calculation est

Usage

```
find_paths_cal(paste_path_result, type = "all", digits = 3, est = "Original")
```

Arguments

`paste_path_result`	find_paths data
`type`	all=res, ind, est
`digits`	round 3
`est`	Original or Mean.Boot

Value

data

Examples

```
## Not run:
#'

# Example usage
sdata <- data.frame(
  paths = c("IMAG -> EXPE", "IMAG -> SAT", "IMAG -> LOY",
   "EXPE -> QUAL", "EXPE -> VAL", "EXPE -> SAT",
            "QUAL -> VAL", "QUAL -> SAT",
            "VAL -> SAT", "SAT -> LOY"),
  Original = c(0.578959128, 0.200724200, 0.275149576,
   0.848344408, 0.105477650, -0.002753995,
              0.676655529, 0.122144566, 0.589330650, 0.495479322),
  Mean.Boot = c(0.58232765, 0.21209094, 0.27486564,
  0.84793899, 0.10992051, -0.01008222,
               0.67780527, 0.15132543, 0.56312717, 0.50757755),
```

```
  Std.Error = c(0.04358293, 0.06056988, 0.07729000,
  0.01864573, 0.06968826, 0.06507903,
               0.07760527, 0.08440236, 0.08723293, 0.07856694),
  perc.025 = c(0.497644719, 0.107561558, 0.140641502,
   0.815794494, -0.025566471, -0.147788188,
              0.519959683, -0.008629516, 0.392664443, 0.351774130),
  perc.975 = c(0.6646198, 0.3275312, 0.4161131,
   0.8828875, 0.2545785, 0.1086856,
              0.8166486, 0.2991045, 0.7012494, 0.6556333)
)

#'

##realdata analysis
data(satisfaction)
sat_path = plspm_paths(
  row_names = c("IMAG","EXPE","QUAL","VAL","SAT","LOY"),
  relationship = list(
    path(from="IMAG", to=c("EXPE","SAT","LOY")),
    path("EXPE", c("QUAL","VAL","SAT")),
    path("QUAL", c("VAL","SAT")),
    path("VAL",c("SAT")),
    path("SAT","LOY"))
)
sat_path

# blokcs
sat_blocks1 <- plspm_blocks(
  IMAG = item(paste0("imag",1:5)),
  EXPE = item(paste0("expe", 1:5)),
  QUAL = item( paste0("qual", 1:5)),
  VAL = item(paste0("val", 1:4)),
  SAT = item(paste0("sat", 1:4)),
  LOY = item(paste0("loy", 1:4))
)

sat_blocks1

satpls_boot = plspm_sem(satisfaction, sat_path, sat_blocks1, scaled = FALSE,
                        boot.val =TRUE, br=100)
#' find_paths(satpls_boot)
find_paths_cal(find_paths(satpls_boot,"all") )

#값을 계산하여 보기
find_paths(jutpls_boot,"all")%>%find_paths_cal()
#Mean.Boot자료 보기
find_paths(jutpls_boot,"all", est="Mean.Boot")%>%find_paths_cal()




## End(Not run)
```

`format_col` *select_format*

Description

select_format

Usage

```
format_col(data, ..., nsmall = 3, digits = 3)
```

Arguments

`data`	data
`...`	col selection
`nsmall`	default 3
`digits`	round 3

Value

data

Examples

```
## Not run:


# Usage example
data2 <- data.frame(a = rnorm(5), b = rnorm(5), c = rnorm(5), d = rnorm(5), e = rnorm(5))
format_col(data2, 1, 2)
format_col(data2, 1:5)
format_col(data2, a)
format_col(data2, a:e)
format_col(data2, "a")

## End(Not run)
```

`format_number` *Define a function: Convert numbers to scientific notation or common number*

Description

Define a function: Convert numbers to scientific notation or common number

Usage

```
format_number(number, n1 = 8, n2 = 3, n3 = 5)
```

Arguments

`number`	number
`n1`	length
`n2`	scientfic
`n3`	scientfic

Value

formatting

Examples

```
## Not run:
## number of data
nchar(sub("\\d+\\.", "", as.character(0.123456)))

numbers <- c(0.123456, 123.456789, 0.0000123456)
sapply(numbers, format_number)

for (i in seq_along(numbers)) {
   cat("Original:", numbers[i], "Formatted:", formatted_numbers[i], "\n")}

format_number(0.123456)


## End(Not run)
```

`Freq_rowcounter` *Freq_rowcounter*

Description

Freq_rowcounter

Usage

```
Freq_rowcounter(df, type = "counter")
```

Arguments

`df`	df
`type`	counter, term, freq_counter, freq_term

Value

df

Examples

```
## Not run:

 Freq_rowcounter(df = eduteck2, type = "counter")
Freq_rowcounter(df = eduteck2, type = "freq_counter")

## End(Not run)
```

`Freq_table` *Frequency analysis table*

Description

Frequency analysis table

Usage

```
Freq_table(
  data,
  ...,
  prop = FALSE,
  plot = FALSE,
  angle = 0,
  size_text = 4,
  size_axis = 10,
  vjust = -0.05,
  hjust = 0.5,
  yadd = 100,
  legend.position = "",
  reorder = FALSE,
  wt = NULL,
  na.rm = TRUE,
  na_treatment = "무응답",
  remove_rows = NULL,
  add_row_sum = FALSE,
  xlab = "Category",
  flip = FALSE,
  size_axis_title = 12,
  sort_bar = TRUE,
  type = "res"
)
```

Arguments

`data`	data.frame
`...`	data.frame col names
`prop`	proportion value, default TRUE
`plot`	plot defualt false
`angle`	angle = 0
`size_text`	default 3, bargraph text

size_axis	default 12, axis test
vjust	default -0.5
hjust	default -0.5
yadd	default 0.8
legend.position	legend.position="" none. top, bottom, left, right
reorder	reorder Sorting Graph Arrays
wt	weight
na.rm	TRUE
na_treatment	" 무응답", # 결측값대체값
remove_rows	Enter the number of rows to hide and they will be removed
add_row_sum	default FALSE, Sum excluding removed rows
xlab	xalb = "Category",
flip	FALSE
size_axis_title	default 12,
sort_bar	TRUE(default)
type	default "res", all, res=data, g=plot=graph

Value

Freqency table

Examples

```
## Not run:
#사용방법
eduteck_s%>% Freq_table("Q9", prop=TRUE, add_row_sum = TRUE)

eduteck_s$Q9 %>% Freq_table(prop=TRUE, add_row_sum = TRUE, na.rm= TRUE)
eduteck_s$Q9 %>% Freq_table(prop=TRUE, add_row_sum = TRUE, na.rm=FALSE)

Mtcars = mtcars
Mtcars$am = factor(Mtcars$am, levels=c(0,1), labels= c("automatic","manual" ))
Mtcars$vs  = factor(Mtcars$vs, levels=c(0,1), labels= c("V-shaped","straight" ))
Mtcars$cyl  = factor(Mtcars$cyl, levels=c(4,6,8), labels= c("cyl-4","cyl-6","cyl-8"))

#vector variables
Freq_table(c("a","b","c","a","b","a","c","c","c"))
Freq_table(c("a","b","c","a","b","a","c","c","c"), prop=TRUE)
Freq_table(c("a","b","c","a","b","a","c","c","c"), plot=T)


Mtcars %>% Freq_table("am")
Mtcars %>% Freq_table("am", prop=F)
Mtcars %>% Freq_table("vs", plot=FALSE)

Mtcars %>% Freq_table("vs")
Mtcars %>% Freq_table("vs","am")

Mtcars %>% Freq_table("vs","am","cyl", type="all")
```

```
#graph
Mtcars %>% Freq_table("vs","am","cyl", type="g")

##only graph
Mtcars %>% Freq_table("vs","am","cyl", angle=90, type="g")
Mtcars %>% Freq_table("vs","am","cyl", angle=90, reorder = TRUE, type="g")

Mtcars %>% Freq_table("vs","am","cyl", angle=90, type="g")+ylim(0,14)
Mtcars %>% Freq_table("vs","am","cyl", angle=90, reorder = TRUE, type="g")+ylim(0,14)

Mtcars %>% Freq_table("vs","am") %>% arrange(am)
Mtcars %>% Freq_table("vs","am") %>% arrange(vs)
Mtcars %>% Freq_table("vs","am") %>% arrange(Freq)

##when the data has already been aggregated once
df_sample <- tribble(
  ~name,    ~gender,   ~ sum,
  "Max",    "male",       10,
  "Sandra", "female",      1,
  "Susan",  "female",      4)
df_sample

#weight frequency
df_sample %>% unCount(sel = "sum") %>% Freq_table("gender")

# this function
df_sample %>% Freq_table("gender", wt="sum")
#'
# 데이터 생성
df <- data.frame(
  term = c("", "10-20분", "20-30분", "30분 이상", "5-10분", "5분 이내"),
  Freq = c(728, 222, 77, 120, 319, 135)
)

# 기본 그래프
Freq_table(df, term, prop = TRUE, remove_rows = 1, add_row_sum = FALSE, plot = TRUE)

# xlab과 flip 옵션 사용
Freq_table(df, term, prop = TRUE, remove_rows = 1, add_row_sum = TRUE, xlab = "응답 시간", flip = TRU
#'

#빈도분석 및 그래프
Freq_table(eduteck2$Q9, prop=TRUE, remove_rows=1, add_row_sum=TRUE, vjust=-0.5,
           plot=T, size_axis = 14, flip=FALSE, yadd=130, xlab="Eduteck Time", size_text = 6) #'

## End(Not run)
```

`Freq_table_apa` *Interpreting frequency analysis results from Freq_table*

Description

Interpreting frequency analysis results from Freq_table

Usage

```
Freq_table_apa(df, title = "", print = TRUE, digits = 2)
```

Arguments

df	data
title	title =””
print	df output
digits	2

Value

apa

Examples

```
## Not run:
#'
Mtcars = mtcars

Mtcars$am = factor(Mtcars$am, levels=c(0,1), labels= c("automatic","manual" ))
Mtcars$vs  = factor(Mtcars$vs, levels=c(0,1), labels= c("V-shaped","straight" ))
Mtcars$cyl  = factor(Mtcars$cyl, levels=c(4,6,8), labels= c("cyl-4","cyl-6","cyl-8"))

Mtcars %>% Freq_table("am","vs", prop=T) %>% Freq_table_apa()
Mtcars %>% table_freq("vs","am","cyl") %>% Freq_table_apa()
Mtcars %>% table_freq("vs","am","cyl", angle=90)%>% Freq_table_apa()

## End(Not run)
```

Freq_table_colnames *Freq_table_colnames caange korean names*

Description

Freq_table_colnames caange korean names

table_df_colnames

Usage

```
Freq_table_colnames(df, input = "구분")

Freq_table_colnames(df, input = "구분")
```

Arguments

df	df
input	” 구분

Value

df

df

Examples

```
## Not run:
#' table_df_multit(eduteck2 %>% select(contains("Q5_"), -Q5_4_TEXT)) %>%
arrange(desc(Freq)) %>%Freq_table_colnames()

## End(Not run)

## Not run:



## End(Not run)
```

`Freq_table_multi` *Create a Frequency Table from Delimited Multiple Responses*

Description

This function creates a frequency table from a column containing delimited multiple responses. It accepts either a data frame with a specified column or a simple character vector.

Usage

```
Freq_table_multi(df, col = 1, sep = ",\\s*")
```

Arguments

`df`	A data frame or a character vector. If a vector is provided, it is converted to a one-column data frame named `response`.
`col`	Column name or index to be processed when `df` is a data frame. Defaults to `1`.
`sep`	A regular expression string for splitting responses. Defaults to `",\s*"` (comma followed by optional spaces).

Value

A data frame with two columns: `item` and `n`, where `item` is the response item and `n` is the count.

Examples

```
# Example 1: Using a character vector
responses <- c("apple, banana", "banana, apple", "banana", "orange, apple")
Freq_table_multi(responses)

# Example 2: Using a data frame column with different separator
df <- data.frame(Q1 = c("cat / dog", "dog", "cat / bird", "bird / dog"))
Freq_table_multi(df, col = "Q1", sep = "\\s*/\\s*")
```

`fs` *Add factor score calculation functions and data*

Description

Add factor score calculation functions and data

Usage

```
fs(data, range = 1:ncol(data), nfactor = NULL, prefix = "", rotate = "varimax")
```

Arguments

`data`	data
`range`	range default 1:ncol(data)
`nfactor`	number of factor
`prefix`	data prefic
`rotate`	varimax

Value

data

Examples

```
## Not run:
#'
# 활용방법 1
factor_scores(mtcars,
              range = c("disp", "hp", "drat", "wt", "qsec"),
              nfactor = 2, prefix="a")

mtcars %>% select("disp", "hp", "drat", "wt", "qsec") %>% factor_scores(nfactor=2)

# 활용방법 2
factor_scores(mtcars, range = c(3:7), nfactor = 2)

factor_scores(mtcars, c("disp", "hp", "drat", "wt", "qsec"), nfactor = 2)
factor_scores(mtcars, c(3, 4, 5, 6, 7), nfactor = 2)

factor_scores(mtcars, range = "disp:qsec", nfactor = 2)


mtcars %>% select("disp", "hp", "drat", "wt", "qsec") %>%
  factor_scores(c("disp","hp"),nfactor=2, prefix="a") %>%
  factor_scores(c("drat", "wt", "qsec"),nfactor=3, prefix="b") %>%head()


## End(Not run)
```

g *Googole Translate korean to english*

Description

Googole Translate korean to english

Usage

```
g(text = "hi.", slang = "ko", elang = "en", show = "translate")
```

Arguments

text	input your text
slang	start language. default Korean
elang	translated language. default English
show	show Methode. 'translate' is console, 'browser' is Print to your computer's browser, 'all' are There are two ways to output it. 'data' is char data

Examples

```
# example code
## Not run:
#korean to english : default
g("안녕하세요 번역을 시작합니다!(by 박중희).", slang ="ko", elang = "en")
g("안녕하세요 번역을 시작합니다!(by 박중희).", "ko", "en")
# english to korean
g("Hello, let's start translating! (by Park Joong-hee", slang ="ko", elang = "en")
g("Hello, let's start translating! (by Park Joong-hee","ko","en")
#japan to korean
g("こんにちは翻□を始めます！(byパク□ジュンヒ", slang ="ja", elang = "ko")

# ko: korean , en: English , ja: Japan, china : zh
# https://cloud.google.com/translate/docs/languages?hl=ko

## End(Not run)
```

general_Sobel_test *Sobeltest toal*

Description

Sobeltest toal

Usage

```
general_Sobel_test(a, SE, roburst = FALSE, n = NULL, k = 2, digits = 5)
```

Arguments

a	est
SE	se
roburst	roburst T whide correction
n	sample size
k	number of regressin coefs, default 2
digits	5
conf	0.90, 0.95, 0.99, 0.999
type	all, res, sobel, aroian, goodman

Value

result

Examples

```
## Not run:
#'
# Example usage
a <- 0.5  # Example coefficient a
b <- 0.3  # Example coefficient b
sa <- 0.1 # Standard error of a
sb <- 0.05 # Standard error of b

Sobel_test(a, b, sa, sb)
Sobel_test(a, b, sa, sb, "all")
Sobel_test(a, b, sa, sb, "Sobel")

Sobel_test(a, b, sa, sb,"aroian")

Sobel_test(a, b, sa, sb,"Goodman")
#'
a <- c(0.5, 0.3, 0.4)
SE <- c(0.1, 0.1, 0.1)
general_Sobel_test(a, SE)
general_Sobel_test(a, SE)%>%unite_ci(digits=4)



## End(Not run)
```

GetCov *text to get covariance matrix*

Description

text to get covariance matrix

Usage

```
GetCov(
  x,
  lower = TRUE,
  diagonal = TRUE,
  sds = NULL,
  names = paste("V", 1:nvar, sep = "")
)
```

Arguments

x	text
lower	True
diagonal	TRUE
sds	NULL if cor is need
names	mat colnames,

Value

matrix

Examples

```
## Not run:
#'
library(jjstat)

lower <- '
 11.834
  6.947   9.364
  6.819   5.091  12.532
  4.783   5.028   7.495   9.986
 -3.839  -3.889  -3.841  -3.625  9.610
-21.899 -18.831 -21.748 -18.775 35.522 450.288 '

GetCov(lower)
#' GetCov(lower, names = c("anomia67", "powerless67", "anomia71", "powerless71","education", "sei"))

wheaton.cov <-
  GetCov(lower, names = c("anomia67", "powerless67",
                          "anomia71", "powerless71",
                          "education", "sei"))
wheaton.cov
wheaton.model <- '
  # latent variables
    ses     =~ education + sei
    alien67 =~ anomia67 + powerless67
    alien71 =~ anomia71 + powerless71
  # regressions
    alien71 ~ alien67 + ses
    alien67 ~ ses
  # correlated residuals
    anomia67 ~~ anomia71
```

```
    powerless67 ~~ powerless71
'
fit <- sem(wheaton.model,
           sample.cov = wheaton.cov,
           sample.nobs = 932)
summary(fit, standardized = TRUE)
diagram2(fit, curve=2)

## End(Not run)
```

`goodman_sobel_test_general` *Goodman Sobel Test generalized function*

Description

Goodman Sobel Test generalized function

Usage

```
goodman_sobel_test_general(
  a,
  SE,
  roburst = FALSE,
  n = NULL,
  k = 2,
  digits = 5,
  conf = 0.95
)
```

Arguments

`a`	est
`SE`	se
`roburst`	roburst T whide correction
`n`	sample size
`k`	number of regressin coefs, default 2
`digits`	5
`conf`	0.90, 0.95, 0.99, 0.999

Value

result

Examples

```
## Not run:
#'
# Example usage
a <- 0.5  # Example coefficient a
b <- 0.3  # Example coefficient b
sa <- 0.1 # Standard error of a
sb <- 0.05 # Standard error of b

Sobel_test(a, b, sa, sb)
Sobel_test(a, b, sa, sb, "all")
Sobel_test(a, b, sa, sb, "Sobel")

Sobel_test(a, b, sa, sb,"Aroian")

Sobel_test(a, b, sa, sb,"Goodman")
#'
# 예제 데이터
a <- c(0.5, 0.3, 0.4)
SE <- c(0.1, 0.1, 0.1)

# 함수 실행 (기본 95% 신뢰구간)
goodman_sobel_test_general(a, SE)

# 함수 실행 (90% 신뢰구간)
result_90 <- goodman_sobel_test_general(a, SE, conf=0.90)
print(result_90)

# 함수 실행 (99% 신뢰구간)
result_99 <- goodman_sobel_test_general(a, SE, conf=0.99)
print(result_99)

# 함수 실행 (99.9% 신뢰구간)
result_999 <- goodman_sobel_test_general(a, SE, conf=0.999)
print(result_999)
#'


## End(Not run)
```

`gtl` *Googole Translate korean to english*

Description

Googole Translate korean to english

Usage

```
gtl(text = "hi.", slang = "ko", elang = "en", show = "data")
```

Arguments

text	input your text
slang	start language. default Korean
elang	translated language. default English
show	show Methode. 'translate' is console, 'browser' is Print to your computer's browser, 'all' are There are two ways to output it. 'data' is char data

Examples

```
# example code
## Not run:
#korean to english : default
g("안녕하세요 번역을 시작합니다!(by 박중희).", slang ="ko", elang = "en")
g("안녕하세요 번역을 시작합니다!(by 박중희).", "ko", "en")
# english to korean
g("Hello, let's start translating! (by Park Joong-hee", slang ="ko", elang = "en")
g("Hello, let's start translating! (by Park Joong-hee","ko","en")
#japan to korean
g("こんにちは翻□を始めます！(byパク□ジュンヒ", slang ="ja", elang = "ko")

# ko: korean , en: English , ja: Japan, china : zh
# https://cloud.google.com/translate/docs/languages?hl=ko

## End(Not run)
```

---

gtl_df *Google translate data.frame #gtl_df() dataframe* 번역 ———- # 영어논문을만들때, 표에들어가는단어를번역하여넣기 # 너무많은수는오래걸림-> 웹페이지를갔다와야하므로... *Title*

Description

Google translate data.frame #gtl_df() dataframe 번역 ———- # 영어논문을만들때, 표에들어가는단어를번역하여넣기 # 너무많은수는오래걸림-> 웹페이지를갔다와야하므로... Title

Usage

```
gtl_df(df, col = 1)
```

Arguments

df	data.frame
col	Select the column you want to translate

Examples

```
## Not run:
 df0 <-  data.frame(word=c("온라인","진행","교육"),
                    n=c(100, 200, 300))
 df0 %>% gtl_df()
```

```
## End(Not run)
```

`hist2` *histogram*

Description

histogram

Usage

```
hist2(data, color = "lightblue", linecolor = "red")
```

Arguments

`data`	vector
`color`	fill color
`linecolor`	line color

Value

graph histogram

Examples

```
 ## Not run:

set.seed(123)

data <- rnorm(100)

hist2(data)


## End(Not run)
```

`histogram` *histogram*

Description

histogram

Usage

```
histogram(data, color = "gray60")
```

Arguments

`data`	vector
`color`	color

Value

plot

Examples

```
## Not run:
set.seed(123)
data <- rnorm(100)
histogram(data)


## End(Not run)
```

`HTMT` *HTMT for Single-level SEM/CFA (lavaan)*

Description

This function computes the Heterotrait–Monotrait ratio (HTMT) for single-level CFA/SEM models fitted with `lavaan`. The output is a lower-triangular HTMT matrix (upper triangle and diagonal are NA).

Usage

```
HTMT(fit, digits = 3)
```

Arguments

`fit`	A `lavaan` cfa/sem fitted object.
`digits`	Integer. Number of digits to round the HTMT values. Default is 3.

Details

HTMT is computed using observed-variable correlations from `lavaan::lavInspect(fit, "cor.ov")`. Only latent variables with at least two indicators are included.

The HTMT for latent variables i and j is computed as: HTMT_ij = mean(|r_xy|) / mean(|r_xx|) * mean(|r_yy|) over 2 where r_xy are heterotrait correlations (indicators across constructs), and r_xx, r_yy are monotrait correlations (within-construct indicators).

Value

A data.frame containing the HTMT matrix (lower triangle only).

References

Henseler, J., Ringle, C. M., & Sarstedt, M. (2015). A new criterion for assessing discriminant validity in variance-based structural equation modeling. Journal of the Academy of Marketing Science, 43(1), 115–135.

Examples

```
## Not run:
HS.model <- '
  visual  =~ x1 + x2 + x3
  textual =~ x4 + x5 + x6
  speed   =~ x7 + x8 + x9
'
fit <- lavaan::cfa(HS.model, data = lavaan::HolzingerSwineford1939)

HTMT(fit)
HTMT(fit, digits = 4)

## End(Not run)
```

---

htmt_mlsem *HTMT for Multilevel SEM*

Description

This function computes the Heterotrait–Monotrait ratio (HTMT) for multilevel CFA/SEM models estimated using lavaan. HTMT is calculated separately for within-level or between-level measurement models based on observed-variable correlations.

Usage

```
htmt_mlsem(fit, level = c("within", "between"), digits = 3)
```

Arguments

`fit`	A lavaan object fitted with a multilevel CFA/SEM model.
`level`	Character. One of `"within"` or `"between"`.
`digits`	Integer. Number of digits to round the HTMT values.

Details

HTMT is defined as the ratio of the mean of heterotrait–heteromethod correlations to the geometric mean of monotrait–heteromethod correlations.

Only latent variables with at least two indicators are included. The function uses level-specific observed-variable correlation matrices extracted via `lavInspect(fit, "cor.ov")`.

Value

A data.frame containing the HTMT matrix (lower triangle only; diagonal and upper triangle are NA).

References

Henseler, J., Ringle, C. M., & Sarstedt, M. (2015). A new criterion for assessing discriminant validity in variance-based structural equation modeling. Journal of the Academy of Marketing Science, 43(1), 115–135.

Examples

```
## Not run:
fit <- lavaan::sem(model, data = Demo.twolevel, cluster = "cluster")

mlsem_htmt(fit, level = "within")
mlsem_htmt(fit, level = "between")

## End(Not run)
```

---

htmt_sem *HTMT for Single-level SEM/CFA (lavaan)*

Description

This function computes the Heterotrait–Monotrait ratio (HTMT) for single-level CFA/SEM models fitted with `lavaan`. The output is a lower-triangular HTMT matrix (upper triangle and diagonal are NA).

Usage

```
htmt_sem(fit, digits = 3)
```

Arguments

`fit`	A `lavaan` cfa/sem fitted object.
`digits`	Integer. Number of digits to round the HTMT values. Default is 3.

Details

HTMT is computed using observed-variable correlations from `lavaan::lavInspect(fit, "cor.ov")`. Only latent variables with at least two indicators are included.

The HTMT for latent variables i and j is computed as: HTMT_ij = mean(|r_xy|) / mean(|r_xx|) * mean(|r_yy|) over 2 where r_xy are heterotrait correlations (indicators across constructs), and r_xx, r_yy are monotrait correlations (within-construct indicators).

Value

A data.frame containing the HTMT matrix (lower triangle only).

References

Henseler, J., Ringle, C. M., & Sarstedt, M. (2015). A new criterion for assessing discriminant validity in variance-based structural equation modeling. Journal of the Academy of Marketing Science, 43(1), 115–135.

Examples

```
## Not run:
HS.model <- '
  visual  =~ x1 + x2 + x3
  textual =~ x4 + x5 + x6
  speed   =~ x7 + x8 + x9
'
fit <- lavaan::cfa(HS.model, data = lavaan::HolzingerSwineford1939)

htmt_sem(fit)
htmt_sem(fit, digits = 4)

## End(Not run)
```

`ICC` *ICC*

Description

ICC

Usage

```
ICC(lmedata, type = "mixed", digits = 4)
```

Arguments

`lmedata`	lmer result
`type`	'mixed', 'logit', 'pois'
`digits`	digits = 3 default

Examples

```
## Not run:
data(sleepstudy)
sleepstudy |> str()
lmer(Reaction ~ Days + (Days | Subject), sleepstudy) |> ICC()


## End(Not run)
```

icc_batch *Batch calculation of ICCs for multiple variables*

Description

This function applies `icc_lme4()` to multiple outcome variables and returns a combined table of ICC results.

Usage

```
icc_batch(data, vars, cluster)
```

Arguments

`data`	data.frame. Dataset containing outcome variables and cluster variable.
`vars`	character vector. Names of numeric outcome variables.
`cluster`	character string. Name of the clustering variable.

Value

A tibble with ICC results for each variable.

Examples

```
## Not run:
# Built-in dataset
data(mtcars)

# Single ICC calculation
icc_lme4(
  y = mtcars$hp,
  cluster = mtcars$cyl
)

# Batch ICC calculation
icc_batch(
  data = mtcars,
  vars = c("mpg", "hp", "wt"),
  cluster = "cyl"
)

## End(Not run)
```

icc_lme4 *Calculate Intraclass Correlation Coefficient (ICC) using lme4*

Description

This function computes the Intraclass Correlation Coefficient (ICC) based on a random-intercept-only linear mixed-effects model.

Usage

```
icc_lme4(y, cluster)
```

Arguments

`y`	numeric vector. Outcome variable.
`cluster`	vector or factor. Grouping (cluster) variable.

Value

A data.frame with the following components:

ICC Intraclass correlation coefficient

between_var Between-cluster variance (tau^2)

within_var Within-cluster variance (sigma^2)

J Number of clusters

kbar Average cluster size

Examples

```
## Not run:

# Example using a built-in R dataset
data(ToothGrowth)

# ICC of tooth length clustered by supplement type
icc_lme4(
  y = ToothGrowth$len,
  cluster = ToothGrowth$supp
)

# ICC of tooth length clustered by dose
icc_lme4(
  y = ToothGrowth$len,
  cluster = ToothGrowth$dose


# Built-in dataset
data(mtcars)

# Single ICC calculation
icc_lme4(
  y = mtcars$mpg,
  cluster = mtcars$cyl
)

# Batch ICC calculation using icc_batch()
icc_batch(
  data = mtcars,
  vars = c("mpg", "hp", "wt"),
  cluster = "cyl"
)

## End(Not run)
```

IE_effect *SEM indirect effect*

Description

SEM indirect effect

Usage

```
IE_effect(model_sem, effect1 = "IE", effect2 = "IE", label = FALSE, ci = TRUE)
```

Arguments

model_sem	lavaan objgect
effect1	”IE”
effect2	”IE”
label	label
ci	confidence interval

Value

parameterEstimates()

Examples

```
## Not run:
#library(lavaan) # only needed once per session

models1 <- '
  # measurement model
    ind60 =~ x1 + x2 + x3
    dem60 =~ y1 + y2 + y3 + y4
    dem65 =~ y5 + y6 + y7 + y8
    # regressions
    dem60 ~ ind60
    dem65 ~ ind60 + dem60
      # residual correlations
    y1 ~~ y5
    y2 ~~ y4 + y6
    y3 ~~ y7
    y4 ~~ y8
    y6 ~~ y8

models2 ='
  # measurement model
    ind60 =~ x1 + x2 + x3
    dem60 =~ y1 + y2 + y3 + y4
    dem65 =~ y5 + y6 + y7 + y8
     # regressions
    dem60 ~ a1*ind60
    dem65 ~ a2*ind60 + a3*dem60
 #NEw
```

```
    ie := a1*a3
    te := a2 +a1*a3
    diff := a1 - a3



fits1 <- sem(models1, data=PoliticalDemocracy)
fits2 <- sem(models2, data=PoliticalDemocracy)
summary(fits1, standardized=TRUE)
summary(fits2, standardized=TRUE)
fits2 %>% IE_effect()

## End(Not run)
```

`indicator_validity_ml` *Indicator validity table for (Multi-level) CFA*

Description

Indicator validity table for (Multi-level) CFA

Usage

```
indicator_validity_ml(fit, level = c("within", "between"), digits = 3)
```

Arguments

`fit`	lavaan SEM / ML-SEM object
`level`	"within" or "between"
`digits`	numeric

Value

data.frame

`interlist` *Functions that compensate for the method of input*

Description

Functions that compensate for the method of input

Usage

```
interlist(...)
```

Arguments

`...`	LONG LIST

Value

list

Examples

```
## Not run:


# want list
intbasic = list(c("IMAG", 1, 2), c("SAT", 1, 2))
intbasic

inta = interlist(list(c("IMAG", 1, 2, 3), c("SAT", 1, 2)));inta
intc = interlist(list("IMAG", 1, 2, 3, "SAT", 1, 2));intc

intb = interlist(c("IMAG", 1, 2, 3), c("SAT", 1, 2));intb
intd = interlist("IMAG", 1, 2, 3, "SAT", 1, 2);intd
#'

# Partially select to create an interaction term
satisfaction%>% plspm_prod(sat_blocks1,
                           list(c("IMAG", 1, 2, 3), c("SAT", 1, 2, 3))) %>%str()

satisfaction%>% plspm_prod(sat_blocks1, inta) %>%str()
satisfaction%>% plspm_prod(sat_blocks1, intb) %>%str()
satisfaction%>% plspm_prod(sat_blocks1, intc) %>%str()



satisfaction%>%
plspm_prod(sat_blocks1, interlist(list("IMAG", 1, 2, 3, "SAT", 1, 2))) %>%str()
satisfaction%>%
plspm_prod(sat_blocks1, interlist(list("IMAG", 1, "SAT", 1, 2))) %>%str()
satisfaction%>%
plspm_prod(sat_blocks1, interlist(list("IMAG", 1,3,  "SAT", 2, 3))) %>%str()




## End(Not run)
```

---

interplot *interplot is using data*

Description

interplot is using data

Usage

```
interplot(
  data_long,
  type = "plot",
  size_point = 4,
  size_text = 5,
  linewidth = 1,
  vjust = -0.4,
  hjust = -2,
  title = NULL,
  xlab = "x_variable",
  ylab = "y_variable",
  legend_title = "v2",
  yadd = 0.5,
  size_element = 12,
  t = FALSE,
  xy = FALSE,
  digits = 3
)
```

Arguments

`data_long`	long foramt data
`size_point`	point size default 4
`size_text`	text size default 5
`linewidth`	line width default 1
`vjust`	vjust
`hjust`	hjust
`title`	title null
`xlab`	xlab
`ylab`	ylab
`legend_title`	Change legend_title
`yadd`	fig size add value
`size_element`	axis element size
`t`	matrix transpose efault FALSE, use when matrix data is wide format
`xy`	plot ranspose default FALSE
`digits`	default 3

Value

plot

Examples

```
## Not run:

## interplot_data use!!
interplot_data( 6, 1, 4, 5, type = "long")%>%
interplot(size_element = 10)
```

```
interplot_data( 2, 1, 4, 5, type="long")%>%interplot()

## 2 *3 matriix
interplot_data( 4, 1, 2, 5, 6, 3, type="long", row=3, col=2)%>%interplot()

##3*2
interplot_data( 4, 1, 2, 5, 6, 3, type="long", row=2, col=3)%>%interplot()

##all data output
interplot_data( 4, 1, 2, 5, 6, 3, type="long", row=2, col=3)%>%interplot("all")

#generated direct data matrix and data frame
matrix(c(4, 1, 2, 5, 6, 3), nrow=2,ncol=3)  #marix

#long format
matrix(c(4, 1, 2, 5, 6, 3), nrow=2,ncol=3) %>%
data.frame() %>% rownames_to_column("v1") %>%
pivot_longer(cols=2:4)

##plot
matrix(c(4, 1, 2, 5, 6, 3), nrow=2,ncol=3) %>%
 data.frame() %>%
rownames_to_column("v1") %>% pivot_longer(cols=2:4)%>%
interplot()

## onestep method when use this function
matrix(c(4, 1, 2, 5, 6, 3), nrow=2, ncol=3) %>% interplot(t=TRUE)


#  3 * 2
matrix(c(4, 1, 2, 5, 6, 3), nrow=3, ncol=2)

matrix(c(4, 1, 2, 5, 6, 3), nrow=3, ncol=2) %>%
data.frame() %>%
rownames_to_column("v1") %>%
pivot_longer(cols=2:3)

matrix(c(4, 1, 2, 5, 6, 3), nrow=3, ncol=2) %>%
 data.frame() %>%
 rownames_to_column("v1") %>% p
 ivot_longer(cols=2:3)%>%
 interplot()


##one step output
matrix(c(4, 1, 2, 5, 6, 3), nrow=3, ncol=2) %>%
interplot(t=TRUE)


#### all data output
matrix(c(4, 1, 2, 5, 6, 3), nrow=3, ncol=2) %>% interplot(t=TRUE, type="all")


## End(Not run)
```

`interplot_data` *interplot_data is matrix, long_data, plotting function*

Description

interplot_data is matrix, long_data, plotting function

Usage

```
interplot_data(
  ...,
  col = 2,
  row = 2,
  type = "long",
  title = NULL,
  xlab = "x",
  ylab = "y",
  vjust = -0.4,
  hjust = 0.5,
  size_element = 12,
  byrow = TRUE,
  xy = FALSE,
  yadd = 0.3
)
```

Arguments

`...`	input data c() or as select()
`col`	matrix ncol setting
`row`	matrix nrow
`type`	type is mat, long, all, plot
`title`	plot title
`xlab`	plot xlab
`ylab`	plot ylab
`vjust`	vjust text in plot postion
`hjust`	hjust text in plot position
`size_element`	plot element text size
`byrow`	byrow=TRUE is default
`xy`	plot transpose default FALSE
`yadd`	plot range add

Value

data and plot

Examples

```
## Not run:
 ## data output
interplot_data( 6, 1, 4, 5, type = "mat")
#' interplot_data( 6, 1, 4, 5, type = "all")
interplot_data( 6, 1, 4, 5, type = "long")
interplot_data( 6, 1, 4, 5, type = "plot")

# 2*3 data
interplot_data( c(6, 1, 4, 5,6,7), type = "mat",  col = 3,row = 2)
interplot_data( c(6, 1, 4, 5,6,7), type = "long",  col = 3,row = 2)
interplot_data( 6, 1, 4, 5,6,7, type = "all",  col = 3,row = 2)
interplot_data( 6, 1, 4, 5,6,7, type = "plot",  col = 3,row = 2)

## 3*2 data
interplot_data( 6, 1, 4, 5,6,7, type = "mat",  col = 2,row = 3)
interplot_data( 6, 1, 4, 5,6,7, type = "all",  col = 2,row = 3)
interplot_data( 6, 1, 4, 5,6,7, type = "long",  col = 2, row = 3)
interplot_data( c(6, 1, 4, 5,6,7), type = "plot",  col = 2,row = 3)

### ii is very hard example
interplot_data( 6, 1, 4, 5) %>%
ggplot(aes(x = v1, y = value))+
geom_point(size=4, aes(color=v2))+
geom_line(aes(group = v2, linetype = v2 ), linewidth = 1)+
geom_text(aes(label = value), vjust= -0.4, hjust= 2, size= 5)+
theme_bw()


## End(Not run)
```

---

interpretation_de *SEM direct effect interpretation*

Description

SEM direct effect interpretation

Usage

```
interpretation_de(data, md = FALSE)
```

Arguments

data	DE_effect obj
md	markdown_table output

Value

report apa

Examples

```
## Not run:

 #library(lavaan) # only needed once per session
 models <- '
   # measurement model
     ind60 =~ x1 + x2 + x3
     dem60 =~ y1 + y2 + y3 + y4
     dem65 =~ y5 + y6 + y7 + y8
 # regressions
     dem60 ~ ind60
     dem65 ~ ind60 + dem60
   # residual correlations
     y1 ~~ y5
     y2 ~~ y4 + y6
     y3 ~~ y7
     y4 ~~ y8
    y6 ~~ y8
 '
 fits <- sem(models, data=PoliticalDemocracy)
 summary(fits, standardized=TRUE)

 ##Direct effect
 DE_effect(fits)
 DE_effect(fits)%>% interpretation_de()


## End(Not run)
```

`interpretation_ie` *interpretation_ie*

Description

interpretation_ie

Usage

```
interpretation_ie(
  data,
  msg = "IE",
  md = FALSE,
  est = 3,
  std = 4,
  sig = 7,
  p = 6
)
```

Arguments

`data`	data

msg	msg=”IE”
md	md = FALSE
est	est=3
std	std=4
sig	sig = 7
p	p=6

Value

ie report

Examples

```
## Not run:
example(sem)
IE_effect(fit)
IE_effect(fit) %>% interpretation_ie()

## End(Not run)
```

---

interpretation_ie_CI *sem_apa_ie_CI*

Description

sem_apa_ie_CI

Usage

```
interpretation_ie_CI(
  data,
  caption = "매개효과 분석 결과 ",
  md = FALSE,
  est = 2,
  std = 3,
  sig = 6,
  CI = 7,
  Z = FALSE,
  tot_filter = "총",
  ind_filter = "IE"
)
```

Arguments

data	lavaan object
caption	” 매개효과분석결과”,
md	markdown
est	est sel 2
std	std sel 3

`sig`	sig sel 6
`CI`	CI seledct 7
`Z`	Z=FALSE
`tot_filter`	tot_filter=" 총",
`ind_filter`	ind_filter="IE"

Value

table

`item` *item*

Description

item

Usage

```
item(...)
```

Arguments

`...`	input

Value

list

`item_CR_AVE` *Item-level CR & AVE table*

Description

Item-level CR & AVE table

Usage

```
item_CR_AVE(fit, level = c("within", "between"), digits = 3)
```

Arguments

`fit`	lavaan (ML-)SEM object
`level`	"within" or "between"
`digits`	numeric

Value

data.frame

`item_parceling` *Item Parceling Utility Function*

Description

This function creates item parcels from observed item-level data using aggregation methods (mean, sum, weighted mean) or factor-score-based methods (EFA / CFA).

Usage

```
item_parceling(
  data,
  ...,
  fun = "mean",
  type = c("new", "data", "all"),
  weights = NULL,
  standardize_items = FALSE,
  min_valid = 1,
  factor_score = c("regression", "bartlett"),
  lavaan_estimator = "ML",
  std.lv = FALSE,
  name_in_vector = FALSE,
  verbose = FALSE
)
```

Arguments

`data`	A data.frame containing observed item variables.
`...`	Parcel definitions.
`fun`	Scoring method. Options include ”mean”, ”sum”, ”median”, ”wmean”, ”factor_efa”, ”fs_efa”, ”factor_cfa”, ”fs_cfa”. Abbreviated forms ”efa” and ”cfa” are allowed with a warning.
`type`	Output type: ”new” (parcel data only), ”data” (append parcels to original data), ”all” (data + metadata).
`weights`	Numeric vector or a named list of numeric vectors. For fun = ”wmean”, parcel-specific weights can be supplied as a list. Parcels without weights will be aggregated using mean().
`standardize_items`	Logical. If TRUE, items are standardized before scoring.
`min_valid`	Minimum number of valid items required per row.
`factor_score`	Factor score method for EFA (”regression”, ”bartlett”).
`lavaan_estimator`	Estimator for CFA.
`std.lv`	Logical. If TRUE, latent variance is fixed to 1 in CFA.
`name_in_vector`	Logical or ”auto”. - TRUE: c(”ParcelName”,”item1”,”item2”,...) - FALSE: ParcelName = c(”item1”,”item2”,...) (default) - ”auto”: if the first element is not a column name, treat it as parcel name.
`verbose`	Logical. If TRUE, prints reliability summaries.

Value

Depending on type, a data.frame or a list.

Examples

```
## ------------------------------------------------
## Example data
## ------------------------------------------------
set.seed(123)
shindata_example <- data.frame(
  efficacy2 = rnorm(100, 3.5, 0.8),
  efficacy3 = rnorm(100, 3.6, 0.7),
  efficacy4 = rnorm(100, 3.7, 0.6),

  Accessibility1 = rnorm(100, 4.0, 0.9),
  Accessibility2 = rnorm(100, 3.9, 0.8),
  Accessibility3 = rnorm(100, 4.1, 0.7),
  Accessibility4 = rnorm(100, 4.0, 0.6),
  Accessibility5 = rnorm(100, 3.8, 0.9),

  Interest1 = rnorm(100, 3.2, 0.7),
  Interest2 = rnorm(100, 3.3, 0.8),
  Interest3 = rnorm(100, 3.4, 0.6),
  Interest4 = rnorm(100, 3.5, 0.7)
)

## ------------------------------------------------
## Basic example: sum
## ------------------------------------------------
parcel_sum <- item_parceling(
  data = shindata_example,
  효능감 = c("efficacy2","efficacy3","efficacy4"),
  연구접근성 = c("Accessibility1","Accessibility2",
                 "Accessibility3","Accessibility4","Accessibility5"),
  fun = "sum",
  type = "new"
)
head(parcel_sum)

## ------------------------------------------------
## Weighted mean: single parcel
## ------------------------------------------------
item_parceling(
  data = shindata_example,
  효능감 = c("efficacy2","efficacy3","efficacy4"),
  fun = "wmean",
  weights = c(0.40, 0.35, 0.25),
  type = "new"
)

## ------------------------------------------------
## Weighted mean: parcel-specific weights (recommended)
## ------------------------------------------------
parcel_wmean <- item_parceling(
  data = shindata_example,
  효능감 = c("efficacy2","efficacy3","efficacy4"),
  연구접근성 = c("Accessibility1","Accessibility2",
```

```
                  "Accessibility3","Accessibility4","Accessibility5"),
  fun = "wmean",
  weights = list(
    효능감 = c(0.40, 0.35, 0.25)
  ),
  type = "new"
)
head(parcel_wmean)

## Not run:
## -------------------------------------------------
## Error example: invalid weights usage
## -------------------------------------------------
item_parceling(
  data = shindata_example,
  효능감 = c("efficacy2","efficacy3","efficacy4"),
  연구접근성 = c("Accessibility1","Accessibility2",
                  "Accessibility3","Accessibility4","Accessibility5"),
  fun = "wmean",
  weights = c(0.40, 0.35, 0.25)
)

## End(Not run)

## -------------------------------------------------
## Factor score (EFA)
## -------------------------------------------------
parcel_fs_efa <- item_parceling(
  data = shindata_example,
  효능감 = c("efficacy2","efficacy3","efficacy4"),
  연구접근성 = c("Accessibility1","Accessibility2",
                  "Accessibility3","Accessibility4","Accessibility5"),
  fun = "factor_efa",
  factor_score = "regression",
  type = "all"
)
parcel_fs_efa$score_info$효능감

## -------------------------------------------------
## EFA alias (warning example)
## -------------------------------------------------
item_parceling(
  data = shindata_example,
  효능감 = c("efficacy2","efficacy3","efficacy4"),
  fun = "efa",
  type = "new"
)

## -------------------------------------------------
## CFA factor score
## -------------------------------------------------
parcel_fs_cfa <- item_parceling(
  data = shindata_example,
  효능감 = c("efficacy2","efficacy3","efficacy4"),
  fun = "factor_cfa",
  type = "all"
)
```

```
parcel_fs_cfa$score_info$효능감$CR

## ------------------------------------------------
## name_in_vector usage
## ------------------------------------------------
item_parceling(
  data = shindata_example,
  c("효능감","efficacy2","efficacy3","efficacy4"),
  c("진로흥미","Interest1","Interest2","Interest3","Interest4"),
  fun = "factor_efa",
  name_in_vector = TRUE,
  type = "new"
)

item_parceling(
  data = shindata_example,
  c("efficacy2","efficacy3","efficacy4"),
  진로흥미 = c("Interest1","Interest2","Interest3","Interest4"),
  fun = "factor_efa",
  name_in_vector = "auto",
  type = "new"
)
```

`jjstat_plot_path` *SEM Path Diagram (Base R)*

Description

Draws a simple SEM path diagram using base R graphics. Only latent-latent edges (B, Phi) are weighted by coefficient size and sign. Measurement paths are drawn with fixed style for readability.

Usage

```
jjstat_plot_path(
  object,
  main = "SEM Path Diagram",
  layout = c("tree", "tree2", "tree3", "circle"),
  rotate = 1,
  show_phi = TRUE,
  show_theta = FALSE,
  std.all = TRUE,
  digits = 3,
  sizeLat = 8,
  sizeLat2 = 4,
  sizeMan = 5,
  sizeMan2 = 3,
  edge.width.range = c(1, 6),
  edge.color.pos = "black",
  edge.color.neg = "red",
  curve = 0.6,
  curve.max = 2,
  zigzag = TRUE,
```

```
  zigzag.amp = 0.035
)
```

Arguments

`object`	A `jjstat_sem` object or a list containing matrices: `Lambda`, `B`, `Phi` (and optionally `Theta`).
`main`	Title.
`layout`	Layout for latent variables: tree/tree2/tree3/circle.
`rotate`	Integer 1~4. Rotate coordinates by 90-degree steps (clockwise).
`show_phi`	Logical. If TRUE, draw latent correlations (Phi).
`show_theta`	Logical. If TRUE, draw residual loops for observed variables (Theta diagonal).
`digits`	Numeric. Rounding digits for coefficients.
`sizeLat, sizeLat2`	Latent ellipse radii (x/y) scaling.
`sizeMan, sizeMan2`	Manifest rectangle half-width/half-height scaling.
`edge.width.range`	Numeric length-2. Width range for latent-latent edges.
`edge.color.pos`	Color for positive coefficients.
`edge.color.neg`	Color for negative coefficients.
`curve`	A single numeric alias to control Phi curvature intensity. Internally mapped to `curve.base`. (Recommended to use this.)
`curve.max`	Maximum curvature for Phi.
`zigzag`	Logical. If TRUE, apply small zigzag offsets to latent positions.
`zigzag.amp`	Numeric. Zigzag amplitude.
`std`	Logical. If TRUE, uses correlation for Phi (standardized visualization).
`curve.base`	Base curvature parameter for Phi (distance-adjusted).

Examples

```
#'
library(jjstat)
# 검증용 python data
df <- data.frame(
  x1 = c( 1.730516,  1.159965, -0.648269, -0.593655,  1.213306,  0.895361, -2.729817, -1.785706,  0.35
  x2 = c( 2.208429,  1.111993, -0.671010, -0.309758,  0.893759,  1.354703, -2.800578, -1.492191,  0.51
  x3 = c( 1.791936,  0.665374, -0.004232, -0.253564,  0.835220,  0.945070, -2.115333, -1.176772,  0.01
  y1 = c( 1.581619,  1.372783, -0.346918, -0.078805,  1.076466,  1.521254, -1.682830, -1.047084,  1.09
  y2 = c( 2.393086,  1.230622, -0.902185, -0.714770,  1.501231,  1.572681, -2.962813, -1.159968,  0.70
  y3 = c( 0.896092,  1.639311, -0.756823, -0.286587,  0.717714,  1.296822, -1.155036, -1.229061,  0.48
)

mydata<-df

mydata
```

```
latent2 <- list(
  n1 = c("x1","x2","x3"),
  n2 = c("y1","y2","y3")
)

res2 <- jjstat_SEM(
  data = mydata,
  latent_list = latent2,
  structural = list(
    path(from="n1", to="n2")
  ),
  residual_cov = list(c("y1","y2")),
  nboot = 300,
  rmseaCI = 0.90
)

res2

summary(res2)

res2$parameters

jjstat_plot_path(res2)
# jjstat_plot_path(res, show_phi = TRUE, show_theta = TRUE, curve = 2, layout="circle")
```

`jjstat_SEM` *Educational Structural Equation Modeling (Closed-form Engine)*

Description

행렬기반수기계산방식으로구조방정식모형을추정하는교육용 SEM 엔진이다. 잠재변수 점수 (unit-weight), OLS 회귀, 모형암시공분산행렬을이용하여 ML 적합도를근사하며, bootstrap 기반표준오차와 RMSEA 신뢰구간을제공한다.

Usage

```
jjstat_SEM(
  data,
  latent_list,
  structural,
  residual_cov = NULL,
  variance = NULL,
  data_format = c("data", "cov", "cor"),
  n_cov = 200,
  nboot = 200,
  seed = 2025,
  rmseaCI = 0.9
)
```

Arguments

`data` data.frame 또는 matrix.

- `data_format="data"`: 관측자료
- `data_format="cov"`: 공분산행렬
- `data_format="cor"`: 상관행렬

`latent_list`	named list. 잠재변수와관측지표의매핑정보.
`structural`	list of `path()`. 잠재변수간구조경로정의.
`residual_cov`	list or NULL. 관측변수잔차공분산설정.
`variance`	list or NULL. 관측·잠재변수분산고정설정.
`data_format`	character. "data", "cov", "cor" 중하나.
`n_cov`	integer. 공분산/상관행렬입력시생성할표본크기.
`nboot`	integer. bootstrap 반복횟수.
`seed`	integer. 난수시드.
`rmseaCI`	numeric. RMSEA bootstrap 신뢰수준 (예: 0.90).

Details

본함수는 lavaan, OpenMx 와달리**모든계산단계를명시적으로드러내는교육용구현**이다. 수기계산검증, SEM 이론강의, 알고리즘이해를목적으로한다.

Value

`jjstat_sem` 클래스객체.

- call: 함수호출정보
- n: 표본크기
- Lambda, B, Phi, Theta, Sigma_hat
- fit: 적합도지수
- parameters: lavaan 스타일파라미터테이블
- bootstrap: bootstrap 결과

Examples

```
latent <- list(
  F1 = c("y1","y2","y3"),
  F2 = c("y4","y5","y6")
)

res <- jjstat_SEM(
  data = HolzingerSwineford1939[,1:6],
  latent_list = latent,
  structural = list(path("F1","F2")),
  nboot = 50
)

summary(res)
```

jtl *Haehyoung translate*

Description

Haehyoung translate

Usage

```
jtl(text = "", slang = "ko", elang = "en", translator = "deepl", print_url = F)
```

Arguments

`text`	text to translate
`slang`	starting language
`elang`	ending language
`translator`	translator type. lists are above.”alibaba”, ”apertium”, ”argos”, ”baidu”, ”bing”,”caiyun”, ”cloudTranslation”, ”deepl”, ”elia”, ”google”,”iciba”, ”iflytek”, ”iflyrec”, ”itranslate”, ”judic”, ”languageWire”, ”lingvanex”, ”niutrans”, ”mglip”, ”mirai”, ”modernMt”, ”myMemory”, ”papago”, ”qqFanyi”, ”qqTranSmart”, ”reverso”, ”sogou”, ”sysTran”, ”tilde”, ”translateCom”,”translateMe”, ”utibet”, ”volcEngine”, ”yandex”, ”yeekit”, ”youdao”
`print_url`	Debugging Setting

k *Kakao translaste*

Description

Kakao translaste

Usage

```
k(text = "", slang = "en", elang = "ko", show = "data")
```

Arguments

`text`	input text
`slang`	”ko”
`elang`	”en”
`show`	nomal, data is each sentence

Value

translate result

`kakaoi` *Kakao translaste*

Description

Kakao translaste

Usage

```
kakaoi(text = "", slang = "en", elang = "ko", show = "data")
```

Arguments

`text`	input text
`slang`	”ko”
`elang`	”en”
`show`	nomal, data is each sentence

Value

translate result

`kge_ca_analysis` *Correspondence analysis visualization functions*

Description

Correspondence analysis visualization functions

Usage

```
kge_ca_analysis(
  dataset,
  typekey,
  selcol_3 = "w1f",
  type = "all",
  arrows = c(F, T),
  selcol_1 = "성조",
  selcol_2 = "onset",
  area = "부산",
  addtext = "",
  xlim = NULL,
  ylim = NULL,
  size_text = 5,
  opt = 1
)
```

Arguments

dataset	data
typekey	filter
selcol_3	col3
type	all
arrows	c(F,T)
selcol_1	col1
selcol_2	col2
area	지역
addtext	add text
xlim	range c(0, 0.3)
ylim	rangec(-0.3, 0.4)
size_text	5
opt	opt 1~ 6

`kge_chisq_table` *kge_chisq_table chisq table observed/Expected table-*

Description

kge_chisq_table chisq table observed/Expected table-

Usage

```
kge_chisq_table(
  dataset,
  v1 = "a1",
  v2 = "성조",
  title = "Table",
  type = "res2",
  digits = 3,
  yadd = 0.4,
  Ncol = NULL,
  trans = FALSE,
  simple = FALSE,
  ko = TRUE,
  simulate.p.value = FALSE,
  correct = FALSE,
  size_bartext = 5,
  strip_size = 16,
  axis_size = 16,
  text_size = 13,
  xlab = "관측기대비율",
  ylab = "성조형",
  cramer = "adjust",
  raw = TRUE
)
```

Arguments

dataset	data.frame
v1	col
v2	row
title	title
type	res2 default
digits	3
yadd	up yadd
Ncol	grid
trans	transpose
simple	* style
ko	graph korean
simulate.p.value	exact
correct	exactt
size_bartext	graph text
strip_size	strip
axis_size	axis_size
text_size	text_size
xlab	xlab
ylab	ylab
cramer	cramer cor
raw	TRUE data.grame FALSE table data

Value

multiple data

Examples

```
## Not run:
kge_chisq_table(mtcars,"am","cyl")

# generate matrix
voteMat = matrix(c(183, 35, 824, 50), ncol=2)
colnames(voteMat) = c("Did not vote", "Voted")
rownames(voteMat) = c("All other ages","Youngest")

#chisq test result
vote_chi <- Chisq_retest(c(183, 35, 824, 50), ncol=2,
                         colname = c("Did not vote", "Voted"),
                          rowname =c("All other ages","Youngest"))

vote_chi

voteMat%>% data.frame() %>%
  long_df("voted", rowname="youth")%>% #long format
```

```
  add_rows_freq() %>%     # add rows
  kge_chisq_table("voted","youth")    #chisq test


## End(Not run)
```

`kge_weigth` *generative weight variable, kge(2024)*

Description

generative weight variable, kge(2024)

Usage

```
kge_weigth(df, type = "res", pattern = "", remove = FALSE)
```

Arguments

`df`	data.frame
`type`	result res, res1, res2, res3, res4
`pattern`	pattern = "", If you want something different, you can specify a different pattern.
`remove`	ubite option

Value

add weight

Examples

```
## Not run:
 # first
 kge_bind0 %>% split_kw_df_match()
#데이터검증: 입력시 " "와 같은 빈칸들이 존재함.
#' kge_bind11 <- kge_bind0 %>% split_kw_df_match()
kge_bind11$A3 %>% unique()
kge_bind11$B3 %>% unique()
kge_bind11$C3 %>% unique()
kge_bind11$D3 %>% unique()
# kge_bind10$d3 %>% unique()
#데이터 전처리: 빈칸 규칙 체크하여 정리
kge_bind11$A3 <- kge_bind11$A3 %>% replace_df(imp = "", pattern = " ")
kge_bind11$B3 <- kge_bind11$B3 %>% replace_df(imp = "", pattern = " ")
kge_bind11$C3 <- kge_bind11$C3 %>% replace_df(imp = "", pattern = " ")
kge_bind11$D3 <- kge_bind11$D3 %>% replace_df(imp = "", pattern = " ")

# result
kge_bind11 %>% data.frame()

# same data
```

```
kge_bind10 %>% data.frame()

# apply kge_weight
kge_bind10%>% kge_weigth() %>%  data.frame()

kge_bind1<- kge_bind10%>% kge_weigth()

kge_bind1 %>% data.frame()
 kge_bind1 %>% data.frame() %>% nrow()  #4924

##
 kge_bind1 %>% filter(성조 != "") %>% nrow() #4360

kge_bind2 %>% data.frame() %>% nrow()#4360


## End(Not run)
```

`kge_weigth_add_ca` *k generative weight variable korean, kge(2024)*

Description

k generative weight variable korean, kge(2024)

Usage

```
kge_weigth_add_ca(df001, type = "res", pattern = "", remove = FALSE)
```

Arguments

`type`	result res, res1, res2, res3, res4
`pattern`	pattern = ””, If you want something different, you can specify a different pattern.
`remove`	ubite option
`df`	data.frame

Value

add weight

Examples

```
## Not run:
 # first
 kge_bind0 %>% split_kw_df_match()
#데이터검증: 입력시 " "와 같은 빈칸들이 존재함.
#' kge_bind11 <- kge_bind0 %>% split_kw_df_match()
kge_bind11$A3 %>% unique()
kge_bind11$B3 %>% unique()
kge_bind11$C3 %>% unique()
```

```
kge_bind11$D3 %>% unique()
# kge_bind10$d3 %>% unique()
#데이터 전처리: 빈칸 규칙 체크하여 정리
kge_bind11$A3 <- kge_bind11$A3 %>% replace_df(imp = "", pattern = " ")
kge_bind11$B3 <- kge_bind11$B3 %>% replace_df(imp = "", pattern = " ")
kge_bind11$C3 <- kge_bind11$C3 %>% replace_df(imp = "", pattern = " ")
kge_bind11$D3 <- kge_bind11$D3 %>% replace_df(imp = "", pattern = " ")

# result
kge_bind11 %>% data.frame()

# same data
kge_bind10 %>% data.frame()

# apply kge_weight
kge_bind10%>% kge_weigth() %>%  data.frame()

kge_bind1<- kge_bind10%>% kge_weigth()

kge_bind1 %>% data.frame()
 kge_bind1 %>% data.frame() %>% nrow()  #4924

##
 kge_bind1 %>% filter(성조 != "") %>% nrow() #4360

kge_bind2 %>% data.frame() %>% nrow()#4360


## End(Not run)
```

`kge_weigth_ko` *generative weight variable korean, kge(2024)*

Description

generative weight variable korean, kge(2024)

Usage

```
kge_weigth_ko(df, type = "res", pattern = "", remove = FALSE)
```

Arguments

`df`	data.frame
`type`	result res, res1, res2, res3, res4
`pattern`	pattern = ””, If you want something different, you can specify a different pattern.
`remove`	ubite option

Value

add weight

Examples

```
## Not run:
 # first
 kge_bind0 %>% split_kw_df_match()
#데이터검증: 입력시 " "와 같은 빈칸들이 존재함.
#' kge_bind11 <- kge_bind0 %>% split_kw_df_match()
kge_bind11$A3 %>% unique()
kge_bind11$B3 %>% unique()
kge_bind11$C3 %>% unique()
kge_bind11$D3 %>% unique()
# kge_bind10$d3 %>% unique()
#데이터 전처리: 빈칸 규칙 체크하여 정리
kge_bind11$A3 <- kge_bind11$A3 %>% replace_df(imp = "", pattern = " ")
kge_bind11$B3 <- kge_bind11$B3 %>% replace_df(imp = "", pattern = " ")
kge_bind11$C3 <- kge_bind11$C3 %>% replace_df(imp = "", pattern = " ")
kge_bind11$D3 <- kge_bind11$D3 %>% replace_df(imp = "", pattern = " ")

# result
kge_bind11 %>% data.frame()

# same data
kge_bind10 %>% data.frame()

# apply kge_weight
kge_bind10%>% kge_weigth() %>%  data.frame()

kge_bind1<- kge_bind10%>% kge_weigth()

kge_bind1 %>% data.frame()
 kge_bind1 %>% data.frame() %>% nrow()  #4924

##
 kge_bind1 %>% filter(성조 != "") %>% nrow() #4360

kge_bind2 %>% data.frame() %>% nrow()#4360


## End(Not run)
```

KURT *kurtosis* 계산함수

Description

kurtosis 계산함수

Usage

```
KURT(x)
```

Arguments

x	vector

Value

kurtosis

`k_df` *k df rep*

Description

k df rep

Usage

```
k_df(df, range = 1:ncol(df), slang = "en", elang = "ko")
```

Arguments

df	data.frame
range	range
slang	en
elang	ko

Value

text

Examples

```
## Not run:

#'
Authentic_Leadership <- data.frame(
  id = paste0("Q",1:16),
  factor= c(
    rep("self Awareness", 4),
    rep("Relational Transparency", 4),
    rep("Internalized Moral perspective", 4),
    rep("Balanced Processing", 4)
  ),
  Question = c(
    "I can list my three greatest weaknesses.",
    "My actions reflect my core values.",
    "I seek others' opinions before making up my own mind.",
    "I openly share my feelings with others.",
    "I can list my three greatest strengths.",
    "I do not allow group pressure to control me.",
    "I listen closely to the ideas of those who disagree with me.",
    "I let others know who I truly am as a person.",
    "I seek feedback as a way of understanding who I really am as a person.",
```

```
    "Other people know where I stand on controversial issues.",
    "I do not emphasize my own point of view at the expense of others.",
    "I rarely present a "false" front to others.",
    "I accept the feelings I have about myself.",
    "My morals guide what I do as a leader.",
    "I listen very carefully to the ideas of others before making decisions.",
    "I admit my mistakes to others."
  ))%>%tibble()
Authentic_Leadership
# 데이터프레임 번역
Alp_ko = k_df(Authentic_Leadership,
              range = 2:3,
              slang = "en",
              elang = "ko") %>%select(3,2)
Alp_ko
Alp_ko %>%replace_df_rep("받아들입니다.","받아들인다.", "압니다","안다")
#'




## End(Not run)
```

LavaanPlot *Lavaan plot draw*

Description

Lavaan plot draw

Usage

```
LavaanPlot(
  x,
  rotation = "LR",
  coefs = TRUE,
  col = "gray60",
  stand = FALSE,
  label = NULL,
  covs = FALSE,
  stars = c("covs", "latent", "regress"),
  overlap = FALSE,
  layout = "dot",
  digits = 3
)
```

Arguments

`x`	lavaan result
`rotation`	rotation 'LR','RL', 'TB','BT'
`coefs`	TRUE
`col`	default 'gray60'
`stand`	std coeff
`label`	input name labels=c(f1_1="exp", f1_2="want",f1_3="problem") etc
`covs`	TRUE
`stars`	stars = c("covs", "latent","regress"),
`overlap`	default FALSE
`layout`	'dot', 'neato', 'fdp', 'sfdp', 'circo', 'twopi', 'nop','nop2', 'osage', 'patchwork'
`digits`	digits = 3#'

Value

plot

Examples

```
# lavaan code

## Bollen (1989), page 332
model <- '
  # latent variable definitions
     ind60 =~ x1 + x2 + x3
     dem60 =~ y1 + a*y2 + b*y3 + c*y4
     dem65 =~ y5 + a*y6 + b*y7 + c*y8

  # regressions
    dem60 ~ ind60
  dem65 ~ ind60 + dem60

  # residual correlations
    y1 ~~ y5
    y2 ~~ y4 + y6
    y3 ~~ y7
    y4 ~~ y8
    y6 ~~ y8
'

fit <- lavaan::sem(model, data = PoliticalDemocracy)
 ## summary
summary(fit, fit.measures = TRUE, stand=T)
##
LavaanPlot(fit, cov=F)
LavaanPlot(fit, cov=F, layout="circo")

LavaanPlot(fit, cov=F, layout="dot",
           label=c(x1="ind1",ind60="IND60"))
```

`lav_change_node_name` *Change node names in semPlot / qgraph object*

Description

semPlot::semPaths() 로생성된구조방정식시각화객체에서노드이름을사용자가지정한 (원래이름, 변경라벨) 쌍에따라일괄적으로변경한다. 본함수는 pipe(

Usage

```
lav_change_node_name(plot_obj, var_named)
```

Arguments

`plot_obj`	semPlot 또는 qgraph 객체
`var_named`	character vector. c(old1, new1, old2, new2, ...)

Value

노드명칭이변경된 semPlot 객체

Examples

```
## Not run:

#################################
# 1. SCCT 구조방정식 모형 정의
#################################

model_scct_indirect <- '

REFF =~ efficacy2 + efficacy3 + efficacy4
OEXP =~ Outcome1 + Outcome2 + Outcome3 + Outcome4 + Outcome5 + Outcome8
INTR =~ Interest1 + Interest2 + Interest3 + Interest4

REFF ~ e1*grade + e2*experience0
OEXP ~ a1*REFF + d1*grade + d2*experience0
INTR ~ b1*REFF + b2*OEXP

intention ~ c1*INTR + c2*OEXP + c3*REFF

grade ~~ experience0

# 간접효과
ind_grade_REFF        := e1 * c3
ind_grade_REFF_INTR   := e1 * b1 * c1
ind_grade_REFF_OEXP   := e1 * a1 * c2
ind_grade_REFF_OEXP_INTR := e1 * a1 * b2 * c1

ind_exp_REFF          := e2 * c3
ind_exp_REFF_INTR     := e2 * b1 * c1
ind_exp_REFF_OEXP     := e2 * a1 * c2
ind_exp_REFF_OEXP_INTR := e2 * a1 * b2 * c1
'
```

```
##################################
# 2. 모형 추정
##################################

library(lavaan)
fit_scct_indirect <- sem(
  model = model_scct_indirect,
  data  = shindata,
  se = "bootstrap",
  bootstrap = 2000
)

##################################
# 3. 노드 명칭 매핑 벡터
##################################

var_named1 <- c(
  "REFF", "연구효능감",
  "efficacy2","연구효능2",
  "efficacy3","연구효능3",
  "efficacy4","연구효능4",

  "INTR", "진로흥미",
  "Interest1","진로흥미1",
  "Interest2","진로흥미2",
  "Interest3","진로흥미3",
  "Interest4","진로흥미4",

  "OEXP", "결과기대",
  "Outcome1","결과기대1",
  "Outcome2","결과기대2",
  "Outcome3","결과기대3",
  "Outcome4","결과기대4",
  "Outcome5","결과기대5",
  "Outcome8","결과기대8",

  "grade","학년",
  "experience0","연구경험",
  "intention","진로의지"
)

##################################
# 4. SCCT 전용 layout 행렬
##################################

layout_scct <- matrix(
  c(
    NA, "efficacy2", "efficacy3", "efficacy4", NA,
        "Interest1", "Interest2", "Interest3", "Interest4", NA,

    "experience0", NA, "REFF", NA, NA,
        NA, "INTR", NA, NA, NA,

    "grade", NA, "OEXP", NA, NA,
        NA, NA, "intention", NA, NA,
```

```
    NA, "Outcome1", "Outcome2", "Outcome3", "Outcome4",
        "Outcome5", "Outcome8", NA, NA, NA
  ),
  nrow = 4,
  byrow = TRUE
)

################################
# 5. 구조방정식 그림 + 노드명 변경
################################

library(semPlot)
library(semptools)

semPlot::semPaths(
  fit_scct_indirect,
  layout        = layout_scct,
  whatLabels    = "std",
  style         = "ram",
  residuals     = FALSE,
  intercepts    = FALSE,
  nCharNodes    = 0,
  sizeLat       = 9,
  sizeMan       = 9,
  edge.label.cex = 0.85,
  edge.label.position = 0.65
) %>%
  semptools::mark_sig(fit_scct_indirect) %>%
  change_node_name(var_named1) %>%  ##### 이것을 이용하여 이름 변경
  plot()


## End(Not run)
```

`lav_de_hypo` *Hypothesis generation functions*

Description

Hypothesis generation functions

Usage

```
lav_de_hypo(model)
```

Arguments

`model`	lavaan syntax

Value

text

Examples

```
## Not run:
#'
# 예제 모델
model1a <- "
EXPE ~ H1*IMAG
QUAL ~ H2*EXPE
VAL ~  H3*QUAL + H4*EXPE
SAT ~ H5*IMAG + H6*EXPE + H7*QUAL + H8*VAL
LOY ~ H9*SAT + H10*IMAG
"

# 함수 실행
lav_de_hypo(model1a)

#'

## End(Not run)
```

`lav_edge_value` *Extract edge labels for semPlot from lavaan / MLSEM object*

Description

lavaan(일반 SEM/CFA) 또는다층 SEM(ML-SEM) 적합객체에서 semPlot::semPaths() 의 edgeLabels 인수에바로넣을수있는" 간선라벨 (edge label) 벡터" 를자동생성하는함수이다.

본함수는 lavaan::parameterEstimates() 를기반으로측정경로 (=~), 구조경로 (~), 공분산 (~~) 을모두인식한다. 다층모형의경우 within/between 수준 (block) 을선택할수있다.

핵심설계목표는다음과같다. (1) 기준지표 (fixed loading=1) 에서 pvalue 가 NA 로반환되는경우에도라벨에"NA" 가붙지않도록처리한다. (2) semPlot 이요구하는형식 (unnamed character vector) 에맞게안전반환한다. (3) 측정경로/공분산값을숨기거나, 유의성별표를제거하는옵션을제공한다. (4) 구조경로 (~) 만뽑아구조모형라벨에만쓰는옵션 (path_only) 을제공한다.

Usage

```
lav_edge_value(
  obj,
  level = c("within", "between"),
  type = c("est", "std"),
  sig = TRUE,
  digits = 3,
  include_cov = TRUE,
  fillin = FALSE,
  hide_mea = "",
  hide_cov = "",
  hide_sig = FALSE,
  hide_sig_mea = "",
```

```
  hide_sig_cov = "",
  path_only = FALSE
)
```

Arguments

obj	lavaan fitted object. (cfa/sem 또는다층 sem 객체)
level	character. "within" 또는"between". 다층모형에서만의미가있으며, within=block 1, between=block 2 로처리한다. 일반 lavaan 단일수준 모형에서는 block 이 0 또는 1 로나올수있어, 이경우 level 을"within" 으로두는것이보통안전하다.
type	character. "est"(비표준화추정치) 또는"std"(표준화계수 std.all).
sig	logical. 유의성별표 (*, **, ***) 를붙일지여부. 기본 TRUE. 단, lavaan 에서 pvalue 가 NA 인경우 (예: 기준지표 fixed loading=1) 는별표를붙이지않는다 (NA-safe).
digits	numeric. 반올림소수점자리수. 기본 3.
include_cov	logical. 공분산 (~~) 을포함할지여부. 기본 TRUE. FALSE 이면 ~~ 는제외된다.
fillin	logical. TRUE 이면측정경로 (=~) 및공분산 (~~) 라벨을사용자가지정한문자로" 대체" 한다. (예: 측정경로는' 지표' 로숨기고, 구조경로 (~) 만수치라벨로보이게)
hide_mea	character. fillin=TRUE 일때 =~ 경로라벨을이문자로대체. 기본""(빈문자열).
hide_cov	character. fillin=TRUE 일때 ~~ 경로라벨을이문자로대체. 기본""(빈문자열).
hide_sig	logical. TRUE 이면모든유의성별표를제거한다. 주의: hide_sig=TRUE 이면 sig 인수는사실상무시된다 (별표제거가우선).
hide_sig_mea	character. hide_sig=TRUE 일때, =~ 경로에대해특정문자열로대체하고싶을때사용한다. 기본""(대체안함).
hide_sig_cov	character. hide_sig=TRUE 일때, ~~ 경로에대해특정문자열로대체하고싶을때사용한다. 기본""(대체안함).
path_only	logical. TRUE 이면구조경로 (~) 만추출한다. 구조모형만그리고싶을때사용한다. 이경우 fillin 옵션은적용되지않는다 (측정/공분산을뽑지않기때문).

Value

unnamed character vector. semPlot::semPaths(edgeLabels=) 에그대로투입가능한형식이다.

Examples

```
## Not run:
# ============================================================
# [0] 전제: fit_mlsem_expB, model_within_plot, diagram_model()이 존재한다고 가정
# ============================================================

# ------------------------------------------------------------
# (1) 기본 사용: within 수준, 표준화 계수(std) + 유의성 별표
# ------------------------------------------------------------
```

```
edge_std <- lav_edge_value(
  obj   = fit_mlsem_expB,
  level = "within",
  type  = "std"
)
edge_std

# diagram_model에 그대로 주입
diagram_model(
  model      = model_within_plot,
  layout     = layout_scct1,
  whatLabels = "none",
  sizeMan    = 8,
  sizeLat    = 10,
  edge.label.position = 0.7,
  edge.label.cex      = 0.9,
  edgeLabels = edge_std,
  color = list(lat="black", man="White")
)

# ------------------------------------------------------------
# (2) 비표준화 계수(est) 라벨
# ------------------------------------------------------------
edge_est <- lav_edge_value(
  obj   = fit_mlsem_expB,
  level = "within",
  type  = "est"
)
edge_est

diagram_model(
  model      = model_within_plot,
  layout     = layout_scct1,
  whatLabels = "none",
  edge.label.position = 0.7,
  edge.label.cex      = 0.9,
  edgeLabels = edge_est
)

# ------------------------------------------------------------
# (3) 측정경로(=~)를 숨기고 구조경로(~)만 숫자로 보여주기
#     - fillin=TRUE + hide_mea="지표"
#     - 공분산(~~)은 빈문자("")로 숨김
# ------------------------------------------------------------
edge_std_path <- lav_edge_value(
  obj      = fit_mlsem_expB,
  level    = "within",
  type     = "std",
  fillin   = TRUE,
  hide_mea = "지표",
  hide_cov = ""
)

diagram_model(
  model      = model_within_plot,
  layout     = layout_scct1,
  whatLabels = "none",
```

```
  edge.label.position = 0.6,
  edge.label.cex      = 0.9,
  edgeLabels = edge_std_path
)

# ------------------------------------------------------------
# (4) 유의성 별표(*,**,***)를 모두 숨기고 숫자만 표시
#     - hide_sig=TRUE가 최우선
# ------------------------------------------------------------
edge_std_nostar <- lav_edge_value(
  obj      = fit_mlsem_expB,
  level    = "within",
  type     = "std",
  hide_sig = TRUE
)

diagram_model(
  model      = model_within_plot,
  layout     = layout_scct1,
  whatLabels = "none",
  edgeLabels = edge_std_nostar
)

# ------------------------------------------------------------
# (5) 구조경로(~)만 추출: 구조모형 라벨만 뽑기
#     - 측정경로/공분산 라벨이 전혀 포함되지 않는다.
# ------------------------------------------------------------
edge_struct_only <- lav_edge_value(
  obj       = fit_mlsem_expB,
  level     = "within",
  type      = "std",
  path_only = TRUE
)
edge_struct_only

# (주의) diagram_model()에서 structural=TRUE를 쓰면
# semPlot이 일부 변수를 자동 제외할 수 있어 레이아웃과 충돌 가능성이 있다.
# 이 경우에는 whatLabels="none" + fillin을 쓰는 방식이 더 안정적일 수 있다.

# ------------------------------------------------------------
# (6) between 수준 라벨 추출 (ML-SEM)
# ------------------------------------------------------------
edge_between <- lav_edge_value(
  obj   = fit_mlsem_expB,
  level = "between",
  type  = "std",
  fillin   = TRUE,
  hide_mea = "지표",
  hide_cov = ""
)
edge_between

## End(Not run)
```

`lav_extract_imlist` *Function to extract interaction terms from the lavaan syntax, excluding commented lines.*

Description

Function to extract interaction terms from the lavaan syntax, excluding commented lines.

Usage

```
lav_extract_imlist(lav_syn, prefix = ".")
```

Arguments

`lav_syn`	lavaan syntax
`prefix`	. dataname prefix

Value

list

Examples

```
## Not run:
#'

# # Example usage:
data(MarshWenHau)
str(MarshWenHau)

mwh = MarshWenHau

model2 <- "
f1 =~ x1 + x2 + x3
f1Xf2 =~ f1:f2
f2 =~ x4 + x5 + x6
f3 =~ y1 + y2 + y3

f3 ~ f1 + f2
f3 ~ f1Xf2

f1Xf2 ~~ 0*f1
f1Xf2 ~~ 0*f2
"

# interaction model
intsem =  SEM(model2, data= MarshWenHau)
summary(intsem)
intsem %>%lav_semPaths2(color=list(lat="darkred",man="gold"))


##2nd
#base model
lav_extract_mm(model2) %>%cat()
#interaction item list
lav_extract_imlist(model3)
```

```
#data
lav_latentProd(mwh, model3)%>%head()

#extracted interaction term
lav_extract_int(model2, lav_extract_imlist(model3))

# or interaction items ; imlist
imlist <- list(
  .f1Xf2 = c("f1", "f2"),
  .i_f1_f4 = c("f1", "f4")
)
# names(imlist)
lav_extract_int(model2, imlist)
#direct
lav_extract_int(model2, imlist= list(.f1Xf2 = c("f1", "f2"),
                                     .i_f1_f4 = c("f1", "f4")))
#estimates model
intsem =  SEM(lav_new_model(model2, lav_extract_imlist(model2)),
              data= lav_latentProd(mwh, model2))


## End(Not run)
```

`lav_extract_ind` *lav_extract_ind, indirect effect*

Description

lav_extract_ind, indirect effect

Usage

```
lav_extract_ind(
  model,
  start_node = NULL,
  end_node = NULL,
  prefix = "a",
  cat = FALSE,
  type = "ind",
  auto = TRUE,
  paths_name = FALSE,
  n_name = 1
)
```

Arguments

`model`	lavaan
`start_node`	path start node
`end_node`	path end node
`prefix`	hypothesis ”a” if U want change ”H”
`cat`	test show cat TRUR

`type`	type ”ind”,”new_parameters”,”combine”
`auto`	TRUE path auto, FLASE all ind path
`paths_name`	TRUE change paths name
`n_name`	Number of letters in the name being extracted, default =1

Value

text

Examples

```
## Not run:
#'

# Example usage
model1 <- "
IMAG =~ imag1 + imag2 + imag3 + imag4 + imag5
EXPE =~ expe1 + expe2 + expe3 + expe4 + expe5
QUAL =~ qual1 + qual2 + qual3 + qual4 + qual5
VAL =~ val1 + val2 + val3 + val4
SAT =~ sat1 + sat2 + sat3 + sat4
LOY =~ loy1 + loy2 + loy3 + loy4

EXPE ~ IMAG
QUAL ~ EXPE
VAL ~  QUAL+ EXPE
SAT ~ IMAG + EXPE + QUAL + VAL
LOY ~ SAT + IMAG
"


# Extracting paths from the model with default start_node = NULL and end_node = NULL
lav_extract_mm(model1) %>%cat("\n")
lav_extract_sm(model1) %>%cat("\n")

lav_extract_ind(model1)%>%cat("\n")
lav_extract_ind(model1, auto=TRUE)%>%cat("\n")

lav_extract_ind(model1,start_node="EXPE", auto=TRUE)%>%cat("\n")
lav_extract_ind(model1,start_node="EXPE")%>%cat("\n")
lav_extract_ind(model1,start_node="QUAL")%>%cat("\n")
lav_extract_ind(model1,start_node="IMAG", auto=TRUE)%>%cat("\n")

lav_extract_ind(lav_extract_sm(model1), de=TRUE) %>%cat("\n")

lav_extract_ind(lav_extract_sm(model1), de=TRUE, auto=TRUE)%>%cat("\n")
lav_extract_ind(lav_extract_sm(model1), start_node="IMAG", de=TRUE) %>%cat('\n')

lav_extract_ind(lav_extract_sm(model1),"IMAG", prefix="H", de=TRUE) %>%cat('\n')

# Example usage
de_paths <- "
EXPE  ~ a1* IMAG
QUAL  ~ a2* EXPE
```

```
VAL  ~ a3* QUAL
VAL  ~ a4* EXPE
SAT  ~ a5* IMAG
SAT  ~ a6* EXPE
SAT  ~ a7* QUAL
SAT  ~ a8* VAL
LOY  ~ a9* SAT
LOY  ~ a10* IMAG
"

lav_extract_ind(de_paths)
#출력
#  lav_extract_ind(model1, cat=TRUE)

lav_extract_ind(de_paths, cat=TRUE)

lav_extract_ind(de_paths)%>%make_c_text()

## End(Not run)
```

`lav_extract_ind_paths` *lav_extract_ind_paths, Create an indirect effect hypothesis*

Description

lav_extract_ind_paths, Create an indirect effect hypothesis

Usage

```
lav_extract_ind_paths(
  paths_data,
  first_node = NULL,
  paths_name = FALSE,
  allpaths = FALSE
)
```

Arguments

`paths_data`	direct path
`first_node`	start node name
`paths_name`	TRUE hypothesis output
`allpaths`	all and ind(FALSE)

Value

text

Examples

```
## Not run:
#'
# 예시 데이터
paths_data <- data.frame(
  hypo = c("H1", "H2", "H3", "H4", "H5",
  "H6", "H7", "H8", "H9", "H10"),
  paths = c("IMAG->EXPE", "EXPE->QUAL",
  "QUAL->VAL", "EXPE->VAL", "IMAG->SAT",
   "EXPE->SAT", "QUAL->SAT", "VAL->SAT", "SAT->LOY", "IMAG->LOY")
)
# 함수 실행
lav_extract_ind_paths(paths_data, allpaths = TRUE)
lav_extract_ind_paths(paths_data, allpaths = FALSE)
#'
lav_extract_ind_paths(paths_data, allpaths = FALSE, paths_name = TRUE)
lav_extract_ind_paths(paths_data, allpaths = FALSE, paths_name = FALSE)
#'
lav_extract_path(model1a)%>%lav_extract_ind_paths()

lav_extract_path(model1)%>%lav_extract_ind_paths()
#'
lav_extract_path(model1a) %>%
  lav_extract_ind_paths() %>%
  sem_lav_ind_hypo()
#'
## 구조모형에서 나타나는 간접효과 가설은 다음과 같다.
## 간접효과[H1]: 간접효과 IMAG -> SAT ->
LOY 는 통계적으로 유의할 것이다.
## 간접효과[H2]: 간접효과 IMAG -> EXPE -
> SAT -> LOY 는 통계적으로 유의할 것이다.
## 간접효과[H3]: 간접효과 IMAG -> EXPE -
> VAL -> SAT -> LOY 는 통계적으로 유의할 것이다.
## 간접효과[H4]: 간접효과 IMAG -> EXPE
-> QUAL -> SAT -> LOY 는 통계적으로 유의할 것이다.
## 간접효과[H5]: 간접효과 IMAG -> EXPE
-> QUAL -> VAL -> SAT -> LOY 는 통계적으로 유의할 것이다.
#' #'
#'

# Example usage with the provided model
model10 <- "
IMAG =~ imag1 + imag2 + imag3 + imag4 + imag5
EXPE =~ expe1 + expe2 + expe3 + expe4 + expe5
QUAL =~ qual1 + qual2 + qual3 + qual4 + qual5
VAL =~ val1 + val2 + val3 + val4
SAT =~ sat1 + sat2 + sat3 + sat4
LOY =~ loy1 + loy2 + loy3 + loy4

EXPE ~ IMAG
QUAL ~ EXPE
VAL ~  QUAL+ EXPE
SAT ~ IMAG + EXPE + QUAL + VAL
LOY ~ SAT + IMAG
"
```

```
# Extract paths from the model
lav_extract_path(model10)
lav_extract_path(model10)%>%lav_extract_ind_paths()

lav_extract_sm(model10)%>%lav_extract_path()%>%lav_extract_ind_paths()
lav_extract_sm(model10)%>%lav_extract_ind_paths()
lav_extract_sm(model10)%>%lav_extract_ind_paths("EXPE")


## End(Not run)
```

`lav_extract_int` *lav_extract_int*

Description

lav_extract_int

Usage

```
lav_extract_int(lav_syn, imlist = NULL)
```

Arguments

`lav_syn`	lavaan
`imlist`	item list

Value

interation data

Examples

```
## Not run:
#'

# # Example usage:
data(MarshWenHau)
str(MarshWenHau)

mwh = MarshWenHau

model2 <- "
f1 =~ x1 + x2 + x3
f1Xf2 =~ f1:f2
f2 =~ x4 + x5 + x6
f3 =~ y1 + y2 + y3

f3 ~ f1 + f2
f3 ~ f1Xf2
```

```
f1Xf2 ~~ 0*f1
f1Xf2 ~~ 0*f2
"

# interaction model
intsem =  SEM(model2, data= MarshWenHau)
summary(intsem)
intsem %>%lav_semPaths2(color=list(lat="darkred",man="gold"))


##2nd
#base model
lav_extract_mm(model2) %>%cat()
#interaction item list
lav_extract_imlist(model3)
#data
lav_latentProd(mwh, model3)%>%head()

#extracted interaction term
lav_extract_int(model2, lav_extract_imlist(model3))

# or interaction items ; imlist
imlist <- list(
  .f1Xf2 = c("f1", "f2"),
  .i_f1_f4 = c("f1", "f4")
)
# names(imlist)
lav_extract_int(model2, imlist)
#direct
lav_extract_int(model2, imlist= list(.f1Xf2 = c("f1", "f2"),
                                     .i_f1_f4 = c("f1", "f4")))
#estimates model
intsem =  SEM(lav_new_model(model2, lav_extract_imlist(model2)),
              data= lav_latentProd(mwh, model2))


## End(Not run)
```

`lav_extract_mm` *extranct measurement*

Description

extranct measurement

Usage

```
lav_extract_mm(lav_syn, text = FALSE)
```

Arguments

`lav_syn`	model lavaan syntax
`text`	text output

Value

text model

Examples

```
## Not run:
#'

# # Example usage:
data(MarshWenHau)
str(MarshWenHau)

mwh = MarshWenHau

model2 <- "
f1 =~ x1 + x2 + x3
f1Xf2 =~ f1:f2
f2 =~ x4 + x5 + x6
f3 =~ y1 + y2 + y3

f3 ~ f1 + f2
f3 ~ f1Xf2

f1Xf2 ~~ 0*f1
f1Xf2 ~~ 0*f2
"

# interaction model
intsem =  SEM(model2, data= MarshWenHau)
summary(intsem)
intsem %>%lav_semPaths2(color=list(lat="darkred",man="gold"))


##2nd
#base model
lav_extract_mm(model2) %>%cat()
#interaction item list
lav_extract_imlist(model3)
#data
lav_latentProd(mwh, model3)%>%head()

#extracted interaction term
lav_extract_int(model2, lav_extract_imlist(model3))

# or interaction items ; imlist
imlist <- list(
  .f1Xf2 = c("f1", "f2"),
  .i_f1_f4 = c("f1", "f4")
)
# names(imlist)
lav_extract_int(model2, imlist)
#direct
lav_extract_int(model2, imlist= list(.f1Xf2 = c("f1", "f2"),
                                     .i_f1_f4 = c("f1", "f4")))
#estimates model
intsem =  SEM(lav_new_model(model2, lav_extract_imlist(model2)),
```

```
                data= lav_latentProd(mwh, model2))

## End(Not run)
```

`lav_extract_mm2` *extranct measurement, Extract all structural models*

Description

extranct measurement, Extract all structural models

Usage

```
lav_extract_mm2(lav_syn, text = FALSE, interact = TRUE)
```

Arguments

`lav_syn`	model lavaan syntax
`text`	text output
`interact`	FALSE exclude :, TRue include :

Value

text model

Examples

```
## Not run:
#'

# # Example usage:
data(MarshWenHau)
str(MarshWenHau)

mwh = MarshWenHau

model2 <- "
f1 =~ x1 + x2 + x3
f1Xf2 =~ f1:f2
f2 =~ x4 + x5 + x6
f3 =~ y1 + y2 + y3

f3 ~ f1 + f2
f3 ~ f1Xf2

f1Xf2 ~~ 0*f1
f1Xf2 ~~ 0*f2
"

# interaction model
intsem =  SEM(model2, data= MarshWenHau)
```

```
summary(intsem)
intsem %>%lav_semPaths2(color=list(lat="darkred",man="gold"))


##2nd
#base model
lav_extract_mm(model2) %>%cat()
#interaction item list
lav_extract_imlist(model3)
#data
lav_latentProd(mwh, model3)%>%head()

#extracted interaction term
lav_extract_int(model2, lav_extract_imlist(model3))

# or interaction items ; imlist
imlist <- list(
  .f1Xf2 = c("f1", "f2"),
  .i_f1_f4 = c("f1", "f4")
)
# names(imlist)
lav_extract_int(model2, imlist)
#direct
lav_extract_int(model2, imlist= list(.f1Xf2 = c("f1", "f2"),
                                     .i_f1_f4 = c("f1", "f4")))
#estimates model
intsem =  SEM(lav_new_model(model2, lav_extract_imlist(model2)),
              data= lav_latentProd(mwh, model2))

#'
model2 <- "
 f1 =~ x1 + x2 + x3
 f1Xf2 =~ f1:f2
 f2 =~ x4 + x5 + x6
 f3 =~ y1 + y2 + y3

 f3 ~ f1 + f2
 f3 ~ f1Xf2

 f1Xf2 ~~ 0*f1
 f1Xf2 ~~ 0*f2
 "

lav_extract_mm2(model2)
lav_extract_mm2(model2, interact=F)
lav_extract_mm2(model2, interact=T)
#'
#'

## End(Not run)
```

---

lav_extract_path *lav_extract_path*

Description

lav_extract_path

Usage

```
lav_extract_path(model)
```

Arguments

`model`	model

Value

text

Examples

```
## Not run:
#'
# 모델 문자열
model1a <- "
EXPE ~ H1*IMAG
QUAL ~ H2*EXPE
VAL ~ H3*QUAL + H4*EXPE
SAT ~ H5*IMAG + H6*EXPE + H7*QUAL + H8*VAL
LOY ~ H9*SAT + H10*IMAG
"

model1 <- "
EXPE ~ IMAG
QUAL ~ EXPE
VAL ~ QUAL + EXPE
SAT ~ IMAG + EXPE + QUAL + VAL
LOY ~ SAT + IMAG
"


# 함수 호출
lav_extract_path(model1a)

lav_extract_path(model1)


## End(Not run)
```

`lav_extract_sm` *lav_extract_sm, Extracting a Structural Path Model from a Model*

Description

lav_extract_sm, Extracting a Structural Path Model from a Model

Usage

```
lav_extract_sm(model, hypo = TRUE, prefix = "a", all = FALSE)
```

Arguments

model	lavaan model
hypo	hypothesis, default TRUE
prefix	”a”
all	all paths

Value

text

Examples

```
## Not run:
#'
# Example usage
model1 <- "
IMAG =~ imag1 + imag2 + imag3 + imag4 + imag5
EXPE =~ expe1 + expe2 + expe3 + expe4 + expe5
QUAL =~ qual1 + qual2 + qual3 + qual4 + qual5
VAL =~ val1 + val2 + val3 + val4
SAT =~ sat1 + sat2 + sat3 + sat4
LOY =~ loy1 + loy2 + loy3 + loy4

EXPE ~ IMAG
QUAL ~ EXPE
VAL ~  QUAL+ EXPE
SAT ~ IMAG + EXPE + QUAL + VAL
LOY ~ SAT + IMAG
"


lav_extract_sm(model1)%>%cat("\n")
lav_extract_sm(model1, hypo=FALSE)%>%cat("\n")
lav_extract_sm(model1, prefix="lamda")%>%cat("\n")
lav_extract_sm(model1, prefix="H")%>%cat("\n")
lav_extract_sm(model1, all=FALSE) %>%cat("\n")
lav_extract_sm(model1, all=TRUE) %>%cat("\n")
lav_extract_sm(model1,FALSE) %>%cat("\n")


lav_extract_sm(model1, all=TRUE) %>%
  diagram_model(sizeLat=6)
#'


## End(Not run)
```

`lav_factor_layout` 자동화된 *semPaths* 모델 *layout*

Description

자동화된 semPaths 모델 layout

Usage

```
lav_factor_layout(
  sem_res,
  positions = NULL,
  nrow = 4,
  ncol = 5,
  new_position = NULL,
  variables = NULL,
  ...
)
```

Arguments

`sem_res`	lavaan object
`positions`	각변수의위치를변경 position = list("motiv" =c(1, 3))
`nrow`	4
`ncol`	5
`new_position`	list(c(1, 1), c(1, 2), c(3, 1), c(2, 3),c(2, 4))
`variables`	직업입력하는변수명
`...`	ETC

Value

MAT

Examples

```
## Not run:
#'
data(motivation)
str(motivation)
# Test the function with debug prints
motive_model <- "
achieve ~ motiv + ses + iq
motiv ~ ses + health + iq
"

# Assuming 'motivation' is your data frame
motiv_sem <- SEM(motive_model, data = motivation)

motiv_sem%>%summary(stand=TRUE)
```

```
# 적용 예시
motiv_sem %>%
  lav_semPaths2(rotation = 2, se=TRUE,
  edge.label.position = 0.4,
                layout =
                lav_factor_layout(motiv_sem,
       new_position = list(c(1, 1), c(1, 2),
       c(3, 1), c(2, 3),c(2, 4)) ),
            rotate_resid = c("motiv" =160, "achieve"=160))


# 적용 예시
motiv_sem %>%
  lav_semPaths2(rotation = 2, se=TRUE,
  edge.label.position = 0.4,
                layout = lav_factor_layout(motiv_sem,
                                           position = list("motiv" =c(1, 3))),
                rotate_resid = c("motiv" =160, "achieve"=160))

#'

## End(Not run)
```

`lav_htmt` *lavaan htmt*

Description

lavaan htmt

Usage

```
lav_htmt(
  sem_res = NULL,
  data = NULL,
  blocks = NULL,
  htmt2 = TRUE,
  sig = TRUE,
  cut = 0.9,
  digits = 3,
  imp = ""
)
```

Arguments

`sem_res`	lavaan result
`data`	sem data
`blocks`	sem blocks list data
`htmt2`	true using htmt2
`sig`	true *
`cut`	cut default 0.9, roburst 0.85, general 1

digits	3
imp	"" upper triangle matrix

Value

data

Examples

```
## Not run:
#'
data(satisfaction)
model1 <- "
IMAG =~ imag1 + imag2 + imag3 + imag4 + imag5
EXPE =~ expe1 + expe2 + expe3 + expe4 + expe5
QUAL =~ qual1 + qual2 + qual3 + qual4 + qual5
VAL =~ val1 + val2 + val3 + val4
SAT =~ sat1 + sat2 + sat3 + sat4
LOY =~ loy1 + loy2 + loy3 + loy4

EXPE ~ IMAG
QUAL ~ EXPE
VAL ~  QUAL + EXPE
SAT ~ IMAG + EXPE + QUAL + VAL
LOY ~ SAT + IMAG
"

sat_sem0 <- sem(model1, data = satisfaction)

#check
lav_htmt(sat_sem0)
# using data
lav_htmt(data= lav_return_data(sat_sem0), blocks= lav_return_blocks(sat_sem0))
#model data
lav_htmt(data= satisfaction, blocks =  lav_extract_mm(model1)%>% plspm_lav2blocks())
# using data
lav_htmt(data= satisfaction, blocks = model1)

#option
lav_htmt(sat_sem0, htmt2=T, cut=0.85)



## End(Not run)
```

---

lav_ind_hypo *Function that takes path data as input and outputs an indirect effect hypothesis.*

Description

Function that takes path data as input and outputs an indirect effect hypothesis.

Usage

```
lav_ind_hypo(paths)
```

Arguments

paths	indpath

Value

text

Examples

```
## Not run:
#'
# 모델 문자열
model1a <- "
EXPE ~ H1*IMAG
QUAL ~ H2*EXPE
VAL ~ H3*QUAL + H4*EXPE
SAT ~ H5*IMAG + H6*EXPE + H7*QUAL + H8*VAL
LOY ~ H9*SAT + H10*IMAG
"

model1 <- "
EXPE ~ IMAG
QUAL ~ EXPE
VAL ~ QUAL + EXPE
SAT ~ IMAG + EXPE + QUAL + VAL
LOY ~ SAT + IMAG
"
lav_extract_path(model1a)%>%
  lav_generate_ind_paths() %>%
  lav_ind_hypo()


## End(Not run)
```

---

lav_interact_model *lav_new_model: Generate interaction new model*

Description

lav_new_model: Generate interaction new model

Usage

```
lav_interact_model(lav_syn, imlist = NULL)
```

Arguments

lav_syn	original lavaan syntax model
imlist	interaction data lav_extract_imlist(model)

Value

model text

Examples

```
## Not run:
#'

# # Example usage:
data(MarshWenHau)
str(MarshWenHau)

mwh = MarshWenHau

model2 <- "
f1 =~ x1 + x2 + x3
f1Xf2 =~ f1:f2
f2 =~ x4 + x5 + x6
f3 =~ y1 + y2 + y3

f3 ~ f1 + f2
f3 ~ f1Xf2

f1Xf2 ~~ 0*f1
f1Xf2 ~~ 0*f2
"

# interaction model
intsem =  SEM(model2, data= MarshWenHau)
summary(intsem)
intsem %>%lav_semPaths2(color=list(lat="darkred",man="gold"))


##2nd
#base model
lav_extract_mm(model2) %>%cat()
#interaction item list
lav_extract_imlist(model3)
#data
lav_latentProd(mwh, model3)%>%head()

#extracted interaction term
lav_extract_int(model2, lav_extract_imlist(model3))

# or interaction items ; imlist
imlist <- list(
  .f1Xf2 = c("f1", "f2"),
  .i_f1_f4 = c("f1", "f4")
)
# names(imlist)
lav_extract_int(model2, imlist)
#direct
lav_extract_int(model2, imlist= list(.f1Xf2 = c("f1", "f2"),
                                     .i_f1_f4 = c("f1", "f4")))
#estimates model
intsem =  sem(lav_interact_model(model2, lav_extract_imlist(model2)),
```

```
                data= lav_latentProd(mwh, model2))



## End(Not run)
```

`lav_latentProd` *Functions to generate data for interaction terms in a dataset*

Description

Functions to generate data for interaction terms in a dataset

Usage

```
lav_latentProd(data, lav_syn, prefix = ".")
```

Arguments

`data`	original data
`lav_syn`	lavaan syntax
`prefix`	. prefix

Value

data.frame

Examples

```
## Not run:
#'

# # Example usage:
data(MarshWenHau)
str(MarshWenHau)

mwh = MarshWenHau

model2 <- "
f1 =~ x1 + x2 + x3
f1Xf2 =~ f1:f2
f2 =~ x4 + x5 + x6
f3 =~ y1 + y2 + y3

f3 ~ f1 + f2
f3 ~ f1Xf2

f1Xf2 ~~ 0*f1
f1Xf2 ~~ 0*f2
"
```

```
# interaction model
intsem =  SEM(model2, data= MarshWenHau)
summary(intsem)
intsem %>%lav_semPaths2(color=list(lat="darkred",man="gold"))


##2nd
#base model
lav_extract_mm(model2) %>%cat()
#interaction item list
lav_extract_imlist(model3)
#data
lav_latentProd(mwh, model3)%>%head()

#extracted interaction term
lav_extract_int(model2, lav_extract_imlist(model3))

# or interaction items ; imlist
imlist <- list(
  .f1Xf2 = c("f1", "f2"),
  .i_f1_f4 = c("f1", "f4")
)
# names(imlist)
lav_extract_int(model2, imlist)
#direct
lav_extract_int(model2, imlist= list(.f1Xf2 = c("f1", "f2"),
                                     .i_f1_f4 = c("f1", "f4")))
#estimates model
intsem =  SEM(lav_new_model(model2, lav_extract_imlist(model2)),
              data= lav_latentProd(mwh, model2))


## End(Not run)
```

`lav_matrix` *paper table to matrix*

Description

paper table to matrix

Usage

```
lav_matrix(text, type = "res")
```

Arguments

`text`	input copy data

Value

matrix

Examples

```
## Not run:

Input data Cho(2014)
text <- "필 요 성 1
         수 업 0.320** 1
         교 사 0.266** 0.647** 1
         계 획 0.506** 0.463** 0.383** 1
         영 향 0.429** 0.548** 0.491** 0.531** 1
         만 족 도 0.401** 0.554** 0.545** 0.491** 0.743** 1"

lav_matrix(text)

#Kim(2014)
kim2014text = "
자기주도성  1
컴퓨터효능감 .284** 1
학습설계  .369** .234** 1
상호작용  .124** .179** .361** 1
강사선호도  .267** .130** .423** .368** 1
학업성취도  .281** .197** .548** .306** .513** 1
동기변화  .246** .108* .453** .248** .405** .445** 1
자신감  .262** .092 .471** .149** .353** .482** .576** 1
흥미  .317** .149** .535** .272** .470** .571** .596** .628** 1"
#
lav_matrix(kim2014text)

# 마이너스가 포함되어 있는 경우
#정성령(2014)
Jungsy2014 = "
수강료 1
강사 .165** 1
수업 .303** .144** 1
개인관리 .494** .069 .284** 1
인지도 .425** .176** .407** .424** 1
이상적영향력 .210** -.056 .212** .286** .222** 1
영감적동기화 .190** -.133** .211** .290** .230** .659** 1
지적자극 .300** -.136** .159** .402** .238** .533** .517** 1
개별적 배려 .202** -.153** .150** .250** .291** .684** .618** .530** 1
"
lav_matrix(Jungsy2014)

Jungsy2014_cor = lav_matrix(Jungsy2014)
Jungsy2014_cor
#'
지경선(2016)
# 개인배경: 수학성적, 기대수준
# 선행학습실태: 학습시간, 학습진도, 반복횟수, 이해정도, 비용, 향후계획
# 선행학습유발요인:입시준비, 학벌사회,학교수업
# 수학학습태도: 흥미와동기, 학습자세, 수학효능감


Jiks2016="
수학성적 1
기대수준 .256*** 1
학습시간 .128** .079 1
```

```
학습진도 .304*** .146** .381*** 1
반복횟수 .115* .073 .155*** .173*** 1
이해정도 .150*** .283*** .142** .070 .236*** 1
비용 .132** .214*** .295*** .212*** .042 .086 1
향후계획 .228*** .368*** .134** .183*** .061 .302*** .226*** 1
입시준비 .201*** .300*** .155*** .249*** .121** .128*** .224*** .624*** 1
학벌사회 .097* .196*** .106* .176*** .063 -.032 .152*** .287*** .497*** 1
학교수업 .072 .258*** .120** .140** .087 .242*** .049 .292*** .235*** .098* 1
흥미동기 .357*** .358*** .113* .182*** .062 .350*** .175*** .514*** .359*** .182*** .373*** 1
학습자세 .257*** .441*** .177*** .193*** .101* .320*** .186*** .478*** .448*** .268*** .391*** .636***
수학효능감 .419*** .385*** .249*** .322*** .121** .373*** .182***
          .505*** .423*** .261*** .459*** .703*** .614*** 1
"
lav_matrix(Jiks2016, "check")
lav_matrix(Jiks2016)
#'
## data
textdf =  "motiv 400
harm      77  300
stabi     32   25   100
ppsych   -25  -25   -40    100
ses       25   26    40    -42 100
verbal    59   58    27    -16  18    100
read      53   42    56    -39  43     36  100
arith     60   44    49    -24  37     38   73   100
spell     59   45    48    -31  33     38   87    72   100"

lav_matrix(textdf)
lav_matrix(textdf,type= "all")

lav_matrix(textdf, "check", init=3)



## End(Not run)
```

`lav_modelfit` *lavaan model fit*

Description

lavaan model fit

Usage

```
lav_modelfit(..., nice_table = FALSE)
```

Arguments

`...`	sem models
`nice_table`	output model fit web

Value

data

Examples

```
## Not run:
#'
example(sem)
lav_modelfit(fit)
lav_modelfit(fit, nice_table = T)

lav_modelfit("model1"=fit,"model2"= fit)

#'

## End(Not run)
```

`lav_new_model` *lav_new_model: Generate interaction new model*

Description

lav_new_model: Generate interaction new model

Usage

```
lav_new_model(lav_syn, imlist = NULL)
```

Arguments

`lav_syn`	original lavaan syntax model
`imlist`	interaction data lav_extract_imlist(model)

Value

model text

Examples

```
## Not run:
#'

# # Example usage:
data(MarshWenHau)
str(MarshWenHau)

mwh = MarshWenHau

model2 <- "
f1 =~ x1 + x2 + x3
```

```
f1Xf2 =~ f1:f2
f2 =~ x4 + x5 + x6
f3 =~ y1 + y2 + y3

f3 ~ f1 + f2
f3 ~ f1Xf2

f1Xf2 ~~ 0*f1
f1Xf2 ~~ 0*f2
"

# interaction model
intsem =  SEM(model2, data= MarshWenHau)
summary(intsem)
intsem %>%lav_semPaths2(color=list(lat="darkred",man="gold"))


##2nd
#base model
lav_extract_mm(model2) %>%cat()
#interaction item list
lav_extract_imlist(model3)
#data
lav_latentProd(mwh, model3)%>%head()

#extracted interaction term
lav_extract_int(model2, lav_extract_imlist(model3))

# or interaction items ; imlist
imlist <- list(
  .f1Xf2 = c("f1", "f2"),
  .i_f1_f4 = c("f1", "f4")
)
# names(imlist)
lav_extract_int(model2, imlist)
#direct
lav_extract_int(model2, imlist= list(.f1Xf2 = c("f1", "f2"),
                                     .i_f1_f4 = c("f1", "f4")))
#estimates model
intsem =  SEM(lav_new_model(model2, lav_extract_imlist(model2)),
              data= lav_latentProd(mwh, model2))


## End(Not run)
```

`lav_Pattern` 응답패턴분석

Description

응답패턴분석

Usage

```
lav_Pattern(data, n = 2, dim = 0L, top = 20)
```

Arguments

data	data
n	구분되는개수
dim	0L
top	20

Value

data

Examples

```
## Not run:
#'
latentPattern(MarshWenHau, n=3)

HS9 <- HolzingerSwineford1939[,c("x1","x2","x3","x4","x5",
                                 "x6","x7","x8","x9")]
latentPattern(HS9)

#explain
HSbinary <- as.data.frame( lapply(HS9, cut, 2, labels=FALSE) )
HSbinary %>%head()
lavTables(HSbinary, dim = 0L, categorical = names(HSbinary)) %>%head(20)

#'
#'

x11()
latentPattern(park516_kor[,1:6], n=5) %>%
  ggplot(aes(x= fct_reorder(pattern, desc(obs.freq)), y= obs.freq))+
  geom_bar(stat="identity")+theme_bw()+
  theme(axis.text.x= element_text(size=14, angle=90))

## End(Not run)
```

---

lav_remodel *lav_remodel, Models that create indirect effects*

Description

lav_remodel, Models that create indirect effects

Usage

```
lav_remodel(
  model,
  prefix = "a",
  auto = TRUE,
```

```
  start_node = NULL,
  end_node = NULL,
  cat = FALSE,
  text = FALSE,
  interact = TRUE,
  paths_name = TRUE,
  n_name = 1,
  add_model = NULL
)
```

Arguments

model	lavaan(original model)
prefix	model hypothesis ”a”, or ”H”
auto	TRUe indirect effec auto, FALSE all path
start_node	NULL If you type it, it starts with
end_node	NULL and that’s it
cat	text puput console
text	Output text to save and type
interact	TRUe :(use) using SESM, FALSE (:) using sem()
paths_name	paths name
n_name	paste name number
add_model	add model syntax

Value

text

Examples

```
## Not run:
#'


data(satisfaction)

# Example usage
model1 <- "
IMAG =~ imag1 + imag2 + imag3 + imag4 + imag5
EXPE =~ expe1 + expe2 + expe3 + expe4 + expe5
QUAL =~ qual1 + qual2 + qual3 + qual4 + qual5
VAL =~ val1 + val2 + val3 + val4
SAT =~ sat1 + sat2 + sat3 + sat4
LOY =~ loy1 + loy2 + loy3 + loy4

EXPE ~ IMAG
QUAL ~ EXPE
VAL ~  QUAL+ EXPE
SAT ~ IMAG + EXPE + QUAL + VAL
LOY ~ SAT + IMAG
"
```

```
# lav_remodel(model1)
lav_remodel(model1)%>%cat("\n")
lav_remodel(model1, interact=FALSE)%>%cat("\n")

lav_remodel(model1, auto=FALSE)%>%cat("\n")

lav_remodel(model1, start="EXPE", prefix="H", cat=TRUE)
lav_remodel(model1, cat=TRUE)

lav_remodel(model1, cat=TRUE, text=TRUE)
lav_remodel(model1, cat=TRUE, text=FALSE)

lav_remodel(model1, cat=FALSE, text=TRUE)
lav_remodel(model1, cat=FALSE, text=FALSE)


satis_sem = sem(lav_remodel(model1), satisfaction)
summary(satis_sem)
model1 %>%class()



## End(Not run)
```

`lav_return_blocks` *lavaan:sem* 에서 *blocks* 을생성

Description

lavaan:sem 에서 blocks 을생성

Usage

```
lav_return_blocks(sem_res)
```

Arguments

`sem_res`	sem result

Value

list

Examples

```
## Not run:
#'
data(satisfaction)
```

```
model1 <- "
IMAG =~ imag1 + imag2 + imag3 + imag4 + imag5
EXPE =~ expe1 + expe2 + expe3 + expe4 + expe5
QUAL =~ qual1 + qual2 + qual3 + qual4 + qual5
VAL =~ val1 + val2 + val3 + val4
SAT =~ sat1 + sat2 + sat3 + sat4
LOY =~ loy1 + loy2 + loy3 + loy4

EXPE ~ IMAG
QUAL ~ EXPE
VAL ~  QUAL + EXPE
SAT ~ IMAG + EXPE + QUAL + VAL
LOY ~ SAT + IMAG
"

sat_sem0 <- sem(model1, data = satisfaction)

# 함수 결과 확인
lav_return_blocks(sat_sem0)

## End(Not run)
```

`lav_return_data` *lavaan* 결과에서데이터추출하는함수

Description

lavaan 결과에서데이터추출하는함수

Usage

```
lav_return_data(sem_res)
```

Arguments

`sem_res`	sem result

Value

data

Examples

```
## Not run:
#'
data(satisfaction)
model1 <- "
IMAG =~ imag1 + imag2 + imag3 + imag4 + imag5
EXPE =~ expe1 + expe2 + expe3 + expe4 + expe5
QUAL =~ qual1 + qual2 + qual3 + qual4 + qual5
VAL =~ val1 + val2 + val3 + val4
SAT =~ sat1 + sat2 + sat3 + sat4
```

```
LOY =~ loy1 + loy2 + loy3 + loy4

EXPE ~ IMAG
QUAL ~ EXPE
VAL ~  QUAL + EXPE
SAT ~ IMAG + EXPE + QUAL + VAL
LOY ~ SAT + IMAG
"

sat_sem0 <- sem(model1, data = satisfaction)

# 함수 결과 확인
lav_return_data(sat_sem0)
lav_return_data(sat_sem0)%>%head()

## End(Not run)
```

`lav_return_model` *lavaan::sem* 으로분석한함수에서 *model* 추출

Description

lavaan::sem 으로분석한함수에서 model 추출

Usage

```
lav_return_model(sem_res, sep = "\n")
```

Arguments

`sem_res`	sem data
`sep`	sep =

Value

text

Examples

```
## Not run:
#'
# 예시 사용법
data(satisfaction)
model1 <- "
IMAG =~ imag1 + imag2 + imag3 + imag4 + imag5
EXPE =~ expe1 + expe2 + expe3 + expe4 + expe5
QUAL =~ qual1 + qual2 + qual3 + qual4 + qual5
VAL =~ val1 + val2 + val3 + val4
SAT =~ sat1 + sat2 + sat3 + sat4
LOY =~ loy1 + loy2 + loy3 + loy4
```

```
EXPE ~ IMAG
QUAL ~ EXPE
VAL ~  QUAL + EXPE
SAT ~ IMAG + EXPE + QUAL + VAL
LOY ~ SAT + IMAG
"

sat_sem0 <- sem(model1, data = satisfaction)

# 함수 결과 확인
lav_return_model(sat_sem0)
lav_return_model(sat_sem0)%>%cat("\n")
#data
lav_return_data(sat_sem0)

#lav sem reuslt  model to blocks
lav_return_model(sat_sem0)%>%
  lav_extract_mm()%>%
  plspm_lav2blocks()



## End(Not run)
```

`lav_rotate_resid` *plspm_rotate_resid*

Description

plspm_rotate_resid

Usage

```
lav_rotate_resid(plotdata, resid_list = NULL, plot = TRUE)
```

Arguments

`plotdata`	plotdata
`resid_list`	resid list
`plot`	TRUE

Value

plot data

Examples

```
## Not run:
#'

pathmodel = "
```

```
자기효능감 =~ C_S1 +  C_S2 + C_S3 + C_S4 + C_S5
진로태도 =~ B02 + B04 + B10 + B14
진로동기 =~ A_M1 + A_M2 + A_M3
진로준비 =~ D_P1 + D_P2 + D_P3

진로동기 ~ H1*자기효능감
진로태도 ~  H2*자기효능감
진로준비 ~ H5 *진로태도
진로준비 ~  H4 * 진로동기
진로준비 ~  H3*자기효능감
"

plspm_diagram_model(pathmodel, layout="tree2",
                    rotation = 2,residuals = T, residScale = 6,
                    edge.label.position = 0.6)%>%
  plspm_rotate_resid(c(진로동기= 160, 진로태도= 160))

#
plspm_diagram_model(pathmodel, layout="tree2",
                    rotation = 2,residuals = T, residScale = 6,
                    edge.label.position = 0.6)%>%
  plspm_curve(c("진로준비~~ 진로동기" = -2))%>%
  lav_rotate_resid(c(진로동기= 160, 진로태도= 30))

## End(Not run)
```

`lav_semPaths` *Drawing Structural Equation Models Using the lavaan Syntax (park JH PhD)*

Description

Drawing Structural Equation Models Using the lavaan Syntax (park JH PhD)

Usage

```
lav_semPaths(
  lav_obj,
  whatLabels = "est",
  rotation = 2,
  residuals = TRUE,
  residScale = 12,
  layout = "tree2",
  edge.label.cex = 0.7,
  edge.label.position = 0.55,
  edge.color = "black",
  sizeMan = 6,
  sizeMan2 = 4,
  sizeLat = 9,
  sizeLat2 = 6,
  style = "lisrel",
```

```
  exoVar = FALSE,
  exoCov = TRUE,
  curve = 1.5,
  asize = 2,
  mar = c(3, 8, 3, 8),
  nDigits = 3,
  shapeLat = "circle",
  shapeMan = "rectangle",
  shapeInt = "triangle",
  border.width = 2,
  edge.width = 1.5,
  Groups = FALSE,
  group_match = "lat",
  structural = FALSE,
  edgeLabels = NULL,
  nodeLabels = NULL,
  fixedStyle = 1,
  intercept = FALSE,
  color = list(lat = "gray10", man = "white"),
  label.cex = 1,
  sig = TRUE,
  rotate_resid = NULL,
  se = FALSE,
  type = "plot"
)
```

Arguments

`whatLabels`	”model
`rotation`	rotation=2
`residuals`	residauls TRUE
`residScale`	residScale=12
`layout`	layout=”tree”
`edge.label.cex`	1
`edge.label.position`	0.5
`edge.color`	edge.color= black
`sizeMan`	8
`sizeMan2`	4
`sizeLat`	10
`sizeLat2`	6
`style`	lisrel
`exoVar`	FALSE
`exoCov`	TRUE
`curve`	1.5
`asize`	1.5
`mar`	c(2,8,3,10)

nDigits	2
shapeLat	”circle, ellipse
shapeMan	retangle
shapeInt	triangle ,
border.width	border.width=1
edge.width	edge.width=1.5
Groups	FALSE
group_match	”FALSE”lat#’
structural	structural = FALSE
edgeLabels	edgeLabels=NULL
nodeLabels	nodeLabels=NULL
fixedStyle	fixedStyle=1 default rbase linetype
color	list(lat=”gray10”, man=”white”)
label.cex	label.cex
sig	FALSE
rotate_resid	c(lv= angle), c(f1 =90, f2=180)
se	se=FALSE
type	type =”plot, and res is lavaan simdata result
plsres_boot	plsres_boot to model syntax

Value

plot jutpls_boot

Examples

```
## Not run:
#'

#data
data(MarshWenHau) # Marsh, Wen & Hau (2000)
# colnames(MarshWenHau) = c( paste0("x",1:6), paste0("y",1:3))
MarshWenHau %>%str()


model1 = "

f1 =~ x1 + x2 + x3
f2 =~ x4 + x5 + x6
f3 =~ y1 + y2 + y3
"

mwh_sem1 = sem(model1, MarshWenHau)
mwh_sem1%>%summary(fit=T)


re_mwh_data = lavPredict(mwh_sem1)%>%data.frame()
re_mwh_data
```

```
# 이방법인 LMS방법임

Intf1f2  = re_mwh_data[,"f1"] *  re_mwh_data[,"f2"]

#fitst
MarshWenHau$f1_f2 = Intf1f2
MarshWenHau %>% str()

model2 = "

f1 =~ x1 + x2 + x3
f2 =~ x4 + x5 + x6
f3 =~ y1 + y2 + y3

f1Xf2 =~ f1_f2

f3 ~ f1 + f2 + f1Xf2

f1Xf2 ~~ 0*f1
f1Xf2 ~~ 0*f2

"

mwh_sem2 = sem(model2, MarshWenHau)
mwh_sem2%>%summary(fit=T)
mwh_sem2%>% sem_effect()
x11()
mwh_sem2%>% lav_semPaths2(exoCov = FALSE, sig=F,
                          rotate_resid = c(f3 = 160))

mwh_sem2%>% lav_semPaths2(exoCov = FALSE, sig=TRUE, Groups=TRUE,
                          rotate_resid = c(f3 = 160))

mwh_sem2%>% lav_semPaths2(exoCov = FALSE, sig=TRUE,se=TRUE,
                          rotate_resid = c(f3 = 160))


#'


## End(Not run)
```

`lav_semPaths2` *Drawing Structural Equation Models Using the lavaan Syntax (park JH PhD)*

Description

Drawing Structural Equation Models Using the lavaan Syntax (park JH PhD)

Usage

```
lav_semPaths2(
  lav_obj,
  whatLabels = "est",
  rotation = 2,
  residuals = TRUE,
  residScale = 12,
  layout = "tree2",
  edge.label.cex = 0.7,
  edge.label.position = 0.55,
  edge.color = "black",
  sizeMan = 6,
  sizeMan2 = 4,
  sizeLat = 9,
  sizeLat2 = 6,
  style = "lisrel",
  exoVar = FALSE,
  exoCov = TRUE,
  curve = 1.5,
  asize = 2,
  mar = c(3, 8, 3, 8),
  nDigits = 3,
  shapeLat = "circle",
  shapeMan = "rectangle",
  shapeInt = "triangle",
  border.width = 2,
  edge.width = 1.5,
  Groups = FALSE,
  group_match = "lat",
  structural = FALSE,
  edgeLabels = NULL,
  nodeLabels = NULL,
  fixedStyle = 1,
  intercept = TRUE,
  color = list(lat = "gray10", man = "white"),
  label.cex = 1,
  sig = TRUE,
  rotate_resid = NULL,
  se = FALSE,
  type = "plot"
)
```

Arguments

whatLabels	”model
rotation	rotation=2
residuals	residauls TRUE
residScale	residScale=12
layout	layout=”tree”
edge.label.cex	1

edge.label.position	0.5
edge.color	edge.color= black
sizeMan	8
sizeMan2	4
sizeLat	10
sizeLat2	6
style	lisrel
exoVar	FALSE
exoCov	TRUE
curve	1.5
asize	1.5
mar	c(2,8,3,10)
nDigits	2
shapeLat	”circle, ellipse
shapeMan	retangle
shapeInt	triangle ,
border.width	border.width=1
edge.width	edge.width=1.5
Groups	FALSE
group_match	”FALSE”lat#’
structural	structural = FALSE
edgeLabels	edgeLabels=NULL
nodeLabels	nodeLabels=NULL
fixedStyle	fixedStyle=1 default rbase linetype
intercept	intercept
color	list(lat=”gray10”, man=”white”)
label.cex	label.cex
sig	FALSE
rotate_resid	c(lv= angle), c(f1 =90, f2=180)
se	se=FALSE
type	type =”plot, and res is lavaan simdata result
plsres_boot	plsres_boot to model syntax

Value

plot jutpls_boot

Examples

```
## Not run:
#'

#data
data(MarshWenHau) # Marsh, Wen & Hau (2000)
# colnames(MarshWenHau) = c( paste0("x",1:6), paste0("y",1:3))
MarshWenHau %>%str()


model1 = "

f1 =~ x1 + x2 + x3
f2 =~ x4 + x5 + x6
f3 =~ y1 + y2 + y3
"

mwh_sem1 = sem(model1, MarshWenHau)
mwh_sem1%>%summary(fit=T)


re_mwh_data = lavPredict(mwh_sem1)%>%data.frame()
re_mwh_data



# 이방법인 LMS방법임

Intf1f2  = re_mwh_data[,"f1"] *  re_mwh_data[,"f2"]

#fitst
MarshWenHau$f1_f2 = Intf1f2
MarshWenHau %>% str()

model2 = "

f1 =~ x1 + x2 + x3
f2 =~ x4 + x5 + x6
f3 =~ y1 + y2 + y3

f1Xf2 =~ f1_f2

f3 ~ f1 + f2 + f1Xf2

f1Xf2 ~~ 0*f1
f1Xf2 ~~ 0*f2

"

mwh_sem2 = sem(model2, MarshWenHau)
mwh_sem2%>%summary(fit=T)
mwh_sem2%>% sem_effect()
x11()
mwh_sem2%>% lav_semPaths2(exoCov = FALSE, sig=F,
                          rotate_resid = c(f3 = 160))
```

```
mwh_sem2%>% lav_semPaths2(exoCov = FALSE, sig=TRUE, Groups=TRUE,
                          rotate_resid = c(f3 = 160))

mwh_sem2%>% lav_semPaths2(exoCov = FALSE, sig=TRUE,se=TRUE,
                          rotate_resid = c(f3 = 160))


#'


## End(Not run)
```

---

layout_mat *layout_mat*

Description

layout_mat

Usage

```
layout_mat(semplot, text = TRUE, byrow = TRUE, variable = TRUE)
```

Arguments

semplot	semPaths2 data
text	text True output text matrix
byrow	byrow TRUE
variable	variable is rownames adapt

Value

data

Examples

```
## Not run:
#'
data(offense)
offense %>%str()

nfl_path = matrix(
  c(0, 0, 0, 0, 0,
    0, 0, 0, 0, 0,
    0, 0, 0, 0, 0,
    0, 1, 1, 0, 0,
    1, 0, 0, 1, 0),
  nrow = 5, ncol = 5, byrow = TRUE )

rownames(nfl_path) <- c("Speical", "Rushing", "Passing", "Offense", "Scoring")
```

```
colnames(nfl_path) <- c("Speical", "Rushing", "Passing", "Offense", "Scoring")
nfl_path

nfl_blocks = list(7:8, 1:3, 4:6,1:6, 9:11)

plspm_blocks_match(data=offense, blocks=nfl_blocks,
                   paths=nfl_path)

nfl_mode =c("B","A","A","A","A")

nfl_pls = plspm(Data=offense, nfl_path, nfl_blocks, modes = nfl_mode)
nfl_pls%>%summary()

#########
nfl_pls_boot = plspm_sem(Data=offense, nfl_path, nfl_blocks, modes = nfl_mode)

#Layouts extracted from mockups
plspm_semPaths2(nfl_pls_boot,sizeLat = 6)%>%
  plspm_factor_layout()

#Make it input material
plspm_semPaths2(nfl_pls_boot,sizeLat =6)%>%
  plspm_factor_layout() %>%layout_mat(text=T)

#Organizing materials for input
plspm_semPaths2(nfl_pls_boot,sizeLat =6)%>%
  plspm_factor_layout() %>%layout_mat()%>%
  mat_text_arrange()

## End(Not run)
```

`lcaGraphData` *lcaGraphData, lcadata to graph data*

Description

lcaGraphData, lcadata to graph data

Usage

```
lcaGraphData(lcaResult, ..., yes_col = 3, yes = `Y = 2`)
```

Arguments

`lcaResult`	lcadata
`...`	iteam names, nothng auto
`yes_col`	yes col , default 3
`yes`	yes name default ‘Y = 2‘

Value

data

Examples

```
## Not run:
#'
#함수를 이용하여 그래프 데이터 생성
jutLca3 %>%lcaGraphData()

jutLca3 %>%lcaGraphData( "학업성적"="L05",
                         "부모지지"="L04",
                         "진로결정"="L03",
                         "준비미흡"="L02",
                         "목표확실"="L01")


## End(Not run)
```

`lca_class_add` *lca_class_add,* 기존데이터에잠재클래스추가

Description

lca_class_add, 기존데이터에잠재클래스추가

Usage

```
lca_class_add(data, lcadata)
```

Arguments

`data`	df
`lcadata`	lcadata

Value

data

Examples

```
## Not run:

jut6 %>% lca_class_add(jutLca3) %>% head()

## End(Not run)
```

lca_class_freq *lca_class_freq cal*

Description

lca_class_freq cal

Usage

```
lca_class_freq(lacdata, show = "")
```

Arguments

lacdata	lca result
show	df, ""

Value

df

Examples

```
## Not run:

 jutLca3 %>%lca_class_freq()%>% data.frame()

## End(Not run)
```

lca_class_plot *lca_class_plot,* 잠재클래스패턴그래프

Description

lca_class_plot, 잠재클래스패턴그래프

Usage

```
lca_class_plot(lcadata_graph, type = "g")
```

Arguments

lcadata_graph	lcaGraphData 데이터처리된것
type	g, full, line

Value

plot

Examples

```
## Not run:
#'
## using
jutLca3 %>%lcaGraphData() %>% lca_class_plot()
jutLca3 %>%lcaGraphData() %>% lca_class_plot("line")

jutLca3 %>%lcaGraphData( "학업성적"="L05",
                         "부모지지"="L04",
                         "진로결정"="L03",
                         "준비미흡"="L02",
                         "목표확실"="L01") %>%
  lca_class_plot()
#full graph
jutLca3 %>%lcaGraphData( "학업성적"="L05",
                         "부모지지"="L04",
                         "진로결정"="L03",
                         "준비미흡"="L02",
                         "목표확실"="L01") %>%
  replace_df_rep(
    "L01", "L01:목표확실",
    "L02", "L02:준비미흡",
    "L03", "L03:진로결정",
    "L04", "L04:부모지지",
    "L05", "L05:학업성적")%>%
  replace_df_rep(
    "Class1", "Class1:적극적 진로준비형",
    "Class2", "Class2:불명확 진로인식형",
    "Class3", "Class3:조건부 진로지향형"
  )%>%lca_class_plot("line")


## End(Not run)
```

`lca_freq_plot` *lca_freq_plot LCA* 빈도그래프

Description

lca_freq_plot LCA 빈도그래프

Usage

```
lca_freq_plot(data, lcadata)
```

Arguments

`data`	df
`lcadata`	lcadata

Value

plot

Examples

```
## Not run:

jut7c %>% lca_freq_plot(jutLca3)+ ylim(0, 380)

## End(Not run)
```

`lca_gamma` *lca_gamma* 사전확률

Description

lca_gamma 사전확률

Usage

```
lca_gamma(lcadata)
```

Arguments

`lcadata`	lcadata

Value

vedtor

Examples

```
## Not run:
jutLca3%>% lca_gamma()

## End(Not run)
```

`lca_gof` *lca_gof*

Description

lca_gof

Usage

```
lca_gof(...)
```

Arguments

`...`	models

Value

data

Examples

```
## Not run:
#'
# 함수 사용 예시
lca_gof(jutLca2, jutLca3, jutLca4)

lca_gof(jutLca3)%>%row2col()

## End(Not run)
```

`lca_GraphData` *lcaGraphData, lcadata to graph data*

Description

lcaGraphData, lcadata to graph data

Usage

```
lca_GraphData(lcaResult, ..., yes_col = 3, yes = `Y = 2`)
```

Arguments

`lcaResult`	lcadata
`...`	iteam names, nothng auto
`yes_col`	yes col , default 3
`yes`	yes name default 'Y = 2'

Value

data

Examples

```
## Not run:
#'
#함수를 이용하여 그래프 데이터 생성
jutLca3 %>%lcaGraphData()

jutLca3 %>%lcaGraphData( "학업성적"="L05",
                         "부모지지"="L04",
                         "진로결정"="L03",
                         "준비미흡"="L02",
                         "목표확실"="L01")


## End(Not run)
```

`lca_result` *lca_result lca* 분석후결과정리

Description

lca_result lca 분석후결과정리

Usage

```
lca_result(lcadata, class_name = NULL)
```

Arguments

`lcadata`	lcadata
`class_name`	input name

Value

data

Examples

```
## Not run:
#'
jutLca3 %>%lca_result() %>%
  replace_df_rep( "L01", "진로목표(L01)",
                  "L02", "준비여부(L02)",
                  "L03", "직업결정(L03)",
                  "L04", "부모추천(L04)",
                  "L05", "학업성적(L05)" ) #변수명을 바꾸기

jutLca3 %>%lca_result(class_name=c(1,2,3))

jutLca3 %>%lca_result(class_name=c("조건부_진로지향형\n(Conditional-Oriented)",
                                   "불명확_진로인식형\n(Unclear-Awareness)",
                                   "적극적_진로준비형\n(Active-Preparedness)")


                      jutLca3 %>%lca_result(class_name=c("조건부_진로지향형",
                                                         "불명확_진로인식형",
                                                         "적극적_진로준비형"))


## End(Not run)
```

`lme_report` *linear mixed model report*

Description

Mixed model summary

Usage

```
lme_report(
  lmedata,
  type = "basic",
  form = "lmer",
  apa = FALSE,
  fit_more = FALSE,
  ranef_sig = FALSE,
  show.effect = FALSE,
  show.ci = FALSE
)
```

Arguments

`lmedata`	lmedata is lmer() function result
`type`	= 'all' is res, 'Fixed_effect','Random_effect','ICC', 'ConfidenceInterval_95','Satterthwaite_r
`form`	'lmer','glmer' = 'logit'='dich', 'poisson' ='pois'
`apa`	= default FALSE, if you REML is FALSE
`fit_more`	= default FALSE, TRUE detailed report
`ranef_sig`	ranef_sig =TRUE random effect test
`show.effect`	if you show.effect = TRUE -> show ranef, fixef, coef
`show.ci`	TRUE calculation 95percent CI

Examples

```
## Not run:
data(sleepstudy)

sleepstudy |> str()

lmer(Reaction ~ Days + (Days | Subject), sleepstudy) |> lme_report()

lmer(Reaction ~ Days + (Days | Subject), sleepstudy) |> lme_report(apa=TRUE)


book.url <- "https://stat.ethz.ch/~meier/teaching/book-anova"
quality <- readRDS(url(file.path(book.url, "data/quality.rds")))
str(quality)
fit.quality <- lmer(score ~ (1 | employee) + (1 | batch) +
                      (1 | employee:batch), data = quality)
summary(fit.quality)
fit.quality|> lme_report()
fit.quality|> lme_report(apa= TRUE) #error the app is FALSE
```

```
fit.quality|> lme_report(apa=FALSE) #error the app is FALSE
fit.quality|> ICC()

lmer(Reaction ~ Days + (Days | Subject), sleepstudy) |>
SEM212::lme_report(apa=TRUE, fit_more = T)
lmer(Reaction ~ Days + (Days | Subject), sleepstudy) |>
SEM212::lme_report(apa=TRUE, fit_more = F)  #default FALSE, it is show that AIC and BIC


## End(Not run)
```

`long_df` *longdata transfomation*

Description

longdata transfomation

Usage

```
long_df(
  data,
  rowname = "rownames",
  names_to = "names",
  values_to = "values",
  cols = NULL,
  fix = NULL,
  rownames_to_column = TRUE
)
```

Arguments

`data`	widedata
`rowname`	defailt accent
`names_to`	name, speaker(kge)
`values_to`	value, freq
`cols`	colrange
`fix`	column fixed (thereofore not using cols)

Value

longdata

Examples

```
## Not run:

mtcars[,1:4] %>%long_df("car", "value")
mtcars[1:3, 1:7] %>% long_df(fix= 1:3)
mtcars[1:3, 1:7] %>% long_df(fix= 2:7)
mtcars[1:3, 1:7] %>% long_df(cols= 3:7, rownames_to_column=F)
```

```
mtcars[1:3, 1:7] %>% long_df(cols= 3:7)
mtcars[1:3, 1:7] %>% long_df(cols= 5:7)


 mtcars[,1:6] %>%long_df("car", "value", rownames_to_column = F, cols = c(1,3:6))



## End(Not run)
```

`lowerdf` *df or mat lower tri*

Description

df or mat lower tri

Usage

```
lowerdf(data)
```

Arguments

`data`	data

Value

matrix

`lowerMat` *lower pull matrix*

Description

lower pull matrix

Usage

```
lowerMat(mat, fill = 0, diag = 0, digits = 3)
```

Arguments

`mat`	matrix data
`fill`	matrix data fill upper 0 or if that's what you want , enter it
`diag`	matrix data diag enter 0

Value

lowermatrix, upper 0

lowerMat_na *lower pull matrix fill NA*

Description

lower pull matrix fill NA

Usage

```
lowerMat_na(
  mat,
  fill = "",
  diag = 1,
  numeric = FALSE,
  digits = 3,
  justify = "right"
)
```

Arguments

mat	matrix data
fill	matrix data NA or .
diag	matrix diagonal data NA or .

Value

lowermatrix, upper 0

lpa *Latent Profile Analysis (LPA) Function*

Description

This function performs latent profile analysis (LPA) using the ‘mclust‘ package, creates visualizations, and returns summary statistics and plots.

Usage

```
lpa(
  df,
  profile = 4,
  model = c("EII", "VII", "EEI", "VEI", "EVI", "VVI", "EEE", "EVE", "VEE", "VVE", "EEV",
    "VEV", "EVV", "VVV"),
  seed = 123,
  size_axis = 14,
  size_strip = 14,
  name_strip = NULL,
  md = FALSE
)
```

Arguments

`df`	Data frame containing the numeric variables for LPA.
`profile`	Integer specifying the number of profiles (clusters). Default is 4.
`model`	Character vector specifying the covariance model to use in the analysis. Options include "EII", "VII", "EEI", "VEI", "EVI", "VVI", "EEE", "EVE", "VEE", "VVE", "EEV", "VEV", "EVV", "VVV". Default is "EEE".
`seed`	Integer for setting the random seed for reproducibility. Default is 123.
`size_axis`	Numeric value specifying the size of the axis text in plots. Default is 14.
`size_strip`	Numeric value specifying the size of the strip text in plots. Default is 14.
`name_strip`	Optional vector of names for customizing profile strip labels.
`md`	Logical indicating whether to convert the combined data to markdown format. Default is FALSE.

Value

A list containing:

`summary_table`	Data frame with raw means, frequencies, and proportions for each profile.
`scaled_summary_table`	Data frame with standardized means, frequencies, and proportions for each profile.
`profile_frequencies`	Data frame with profile frequencies and proportions.
`combined_data`	Combined data in either raw or markdown format.
`mclust_object`	Summary of the Mclust object from the analysis.
`raw_plot`	Plot of raw scores by profile.
`scaled_plot`	Plot of standardized scores by profile.
`combined_plot`	Combined plot of raw and standardized scores.

Examples

```
## Not run:
data <- mtcars[, c("mpg", "disp", "hp", "drat", "wt")]
result <- lpa(data, profile = 3, model = "EEE", seed = 123)
result$raw_plot
result$scaled_plot
result$combined_data

## End(Not run)
```

`lpa_BIC_plot` *lpa_BIC_plot: Visualize BIC Values for Latent Profile Analysis*

Description

This function generates a plot to visualize BIC values from latent profile analysis.

Usage

```
lpa_BIC_plot(
  data,
  n_profiles_range = 1:6,
  type = "all",
  legend_position = "top",
  covar_model = c("EII", "EEE", "VII", "EEV", "EEI", "VVI", "VVV"),
  set_n_profile = NULL,
  xintercept = 4,
  size.text = 4,
  flip = TRUE,
  na2zero = FALSE,
  new.window = TRUE,
  basic = TRUE
)
```

Arguments

`data`	A data frame containing BIC values for different models and profiles.
`n_profiles_range`	A numeric vector specifying the range of profiles to analyze.
`type`	Type of output to return. Options include "all", "res", "data", "long_data", "wide_data", or "graph".
`legend_position`	The position of the legend in the plot (e.g., "top", "bottom").
`covar_model`	A character vector specifying covariance models to include in the analysis.
`set_n_profile`	A fixed number of profiles to use in the analysis (optional).
`xintercept`	The x-axis intercept line for visualization.
`size.text`	The size of text labels in the plot.
`flip`	Logical. If TRUE, flips the y-axis values for visualization. Default is TRUE.
`na2zero`	Logical. If TRUE, replaces NA values with zero. Default is FALSE.
`new.window`	Logical. If TRUE, opens a new graphics window for the plot. Default is TRUE.
`basic`	Logical. If TRUE, uses only basic covariance models (e.g., "EII", "EEE"). Default is TRUE.

Value

A list containing:

`data`	The wide-format data used for plotting.
`graph`	The ggplot object for visualization.
`to_plot`	The long-format data used for plotting.

Examples

```
## Not run:
# Example usage of lpa_BIC_plot:
lpa_BIC_plot(data, n_profiles_range = 1:6, basic = FALSE, set_n_profile = 4)

## End(Not run)
```

`lpa_bind_classdata` *lpa_bind_classdata*

Description

lpa_bind_classdata

Usage

```
lpa_bind_classdata(
  data,
  n_profiles = 3,
  models = NULL,
  package = c("mclust"),
  type = "all",
  add_line = TRUE,
  var_model = "EII",
  Class_colors = 1:4,
  linewidth = 0.9,
  sd = TRUE,
  set_seed = 123
)
```

Arguments

`data`	data
`n_profiles`	default 3
`package`	mclucs , MplusAutomation
`type`	all, res, fit, est, data, all, plot, group_plot
`add_line`	add_lineTRUE
`var_model`	”EEE”,”EEI,”VVV”,”EII”
`Class_colors`	1:4
`linewidth`	linewidth 0.9
`sd`	TRUE graph

Value

data

Examples

```
## Not run:

#'
lpa_bind_classdata(select(iris, -Species))

lpa_bind_classdata(select(iris, -Species), type="res")
lpa_bind_classdata(select(iris, -Species), type="fit")
lpa_bind_classdata(select(iris, -Species), type="est")
lpa_bind_classdata(select(iris, -Species), type="data")%>%Round(3)%>%dall
lpa_bind_classdata(select(iris, -Species), type="all")
#'
# "EEE": variances = "equal", covariances = "equal"
# 프로파일 간 분산과 공분산이 모두 동일하다고 가정.
# "EEV": variances = "equal", covariances = "varying"
# 분산은 동일하지만, 공분산은 프로파일마다 다르게 추정.
# "EEI": variances = "equal", covariances = "zero"
# 분산은 동일하지만, 공분산은 추정하지 않음 (공변량 간 독립성 가정).
# "EII": variances = "equal", covariances = "zero"
# 동일한 가정으로 분산이 동일하고, 공분산은 추정되지 않음.

## End(Not run)
```

---

lpa_bind_classdata_tidyLPA

lpa_bind_classdata_tidyLPA

Description

lpa_bind_classdata_tidyLPA

Usage

```
lpa_bind_classdata_tidyLPA(
  data,
  n_profiles = 3,
  models = NULL,
  package = c("mclust", "MplusAutomation"),
  type = "all",
  add_line = TRUE,
  var_model = "EEE",
  Class_colors = 1:4,
  linewidth = 0.9,
  sd = TRUE
)
```

Arguments

`data`	data
`n_profiles`	default 3
`package`	mclucs , MplusAutomation
`type`	all, res, fit, est, data, all, plot, group_plot
`add_line`	add_lineTRUE
`var_model`	”EEE”,”EEI,”VVV”,”EII”
`Class_colors`	1:4
`linewidth`	linewidth 0.9
`sd`	TRUE graph

Value

data

Examples

```
## Not run:

#'
lpa_bind_classdata(select(iris, -Species))

lpa_bind_classdata(select(iris, -Species), type="res")
lpa_bind_classdata(select(iris, -Species), type="fit")
lpa_bind_classdata(select(iris, -Species), type="est")
lpa_bind_classdata(select(iris, -Species), type="data")%>%Round(3)%>%dall
lpa_bind_classdata(select(iris, -Species), type="all")
#'
# "EEE": variances = "equal", covariances = "equal"
# 프로파일 간 분산과 공분산이 모두 동일하다고 가정.
# "EEV": variances = "equal", covariances = "varying"
# 분산은 동일하지만, 공분산은 프로파일마다 다르게 추정.
# "EEI": variances = "equal", covariances = "zero"
# 분산은 동일하지만, 공분산은 추정하지 않음 (공변량 간 독립성 가정).
# "EII": variances = "equal", covariances = "zero"
# 동일한 가정으로 분산이 동일하고, 공분산은 추정되지 않음.

## End(Not run)
```

`lpa_create_profiles` *lpa_create_profiles*

Description

lpa_create_profiles

Usage

```
lpa_create_profiles(
  df,
  n_profiles = 3,
  variance_structure = "freed",
  covariance_structure = "freed"
)
```

Arguments

df	df
n_profiles	default = 3
variance_structure	freed
covariance_structure	freed

Value

data

Examples

```
## Not run:
#'
iris[,-5]%>% lpa_create_profiles(n_profiles =  4,
                                 variance_structure = "constrained",
                                 covariance_structure = "constrained")

## End(Not run)
```

---

lpa_create_profiles2 *lpa_create_profiles2*

Description

lpa_create_profiles2

Usage

```
lpa_create_profiles2(
  df,
  n_profiles = 4,
  model_name = NULL,
  type = "mean",
  seed = 123
)
```

Arguments

`df`	data
`n_profiles`	3, ...
`model_name`	EEI, VVI, EEE, VVV

Value

data

Examples

```
## Not run:
#'

iris[,-5] %>% lpa_create_profiles2(n_profiles = 4)

#v:constrained, Co: fixed
iris[,-5] %>% lpa_create_profiles2(n_profiles = 4, model_name="EEE")
#variance_structure == "fixed"
iris[,-5] %>% lpa_create_profiles2(n_profiles = 4, model_name="VVV")


# variance_structure == "freed" &covariance_structure == "fixed"
iris[,-5] %>% lpa_create_profiles2(n_profiles = 4, model_name="VVI")


#variance_structure == "constrained" & covariance_structure == "fixed"
iris[,-5] %>% lpa_create_profiles2(n_profiles = 4, model_name="EEI")
#'

## End(Not run)
```

`lpa_explore_modelfit` *lpa_explore_modelfit*

Description

lpa_explore_modelfit

Usage

```
lpa_explore_modelfit(
  df,
  n_profiles_range = 1:9,
  modelNames = c("EEV", "EEI", "EEE", "EII", "VVI", "VII", "VVV"),
  chk = FALSE,
  basic = FALSE,
  set_n_profile = NULL,
  na2zero = TRUE
)
```

Arguments

df	data
n_profiles_range	1:9
modelNames	"EEV", "EEI", "EEE", "EII", "VVI", "VII", "VVV"
chk	default FALSE, TRUE show
basic	select EII, EEE, VII, EEI
set_n_profile	default select row number then result show
na2zero	na2zero is NA data to 0

Value

data

Examples

```
## Not run:

df <- select(iris, -Species)
lpa_explore_modelfit(df, 1:4)
#' lpa_explore_modelfit(df, 1:9)
lpa_explore_modelfit(pwjlpa,1:4, basic=TRUE)
lpa_explore_modelfit(pwjlpa,1:4, basic=TRUE, set_n_profile=4)
lpa_explore_modelfit(pwjlpa,1:6, basic=TRUE)

lpa_explore_modelfit(pwjlpa,1:6, basic=FALSE)
lpa_explore_modelfit(pwjlpa,1:10, basic=TRUE, set_n_profile=4)
lpa_explore_modelfit(pwjlpa,1:10, basic=TRUE, set_n_profile=4, chk=TRUE)


## End(Not run)
```

---

lpa_gmm *lpa_gmm: Latent Profile Analysis using Gaussian Mixture Models*

Description

This function performs latent profile analysis (LPA) using Gaussian Mixture Models (GMM). It provides model parameters, profile classifications, and visualizations for the analysis.

Usage

```
lpa_gmm(
  df,
  profiles = 4,
  modelNames = "EII",
  g_type = c("uncertainty", "classification")
)
```

Arguments

df	A data frame containing the variables to be analyzed.
profiles	An integer specifying the number of profiles to extract. Default is 4.
modelNames	A character string specifying the GMM covariance model. Default is "EII".
g_type	A character string specifying the type of GMM plot to generate. Choices are "uncertainty" or "classification". Default is "uncertainty".

Value

A list containing:

result The GMM parameters for the fitted model.

lpa_reuslt A summary table of profile means and frequencies.

graph The GMM plot based on the specified type.

profile_plot A bar plot visualizing profile frequencies.

table A formatted markdown table summarizing profile results.

Examples

```
## Not run:
# Example usage of lpa_gmm
library(mclust)
library(dplyr)
library(ggplot2)

# Example data
set.seed(123)
df <- data.frame(x = rnorm(100), y = rnorm(100))

# Perform LPA with 3 profiles
lpa_results <- lpa_gmm(df, profiles = 3)

# View results
print(lpa_results$lpa_reuslt)
plot(lpa_results$graph)

## End(Not run)
```

---

lpa_plot *Latent Profile Analysis Profile Plot*

Description

Latent Profile Analysis Profile Plot

Usage

```
lpa_plot(
  data,
  n_profiles = 3,
  model_name = "EEE",
  view = "pair",
  alpha = 0.6,
  gtype = "normal",
  angle = 90,
  size.x = 13,
  size.p = 5,
  ncol = 2,
  fct_reorder = NULL,
  show.legend = FALSE,
  legend.text.size = 16,
  legend_position = "top",
  linewidth = 1,
  title_size = 18,
  seed = 123
)
```

Arguments

`data`	A data frame containing numeric variables for latent profile analysis.
`n_profiles`	Integer. Number of latent profiles to identify. Default is 3.
`model_name`	Character. Model name to use in Mclust for Gaussian mixture modeling (e.g., "EEE", "EEI", "VVI", "VVV"). Default is "EEE".
`view`	Character. Determines the view type of the plots ("pair", "each"). Default is "pair".
`alpha`	Numeric. Transparency level for bar plots. Default is 0.6.
`gtype`	Character. Graph type to generate ("normal", "trans", "each"). Default is "normal".
`angle`	Numeric. Angle for the x-axis text labels. Default is 90.
`size.x`	Numeric. Font size for the x-axis text labels. Default is 13.
`size.p`	Numeric. Size of points in the plot. Default is 5.
`ncol`	Integer. Number of columns for the facet plots in "each" view. Default is 2.
`fct_reorder`	Character vector or NULL. Specifies the order of factor levels for plotting. Default is NULL.
`show.legend`	Logical. Whether to show legends for line plots. Default is FALSE.
`legend.text.size`	Numeric. Font size for legend text. Default is 16.
`legend_position`	Character. Position of the legend ("top", "bottom", "left", "right"). Default is "top".
`linewidth`	Numeric. Line width for plots. Default is 1.
`title_size`	Numeric. Font size for plot titles. Default is 18.
`seed`	Integer or NULL. Seed value for reproducibility. Default is 123.

Value

A list containing:

`std`	Standardized mean data for each profile.
`est`	Estimated raw mean data for each profile.
`graph`	Combined plots for the profiles.
`plot_data`	Data used for plotting.
`Class_freq`	Frequency table of profile classifications.

Examples

```
## Not run:
# Example usage with iris dataset
result <- lpa_plot(iris[, -5], n_profiles = 3, model_name = "EEE")
print(result$graph)

## End(Not run)
```

`lpa_profile_plot` *Latent Profile Analysis (LPA) Profile Plot Function*

Description

This function generates plots to visualize latent profile analysis results, including raw scores and standardized scores for profiles.

Usage

```
lpa_profile_plot(
  data,
  n_profiles = 3,
  model_name = "EEE",
  view = "pair",
  alpha = 0.6,
  gtype = "normal",
  angle = 90,
  size.x = 13,
  size.p = 5,
  ncol = 2,
  fct_reorder = NULL,
  show.legend = FALSE,
  legend.text.size = 16,
  legend_position = "top",
  linewidth = 1,
  title_size = 18,
  seed = 123
)
```

Arguments

data	A data frame containing numeric variables for latent profile analysis.
n_profiles	Integer. Number of latent profiles to identify. Default is 3.
model_name	Character. Model name for Gaussian mixture modeling in Mclust (e.g., "EEE", "EEI", "VVI", "VVV"). Default is "EEE".
view	Character. Determines the view type of the plots ("pair", "each"). Default is "pair".
alpha	Numeric. Transparency level for bar plots. Default is 0.6.
gtype	Character. Type of graph to generate ("normal", "trans"). Default is "normal".
angle	Numeric. Angle for the x-axis text labels. Default is 90.
size.x	Numeric. Font size for the x-axis text labels. Default is 13.
size.p	Numeric. Size of points in the plot. Default is 5.
ncol	Integer. Number of columns for grid-arranged plots. Default is 2.
fct_reorder	Character vector or NULL. Specifies the order of factor levels for plotting. Default is NULL.
show.legend	Logical. Whether to show legends for line plots. Default is FALSE.
legend.text.size	Numeric. Font size for legend text. Default is 16.
legend_position	Character. Position of the legend ("top", "bottom", "left", "right"). Default is "top".
linewidth	Numeric. Line width for line plots. Default is 1.
title_size	Numeric. Font size for plot titles. Default is 18.
seed	Integer or NULL. Seed value for reproducibility. Default is 123.

Value

A list containing:

std	Standardized mean data for each profile.
est	Estimated raw mean data for each profile.
graph	Combined plots for the profiles.
plot_data	Data used for plotting.
Class_freq	Frequency table of profile classifications.

Examples

```
## Not run:
# Example usage with the iris dataset
result <- lpa_profile_plot(iris[, -5], n_profiles = 3, model_name = "EEE", gtype = "normal")
print(result$graph)

## End(Not run)
```

`lpa_profile_plot2` *profile_plot2 Brargrahp*

Description

profile_plot2 Brargrahp

Usage

```
lpa_profile_plot2(data, n_profiles = 3, model_name = NULL, view = "pair")
```

Arguments

`data`	data
`n_profiles`	profiles number
`model_name`	”EEE”,”EEI”, ”VVV”,”VVI”,
`view`	each, pair

Value

plot

Examples

```
## Not run:

iris[,-5] %>%profile_plot2(n_profiles = 4, model_name = "EEE")


## End(Not run)
```

`lpa_profile_plot_summary` *lpa_profile_plot summary data*

Description

lpa_profile_plot summary data

Usage

```
lpa_profile_plot_summary(resl, digits = 3)
```

Arguments

`resl`	result
`digits`	default 3

Value

df

Examples

```
## Not run:
#'
lpa_profile_plot(edlcarename, n_profiles = 4, model_name = "EII", gtype="trans",
                 fct_reorder= f_order,
                 size.x=16, alpha = 0.2)%>%
  lpa_profile_plot_summary() %>%
  nice_table()
#'

## End(Not run)
```

`LRT_apa` *LRT_apa*

Description

LRT_apa

Usage

```
LRT_apa(data, md = FALSE)
```

Arguments

`data`	anova data, LRT data
`md`	md=TRUE

Value

table explain

Examples

```
## Not run:
#'

library(jjstat)
HS.model <- '
    visual  =~ x1 + b1*x2 + x3
    textual =~ x4 + b2*x5 + x6
    speed   =~ x7 + b3*x8 + x9
'
fit1 <- cfa(HS.model, data = HolzingerSwineford1939)
fit0 <- cfa(HS.model, data = HolzingerSwineford1939,
            orthogonal = TRUE)

fit1 %>% sem_effect(effect ="=~")
```

```
fit0 %>% sem_effect(effect ="=~")
lavTestLRT(fit1, fit0)
lavTestLRT(fit1, fit0) %>% LRT_apa()

## End(Not run)
```

`LRT_apa2` *LRT_apa*

Description

LRT_apa

Usage

```
LRT_apa2(data, md = FALSE)
```

Arguments

`data`	anova data, LRT data
`md`	md=TRUE

Value

table explain

Examples

```
## Not run:
#'

library(jjstat)
HS.model <- '
    visual  =~ x1 + b1*x2 + x3
    textual =~ x4 + b2*x5 + x6
    speed   =~ x7 + b3*x8 + x9
'
fit1 <- cfa(HS.model, data = HolzingerSwineford1939)
fit0 <- cfa(HS.model, data = HolzingerSwineford1939,
            orthogonal = TRUE)

fit1 %>% sem_effect(effect ="=~")
fit0 %>% sem_effect(effect ="=~")
lavTestLRT(fit1, fit0)
lavTestLRT(fit1, fit0) %>% LRT_apa()

## End(Not run)
```

make_c_text *make_c_text*

Description

make_c_text

Usage

```
make_c_text(data)
```

Arguments

data	vector data

Value

text

Examples

```
## Not run:

# Example usage
data <- c("FieldGoals", "OtherTDs", "YardsRushAtt", "RushYards",
          "RushFirstDown", "YardsPassComp", "PassYards", "PassFirstDown",
          "YardsRushAtt", "RushYards", "RushFirstDown", "YardsPassComp",
          "PassYards", "PassFirstDown", "PointsGame", "OffensTD",
          "TDGame", "Speical", "Rushing", "Passing",
          "Offense", "Scoring", 123, 456.78)

make_c_text(data)


## End(Not run)
```

make_df_text *data.frame to input text*

Description

data.frame to input text

Usage

```
make_df_text(data, rownames = FALSE, text = TRUE, col_text = 1)
```

Arguments

`data`	data.frame
`rownames`	rownames=TRUE, FALSE -> nothing rownames
`text`	Convert all configurations to char, e.g. '10'
`col_text`	columns to be used as letters, the rest as numeric

Value

text

Examples

```
## Not run:

# 데이터 프레임 생성
cho2013_cor <- data.frame(
  필요성 = c(1.000, 0.320, 0.266, 0.506, 0.429, 0.401),
  수업 = c(0.320, 1.000, 0.647, 0.463, 0.548, 0.554),
  교사 = c(0.266, 0.647, 1.000, 0.383, 0.491, 0.545),
  계획 = c(0.506, 0.463, 0.383, 1.000, 0.531, 0.491),
  영향 = c(0.429, 0.548, 0.491, 0.531, 1.000, 0.743),
  만족도 = c(0.401, 0.554, 0.545, 0.491, 0.743, 1.000)
)
cho2013_cor %>% make_df_text()



## End(Not run)
```

`make_label_list` *Create Label Mapping List for Node Renaming*

Description

이함수는 (원래이름, 변경라벨) 쌍으로구성된문자벡터를입력받아, 각쌍을 'list(node = ..., to = ...)' 형태의리스트로변환한다. 주로 semPlot, qgraph, 사용자정의시각화함수에서노드이름일괄변경을위해사용된다.

Usage

```
make_label_list(x)
```

Arguments

`x`	character vector. 반드시 (원래이름, 라벨) 쌍으로구성되어야하며, 길이는짝수여야한다.

Details

입력벡터의길이가홀수인경우, (원래이름, 라벨) 쌍이완성되지않으므로오류를발생시킨다.

내부적으로는 'split()'을사용하여 2 개씩그룹화한뒤, 'lapply()'를통해리스트형태로변환한다.

Value

A list of lists. 각원소는다음구조를가진다:

- node: 원래변수명
- to: 변경할라벨

Examples

```
# Example 1: 기본 사용
labels <- c(
  "REFF", "연구효능감",
  "INTR", "진로흥미",
  "OEXP", "결과기대"
)

make_label_list(labels)

# Example 2: semPlot/qgraph용 라벨 매핑
var_named <- c(
  "efficacy2", "연구효능2",
  "efficacy3", "연구효능3",
  "efficacy4", "연구효능4"
)

label_list <- make_label_list(var_named)

# label_list[[1]]
# $node
# [1] "efficacy2"
# $to
# [1] "연구효능2"
```

---

make_layout *Create a Custom Layout Matrix*

Description

Create a Custom Layout Matrix

Usage

```
make_layout(nc = 10, nr = NULL)
```

Arguments

nc	Integer. Number of columns in the layout matrix. Default is 10.
nr	Integer or NULL. Number of rows in the layout matrix. If NULL, the number of rows will be automatically determined. Default is NULL.

Value

A matrix representing the layout, optionally edited interactively using the 'edit' function.

Examples

```
## Not run:
make_layout(nc = 7)
make_layout(nc = 7, nr = 6)
make_layout(3, 6)

## End(Not run)
```

---

make_list_text *make_list_text*

Description

make_list_text

Usage

```
make_list_text(data)
```

Arguments

data	list data

Value

text

Examples

```
## Not run:
#

# Example data (characters)
data1 <- list(
  Speical = c("FieldGoals", "OtherTDs"),
  Rushing = c("YardsRushAtt", "RushYards", "RushFirstDown"),
  Passing = c("YardsPassComp", "PassYards", "PassFirstDown"),
  Offense = c("rush1", "pass1"),
  Scoring = c("PointsGame", "OffensTD", "TDGame")
)
make_list_text(data1)
```

```
# Example data (numeric)
data2 <- list(
  Speical = 7:8,
  Rushing = 1:3,
  Passing = 4:6,
  Offense = 18:19,
  Scoring = 9:11
)
make_list_text(data2)



# Example data (numeric)
list(7:8, 1:3, 4:6, 18:19, 9:11)%>%make_list_text()
#
#

## End(Not run)
```

`make_mat_text` *make_mat_text*

Description

Converts a matrix into a formatted text representation for layout creation.

Usage

```
make_mat_text(mat, text = TRUE, byrow = TRUE)
```

Arguments

`mat`	Matrix data.
`text`	Logical. If 'TRUE', the elements are wrapped in quotes. If 'FALSE', numbers or 'NA' are returned.
`byrow`	Logical. If 'TRUE', the matrix is converted row by row; otherwise, column by column.

Value

Formatted text representation of the matrix.

Examples

```
## Not run:
# Example usage
mmm <- matrix(c(1, 2, 3, 4, 5, 6, 7, 8, 9,
10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30),
 nrow = 5, ncol = 6)
```

```
make_mat_text(mmm, text = TRUE, byrow = TRUE)
make_mat_text(mmm, text = TRUE, byrow = FALSE)
make_mat_text(mmm, text = FALSE, byrow = TRUE)
make_mat_text(mmm, text = FALSE, byrow = FALSE)

# Example with NA
mmm_with_na <- matrix(c(1, NA, 3, NA, 5, 6, NA, 8, 9, 10), nrow = 2)
make_mat_text(mmm_with_na, text = FALSE, byrow = TRUE)

data(offense)
offense %>%str()

nfl_path = matrix(
  c(0, 0, 0, 0, 0,
    0, 0, 0, 0, 0,
    0, 0, 0, 0, 0,
    0, 1, 1, 0, 0,
    1, 0, 0, 1, 0),
  nrow = 5, ncol = 5, byrow = TRUE )

rownames(nfl_path) <- c("Speical", "Rushing",
"Passing", "Offense", "Scoring")
colnames(nfl_path) <- c("Speical", "Rushing",
"Passing", "Offense", "Scoring")
nfl_path

nfl_blocks = list(7:8, 1:3, 4:6,1:6, 9:11)

plspm_blocks_match(data=offense, blocks=nfl_blocks,
                   paths=nfl_path)

nfl_mode =c("B","A","A","A","A")

nfl_pls = plspm(Data=offense, nfl_path, nfl_blocks, modes = nfl_mode)
nfl_pls%>%summary()

#########
nfl_pls_boot = plspm_sem(Data=offense, nfl_path,
 nfl_blocks, modes = nfl_mode)

#Layouts extracted from mockups
plspm_semPaths2(nfl_pls_boot,sizeLat = 6)%>%
  plspm_factor_layout()

#Make it input material
plspm_semPaths2(nfl_pls_boot,sizeLat =6)%>%
  plspm_factor_layout() %>%layout_mat(text=T)

#Organizing materials for input
plspm_semPaths2(nfl_pls_boot,sizeLat =6)%>%
  plspm_factor_layout() %>%layout_mat()%>%
  make_mat_text2()

## End(Not run)
```

make_mat_text2 *Functions that let you input data in matrix form*

Description

Functions that let you input data in matrix form

Usage

```
make_mat_text2(data, text = TRUE, byrow = FALSE)
```

Arguments

data	input dat a
text	text = TRUE is text FALSE numeric
byrow	byrow=FALSE or TRUE

Value

make test

Examples

```
## Not run:
lay
V1      V2 V3        V4        V5       V6        V7        V8 V9
[1,] "ceo_8" NA "satis_5" "satis_4" ""       "satis_3" "satis_1" NA ""
[2,] ""      NA ""        ""        ""       ""        ""        NA ""
[3,] "ceo_7" NA ""        ""        ""       ""        ""        NA ""
[4,] ""      NA ""        ""        "SATISF" ""        ""        NA ""
[5,] "ceo_6" NA ""        ""        ""       ""        ""        NA "stay_2"
[6,] ""      NA ""        ""        ""       ""        ""        NA ""
[7,] "ceo_4" NA "CEOCOM"  ""        ""       ""        "STAYON"  NA ""
[8,] ""      NA ""        ""        ""       ""        ""        NA ""
[9,] "ceo_3" NA ""        ""        ""       ""        ""        NA "stay_1"
[10,] ""      NA ""        ""        ""       ""        ""        NA ""
[11,] "ceo_2" NA ""        ""        ""       ""        ""        NA ""
[12,] ""      NA ""        ""        ""       ""        ""        NA ""
[13,] "ceo_1" NA ""        ""        ""       ""        ""        NA ""

lay %>% make_mat_text2()


NewMat = matrix(c('ceo_8', NA, 'ceo_7', NA, 'ceo_6',
NA, 'ceo_4', NA, 'ceo_3', NA, 'ceo_2', NA, 'ceo_1',
NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,
'satis_5', NA, NA, NA, NA, NA, 'CEOCOM', NA, NA, NA,
NA, NA, NA, 'satis_4', NA, NA, NA, NA, NA, NA, NA,
 NA, NA, NA, NA, NA, NA, NA, NA, 'SATISF', NA, NA,
  NA, NA, NA, NA, NA, NA, NA, 'satis_3', NA, NA,
  NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 'satis_1',
  NA, NA, NA, NA, NA, 'STAYON', NA, NA, NA, NA, NA,
  NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,
```

```
   NA, NA, NA, NA, NA, NA, 'stay_2', NA, NA, NA,
   'stay_1', NA, NA, NA, NA), nrow = 13, ncol = 9)


colnames(NewMat) <- c("V1", "V2", "V3", "V4", "V5", "V6", "V7", "V8", "V9")

#as.numeric
# NewMat <- apply(NewMat, 2, function(x) as.numeric(gsub("'", "", x)))

V1      V2 V3        V4        V5       V6        V7        V8 V9
[1,] "ceo_8" NA "satis_5" "satis_4" NA       "satis_3" "satis_1" NA NA
[2,] NA      NA NA        NA        NA       NA        NA        NA NA
[3,] "ceo_7" NA NA        NA        NA       NA        NA        NA NA
[4,] NA      NA NA        NA        "SATISF" NA        NA        NA NA
[5,] "ceo_6" NA NA        NA        NA       NA        NA        NA "stay_2"
[6,] NA      NA NA        NA        NA       NA        NA        NA NA
[7,] "ceo_4" NA "CEOCOM"  NA        NA       NA        "STAYON"  NA NA
[8,] NA      NA NA        NA        NA       NA        NA        NA NA
[9,] "ceo_3" NA NA        NA        NA       NA        NA        NA "stay_1"
[10,] NA      NA NA        NA        NA       NA        NA        NA NA
[11,] "ceo_2" NA NA        NA        NA       NA        NA        NA NA
[12,] NA      NA NA        NA        NA       NA        NA        NA NA
[13,] "ceo_1" NA NA        NA        NA       NA        NA        NA NA
#' #'
 lay_str1 %>%make_mat_text3(byrow=T)


 NewMat = matrix(c(NA, NA, 'PEOU', NA, NA, NA, NA,
 NA, NA, NA, NA, NA, NA, NA, 'CUX', NA, NA, NA,
  NA, NA, NA, NA, NA, NA, NA, 'AT', NA, 'USE',
  'SUX', NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,
  NA, NA, NA, NA, NA, 'PU', NA, NA, NA, NA),
  nrow = 7, ncol = 7, byrow = TRUE )

 rownames(NewMat) <- c("row1", "row2", "row3", "row4", "row5", "row6", "row7")
colnames(NewMat) <- c("col1", "col2", "col3", "col4", "col5", "col6", "col7")

NewMat %>%make_mat_text2(byrow=FALSE)
NewMat %>%make_mat_text2(byrow=TRUE)
#'
#'

## End(Not run)
```

`make_mat_text_col` 첫번째열이존재하는경우에사용하는 *make_mat_text2*

Description

첫번째열이존재하는경우에사용하는 make_mat_text2

Usage

```
make_mat_text_col(data, text = TRUE, firstcol = 1)
```

Arguments

data	data.frame
text	text TRUE
firstcol	ncol setting

Value

text

Examples

```
## Not run:
#'
data_text = "Tobacco_Dependence (0,5] (5,10] (10,15] (15,20] (20,98]
No_Nicotine_Dependence 130 210 43 114 20
Nicotine_Dependence 119 267 91 254 67"
nt=data_text%>% text2df(header=T)%>%tibble::column_to_rownames("Tobacco_Dependence")
nt %>%tibble()
nt=Char2num(nt,1:5)
nt

## End(Not run)
```

---

make_text_df *Text to data.frame /It is characterized by the use of blank spaces to separate content.*

Description

Text to data.frame /It is characterized by the use of blank spaces to separate content.

Usage

```
make_text_df(text, header = TRUE, type = "res", add_vars = NULL)
```

Arguments

text	text(Row variable names must not contain spaces.)
header	header = TRUE
type	res, check
add_vars	additinal factor

Value

table data.frame

Examples

```
## Not run:
#'
text <- "변인 평균 표준편차 왜도 첨도
성취감 4.24 0.73 -0.82 0.39
인정감 4.19 0.60 -0.36 -0.42
책임감 4.36 0.56 -0.61 -0.29
발전성 3.65 0.89 -0.50 -0.04
감독 3.59 0.88 -0.46 0.13
보수 3.56 0.90 -0.41 -0.18
대인관계 3.75 0.81 -0.14 -0.54
작업환경 3.83 0.75 -0.21 -0.44
안정성 3.28 1.08 -0.28 -0.68
감정이입 3.96 0.72 -0.40 -0.33
직무적합 4.00 0.75 -0.35 -0.78
일체감 4.02 0.75 -0.59 -0.13
현학원재직 3.97 0.79 -0.77 0.37
타학원이직 2.99 1.06 -0.41 -0.77
타직종이직 2.76 1.07 -0.13 -0.70"

text2df(text)

text2df(text, vars=c(rep("동기요인", 4),
             rep("위생요인",5), rep("개인요인",6)  ))

text2df(text, T)%>% make_df_text()

#direct output
"var B se b t
성취감 -.03 .09 -.03 -.33
인정감 .24 .09 .20 2.50
책임감 .30 .08 .23 3.56***
발전성 -.08 .06 -.10 -1.29
감독 .06 .05 .08 1.35
보수 -.14 .07 -.18 -2.01*
대인관계 .06 .06 .06 .89
작업환경 .33 .09 .34 3.82***
안정성 .17 .05 .26 3.42**
성별 -.05 .09 -.03 -.52
연령 -.20 .07 -.22 -3.01**
학력 .05 .07 .04 .70
소득 .00 .04 .01 .12
종사기간 .17 .06 .23 2.71**
기관규모 -.10 .03 -.20 -3.36**" %>%text2df()

reg_text = "var B se b t
성취감 -.03 .09 -.03 -.33
인정감 .24 .09 .20 2.50
책임감 .30 .08 .23 3.56***
발전성 -.08 .06 -.10 -1.29
감독 .06 .05 .08 1.35
보수 -.14 .07 -.18 -2.01*
대인관계 .06 .06 .06 .89
작업환경 .33 .09 .34 3.82***
안정성 .17 .05 .26 3.42**
```

```
성별 -.05 .09 -.03 -.52
연령 -.20 .07 -.22 -3.01**
학력 .05 .07 .04 .70
소득 .00 .04 .01 .12
종사기간 .17 .06 .23 2.71**
기관규모 -.10 .03 -.20 -3.36**"

make_text_to_df(reg_text)
#'
make_text_to_df(reg_text) %>% tibble()

#text to numeric
make_text_to_df(reg_text)%>% Char2num() %>% tibble()

make_text_to_df(reg_text)%>% Char2num(iter=2:4) %>% tibble()
#'

text ="d           Estimate  Std.Err  z-value  P(>|z|)
  read~ppsych           -0.275    0.037   -7.385    0.000
    read~motiv             0.461    0.037   12.404    0.000
arith~motiv             0.600    0.036   16.771    0.000
"
make_text_to_df(text)
make_text_to_df(text) %>% tibble::tibble()

text ="             Estimate Std.Error tvalue Pr(>|t|)
(Intercept) -1.336e-07  3.608e-01   0.000        1
ppsych      -2.747e-01  3.730e-02  -7.363  7.51e-13***
motiv        4.613e-01  3.730e-02  12.367  <2e-16***
"
make_text_to_df(text) %>%tibble()
make_text_to_df(text) %>%make_df_text()


text ="                   Estimate  Std.Err  z-value  P(>|z|)

    read~ppsych           -0.275    0.037   -7.385    0.000
    read~motiv             0.461    0.037   12.404    0.000

    arith~ppsych           -0.096    0.037   -2.616    0.009
     arith~motiv             0.576    0.037   15.695    0.000
"
make_text_to_df(text)



text =" 과장 대리 부장 사원 주임 차장 SUM
남자 30(39.5%) 21(38.9%) 34(79.1%) 30(49.2%)  10(37%)     28(71.8%) 153
여자 46(60.5%) 33(61.1%) 9(20.9%) 31(50.8%) 17(63%)    11(28.2%) 147
SUM 76 54 43 61 27 39 300
"
make_text_to_df(text)
#'
## End(Not run)
```

`markdown_table` *A tool that outputs data to the viewer*

Description

knitr's kable function is the foundation of this package. However, it has many latex/html specific arguments hidden under the ground unless you check its source code. This wrapper function is created to provide better documentation (and auto-complete yay) and at the same time, solve the auto format setting in a better way. kableExtra uses the built-in bootstrap themes by default in kable_styling(). Alternatively, you can use a customized table themes for your table. This lightable table style sheet comes with three formats, namely lightable-minimal, lightable-classic, lightable-material and lightable-material-dark with hover and striped options.

Usage

```
markdown_table(
  data,
  caption = "Table 1. Capiton title input",
  digits = 3,
  font_size = 13,
  full_width = F,
  table = "paper",
  show = "data",
  format = "html",
  row.names = NA,
  col.names = NA,
  align = NULL,
  label = NULL,
  format.args = list(),
  escape = TRUE,
  table.attr = "",
  booktabs = FALSE,
  longtable = FALSE,
  valign = "t",
  position = "center",
  centering = TRUE,
  lightable_options = "basic",
  html_font = "\"Arial Narrow\", arial, helvetica, sans-serif",
  general_title = "Note: ",
  number = NULL,
  alphabet = NULL,
  catout = FALSE,
  general = NULL
)
```

Arguments

`data`	data.frame or etc data, For , is an R object, which is typically a matrix or data frame. For , a list with each element being a returned value from .kable()xkables()kable()

caption	table title
digits	Maximum number of digits for numeric columns, passed to round(). This can also be a vector of length ncol(x), to set the number of digits for individual columns.
font_size	table font size
full_width	table wide and fit values
table	tbble type 'paper', 'classic', 'dark', 'minimal'
show	'markdown' is output to the viewer, and 'data' is output to the console.
format	'markdown' and 'html'. default 'html'. A character string. Possible values are , , (Pandoc's pipe tables), (Pandoc's simple tables), and . The value of this argument will be automatically determined if the function is called within a knitr document. The value can also be set in the global option . If is a function, it must return a character string.latexhtmlpipesimplerstformatknitr.table.format
row.names	row names in table(Set if necessary), Logical: whether to include row names. By default, row names are included if is neither nor identical to .rownames(x)NULL1:nrow(x)
col.names	column names in table (Set if necessary)
align	Table value align, Column alignment: a character vector consisting of 'l' (left), 'c' (center) and/or 'r' (right). By default or if align = NULL, numeric columns are right-aligned, and other columns are left-aligned. If length(align) == 1L, the string will be expanded to a vector of individual letters, e.g. 'clc' becomes c('c', 'l', 'c'), unless the output format is LaTeX.
label	Table label, he table reference label. By default, the label is obtained from .knitr::opts_current$get('label')
format.args	A list of arguments to be passed to to format table values, e.g. .format()list(big.mark = ',')
escape	Boolean; whether to escape special characters when producing HTML or LaTeX tables. When , you have to make sure that special characters will not trigger syntax errors in LaTeX or HTML.escape = FALSE
table.attr	A character string for addition HTML table attributes. This is convenient if you simply want to add a few HTML classes or styles. For example, you can put 'class="table" style="color: red"'.
booktabs	booktabs - T/F for whether to use the longtable format. If you have a table that will span over two or more pages, you will have to turn this on.
longtable	longtable
valign	You probably won't need to adjust this latex option very often. If you are familar with latex tables, this is the optional position for the tabular environment controling the vertical position of the table relative to the baseline of the surrounding text. Possible choices are , and (default)
position	kable_styling: A character string determining how to position the table on a page. Possible values include left, center, right, float_left and float_right. Please see the package doc site for demonstrations. For a LaTeX table, if float_* is selected, LaTeX package wrapfig will be imported.
centering	Align cener in Table default TRUE
lightable_options	Options to customize lightable. Similar with bootstrap_options in kable_styling. Choices include basic, striped and hover.

`html_font`	A string for HTML css font. For example, html_font = '"Arial Narrow", arial, helvetica, sans-serif'.Everything else you need to specify in kable_styling.
`general_title`	Section header for general footnotes. Default is "Note: ".
`number`	A vector of footnote texts. Footnotes here will be numbered. There is no upper cap for the number of footnotes here
`alphabet`	A vector of footnote texts, Footnotes here will be labeled with abc. The vector here should not have more than 26 elements.
`catout`	#p value msg
`general`	#p value msg

Examples

```
# example code

  ## Not run:
  markdown_table(mtcars[1:5,], table="minimal")

  ###*** : p < .001, ** : p < .01, * : p < .05"

## End(Not run)
```

`mat_text_arrange` *mat_text_arrange*

Description

mat_text_arrange

Usage

```
mat_text_arrange(mat, text = TRUE, byrow = FALSE)
```

Arguments

`mat`	matrix data
`text`	TRUE " FALSE number
`byrow`	FALSE, byrow =TRUE fill in row

Value

text

Examples

```
## Not run:
data(offense)
offense %>%str()

nfl_path = matrix(
  c(0, 0, 0, 0, 0,
```

```
    0, 0, 0, 0, 0,
    0, 0, 0, 0, 0,
    0, 1, 1, 0, 0,
    1, 0, 0, 1, 0),
  nrow = 5, ncol = 5, byrow = TRUE )

rownames(nfl_path) <- c("Speical", "Rushing", "Passing", "Offense", "Scoring")
colnames(nfl_path) <- c("Speical", "Rushing", "Passing", "Offense", "Scoring")
nfl_path

nfl_blocks = list(7:8, 1:3, 4:6,1:6, 9:11)

plspm_blocks_match(data=offense, blocks=nfl_blocks,
                   paths=nfl_path)

nfl_mode =c("B","A","A","A","A")

nfl_pls = plspm(Data=offense, nfl_path, nfl_blocks, modes = nfl_mode)
nfl_pls%>%summary()

#########
nfl_pls_boot = plspm_sem(Data=offense, nfl_path, nfl_blocks, modes = nfl_mode)

#Layouts extracted from mockups
plspm_semPaths2(nfl_pls_boot,sizeLat = 6)%>%
  plspm_factor_layout()

#Make it input material
plspm_semPaths2(nfl_pls_boot,sizeLat =6)%>%
  plspm_factor_layout() %>%layout_mat(text=T)

#Organizing materials for input
plspm_semPaths2(nfl_pls_boot,sizeLat =6)%>%
  plspm_factor_layout() %>%layout_mat()%>%
  mat_text_arrange()

## End(Not run)
```

`mcnemar_test` *mcnemar_test*

Description

mcnemar_test

Usage

```
mcnemar_test(mat, correct = TRUE, exact = TRUE)
```

Arguments

`mat`	matrix
`correct`	McNemar's Chi-squared test with continuity correction
`exact`	CI check binom.test

Value

reusult

Examples

```
## Not run:
#'
Buying_car  = matrix(c(23,28,7,12), nrow=2,
                     dimnames= list(c("시승전_있음","시승전_없음"),
                                    c("시승후_있음","시승후_없음")))
Buying_car
mcnemar_test(Buying_car)
mcnemar_test(Buying_car, correct=FALSE)

gen_table = matrix(c(c(9, 3, 13, 10)),ncol=2,
                   dimnames=list("case"=c("yes", "no"),
                                 "control"=c("yes", "no")   ))
gen_table
#Rbase
mcnemar.test(gen_table, correct = FALSE)
mcnemar_test(gen_table, correct = FALSE)



#모든 종류의 분석을 시행한 경우
bind_cols(
  "TEST"= c("correct and exact", "not Correct"," Not exact", "Not correct, exact"),
  bind_rows(
    mcnemar_test(Buying_car),  # 디폴트—
    mcnemar_test(Buying_car, correct=FALSE), # 연속보정 중지
    mcnemar_test(Buying_car,exact=FALSE), # 정확도 중지
    mcnemar_test(Buying_car,exact=FALSE, correct=FALSE)# 연속성 보정, 정확도 중지,SAS
  )
)%>%select(-b,-c,-odds_ratio)

## End(Not run)
```

`md` *md: Generate Styled Data Tables for Reports*

Description

This function generates styled tables for reports, supporting different formats and options for enhanced visual presentation.

Usage

```
md(
  data,
  caption = "Table 1. Capiton title input",
  digits = 3,
  font_size = 13,
  full_width = FALSE,
```

```
  table = "paper",
  show = "data",
  format = "html",
  row.names = NA,
  col.names = NA,
  align = NULL,
  label = NULL,
  format.args = list(),
  escape = TRUE,
  table.attr = "",
  booktabs = FALSE,
  longtable = FALSE,
  valign = "t",
  position = "center",
  centering = TRUE,
  lightable_options = "basic",
  html_font = "\"Arial Narrow\", arial, helvetica, sans-serif",
  general_title = "Note: ",
  number = NULL,
  alphabet = NULL,
  catout = FALSE,
  general = NULL,
  pass = FALSE
)
```

Arguments

`data`	A data frame or tibble to be styled and displayed.
`caption`	A character string for the table caption. Default is "Table 1. Capiton title input".
`digits`	An integer indicating the number of decimal places. Default is 3.
`font_size`	A numeric value specifying the font size. Default is 13.
`full_width`	Logical, whether the table should use full width. Default is FALSE.
`table`	A character string specifying the table style. Options are "paper", "classic", "dark", "minimal", and "basic". Default is "paper".
`show`	A character string specifying the type of display. Options are "data", "lm", or "p_add". Default is "data".
`format`	The output format of the table. Default is "html".
`row.names`	Logical or character vector for row names. Default is NA.
`col.names`	A character vector specifying column names. Default is NA.
`align`	A character vector specifying column alignment. Default is NULL.
`label`	A character string for table label. Default is NULL.
`format.args`	A list of additional arguments passed to 'format()'. Default is an empty list.
`escape`	Logical, whether to escape special characters. Default is TRUE.
`table.attr`	A character string for additional HTML attributes. Default is an empty string.
`booktabs`	Logical, whether to use LaTeX booktabs style. Default is FALSE.

longtable	Logical, whether to use LaTeX longtable style. Default is FALSE.
valign	A character string specifying vertical alignment. Default is "t".
position	A character string for table position. Default is "center".
centering	Logical, whether to center align the table. Default is TRUE.
lightable_options	A character string for lightable styling options. Default is "basic".
html_font	A character string specifying the font for HTML output. Default is '"Arial Narrow", arial, helvetica, sans-serif'.
general_title	A character string specifying the title for footnotes. Default is "Note: ".
number	A numeric vector specifying footnote numbering. Default is NULL.
alphabet	A character vector specifying footnote alphabet markers. Default is NULL.
catout	Logical, whether to output significance markers (e.g., **, *). Default is FALSE.
general	A character string specifying a general footnote. Default is NULL.
pass	Logical, whether to process the table styling. If TRUE, returns the original data. Default is FALSE.

Value

A styled table if 'pass = TRUE', otherwise returns the original data.

Examples

```
## Not run:
  data <- data.frame(A = c(1.234, 2.345, 3.456), B = c(4.567, 5.678, 6.789))
  md(data, caption = "Example Table", table = "classic")

## End(Not run)
```

---

meanCentering *centering each vector*

Description

centering each vector

Usage

```
meanCentering(variable, na.rm = TRUE)
```

Arguments

variable	vector variable in data
na.rm	missing data chedk

meanCentering_df *meancentering mixed model*

Description

meancentering mixed model

Usage

```
meanCentering_df(data, grp_id, gv, ...)
```

Arguments

data	data.frame
grp_id	group id
gv	group variable, Group Variables That Separate Groups
...	individual measurement independent variables evaluated at individual level

Examples

```
## Not run:
meanCentering_df(mtcars[,c("cyl","mpg","wt","hp")],"cyl","mpg","wt","hp")|> data.frame()

## End(Not run)
```

mean_apa *mean_apa: Generate descriptive statistics in APA style*

Description

mean_apa: Generate descriptive statistics in APA style

Usage

```
mean_apa(df, sd = FALSE)
```

Arguments

df	A data frame containing descriptive statistics, including variables such as N, MEAN, SD, Skew, and Kurt.
sd	A logical value. If TRUE, includes standard deviation (SD) along with the mean in the output. Default is FALSE.

Value

A string formatted in APA style, summarizing the descriptive statistics in order of mean values and assessing normality based on skewness and kurtosis.

Examples

```
df <- data.frame(
  var = c("Q33_1", "Q33_2", "Q33_3"),
  N = c(4542, 4542, 4542),
  MEAN = c(1.9489, 1.9337, 1.9243),
  SD = c(0.84679, 0.86568, 0.86960),
  Skew = c(0.72576, 0.63900, 0.70717),
  Kurt = c(0.63369, 0.087763, 0.27470)
)
mean_apa(df, sd = TRUE)

# Example Output:
# "The results of the descriptive statistics analysis are as follows.
# Variables ordered by mean are Q33_1 (M=1.9489, SD=0.84679) > Q33_2 (M=1.9337, SD=0.86568) >
# Q33_3 (M=1.9243, SD=0.86960). Skewness ranged from 0.63900 to 0.72576, and kurtosis ranged from
# 0.087763 to 0.63369. Following Kline (2005), skewness values exceeding an absolute value of 3
# and kurtosis values exceeding an absolute value of 8 or 10 are considered non-normal. Skewness
# and kurtosis met the criteria for normality."
```

`mean_bar` *mean_bar: Bar plot of means with optional sorting and customizations*

Description

This function generates a bar plot displaying mean values, allowing customization of axis labels, text size, angle, and sorting.

Usage

```
mean_bar(
  df,
  x.title = "문항",
  sort = FALSE,
  size.text = 4,
  text.angle = 0,
  size.axis = 12
)
```

Arguments

`df`	A data frame containing 'var' and 'MEAN' columns.
`x.title`	A character string for the x-axis title. Default is " 문항".
`sort`	Logical. If TRUE, sorts the data by mean values in ascending order. Default is FALSE.
`size.text`	Numeric. Size of the text labels on bars. Default is 4.
`text.angle`	Numeric. Angle of the x-axis text labels. Default is 0.
`size.axis`	Numeric. Size of the axis text labels. Default is 12.

Value

A ggplot object showing a bar plot of means.

Examples

```
## Not run:
# Example dataset
df <- data.frame(var = c("Q1", "Q2", "Q3"), MEAN = c(3.5, 4.2, 2.8))

# Default bar plot
mean_bar(df)

# Sorted bar plot with customized text size and angle
mean_bar(df, sort = TRUE, size.text = 5, text.angle = 45)

## End(Not run)
```

`med_effect` *DE,IE,TE Effect extraction*

Description

DE,IE,TE Effect extraction

Usage

```
med_effect(x, effect1 = "", caption = "", effect2 = ".", option = 1)
```

Arguments

`x`	lavaan result
`effect1`	first, you must select effect1 = IE, DE, TE
`caption`	title
`effect2`	‘’ 2nd oprion
`option`	1(p_value) or 2(ci)

Value

effect result

`metacorMat` *Creating a meta-structural equation correlation matrix JH, Park*

Description

Creating a meta-structural equation correlation matrix JH, Park

Usage

```
metacorMat(data, startcol, var = "")
```

Arguments

`data`	data.frame, columns: study, n, var1_var2, var1_var2, ...
`startcol`	The column where the correlation coefficient starts, ej, and the column where the correlation coefficient ends are automatically calculated.
`var`	Unique variable name with correlation analysis

Examples

```
##make matrix
## Not run:
 ## making correlation matrix ##
cormat1 = metacorMat(rd2, startcol= 3, var=c("po","ne","se","jp"))
 ##variable : "po_ne" "po_se" "po_jp" "ne_se" "ne_jp" "se_jp"

rd2<- data.frame(
  stringsAsFactors = FALSE,
  study = c(1L,
            2L,3L,4L,5L,6L,7L,8L,9L,10L,11L,12L,
            13L,14L,15L,16L,17L,18L,19L,20L,
            21L,22L,23L,24L,25L,26L,27L,28L,29L,
            30L,31L,32L,33L,34L,35L,36L),
  n = c(671L,
        92L,73L,321L,214L,947L,321L,231L,
        531L,567L,213L,928L,269L,593L,78L,983L,
        1504L,247L,79L,875L,68L,201L,891L,
        197L,431L,98L,179L,69L,301L,679L,273L,
        254L,316L,781L,273L,268L),
  po_ne = c(NA,
            NA,-0.33,NA,NA,NA,NA,-0.32,NA,NA,NA,
            NA,NA,NA,-0.21,-0.49,NA,-0.14,-0.29,
            -0.27,NA,-0.37,NA,NA,-0.35,-0.17,NA,
            -0.11,NA,NA,-0.41,NA,NA,NA,NA,-0.39),
  po_se = c(NA,
            0.37,0.29,0.16,0.41,NA,NA,0.21,0.59,
            0.53,0.31,0.38,NA,NA,0.19,NA,NA,NA,
            NA,0.21,0.07,0.57,0.45,NA,0.18,NA,
            0.45,NA,NA,0.09,NA,NA,0.37,0.19,0.23,
            NA),
  po_jp = c(0.16,
            0.39,0.09,0.07,NA,0.04,NA,NA,0.33,
            0.11,NA,0.04,NA,NA,0.21,0.23,0.09,
            0.11,0.31,0.11,-0.07,0.14,NA,0.19,0.13,
            0.25,0.35,0.29,0.35,NA,0.27,0.19,NA,
            NA,0.13,0.19),
  ne_se = c(NA,
            -0.31,NA,NA,NA,NA,NA,-0.19,NA,NA,NA,
            NA,-0.49,NA,-0.45,NA,NA,NA,NA,-0.38,
            NA,-0.39,NA,NA,-0.21,NA,NA,NA,NA,
            -0.23,NA,NA,NA,NA,-0.44,NA),
  ne_jp = c(NA,
            -0.41,NA,NA,NA,-0.18,-0.11,NA,NA,NA,
            NA,NA,-0.21,-0.15,-0.36,-0.29,NA,
            -0.27,-0.29,-0.23,NA,-0.27,NA,NA,-0.19,
            -0.07,NA,-0.31,NA,NA,0.07,NA,NA,NA,
            -0.27,-0.05),
  se_jp = c(NA,
```

```
            NA,NA,0.07,NA,NA,NA,NA,0.27,0.07,NA,
            NA,0.48,0.26,0.41,NA,NA,NA,NA,0.31,
            0.19,0.34,NA,NA,0.19,NA,0.39,NA,
            0.57,NA,NA,NA,NA,NA,NA,NA),
  jobyear = c("f",
              "f","s","s","s","s","f","f","s","s",
              "s","s","f","f","f","f","f","f","s",
              "s","s","s","s","f","s","f","f",
              "f","f","f","f","s","s","s","f","f"),
  wage = c(9L,
           1L,1L,1L,4L,9L,2L,3L,3L,3L,2L,7L,
           1L,8L,6L,3L,5L,7L,8L,4L,7L,1L,4L,
           6L,7L,1L,9L,6L,7L,7L,7L,9L,2L,5L,
           7L,3L)
)

## cormat1
## [[1]]
## po ne se   jp
## po 1.00 NA NA 0.16
## ne   NA NA NA   NA
## se   NA NA NA   NA
## jp 0.16 NA NA 1.00
##
## [[2]]
## po    ne    se    jp
## po   NA    NA  0.37  0.39
## ne   NA  1.00 -0.31 -0.41
## se 0.37 -0.31    NA    NA
## jp 0.39 -0.41    NA  1.00
## 출처 meta-analaysis 경로분석모델 1단계 - 상관행렬추정 correlation using metaSEM|자유자재Paper

### 1. How to obtain the correlation matrix using the fixed effects model
library(metaSEM)
fixed1 <- tssem1(Cov=cormat1, n=rd2$n, method="FEM")
summary(fixed1)
coef(fixed1)

### random effects model
random = tssem1(Cov=cormat1, n=rd2$n, method="REM" , RE.type = "Diag")
summary(random)

# 2. How to find correlation matrix using random effects model
remodel = tssem1(Cov=cormat1, n=rd2$n, method="REM" , RE.type = "Diag")
random1 = summary(remodel)
mat1 <- vec2symMat(matrix(random1$coefficients[1:6,1]), diag = FALSE)
colnames(mat1)=c("po","ne","se","jp")
rownames(mat1)=c("po","ne","se","jp")
mat1

##path modeling
### step1
rem = tssem1(Cov=cormat1, n=rd2$n, method="REM" , RE.type = "Diag")
summary(rem)

## path model to A
## se = b31*po + b32*ne +e3
```

```
## jp = b41*po + b42*ne + b43*se +e4
A= create.mxMatrix(
c(0,0,0,0,
  0,0,0,0,
  "0.1*b31","0.1*b32",0,0,
  "0.1*b41","0.1*b42","0.1*b43",0),
type="Full", nrow=4, ncol=4, byrow=TRUE, name="A")
A

#exogeous correlation matrix
varnames <- c("po","ne","se","jp")
S= create.mxMatrix(
c(1,
  "0.1*p21",1,
  0,        0,"1*p33",
  0,        0,      0,"1*p44"),
  type="Symm", byrow = TRUE,
  name="S",
  dimnames=list(varnames, varnames)
)
S


### step 2
stage2A = tssem2(rem, Amatrix = A, Smatrix = S, diag.constraints = TRUE)
summary(stage2A)
plot(stage2A, what="est", edge.label.cex = 0.9, edge.label.position=0.7,
    nDigits=3, layout = "spring")


###Alternative model
A2 = create.mxMatrix(
c(0,0,0,0,
  0,0,0,0,
  "0.1*b31","0.1*b32",0,0,
  0,"0.1*b42","0.1*b43",0),
type="Full", nrow=4, ncol=4, byrow=TRUE, name="A")
 A2


stage3A = tssem2(rem, Amatrix = A2, Smatrix = S,
                 diag.constraints = TRUE, intervals.type = "LB")
summary(stage3A)

plot(stage3A, what="est", edge.label.cex = 1, edge.color="gray20",
edge.label.position=0.5, nDigits=4,
layout = "spring", style="lisrel",residScale=16)

# Test the significance of indirect effect
stage4A = tssem2(rem, Amatrix = A2, Smatrix = S, diag.constraints = TRUE,
          intervals.type = "LB",
          mx.algebras = list(
                   IDEpotojp = mxAlgebra(b31*b43, name="IDEpotojp"),
                   IDEnetojp = mxAlgebra(b32*b43, name="IDEnetojp")
                  ))
  summary(stage4A)
```

```
## mxAlgebras objects (and their 95% likelihood-based CIs):
## lbound    Estimate      ubound
## IDEpotojp[1,1]  0.05315542  0.08413089  0.11799399
## IDEnetojp[1,1] -0.10185142 -0.06355753 -0.03440116


## End(Not run)
```

`min_max_scaler` 비율척도로변환하는함수정의

Description

비율척도로변환하는함수정의

Usage

```
min_max_scaler(data, columns = NULL)
```

Arguments

`data`	data
`columns`	colum all

Value

df

Examples

```
## Not run:
#'
 사용 예
data_frame: 데이터프레임 객체
변환할 열 이름을 명시적으로 지정하거나 NULL로 모든 열을 변환
scaled_data <- min_max_scaler(data_frame, columns = c("V1_1", "V2_1", "V3_1"))
print(scaled_data)

## End(Not run)
```

`missCheck` *Check and resolve missing data-*

Description

If there are missing values, print how many there are

Usage

```
missCheck(data, plot = FALSE)
```

Arguments

data	data.frame
plot	missing data plot

Value

Results of checking for missingness

Examples

```
## Not run:
 sleep %>% str()
sleep %>% missCheck()

## End(Not run)
```

---

missfillin *missfillin: Fill Missing Values in a Data Frame*

Description

This function fills missing values in a data frame using either a specified function (e.g., 'mean' or 'median') or a method such as 'pmm' (predictive mean matching) through the 'mice' package.

Usage

```
missfillin(data, fn = mean, method = NULL)
```

Arguments

data	A data frame containing missing values.
fn	A function to apply for filling missing values (default: 'mean').
method	The method to use for imputing missing values. Accepts 'NULL' (default function-based) or '"pmm"'.

Value

A data frame with missing values imputed.

Examples

```
## Not run:
# Example 1: Impute with mean function
data <- data.frame(a = c(1, 2, NA, 4), b = c(NA, 3, 5, 6))
result <- missfillin(data, fn = mean)
print(result)

# Example 2: Impute with median function
data <- data.frame(a = c(NA, 1, 2, 3), b = c(4, NA, NA, 8))
```

```
result <- missfillin(data, fn = median)
print(result)

# Example 3: Impute using predictive mean matching (pmm)
data <- data.frame(a = c(1, NA, 3, 4), b = c(NA, 2, NA, 5))
result <- missfillin(data, method = "pmm")
print(result)

# Example 4: Handle data without missing values
data <- data.frame(a = c(1, 2, 3, 4), b = c(4, 5, 6, 7))
result <- missfillin(data)
print(result)

# Example 5: Error when insufficient columns for pmm
data <- data.frame(a = c(1, 2, NA, 4))
result <- tryCatch(missfillin(data, method = "pmm"), error = function(e) e)
print(result)

## End(Not run)
```

`missfillinFor` *missfillinFor: Fill Missing Values in Selected Columns of a Data Frame*

Description

This function fills missing values in a specified range of columns within a data frame using either a specified function (e.g., ‘mean‘, ‘median‘) or a method such as ‘pmm‘ (predictive mean matching).

Usage

```
missfillinFor(data, fn = mean, method = NULL, range = NULL)
```

Arguments

`data`	A data frame containing missing values.
`fn`	A function to apply for filling missing values (default: ‘mean‘).
`method`	The method to use for imputing missing values. Accepts ‘NULL‘ (default function-based) or ‘”pmm”‘.
`range`	A numeric vector indicating column indices or a character string specifying the range of column names (e.g., ‘”col1:col3”‘). If ‘NULL‘, all columns are considered.

Value

A data frame with missing values imputed in the specified range of columns.

Examples

```
## Not run:
# Example 1: Impute all columns with mean function
data <- data.frame(a = c(1, 2, NA, 4), b = c(NA, 3, 5, 6), c = c(NA, NA, 7, 8))
result <- missfillinFor(data, fn = mean)
print(result)

# Example 2: Impute specific column range with median function
data <- data.frame(a = c(1, NA, 3, 4), b = c(NA, 2, NA, 5), c = c(6, 7, 8, 9))
result <- missfillinFor(data, fn = median, range = 1:2)
print(result)

# Example 3: Impute using predictive mean matching (pmm)
data <- data.frame(a = c(1, NA, 3, 4), b = c(NA, 2, NA, 5), c = c(6, 7, 8, 9))
result <- missfillinFor(data, method = "pmm", range = "a:b")
print(result)

# Example 4: Impute using column names
data <- data.frame(a = c(1, 2, NA, 4), b = c(NA, 3, 5, 6), c = c(NA, NA, 7, 8))
result <- missfillinFor(data, fn = mean, range = "a:c")
print(result)

# Example 5: Handle insufficient columns for pmm
data <- data.frame(a = c(1, 2, NA, 4))
result <- tryCatch(missfillinFor(data, method = "pmm", range = 1), error = function(e) e)
print(result)

## End(Not run)
```

`mlsem_fit_indices` *Extract Model Fit Indices for Multilevel SEM*

Description

This function extracts model fit indices from a fitted multilevel SEM object (lavaan), with robust fit measures prioritized when available. The output can be returned in long (APA-style) or wide format depending on the selected type.

Usage

```
mlsem_fit_indices(
  fit,
  type = c("data", "long", "wide", "apa"),
  robust = TRUE,
  digits = 3
)
```

Arguments

`fit`	A fitted lavaan object (e.g., from `sem()` with `cluster` specified).
`type`	Output type. One of `"data"`, `"long"`, `"wide"`, or `"apa"`.

- `"data"`: APA-style long format (Index–Value).

- `"long"`: Same as `"data"`.
- `"wide"`: One-row wide data.frame.
- `"apa"`: Same as `"wide"` (alias for reporting).

`robust` Logical. If TRUE (default), robust fit indices (e.g., robust CFI, TLI, RMSEA) are used when available.

`digits` Integer. Number of decimal places for rounding.

Details

This function is designed to be used upstream of `mlsem_modelfit_paper()`. It performs no table formatting and returns numeric values only.

Value

A data.frame containing model fit indices. The structure depends on `type`.

Examples

```
## Not run:
##############################################
# Example 1. Typical usage after ML-SEM
##############################################

fit_tab_long <- mlsem_fit_indices(fit_mlsem, type = "long")
fit_tab_data <- mlsem_fit_indices(fit_mlsem, type = "data")
fit_tab_wide <- mlsem_fit_indices(fit_mlsem, type = "wide")
fit_tab_apa  <- mlsem_fit_indices(fit_mlsem, type = "apa")

# Pipe-friendly usage
mlsem_fit_indices(fit_mlsem) %>%
  mlsem_modelfit_paper(type = "kable", format = "markdown")


##############################################
# Example 2. Simple Multilevel SEM example
##############################################

library(lavaan)

# Simulated multilevel data
set.seed(123)
dat <- data.frame(
  university = rep(1:25, each = 40),
  x1 = rnorm(1000),
  x2 = rnorm(1000),
  y1 = rnorm(1000),
  y2 = rnorm(1000)
)

# Multilevel SEM model
model_ml <- '
  level: 1
    X =~ x1 + x2
    Y =~ y1 + y2
    Y ~ X
```

```
  level: 2
    Y ~ X
'

fit_mlsem <- sem(
  model_ml,
  data      = dat,
  cluster   = "university",
  estimator = "MLR"
)

# Extract fit indices (APA long format)
fit_tab <- mlsem_fit_indices(fit_mlsem, type = "long")

# Convert to paper-ready table
mlsem_modelfit_paper(fit_tab, type = "kable", format = "markdown")

## End(Not run)
```

`mlsem_fornell_larcker` *Fornell–Larcker Table with Significance (Multilevel SEM)*

Description

Fornell–Larcker Table with Significance (Multilevel SEM)

Usage

```
mlsem_fornell_larcker(fit, level = c("within", "between"), digits = 3)
```

Arguments

`fit`	lavaan multilevel SEM object
`level`	”within” or ”between”
`digits`	numeric

`mlsem_htmt` *HTMT for Multilevel SEM*

Description

This function computes the Heterotrait–Monotrait ratio (HTMT) for multilevel CFA/SEM models estimated using lavaan. HTMT is calculated separately for within-level or between-level measurement models based on observed-variable correlations.

Usage

```
mlsem_htmt(fit, level = c("within", "between"), digits = 3)
```

Arguments

`fit`	A lavaan object fitted with a multilevel CFA/SEM model.
`level`	Character. One of `"within"` or `"between"`.
`digits`	Integer. Number of digits to round the HTMT values.

Details

HTMT is defined as the ratio of the mean of heterotrait–heteromethod correlations to the geometric mean of monotrait–heteromethod correlations.

Only latent variables with at least two indicators are included. The function uses level-specific observed-variable correlation matrices extracted via `lavInspect(fit, "cor.ov")`.

Value

A data.frame containing the HTMT matrix (lower triangle only; diagonal and upper triangle are NA).

References

Henseler, J., Ringle, C. M., & Sarstedt, M. (2015). A new criterion for assessing discriminant validity in variance-based structural equation modeling. Journal of the Academy of Marketing Science, 43(1), 115–135.

Examples

```
## Not run:
fit <- lavaan::sem(model, data = Demo.twolevel, cluster = "cluster")

mlsem_htmt(fit, level = "within")
mlsem_htmt(fit, level = "between")

## End(Not run)
```

`mlsem_modelfit_paper` *Model Fit Table for Multilevel SEM (Paper-ready)*

Description

This function formats multilevel SEM model fit indices extracted by `mlsem_fit_indices()` into a publication-ready table. It supports raw data output as well as markdown/html tables via kable.

Usage

```
mlsem_modelfit_paper(
  x,
  type = c("data", "kable"),
  format = c("raw", "markdown", "html"),
  caption = "Model Fit Indices (Multilevel SEM)"
)
```

Arguments

`x`	A data.frame in APA long format with columns `Index` and `Value`. Typically returned from `mlsem_fit_indices(fit, type = "long")`.
`type`	Output type. One of `"data"` or `"kable"`.
`format`	Output format when `type = "kable"`. One of `"raw"`, `"markdown"`, or `"html"`.
`caption`	Table caption used for kable output.

Details

This function is designed to be used after `mlsem_fit_indices()`. All internal formatting helpers are intentionally hidden (prefixed with `.`) to keep the public API minimal and stable.

Value

A data.frame (when `type = "data"` or `format = "raw"`), or a formatted kable object (markdown/html).

Examples

```
## Not run:
##########################################
# Example 1. Typical workflow
##########################################

fit_tab <- mlsem_fit_indices(fit_mlsem, type = "long")

mlsem_modelfit_paper(fit_tab)
mlsem_modelfit_paper(fit_tab, type = "data")
mlsem_modelfit_paper(fit_tab, type = "kable")
mlsem_modelfit_paper(fit_tab, type = "kable", format = "raw")
mlsem_modelfit_paper(fit_tab, type = "kable", format = "markdown")
mlsem_modelfit_paper(fit_tab, type = "kable", format = "html")

# Pipe-friendly usage
mlsem_fit_indices(fit_mlsem) %>%
  mlsem_modelfit_paper(type = "kable", format = "markdown")


##########################################
# Example 2. Simple Multilevel SEM example
##########################################

library(lavaan)

# Simulated data
set.seed(123)
dat <- data.frame(
  univ = rep(1:30, each = 40),
  x1 = rnorm(1200),
  x2 = rnorm(1200),
  y1 = rnorm(1200),
  y2 = rnorm(1200)
)
```

```
# Multilevel SEM model
model_ml <- '
  level: 1
    X =~ x1 + x2
    Y =~ y1 + y2
    Y ~ X

  level: 2
    Y ~ X
'

fit_mlsem <- sem(
  model_ml,
  data    = dat,
  cluster = "univ",
  estimator = "MLR"
)

# Extract fit indices
fit_tab <- mlsem_fit_indices(fit_mlsem, type = "long")

# Paper-ready table
mlsem_modelfit_paper(fit_tab, type = "kable", format = "markdown")

## End(Not run)
```

`mlsem_reliability` *Multilevel SEM Reliability (Cronbach's alpha, CR, AVE)*

Description

This function computes reliability indices for multilevel SEM models fitted with lavaan. Reliability is calculated separately for within- and between-level measurement models.

Usage

```
mlsem_reliability(fit, level = c("within", "between"), digits = 3)
```

Arguments

`fit`	A fitted lavaan multilevel SEM object.
`level`	Character. One of `"within"` or `"between"`.
`digits`	Integer. Number of digits to round the results.

Details

The function returns Cronbach's alpha (model-based, using observed correlation matrix), Composite Reliability (CR), and Average Variance Extracted (AVE) based on standardized factor loadings.

- Cronbach's alpha is computed using the model-implied observed correlation matrix (lavInspect(fit, "cor.ov")).
- CR and AVE are computed from standardized factor loadings.
- Latent variables with fewer than two indicators are excluded.

Value

A data.frame with latent variables and their reliability indices:

Latent Latent variable name

Cronbach Cronbach's alpha

CR Composite Reliability

AVE Average Variance Extracted

Examples

```
## Not run:
model <- '
level: 1
  fw =~ y1 + y2 + y3

level: 2
  fb =~ y1 + y2 + y3
'

fit <- lavaan::sem(
  model,
  data    = Demo.twolevel,
  cluster = "cluster"
)

mlsem_reliability(fit, level = "within")
mlsem_reliability(fit, level = "between", digits = 4)

## End(Not run)
```

`modelfit_paper` *Model Fit Table for Paper*

Description

This function formats model fit indices into a publication-ready table or returns the underlying data.frame depending on the selected options. It is designed to be used after extracting model fit information from a CFA or SEM object (e.g., lavaan output).

Usage

```
modelfit_paper(
  x,
  type = c("kable", "data"),
  format = c("markdown", "raw", "html"),
  caption = "Model Fit Indices"
)
```

Arguments

`x`	A data.frame or matrix containing model fit information. The object must include rows named `"criterian"` and `"Value"`, and columns such as `chisq`, `df`, `pvalue`, `rmsea`, `srmr`, `cfi`, and `tli`.
`type`	Character. Output type. One of `"kable"` (default) or `"data"`.
`format`	Character. Output format when `type = "kable"`. One of `"markdown"`, `"raw"`, or `"html"`.
`caption`	Character. Table caption used for kable output.

Details

The function internally converts the degrees of freedom (df) value to an integer to avoid outputs such as 97.000 in publication tables. Formatting using `knitr::kable()` is applied only at the final output stage, ensuring that raw data remain accessible for further processing when needed.

Value

- If `type = "data"`: a data.frame.
- If `type = "kable"` and `format = "raw"`: a data.frame.
- If `type = "kable"` and `format = "markdown"` or `"html"`: a formatted table object.

Examples

```
## Not run:
# Assuming modelfitdata is returned from cfa2(type = "modelfit")

# Raw data.frame
modelfit_paper(modelfitdata, type = "data")

# Markdown table for papers
modelfit_paper(modelfitdata, type = "kable", format = "markdown")

# HTML table for Viewer or browser
modelfit_paper(modelfitdata, type = "kable", format = "html")

## End(Not run)
```

`Modindices` *Modification Indices*

Description

Given a fitted lavaan object, compute the modification indices (= univariate score tests) for a selected set of fixed-to-zero parameters.

Usage

```
Modindices(x, mi.level = 10, op1 = "~~")
```

Arguments

`x`	lavaan result
`mi.level`	mi over 10, default
`op1`	regression '~~'

Value

chisquare

Examples

```
## Not run:
library(lavaan)
model1 <- '
  # measurement model
    ind60 =~ x1 + x2 + x3
    dem60 =~ y1 + y2 + y3 + y4
    dem65 =~ y5 + y6 + y7 + y8
  # regressions
    dem60 ~ ind60
    dem65 ~ ind60 + dem60
 # residual correlations
    y1 ~~ y5
    y2 ~~ y4 + y6
    y3 ~~ y7
    y4 ~~ y8
    y6 ~~ y8
'

fit1 <- sem(model1, data = PoliticalDemocracy)

 summary(fit1, standardized = TRUE)
cfa2(fit1)

Modindices(fit1)

## End(Not run)
```

`ModSEM` *Title*

Description

Title

Usage

```
ModSEM(
  model = NULL,
  data = NULL,
```

```
  type = "res",
  ind = TRUE,
  cat = FALSE,
  interact = TRUE,
  auto = TRUE,
  n_name = 1,
  prefix = "a",
  paths_name = TRUE,
  add_model = NULL,
  ordered = NULL,
  sampling.weights = NULL,
  sample.cov = NULL,
  sample.mean = NULL,
  sample.th = NULL,
  sample.nobs = NULL,
  group = NULL,
  cluster = NULL,
  constraints = "",
  WLS.V = NULL,
  NACOV = NULL,
  ov.order = "model",
  ...
)
```

Arguments

`model`	A description of the user-specified model. Typically, the model is described using the lavaan model syntax. See model.syntax for more information. Alternatively, a parameter table (eg. the output of the lavaanify() function) is also accepted.
`data`	An optional data frame containing the observed variables used in the model. If some variables are declared as ordered factors, lavaan will treat them as ordinal variables.
`type`	res, fit, New_model, New_data, imlist
`ind`	Automatically Adding Indirect Effects
`cat`	output syntax default FALSE
`interact`	interaction term extract possinle TRUE
`auto`	auto genrate indirect path
`n_name`	indirect name number n_name =1
`prefix`	prefix "a" –"H" change possible
`paths_name`	inde path name default TRUE
`add_model`	add model syntax
`ordered`	Character vector. Only used if the data is in a data.frame. Treat these variables as ordered (ordinal) variables, if they are endogenous in the model. Importantly, all other variables will be treated as numeric (unless they are declared as ordered in the data.frame.) Since 0.6-4, ordered can also be logical. If TRUE, all observed endogenous variables are #' treated as ordered (ordinal). If FALSE, all observed endogenous variables are considered to be numeric (again, unless they are declared as ordered in the data.frame.)

`sampling.weights`
: A variable name in the data frame containing sampling weight information. Currently only available for non-clustered data. Depending on the sampling.weights.normalization option, these weights may be rescaled (or not) so that their sum equals the number of observations (total or per group).

`sample.cov`
: Numeric matrix. A sample variance-covariance matrix. The rownames and/or colnames must contain the observed variable names. For a multiple group analysis, a list with a variance-covariance matrix for each group.

`sample.mean`
: A sample mean vector. For a multiple group analysis, a list with a mean vector for each group.

`sample.th`
: Vector of sample-based thresholds. For a multiple group analysis, a list with a vector of thresholds for each group.

`sample.nobs`
: Number of observations if the full data frame is missing and only sample moments are given. For a multiple group analysis, a list or a vector with the number of observations for each group.

`group`
: Character. A variable name in the data frame defining the groups in a multiple group analysis.

`cluster`
: Character. A (single) variable name in the data frame defining the clusters in a two-level dataset.

`constraints`
: Additional (in)equality constraints not yet included in the model syntax. See model.syntax for more information.

`WLS.V`
: A user provided weight matrix to be used by estimator "WLS"; if the estimator is "DWLS", only the diagonal of this matrix will be used. For a multiple group analysis, a list with a weight matrix for each group. The elements of the weight matrix should be in the following order (if all data is continuous): first the means (if a meanstructure is involved), then the lower triangular elements of the covariance matrix including the diagonal, ordered column by column. In the categorical case: first the thresholds (including the means for continuous variables), then the slopes (if any), the variances of continuous variables (if any), and finally the lower triangular elements of the correlation/covariance matrix excluding the diagonal, ordered column by column.

`NACOV`
: A user provided matrix containing the elements of (N times) the asymptotic variance-covariance matrix of the sample statistics. For a multiple group analysis, a list with an asymptotic variance-covariance matrix for each group. See the WLS.V argument for information about the order of the elements.

`ov.order`
: Character. If "model" (the default), the order of the observed variable names (as reflected for example in the output of lavNames()) is determined by the model syntax. If "data", the order is determined by the data (either the full data.frame or the sample (co)variance matrix). If the WLS.V and/or NACOV matrices are provided, this argument is currently set to "data".

`...`
: Many more additional options can be defined, using 'name = value'. See lavOptions for a complete list.

Value

data

Examples

```
## Not run:
#'
## The industrialization and Political Democracy Example
## Bollen (1989), page 332
model <- '
  # latent variable definitions
     ind60 =~ x1 + x2 + x3
     dem60 =~ y1 + a*y2 + b*y3 + c*y4
     dem65 =~ y5 + a*y6 + b*y7 + c*y8

  # regressions
    dem60 ~ ind60
    dem65 ~ ind60 + dem60

  # residual correlations
    y1 ~~ y5
    y2 ~~ y4 + y6
    y3 ~~ y7
    y4 ~~ y8
    y6 ~~ y8
'

fit <- sem(model, data = PoliticalDemocracy)
summary(fit, fit.measures = TRUE)
#'
#'

# # Example usage:
data(MarshWenHau)
str(MarshWenHau)

mwh = MarshWenHau

model2 <- "
f1 =~ x1 + x2 + x3
f1Xf2 =~ f1:f2
f2 =~ x4 + x5 + x6
f3 =~ y1 + y2 + y3

f3 ~ f1 + f2
f3 ~ f1Xf2

f1Xf2 ~~ 0*f1
f1Xf2 ~~ 0*f2
"

# interaction model
intsem =  SEM(model2, data= MarshWenHau)
summary(intsem)
intsem %>%lav_semPaths2(color=list(lat="darkred",man="gold"))


##2nd---------
#base model
```

```
lav_extract_mm(model2) %>%cat()
#interaction item list
lav_extract_imlist(model3)
#data
lav_latentProd(mwh, model3)%>%head()

#extracted interaction term
lav_extract_int(model2, lav_extract_imlist(model3))

# or interaction items ; imlist
imlist <- list(
  .f1Xf2 = c("f1", "f2"),
  .i_f1_f4 = c("f1", "f4")
)
# names(imlist)
lav_extract_int(model2, imlist)
#direct
lav_extract_int(model2, imlist= list(.f1Xf2 = c("f1", "f2"),
                                     .i_f1_f4 = c("f1", "f4")))
#estimates model
intsem =  SEM(lav_new_model(model2, lav_extract_imlist(model2)),
              data= lav_latentProd(mwh, model2))

#'

## End(Not run)
```

`move_col` *move_col*

Description

move_col

Usage

```
move_col(data, from, to, history = FALSE)
```

Arguments

`data`	df
`from`	from
`to`	to
`history`	histor show text

Value

df

Examples

```
## Not run:
#'
# 사용 예시
df <- data.frame(
  A = 1:5,
  B = 6:10,
  C = 11:15,
  D = 16:20
)

# 2번째 열을 4번째 위치로 이동
result_df <- move_col(df, from = 2, to = 4, history = TRUE)
print(result_df)


## End(Not run)
```

`move_cols` *move_cols replication*

Description

move_cols replication

Usage

```
move_cols(data, ..., history = TRUE)
```

Arguments

`data`	data frame
`...`	from to, from to
`history`	history show text

Value

df

Examples

```
## Not run:
#'
# 사용 예시
df <- data.frame(
  A = 1:5,
  B = 6:10,
  C = 11:15,
  D = 16:20
)

# 여러 열을 이동: 2번째 열을 4번째로, 3번째 열을 1번째로 이동
```

```
result_df <- move_cols(df, 2, 4, 3, 1, history = TRUE)
print(result_df)


## End(Not run)
```

`move_mat` *move_mat matrix element coord position*

Description

move_mat matrix element coord position

Usage

```
move_mat(data, ..., imp = NA, replace = FALSE)
```

Arguments

`data`	mat
`...`	coord
`imp`	NA
`replace`	FALSE

Value

mat

Examples

```
## Not run:
#'
# example
data1 <- matrix(c(NA, NA, NA, NA, NA, NA, NA, NA, NA, "D_P3",
                  NA, NA, NA, NA, NA, NA, "진로태도", NA, NA, "D_P2",
                  NA, NA, NA, NA, NA, NA, NA, NA, NA, "D_P1",
                  "C_S5", NA, NA, NA, NA, NA, NA, NA, NA, "B14",
                  "C_S4", NA, NA, NA, NA, NA, NA, NA, NA, "B10",
                  "C_S3", NA, "자기효능감", NA, NA, NA, "진로준비", NA, NA, "B04",
                  "C_S2", NA, NA, NA, NA, NA, NA, NA, NA, "B02"))

# Test with replace = FALSE
move_mat(data, list(10, 10, 1, 2), list(9, 10, 1, 3), list(8, 10, 6,3), replace = FALSE)
move_mat(data, c(10, 10, 1, 2), c(9, 10, 1, 3), c(8, 10, 6,3), replace = FALSE)


# Test with replace = TRUE
move_mat(data, list(10, 10, 1, 2), list(9, 10, 1, 3), list(8, 10, 6,3), replace = TRUE)
move_mat(data, c(10, 10, 1, 2), c(9, 10, 1, 3), c(8, 10, 6,3), replace = TRUE)

## End(Not run)
```

`move_matpos` *move_matpos is element change position*

Description

move_matpos is element change position

Usage

```
move_matpos(data, ..., imp = NA, replace = FALSE)
```

Arguments

`data`	mat
`...`	position var
`imp`	NA
`replace`	FALSE

Value

mat

Examples

```
## Not run:
#'
# # Original matrix
layout_jut <- matrix(
  c(NA, NA, NA, NA, NA, NA, NA, NA, NA, 'D_P3',
    NA, NA, NA, NA, NA, NA, '진로태도', NA, NA, 'D_P2',
    NA, NA, NA, NA, NA, NA, NA, NA, NA, 'D_P1',
    'C_S5', NA, NA, NA, NA, NA, NA, NA, NA, 'B14',
    'C_S4', NA, NA, NA, NA, NA, NA, NA, NA, 'B10',
    'C_S3', NA, '자기효능감', NA, NA, NA, '진로준비', NA, NA, 'B04',
    'C_S2', NA, NA, NA, NA, NA, NA, NA, NA, 'B02',
    'C_S1', NA, NA, NA, NA, NA, NA, NA, NA, 'A_M3',
    NA, NA, NA, NA, NA, NA, NA, NA, NA, 'A_M2',
    NA, NA, NA, NA, NA, NA, '진로동기', NA, NA, 'A_M1'),
  nrow = 10, ncol = 10, byrow = TRUE)
layout_jut
# Apply the matpos function
layout_jut%>% move_matpos(
  list("진로태도", 3, 5),
  list("진로동기", 8, 5),
  list("B02", 1, 3),
  list("B04", 1, 5),
  list("B10", 1, 7),
  list("B14", 1, 9),
  list("D_P3", 4, 10),
  list("D_P2", 6, 10),
  list("D_P1", 8, 10),
  list("A_M1", 10, 3),
```

```
  list("A_M2", 10, 4),
  list("A_M3", 10, 5),
  replace = FALSE
)


matrix(
  c(NA, NA, NA, NA, NA, NA, NA, NA, NA, 'D_P3',
    NA, NA, NA, NA, NA, NA, '진로태도', NA, NA, 'D_P2',
    NA, NA, NA, NA, NA, NA, NA, NA, NA, 'D_P1',
    'C_S5', NA, NA, NA, NA, NA, NA, NA, NA, 'B14',
    'C_S4', NA, NA, NA, NA, NA, NA, NA, NA, 'B10',
    'C_S3', NA, '자기효능감', NA, NA, NA, '진로준비', NA, NA, 'B04',
    'C_S2', NA, NA, NA, NA, NA, NA, NA, NA, 'B02',
    'C_S1', NA, NA, NA, NA, NA, NA, NA, NA, 'A_M3',
    NA, NA, NA, NA, NA, NA, NA, NA, NA, 'A_M2',
    NA, NA, NA, NA, NA, NA, '진로동기', NA, NA, 'A_M1'),
  nrow = 10, ncol = 10, byrow = TRUE)%>% move_matpos(
    list("진로태도", 3, 5),
    list("진로동기", 8, 5),
    list("B02", 1, 3),
    list("B04", 1, 5),
    list("B10", 1, 7),
    list("B14", 1, 9),
    list("D_P3", 4, 10),
    list("D_P2", 6, 10),
    list("D_P1", 8, 10),
    list("A_M1", 10, 3),
    list("A_M2", 10, 4),
    list("A_M3", 10, 5),
    replace = FALSE
  )


## End(Not run)
```

`move_mat_change` *Functions that replace a single component*

Description

Functions that replace a single component

Usage

```
move_mat_change(mat, pattern, imp)
```

Arguments

`mat`	matrix
`pattern`	pattern
`imp`	imputation

Value

matrix

Examples

```
## Not run:
#'
plspm_boot_factor_layout(pls_ts)%>%
  move_mat_cut(1,3)%>%
  move_mat_change("int","i_I")%>%make_mat_text()

layout_New = matrix(
  c( NA, 'IMA',  NA, NA, 'i_I',
     NA, 'QUA',NA,  'VAL', 'LOY',
     'EXP', NA, 'SAT',NA, NA),
  nrow = 3, ncol = 5, byrow = TRUE)

## End(Not run)
```

---

move_mat_cut *move_mat_cut*

Description

move_mat_cut

Usage

```
move_mat_cut(mat, start = 1, n = 3)
```

Arguments

mat	matrix
start	1
n	width = 3

Value

matrix

Examples

```
## Not run:
#'
# 테스트용 행렬 데이터
mat <- matrix(c(NA, "진로동기", NA, "자기효능감", NA,
    "진로준비", NA, "진로태도", NA), nrow = 3, byrow = TRUE)

# 함수 실행
```

```
move_mat_cut(mat, start = 1, n = 3)

## End(Not run)
```

`move_mat_nc` *move_mat_nc*

Description

move_mat_nc

Usage

```
move_mat_nc(mat, row, col, name)
```

Arguments

`mat`	mat
`row`	row
`col`	cp;
`name`	change name

Value

mat

`move_mat_ori` *move_mat_ori*

Description

move_mat_ori

Usage

```
move_mat_ori(mat, sort = TRUE, type = "res")
```

Arguments

`mat`	mat
`sort`	sort =TRUE
`type`	res, coord

Value

coordnate

Examples

```
## Not run:
#'
layout_jut <- matrix(
  c(NA, NA, NA, NA, NA, NA, NA, NA, NA, 'D_P3',
    NA, NA, NA, NA, NA, NA, '진로태도', NA, NA, 'D_P2',
    NA, NA, NA, NA, NA, NA, NA, NA, NA, 'D_P1',
    'C_S5', NA, NA, NA, NA, NA, NA, NA, NA, 'B14',
    'C_S4', NA, NA, NA, NA, NA, NA, NA, NA, 'B10',
    'C_S3', NA, '자기효능감', NA, NA, NA, '진로준비', NA, NA, 'B04',
    'C_S2', NA, NA, NA, NA, NA, NA, NA, NA, 'B02',
    'C_S1', NA, NA, NA, NA, NA, NA, NA, NA, 'A_M3',
    NA, NA, NA, NA, NA, NA, NA, NA, NA, 'A_M2',
    NA, NA, NA, NA, NA, NA, '진로동기', NA, NA, 'A_M1'),
  nrow = 10, ncol = 10, byrow = TRUE)
move_mat_ori(layout_jut)

#coord

move_mat_ori(layout_jut)%>%
mutate(Mat = paste0(element," = c(", row_col,")" ))%>%
pull(Mat)%>%
  paste(collapse=", ")%>%
  cat("\n")

##treatment
 move_mat_ori(layout_jut, type="coord")
#'
plspm_make_layout_mat(
  A_M1 = c(10, 10), A_M2 = c(9, 10),
  A_M3 = c(8, 10), B02 = c(7, 10),
  B04 = c(6, 10), B10 = c(5, 10),
   B14 = c(4, 10), C_S1 = c(8, 1),
   C_S2 = c(7, 1), C_S3 = c(6, 1),
   C_S4 = c(5, 1), C_S5 = c(4, 1),
   D_P1 = c(3, 10), D_P2 = c(2, 10),
    D_P3 = c(1, 10), 자기효능감 = c(6, 3),
    진로동기 = c(10, 7), 진로준비 = c(6, 7),
     진로태도 = c(2, 7)
)


## End(Not run)
```

`move_mat_substr` *move_mat_cut*

Description

move_mat_cut

Usage

```
move_mat_substr(mat, start = 1, n = 3)
```

Arguments

mat	matrix
start	1
n	width = 3

Value

matrix

Examples

```
## Not run:
#'
# 테스트용 행렬 데이터
mat <- matrix(c(NA, "진로동기", NA, "자기효능감", NA,
    "진로준비", NA, "진로태도", NA), nrow = 3, byrow = TRUE)

# 함수 실행
move_mat_cut(mat, start = 1, n = 3)


## End(Not run)
```

---

move_row *Functions to move rows to a desired position*

Description

Functions to move rows to a desired position

Usage

```
move_row(data, from, to, history = FALSE)
```

Arguments

data	data.fram
from	row from
to	row to
history	history

Value

data

Examples

```
## Not run:
#' #'
#' # Example data
car = data.frame(
  rownames = c(paste0("Row_",1:6)),
  mpg = c('21', '21', '22.8', '21.4', '18.7', '18.1'),
  cyl = c('6', '6', '4', '6', '8', '6'),
  disp = c('160', '160', '108', '258', '360', '225')
)

car
car%>%move_row(1,6)
car%>%move_row(6,1)
car%>%move_row(6,2)
car%>%move_row(6,3)
car%>%move_row(1,3)
car%>%move_row(6,5)

car
car%>%
  move_row(6,1)%>%
  move_row(1,3)%>%
  move_row(3,2)
#'

## End(Not run)
```

`move_rows` *move_row rep function*

Description

move_row rep function

Usage

```
move_rows(data, ..., history = TRUE)
```

Arguments

`data`	data
`...`	movelist c(a,b),c(c,d)...

Value

data

Examples

```
## Not run:
#'
data <- data.frame(
  source = c("A05", "A08", "A04", "A06",
  "A09", "A11", "A13", "A02", "A03", "A01",
  "A14", "A15", "A07", "A12", "A10",
  "eigen_value", "Proportion_Var", "Cumulative_Var"),
  MR1 = c("", "", "", "", "", "", "",
  "0.362", "0.449", "0.457", "0.47",
  "0.538", "0.62", "0.792", "0.96",
  "3.363", "0.224", "0.224"),
  MR2 = c("", "0.382", "", "", "0.445",
  "0.724", "0.906", "0.392", "", "",
  "0.487", "", "", "", "", "2.095",
   "0.14", "0.364"),
  MR3 = c("", "", "0.402", "0.963", "", "",
   "", "", "", "", "", "", "", "",
    "", "1.691", "0.113", "0.477")
)
data
#'
move_row(data, 14,1)
move_row(data, 14,1)%>% move_row(1,2)
move_row(data, 14,1)%>% move_row(1,2)%>%
 move_row(2,3)
#'
data %>% move_rows(c(14, 1), c(1, 2))
data %>% move_rows(c(14, 1), c(1, 2), c(2, 3))
#'
car = data.frame(
  rownames = c(paste0("Row_",1:6)),
  mpg = c('21', '21', '22.8', '21.4', '18.7', '18.1'),
  cyl = c('6', '6', '4', '6', '8', '6'),
  disp = c('160', '160', '108', '258', '360', '225'))
cars

#repeated
cars
car %>%
  move_row(6,1)%>%
  move_row(1,3)%>%
  move_row(3,2)

#history
car %>%
  move_row(6,1, history=TRUE)%>%
  move_row(1,3, history=TRUE)%>%
  move_row(3,2, history=TRUE)%>%
  move_row(2,6, history=TRUE)

car%>%move_rows(6,1) #errorcheck
car%>%move_rows(c(6,1), c(1,2),c(2,6))
car%>%move_rows(c(6,1), c(1,2),c(2,6), history=T)
```

```
# simple input method
move_rows(car, 6, 1, 1, 2, 2, 6, history=TRUE)
car%>% move_rows( 6, 1, 1, 2, 2, 6, 6,3, history=TRUE)
# Testing the function with direct pair inputs
move_rows(car, 6, 1, 1, 2, 2, 6, history=TRUE)

# Testing the function with list of pairs inputs
move_rows(car, c(6, 1), c(1, 2), c(2, 6), history=TRUE)


# Testing the function with direct pair inputs
move_rows(car, 6, 1,  history=FALSE)
move_rows(car, 6, 1,1,2)
move_rows(car, 6, 1, 1, 2, 2, 6, history=TRUE)
car%>%move_rows(6, 1, 1, 2, 2, 6, history=TRUE)
move_rows(car, 6, 1, 1, 2, 2, 6, 6,3, history=TRUE)
car%>%move_rows(c(6,1), c(1,2),c(2,6),c(6,5), history=T)
car%>%move_rows(6, 1, 1, 2, 2, 6, 6,3, history=T)


# Testing the function with list of pairs inputs
move_rows(car, c(6, 1), history=TRUE)
move_rows(car, c(6, 1),c(1,2), history=TRUE)
move_rows(car, c(6, 1), c(1, 2), c(2, 6),c(6,3), history=TRUE)
#'
#'




## End(Not run)
```

`mplus_est_df` *mplus_est_df*

Description

mplus_est_df

Usage

```
mplus_est_df(text, digits = 3)
```

Arguments

`text`	text est
`digits`	3

Value

data.frame

Examples

```
## Not run:
#'
esttest <- "MODEL RESULTS

                                                    Two-Tailed
                   Estimate       S.E.  Est./S.E.    P-Value

 F1       BY
    X1                1.000      0.000    999.000    999.000
    X2                0.771      0.034     22.380      0.000
    X3                0.683      0.033     20.510      0.000

 F2       BY
    X4                1.000      0.000    999.000    999.000
    X5                0.722      0.032     22.400      0.000
    X6                0.702      0.033     21.320      0.000

 F3       BY
    Y1                1.000      0.000    999.000    999.000
    Y2                0.705      0.029     24.023      0.000
    Y3                0.695      0.029     24.243      0.000

 F2       WITH
    F1               -0.482      0.082     -5.912      0.000

 F3       WITH
    F1                0.522      0.084      6.203      0.000
    F2               -0.558      0.088     -6.315      0.000

 Variances
    F1                1.347      0.121     11.095      0.000
    F2                1.492      0.131     11.382      0.000
    F3                1.599      0.135     11.868      0.000

 Residual Variances
    X1                0.364      0.046      7.916      0.000
    X2                0.245      0.029      8.583      0.000
    X3                0.320      0.029     11.160      0.000
    X4                0.347      0.048      7.224      0.000
    X5                0.279      0.029      9.579      0.000
    X6                0.342      0.031     10.865      0.000
    Y1                0.302      0.044      6.931      0.000
    Y2                0.290      0.028     10.444      0.000
    Y3                0.269      0.026     10.230      0.000"

mplus_est_df(esttest)
#'

## End(Not run)
```

`mplus_fit_df` *mplus_fit_df: model fit from Mplus result*

Description

mplus_fit_df: model fit from Mplus result

Usage

```
mplus_fit_df(text, digits = 3)
```

Arguments

`text`	mplut copy data
`digits`	3

Value

fit data.frame

Examples

```
## Not run:


text= c("MODEL FIT INFORMATION

Number of Free Parameters                       21

Loglikelihood

          H0 Value                       -4568.339
          H1 Value                       -4551.194

Information Criteria

          Akaike (AIC)                    9178.677
          Bayesian (BIC)                  9263.623
          Sample-Size Adjusted BIC        9196.983
            (n* = (n + 2) / 24)

Chi-Square Test of Model Fit

          Value                             34.290
          Degrees of Freedom                    24
          P-Value                           0.0796

RMSEA (Root Mean Square Error Of Approximation)

          Estimate                           0.032
          90 Percent C.I.                    0.000  0.054
          Probability RMSEA <= .05           0.901
```

```
CFI/TLI

          CFI                                 0.996
          TLI                                 0.994

Chi-Square Test of Model Fit for the Baseline Model

          Value                            2533.266
          Degrees of Freedom                 36
          P-Value                          0.0000

SRMR (Standardized Root Mean Square Residual)

          Value                               0.022")

mplus_fit_df(text)
#'
## End(Not run)
```

`mr` *Functions to move rows to a desired position*

Description

Functions to move rows to a desired position

Usage

```
mr(data, from, to, history = FALSE)
```

Arguments

`data`	data.fram
`from`	row from
`to`	row to
`history`	history

Value

data

Examples

```
## Not run:
#'
#' # Example data
car = data.frame(
  rownames = c(paste0("Row_",1:6)),
  mpg = c('21', '21', '22.8', '21.4', '18.7', '18.1'),
  cyl = c('6', '6', '4', '6', '8', '6'),
  disp = c('160', '160', '108', '258', '360', '225')
```

```
)
car
car %>% mr(1,6)
car %>% mr(6,1)
car %>% mr(6,2)
car %>% mr(6,3)
car %>% mr(1,3)
car %>% mr(6,5)

#history
car %>%
  mr(6,1)%>%
  mr(1,3)%>%
  mr(3,2)
#'

## End(Not run)
```

`mrs` *move_row rep function*

Description

move_row rep function

Usage

```
mrs(data, ..., history = TRUE)
```

Arguments

`data`	data
`...`	movelist c(a,b),c(c,d)...

Value

data

Examples

```
## Not run:
#'
data <- data.frame(
  source = c("A05", "A08", "A04",
  "A06", "A09", "A11", "A13", "A02",
  "A03", "A01", "A14", "A15", "A07",
  "A12", "A10", "eigen_value", "Proportion_Var", "Cumulative_Var"),
  MR1 = c("", "", "", "", "", "", "",
  "0.362", "0.449", "0.457", "0.47",
  "0.538", "0.62", "0.792", "0.96",
  "3.363", "0.224", "0.224"),
```

```
  MR2 = c("", "0.382", "", "", "0.445",
  "0.724", "0.906", "0.392", "", "",
  "0.487", "", "", "", "", "2.095", "0.14", "0.364"),
  MR3 = c("", "", "0.402", "0.963", "",
   "", "", "", "", "", "", "", "",
    "", "", "1.691", "0.113", "0.477")
)
data
#'
move_row(data, 14,1)
move_row(data, 14,1)%>% move_row(1,2)
move_row(data, 14,1)%>% move_row(1,2)%>% move_row(2,3)
#'
data %>% move_rows(c(14, 1), c(1, 2))
data %>% move_rows(c(14, 1), c(1, 2), c(2, 3))
#'
car = data.frame(
  rownames = c(paste0("Row_",1:6)),
  mpg = c('21', '21', '22.8', '21.4', '18.7', '18.1'),
  cyl = c('6', '6', '4', '6', '8', '6'),
  disp = c('160', '160', '108', '258', '360', '225'))
cars

#repeated
cars
car %>%
  move_row(6,1)%>%
  move_row(1,3)%>%
  move_row(3,2)

#history
car %>%
  move_row(6,1, history=TRUE)%>%
  move_row(1,3, history=TRUE)%>%
  move_row(3,2, history=TRUE)%>%
  move_row(2,6, history=TRUE)

car%>%move_rows(6,1) #errorcheck
car%>%move_rows(c(6,1), c(1,2),c(2,6))
car%>%move_rows(c(6,1), c(1,2),c(2,6), history=T)

# simple input method
move_rows(car, 6, 1, 1, 2, 2, 6, history=TRUE)
car%>% move_rows( 6, 1, 1, 2, 2, 6, 6,3, history=TRUE)
# Testing the function with direct pair inputs
move_rows(car, 6, 1, 1, 2, 2, 6, history=TRUE)

# Testing the function with list of pairs inputs
move_rows(car, c(6, 1), c(1, 2), c(2, 6), history=TRUE)


# Testing the function with direct pair inputs
move_rows2(car, 6, 1,  history=FALSE)
move_rows2(car, 6, 1,1,2)
move_rows2(car, 6, 1, 1, 2, 2, 6, history=TRUE)
car%>%move_rows2(6, 1, 1, 2, 2, 6, history=TRUE)
move_rows2(car, 6, 1, 1, 2, 2, 6, 6,3, history=TRUE)
```

```
car%>%move_rows2(c(6,1), c(1,2),c(2,6),c(6,5), history=T)
car%>%move_rows2(6, 1, 1, 2, 2, 6, 6,3, history=T)


# Testing the function with list of pairs inputs
move_rows2(car, c(6, 1), history=TRUE)
move_rows(car, c(6, 1),c(1,2), history=TRUE)
move_rows2(car, c(6, 1), c(1, 2), c(2, 6),c(6,3), history=TRUE)
#'
#'




## End(Not run)
```

`msdn_contrast` *Contrast test with mean, standard deviation, and number of samples*

Description

Contrast test with mean, standard deviation, and number of samples

Usage

```
msdn_contrast(M, sd, n, ..., digits = 3, type = "res", meanadd = TRUE)
```

Arguments

`M`	means M = c(26.66, 19.74, 15.10)
`sd`	sd sd = c(4.51, 1.45, 2.56)
`n`	sample n = c(11, 7, 14)
`...`	contrast c(1, -1, 0), c(1, 0, -1), c(0, 1, -1) ....
`digits`	default 3
`type`	res, res_ci
`meanadd`	include means

Value

conrast result

Examples

```
## Not run:


msdn_contrast(M = c(26.66, 19.74, 15.10),
              sd = c(4.51, 1.45, 2.56),
              n = c(11, 7, 14),
              c(1, -1, 0) )
```

```
contrast_msdn(M = c(26.66, 19.74, 15.10),
              sd = c(4.51, 1.45, 2.56),
              n = c(11, 7, 14),
              c(1, -1, 0) ,
              meanadd = FALSE)

msdn_contrast(M = c(26.66, 19.74, 15.10),
              sd = c(4.51, 1.45, 2.56),
              n = c(11, 7, 14),
              c(1, -1, 0) ,
             meanadd = FALSE,
              type="res_ci")


msdn_contrast(M = c(26.66, 19.74, 15.10),
              sd = c(4.51, 1.45, 2.56),
             n = c(11, 7, 14),
              c(1, -1,  0),
              c(1,  0, -1),
              c(0,  1, -1)
)
msdn_contrast(M = c(26.66, 19.74, 15.10),
              sd = c(4.51, 1.45, 2.56),
              n = c(11, 7, 14),
              contr1 = c(1, -1,  0),
              contr2= c(1,  0, -1),
              contr3= c(0,  1, -1),
              meanadd=F
)
msdn_contrast(M = c(26.66, 19.74, 15.10),
              sd = c(4.51, 1.45, 2.56),
              n = c(11, 7, 14),
             contr1 = c(1, -1,  0),
             contr2= c(1,  0, -1),
              contr3= c(0,  1, -1),
              meanadd=FALSE, type="res_ci"
)
#error
msdn_contrast(M = c(26.66, 19.74, 15.10),
              sd = c(4.51, 1.45, 2.56),
             n = c(11, 7, 14),
              contr = list(c(1, -1,  0),
                           c(1,  0, -1),
                          c(0,  1, -1)
              ))



## End(Not run)
```

`msdn_ttest` *Using ttest with mean, sd, and n*

Description

Using ttest with mean, sd, and n

Usage

```
msdn_ttest(
  n1,
  m1,
  sd1,
  n2,
  m2,
  sd2,
  var1 = NULL,
  var2 = NULL,
  var.equal = FALSE,
  digits = 2
)
```

Arguments

n1	sample1
m1	mean1
sd1	sd1
n2	n2
m2	m2
sd2	sd2
var1	var1 name
var2	var2 name
var.equal	TRUE Student FALSE, whech
digits	3

Value

ttest result

Examples

```
## Not run:
#'
# # Example usage
msdn_ttest(n1 = 603, m1 = 59.6, sd1 = 0.6,
           n2 = 10785, m2 = 46.7, sd2 = 0.3)

#  mysummaryBy(mpg ~ vs, data = mtcars)

# library(jjstat)
msdn_ttest(n1 = 18, m1 = 16.617, sd1 = 3.8607,
           n2 = 14, m2 = 24.557, sd2 = 5.3790,  var.equal=FALSE)
t.test(mpg ~ vs, data = mtcars, var.equal=F)%>%tidy()%>%dall

msdn_ttest(n1 = 18, m1 = 16.617, sd1 = 3.8607,
```

```
            n2 = 14, m2 = 24.557, sd2 = 5.3790, var.equal=TRUE)
t.test(mpg ~ vs, data = mtcars, var.equal=T)%>%tidy()

#'
msdn_ttest(n1 = 18, m1 = 16.617, sd1 = 3.8607, var1 = "vs0",
            n2 = 14, m2 = 24.557, sd2 = 5.3790, var2 = "vs1", var.equal = FALSE)
msdn_ttest(n1 = 18, m1 = 16.617, sd1 = 3.8607, var1 = "vs1",
            n2 = 14, m2 = 24.557, sd2 = 5.3790, var2 = "vs2", var.equal = FALSE)



## End(Not run)
```

`mutate_col` *Create, modify, and delete columns col position*

Description

Create, modify, and delete columns col position

Usage

```
mutate_col(data, ..., col = ncol(data))
```

Arguments

`data`	data.frame
`...`	creat name
`col`	position

Value

data

Examples

```
## Not run:
#'
fm <- data.frame(
  age = c("20대", "20대", "20대", "30대",
  "30대", "30대", "40대", "40대", "40대"),
  manufacturer = c("google", "apple", "samsung",
  "google", "apple", "samsung", "google",
   "apple", "samsung"),
  values = c(794, 804, 1243, 987, 1055,
  2517, 1528, 1519, 2255)
)
fm

#data
fm %>% mutate_col(gender= rep("a",9))
fm %>% mutate_col(gender= rep("a",9), col = 2)
```

```
#position
fm %>% mutate_col(gender = rep("female", nrow(fm)), col = 2)
fm %>% mutate_col(gender = rep("female", nrow(fm)), col = 1)
fm %>% mutate_col(gender = rep("female", nrow(fm)), col = 3)
fm %>% mutate_col(gender = rep("female", nrow(fm)), col = 4)

#Build and adjust with data from within
fm %>% mutate(gender = age)
fm %>% mutate_col(age_int = substring(fm$age,1,2))
fm %>% mutate_col(gender = paste(fm$age,fm$values,sep="//"))

fm %>% mutate_col(gender = paste(.$age,.$values,sep="//"), col=2)
# Double data generation
fm %>%
  mutate_col(insert_2 = rep("female", nrow(fm)), col = 2) %>%
  mutate_col( insert_4 = rep("a",9), col = 4)


# second data
data2 <- data.frame(
  weight = c(4.17, 5.58, 5.18, 6.11, 4.50, 4.61, 5.17, 4.53, 5.33, 5.14,
             4.81, 4.17, 4.41, 3.59, 5.87, 3.83, 6.03, 4.89, 4.32, 4.69,
             6.31, 5.12, 5.54, 5.50, 5.37, 5.29, 4.92, 6.15, 5.80, 5.26),
  ids = c(paste0("C", 1:10), paste0("T",1:10), paste0("S", 1:10))
)
data2 %>%mutate_col(group= .$weight, col=2)


## End(Not run)
```

`mydes` *mydes 'n','mean','sd','min','max'*

Description

mydes 'n','mean','sd','min','max'

Usage

```
mydes(myvariable, var = NULL, digits = 2)
```

Arguments

`myvariable`	variable vector
`var`	input variable name
`digits`	digits

Examples

```
## Not run:
## view variable
mydes(mtcars$mpg,"mpg")
##not see variable
mydes(mtcars$mpg)


## End(Not run)
```

`mysummary` *my summary descriptive statistics*

Description

my summary descriptive statistics

Usage

```
mysummary(myobject, ..., digits = 5, msdn = FALSE, na.rm = FALSE)
```

Arguments

`myobject`	data.frame, matrix
`...`	column variable
`digits`	round default 3
`msdn`	TRUE only output var, N, mean. sd
`na.rm`	FALSE, TRUE is remove NA

Value

size MEAN SD MIN MAX

Examples

```
## Not run:
mysummary(mtcars, "mpg","wt","hp")
## size      MEAN         SD    MIN     MAX
## 1   32  20.09062  6.0269481 10.400  33.900
## 2   32   3.21725  0.9784574  1.513   5.424
## 3   32 146.68750 68.5628685 52.000 335.000

## all variable
 jjstat::mysummary(mtcars, colnames(mtcars))

mysummary(mtcars, colnames(mtcars))

mysummary(mtcars)

mysummary(mtcars, all=FALSE)


## End(Not run)
```

`mysummaryBy` *statistics by group*

Description

statistics by group

Usage

```
mysummaryBy(
  data,
  formula,
  add_var = NULL,
  stat = FALSE,
  agg = FALSE,
  gm = FALSE,
  fun = NULL,
  msdn = FALSE,
  t = FALSE
)
```

Arguments

`data`	data.frame
`formula`	general formula
`add_var`	one sample name, Used to obtain statistics for one variable
`stat`	TRUE t.test and aov(), thes stat ="t.test", or 'aov'
`agg`	TRUE, mnay to many variable is showed. This functuion tha makes aggregate() work full
`gm`	default FALSE, TRUE Use group data obtained from group descriptive statistics to obtain group-level descriptive statistics (means, standard deviations, etc.) and to analyze a mixed model
`msdn`	msdn=TRUE is output mean, sd, N
`t`	transpose when agg=TRUE

Value

Mea, SD, N, min, max, skew, kurt

Examples

```
## Not run:
## Used to obtain statistics for one variable
mysummaryBy(mpg ~ 1, data = mtcars)
mysummaryBy(mpg ~ 1, data = mtcars, "mpg")
mysummaryBy(mpg ~ 1, data = mtcars, add_var = "mpg")

##The group variable is 2 or greater
mysummaryBy(mpg ~ vs, data = mtcars)
mysummaryBy(mpg ~ vs+am, data = mtcars)
```

```
mysummaryBy(mpg ~ vs+am+cyl, data = mtcars)

##statistic data output
mysummaryBy(mpg ~ 1, data = mtcars, "mpg", stat="t.test")
mysummaryBy(mpg ~ vs, data = mtcars, stat="t.test")
mysummaryBy(mpg ~ vs+am, data = mtcars, stat="aov")
mysummaryBy(mpg ~ vs*cyl, data = mtcars, stat="aov")

##new group mean by group
#' group_by(mtcars, am) %>%
 summarise(mpg = mean(mpg)) %>%
  summarise(mean(mpg),sd(mpg),length(mpg), min(mpg),max(mpg))

### A tibble: 1 × 5
###`mean(mpg)` `sd(mpg)` `length(mpg)` `min(mpg)` `max(mpg)`
###<dbl>     <dbl>          <int>      <dbl>      <dbl>
###  1        20.8      5.12              2       17.1       24.4

mysummaryBy(mpg ~ am , data= mtcars) %>%
    rename(mpg = Mean) %>%   #change name
      mysummaryBy(formula = mpg ~ 1)  # mean of mpg

mysummaryBy(mpg ~ am , data= mtcars) %>%
    rename(mpg = Mean) %>%   #change name
      mysummary("mpg")  # mean of mpg

#multi group transpose table
mysummaryBy(mpg ~ vs+am, data = mtcars, agg=TRUE)

## transpose
mysummaryBy(mpg ~ vs+am, data = mtcars, agg=TRUE, t=TRUE)

##twoway
mysummaryBy(mpg ~ vs+am, data = mtcars, agg=TRUE)


## twoway interaction
mysummaryBy(mpg ~ vs*am, data = mtcars, agg=TRUE, stat="aov")

#add variable
mysummaryBy(mpg ~ vs, data = mtcars, gm=T)
# A tibble: 1 × 9
# grp   dv        N  MEAN    SD   MIN   MAX  Skew  Kurt
# <chr> <chr> <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
#  vs    mpg       2  20.6  5.61  16.6  24.6     0 -2.75


## End(Not run)
```

`mysummary_apa` *mysummary_apa: Generate descriptive statistics in APA style (English or Korean)*

Description

mysummary_apa: Generate descriptive statistics in APA style (English or Korean)

Usage

```
mysummary_apa(df, sd = FALSE, eng = FALSE)
```

Arguments

df	A data frame containing descriptive statistics, including variables such as N, MEAN, SD, Skew, and Kurt.
sd	A logical value. If TRUE, includes standard deviation (SD) along with the mean in the output. Default is FALSE.
eng	A logical value. If TRUE, the output is in English. If FALSE, the output is in Korean. Default is TRUE.

Value

A string formatted in APA style (English or Korean), summarizing the descriptive statistics in order of mean values and assessing normality based on skewness and kurtosis.

Examples

```
## Not run:
df <- data.frame(
  var = c("Q33_1", "Q33_2", "Q33_3"),
  N = c(4542, 4542, 4542),
  MEAN = c(1.9489, 1.9337, 1.9243),
  SD = c(0.84679, 0.86568, 0.86960),
  Skew = c(0.72576, 0.63900, 0.70717),
  Kurt = c(0.63369, 0.087763, 0.27470)
)
mysummary_apa(df, sd = TRUE, eng = TRUE)
mysummary_apa(df, sd = TRUE, eng = FALSE)

## End(Not run)
```

---

na2zero *NA replace to value default 0*

Description

NA replace to value default 0

Usage

```
na2zero(data, imp = 0, range = NULL)
```

Arguments

data	data.frame or matrix
imp	imputation value default 0
range	range is column

Value

dat

Examples

```
## Not run:

data %>% na2zero()
data %>% na2zero(0)
data %>% na2zero(1)

#'

# Example data frame entry
df <- data.frame(a = c(1, NA, 3), b = c(NA, 2, 3))
na2zero(df)

# Vector input examples
vec <- c(1, NA, 3)
print(na2zero(vec))

# Character Vector Input Example
char_vec <- c("a", NA, "b")
print(na2zero(char_vec))
#'


## End(Not run)
```

---

na_remover *na_remover row unite*

Description

na_remover row unite

Usage

```
na_remover(df_var, pattern = "", type = "data")
```

Arguments

df_var	df vector
pattern	""
type	data

Value

df

`nice_aov_df` *Nice tables for papers in aov_df results to word, html*

Description

Nice tables for papers in aov_df results to word, html

Usage

```
nice_aov_df(df, show = TRUE, tibble = FALSE, digits = 2)
```

Arguments

`df`	aov_Df
`show`	TRUE nice, FALSE pass
`digits`	2

Value

result

Examples

```
## Not run:
#'
aov_df(data = mtcars,
       dv_var = "mpg",
       iv_vars = c("cyl", "gear", "carb") )

## generate nice table
aov_df(data = mtcars,
       dv_var = "mpg",
       iv_vars = c("cyl", "gear", "carb") )%>%
  nice_aov_df()


aov_df(data = mtcars,
       dv_var = "mpg",
       iv_vars = c("cyl", "gear", "carb") )%>%
  nice_aov_df() %>%web()



## End(Not run)
```

nice_table 같은것이반복되면”” 처리

Description

같은것이반복되면”” 처리

Usage

```
nice_table(data, exclude = NULL)
```

Arguments

data	data
exclude	not adapt col

Value

data ressult

Examples

```
## Not run:
#'
# Example usage
data <- data.frame(
  Var = c("a", "a", "a", "a", "a", "a", "a", "b",
  "b", "b", "b", "b", "c", "c", "c", "c"),
  alpha_95.CI = c("alpha = 0.855, 95%CI[0.8, 0.9]",
  "alpha = 0.855, 95%CI[0.8, 0.9]",
                  "alpha = 0.855, 95%CI[0.8, 0.9]",
                  "alpha = 0.855, 95%CI[0.8, 0.9]",
                  "alpha = 0.855, 95%CI[0.8, 0.9]",
                  "alpha = 0.855, 95%CI[0.8, 0.9]",
                  "alpha = 0.805, 95%CI[0.74, 0.86]",
                  "alpha = 0.805, 95%CI[0.74, 0.86]",
                  "alpha = 0.805, 95%CI[0.74, 0.86]",
                  "alpha = 0.805, 95%CI[0.74, 0.86]",
                  "alpha = 0.805, 95%CI[0.74, 0.86]",
                  "alpha = 0.534, 95%CI[0.36, 0.67]",
                  "alpha = 0.534, 95%CI[0.36, 0.67]",
                  "alpha = 0.534, 95%CI[0.36, 0.67]",
                  "alpha = 0.534, 95%CI[0.36, 0.67]"),
  subfactor = c("B02", "B14", "B10", "B04", "B01", "B15",
   "B03", "B09", "B25", "B07", "B06", "B05", "B17",
   "B18", "B21", "B22"),
  cronbach_alpha = c(0.8175800, 0.8192332,
  0.8375942, 0.8163753, 0.8505183, 0.8488448,
   0.8445658, 0.7607713, 0.7494337,
                     0.7534147, 0.7620378,
                      0.8099670, 0.4376783,
                       0.5111512, 0.4434714, 0.4554295)
```

```
)

# Apply the function to the data
nice_table(data)


## End(Not run)
```

`nice_Var` *bind_alpha_table* 문자정리함수

Description

bind_alpha_table 문자정리함수

nice_Var alpha 분석한경우첫열의 a1->a

Usage

```
nice_Var(data)

nice_Var(data)
```

Arguments

`data`	alpha data

Value

table data.frame

data.frame

Examples

```
## Not run:
#'
jutr2 %>% select(B01:B25) %>%
  bind_alpha_table(c(1:5),c(6:10),c(11:15),c(16:20),c(21:25))%>%
  nice_Var()%>%  # <----this
  nice_table()

## End(Not run)
```

`Normality_test` *Multivariate normality tests*

Description

Multivariate normality tests

Usage

```
Normality_test(multiVariable_data)
```

Arguments

`multiVariable_data` data

Value

normality

Examples

```
## Not run:
mtcars%>%Normality_test()

## End(Not run)
```

`obs_exp_table` *Observation Expectation Table with calculate_chi_sig*

Description

Observation Expectation Table with calculate_chi_sig

Usage

```
obs_exp_table(obs_data)
```

Arguments

`obs_data` matrix

Value

matrix

oneway_aov *oneway ANVOA calculation*

Description

oneway ANVOA calculation

Usage

```
oneway_aov(formula, data, format = "pandoc")
```

Arguments

formula	ex)mpg ~ cyl
data	data.frame
format	'markdwon', 'pandoc', 'html'

Value

anova result

Examples

```
## Not run:
oneway_aov(mpg ~ cyl, data = mtcars)

## End(Not run)
```

pall *Print and Inspect Data with Multiple Output Modes*

Description

A unified data inspection function supporting console printing, RStudio viewing, interactive DataTables (DT), and publication-ready tables via knitr::kable().

Usage

```
pall(
  data,
  n = Inf,
  type = c("print", "view", "dt", "kable"),
  dt_browser = TRUE,
  dt_page = 20,
  dt_scroll_y = 650,
  dt_digits = 2,
  dt_highlight_na = TRUE,
  kable_format = c("markdown", "html", "latex", "pipe"),
  kable_digits = 3,
```

```
  max_n = 5000,
  summary = TRUE,
  return_data = FALSE
)
```

Arguments

data	A data.frame or tibble.
n	Number of rows to display. Default is Inf.
type	Output type: "print", "view", "dt", or "kable".
dt_browser	Logical. If TRUE (default), DT output opens in browser.
dt_page	Number of rows per DT page.
dt_scroll_y	Vertical scroll height (px) for DT.
dt_digits	Decimal digits for numeric variables in DT.
dt_highlight_na	Logical; highlight missing values in DT.
kable_format	Output format for kable: "markdown", "html", "latex", or "pipe".
kable_digits	Decimal digits for numeric values in kable output.
max_n	Safety limit for maximum rows.
summary	Logical; print row/column summary.
return_data	Logical; return sliced data invisibly.

Details

In DT mode, the table is rendered in a web browser by default, with automatic variable-type detection and a summary panel displayed above the table.

In kable mode, output is routed automatically: markdown / pipe / latex -> console html -> web browser

Value

Invisibly returns the sliced data.

Examples

```
pall(mtcars, n = 10)
pall(iris, type = "dt")
pall(head(mtcars), type = "kable", kable_format = "markdown")
pall(head(mtcars), type = "kable", kable_format = "htmml")
```

path *Define a Structural Path (DSL)*

Description

구조방정식에서잠재변수간회귀경로를정의하기위한 DSL 함수이다.

Usage

```
path(from, to)

path(from, to)
```

Arguments

from	var1
to	var2

Value

list with elements `from`, `to`.

Examples

```
path(c("F1","F2"), "F3")
```

patternGraph *patternGraph*

Description

patternGraph

Usage

```
patternGraph(
  data,
  size = 4,
  strip = 14,
  text = 12,
  axis = 16,
  hjust = -0.4,
  yadd = 10,
  type = "g",
  xlab = "성조형",
  ylab = "빈도",
  show = TRUE,
  tolong = TRUE
)
```

Arguments

`data`	data.frame
`size`	bar text size
`strip`	strip size
`text`	axis text test size
`axis`	aixs size
`hjust`	hjust
`yadd`	yadd
`type`	type ”g”, ”data”, ”all”
`xlab`	xlab
`ylab`	ylab
`show`	show.legend
`tolong`	long format transpose

Value

graph

Examples

```
## Not run:
aaa=kge_bind2 %>% filter(지역 =="부산" & N ==1) %>%
  auto_pattern("고유어1") %>%
  kge_chisq_table("a1","성조", "고유어1 부산")

#sAVE
data_long<- aaa$chi_table %>%
  rownames_to_column("syllabic") %>%
  pivot_longer(names_to = "accent", values_to = 'ratio', cols=2:4)
data_long

data_long%>% ggplot(aes(x = accent, y = ratio))+
  geom_bar(stat = "identity", aes( fill = accent),
           position = "dodge", show.legend = FALSE)+
  geom_hline(yintercept = 1, linetype=2, color="gray80")+
  geom_text(aes(label = round(ratio,2)), hjust = -0.1, size = 4)+
  ylim(0,max(data1$ratio)+0.07)+
  coord_flip()+
  theme_bw()+
  theme(axis.text = element_text(size= 12),
        axis.title = element_text(size= 12),
        strip.text = element_text(size= 14)
  )+
  scale_fill_grey(start = 0, end = 0.7) +
  facet_wrap(~ syllabic )

aaa%>% patternGraph()


## End(Not run)
```

patternGraph2 *patternGraph2*

Description

patternGraph2

Usage

```
patternGraph2(
  data,
  type = "g",
  raw = TRUE,
  Ncol = NULL,
  yadd = 0.45,
  xlab = "음절",
  ylab = "관측기대비율",
  strip_size = 16,
  axis_size = 16,
  text_size = 14,
  Colsnames = NULL,
  size_bartext = 5
)
```

Arguments

data	data,frame
type	g
raw	raw data
Ncol	grid
yadd	up yadd
xlab	xlab
ylab	ylab
strip_size	strip
axis_size	axis
text_size	text
Colsnames	new select col pivot long
size_bartext	bartext

Value

result fill data

Examples

```
## Not run:
aaa=kge_bind2 %>% filter(지역 =="부산" & N ==1) %>%
  auto_pattern("고유어1") %>%
  kge_chisq_table("a1","성조", "고유어1 부산")

#sAVE
data_long<- aaa$chi_table %>%
  rownames_to_column("syllabic") %>%
  pivot_longer(names_to = "accent", values_to = 'ratio', cols=2:4)
data_long

data_long%>% ggplot(aes(x = accent, y = ratio))+
  geom_bar(stat = "identity", aes( fill = accent),
           position = "dodge", show.legend = FALSE)+
  geom_hline(yintercept = 1, linetype=2, color="gray80")+
  geom_text(aes(label = round(ratio,2)), hjust = -0.1, size = 4)+
  ylim(0,max(data1$ratio)+0.07)+
  coord_flip()+
  theme_bw()+
  theme(axis.text = element_text(size= 12),
        axis.title = element_text(size= 12),
        strip.text = element_text(size= 14)
  )+
  scale_fill_grey(start = 0, end = 0.7) +
  facet_wrap(~ syllabic )

aaa%>% patternGraph_obs_exp()


## End(Not run)
```

---

patternGraph_obs_exp *patternGraph_obs_exp*

Description

patternGraph_obs_exp

Usage

```
patternGraph_obs_exp(
  data,
  raw = TRUE,
  Ncol = NULL,
  yadd = 0.3,
  strip_size = 16,
  axis_size = 15,
  text_size = 13,
  size_bartext = 4,
  xlab = "성조형",
  ylab = "관측기대비율",
  Colsnames = NULL,
```

```
  type = "g"
)
```

Arguments

data	data,frame
raw	raw data
Ncol	grid
yadd	up yadd
strip_size	strip
axis_size	axis
text_size	text
size_bartext	bartext
xlab	xlab
ylab	ylab
Colsnames	new select col pivot long
type	res

Value

result fill data

Examples

```
## Not run:
aaa=kge_bind2 %>% filter(지역 =="부산" & N ==1) %>%
  auto_pattern("고유어1") %>%
  kge_chisq_table("a1","성조", "고유어1 부산")

#sAVE
data_long<- aaa$chi_table %>%
  rownames_to_column("syllabic") %>%
  pivot_longer(names_to = "accent", values_to = 'ratio', cols=2:4)
data_long

data_long%>% ggplot(aes(x = accent, y = ratio))+
  geom_bar(stat = "identity", aes( fill = accent),
           position = "dodge", show.legend = FALSE)+
  geom_hline(yintercept = 1, linetype=2, color="gray80")+
  geom_text(aes(label = round(ratio,2)), hjust = -0.1, size = 4)+
  ylim(0,max(data1$ratio)+0.07)+
  coord_flip()+
  theme_bw()+
  theme(axis.text = element_text(size= 12),
        axis.title = element_text(size= 12),
        strip.text = element_text(size= 14)
  )+
  scale_fill_grey(start = 0, end = 0.7) +
  facet_wrap(~ syllabic )

aaa%>% patternGraph_obs_exp()
```

```
#신진영(2013)
shin2013_mat =matrix(c(14,1,23,3,4,0,9,0,13,18,89,8,11,40,46,12,3,4,22,0), ncol= 5)
# colnames(shin2013_mat)=c("지능지수가높다","학업성취도가높다","창의적사고를한다","특정분야에서뛰어난
colnames(shin2013_mat)=c("지능지수","학업성취도","창의적사고","특정분야","모든분야")
rownames(shin2013_mat)=c("고졸","초대졸","대졸","대학원이상")
shin2013_mat

shin2013_mat%>% data.frame() %>%
  long_df("학력","영재성", cols=2:6) %>%
  add_rows_freq("real") %>%
  chisq_test_kge("학력","영재성", type="data_graph") %>%
  patternGraph_obs_exp_ko()


## End(Not run)
```

---

patternGraph_obs_exp_ko

patternGraph_obs_exp

Description

patternGraph_obs_exp

Usage

```
patternGraph_obs_exp_ko(
  data,
  raw = TRUE,
  Ncol = NULL,
  yadd = 0.5,
  strip_size = 16,
  axis_size = 16,
  text_size = 13,
  size_bartext = 5,
  xlab = "기준변수",
  ylab = "관측기대비율",
  Colsnames = NULL,
  trans = FALSE,
  type = "g"
)
```

Arguments

data	data,frame
raw	raw data
Ncol	grid
yadd	up yadd
strip_size	strip
axis_size	axis
text_size	text

size_bartext	bartext
xlab	xlab
ylab	ylab
Colsnames	new select col pivot long
trans	trans plot transpose
type	res

Value

result fill data

Examples

```
## Not run:
aaa=kge_bind2 %>% filter(지역 =="부산" & N ==1) %>%
  auto_pattern("고유어1") %>%
  kge_chisq_table("a1","성조", "고유어1 부산")

#sAVE
data_long<- aaa$chi_table %>%
  rownames_to_column("syllabic") %>%
  pivot_longer(names_to = "accent", values_to = 'ratio', cols=2:4)
data_long

data_long%>% ggplot(aes(x = accent, y = ratio))+
  geom_bar(stat = "identity", aes( fill = accent),
           position = "dodge", show.legend = FALSE)+
  geom_hline(yintercept = 1, linetype=2, color="gray80")+
  geom_text(aes(label = round(ratio,2)), hjust = -0.1, size = 4)+
  ylim(0,max(data1$ratio)+0.07)+
  coord_flip()+
  theme_bw()+
  theme(axis.text = element_text(size= 12),
        axis.title = element_text(size= 12),
        strip.text = element_text(size= 14)
  )+
  scale_fill_grey(start = 0, end = 0.7) +
  facet_wrap(~ syllabic )

aaa%>% patternGraph_obs_exp_ko()

#신진영(2013)
shin2013_mat =matrix(c(14,1,23,3,4,0,9,0,13,18,89,8,11,40,46,12,3,4,22,0), ncol= 5)
# colnames(shin2013_mat)=c("지능지수가높다","학업성취도가높다","창의적사고를한다","특정분야에서뛰어난
colnames(shin2013_mat)=c("지능지수","학업성취도","창의적사고","특정분야","모든분야")
rownames(shin2013_mat)=c("고졸","초대졸","대졸","대학원이상")
shin2013_mat

shin2013_mat%>% data.frame() %>%
  long_df("학력","영재성", cols=2:6) %>%
  add_rows_freq("real") %>%
  chisq_test_kge("학력","영재성", type="data_graph") %>%
  patternGraph_obs_exp_ko()
```

```
## End(Not run)
```

`pattern_detector` *pattern_detector: missing data detect*

Description

pattern_detector: missing data detect

Usage

```
pattern_detector(df, missing = TRUE, data = FALSE, pattern = "")
```

Arguments

`df`	data
`missing`	missing TRUE detecte missing T,F
`data`	TRUE data print, FALSE rownames number
`pattern`	pattern "" or NA

Value

data

Examples

```
## Not run:

ed_tam %>% pattern_detector()
ed_tam %>% pattern_detector(FALSE,TRUE)%>% head()
# missing data chek
ed_tam %>% pattern_detector(missing=T, data=F)
#missing data output
ed_tam %>% pattern_detector(missing=T, data=T) %>%head(30)

# not missing
ed_tam %>% pattern_detector(data=T,missing=F) %>%head()

# ot missing , not data
ed_tam %>% pattern_detector(data=F,missing=F)



## End(Not run)
```

`pcfa` *principal component factor analysis function*

Description

principal component factor analysis function

Usage

```
pcfa(
  R = NULL,
  nfactor = NULL,
  rowname = NULL,
  digits = 2,
  xlim = NULL,
  ylim = NULL,
  cex = 1.2,
  vjust = -0.5,
  yadd = 10,
  name = "auto",
  pos = 1
)
```

Arguments

`R`	correlation matrix
`nfactor`	number of factor
`rowname`	matrix rowname
`digits`	digits = 3
`xlim`	NULL
`ylim`	NULL
`cex`	size =1
`yadd`	yadd = 10,
`name`	'auto', 'manual'
`pos`	positon 1

Value

result table, biplot, dimension

Examples

```
## Not run:
R= matrix(
c(1.00, 0.83, 0.78, 0.70, 0.66, 0.63,
  0.83, 1.00, 0.67, 0.67, 0.65, 0.57,
  0.78, 0.67, 1.00, 0.64, 0.54, 0.51,
  0.70, 0.67, 0.64, 1.00, 0.45, 0.51,
  0.66, 0.65, 0.54, 0.45, 1.00, 0.40,
  0.63, 0.57, 0.51, 0.51, 0.40,1.00),
```

```
 byrow=T, nrow=6)
subject=c("classic","france","english","math","pitch","music")

colnames(R)=subject
rownames(R)=subject
R
pcfa(R, nfactor = 2)

library(MVT)
data(examScor)

#if you when input cor
pcfa(cor(examScor), nfactor = 2)

## if you When input data
pcfa( examScor, nfactor = 2)


## End(Not run)
```

`pcfa_porp_plot` *pcfa_porp_plot*

Description

pcfa_porp_plot

Usage

```
pcfa_porp_plot(Data, digits = 2, prop = FALSE, ymax = NA)
```

Arguments

`Data`	factanal result uniqueness
`digits`	digits 3
`prop`	FALSE
`ymax`	plot y size

Value

graph

Examples

```
## Not run:

library(MVT)
library(GPArotation)
data(examScor)
X = examScor
Z = scale(X, scale=T)  # scale
R = cor(X)  # correlation matrix
```

```
## factor analysis tools : factanal
mlfa <- factanal(Z, factors = 2, rotation = "varimax")

mlfa|> pcfa_porp_plot()


## End(Not run)
```

`pdata` *pdata plspm group change long data format*

Description

pdata plspm group change long data format

Usage

```
pdata(widedata, rownames_to_column=FALSE)
```

Arguments

`data`	wide data

Value

long

`perform_chi_square_test`
Various output functions after chi-square analysis

Description

Various output functions after chi-square analysis

Usage

```
perform_chi_square_test(observed, type = "data", simple = FALSE)
```

Arguments

`observed`	table data
`type`	type data, res, chisq_test, cramersv, observed_over_expected_ratios, cell_p_values, p_sig
`simple`	* or ***

Value

chisq table obs/exp

Examples

```
## Not run:

#' #'mat = matrix(c(36, 67, 11,
          0, 35, 44,
         41, 108, 1,
          56, 87, 54),
           nrow = 4, byrow = TRUE.
   dimnames = list(c("공명음", "마찰음",
     "유기음_경음", "평파열음_평파찰음"),  c("H", "H(H)", "L")))

 mat %>% perform_chi_square_test()

 #perform_chi_square_test
mtcars%>%select(am, cyl)%>%table()%>%perform_chi_square_test()
mtcars%>%select(am, cyl)%>%table()%>%perform_chi_square_test("all")

## End(Not run)
```

`pivot_table` *pivot*

Description

pivot

Usage

```
pivot_table(
  data,
  formula,
  names_from = NULL,
  unite = FALSE,
  col = 1:2,
  remove = TRUE,
  change_col = TRUE
)
```

Arguments

`data`	data
`formula`	formul
`names_from`	column
`unite`	1,2 colum unite
`col`	합쳐질컬럼
`remove`	기존것지우기
`change_col`	열의위치바꾸기

Value

데이터

Examples

```
## Not run:

#' kge_bind3a %>%
filter(type == "고유어1")%>%
  pivot_table(~ area + age + 성조, unite=TRUE)
#'
kge_bind3a %>%
  filter(type == "고유어1")%>%
  pivot_table(~  age + area+ 성조, unite=FALSE)

#고유어1 공명음  지역별 연령
kge_bind3a %>%
  filter(type == "고유어1" & a1=="공명음")%>%
  pivot_table(~ age + area + 성조, unite=T)  #%>% md()



kge_bind3a %>%
  filter(type == "고유어1")%>%
  pivot_table(~  age +  성조, unite=FALSE)

#고유어1 공명음  지역별 연령
kge_bind3a %>%
  filter(type == "고유어1" & a1=="공명음")%>%
  pivot_table(~  area + 성조, unite=F)  #%>% md()



## End(Not run)
#'
```

---

plot_lpa *Latent Profile Analysis Profile Plot*

Description

Latent Profile Analysis Profile Plot

Usage

```
plot_lpa(
  data,
  n_profiles = 3,
  model_name = "EEE",
  view = "pair",
  alpha = 0.6,
```

```
  gtype = "normal",
  angle = 90,
  size.x = 13,
  size.p = 5,
  ncol = 2,
  fct_reorder = NULL,
  show.legend = FALSE,
  legend.text.size = 16,
  legend_position = "top",
  linewidth = 1,
  title_size = 18,
  seed = 123
)
```

Arguments

`data`	A data frame containing numeric variables for latent profile analysis.
`n_profiles`	Integer. Number of latent profiles to identify. Default is 3.
`model_name`	Character. Model name to use in Mclust for Gaussian mixture modeling (e.g., "EEE", "EEI", "VVI", "VVV"). Default is "EEE".
`view`	Character. Determines the view type of the plots ("pair", "each"). Default is "pair".
`alpha`	Numeric. Transparency level for bar plots. Default is 0.6.
`gtype`	Character. Graph type to generate ("normal", "trans", "each"). Default is "normal".
`angle`	Numeric. Angle for the x-axis text labels. Default is 90.
`size.x`	Numeric. Font size for the x-axis text labels. Default is 13.
`size.p`	Numeric. Size of points in the plot. Default is 5.
`ncol`	Integer. Number of columns for the facet plots in "each" view. Default is 2.
`fct_reorder`	Character vector or NULL. Specifies the order of factor levels for plotting. Default is NULL.
`show.legend`	Logical. Whether to show legends for line plots. Default is FALSE.
`legend.text.size`	Numeric. Font size for legend text. Default is 16.
`legend_position`	Character. Position of the legend ("top", "bottom", "left", "right"). Default is "top".
`linewidth`	Numeric. Line width for plots. Default is 1.
`title_size`	Numeric. Font size for plot titles. Default is 18.
`seed`	Integer or NULL. Seed value for reproducibility. Default is 123.

Value

A list containing:

`std`	Standardized mean data for each profile.
`est`	Estimated raw mean data for each profile.
`graph`	Combined plots for the profiles.
`plot_data`	Data used for plotting.
`Class_freq`	Frequency table of profile classifications.

Examples

```
## Not run:
# Example usage with iris dataset
result <- plot_lpa(iris[, -5], n_profiles = 3, model_name = "EEE")
print(result$graph)

## End(Not run)
```

---

plot_pie_chart *pie chart(using fei)*

Description

pie chart(using fei)

Usage

```
plot_pie_chart(
  data,
  title = "",
  size.text = 5,
  title.size = 20,
  vjust = 0.5,
  palette = "Set2"
)
```

Arguments

data	data
title	title
size.text	5
title.size	20
vjust	0.5
palette	”Set2”, ”Set3”

Value

graph

Examples

```
## Not run:
#'
# 예시 데이터
data <- data.frame(
  구분 = c("전혀 아니다", "아니다", "중간이다", "그렇다", "매우 그렇다"),
  빈도 = c(52, 90, 304, 257, 114),
  비율 = c(6.36, 11.02, 37.21, 31.46, 13.95)
)
```

```
# 함수 호출
plot_pie_chart(data,,"Q1응답",0.5)

## End(Not run)
```

`plspm_add_blocks` *plspm_add_blocks, indicator blocks*

Description

plspm_add_blocks, indicator blocks

Usage

```
plspm_add_blocks(newData, blocks, interactionTerm)
```

Arguments

`newData`	plspm_make_prod data
`blocks`	clocks
`interactionTerm`	interaction Term list data

Value

list

Examples

```
## Not run:
#' data(satisfaction)
#'
newData1 <- plspm_make_prod(satisfaction, sat_blocks1,
                            list(intIE = interlist("IMAG",1,2, "EXPE",1,2) ) )


interactionTerm1 = list(list(from = "IMAG*EXPE", to = "LOY"))
plspm_add_blocks(newData1, sat_blocks1, interactionTerm1)


## End(Not run)
```

plspm_add_inter_paths *plspm_add_inter_paths*

Description

plspm_add_inter_paths

Usage

```
plspm_add_inter_paths(path_mat, paths)
```

Arguments

path_mat	path matrix data
paths	add path, interactionTerm

Value

mat

Examples

```
## Not run:
#'
data <- data.frame(
  IMAG = c(0, 1, 0, 0, 1, 1),
  EXPE = c(0, 0, 1, 1, 1, 0),
  QUAL = c(0, 0, 0, 1, 1, 0),
  VAL  = c(0, 0, 0, 0, 1, 0),
  SAT  = c(0, 0, 0, 0, 0, 1),
  LOY  = c(0, 0, 0, 0, 0, 0),
  row.names = c("IMAG", "EXPE", "QUAL", "VAL", "SAT", "LOY")
)

# 함수 호출 예시
plspm_add_inter_paths(data, list(
  int1 = list(from = "IMAG*EXPE", to = "LOY"),
  int2 = list(from = "IMAG*QUAL", to = "LOY")
))

plspm_add_inter_paths(data, list(
  list(from = "IMAG*EXPE", to = "LOY"),
  list(from = "IMAG*QUAL", to = "LOY")
))



## End(Not run)
```

`plspm_apa_boot_paths` *bootstrap path effect apa analysis*

Description

bootstrap path effect apa analysis

Usage

```
plspm_apa_boot_paths(data)
```

Arguments

`data`	plspm_boot_paths_sig

Value

text

Examples

```
## Not run:

#'

# 예시 데이터로 함수 호출
nfl_pls_boot2 <- data.frame(
  paths = c("Speical -> Scoring", "Rushing -> Offense", "Passing -> Offense", "Offense -> Scoring"),
  Original = c(0.311, 0.535, 0.789, 0.810),
  Mean.Boot = c(0.098, 0.522, 0.756, 0.762),
  Std.Error = c(0.317, 0.044, 0.252, 0.227),
  t = c("0.310", "11.840***", "2.995**", "3.361***"),
  `95%CI` = I(list(c(-0.484, 0.450), c(0.411, 0.592), c(0.563, 0.948), c(0.587, 0.895)))
)

result <- plspm_apa_boot_paths(nfl_pls_boot2)
cat(result)
#'

## End(Not run)
```

`plspm_apa_grp` *plspm_apa_grp*

Description

plspm_apa_grp

Usage

```
plspm_apa_grp(data)
```

Arguments

data	plspm_grp_summary

Value

text

Examples

```
## Not run:

# Example usage
data14 <- tibble::tribble(
  ~paths, ~global, ~`group.고등학교`, ~`group.중학교`, ~diff, ~t, ~p.value, ~sig,
  "자기효능감->진로동기", 0.88, 0.894, 0.81, 0.085, 0.908, 0.183, "no",
  "자기효능감->진로태도", -0.674, -0.749, 0.438, 1.19, 2.49, 0.007, "yes",
  "자기효능감->진로준비", 0.429, 0.542, 0.595, 0.053, 0.395, 0.347, "no",
  "진로동기->진로준비", 0.446, 0.431, 0.385, 0.046, 0.232, 0.408, "no",
  "진로태도->진로준비", 0.007, 0.098, -0.126, 0.224, 0.626, 0.266, "no"
)

plspm_apa_grp(data14)


## End(Not run)
```

---

plspm_apa_ind *Function to interpret plspm results*

Description

Function to interpret plspm results

Usage

```
plspm_apa_ind(data)
```

Arguments

data	plspm_ind_effect_test data

Value

text

Examples

```
## Not run:
#'
# Example data frame
data11 <- tibble::tibble(
  paths = c("IMAG -> EXPE -> QUAL -> VAL -> SAT -> LOY",
            "IMAG -> EXPE -> QUAL -> SAT -> LOY",
            "IMAG -> EXPE -> VAL -> SAT -> LOY",
            "IMAG -> EXPE -> SAT -> LOY",
            "IMAG -> SAT -> LOY"),
  ind_effect = c(0.0970, 0.0297, 0.0178, -0.000790, 0.0995),
  SE = c(4.44e-4, 4.07e-3, 2.98e-3, 1.91e-2, 3.35e-2),
  Z = c(218.39, 7.31, 5.98, -0.0413, 2.97),
  p.value = c(0, 2.76e-13, 2.26e-9, 9.67e-1, 3.00e-3),
  sig = c("***", "***", "***", "ns", "**")
)

plspm_apa_ind(data11)

## exaample
plspm_ind_effect_test(satpls_boot)%>%plspm_apa_ind()


## End(Not run)
```

---

plspm_apa_paths *plspm_apa_paths*

Description

plspm_apa_paths

Usage

```
plspm_apa_paths(data)
```

Arguments

data	plspm_paths_sig

Value

interpretation

Examples

```
## Not run:
#'
# 예시 데이터 프레임
jutpls_boot1 <- data.frame(
  endogenous = c("진로동기", "진로태도", "진로준비", "진로준비", "진로준비"),
  exogenous = c("자기효능감", "자기효능감", "자기효능감", "진로동기", "진로태도"),
```

```
  Est = c(0.880, -0.674, 0.429, 0.446, 0.007),
  SE = c(0.050, 0.078, 0.129, 0.121, 0.078),
  t = c(17.597, -8.656, 3.330, 3.694, 0.087),
  p.value = c("<.001", "<.001", 0.001, "<.001", 0.931),
  sig = c("***", "***", "**", "***", "ns")
)

# 함수 실행
plspm_apa_paths(jutpls_boot)

jutpls_boot %>% plspm_paths_sig()%>% plspm_apa_paths()
#'
jutpls_boot %>% plspm_apa_paths()
jutpls_boot %>% plspm_paths_sig()%>% plspm_apa_paths()
jutpls_boot %>% plspm_paths_sig(unite=T)%>% plspm_apa_paths()



## End(Not run)
```

`plspm_auto_blocks` *plspm_auto_blocks*

Description

plspm_auto_blocks

Usage

```
plspm_auto_blocks(mat, to_lower = TRUE)
```

Arguments

`mat`	path matrix
`to_lower`	Change to lowercase

Value

data

Examples

```
## Not run:

#'
mat <- matrix(c(0, 1, 0, 0, 0, 1, 0, 0,
                0, 0, 1, 1, 1, 0, 0, 0,
                0, 0, 0, 1, 0, 0, 0, 0,
                0, 0, 0, 1, 0, 0, 0, 0,
                0, 0, 0, 0, 0, 1, 0, 0,
                0, 0, 0, 0, 0, 0, 1, 1,
```

```
                0, 0, 0, 0, 0, 0, 0, 0,
                0, 0, 0, 0, 0, 0, 0, 0),
              nrow = 8, byrow = TRUE)

rownames(mat) <- c("IMAG", "EXPE", "QUAL", "VAL", "SAT", "LOY", "int_IMEX", "int_IMQU")
colnames(mat) <- c("IMAG", "EXPE", "QUAL", "VAL", "SAT", "LOY", "int_IMEX", "int_IMQU")
mat
# 함수 호출
plspm_auto_blocks(mat)

plspm_add_inter_paths(satpls_boot$model$IDM, list(
  list(from = "IMAG*EXPE", to = "LOY"),
  list(from = "IMAG*QUAL", to = "LOY")
))%>%plspm_auto_blocks()


## End(Not run)
```

`plspm_auto_genData` *plspm_auto_genData*

Description

plspm_auto_genData

Usage

```
plspm_auto_genData(data, interactions, to_lower = TRUE)
```

Arguments

`data`	data.frame
`interactions`	interaction term list data
`to_lower`	Change to lowercase

Value

data

Examples

```
## Not run:
#'
# 예시 데이터
data1 <- data.frame(
  IMAG = c(-0.06448267, 0.83858784, 0.47542512, 0.42690176, 1.04176442, 0.27894142),
  EXPE = c(0.3442433, 0.7974660, 0.5531210, 0.1281051, 0.4492096, 0.6914954),
  QUAL = c(-0.5254008, 0.3971273, 0.1642778, -0.6892055, 0.7255953, -0.2088824),
  VAL = c(-0.07103969, 0.44321149, 0.56088128, -0.81127296, 1.21587873, -0.29482532),
  SAT = c(-0.20583024, 0.36908558, 0.83019100, -0.05045141, 0.53174538, -0.20212349),
  LOY = c(0.1664431, 0.4924859, 0.8117161, -0.7969487, 0.7348195, 0.1714641)
)
```

```
# 함수 실행 예시
plspm_auto_genData(data1, list(
  int1 = list(from = "IMAG*EXPE", to = "LOY"),
  int2 = list(from = "IMAG*QUAL", to = "LOY")
))

# 함수 실행 예시
plspm_auto_genData(satpls_boot$scores , list(
  int1 = list(from = "IMAG*EXPE", to = "LOY"),
  int2 = list(from = "IMAG*QUAL", to = "LOY")
)) %>%head()

# 함수 실행 예시
plspm_auto_genData(satpls_boot$scores , list(
  list(from = "IMAG*EXPE", to = "LOY"),
  list(from = "IMAG*QUAL", to = "LOY")
))%>%head()
#'

## End(Not run)
```

`plspm_blocks` *Function to create blocks*————

Description

Function to create blocks————

Usage

```
plspm_blocks(..., dataset = NULL, output = "match")
```

Arguments

`...`	list data
`dataset`	data.frame
`output`	'match', 'match_colnames' is compare data.frame

Value

list blocks

Examples

```
## Not run:

#'
문자열 블록:
  블록의 모든 항목이 문자열이고 output이 "match"일 경우,
   블록을 그대로 유지합니다.
```

숫자 블록:
  블록의 모든 항목이 숫자이고 output이 "match"일 경우,
  블록을 숫자로 변환합니다.

"match_colnames" 출력 옵션:
  output이 "match_colnames"일 경우, 블록이 숫자인
  경우 해당 인덱스를 데이터셋의 컬럼 이름으로
  변환합니다. 변환된 컬럼 이름 중 데이터셋에
  존재하는 이름만 유지합니다.

기타:
#'   블록의 항목이 혼합되어 있거나 output이
"match_colnames"가 아닌 경우, 블록을 숫자로
변환하고 last_value를 기준으로 연속된 숫자로
변환합니다. 빈 블록의 경우 빈 정수 벡터를
반환합니다.

```
bl1 <- c("A", "B", "C")
bl2 <- c(1, 2, 3)
dataset <- data.frame(A = 1:5, B = 1:5, C = 1:5, D = 1:5)
dataset
# 함수 호출
plspm_blocks(block1 = bl1, block2 = bl2,
dataset = dataset, output = "match")
plspm_blocks(block1 = bl1, block2 = bl2,output = "match")
plspm_blocks(block1 = bl1, block2 = bl2,
dataset = dataset, output = "match_colnames")
plspm_blocks(block1 = bl1, block2 = bl2,
output = "match_colnames") #character(0)
#'
#'
#'
# selected indicators from satisfaction
satisfaction2 = satisfaction[, c(20:22, 24:26)]
# add dummy variables to satisfaction2
satisfaction2$dummy1 = dummy1
satisfaction2$dummy2 = dummy2
# add product terms to satisfaction2
satisfaction2$sat1m1 = satisfaction2$sat1 * dummy1
satisfaction2$sat2m1 = satisfaction2$sat2 * dummy1
satisfaction2$sat3m1 = satisfaction2$sat3 * dummy1
satisfaction2$sat1m2 = satisfaction2$sat1 * dummy2
satisfaction2$sat2m2 = satisfaction2$sat2 * dummy2
satisfaction2$sat3m2 = satisfaction2$sat3 * dummy2

satisfaction2 %>%str()
#'
#'

plspm_blocks(
  Satis = item(1:3),
  M1= item(7),
  SatisM1 = item(9:11),
  M2 = item(8),
  SatisM2 = item(12:14),
  Loyalty = item(4:6),
  dataset=satisfaction2,
```

```
  output="match_colnames"
)


plspm_blocks(
  Satis = 1:3,
  M1 = 7,
  SatisM1 = 9:11,
  M2 = 8,
  SatisM2 = 12:14,
  Loyalty = 4:6,
  dataset = satisfaction2,
  output = "match_colnames"
)

plspm_blocks(
  Satis = 1:3,
  M1 = 7,
  SatisM1 = 9:11,
  M2 = 8,
  SatisM2 = 12:14,
  Loyalty = 4:6,
  dataset = satisfaction2,
  output = "match"
)



#참고
plspm_blocks(
  Satis = item(1:3),
  M1= item(7),
  SatisM1 = item(9:11),
  M2 = item(8),
  SatisM2 = item(12:14),
  Loyalty = item(4:6),
  dataset=satisfaction2,
  output="match"
)



edu_blocks_named2 <- plspm_blocks(
  Support = item(1:4),
  Advising = item(5:8),
  Tutoring= item( 9:12),
  Value = item(13:16),
  Satisfaction = item(17:19),
  Loyalty =item(20:23),
  output = "match_colnames"
edu_blocks_named2

# to lavaan syntax
edu_blocks_named2 %>% plspm_blocks2lav()%>%cat("\n")

# # Example usage
edu_blocks_named1 <- plspm_blocks(
```

```
  Support = item("sup.help", "sup.under", "sup.safe", "sup.conc"),
  Advising = item("adv.comp", "adv.acces", "adv.comm", "adv.qual"),
  Tutoring = item("tut.prof", "tut.sched", "tut.stud", "tut.qual"),
  Value = item("val.devel", "val.deci", "val.meet", "val.info"),
  Satisfaction = item("sat.glad", "sat.expe", "sat.over"),
  Loyalty = item("loy.proud", "loy.recom", "loy.asha", "loy.back"),
  dataset = education , # Provide your dataset here
  output = "match_colnames"
)
#'
sat_blocks1 <- plspm_blocks(
  IMAG = item(paste0("imag",1:5)),
  EXPE = item(paste0("expe", 1:5)),
  QUAL = item( paste0("qual", 1:5)),
  VAL = item(paste0("val", 1:4)),
  SAT = item(paste0("sat", 1:4)),
  LOY = item(paste0("loy", 1:4))
)
sat_blocks1
##to lavaan syntax
sat_blocks1 %>% plspm_blocks2lav()%>%cat("\n")



## End(Not run)
```

`plspm_blocks2lav` *blocks to lavaan syntax*

Description

blocks to lavaan syntax

Usage

```
plspm_blocks2lav(listdata, name = FALSE)
```

Arguments

`listdata`	block list data
`name`	defulat FALSE, TRUE generate blocks_model=

Value

lavaan syntax

Examples

```
## Not run:
#'
listdata <- list(
```

```
  IMAG = c("imag1", "imag2", "imag3", "imag4", "imag5"),
  EXPE = c("expe1", "expe2", "expe3", "expe4", "expe5"),
  QUAL = c("qual1", "qual2", "qual3", "qual4", "qual5"),
  VAL = c("val1", "val2", "val3", "val4"),
  SAT = c("sat1", "sat2", "sat3", "sat4"),
  LOY = c("loy1", "loy2", "loy3", "loy4")
)
listdata
# Function calls and result output
plspm_blocks2lav(listdata)
plspm_blocks2lav(listdata)%>%cat("\n")

## End(Not run)
```

`plspm_blocks_match` *Matching plspm blocks names*

Description

Matching plspm blocks names

Usage

```
plspm_blocks_match(data, blocks, paths = NULL, paths_name = NULL)
```

Arguments

`data`	data
`blocks`	plspm blocks
`paths`	plspm paths
`paths_name`	path colnames

Value

block data

Examples

```
## Not run:

data(satisfaction)


# path matrix
IMAG = c(0,0,0,0,0,0)
EXPE = c(1,0,0,0,0,0)
QUAL = c(0,1,0,0,0,0)
VAL = c(0,1,1,0,0,0)
SAT = c(1,1,1,1,0,0)
LOY = c(1,0,0,0,1,0)
sat_path = rbind(IMAG, EXPE, QUAL, VAL, SAT, LOY)
```

```
colnames(sat_path) = rownames(sat_path)
sat_path

# blocks of outer model
sat_blocks = list(1:5, 6:10, 11:15, 16:19, 20:23, 24:27)
sat_blocks


#by hand Very Very tired !!!!
cnames = satisfaction%>%colnames()
latent = colnames(sat_path) #"IMAG", "EXPE", "QUAL", "VAL",  "SAT",  "LOY"
latent

list(IMAGE = cnames[1:5],
     EXPE = cnames[6:10],
     QUAL= cnames[11:15],
     VAL= cnames[16:19],
     SAT= cnames[20:23],
     LOY= cnames[24:27])

#' #Using function : all input
plspm_blocks_match(data=satisfaction, blocks= sat_blocks, paths=sat_path )

# nothing paths --> auto names
plspm_blocks_match(data= satisfaction, blocks=sat_blocks )

# Input paths_name
plspm_blocks_match(data= satisfaction, blocks=sat_blocks,
                   paths_name =c("IMAG", "EXPE", "QUAL", "VAL",  "SAT",  "LOY") )


sat_blocks1 = plspm_blocks_match(satisfaction, paths=sat_path, blocks=sat_blocks )
sat_blocks1

#YOu draw models
paste(plspm_paths2lav(sat_path) ,
      plspm_blocks2lav(sat_blocks1))%>%
  diagram_model(rotation = 1, sizeLat = 6, sizeMan = 4, sizeMan2=3,
                mar=c(2,2,2,2))

## End(Not run)
```

`plspm_boot_effect` *plspm_boot_effect Total effect*

Description

plspm_boot_effect Total effect

Usage

```
plspm_boot_effect(plsres, accept = FALSE, type = "res", digits = 3)
```

Arguments

plsres	plsdata
accept	Hypothesis
type	res, all
digits	3

Value

data result

Examples

```
## Not run:

library(plspm)
data(satisfaction)

# path matrix
IMAG = c(0,0,0,0,0,0)
EXPE = c(1,0,0,0,0,0)
QUAL = c(0,1,0,0,0,0)
VAL = c(0,1,1,0,0,0)
SAT = c(1,1,1,1,0,0)
LOY = c(1,0,0,0,1,0)
sat_path = rbind(IMAG, EXPE, QUAL, VAL, SAT, LOY)

# blocks of outer model
sat_blocks = list(1:5, 6:10, 11:15, 16:19, 20:23, 24:27)
sat_mod=c(rep("A", ncol(sat_path)))
# apply plspm
satpls = plspm(satisfaction, sat_path, sat_blocks, modes = sat_mod,
               scaled = FALSE)



  # bootstrap data
pls_boot_effect(satpls_boot)
pls_boot_effect(satpls_boot, accept=T)


## End(Not run)
```

---

plspm_boot_factor_layout

plspm_boot_factor_layout

Description

plspm_boot_factor_layout

Usage

```
plspm_boot_factor_layout(
  plsres_boot,
  positions = NULL,
  nrow = 4,
  ncol = 5,
  ...
)
```

Arguments

plsres_boot	bootresult
positions	position change
nrow	4,
ncol	5
...	add c()

Value

matrix

Examples

```
## Not run:
#'
# 예시 호출
plsres_boot <- list(model = list(gens = list(
                  lvs_names = c("LV1", "LV2", "LV3", "LV4"))))
result <- plspm_boot_factor_layout(plsres_boot)
print(result)

# 예시 호출
plsres_boot <- list(model =
list(gens = list(
lvs_names = c("LV1", "LV2", "LV3", "LV4", "LV5"))))

plspm_boot_factor_layout(plsres_boot)

# 예시 호출
plsres_boot <- list(model = list(gens =
list(lvs_names = c("LV1", "LV2", "LV3",
"LV4", "LV5", "LV6", "LV7","LV8"))))


plspm_boot_factor_layout(plsres_boot )
###EX)
positions <- list("LV1" = c(2, 3), "LV2" = c(1, 4))


## End(Not run)
```

`plspm_boot_factor_point_to` *plspm_boot_factor_point_to*

Description

plspm_boot_factor_point_to

Usage

```
plspm_boot_factor_point_to(
  plsres_boot,
  point = NULL,
  positions = NULL,
  nrow = 4,
  ncol = 5
)
```

Arguments

`plsres_boot`	plsres_boot
`point`	point point = list(up = c(2,1))
`positions`	positions positions = list(up = c(2,1))
`nrow`	3
`ncol`	3

Value

data

Examples

```
## Not run:
#'
plspm_boot_factor_point_to(jutpls_boot)
plspm_boot_factor_point_to(satpls_boot)
# plspm_factor_point_mat(jutpls_boot, point=list())
# point <- list(up = c(1, 2))
plspm_boot_factor_point_to(jutpls_boot, point = list(up = c(2,1)))
plspm_boot_factor_point_to(jutpls_boot, point = list(up = c(2,1), right =c(1,2)))
#'

## End(Not run)
```

`plspm_boot_ind_factor` *plspm_indicator_factor*

Description

plspm_indicator_factor

Usage

```
plspm_boot_ind_factor(plsres_boot)
```

Arguments

`plsres_boot`	plsres_boot

Value

plsres_boot

Examples

```
## Not run:
plspm_indicator_factor(nfl_pls_boot)

## End(Not run)
```

`plspm_boot_ind_order` *plspm_indicator_order*

Description

plspm_indicator_order

Usage

```
plspm_boot_ind_order(plsres_boot)
```

Arguments

`plsres_boot`	plsres_boot

Value

plsres_boot

Examples

```
## Not run:
#'

## End(Not run)
```

plspm_boot_ind_push *plspm_ind_push*

Description

plspm_ind_push

Usage

```
plspm_boot_ind_push(pls_boot, set = 2)
```

Arguments

pls_boot	pls_boot
set	set=1.5 or c(set= c(2,2,1.5,1.5))

Value

vector

Examples

```
## Not run:

# 예시 사용법
jutpls_boot <- list(model = list(gens = list(lvs_names = c("자기효능감", "진로동기", "진로태도", "진로

# 기본값 사용하는 경우
result_default <- plspm_ind_push(jutpls_boot)
print(result_default)

# 이름과 값이 있는 경우
result_named <- plspm_ind_push(jutpls_boot, set = c(진로동기=2, 자기효능감=3, 진로준비=1.5, 진로태도=

print(result_named)

# 값만 있는 경우
result_unnamed <- plspm_ind_push(jutpls_boot, set = c(2, 2, 1.5, 1.5))
print(result_unnamed)

# 부분적으로 값이 있는 경우
result_partial <- plspm_ind_push(jutpls_boot, set = c(진로동기=2, 자기효능감=3))
print(result_partial)


## End(Not run)
```

`plspm_boot_paths_sig` *boot res sig table*

Description

boot res sig table

Usage

```
plspm_boot_paths_sig(plsres, type = "df", digits = 3)
```

Arguments

`plsres`	plspm result
`type`	df, vec, vector
`digits`	digits=3

Value

data, vector

`plspm_boot_paths_sig_vec` *boot res sig table*

Description

boot res sig table

Usage

```
plspm_boot_paths_sig_vec(plsres, type = "vec")
```

Arguments

`plsres`	plspm result
`type`	df, vec, vector

Value

data, vector

plspm_boot_summary *plspm_boot_summary*

Description

plspm_boot_summary

Usage

```
plspm_boot_summary(pls_boot, type = "res")
```

Arguments

pls_boot	pls boot data
type	res, all, weight, loadings, paths, rsq, total.efs

Value

data

Examples

```
## Not run:

jutpls_boot%>%plspm_boot_summary()
jutpls_boot$boot%>%plspm_boot_summary()


## End(Not run)
```

plspm_cfa *plspm CFA*

Description

plspm CFA

Usage

```
plspm_cfa(plsres_boot, type = "all", axis_x = 1.2)
```

Arguments

plsres_boot	bootstrap plspm data
type	all, cfa1, cfa2, item, CRAVE, fl1981, htmt, f2. total_effect
axis_x	1.2 setting efffecbar text size

Value

multiple data

Examples

```
## Not run:

data(satisfaction)
sat_path = plspm_paths(
  row_names = c("IMAG","EXPE","QUAL","VAL","SAT","LOY"),
  relationship = list(
    path(from="IMAG", to=c("EXPE","SAT","LOY")),
    path("EXPE", c("QUAL","VAL","SAT")),
    path("QUAL", c("VAL","SAT")),
    path("VAL",c("SAT")),
    path("SAT","LOY"))
)
sat_path

# blokcs
sat_blocks1 <- plspm_blocks(
  IMAG = item(paste0("imag",1:5)),
  EXPE = item(paste0("expe", 1:5)),
  QUAL = item( paste0("qual", 1:5)),
  VAL = item(paste0("val", 1:4)),
  SAT = item(paste0("sat", 1:4)),
  LOY = item(paste0("loy", 1:4))
)

sat_blocks1

# apply plspm
# satpls = plspm(satisfaction,
#                path_matrix = sat_path,
#                blocks = sat_blocks1,
#                scaled = FALSE)

satpls_boot = plspm_sem(satisfaction, sat_path, sat_blocks1, scaled = FALSE,
                        boot.val =TRUE, br=100)
##
satpls_boot%>%plspm_cfa()
satpls_boot%>%plspm_cfa("res")

satpls_boot%>%plspm_cfa(type="cfa1")
satpls_boot%>%plspm_cfa(type="cfa2")
satpls_boot%>%plspm_cfa(type="cor")
satpls_boot%>%plspm_cfa(type="model")
satpls_boot%>%plspm_cfa(type="inner_model") %>%plspm_inner_model_sig()
satpls_boot%>%plspm_cfa(type="outer_model")
satpls_boot%>%plspm_cfa(type="effectbar")
total

## End(Not run)
```

plspm_change_nodeLabels *plspm_change_nodeLabels*

Description

plspm_change_nodeLabels

Usage

```
plspm_change_nodeLabels(
  data,
  ...,
  label.cex = 1.2,
  label.scale = TRUE,
  label.prop = 1,
  label.norm = "width",
  plot = TRUE
)
```

Arguments

data	plspm_semPaths
...	change node from name , to name
label.cex	label.cex
label.scale	label.scale
label.prop	label.prop
label.norm	label.norm
plot	TRUE

Value

plot

Examples

```
## Not run:


pathmodel = "
자기효능감 =~ C_S1 +  C_S2 + C_S3 + C_S4 + C_S5
진로태도 =~ B02 + B04 + B10 + B14
진로동기 =~ A_M1 + A_M2 + A_M3
진로준비 =~ D_P1 + D_P2 + D_P3

진로동기 ~ H1*자기효능감
진로태도 ~  H2*자기효능감
진로준비 ~ H5 *진로태도
진로준비 ~  H4 * 진로동기
진로준비 ~  H3*자기효능감
```

```
"

plspm_semPaths2(jutpls_boot,   curve=1, layout="tree2",rotation=1,
                edge.label.cex=0.7, mar=c(2,1,2,1),
                edge.label.position=0.6, label.cex=1,
                sizeLat= 10)%>%
  plspm_change_nodeLabels("B02","결정성",
                          "B04", "독립성",
                          "B10", "계획성",
                          "B14", "성향"
  )









## End(Not run)
```

`plspm_col2lav` *plspm_col2lav, path, or measurement*

Description

plspm_col2lav, path, or measurement

Usage

```
plspm_col2lav(data, lhs, rhs, coef, op = "=~")
```

Arguments

`data`	plspm$boot$loadings
`lhs`	lhs
`rhs`	rhs
`coef`	coeff
`op`	”=~”,”~”

Value

lavaan lavaan syntac

Examples

```
## Not run:
#'
satpls_boot <- data.frame(
  block = c("IMAG", "IMAG", "IMAG",
```

```
  "IMAG", "IMAG", "EXPE", "EXPE",
  "EXPE", "EXPE", "EXPE", "QUAL",
  "QUAL", "QUAL", "QUAL", "QUAL",
  "VAL", "VAL", "VAL", "VAL", "SAT",
  "SAT", "SAT", "SAT", "LOY", "LOY",
  "LOY", "LOY"),
  name = c("imag1", "imag2", "imag3",
   "imag4", "imag5", "expe1", "expe2",
    "expe3", "expe4", "expe5", "qual1",
     "qual2", "qual3", "qual4", "qual5",
     "val1", "val2", "val3", "val4",
     "sat1", "sat2", "sat3", "sat4",
     "loy1", "loy2", "loy3", "loy4"),
  Original = c(0.709, 0.877, 0.842,
   0.569, 0.778, 0.766, 0.837, 0.76,
   0.718, 0.837, 0.781, 0.882, 0.794,
    0.789, 0.807, 0.865, 0.796, 0.75,
    0.844, 0.92, 0.916, 0.826, 0.818,
     0.907, 0.671, 0.905, 0.682)
)
#data
satpls_boot

#method 1
satpls_boot %>%
plspm_col2lav(lhs = "block", rhs = "name", coef = "Original")

#method 2
satpls_boot %>% plspm_col2lav(1,2,3)

#ethod 3
satpls_boot %>%
plspm_col2lav( lhs = 1, rhs = 2, coef = 3)

# draw model
satpls_boot %>%
plspm_col2lav( lhs = 1, rhs = 2, coef = 3)%>%
  diagram_model(whatLabels = "est", exoCov = F,
  sizeLat=6, sizeMan=6,sizeMan2=2,
                edge.label.cex = 0.6)



## End(Not run)
```

`plspm_cor` *Tplspm result structral cor*

Description

Tplspm result structral cor

Usage

```
plspm_cor(plsres)
```

Arguments

`plsres`	plspm result

Value

matrix

`plspm_CRAVE` *plspm_CRAVE*

Description

plspm_CRAVE

Usage

```
plspm_CRAVE(plsdata, digits = 3)
```

Arguments

`plsdata`	plspmdata
`digits`	digists 3

Value

CR and AVE

Examples

```
## Not run:

data(satisfaction)
#model 1
model1 = "
EXPE ~ IMAG
QUAL ~ EXPE
VAL ~  QUAL+ EXPE
SAT ~ IMAG + EXPE + QUAL + VAL
LOY ~ SAT + IMAG
"
# # Generate the matrix
sat_path = plspm_lav2path(model1,
                          fct_order=c("IMAG", "EXPE","QUAL", "VAL", "SAT", "LOY" ))
# sat_path

# Method 2: paths
sat_path = plspm_paths(
  row_names = c("IMAG","EXPE","QUAL","VAL","SAT","LOY"),
  relationship = list(
    path(from="IMAG", to=c("EXPE","SAT","LOY")),
    path("EXPE", c("QUAL","VAL","SAT")),
    path("QUAL", c("VAL","SAT")),
```

```
    path("VAL",c("SAT")),
    path("SAT","LOY")
  )
)
# blokcs
sat_blocks1 <- plspm_blocks(
  IMAG = item(paste0("imag",1:5)),
  EXPE = item(paste0("expe", 1:5)),
  QUAL = item( paste0("qual", 1:5)),
  VAL = item(paste0("val", 1:4)),
  SAT = item(paste0("sat", 1:4)),
  LOY = item(paste0("loy", 1:4))
)
sat_blocks1

# vector of modes (reflective indicators):auto
sat_mod = rep("A", 6)
##############

# apply plspm
satpls = plspm(satisfaction,
                  path_matrix = sat_path,
                  blocks = sat_blocks1,
                  scaled = FALSE)

satpls_boot = plspm_sem(satisfaction, sat_path, sat_blocks1, scaled = FALSE,
                        boot.val =TRUE, br=100)

#CR a AVE
satpls%>% plspm_CRAVE()

satpls_summary%>% plspm_CRAVE()

satpls_boot%>% plspm_CRAVE()

satpls_boot_summary%>% plspm_CRAVE()
#'

## End(Not run)
```

`plspm_crossloadings` *plspm_crossloadings*

Description

plspm_crossloadings

Usage

```
plspm_crossloadings(plsres, digits = 3)
```

Arguments

`plsres`	boot result
`digits`	reound 3

Value

data

`plspm_crossloadings_plot`
plspm_crossloadings_plot

Description

plspm_crossloadings_plot

Usage

```
plspm_crossloadings_plot(plsres, wide = F)
```

Arguments

`plsres`	plspm result
`wide`	plot type TRUE block, FALSE wide plot

Value

pplot

Examples

```
## Not run:

data(satisfaction)
#model 1
model1 = "
EXPE ~ IMAG
QUAL ~ EXPE
VAL ~  QUAL+ EXPE
SAT ~ IMAG + EXPE + QUAL + VAL
LOY ~ SAT + IMAG
"
# # Generate the matrix
sat_path = plspm_lav2path(model1,
                          fct_order=c("IMAG", "EXPE","QUAL", "VAL", "SAT", "LOY" ))
# sat_path

# Method 2: paths
sat_path = plspm_paths(
  row_names = c("IMAG","EXPE","QUAL","VAL","SAT","LOY"),
  relationship = list(
```

```
    path(from="IMAG", to=c("EXPE","SAT","LOY")),
    path("EXPE", c("QUAL","VAL","SAT")),
    path("QUAL", c("VAL","SAT")),
    path("VAL",c("SAT")),
    path("SAT","LOY")
  )
)
# blokcs
sat_blocks1 <- plspm_blocks(
  IMAG = item(paste0("imag",1:5)),
  EXPE = item(paste0("expe", 1:5)),
  QUAL = item( paste0("qual", 1:5)),
  VAL = item(paste0("val", 1:4)),
  SAT = item(paste0("sat", 1:4)),
  LOY = item(paste0("loy", 1:4))
)
sat_blocks1

# vector of modes (reflective indicators):auto
sat_mod = rep("A", 6)
##############

# apply plspm
satpls = plspm(satisfaction,
                   path_matrix = sat_path,
                   blocks = sat_blocks1,
                   scaled = FALSE)

satpls_boot = plspm_sem(satisfaction, sat_path, sat_blocks1, scaled = FALSE,
                        boot.val =TRUE, br=100)


#'
satpls %>% plspm_crossloadings_plot()
satpls %>% plspm_crossloadings_plot(T)
satpls$crossloadings %>% plspm_crossloadings_plot()
satpls$crossloadings %>% plspm_crossloadings_plot(T)
#'
#'

## End(Not run)
```

`plspm_curve` *plspm_curve*

Description

plspm_curve

Usage

```
plspm_curve(plotdata, curve_list = NULL, plot = TRUE)
```

Arguments

`plotdata`	plot data
`curve_list`	curve list
`plot`	TRUE

Value

plot data

Examples

```
## Not run:
#'

pathmodel = "
자기효능감 =~ C_S1 +  C_S2 + C_S3 + C_S4 + C_S5
진로태도 =~ B02 + B04 + B10 + B14
진로동기 =~ A_M1 + A_M2 + A_M3
진로준비 =~ D_P1 + D_P2 + D_P3

진로동기 ~ H1*자기효능감
진로태도 ~  H2*자기효능감
진로준비 ~ H5 *진로태도
진로준비 ~  H4 * 진로동기
진로준비 ~  H3*자기효능감
"

plspm_diagram_model(pathmodel, layout="tree2",
                    rotation = 2,residuals = T, residScale = 6,
                    edge.label.position = 0.6)%>%
  plspm_rotate_resid(c(진로동기= 160, 진로태도= 160))

#
plspm_diagram_model(pathmodel, layout="tree2",
                    rotation = 2,residuals = T, residScale = 6,
                    edge.label.position = 0.6)%>%
  plspm_curve(c("진로준비~~ 진로동기" = -2))%>%
  plspm_rotate_resid(c(진로동기= 160, 진로태도= 30))

## End(Not run)
```

`plspm_diagram_model` *Drawing Structural Equation Models Using the lavaan Syntax (park JH PhD)*

Description

Drawing Structural Equation Models Using the lavaan Syntax (park JH PhD)

Usage

```
plspm_diagram_model(
  model,
  residuals = FALSE,
  whatLabels = "model",
  rotation = 2,
  residScale = 12,
  layout = "tree2",
  edge.label.cex = 1,
  edge.label.position = 0.5,
  edge.color = "black",
  edge.width = 1.5,
  sizeMan = 8,
  sizeMan2 = 3,
  sizeLat = 10,
  sizeLat2 = 4,
  style = "lisrel",
  sig = FALSE,
  exoVar = FALSE,
  exoCov = FALSE,
  curve = 1.5,
  asize = 2,
  mar = c(3, 3, 3, 3),
  nDigits = 3,
  shapeLat = "circle",
  shapeMan = "rectangle",
  shapeInt = "triangle",
  sample.nobs = 100,
  border.width = 2,
  Groups = FALSE,
  group_match = "lat",
  growth = FALSE,
  structural = FALSE,
  edgeLabels = NULL,
  nodeLabels = NULL,
  fixedStyle = 1,
  color = list(lat = "skyblue", man = "yellow"),
  type = "plot"
)
```

Arguments

`model`	lavaan syntax
`residuals`	residauls TRUE
`whatLabels`	”model
`rotation`	rotation=2
`residScale`	residScale=12
`layout`	layout=”tree”
`edge.label.cex`	1

`edge.label.position`	0.5
`edge.color`	edge.color= black
`edge.width`	edge.width=1.5
`sizeMan`	8
`sizeMan2`	4
`sizeLat`	10
`sizeLat2`	6
`style`	lisrel
`sig`	FALSE
`exoVar`	FALSE
`exoCov`	TRUE
`curve`	1.5
`asize`	1.5
`mar`	c(2,8,3,10)
`nDigits`	2
`shapeLat`	”circle, ellipse
`shapeMan`	retangle
`shapeInt`	triangle ,
`sample.nobs`	sample.nobs = 100
`border.width`	border.width=1
`Groups`	FALSE
`group_match`	”FALSE”lat#’
`growth`	growth= FALSE, TRUE model.type =”grouwth”
`structural`	structural = FALSE
`edgeLabels`	edgeLabels=NULL
`nodeLabels`	nodeLabels=NULL
`fixedStyle`	fixedStyle=1 default rbase linetype
`color`	lat= skyblue, man yellow
`type`	type =”plot, and res is lavaan simdata result

Value

plot

Examples

```
## Not run:
#'
model_test <- '
  # latent variable definitions
     ind60 =~ x1 + x2 + x3
     dem60 =~ y1 + a*y2 + b*y3 + c*y4
     dem65 =~ y5 + a*y6 + b*y7 + c*y8
```

```
  # regressions
    dem60 ~ ind60
    dem65 ~ ind60 + dem60

  # residual correlations
    # y1 ~~ y5
    # y2 ~~ y4 + y6
    # y3 ~~ y7
    # y4 ~~ y8
    # y6 ~~ y8
'

HSmodel <- ' visual  =~ x1 + x2 + x3
             textual =~ x4 + x5 + x6
             speed   =~ x7 + x8 + x9 '


m1a <- "
y1 =~ x1 + x2 +x5
y2 =~ x3 + x4 +x6
y1 ~~ y2"

diagram_model(model= HSmodel, groups=F,  residuals=F)

diagram_model(model= model_test, style="ram", groups=F)
diagram_model(model= model_test, residuals=T)
diagram_model(model= m1a, residuals=T)


#plspm model
#model 1
model1 = "
EXPE ~ IMAG
QUAL ~ EXPE
VAL ~  QUAL+ EXPE
SAT ~ IMAG + EXPE + QUAL + VAL
LOY ~ SAT + IMAG
"
sat_path = plspm_lav2path(model1,
                          fct_order=c("IMAG", "EXPE","QUAL", "VAL", "SAT", "LOY" ))
sat_path

blocks_model ="
IMAG =~ imag1 + imag2 + imag3 + imag4 + imag5
EXPE =~ expe1 + expe2 + expe3 + expe4 + expe5
QUAL =~ qual1 + qual2 + qual3 + qual4 + qual5
VAL =~ val1 + val2 + val3 + val4
SAT =~ sat1 + sat2 + sat3 + sat4
LOY =~ loy1 + loy2 + loy3 + loy4
"
blocks_model
plspm_lav2blocks(blocks_model)

#model 그리기
paste(model1, blocks_model)%>%

  plspm_diagram_model(rotation = 1, sizeLat = 6, sizeMan = 4, sizeMan2=3,
```

```
                mar=c(2,2,2,2))

#'
#'
#'

## End(Not run)
```

`plspm_dULS_modelfit` *Bollen-Stine Bootstrap Based d_ULS and d_G Calculation*

Description

This function calculates the d_ULS and d_G metrics for a PLS model using the Bollen-Stine bootstrap method. Confidence intervals are estimated for these indices.

Usage

```
plspm_dULS_modelfit(pls_model, blocks, bootstrap = 1000, ci = 0.95)
```

Arguments

`pls_model`	A PLS model object containing model results.
`blocks`	A list defining the measurement model blocks. Each block should contain variable names.
`bootstrap`	The number of bootstrap samples to use. Default is 1000.
`ci`	The confidence level for the intervals. Default is 0.95.

Value

A data frame with the calculated d_ULS and d_G metrics, along with their confidence intervals and significance indicators. The columns include:

- `Method`: Bootstrap method used (e.g., "Bollen-Stine").
- `d_ULS`: Computed d_ULS value.
- `d_ULS_95CI`: Confidence interval for d_ULS.
- `dUsig`: Significance of d_ULS (whether it falls within the confidence interval).
- `d_G`: Computed d_G value.
- `d_G_95CI`: Confidence interval for d_G.
- `dGsig`: Significance of d_G (whether it falls within the confidence interval).

Examples

```
## Not run:
# Example call with 1000 bootstrap samples
result <- plspm_dULS_modelfit(pls_model = edt_pls, blocks = edblock, bootstrap = 1000, ci = 0.95)
print(result)

## End(Not run)
```

plspm_edges_coefs *plspm* 의구조모형의유의성자료를넣는함수

Description

plspm 의구조모형의유의성자료를넣는함수

Usage

```
plspm_edges_coefs(plsdata, digits = 3, unite = TRUE, type = "df")
```

Arguments

plsdata	plsdata
digits	round 3
unite	data combine unite TRUE -> estimat + sig
type	vec, df

Value

estimates vectors

Examples

```
## Not run:

data(satisfaction)
#model 1
model1 = "
EXPE ~ IMAG
QUAL ~ EXPE
VAL ~  QUAL+ EXPE
SAT ~ IMAG + EXPE + QUAL + VAL
LOY ~ SAT + IMAG
"
# # Generate the matrix
sat_path = plspm_lav2path(model1,
                          fct_order=c("IMAG", "EXPE","QUAL", "VAL", "SAT", "LOY" ))
# sat_path

# Method 2: paths
sat_path = plspm_paths(
  row_names = c("IMAG","EXPE","QUAL","VAL","SAT","LOY"),
  relationship = list(
    path(from="IMAG", to=c("EXPE","SAT","LOY")),
    path("EXPE", c("QUAL","VAL","SAT")),
    path("QUAL", c("VAL","SAT")),
    path("VAL",c("SAT")),
    path("SAT","LOY")
  )
)
# blokcs
```

```
sat_blocks1 <- plspm_blocks(
  IMAG = item(paste0("imag",1:5)),
  EXPE = item(paste0("expe", 1:5)),
  QUAL = item( paste0("qual", 1:5)),
  VAL = item(paste0("val", 1:4)),
  SAT = item(paste0("sat", 1:4)),
  LOY = item(paste0("loy", 1:4))
)
sat_blocks1

# vector of modes (reflective indicators):auto
sat_mod = rep("A", 6)
#############

# apply plspm
satpls = plspm_sem(satisfaction,
                   path_matrix = sat_path,
                   blocks = sat_blocks1,
                   scaled = FALSE)

satpls_boot = plspm_sem(satisfaction, sat_path, sat_blocks1, scaled = FALSE,
                        boot.val =TRUE, br=100)

plspm_edges_values(satpls)
plspm_edges_values(satpls, unite=F) #no star
plspm_edges_values(satpls$inner_model)
plspm_edges_values(satpls$inner_model, unite=FALSE) #No star
#' #'

lay_p = matrix(c('IMA', NA, NA, NA, NA, NA, NA, NA,
                 NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 'EXP', NA,
                 NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 'QUA',
                 NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,
                 NA, NA, NA, NA, NA, NA, NA, NA, NA, 'VAL', NA, NA, NA, NA,
                 NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 'SAT', NA, NA, NA,
                 NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 'LOY',
                 NA, NA, NA, NA, NA, NA, NA, NA),
               nrow = 10, ncol = 10, byrow = FALSE)
lay_p
#  Specify a new value for the edgeLabels parameter when calling qgraph
qgraph(qgraph(t(satpls$path_coefs)),
       layout = lay_p,
       posCol = "black",
       fade = FALSE,
       edge.labels = plspm_edges_values(satpls),
       edge.label.cex = 1,
       edge.label.position = 0.4,
       groups = NULL,
       palette = 'pastel'
)
# Specify a new value for the edgeLabels parameter when calling qgraph
qgraph(qgraph(t(satpls$path_coefs)),
       layout = "spring",
       posCol = "black",
       fade = FALSE,
       edge.labels = plspm_edges_values(satpls),
       edge.label.cex = 1,
```

```
       edge.label.position = 0.4,
       groups = NULL,
       palette = 'pastel'
)

 Specify a new value for the edgeLabels parameter when calling qgraph
qgraph(qgraph(t(satpls$path_coefs)),
       layout = lay_p,
       posCol = "black",
       fade = FALSE,
       edge.labels = matrix(sprintf("%.4f", t(satpls$path_coefs)),
                            nrow=nrow(t(satpls$path_coefs)),
                            ncol=ncol(t(satpls$path_coefs))),
       edge.label.cex = 1,
       edge.label.position=0.4,
       groups = NULL, palette='pastel' )

 plspm_edges_values(satpls)
satpls %>% plspm_path_coefs_plot()
satpls %>% plspm_path_coefs_plot(node =TRUE)



## End(Not run)
```

`plspm_edges_loadings` *plspm lodaings significant vecgtor or df*

Description

plspm lodaings significant vecgtor or df

Usage

```
plspm_edges_loadings(plsres, type = "df")
```

Arguments

`plsres`	plspm result
`type`	df, vec, data.frame, vector

Value

data

Examples

```
## Not run:

# Method 2: paths
sat_path = plspm_paths(
  row_names = c("IMAG","EXPE","QUAL","VAL","SAT","LOY"),
  relationship = list(
```

```
    path(from="IMAG", to=c("EXPE","SAT","LOY")),
    path("EXPE", c("QUAL","VAL","SAT")),
    path("QUAL", c("VAL","SAT")),
    path("VAL",c("SAT")),
    path("SAT","LOY")
  )
)
# blokcs
sat_blocks1 <- plspm_blocks(
  IMAG = item(paste0("imag",1:5)),
  EXPE = item(paste0("expe", 1:5)),
  QUAL = item( paste0("qual", 1:5)),
  VAL = item(paste0("val", 1:4)),
  SAT = item(paste0("sat", 1:4)),
  LOY = item(paste0("loy", 1:4))
)
sat_blocks1

# vector of modes (reflective indicators):auto
sat_mod = rep("A", 6)
##############

# apply plspm
satpls = plspm_sem(satisfaction,
                   path_matrix = sat_path,
                   blocks = sat_blocks1,
                   scaled = FALSE)

satpls_boot = plspm_sem(satisfaction, sat_path, sat_blocks1, scaled = FALSE,
                        boot.val =TRUE, br=100)




# plspm_loadings_values
satpls %>%plspm_edge_loadings()
satpls %>%plspm_edge_loadings("vec")
satpls$boot %>%plspm_edge_loadings()
satpls %>%plspm_edge_loadings()
satpls$boot$loadings%>%plspm_edge_loadings()
satpls$boot$loadings%>%plspm_edge_loadings("vec")


## End(Not run)
```

`plspm_edges_values` *plspm* 의구조모형의유의성자료를넣는함수

Description

plspm 의구조모형의유의성자료를넣는함수

Usage

```
plspm_edges_values(plsdata, digits = 3, unite = TRUE, type = "df")
```

Arguments

plsdata	plsdata
digits	round 3
unite	data combine unite TRUE -> estimat + sig
type	vec, df

Value

estimates vectors

Examples

```
## Not run:

data(satisfaction)
#model 1
model1 = "
EXPE ~ IMAG
QUAL ~ EXPE
VAL ~  QUAL+ EXPE
SAT ~ IMAG + EXPE + QUAL + VAL
LOY ~ SAT + IMAG
"
# # Generate the matrix
sat_path = plspm_lav2path(model1,
                          fct_order=c("IMAG", "EXPE","QUAL", "VAL", "SAT", "LOY" ))
# sat_path

# Method 2: paths
sat_path = plspm_paths(
  row_names = c("IMAG","EXPE","QUAL","VAL","SAT","LOY"),
  relationship = list(
    path(from="IMAG", to=c("EXPE","SAT","LOY")),
    path("EXPE", c("QUAL","VAL","SAT")),
    path("QUAL", c("VAL","SAT")),
    path("VAL",c("SAT")),
    path("SAT","LOY")
  )
)
# blokcs
sat_blocks1 <- plspm_blocks(
  IMAG = item(paste0("imag",1:5)),
  EXPE = item(paste0("expe", 1:5)),
  QUAL = item( paste0("qual", 1:5)),
  VAL = item(paste0("val", 1:4)),
  SAT = item(paste0("sat", 1:4)),
  LOY = item(paste0("loy", 1:4))
)
sat_blocks1

# vector of modes (reflective indicators):auto
sat_mod = rep("A", 6)
##############
```

```
# apply plspm
satpls = plspm_sem(satisfaction,
                   path_matrix = sat_path,
                   blocks = sat_blocks1,
                   scaled = FALSE)

satpls_boot = plspm_sem(satisfaction, sat_path, sat_blocks1, scaled = FALSE,
                        boot.val =TRUE, br=100)

plspm_edges_values(satpls)
plspm_edges_values(satpls, unite=F) #no star
plspm_edges_values(satpls$inner_model)
plspm_edges_values(satpls$inner_model, unite=FALSE) #No star
#' #'

lay_p = matrix(c('IMA', NA, NA, NA, NA, NA, NA, NA,
                 NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 'EXP', NA,
                 NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 'QUA',
                 NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,
                 NA, NA, NA, NA, NA, NA, NA, NA, NA, 'VAL', NA, NA, NA, NA,
                 NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 'SAT', NA, NA, NA,
                 NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 'LOY',
                 NA, NA, NA, NA, NA, NA, NA, NA),
               nrow = 10, ncol = 10, byrow = FALSE)
lay_p
#  Specify a new value for the edgeLabels parameter when calling qgraph
qgraph(qgraph(t(satpls$path_coefs)),
       layout = lay_p,
       posCol = "black",
       fade = FALSE,
       edge.labels = plspm_edges_values(satpls),
       edge.label.cex = 1,
       edge.label.position = 0.4,
       groups = NULL,
       palette = 'pastel'
)
# Specify a new value for the edgeLabels parameter when calling qgraph
qgraph(qgraph(t(satpls$path_coefs)),
       layout = "spring",
       posCol = "black",
       fade = FALSE,
       edge.labels = plspm_edges_values(satpls),
       edge.label.cex = 1,
       edge.label.position = 0.4,
       groups = NULL,
       palette = 'pastel'
)

 Specify a new value for the edgeLabels parameter when calling qgraph
qgraph(qgraph(t(satpls$path_coefs)),
       layout = lay_p,
       posCol = "black",
       fade = FALSE,
       edge.labels = matrix(sprintf("%.4f", t(satpls$path_coefs)),
                            nrow=nrow(t(satpls$path_coefs)),
                            ncol=ncol(t(satpls$path_coefs))),
       edge.label.cex = 1,
```

```
        edge.label.position=0.4,
        groups = NULL, palette='pastel' )

 plspm_edges_values(satpls)
satpls %>% plspm_path_coefs_plot()
satpls %>% plspm_path_coefs_plot(node =TRUE)



## End(Not run)
```

`plspm_edge_position` *plspm_edge_position*

Description

plspm_edge_position

Usage

```
plspm_edge_position(semPaths_plot, ..., plot = TRUE)
```

Arguments

`semPaths_plot`	plot data generated from a PLS-PM model.
`...`	triples of arguments specifying 'from' node, 'to' node, and 'new_position'.
`plot`	logical, if 'TRUE', the plot will be displayed with updated edge positions. Default is 'TRUE'.

Value

Modified plot object with updated edge positions.

Examples

```
## Not run:
# Example 1: Adjusting edge positions
pathmodel <- "
자기효능감 =~ C_S1 +  C_S2 + C_S3 + C_S4 + C_S5
진로태도 =~ B02 + B04 + B10 + B14
진로동기 =~ A_M1 + A_M2 + A_M3
진로준비 =~ D_P1 + D_P2 + D_P3

진로동기 ~ H1*자기효능감
진로태도 ~  H2*자기효능감
진로준비 ~ H5 *진로태도
진로준비 ~  H4 * 진로동기
진로준비 ~  H3*자기효능감
"

semPaths_plot <- plspm_diagram_model(pathmodel, layout="tree2",
                    rotation = 2, residuals = TRUE, residScale = 6,
                    edge.label.position = 0.6)
```

```
semPaths_plot %>%
  plspm_edge_position("진로태도", "진로준비", 0.4,
                      "자기효능감", "진로동기", 0.3)

# Example 2: Adjusting multiple edge positions
semPaths_plot %>%
  plspm_edge_position("진로동기", "진로태도", 0.5,
                      "진로태도", "진로준비", 0.7,
                      "자기효능감", "진로준비", 0.6)

## End(Not run)
```

`plspm_edge_values` *Functions to include significance data for structural models in PLSPM*

Description

Functions to include significance data for structural models in PLSPM

Usage

```
plspm_edge_values(plsdata, type = "vec", digits = 3, unite = TRUE)
```

Arguments

`plsdata`	plsdata
`type`	vec, df
`digits`	round 3
`unite`	data combine unite TRUE -> estimat + sig

Value

estimates vectors

`plspm_effect` *plspm_effect*

Description

plspm_effect

Usage

```
plspm_effect(pls_boot, digits = 3)
```

Arguments

`pls_boot`	pls boot
`digits`	round 3

Value

data

`plspm_effectbar` *plspm_effectbar: Bar graph with direct, indirect, and total effects*

Description

This function creates a bar graph visualizing direct, indirect, and total effects using plspm result data. Users can input plspm results directly or provide custom data.

Usage

```
plspm_effectbar(
  x_pls,
  axis_x = 1.3,
  axis_y = 1.3,
  angle = 90,
  hjust = 1,
  axis_x_pos = 1,
  size_text = 4,
  legend.position = "top",
  input = FALSE,
  user_data = NULL,
  ylim = c(0, 1),
  col = c("black", "gray")
)
```

Arguments

`x_pls`	A plspm result object or a custom data frame with columns: "relationships", "direct", "indirect".
`axis_x`	Size of x-axis text. Default is 1.3.
`axis_y`	Size of y-axis text. Default is 1.3.
`angle`	Angle of x-axis text. Default is 90 degrees.
`hjust`	Horizontal alignment of x-axis text. Default is 1.
`axis_x_pos`	Adjust x-axis position when axis labels are too large. Default is 1.
`size_text`	Size of text displayed on the bars. Default is 4.
`legend.position`	Position of the legend. Default is "top". Options include "top", "bottom", "left", "right", or "none".
`input`	Boolean indicating whether the user provides a custom dataset. Default is FALSE.
`user_data`	A data frame containing relationships, direct, and indirect columns if 'input = TRUE'.
`ylim`	A numeric vector of length 2 specifying the y-axis limits. Default is c(0, 1).
`col`	A character vector specifying bar colors for "direct" and "indirect" effects. Default is c("black", "gray").

Value

Returns the processed data frame with rounded values. It also prints a ggplot2 object.

Examples

```
# Basic usage with plspm results
edt_pls %>% plspm_effectbar()

# Directly specifying data with variable renaming
edt_pls$effects %>% select(1:3) %>%
    replace_df_rep(var_named) %>%
    plspm_effectbar(axis_x = 1.6, legend.position = "bottom")
```

`plspm_effect_bar` *plspm_compare_bar*

Description

plspm_compare_bar

Usage

```
plspm_effect_bar(
  data,
  x = NULL,
  y = NULL,
  fill = NULL,
  size = 3,
  size_x = 12,
  angle = 90,
  position = "dodge",
  xlab = "",
  ylab = "Values",
  text = TRUE,
  digits = 3
)
```

Arguments

`data`	arragne data
`x`	paths
`y`	values
`fill`	group
`size`	bargrahp text size
`size_x`	textsize
`angle`	xangle
`position`	dodge, stack
`text`	hide bartext

Value

graph

Examples

```
## Not run:
#'
# 예시 데이터
REs <- tibble::tribble(
  ~paths, ~grp, ~values,
  "자기효능감->진로동기", "group.남자", 0.902,
  "자기효능감->진로동기", "group.여자", 0.866,
  "자기효능감->진로태도", "group.남자", -0.639,
  "자기효능감->진로태도", "group.여자", -0.672,
  "자기효능감->진로준비", "group.남자", 0.149,
  "자기효능감->진로준비", "group.여자", 0.487,
  "진로동기->진로준비", "group.남자", 0.63,
  "진로동기->진로준비", "group.여자", 0.432,
  "진로태도->진로준비", "group.남자", -0.153,
  "진로태도->진로준비", "group.여자", 0.059
)

# # 함수 호출 예시
REs %>% plspm_effect_bar()
REs %>% plspm_effect_bar(text=FALSE)

## End(Not run)
```

---

plspm_extract_blocks *olsom boot result model*

Description

olsom boot result model

Usage

```
plspm_extract_blocks(plsres)
```

Arguments

plsres	result model

Value

lsit

Examples

```
## Not run:
# using htmt

## End(Not run)
```

`plspm_f2` *Effect Size Black*

Description

Effect Size Black

Usage

```
plspm_f2(plsres, type = "res")
```

Arguments

`plsres`	pls result
`type`	res, f2, R2, ceofs, inner

Value

data

Examples

```
## Not run:
#'
#' #'
sat_path = plspm_paths(
  row_names = c("IMAG","EXPE","QUAL","VAL","SAT","LOY"),
  relationship = list(
    path(from="IMAG", to=c("EXPE","SAT","LOY")),
    path("EXPE", c("QUAL","VAL","SAT")),
    path("QUAL", c("VAL","SAT")),
    path("VAL",c("SAT")),
    path("SAT","LOY"))
)
sat_path

# blokcs
sat_blocks1 <- plspm_blocks(
  IMAG = item(paste0("imag",1:5)),
  EXPE = item(paste0("expe", 1:5)),
  QUAL = item( paste0("qual", 1:5)),
  VAL = item(paste0("val", 1:4)),
  SAT = item(paste0("sat", 1:4)),
  LOY = item(paste0("loy", 1:4))
```

```
)

sat_blocks1

satpls = plspm(satisfaction,
               path_matrix = sat_path,
               blocks = sat_blocks1,
               scaled = FALSE)

satpls_boot = plspm_sem(satisfaction, sat_path, sat_blocks1, scaled = FALSE,
                        boot.val =TRUE, br=100)
# etc
satpls_boot%>%plspm_f2()
satpls_boot%>%plspm_f2("f2")
satpls_boot%>%plspm_f2("coefs")
satpls_boot%>%plspm_f2("r2")
satpls_boot%>%plspm_f2("inner")

## End(Not run)
```

`plspm_f2_plot` *Function to plot f2 values*

Description

Function to plot f2 values

Usage

```
plspm_f2_plot(plsres)
```

Arguments

`plsres`	pls result

Value

plot

Examples

```
## Not run:

#'
# Example data for plsres
plsres <- list(
  path_coefs = data.frame(
    자기효능감 = c(0, 0.884, 0.681, 0.373),
    진로동기 = c(0, 0, 0, 0.353),
    진로태도 = c(0, 0, 0, 0.188),
    진로준비 = c(0, 0, 0, 0),
    row.names = c("자기효능감", "진로동기", "진로태도", "진로준비")
```

```
  ),
  inner_summary = data.frame(
    R2 = c(0.0000000, 0.7817016, 0.4631848, 0.7195506),
    row.names = c("자기효능감", "진로동기", "진로태도", "진로준비")
  )
)

# Run the function
plspm_f2(plsres, type="res")



plspm_f2_plot(plsres)

## End(Not run)
```

`plspm_factor_layout` *Extracting the layout matrix of a plspm model*

Description

Extracting the layout matrix of a plspm model

Usage

```
plspm_factor_layout(plspm_model)
```

Arguments

`plspm_model`	sempaths

Value

mat

Examples

```
## Not run:
#'
data(offense)
offense %>%str()

nfl_path = matrix(
  c(0, 0, 0, 0, 0,
    0, 0, 0, 0, 0,
    0, 0, 0, 0, 0,
    0, 1, 1, 0, 0,
    1, 0, 0, 1, 0),
  nrow = 5, ncol = 5, byrow = TRUE )

rownames(nfl_path) <- c("Speical", "Rushing", "Passing", "Offense", "Scoring")
colnames(nfl_path) <- c("Speical", "Rushing", "Passing", "Offense", "Scoring")
```

```
nfl_path

nfl_blocks = list(7:8, 1:3, 4:6,1:6, 9:11)

plspm_blocks_match(data=offense, blocks=nfl_blocks,
                   paths=nfl_path)

nfl_mode =c("B","A","A","A","A")

nfl_pls = plspm(Data=offense, nfl_path, nfl_blocks, modes = nfl_mode)
nfl_pls%>%summary()

#########
nfl_pls_boot = plspm_sem(Data=offense, nfl_path, nfl_blocks, modes = nfl_mode)

#Layouts extracted from mockups
plspm_semPaths2(nfl_pls_boot,sizeLat = 6)%>%
  plspm_factor_layout()

#Make it input material
plspm_semPaths2(nfl_pls_boot,sizeLat =6)%>%
  plspm_factor_layout() %>%layout_mat(text=T)

#Organizing materials for input
plspm_semPaths2(nfl_pls_boot,sizeLat =6)%>%
  plspm_factor_layout() %>%layout_mat()%>%
  mat_text_arrange()

## End(Not run)
```

`plspm_fl` *Fornell and Locker 1981 method, discrimninat validation*

Description

Fornell and Locker 1981 method, discrimninat validation

Usage

```
plspm_fl(plsdata)
```

Arguments

`plsdata`	plsdata or pls summary

Value

corand AVE

Examples

```
## Not run:

data(satisfaction)
#model 1
model1 = "
EXPE ~ IMAG
QUAL ~ EXPE
VAL ~  QUAL+ EXPE
SAT ~ IMAG + EXPE + QUAL + VAL
LOY ~ SAT + IMAG
"
# # Generate the matrix
sat_path = plspm_lav2path(model1,
                          fct_order=c("IMAG", "EXPE","QUAL", "VAL", "SAT", "LOY" ))
# sat_path

# Method 2: paths
sat_path = plspm_paths(
  row_names = c("IMAG","EXPE","QUAL","VAL","SAT","LOY"),
  relationship = list(
    path(from="IMAG", to=c("EXPE","SAT","LOY")),
    path("EXPE", c("QUAL","VAL","SAT")),
    path("QUAL", c("VAL","SAT")),
    path("VAL",c("SAT")),
    path("SAT","LOY")
  )
)
# blokcs
sat_blocks1 <- plspm_blocks(
  IMAG = item(paste0("imag",1:5)),
  EXPE = item(paste0("expe", 1:5)),
  QUAL = item( paste0("qual", 1:5)),
  VAL = item(paste0("val", 1:4)),
  SAT = item(paste0("sat", 1:4)),
  LOY = item(paste0("loy", 1:4))
)
sat_blocks1

# vector of modes (reflective indicators):auto
sat_mod = rep("A", 6)
##############

# apply plspm
satpls = plspm_sem(satisfaction,
                   path_matrix = sat_path,
                   blocks = sat_blocks1,
                   scaled = FALSE)

satpls_boot = plspm_sem(satisfaction, sat_path, sat_blocks1, scaled = FALSE,
                        boot.val =TRUE, br=100)

satpls_boot %>% plspm_fl()
satpls_boot_summary %>% plspm_fl()
```

```
## End(Not run)
```

`plspm_grp_comp` *plspm_grp_comp:Creating multigroup list data into a data frame*

Description

plspm_grp_comp:Creating multigroup list data into a data frame

Usage

```
plspm_grp_comp(lst_data)
```

Arguments

`lst_data`	grp data

Value

data

Examples

```
## Not run:
#'
plspm_grp_comp(teacher_compare) %>%
  nice_table(exclude=c("df", "sig"))%>%
  replace_df_rep(var_named) %>%Round(2)%>%pall()

## End(Not run)
```

`plspm_grp_compare_bar`
plspm_grp_compare_bar

Description

plspm_grp_compare_bar

Usage

```
plspm_grp_compare_bar(
  data,
  x = NULL,
  y = NULL,
  fill = NULL,
  size = 3,
  size_x = 12,
  angle = 90,
  position = "dodge",
  xlab = "",
  ylab = "Values",
  text = TRUE
)
```

Arguments

data	arragne data
x	paths
y	values
fill	group
size	bargrahp text size
size_x	textsize
angle	xangle
position	dodge, stack
xlab	default ""
ylab	values
text	hide bartext

Value

graph

Examples

```
## Not run:
#'
# 예시 데이터
REs <- tibble::tribble(
  ~paths, ~grp, ~values,
  "자기효능감->진로동기", "group.남자", 0.902,
  "자기효능감->진로동기", "group.여자", 0.866,
  "자기효능감->진로태도", "group.남자", -0.639,
  "자기효능감->진로태도", "group.여자", -0.672,
  "자기효능감->진로준비", "group.남자", 0.149,
  "자기효능감->진로준비", "group.여자", 0.487,
  "진로동기->진로준비", "group.남자", 0.63,
  "진로동기->진로준비", "group.여자", 0.432,
  "진로태도->진로준비", "group.남자", -0.153,
  "진로태도->진로준비", "group.여자", 0.059
```

```
)

# # 함수 호출 예시
REs %>% compare_bar()
REs %>% compare_bar(text=FALSE)

## End(Not run)
```

`plspm_grp_comp_apa` *plspm_grp_comp_apa: Generate APA-style Group Comparison Descriptions*

Description

This function generates APA-style text descriptions for PLS-PM group comparisons.

Usage

```
plspm_grp_comp_apa(df, edteck = TRUE)
```

Arguments

`df`	A data frame containing the group comparison results. It should have the columns: - 'comp': Group comparison identifiers (e.g., "1_vs_2"). - 'Paths': The path being analyzed. - 't': The t-value for the comparison. - 'p_value': The p-value for the comparison. - 'df': Degrees of freedom for the t-test. - 'sig': Significance indicator ("yes" if significant, otherwise "no").
`edteck`	Logical. If 'TRUE', predefined group names are used for educational technology profiles. If 'FALSE', group names are auto-generated as " 그룹 1", " 그룹 2", etc.

Value

None. The function prints APA-style descriptions for significant group comparisons.

Examples

```
## Not run:
df <- data.frame(
  comp = c("1_vs_2", "2_vs_3"),
  Paths = c("Engagement", "Performance"),
  t = c(3.21, 2.45),
  p_value = c(0.0023, 0.0184),
  df = c(45, 50),
  sig = c("yes", "no")
)
plspm_grp_comp_apa(df, edteck = TRUE)

## End(Not run)
```

`plspm_grp_summary` *plspm_grp_summary*

Description

plspm_grp_summary

Usage

```
plspm_grp_summary(plsres, digits = 3)
```

Arguments

`plsres`	plspm boot group
`digits`	round 3

Value

table

Examples

```
## Not run:
#'
jutpath1 = plspm_paths(
  row_names = c("자기효능감","진로동기","진로태도","진로준비"),
  relationship = list(
    path(from="자기효능감", to=c("진로동기","진로태도","진로준비")),
    path("진로동기", c("진로준비")),
    path("진로태도", c("진로준비"))
  )  )
jutpath1

plspm_paths2lav(jutpath1)%>%cat()
#측정모형
jut_blocks <- plspm_blocks(
  자기효능감 = item("C_S1","C_S2","C_S3","C_S4","C_S5"),
  진로동기 = item("A_M1","A_M2","A_M3"),
  진로태도 = item(  "B02", "B04", "B10","B14" ),#결정성
  진로준비 = item("D_P1","D_P2","D_P3")
)

jutpls_boot = plspm_sem(Data= jut6, path_matrix = jutpath1,
                        blocks = jut_blocks,
                        br = 500, summary= TRUE)

#'
jut_gender = plspm.groups(jutpls_boot,
                          group= factor(jut6$성별),
                          method="bootstrap")
jut_gender_p = plspm.groups(jutpls_boot,
                            group= factor(jut6$성별),
                            method="permutation")
```

```
jut_gender%>%plspm_grp_summary()
jut_gender_p%>%plspm_grp_summary()

## End(Not run)
```

`plspm_htmt` *Define a function to calculate HTMT*

Description

Define a function to calculate HTMT

Usage

```
plspm_htmt(
  plsres = NULL,
  data = NULL,
  blocks = NULL,
  htmt2 = TRUE,
  sig = TRUE,
  cut = 0.9,
  digits = 3,
  imp = ""
)
```

Arguments

`data`	data
`blocks`	latent =~ item, plspl blocks
`htmt2`	TRUE Apply geometric mean
`sig`	sig= TRUE- > show star
`cut`	= 0.9, 0.85. 1
`digits`	= 3
`imp`	””

Value

result

Examples

```
## Not run:
#'
sat_path = plspm_paths(
row_names = c("IMAG","EXPE","QUAL","VAL","SAT","LOY"),
relationship = list(
  path(from="IMAG", to=c("EXPE","SAT","LOY")),
  path("EXPE", c("QUAL","VAL","SAT")),
  path("QUAL", c("VAL","SAT")),
```

```
  path("VAL",c("SAT")),
  path("SAT","LOY"))
)

sat_blocks1 <- plspm_blocks(
  IMAG = item(paste0("imag",1:5)),
  EXPE = item(paste0("expe", 1:5)),
  QUAL = item( paste0("qual", 1:5)),
  VAL = item(paste0("val", 1:4)),
  SAT = item(paste0("sat", 1:4)),
  LOY = item(paste0("loy", 1:4))
)

sat_blocks1
# plspm_blocks2lav(sat_blocks1) %>% cat("\n")
data(satisfaction)
plspm_htmt(satisfaction, sat_blocks1)

#'
#'
plspm_htmt(satisfaction, sat_blocks1)
plspm_htmt(satisfaction, sat_blocks1, sig=F)

plspm_htmt(satisfaction, sat_blocks1,sig=F, htmt2=FALSE)
plspm_htmt(satisfaction, sat_blocks1,sig=T, cut=0.9, htmt2=FALSE)

## Reference
semTools::htmt(plspm_blocks2lav(sat_blocks1),
               satisfaction,
               htmt2 = T)

 satpls_boot = plspm_sem(satisfaction, sat_path, sat_blocks1,
  scaled = FALSE, boot.val =TRUE, br=100)
 #If you've bootstrapped
 plspm_htmt(satpls_boot)
#'


## End(Not run)
```

`plspm_htmt2` *Define a function to calculate HTMT*

Description

Define a function to calculate HTMT

Usage

```
plspm_htmt2(
  data,
  blocks,
```

```
  htmt2 = TRUE,
  sig = TRUE,
  cut = 0.9,
  digits = 3,
  imp = ""
)
```

Arguments

data	data
blocks	latent =~ item, plspl blocks
htmt2	TRUE Apply geometric mean
sig	sig= TRUE- > show star
cut	= 0.9, 0.85. 1
digits	= 3
imp	imputation

Value

data

Examples

```
## Not run:
#'
sat_blocks1 <- plspm_blocks(
  IMAG = item(paste0("imag",1:5)),
  EXPE = item(paste0("expe", 1:5)),
  QUAL = item( paste0("qual", 1:5)),
  VAL = item(paste0("val", 1:4)),
  SAT = item(paste0("sat", 1:4)),
  LOY = item(paste0("loy", 1:4))
)

sat_blocks1
# plspm_blocks2lav(sat_blocks1) %>% cat("\n")
data(satisfaction)
plspm_htmt2(satisfaction, sat_blocks1)

#'
#'
plspm_htmt2(satisfaction, sat_blocks1)
plspm_htmt2(satisfaction, sat_blocks1, sig=F)

plspm_htmt2(satisfaction, sat_blocks1,sig=F, htmt2=FALSE)
plspm_htmt2(satisfaction, sat_blocks1,sig=T, cut=0.9, htmt2=FALSE)

## Reference
semTools::htmt(plspm_blocks2lav(sat_blocks1),
               satisfaction,
               htmt2 = T)
#'
```

```
## End(Not run)
```

---

plspm_indicator_factor *plspm_indicator_factor*

Description

plspm_indicator_factor

Usage

```
plspm_indicator_factor(model_syntax)
```

Arguments

model_syntax	lavaan

Value

text

Examples

```
## Not run:
#'
mod <-
  'f1 =~ x01 + x02 + x03 + x06
    f3 =~ x08 + x09 + x10 + x03
    f2 =~ x04 + x05 + x06 + x07
    f4 =~ x11 + x12 + x13 + x14
    f3 ~ f1 + f2
    f4 ~ f3
   '
plspm_indicator_order(mod)

plspm_indicator_factor(mod)

## End(Not run)
```

plspm_indicator_order *plspm_indicator_order*

Description

plspm_indicator_order

Usage

```
plspm_indicator_order(model_syntax)
```

Arguments

model_syntax	lavaan

Value

text

Examples

```
## Not run:
#'
mod <-
  'f1 =~ x01 + x02 + x03 + x06
    f3 =~ x08 + x09 + x10 + x03
    f2 =~ x04 + x05 + x06 + x07
    f4 =~ x11 + x12 + x13 + x14
    f3 ~ f1 + f2
    f4 ~ f3
   '
plspm_indicator_order(mod)

plspm_indicator_factor(mod)

## End(Not run)
```

plspm_indicator_test 지표의균일성평가를위한 *Permutation Test* 함수 *(*병렬처리적용*)*

Description

지표의균일성평가를위한 Permutation Test 함수 (병렬처리적용)

Usage

```
plspm_indicator_test(
  Data,
  path_matrix,
  blocks,
  grp,
  n_perm = 100,
  n_boot = 100,
  n_cores = 12
)
```

Arguments

Data	data
path_matrix	path
blocks	measurement
grp	grp
n_perm	permutation number
n_boot	boot n
n_cores	cores=8 (recently 12~15), exceapt one !!!

Value

data

Examples

```
## Not run:
#'
# 예시: 지표의 균일성 평가 실행 (병렬 처리 적용)
result_df2 <- plspm_indicator_test(Data = jut7c, path_matrix = jutpath1, blocks = jut_blocks, grp = "
print(result_df2)

## End(Not run)
```

---

plspm_indicator_test_multi

Perform Bootstrap and Permutation Tests for PLS-PM Indicators Across Groups

---

Description

This function performs bootstrap and permutation tests for PLS-PM indicators across selected group pairs or all possible group pairs in the dataset. The results include group-specific estimates, standard errors, p-values, and invariance checks.

Usage

```
plspm_indicator_test_multi(
  Data,
  path_matrix,
  blocks,
  grp,
  n_perm = 50,
  n_boot = 50,
  n_cores = 12,
  grp_select = NULL
)
```

Arguments

Data	A data frame containing the dataset to be analyzed.
path_matrix	A square matrix defining the path relationships among constructs in the PLS-PM model.
blocks	A list defining the measurement blocks (i.e., sets of observed variables) for each construct.
grp	A string specifying the name of the grouping variable in 'Data'.
n_perm	Number of permutation tests to perform. Default is 50.
n_boot	Number of bootstrap resamples to perform. Default is 50.
n_cores	Number of cores to use for parallel processing. Default is 12.
grp_select	A character vector of length 2 specifying the group pair to compare. Default is NULL (compares all pairs).

Value

A tibble containing the test results for all indicators, including estimates, standard errors, p-values, and comparisons. The returned tibble includes the following columns: - 'indicator': The indicator being tested. - 'g1_est': The estimate for the first group. - 'g2_est': The estimate for the second group. - 'g1_se': The standard error for the first group. - 'g2_se': The standard error for the second group. - 'se_diff': The difference in standard errors between the groups. - 'p_value': The p-value for the test. - 'invariance': Whether the indicator is invariant between the groups ('"OK"' or '"ns"'). - 'comparison': The comparison between the two groups (e.g., "group1_vs_group2").

Examples

```
## Not run:
result <- plspm_indicator_test_multi(
  Data = ed_tam2_st,
  path_matrix = edmodel,
  blocks = edblock,
  grp = "Class",
  grp_select = c("1", "3"),
  n_perm = 100,
  n_boot = 50,
  n_cores = 13
)
print(result)

## End(Not run)
```

`plspm_ind_effect` *plspm_ind_effect*

Description

plspm_ind_effect

Usage

```
plspm_ind_effect(boot_data, from, through, to, type = "res", digits = 3)
```

Arguments

`boot_data`	boot_data
`from`	from
`through`	through
`to`	to
`type`	res, all
`digits`	3

Value

data

Examples

```
## Not run:
#'
# Example usage of the function with the provided PLS-PM model
plspm_ind_effect(
  boot_data = jutpls_boot,
  from = "자기효능감",
  through = c("진로동기", "진로태도"),
  to = "진로준비"
)

plspm_ind_effect(
  boot_data = jutpls_boot,
  from = "자기효능감",
  through = c("진로동기", "진로태도"),
  to = "진로준비", type="all"
)

plspm_ind_effect(
  boot_data = jutpls_boot,
  from = "자기효능감",
  through = c("진로동기"),
  to = "진로준비", type="res"
)
```

```
## End(Not run)
```

`plspm_ind_effect_boot` *plspm ind efffect bootstrap*

Description

plspm ind efffect bootstrap

Usage

```
plspm_ind_effect_boot(
  plsres = NULL,
  ...,
  data = NULL,
  path_matrix = NULL,
  blocks = NULL,
  modes = NULL,
  B = 100,
  n_cores = NULL
)
```

Arguments

`plsres`	plspm result boot
`...`	paths
`data`	data
`path_matrix`	paths
`blocks`	blocks
`modes`	models
`B`	b= 100
`n_cores`	n_cores =4, CPU core

Value

resutl paths

Examples

```
## Not run:

#'
data(satisfaction)
sat_path = plspm_paths(
  row_names = c("IMAG","EXPE","QUAL","VAL","SAT","LOY"),
  relationship = list(
    path(from="IMAG", to=c("EXPE","SAT","LOY")),
```

```
    path("EXPE", c("QUAL","VAL","SAT")),
    path("QUAL", c("VAL","SAT")),
    path("VAL",c("SAT")),
    path("SAT","LOY"))
)
sat_path

# blokcs
sat_blocks1 <- plspm_blocks(
  IMAG = item(paste0("imag",1:5)),
  EXPE = item(paste0("expe", 1:5)),
  QUAL = item( paste0("qual", 1:5)),
  VAL = item(paste0("val", 1:4)),
  SAT = item(paste0("sat", 1:4)),
  LOY = item(paste0("loy", 1:4))
)

sat_blocks1

# vector of modes (reflective indicators):auto
sat_mod = rep("A", 6)

# satpls = plspm(satisfaction, sat_path, sat_blocks1, scaled = FALSE)
# satpls
# satpls %>%summary()

satpls_boot = plspm_sem(satisfaction, sat_path, sat_blocks1, scaled = FALSE,
                        boot.val =TRUE, br=100)


long_paths <- c(
  "IMAG -> EXPE -> QUAL -> VAL -> SAT -> LOY",
  "IMAG -> EXPE -> QUAL -> SAT -> LOY",
  "IMAG -> EXPE -> VAL -> SAT -> LOY",
  "IMAG -> EXPE -> SAT -> LOY",
  "IMAG -> SAT -> LOY"
)

result <- plspm_get_boot(satpls_boot, long_paths, B = 100)
result
result %>% Round(3)
result %>% unite_ci()

resultall <- plspm_get_boot(satpls_boot, find_paths(satpls_boot), B = 100)
resultall
#'
plspm_ind_effect_boot(
  plsres = satpls_boot,
  "IMAG -> SAT -> LOY",
  "IMAG -> EXPE -> SAT -> LOY",
  "IMAG -> EXPE -> VAL -> SAT -> LOY",
  "IMAG -> EXPE -> QUAL -> SAT -> LOY",
  "IMAG -> EXPE -> QUAL -> VAL -> SAT -> LOY",
  B = 1000,
  n_cores = 4
)%>%system.time()
#'
```

```
#'
plspm_ind_effect_boot(
  plsres = satpls_boot,
  "IMAG -> SAT -> LOY",
  "IMAG -> EXPE -> SAT -> LOY",
  "IMAG -> EXPE -> VAL -> SAT -> LOY",
  "IMAG -> EXPE -> QUAL -> SAT -> LOY",
  "IMAG -> EXPE -> QUAL -> VAL -> SAT -> LOY",
  B = 1000,
  n_cores = 4
)
#'



## End(Not run)
```

`plspm_ind_effect_sig` *plspm_ind_effect_sig*

Description

plspm_ind_effect_sig

Usage

```
plspm_ind_effect_sig(
  boot_data,
  from,
  through,
  to,
  est = "Mean.Boot",
  digits = 3
)
```

Arguments

`boot_data`	bootstrap data
`from`	from
`through`	era
`to`	to
`est`	Mean.Boot, Original
`digits`	3

Value

data

Examples

```
## Not run:
#'
# 함수 호출 예시
plspm_ind_effect_sig(
  boot_data = satpls_boot,
  from = "IMAG",
  through = c("EXPE", "QUAL", "SAT"),
  to = "VAL"
)


## End(Not run)
```

---

plspm_ind_effect_test

Test for all indirect effects

Description

Test for all indirect effects

Usage

```
plspm_ind_effect_test(
  bootdata,
  ...,
  show = TRUE,
  est = "Mean.Boot",
  sobel = "aroian"
)
```

Arguments

`...`	paths
`show`	show paths details
`est`	Original or Mean.Boot
`sobel`	aroian, sobel, goodman
`bootdataboot`	data

Value

data result

Examples

```
## Not run:
#'
#'
ple usage:
  satpls_boot <- list(boot = list(paths = data.frame(
    Original = c(0.578959128, 0.200724200, 0.275149576, 0.848344408,
                 0.105477650, -0.002753995, 0.676655529, 0.122144566, 0.589330650, 0.495479322),
    Mean.Boot = c(0.58232765, 0.21209094, 0.27486564, 0.84793899,
                  0.10992051, -0.01008222, 0.67780527, 0.15132543, 0.56312717, 0.50757755),
    Std.Error = c(0.04358293, 0.06056988, 0.07729000, 0.01864573,
                  0.06968826, 0.06507903, 0.07760527, 0.08440236, 0.08723293, 0.07856694),
    perc.025 = c(0.497644719, 0.107561558, 0.140641502, 0.815794494,
                 -0.025566471, -0.147788188, 0.519959683, -0.008629516, 0.392664443, 0.351774130),
    perc.975 = c(0.6646198, 0.3275312, 0.4161131, 0.8828875,
                 0.2545785, 0.1086856, 0.8166486, 0.2991045, 0.7012494, 0.6556333),
    row.names = c("IMAG -> EXPE", "IMAG -> SAT", "IMAG -> LOY",
                  "EXPE -> QUAL", "EXPE -> VAL", "EXPE -> SAT",
                  "QUAL -> VAL", "QUAL -> SAT", "VAL -> SAT", "SAT -> LOY")
  )))
  # bootdata$boot$paths



  plspm_ind_effect_test(satpls_boot,
                        "IMAG -> EXPE -> VAL -> SAT -> LOY",
                        "IMAG -> SAT -> LOY")

  plspm_ind_effect_test(satpls_boot$boot$paths,
                          "IMAG -> EXPE -> VAL -> SAT -> LOY",
                          "IMAG -> SAT -> LOY")

  plspm_ind_effect_test(satpls_boot,
                        "IMAG -> EXPE -> VAL -> SAT -> LOY",
                        "IMAG -> SAT -> LOY", est = "Mean.Boot")

  plspm_ind_effect_test(satpls_boot$boot$paths,
                          "IMAG -> EXPE -> VAL -> SAT -> LOY",
                          "IMAG -> SAT -> LOY", est = "Mean.Boot")
  plspm_ind_effect_test(satpls_boot$boot$paths,
                          "IMAG -> EXPE -> VAL -> SAT -> LOY",
                          "IMAG -> SAT -> LOY",
                          est = "Mean.Boot",
                          sobel="sobel")

  plspm_ind_effect_test(satpls_boot$boot$paths,
                          "IMAG -> EXPE -> VAL -> SAT -> LOY",
                          "IMAG -> SAT -> LOY",
                          est = "Mean.Boot",
                          sobel="goodman")


  # For individual path strings
  plspm_ind_effect_test(satpls_boot)
```

```
plspm_ind_effect_test(satpls_boot, show=TRUE)
plspm_ind_effect_test(satpls_boot, show=FALSE)

plspm_ind_effect_test(satpls_boot,
                    "IMAG -> EXPE -> VAL -> SAT -> LOY",
                    "IMAG -> SAT -> LOY")

# For a list of paths from find_paths(satpls_boot)$paths
plspm_ind_effect_test(satpls_boot, find_paths(satpls_boot))

plspm_ind_effect_test(satpls_boot, "IMAG -> EXPE -> SAT -> LOY")
plspm_ind_effect_test(satpls_boot,  "IMAG -> EXPE -> VAL -> SAT -> LOY",
 "IMAG -> SAT -> LOY")


find_paths(satpls_boot, type= "ind")
#default
find_paths(satpls_boot, type= "paths")

RRR = find_paths(bootdata, type="paths")
RRR[1]
RRR[2]
RRR[3]
RRR[4]
RRR[5]
RRR[6]

plspm_ind_effect_test(satpls_boot,  RRR[1],RRR[2],RRR[3],RRR[4],RRR[5] )

plspm_ind_effect_test(satpls_boot, find_paths(satpls_boot,"ind")%>%unlist())




## End(Not run)
```

`plspm_ind_factor` *plspm_indicator_factor*

Description

plspm_indicator_factor

Usage

```
plspm_ind_factor(model_syntax)
```

Arguments

`model_syntax`	lavaan

Value

text

Examples

```
## Not run:
#'
mod <-
  'f1 =~ x01 + x02 + x03 + x06
    f3 =~ x08 + x09 + x10 + x03
    f2 =~ x04 + x05 + x06 + x07
    f4 =~ x11 + x12 + x13 + x14
    f3 ~ f1 + f2
    f4 ~ f3
   '
plspm_indicator_order(mod)

plspm_indicator_factor(mod)

## End(Not run)
```

---

plspm_ind_order *plspm_indicator_order*

Description

plspm_indicator_order

Usage

```
plspm_ind_order(model_syntax)
```

Arguments

model_syntax	lavaan

Value

text

Examples

```
## Not run:
#'
mod <-
  'f1 =~ x01 + x02 + x03 + x06
    f3 =~ x08 + x09 + x10 + x03
    f2 =~ x04 + x05 + x06 + x07
    f4 =~ x11 + x12 + x13 + x14
    f3 ~ f1 + f2
    f4 ~ f3
   '
plspm_indicator_order(mod)

plspm_indicator_factor(mod)
```

```
## End(Not run)
```

`plspm_ind_order2` *plspm_ind_order2*

Description

plspm_ind_order2

Usage

```
plspm_ind_order2(model_syntax)
```

Arguments

`model_syntax`	lavaan

Value

input text

Examples

```
## Not run:
#'
mod <-
  'f1 =~ x01 + x02 + x03 + x06
    f3 =~ x08 + x09 + x10 + x03
    f2 =~ x04 + x05 + x06 + x07
    f4 =~ x11 + x12 + x13 + x14
    f3 ~ f1 + f2
    f4 ~ f3
   '
plspm_ind_order2(mod)


## End(Not run)
```

`plspm_ind_push` *plspm_ind_push*

Description

plspm_ind_push

Usage

```
plspm_ind_push(pls_boot, set = 2)
```

Arguments

pls_boot	pls_boot
set	set=1.5 or c(set= c(2,2,1.5,1.5))

Value

vector

Examples

```
## Not run:

# 예시 사용법
jutpls_boot <- list(model = list(gens = list(lvs_names = c("자기효능감", "진로동기", "진로태도", "진로

# 기본값 사용하는 경우
result_default <- plspm_ind_push(jutpls_boot)
print(result_default)

# 이름과 값이 있는 경우
result_named <- plspm_ind_push(jutpls_boot, set = c(진로동기=2, 자기효능감=3, 진로준비=1.5, 진로태도=

# 값만 있는 경우
result_unnamed <- plspm_ind_push(jutpls_boot, set = c(2, 2, 1.5, 1.5))
print(result_unnamed)

# 부분적으로 값이 있는 경우
result_partial <- plspm_ind_push(jutpls_boot, set = c(진로동기=2, 자기효능감=3))
print(result_partial)


## End(Not run)
```

---

plspm_ind_spread *plspm_ind_push*

Description

plspm_ind_push

Usage

```
plspm_ind_spread(pls_boot, set = 2)
```

Arguments

pls_boot	pls_boot
set	set=1.5 or c(set= c(2,2,1.5,1.5))

Value

vector

Examples

```
## Not run:
#'
# 예시 사용법
jutpls_boot <- list(model = list(gens = list(lvs_names = c("자기효능감", "진로동기", "진로태도", "진로

# 기본값 사용하는 경우
result_default <- plspm_ind_spread(jutpls_boot)
print(result_default)

# 이름과 값이 있는 경우
result_named <- plspm_ind_spread(jutpls_boot, set = c(진로동기=2, 자기효능감=3, 진로준비=1.5, 진로태도
print(result_named)

# 값만 있는 경우
result_unnamed <- plspm_ind_spread(jutpls_boot, set = c(2, 2, 1.5, 1.5))
print(result_unnamed)

# 부분적으로 값이 있는 경우
result_partial <- plspm_ind_spread(jutpls_boot, set = c(진로동기=2, 자기효능감=3))
print(result_partial)


## End(Not run)
```

---

plspm_innerplot *inner_plot(Pre-draw a mockup)*

Description

inner_plot(Pre-draw a mockup)

Usage

```
plspm_innerplot(
  path_mat,
  edge.label.cex = 2,
  edge.label.position = 0.5,
  edge.labels = TRUE,
  edge.width = 3,
  label.cex = 1.1,
  rotation = 0,
  angle = 90,
  sizeLat = 14,
  groups = NULL,
```

```
  layout = "circle",
  negCol = "red",
  fade = FALSE,
  asize = 5,
  borders = TRUE,
  border.width = 4,
  border.color = "gray10",
  labels = NULL,
  label.color = "darkblue",
  posCol = "gray10",
  palette = "pastel",
  curve = 0.2,
  width = 2,
  mar = c(3, 3, 3, 3),
  edge_size = 0.7,
  hypo = TRUE,
  digits = 3
)
```

Arguments

path_mat	mati
edge.label.cex	1.2
edge.label.position	0.5
edge.labels	FALSE
edge.width	2
label.cex	1.2
rotation	1
angle	rotation angle
groups	NULL
layout	spring or layout
negCol	red
fade	false
asize	4
borders	TRUE
border.width	2
border.color	gray40
labels	NULL
label.color	gray30
posCol	black
palette	pastel
curve	0.2
width	1
mar	c(3,3,3,3)

edge_size	edge.width size control
hypo	paste H
digits	3#'
vsize	8

Value

graph

Examples

```
## Not run:

# path matrix
IMAG = c(0,0,0,0,0,0)
EXPE = c(1,0,0,0,0,0)
QUAL = c(0,1,0,0,0,0)
VAL = c(0,1,1,0,0,0)
SAT = c(1,1,1,1,0,0)
LOY = c(1,0,0,0,1,0)
sat_path = rbind(IMAG, EXPE, QUAL, VAL, SAT, LOY)

sat_path%>%plspm_innerplot()
#'
path1 = plspm_path(
  row_names = c("IMAG","EXPE","QUAL","VAL","SAT","LOY"),

  relationship = list(
    path(from="IMAG", to=c("EXPE","SAT","LOY")),
    path("EXPE", c("QUAL","VAL","SAT")),
    path("QUAL", c("VAL","SAT")),
    path("VAL",c("SAT")),
    path("SAT","LOY")
  )
)
path1
path1%>%plspm_innerplot()

#'
model1 = "
EXPE ~ IMAG
QUAL ~ EXPE
VAL ~  QUAL+ EXPE
SAT ~ IMAG + EXPE + QUAL + VAL
LOY ~ SAT + IMAG
"
# Generate the matrix
lavpath = plspm_lav2path(model1)


lavpath%>%plspm_innerplot()

#color
sat_path %>%plspm_innerplot(groups=1:6)
sat_path %>%plspm_innerplot(groups=1:6, hypo= FALSE ) #putput 1
```

```
sat_path %>%
 plspm_innerplot(groups=1:6, rotation = 3 )


lay_p = matrix(c('IMA', NA, NA, NA, NA, NA, NA, NA,
                 NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 'EXP', NA,
                 NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 'QUA',
                 NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,
                 NA, NA, NA, NA, NA, NA, NA, NA, NA, 'VAL', NA, NA, NA, NA,
                 NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 'SAT', NA, NA, NA,
                 NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 'LOY',
                 NA, NA, NA, NA, NA, NA, NA, NA),
               nrow = 10, ncol = 10, byrow = FALSE)
lay_p
#'
#'

## End(Not run)
```

`plspm_inner_model_sig` *Showing significance using PLSPM regression*

Description

Showing significance using PLSPM regression

Usage

```
plspm_inner_model_sig(
  data,
  unite = FALSE,
  rhs = "endogenous",
  lhs = "exogenous",
  digits = 3
)
```

Arguments

`data`	plspm data
`unite`	TRUE
`rhs`	from
`lhs`	to
`digits`	round 3

Value

data result

Examples

```
## Not run:
library(plspm)
data(satisfaction)

# path matrix
IMAG = c(0,0,0,0,0,0)
EXPE = c(1,0,0,0,0,0)
QUAL = c(0,1,0,0,0,0)
VAL = c(0,1,1,0,0,0)
SAT = c(1,1,1,1,0,0)
LOY = c(1,0,0,0,1,0)
sat_path = rbind(IMAG, EXPE, QUAL, VAL, SAT, LOY)

# blocks of outer model
sat_blocks = list(1:5, 6:10, 11:15, 16:19, 20:23, 24:27)
sat_mod=c(rep("A", ncol(sat_path)))
# apply plspm
satpls = plspm(satisfaction, sat_path, sat_blocks, modes = sat_mod,
               scaled = FALSE)

#default
satpls%>%plspm_paths_sig()
satpls$inner_model %>%plspm_paths_sig()

satpls %>%plspm_paths_sig(unite=FALSE, lhs="row", rhs="col")
satpls$inner_model  %>%innermodel_sig(unite=FALSE, lhs="row", rhs="col")

satpls %>%plspm_paths_sig(unite=FALSE, lhs="row", rhs="col")
satpls$inner_model %>%plspm_paths_sig(unite=FALSE, lhs="row", rhs="col")

#'

## End(Not run)
```

---

plspm_interaction *PLS-PMinteraction effect Partial Least Squares Path Modeling*

Description

PLS-PMinteraction effect Partial Least Squares Path Modeling

Usage

```
plspm_interaction(
  Data,
  path_matrix,
  blocks,
  interactionTerm = NULL,
  indicator_prod = NULL,
  inter_modes = NULL,
  method = "two_stage",
```

```
  modes = rep("A", ncol(path_matrix)),
  scaling = NULL,
  scheme = "centroid",
  scaled = TRUE,
  tol = 1e-06,
  maxiter = 100,
  plscomp = NULL,
  boot.val = TRUE,
  br = 500,
  seed = NULL,
  dataset = TRUE
)
```

Arguments

`Data`	matrix or data frame containing the manifest variables.
`path_matrix`	A square (lower triangular) boolean matrix representing the inner model (i.e. the path relationships between latent variables).
`blocks`	list of vectors with column indices or column names from Data indicating the sets of manifest variables forming each block (i.e. which manifest variables correspond to each block).#'
`interactionTerm`	interaction terms
`indicator_prod`	indicator producst
`inter_modes`	new modes auto or direct input
`method`	Two_stage, indicator
`modes`	optional argument for runing the non-metric approach; it is a list of string vectors indicating the type of measurement scale for each manifest variable specified in blocks. scaling must be specified when working with non-metric variables. Possible values: "num" (linear transformation, suitable for numerical variables), "raw" (no transformation), "nom" (non-monotonic transformation, suitable for nominal variables), and "ord" (monotonic transformation, suitable for ordinal variables).
`scaling`	character vector indicating the type of measurement f or each block. Possible values are: "A", "B", "newA", "PLScore", "PLScow". The length of modes must be equal to the length of blocks.
`scheme`	string indicating the type of inner weighting scheme. Possible values are "centroid", "factorial", or "path".
`scaled`	standardized. Only used when scaling = NULL. When (TRUE, data is scaled to standardized values (mean=0 and variance=1). The variance is calculated dividing by N instead of N-1).
`tol`	decimal value indicating the tolerance criterion for the iterations (tol=0.000001). Can be specified between 0 and 0.001.
`maxiter`	integer indicating the maximum number of iterations (maxiter=100 by default). The minimum value of maxiter is 100.
`plscomp`	optional vector indicating the number of PLS components (for each block) to be used when handling non-metric data (only used if scaling is provided)

boot.val	whether bootstrap validation should be performed. (FALSE by default).
br	number bootstrap resamples. Used only when boot.val=TRUE. When boot.val=TRUE, the default number of re-samples is 100.
dataset	whether the data matrix used in the computations should be retrieved (TRUE by default).

Value

resutl

Examples

```
## Not run:
#'

data(satisfaction)
sat_path3 = plspm_paths(
  row_names = c("IMAG","EXPE","QUAL","VAL","SAT","LOY"),
  relationship = list(
    path(from="IMAG", to=c("EXPE","SAT","LOY")),
    path("EXPE", c("QUAL","VAL","SAT")),
    path("QUAL", c("VAL","SAT")),
    path("VAL",c("SAT")),
    path("SAT","LOY"))
)

sat_path2

# blokcs
sat_blocks1 <- plspm_blocks(
  IMAG = item(paste0("imag",1:5)),
  EXPE = item(paste0("expe", 1:5)),
  QUAL = item( paste0("qual", 1:5)),
  VAL = item(paste0("val", 1:4)),
  SAT = item(paste0("sat", 1:4)),
  LOY = item(paste0("loy", 1:4))
)

sat_blocks1

# vector of modes (reflective indicators):auto
sat_mod = rep("A", 6)

resinter = plspm_interaction(Data=satisfaction,
                     sat_path2, blocks = sat_blocks1, modes= sat_mod,
                     interactionTerm = list(list(from = "IMAG*EXPE", to = "LOY"),
                     list(from = "IMAG*QUAL", to = "LOY")))
resinter%>%summary()
x11()
plot(resinter)

# resinter %>%plspm_boot_effect()%>%cut_print(rm_row=TRUE)
resinter %>%plspm_effect()%>%cut_print(rm_row=TRUE)
```

```
plspm_paths_sig(resinter, unite=TRUE)
plspm_paths_sig_plot(resinter)
plspm_path_coefs_plot(resinter)

plspm_semPaths2(resinter, structural = T)


#'
## Two stage approach
pls_ts = plspm_interaction(satisfaction, sat_path2, sat_blocks1,
               interactionTerm = list(list(from = "IMAG*EXPE", to = "LOY")) ,
               indicator_prod = list(intIE = interlist("IMAG",1,2, "EXPE",1,2)),
               method="two_stage")
pls_ts %>%summary()


plspm_path_coefs_plot(pls_ts,"circle")
plspm_semPaths2(pls_ts)

# plspm_boot_factor_layout(pls_ts)

## indicator arpproach
pls_int = plspm_interaction(satisfaction, sat_path2, sat_blocks1,
                  interactionTerm = list(list(from = "IMAG*EXPE", to = "LOY")) ,
                indicator_prod = list(intIE = interlist("IMAG", "EXPE")),
                  method="indicator")
pls_int %>%summary()


plspm_path_coefs_plot(pls_int, "circle")
plspm_path_coefs_plot(pls_int)
plspm_semPaths2(pls_int)
#'
#'



## End(Not run)
```

`plspm_lav2blocks` *lavaan syntax to plspm blocks*

Description

lavaan syntax to plspm blocks

Usage

```
plspm_lav2blocks(blocks_model)
```

Arguments

`blocks_model`	lavaan syntax

Value

list

Examples

```
## Not run:

# Input data
blocks_model <- "
IMAG =~ imag1 + imag2 + imag3 + imag4 + imag5
EXPE =~ expe1 + expe2 + expe3 + expe4 + expe5
QUAL =~ qual1 + qual2 + qual3 + qual4 + qual5
VAL =~ val1 + val2 + val3 + val4
SAT =~ sat1 + sat2 + sat3 + sat4
LOY =~ loy1 + loy2 + loy3 + loy4
"

# Example function calls
plspm_lav2blocks(blocks_model)


## End(Not run)
```

---

plspm_lav2paths *lavaan syntax to plspm path matrix*

Description

lavaan syntax to plspm path matrix

Usage

```
plspm_lav2paths(model_syntax, fct_order = NULL)
```

Arguments

model_syntax	lavaan syntax
fct_order	fct_order is reorder option for rownamnes of matrix

Value

matrix

Examples

```
## Not run:
#'
# Example usage
model1 <- "
EXPE ~ IMAG; QUAL ~ EXPE
VAL ~ QUAL + EXPE
SAT ~ IMAG + EXPE + QUAL + VAL
LOY ~ SAT + IMAG
"
```

```
 #or
model1 = "
EXPE ~ IMAG
QUAL ~ EXPE
VAL ~  QUAL+ EXPE
SAT ~ IMAG + EXPE + QUAL + VAL
LOY ~ SAT + IMAG
"
# Generate the matrix
plspm_lav2paths(model1)

plspm_lav2paths(model1, fct_order=c("IMAG", "EXPE","QUAL", "VAL", "SAT", "LOY" ))



## End(Not run)
```

`plspm_layout_mat` *layout_mat*

Description

layout_mat

Usage

```
plspm_layout_mat(semplot, text = TRUE, byrow = TRUE, variable = TRUE)
```

Arguments

`semplot`	semPaths2 data
`text`	text True output text matrix
`byrow`	byrow TRUE
`variable`	variable is rownames adapt

Value

data

Examples

```
## Not run:
#'
data(offense)
offense %>%str()

nfl_path = matrix(
  c(0, 0, 0, 0, 0,
    0, 0, 0, 0, 0,
```

```
    0, 0, 0, 0, 0,
    0, 1, 1, 0, 0,
    1, 0, 0, 1, 0),
  nrow = 5, ncol = 5, byrow = TRUE )

rownames(nfl_path) <- c("Speical", "Rushing", "Passing", "Offense", "Scoring")
colnames(nfl_path) <- c("Speical", "Rushing", "Passing", "Offense", "Scoring")
nfl_path

nfl_blocks = list(7:8, 1:3, 4:6,1:6, 9:11)

plspm_blocks_match(data=offense, blocks=nfl_blocks,
                   paths=nfl_path)

nfl_mode =c("B","A","A","A","A")

nfl_pls = plspm(Data=offense, nfl_path, nfl_blocks, modes = nfl_mode)
nfl_pls%>%summary()

#########
nfl_pls_boot = plspm_sem(Data=offense, nfl_path, nfl_blocks, modes = nfl_mode)

#Layouts extracted from mockups
plspm_semPaths2(nfl_pls_boot,sizeLat = 6)%>%
  plspm_factor_layout()

#Make it input material
plspm_semPaths2(nfl_pls_boot,sizeLat =6)%>%
  plspm_factor_layout() %>%plspm_layout_mat(text=T)

#Organizing materials for input
plspm_semPaths2(nfl_pls_boot,sizeLat =6)%>%
  plspm_factor_layout() %>%plspm_layout_mat()%>%
  mat_text_arrange()

## End(Not run)
```

`plspm_loadings` *plspm loadings output*

Description

plspm loadings output

Usage

```
plspm_loadings(plsres, digits = 3, unite = TRUE)
```

Arguments

`plsres`	plspm result
`digits`	round =3
`unite`	unite t+star

Value

data table

Examples

```
## Not run:

plsres %>% plspm_loadings()
plsres$boot %>% plspm_loadings()
plsres$boot$paths %>% plspm_loadings()



## End(Not run)
```

---

plspm_loadings_plot *plspm_loadings_plot*

Description

plspm_loadings_plot

Usage

```
plspm_loadings_plot(
  plsres,
  size.x = 14,
  size.y = 12,
  size.legend = 12,
  size.text = 5,
  show = FALSE
)
```

Arguments

plsres	plspm data
size.x	axis text 14
size.y	12
size.legend	12
size.text	show data

Value

plot

Examples

```
## Not run:

data(satisfaction)
#model 1
model1 = "
EXPE ~ IMAG
QUAL ~ EXPE
VAL ~  QUAL+ EXPE
SAT ~ IMAG + EXPE + QUAL + VAL
LOY ~ SAT + IMAG
"
# # Generate the matrix
sat_path = plspm_lav2path(model1,
                          fct_order=c("IMAG", "EXPE","QUAL", "VAL", "SAT", "LOY" ))
# sat_path

# Method 2: paths
sat_path = plspm_paths(
  row_names = c("IMAG","EXPE","QUAL","VAL","SAT","LOY"),
  relationship = list(
    path(from="IMAG", to=c("EXPE","SAT","LOY")),
    path("EXPE", c("QUAL","VAL","SAT")),
    path("QUAL", c("VAL","SAT")),
    path("VAL",c("SAT")),
    path("SAT","LOY")
  )
)
# blokcs
sat_blocks1 <- plspm_blocks(
  IMAG = item(paste0("imag",1:5)),
  EXPE = item(paste0("expe", 1:5)),
  QUAL = item( paste0("qual", 1:5)),
  VAL = item(paste0("val", 1:4)),
  SAT = item(paste0("sat", 1:4)),
  LOY = item(paste0("loy", 1:4))
)
sat_blocks1

# vector of modes (reflective indicators):auto
sat_mod = rep("A", 6)
##############

# apply plspm
satpls = plspm(satisfaction,
                   path_matrix = sat_path,
                   blocks = sat_blocks1,
                   scaled = FALSE)

satpls_boot = plspm_sem(satisfaction, sat_path, sat_blocks1, scaled = FALSE,
                        boot.val =TRUE, br=100)


#plot
satpls %>%plspm_loadings_plot()
```

```
satpls$outer_model%>%plspm_loadings_plot()


## End(Not run)
```

`plspm_loadings_values` *plspm lodaings significant vecgtor or df*

Description

plspm lodaings significant vecgtor or df

Usage

```
plspm_loadings_values(plsres, type = "df")
```

Arguments

`plsres`	plspm result
`type`	df, vec, data.frame, vector

Value

data

Examples

```
## Not run:

# Method 2: paths
sat_path = plspm_paths(
  row_names = c("IMAG","EXPE","QUAL","VAL","SAT","LOY"),
  relationship = list(
    path(from="IMAG", to=c("EXPE","SAT","LOY")),
    path("EXPE", c("QUAL","VAL","SAT")),
    path("QUAL", c("VAL","SAT")),
    path("VAL",c("SAT")),
    path("SAT","LOY")
  )
)
# blokcs
sat_blocks1 <- plspm_blocks(
  IMAG = item(paste0("imag",1:5)),
  EXPE = item(paste0("expe", 1:5)),
  QUAL = item( paste0("qual", 1:5)),
  VAL = item(paste0("val", 1:4)),
  SAT = item(paste0("sat", 1:4)),
  LOY = item(paste0("loy", 1:4))
)
sat_blocks1
```

```
# vector of modes (reflective indicators):auto
sat_mod = rep("A", 6)
##############

# apply plspm
satpls = plspm_sem(satisfaction,
                   path_matrix = sat_path,
                   blocks = sat_blocks1,
                   scaled = FALSE)

satpls_boot = plspm_sem(satisfaction, sat_path, sat_blocks1, scaled = FALSE,
                        boot.val =TRUE, br=100)




# plspm_loadings_values
satpls %>%plspm_loadings_values()
satpls %>%plspm_loadings_values("vec")
satpls$boot %>%plspm_loadings_values()
satpls %>%plspm_loadings_values()
satpls$boot$loadings%>%plspm_loadings_values()
satpls$boot$loadings%>%plspm_loadings_values("vec")


## End(Not run)
```

`plspm_loading_lav` *plspm_loading to lavaan syntax*

Description

plspm_loading to lavaan syntax

Usage

```
plspm_loading_lav(input_data, col = "loading")
```

Arguments

`input_data`	plspm result
`col`	selected col

Value

lavaan syntax

Examples

```
## Not run:
#'
data(satisfaction)
sat_path = plspm_paths(
  row_names = c("IMAG","EXPE","QUAL","VAL","SAT","LOY"),
```

```
  relationship = list(
    path(from="IMAG", to=c("EXPE","SAT","LOY")),
    path("EXPE", c("QUAL","VAL","SAT")),
    path("QUAL", c("VAL","SAT")),
    path("VAL",c("SAT")),
    path("SAT","LOY"))
)
sat_path

# blokcs
sat_blocks1 <- plspm_blocks(
  IMAG = item(paste0("imag",1:5)),
  EXPE = item(paste0("expe", 1:5)),
  QUAL = item( paste0("qual", 1:5)),
  VAL = item(paste0("val", 1:4)),
  SAT = item(paste0("sat", 1:4)),
  LOY = item(paste0("loy", 1:4))
)
sat_blocks1

# vector of modes (reflective indicators):auto
sat_mod = rep("A", 6)
##############

# apply plspm
satpls = plspm(satisfaction,
               path_matrix = sat_path,
               blocks = sat_blocks1,
               scaled = FALSE)
#lavaan 1
satpls%>%plspm_loading_lav() %>%cat("\n")

#method 2, lavan
satpls$outer_model%>%plspm_loading_lav() %>%cat("\n")

#diagram
x11()
satpls%>%plspm_loading_lav() %>%
  diagram_model(whatLabels = "est",exoCov=F,
                edge.label.cex = 0.6, rotation=4,
                sizeLat=6,
                sizeMan = 6, sizeMan2=1.5, mar=c(1,10,1,10))

## End(Not run)
```

---

plspm_make_layout_mat *plspm_make_layout_mat*

Description

plspm_make_layout_mat

Usage

```
plspm_make_layout_mat(...)
```

Arguments

`...`	variable

Value

mat

Examples

```
## Not run:
#'
plspm_make_layout_mat(x1 = c(1, 1),
                      x2 = c(2, 1),
                      x3 = c(2, 2),
                      x4 = c(3, 2),
                      x5 = c(3, 5))

## End(Not run)
```

`plspm_make_prod` *plspm_make_prod , Generate data for indicator interaction multiplied terms*

Description

plspm_make_prod , Generate data for indicator interaction multiplied terms

Usage

```
plspm_make_prod(data, blocks, indicator, intname = "int", n_name = 2)
```

Arguments

`data`	ori data
`blocks`	ori blocks
`indicator`	indicator product
`intname`	”int” default
`n_name`	name 2, more 3 int_AAABBB

Value

data

Examples

```
## Not run:
#' data(satisfaction)
#상황이 바뀌는 경우, 데이터를 바꾸어서 생성하는 경우,
plspm_make_prod(satisfaction,
                sat_blocks1,
                indicator= list(intIE = interlist("IMAG",1,2, "EXPE",1,2),
                                intIV = interlist("IMAG",1,2, "QUAL",1)  )
) %>%str()

## End(Not run)
```

---

plspm_mat_lav *plspm_mat_lavaan syntax*

Description

plspm_mat_lavaan syntax

Usage

```
plspm_mat_lav(data, free = FALSE, hypo = FALSE)
```

Arguments

data	matrix
free	TRUE is free parameter
hypo	hypothesis paste

Value

syntax lavaan

Examples

```
## Not run:
#'

# Example data for testing
data_b <- matrix(c(
  0.000, 0.000, 0.000, 0.000, 0.000, 0,
  0.471, 0.000, 0.000, 0.000, 0.000, 0,
  0.000, 0.834, 0.000, 0.000, 0.000, 0,
  0.000, 0.046, 0.701, 0.000, 0.000, 0,
  0.245, -0.017, 0.222, 0.527, 0.000, 0,
  0.182, 0.000, 0.000, 0.000, 0.628, 0
), nrow = 6, byrow = TRUE,
dimnames=list(
  c("IMAG", "EXPE", "QUAL", "VAL", "SAT", "LOY"),
  c("IMAG", "EXPE", "QUAL", "VAL", "SAT", "LOY")
```

```
))
plspm_mat_lav(data_b)%>%cat()
plspm_mat_lav(data_b)%>%
  diagram_model(whatLabels = "est", layout="tree2", nDigits=3,
                edge.label.position=0.65, rotation=1)


data_r <- matrix(c(
  0.000, 0.000, 0.000, 0.000, 0.000, 0,
  0.471, 0.000, 0.000, 0.000, 0.000, 0,
  0.000, 0.834, 0.000, 0.000, 0.000, 0,
  0.000, 0.046, 0.701, 0.000, 0.000, 0,
  0.245, -0.017, 0.222, 0.527, 0.000, 0,
  0.182, 0.000, 0.000, 0.000, 0.628, 0
), nrow = 6, byrow = TRUE,
dimnames=list(c("IMAG", "EXPE", "QUAL", "VAL", "SAT", "LOY")))
data_r

plspm_mat_lav(data_r)%>%cat()

plspm_mat_lav(data_r)%>%
  diagram_model(whatLabels = "est", layout="tree2", nDigits=3,
                edge.label.position=0.65, rotation=1)

data_n <- matrix(c(
  0.000, 0.000, 0.000, 0.000, 0.000, 0,
  0.471, 0.000, 0.000, 0.000, 0.000, 0,
  0.000, 0.834, 0.000, 0.000, 0.000, 0,
  0.000, 0.046, 0.701, 0.000, 0.000, 0,
  0.245, -0.017, 0.222, 0.527, 0.000, 0,
  0.182, 0.000, 0.000, 0.000, 0.628, 0
), nrow = 6, byrow = TRUE)
data_n
plspm_mat_lav(data_n)%>%cat()
plspm_mat_lav(data_n)%>%
  diagram_model(whatLabels = "est", layout="tree2", nDigits=3,
                edge.label.position=0.65, rotation=1)



data_null <- matrix(c(
  0.000, 0.000, 0.000, 0.000, 0.000, 0,
  "H1", 0.000, 0.000, 0.000, 0.000, 0,
  0.000, "H2", 0.000, 0.000, 0.000, 0,
  0.000, "H3", "H4", 0.000, 0.000, 0,
  "H5", "H6", "H7", "H8", 0.000, 0,
  "H9", 0.000, 0.000, 0.000, "H10", 0
), nrow = 6, byrow = TRUE,
dimnames=list(
  c("IMAG", "EXPE", "QUAL", "VAL", "SAT", "LOY"),
  c("IMAG", "EXPE", "QUAL", "VAL", "SAT", "LOY")
))

plspm_mat_lav(data_null)%>%cat()

plspm_mat_lav(data_null)%>%
  diagram_model(whatLabels = "model", layout="tree2", nDigits=3,
```

```
                edge.label.position=0.65, rotation=1)

#'
lay_p = matrix(c('IMA', NA, NA, NA, NA, NA, NA, NA,
                 NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 'EXP', NA,
                 NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 'QUA',
                 NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,
                 NA, NA, NA, NA, NA, NA, NA, NA, NA, 'VAL', NA, NA, NA, NA,
                 NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 'SAT', NA, NA, NA,
                 NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 'LOY',
                 NA, NA, NA, NA, NA, NA, NA, NA),
               nrow = 10, ncol = 10, byrow = FALSE)
lay_p

# Using digaram_model

satpls$path_coefs %>% plspm_mat_lav() %>%
  diagram_model(layout="tree2", whatLabels = "est",
                rotation =1, edge.label.position=0.6)

satpls$path_coefs %>% plspm_mat_lav() %>%
  diagram_model(layout="tree2", whatLabels = "est",
                rotation =1, edge.label.position=0.6,
                edgeLabels=satpls_fit$inner_model %>%plspm_edge_values())
#'
#' #'
data <- matrix(c(0, 1, 0, 1, 0, 1, 1, 0, 0), nrow=3, byrow=TRUE)
rownames(data) <- c("자기효능감", "진로동기", "진로태도")
colnames(data) <- c("진로동기", "진로태도", "진로준비")
data
# hypo=TRUE인 경우
cat(plspm_mat_lav(data, hypo=TRUE))
# 진로동기 ~ H1*자기효능감
# 진로태도 ~ H2*자기효능감
# 진로준비 ~ H3*자기효능감
# 진로준비 ~ H4*진로동기
# 진로준비 ~ H5*진로태도

# free=TRUE, hypo=FALSE인 경우
cat(plspm_mat_lav(data, free=TRUE, hypo=FALSE))
# 진로동기 ~ 자기효능감
# 진로태도 ~ 자기효능감
# 진로준비 ~ 자기효능감
# 진로준비 ~ 진로동기
# 진로준비 ~ 진로태도

# free=FALSE인 경우
cat(plspm_mat_lav(data, free=FALSE, hypo=FALSE))
# 진로동기 ~ 1*자기효능감
# 진로태도 ~ 1*자기효능감
# 진로준비 ~ 1*자기효능감
# 진로준비 ~ 1*진로동기
# 진로준비 ~ 1*진로태도


## End(Not run)
```

plspm_modelfit *Calculate Model Fit Indices for PLS Models*

Description

This function computes various model fit indices for a Partial Least Squares (PLS) model, including SRMR, NFI, d_ULS, and d_G. Bootstrap confidence intervals for d_ULS and d_G are also provided.

Usage

```
plspm_modelfit(
  pls_model,
  blocks,
  bootstrap = 1000,
  ci = 0.95,
  digits = 5,
  trans = TRUE
)
```

Arguments

pls_model	A PLS model object containing model results.
blocks	A list defining the measurement model blocks. Each block should contain variable names.
bootstrap	The number of bootstrap samples to use. Default is 1000.
ci	The confidence level for confidence intervals. Default is 0.95.
digits	The number of decimal places to round results. Default is 5.
trans	If TRUE, transforms the result into a transposed data frame. Default is TRUE.

Value

A data frame containing the computed fit indices and their evaluations.

plspm_model_lav *plspm_model_lav*

Description

plspm_model_lav

Usage

```
plspm_model_lav(paths, blocks)
```

Arguments

paths	paths
blocks	blocks

Value

lavaan syntax

Examples

```
## Not run:
#'
jutpath1 = plspm_paths(
  row_names = c("자기효능감","진로동기","진로태도","진로준비"),
  relationship = list(
    path(from="자기효능감", to=c("진로동기","진로태도","진로준비")),
    path("진로동기", c("진로준비")),
    path("진로태도", c("진로준비"))
  )  )
jutpath1

#측정모형
jut_blocks <- plspm_blocks(
  자기효능감 = item("C_S1","C_S2","C_S3","C_S4","C_S5"),
  진로동기 = item("A_M1","A_M2","A_M3"),
  진로태도 = item(  "B02", "B04", "B10","B14" ),#결정성
  진로준비 = item("D_P1","D_P2","D_P3")
)

jut_blocks
#'
plspm_model_syntax(jutpath1, jut_blocks) %>%cat()

## End(Not run)
```

`plspm_model_plot` *Drawing Structural Equation Models Using the lavaan Syntax (park JH PhD)*

Description

Drawing Structural Equation Models Using the lavaan Syntax (park JH PhD)

Usage

```
plspm_model_plot(
  model,
  residuals = FALSE,
  whatLabels = "model",
  rotation = 2,
  residScale = 12,
  layout = "tree2",
  edge.label.cex = 1.2,
  edge.label.position = 0.55,
  edge.color = "black",
  edge.width = 1.5,
```

```
  sizeMan = 12,
  sizeMan2 = 6,
  sizeLat = 10,
  sizeLat2 = 5,
  label.cex = 1,
  border.width = 2,
  color = list(lat = "gray10", man = "gray90"),
  mar = c(4, 4, 4, 4),
  style = "lisrel",
  shapeLat = "ellipse",
  shapeMan = "circle",
  shapeInt = "triangle",
  exoVar = FALSE,
  exoCov = FALSE,
  curve = 1.5,
  asize = 2,
  nDigits = 3,
  sample.nobs = 100,
  Groups = FALSE,
  group_match = "lat",
  growth = FALSE,
  structural = FALSE,
  edgeLabels = NULL,
  nodeLabels = NULL,
  fixedStyle = 1,
  sig = FALSE,
  type = "plot"
)
```

Arguments

`residuals`	residauls TRUE
`whatLabels`	”model
`rotation`	rotation=2
`residScale`	residScale=12
`layout`	layout=”tree”
`edge.label.cex`	1
`edge.label.position`	0.5
`edge.color`	edge.color= black
`edge.width`	edge.width=1.5
`sizeMan`	8
`sizeMan2`	4
`sizeLat`	10
`sizeLat2`	6
`label.cex`	label.cex
`border.width`	border.width=1
`color`	list(lat=”skyblue”, man=”yellow”)

mar	c(2,8,3,10)
style	lisrel
shapeLat	”circle, ellipse
shapeMan	retangle
shapeInt	triangle ,
exoVar	FALSE
exoCov	TRUE
curve	1.5
asize	1.5
nDigits	2
sample.nobs	sample.nobs = 100
Groups	FALSE
group_match	”FALSE”lat#’
growth	growth= FALSE, TRUE model.type =”grouwth”
structural	structural = FALSE
edgeLabels	edgeLabels=NULL
nodeLabels	nodeLabels=NULL
fixedStyle	fixedStyle=1 default rbase linetype
sig	FALSE
type	type =”plot, and res is lavaan simdata result
plsres_boot	plsres_boot to model syntax

Value

plot jutpls_boot

Examples

```
## Not run:
#'
model_test <- '
  # latent variable definitions
     ind60 =~ x1 + x2 + x3
     dem60 =~ y1 + a*y2 + b*y3 + c*y4
     dem65 =~ y5 + a*y6 + b*y7 + c*y8

  # regressions
    dem60 ~ ind60
    dem65 ~ ind60 + dem60

  # residual correlations
    # y1 ~~ y5
    # y2 ~~ y4 + y6
    # y3 ~~ y7
    # y4 ~~ y8
    # y6 ~~ y8
'

HSmodel <- ' visual  =~ x1 + x2 + x3
```

```
              textual =~ x4 + x5 + x6
              speed   =~ x7 + x8 + x9 '


m1a <- "
y1 =~ x1 + x2 +x5
y2 =~ x3 + x4 +x6
y1 ~~ y2"

diagram_model(model= HSmodel, groups=F,  residuals=F)

diagram_model(model= model_test, style="ram", groups=F)
diagram_model(model= model_test, residuals=T)
diagram_model(model= m1a, residuals=T)


#plspm model
#model 1
model1 = "
EXPE ~ IMAG
QUAL ~ EXPE
VAL ~  QUAL+ EXPE
SAT ~ IMAG + EXPE + QUAL + VAL
LOY ~ SAT + IMAG
"
sat_path = plspm_lav2path(model1,
                          fct_order=c("IMAG", "EXPE","QUAL", "VAL", "SAT", "LOY" ))
sat_path

blocks_model ="
IMAG =~ imag1 + imag2 + imag3 + imag4 + imag5
EXPE =~ expe1 + expe2 + expe3 + expe4 + expe5
QUAL =~ qual1 + qual2 + qual3 + qual4 + qual5
VAL =~ val1 + val2 + val3 + val4
SAT =~ sat1 + sat2 + sat3 + sat4
LOY =~ loy1 + loy2 + loy3 + loy4
"
blocks_model
plspm_lav2blocks(blocks_model)

#model 그리기
paste(model1, blocks_model)%>%
  plspm_model_plot(rotation = 1, sizeLat = 6, sizeMan = 4, sizeMan2=3,
                mar=c(2,2,2,2))

#'
#'
#'

## End(Not run)
```

`plspm_multigroups` *plspm* 에서 *3* 개그룹을비교하는경우, *p* 보정방법포함

Description

plspm 에서 3 개그룹을비교하는경우, p 보정방법포함

Usage

```
plspm_multigroups(
  plspm_boot,
  group,
  method = "bootstrap",
  type = "all",
  adjust = FALSE,
  adjust.method = "bonferroni"
)
```

Arguments

plspm_boot	bootstap data
group	group variable set =”grade”
method	bootstrap, permutation
type	all, grp1, grp2, grp3, ...
adjust	default FALSE
adjust.method	”bonferroni”, ”holm”, ”hochberg”, ”BH”, ”BY”

Value

data

Examples

```
## Not run:
#'
# 사용 예시

#구조모형
jutpath1 = plspm_paths(
  row_names = c("SelfEff","CarMot","CarAtt","CarAct"),
  relationship = list(
    path(from="SelfEff", to=c("CarMot","CarAtt","CarAct")),
    path("CarMot", c("CarAct")),
    path("CarAtt", c("CarAct"))
  )  )
jutpath1

jutpath1 %>% plspm_paths2lav()%>%cat()


# 측정 모형
jut_blocks <- plspm_blocks(
  SelfEff = item("sef01", "sef02", "sef03", "sef04", "sef05"),
  CarMot = item("mot01","mot02","mot03"),
  #   CarAtt = item( "att01", "att02","att03","att04","att05" ),
  CarAtt = item("att03","att04","att05" ),
  CarAct = item("act01", "act02","act03")
)
```

```
jut_blocks

jut_blocks %>%plspm_blocks2lav()%>%cat()



# 데이터 분석 : bootstrap
jutpls_boot = plspm_sem(Data= jut7c,
                        path_matrix = jutpath1,
                        blocks = jut_blocks,seed=19988,
                        br = 5000, summary= TRUE)
# p.adjust() 함수: R에서 제공하는 함수로, 선택된 보정 방법에 맞춰 p-value를 보정한다. p.adjust() 함수
# "bonferroni": 보수적인 Bonferroni 보정.
# "holm": Holm 보정.
# "hochberg": Hochberg 보정.
# "BH": Benjamini-Hochberg 보정.
# "BY": Benjamini-Yekutieli 보정.
# Benjamini-Hochberg 보정을 적용하여 결과 조회
plspm_multigroups(jutpls_boot, group = jut7c$LCA_class,
                  type = "all", adjust = TRUE, adjust.method = "bonferroni")

plspm_multigroups(jutpls_boot, group = jut7c$LCA_class,
                  type = "all", adjust = FALSE, adjust.method = "bonferroni")

plspm_multigroups(jutpls_boot, group = jut7c$LCA_class, method="permutation",
                  type = "all", adjust = FALSE, adjust.method = "bonferroni")

plspm_multigroups(jutpls_boot, group = jut7c$LCA_class, type = "grp1")
plspm_multigroups(jutpls_boot, group = jut7c$LCA_class, type = "grp2")
plspm_multigroups(jutpls_boot, group = jut7c$LCA_class, type = "grp3")

#정한 방법
jut_lca_comparison <- plspm_multigroups(jutpls_boot, group = jut7c$LCA_class)
jut_lca_comparison$Class1_vs_Class2$test
jut_lca_comparison$Class1_vs_Class3$test
jut_lca_comparison$Class2_vs_Class3$test


plspm_multigroups(jutpls_boot, group = jut7c$LCA_class,
                  type = "all", adjust = TRUE, adjust.method = "holm")

plspm_multigroups(jutpls_boot, group = jut7c$LCA_class,
                  type = "all", adjust = FALSE, adjust.method = "holm")

plspm_multigroups(jutpls_boot, group = jut7c$LCA_class,
                  type = "all", adjust = TRUE, adjust.method = "hochberg")
plspm_multigroups(jutpls_boot, group = jut7c$LCA_class,
                  type = "all", adjust = FALSE, adjust.method = "hochberg")

plspm_multigroups(jutpls_boot, group = jut7c$LCA_class,
                  type = "all", adjust = TRUE, adjust.method = "BH")

plspm_multigroups(jutpls_boot, group = jut7c$LCA_class,
                  type = "all", adjust = TRUE, adjust.method = "BY")
#'
#'
```

```
## End(Not run)
```

`plspm_paths` *plspm path create*

Description

plspm path create

Usage

```
plspm_paths(row_names = NULL, col_names = NULL, relationship = NULL)
```

Arguments

`relationship`	input path
`rownames`	rowname
`colnames`	colname

Value

matrix

Examples

```
## Not run:
edu_path <- plspm_path(
  rownames = c("Support", "Advising", "Tutoring", "Value", "Satisfaction", "Loyalty"),
  colnames = c("Support", "Advising", "Tutoring", "Value", "Satisfaction", "Loyalty"),
  relationship = list(
    list(from = "Value",
         to = c("Support", "Advising", "Tutoring")),
    list(from = "Satisfaction",
         to = c("Support", "Advising", "Tutoring","Value")),
    list(from = "Loyalty",
         to =c("Satisfaction"))
  )
)
edu_path


edu_path <- plspm_path(
  rownames = c("Support", "Advising", "Tutoring", "Value", "Satisfaction", "Loyalty"),
  colnames = c("Support", "Advising", "Tutoring", "Value", "Satisfaction", "Loyalty"),
  relationship = list(

    path(form = "Support", to ="Advising"),
    path(form = "Tutoring", to ="Advising"),
    path(from = "Value",
         to = c("Support", "Advising", "Tutoring")),
    path(from = "Satisfaction",
         to = c("Support", "Advising", "Tutoring","Value")),
    path (from = "Loyalty",
```

```
          to =c("Satisfaction"))
  )
)
edu_path
#'

plspm_paths(
  relationship = list(
    list(from="IMAG", to=c("EXPE","SAT","LOY")),
    list(from="EXPE", to=c("QUAL","VAL","SAT")),
    list(from="QUAL", to=c("VAL","SAT")),
    list(from="VAL", to="SAT"),
    list(from="SAT", to="LOY")
  )
)

# 결과 출력

sat_path1 = plspm_paths(
  row_names = c("IMAG","EXPE","QUAL","VAL","SAT","LOY"),
  relationship = list(
    path(from="IMAG", to=c("EXPE","SAT","LOY")),
    path("EXPE", c("QUAL","VAL","SAT")),
    path("QUAL", c("VAL","SAT")),
    path("VAL",c("SAT")),
    path("SAT","LOY"))
)
sat_path1

#'

sat_path <- plspm_paths(relationship = list(
  path(from="IMAG", to=c("EXPE","SAT","LOY")),
  path(from="EXPE", to=c("QUAL","VAL","SAT")),
  path(from="QUAL", to=c("VAL","SAT")),
  path(from="VAL", to="SAT"),
  path(from="SAT", to="LOY")
) )

sat_path <- plspm_paths(relationship = list(
  path(from="IMAG", to=c("EXPE","SAT","LOY")),
  path(from="EXPE", to=c("QUAL","VAL","SAT")),
  path(from="QUAL", to=c("VAL","SAT")),
  path(from="VAL", to="SAT"),
  path(from="SAT", to="LOY")
),
row_names =  c("IMAG", "EXPE", "QUAL", "VAL", "SAT", "LOY")
)

# 결과 출력
print(sat_path)



## End(Not run)
```

plspm_paths2lav *plspm_paths2lav*

Description

plspm_paths2lav

Usage

```
plspm_paths2lav(path_matrix, name = FALSE, hypo = FALSE)
```

Arguments

path_matrix	matrix paths
name	default FALSE, TRUE : paths_model=

Value

lavaan syntax

Examples

```
## Not run:
#'
#' # Example path_matrix
path_matrix <- matrix(c(0, 1, 0, 0, 1, 1,
                        0, 0, 1, 1, 1, 0,
                        0, 0, 0, 1, 1, 0,
                        0, 0, 0, 0, 1, 0,
                        0, 0, 0, 0, 0, 1,
                        0, 0, 0, 0, 0, 0),
                      nrow = 6, ncol = 6, byrow = FALSE)

rownames(path_matrix) <- c("IMAG", "EXPE", "QUAL", "VAL", "SAT", "LOY")
colnames(path_matrix) <- c("IMAG", "EXPE", "QUAL", "VAL", "SAT", "LOY")

cat(plspm_paths2lav(path_matrix))

#Add hypothesis markers
cat(plspm_paths2lav(path_matrix, hypo=TRUE))
#'

## End(Not run)
```

`plspm_paths_coefs_plot` *Draw a Model of a Measured PLSPM Structure*

Description

Draw a Model of a Measured PLSPM Structure

Usage

```
plspm_paths_coefs_plot(
  plsdata,
  layout = "spring",
  boot = TRUE,
  grp = TRUE,
  groups = NULL,
  digits = 3,
  fade = FALSE,
  node = FALSE,
  border.width = 1.5,
  border.color = "gray90",
  edge.label.cex = 1,
  edge.label.position = 0.6,
  asize = 4,
  vsize = 8,
  esize = 10,
  edge_labels_sig = TRUE,
  posCol = "gray20",
  negCol = "red",
  shape = "circle",
  pastel = "pastel",
  trans = TRUE
)
```

Arguments

`layout`	default spring
`boot`	FALSE inner_model TRUE boot
`grp`	groups on T
`groups`	NULL
`digits`	3
`fade`	FALSE
`node`	nodeNames legeng when node =TRUE
`border.width`	1.5
`border.color`	gray30
`edge.label.cex`	1
`edge.label.position`	0.5

asize	3 arrow size
vsize	8 nodesixe
esize	10 edgesize
edge_labels_sig	Tedge_labels_sig= TRUE add sig star
posCol	gray20
negCol	red
shape	cicle
pastel	pastel
trans	true
data	plspm data

Value

plot

Examples

```
## Not run:

# path matrix
IMAG = c(0,0,0,0,0,0)
EXPE = c(1,0,0,0,0,0)
QUAL = c(0,1,0,0,0,0)
VAL = c(0,1,1,0,0,0)
SAT = c(1,1,1,1,0,0)
LOY = c(1,0,0,0,1,0)
sat_path = rbind(IMAG, EXPE, QUAL, VAL, SAT, LOY)

# blocks of outer model
sat_blocks = list(1:5, 6:10, 11:15, 16:19, 20:23, 24:27)

sat_mod = c(rep("A", 6))
# apply plspm
satpls = plspm(satisfaction, sat_path, sat_blocks, modes = sat_mod,
               scaled = FALSE)


 ##jjstat
data(satisfaction)
#model 1
model1 = "
EXPE ~ IMAG
QUAL ~ EXPE
VAL ~  QUAL+ EXPE
SAT ~ IMAG + EXPE + QUAL + VAL
LOY ~ SAT + IMAG
"
# # Generate the matrix
sat_path = plspm_lav2path(model1,
                          fct_order=c("IMAG", "EXPE","QUAL", "VAL", "SAT", "LOY" ))
# sat_path
```

```
# Method 2: paths
sat_path = plspm_paths(
  row_names = c("IMAG","EXPE","QUAL","VAL","SAT","LOY"),
  relationship = list(
    path(from="IMAG", to=c("EXPE","SAT","LOY")),
    path("EXPE", c("QUAL","VAL","SAT")),
    path("QUAL", c("VAL","SAT")),
    path("VAL",c("SAT")),
    path("SAT","LOY")
  )
)
# blokcs
sat_blocks1 <- plspm_blocks(
  IMAG = item(paste0("imag",1:5)),
  EXPE = item(paste0("expe", 1:5)),
  QUAL = item( paste0("qual", 1:5)),
  VAL = item(paste0("val", 1:4)),
  SAT = item(paste0("sat", 1:4)),
  LOY = item(paste0("loy", 1:4))
)
sat_blocks1

# vector of modes (reflective indicators):auto
sat_mod = rep("A", 6)
##############

# apply plspm
satpls = plspm_sem(satisfaction,
                   path_matrix = sat_path,
                   blocks = sat_blocks1,
                   scaled = FALSE)

satpls_boot = plspm_sem(satisfaction, sat_path, sat_blocks1, scaled = FALSE,
                        boot.val =TRUE, br=100)

#default
satpls %>% plspm_path_coefs_plot()



lay_p = matrix(c('IMA', NA, NA, NA, NA, NA, NA,
 NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,
 'EXP', NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,
 NA, NA, NA, NA, 'QUA', NA, NA, NA, NA, NA, NA,
 NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,
 NA, NA, NA, NA, NA, NA, 'VAL', NA, NA, NA, NA,
 NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 'SAT',
 NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,
 NA, NA, NA, NA, 'LOY', NA, NA, NA, NA, NA, NA, NA, NA),
 nrow = 10, ncol = 10, byrow = FALSE)

satpls %>% plspm_path_coefs_plot(layout=lay_p, edge.label.position = 0.6,
                              border.color="gray99", groups=1:6)


# various type
satpls %>% plspm_path_coefs_plot()
```

```
satpls %>% plspm_path_coefs_plot(node =TRUE)

satpls %>% plspm_path_coefs_plot(layout=lay_p, edge.label.position = 0.6,
                                 border.color="gray99", groups=1:6)
satpls %>% plspm_path_coefs_plot(layout=lay_p, edge.label.position = 0.6,
                                 border.color="gray99", groups=1:6, node=TRUE)

satpls %>% plspm_path_coefs_plot(Dshape="square", layout=lay_p,
                                 edge.label.position = 0.6)
#'
satpls_boot%>%plspm_path_coefs_plot()
satpls_boot%>%plspm_path_coefs_plot(boot=F)
#'
satpls %>% plspm_path_coefs_plot()
satpls %>% plspm_path_coefs_plot(node =TRUE)
satpls %>% plspm_path_coefs_plot(grp=FALSE)
satpls %>% plspm_path_coefs_plot(node = FALSE,grp=TRUE)
satpls %>% plspm_path_coefs_plot(node = TRUE,grp=FALSE)
satpls %>% plspm_path_coefs_plot(node = FALSE,grp=FALSE)
satpls %>% plspm_path_coefs_plot(node = TRUE,grp=TRUE)

satpls_boot %>%
plspm_path_coefs_plot(layout =  plspm_boot_factor_layout(satpls_boot)%>%
 move_mat_cut(n=3))

#



## End(Not run)
```

`plspm_paths_sig` *Showing significance using PLSPM regression*

Description

Showing significance using PLSPM regression

Usage

```
plspm_paths_sig(
  data,
  unite = FALSE,
  rhs = "endogenous",
  lhs = "exogenous",
  digits = 3
)
```

Arguments

`data`	plspm data
`unite`	TRUE
`rhs`	from

`lhs`	to
`digits`	round 3

Value

data result

Examples

```
## Not run:
library(plspm)
data(satisfaction)

# path matrix
IMAG = c(0,0,0,0,0,0)
EXPE = c(1,0,0,0,0,0)
QUAL = c(0,1,0,0,0,0)
VAL = c(0,1,1,0,0,0)
SAT = c(1,1,1,1,0,0)
LOY = c(1,0,0,0,1,0)
sat_path = rbind(IMAG, EXPE, QUAL, VAL, SAT, LOY)

# blocks of outer model
sat_blocks = list(1:5, 6:10, 11:15, 16:19, 20:23, 24:27)
sat_mod=c(rep("A", ncol(sat_path)))
# apply plspm
satpls = plspm(satisfaction, sat_path, sat_blocks, modes = sat_mod,
               scaled = FALSE)

#default
satpls%>%plspm_paths_sig()
satpls$inner_model %>% plspm_paths_sig()

satpls %>%plspm_paths_sig(unite=FALSE, lhs="row", rhs="col")
satpls$inner_model  %>%innermodel_sig(unite=FALSE, lhs="row", rhs="col")

satpls %>%plspm_paths_sig(unite=FALSE, lhs="row", rhs="col")
satpls$inner_model %>% plspm_paths_sig(unite=FALSE, lhs="row", rhs="col")

#'

## End(Not run)
```

`plspm_paths_sig_plot` *Draw a Model of a Measured PLSPM Structure*

Description

Draw a Model of a Measured PLSPM Structure

Usage

```
plspm_paths_sig_plot(
  plsdata,
  layout = "spring",
  boot = TRUE,
  grp = TRUE,
  groups = NULL,
  digits = 3,
  fade = FALSE,
  node = FALSE,
  border.width = 1.5,
  border.color = "gray90",
  edge.label.cex = 1,
  edge.label.position = 0.6,
  asize = 4,
  vsize = 8,
  esize = 10,
  edge_labels_sig = TRUE,
  posCol = "gray20",
  negCol = "red",
  shape = "circle",
  pastel = "pastel",
  trans = TRUE
)
```

Arguments

`layout`	default spring
`boot`	FALSE inner_model TRUE boot
`grp`	groups on T
`groups`	NULL
`digits`	3
`fade`	FALSE
`node`	nodeNames legeng when node =TRUE
`border.width`	1.5
`border.color`	gray30
`edge.label.cex`	1
`edge.label.position`	0.5
`asize`	3 arrow size
`vsize`	8 nodesixe
`esize`	10 edgesize
`edge_labels_sig`	Tedge_labels_sig= TRUE add sig star
`posCol`	gray20
`negCol`	red
`shape`	cicle

`pastel`	pastel
`trans`	true
`data`	plspm data

Value

plot

Examples

```
## Not run:

# path matrix
IMAG = c(0,0,0,0,0,0)
EXPE = c(1,0,0,0,0,0)
QUAL = c(0,1,0,0,0,0)
VAL = c(0,1,1,0,0,0)
SAT = c(1,1,1,1,0,0)
LOY = c(1,0,0,0,1,0)
sat_path = rbind(IMAG, EXPE, QUAL, VAL, SAT, LOY)

# blocks of outer model
sat_blocks = list(1:5, 6:10, 11:15, 16:19, 20:23, 24:27)

sat_mod = c(rep("A", 6))
# apply plspm
satpls = plspm(satisfaction, sat_path, sat_blocks, modes = sat_mod,
               scaled = FALSE)


 ##jjstat
data(satisfaction)
#model 1
model1 = "
EXPE ~ IMAG
QUAL ~ EXPE
VAL ~  QUAL+ EXPE
SAT ~ IMAG + EXPE + QUAL + VAL
LOY ~ SAT + IMAG
"
# # Generate the matrix
sat_path = plspm_lav2path(model1,
                          fct_order=c("IMAG", "EXPE","QUAL", "VAL", "SAT", "LOY" ))
# sat_path

# Method 2: paths
sat_path = plspm_paths(
  row_names = c("IMAG","EXPE","QUAL","VAL","SAT","LOY"),
  relationship = list(
    path(from="IMAG", to=c("EXPE","SAT","LOY")),
    path("EXPE", c("QUAL","VAL","SAT")),
    path("QUAL", c("VAL","SAT")),
    path("VAL",c("SAT")),
    path("SAT","LOY")
  )
)
```

```
# blokcs
sat_blocks1 <- plspm_blocks(
  IMAG = item(paste0("imag",1:5)),
  EXPE = item(paste0("expe", 1:5)),
  QUAL = item( paste0("qual", 1:5)),
  VAL = item(paste0("val", 1:4)),
  SAT = item(paste0("sat", 1:4)),
  LOY = item(paste0("loy", 1:4))
)
sat_blocks1

# vector of modes (reflective indicators):auto
sat_mod = rep("A", 6)
##############

# apply plspm
satpls = plspm_sem(satisfaction,
                   path_matrix = sat_path,
                   blocks = sat_blocks1,
                   scaled = FALSE)

satpls_boot = plspm_sem(satisfaction, sat_path, sat_blocks1, scaled = FALSE,
                        boot.val =TRUE, br=100)

#default
satpls %>% plspm_paths_sig_plot()



lay_p = matrix(c('IMA', NA, NA, NA, NA, NA, NA,
 NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,
 'EXP', NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,
 NA, NA, NA, NA, 'QUA', NA, NA, NA, NA, NA, NA,
 NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,
 NA, NA, NA, NA, NA, NA, 'VAL', NA, NA, NA, NA,
 NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 'SAT',
 NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,
 NA, NA, NA, NA, 'LOY', NA, NA, NA, NA, NA, NA, NA, NA),
 nrow = 10, ncol = 10, byrow = FALSE)

satpls %>% plspm_paths_sig_plot(layout=lay_p, edge.label.position = 0.6,
                             border.color="gray99", groups=1:6)


# various type
satpls %>% plspm_paths_sig_plot()
satpls %>% plspm_paths_sig_plot(node =TRUE)

satpls %>% plspm_paths_sig_plot(layout=lay_p, edge.label.position = 0.6,
                                  border.color="gray99", groups=1:6)
satpls %>% plspm_paths_sig_plot(layout=lay_p, edge.label.position = 0.6,
                                  border.color="gray99", groups=1:6, node=TRUE)

satpls %>% plspm_paths_sig_plot(Dshape="square", layout=lay_p,
                                  edge.label.position = 0.6)
#'
satpls_boot%>%plspm_paths_sig_plot()
```

```
satpls_boot%>%plspm_paths_sig_plot(boot=F)
#'
satpls %>% plspm_paths_sig_plot()
satpls %>% plspm_paths_sig_plot(node =TRUE)
satpls %>% plspm_paths_sig_plot(grp=FALSE)
satpls %>% plspm_paths_sig_plot(node = FALSE,grp=TRUE)
satpls %>% plspm_paths_sig_plot(node = TRUE,grp=FALSE)
satpls %>% plspm_paths_sig_plot(node = FALSE,grp=FALSE)
satpls %>% plspm_paths_sig_plot(node = TRUE,grp=TRUE)

satpls_boot %>%
plspm_paths_sig_plot(layout =  plspm_boot_factor_layout(satpls_boot)%>%
 move_mat_cut(n=3))

#



## End(Not run)
```

`plspm_path_coefs_plot` *Draw a Model of a Measured PLSPM Structure*

Description

Draw a Model of a Measured PLSPM Structure

Usage

```
plspm_path_coefs_plot(
  plsdata,
  layout = "spring",
  boot = TRUE,
  grp = TRUE,
  groups = NULL,
  digits = 3,
  fade = FALSE,
  node = FALSE,
  border.width = 1.5,
  border.color = "gray90",
  edge.label.cex = 1,
  edge.label.position = 0.6,
  asize = 4,
  vsize = 8,
  esize = 10,
  edge_labels_sig = TRUE,
  posCol = "gray20",
  negCol = "red",
  shape = "circle",
  pastel = "pastel",
  trans = TRUE
)
```

Arguments

layout	default spring
boot	FALSE inner_model TRUE boot
grp	groups on T
groups	NULL
digits	3
fade	FALSE
node	nodeNames legeng when node =TRUE
border.width	1.5
border.color	gray30
edge.label.cex	1
edge.label.position	0.5
asize	3 arrow size
vsize	8 nodesixe
esize	10 edgesize
edge_labels_sig	Tedge_labels_sig= TRUE add sig star
posCol	gray20
negCol	red
shape	cicle
pastel	pastel
trans	true
data	plspm data

Value

plot

Examples

```
## Not run:

# path matrix
IMAG = c(0,0,0,0,0,0)
EXPE = c(1,0,0,0,0,0)
QUAL = c(0,1,0,0,0,0)
VAL = c(0,1,1,0,0,0)
SAT = c(1,1,1,1,0,0)
LOY = c(1,0,0,0,1,0)
sat_path = rbind(IMAG, EXPE, QUAL, VAL, SAT, LOY)

# blocks of outer model
sat_blocks = list(1:5, 6:10, 11:15, 16:19, 20:23, 24:27)

sat_mod = c(rep("A", 6))
# apply plspm
```

```
satpls = plspm(satisfaction, sat_path, sat_blocks, modes = sat_mod,
               scaled = FALSE)


 ##jjstat
data(satisfaction)
#model 1
model1 = "
EXPE ~ IMAG
QUAL ~ EXPE
VAL ~  QUAL+ EXPE
SAT ~ IMAG + EXPE + QUAL + VAL
LOY ~ SAT + IMAG
"
# # Generate the matrix
sat_path = plspm_lav2path(model1,
                          fct_order=c("IMAG", "EXPE","QUAL", "VAL", "SAT", "LOY" ))
# sat_path

# Method 2: paths
sat_path = plspm_paths(
  row_names = c("IMAG","EXPE","QUAL","VAL","SAT","LOY"),
  relationship = list(
    path(from="IMAG", to=c("EXPE","SAT","LOY")),
    path("EXPE", c("QUAL","VAL","SAT")),
    path("QUAL", c("VAL","SAT")),
    path("VAL",c("SAT")),
    path("SAT","LOY")
  )
)
# blokcs
sat_blocks1 <- plspm_blocks(
  IMAG = item(paste0("imag",1:5)),
  EXPE = item(paste0("expe", 1:5)),
  QUAL = item( paste0("qual", 1:5)),
  VAL = item(paste0("val", 1:4)),
  SAT = item(paste0("sat", 1:4)),
  LOY = item(paste0("loy", 1:4))
)
sat_blocks1

# vector of modes (reflective indicators):auto
sat_mod = rep("A", 6)
##############

# apply plspm
satpls = plspm_sem(satisfaction,
                   path_matrix = sat_path,
                   blocks = sat_blocks1,
                   scaled = FALSE)

satpls_boot = plspm_sem(satisfaction, sat_path, sat_blocks1, scaled = FALSE,
                        boot.val =TRUE, br=100)

#default
satpls %>% plspm_path_coefs_plot()
```

```
lay_p = matrix(c('IMA', NA, NA, NA, NA, NA, NA,
 NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,
 'EXP', NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,
 NA, NA, NA, NA, 'QUA', NA, NA, NA, NA, NA, NA,
 NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,
 NA, NA, NA, NA, NA, NA, 'VAL', NA, NA, NA, NA,
 NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 'SAT',
 NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,
 NA, NA, NA, NA, 'LOY', NA, NA, NA, NA, NA, NA, NA, NA),
 nrow = 10, ncol = 10, byrow = FALSE)

satpls %>% plspm_path_coefs_plot(layout=lay_p, edge.label.position = 0.6,
                            border.color="gray99", groups=1:6)


# various type
satpls %>% plspm_path_coefs_plot()
satpls %>% plspm_path_coefs_plot(node =TRUE)

satpls %>% plspm_path_coefs_plot(layout=lay_p, edge.label.position = 0.6,
                                 border.color="gray99", groups=1:6)
satpls %>% plspm_path_coefs_plot(layout=lay_p, edge.label.position = 0.6,
                                 border.color="gray99", groups=1:6, node=TRUE)

satpls %>% plspm_path_coefs_plot(Dshape="square", layout=lay_p,
                                 edge.label.position = 0.6)
#'
satpls_boot%>%plspm_path_coefs_plot()
satpls_boot%>%plspm_path_coefs_plot(boot=F)
#'
satpls %>% plspm_path_coefs_plot()
satpls %>% plspm_path_coefs_plot(node =TRUE)
satpls %>% plspm_path_coefs_plot(grp=FALSE)
satpls %>% plspm_path_coefs_plot(node = FALSE,grp=TRUE)
satpls %>% plspm_path_coefs_plot(node = TRUE,grp=FALSE)
satpls %>% plspm_path_coefs_plot(node = FALSE,grp=FALSE)
satpls %>% plspm_path_coefs_plot(node = TRUE,grp=TRUE)

satpls_boot %>%
plspm_path_coefs_plot(layout =  plspm_boot_factor_layout(satpls_boot)%>%
 move_mat_cut(n=3))

#



## End(Not run)
```

---

plspm_permutation_test
잠재변수평균값과분산의동일성을평가하는 *Permutation Test* 함수

Description

잠재변수평균값과분산의동일성을평가하는 Permutation Test 함수

Usage

```
plspm_permutation_test(Data, path_matrix, blocks, grp, n_perm = 100)
```

Arguments

`Data`	Data
`path_matrix`	path_matrix
`blocks`	blocks
`grp`	grp = "gender"
`n_perm`	default 100

Value

data

Examples

```
## Not run:
#'
#'
jutpath1 = plspm_paths(
  row_names = c("SelfEff","CarMot","CarAtt","CarAct"),
  relationship = list(
    path(from="SelfEff", to=c("CarMot","CarAtt","CarAct")),
    path("CarMot", c("CarAct")),
    path("CarAtt", c("CarAct"))
  )  )
jutpath1


jut_blocks <- plspm_blocks(
  SelfEff = list("sef01", "sef02", "sef03", "sef04", "sef05"),
  CarMot = list("mot01","mot02","mot03"),
  #   CarAtt = item( "att01", "att02","att03","att04","att05" ),
  CarAtt = list("att03","att04","att05" ),
  CarAct = list("act01", "act02","act03")
)
jut_blocks


result_df <- plspm_permutation_test(Data = jut7c, path_matrix = jutpath1, blocks = jut_blocks, grp = '


result_df
result_df %>%Round(17)%>%dall()



## End(Not run)
```

`plspm_permutation_test_multi`
plspm_permutation_test_multi: Perform pairwise permutation tests for multiple groups

---

Description

plspm_permutation_test_multi: Perform pairwise permutation tests for multiple groups

Usage

```
plspm_permutation_test_multi(Data, path_matrix, blocks, grp, n_perm = 100)
```

Arguments

`Data`	Data frame containing the data.
`path_matrix`	Path matrix for the PLS-PM model.
`blocks`	List of blocks for the PLS-PM model.
`grp`	Grouping variable name (as a string).
`n_perm`	Number of permutations to perform (default: 100).

Value

A list of data frames, where each data frame contains results for a pairwise group comparison.

Examples

```
## Not run:
# Example usage
data <- iris  # Example dataset
path_matrix <- matrix(c(0, 1, 1, 0), 2, 2)  # Example path matrix
blocks <- list(1:2, 3:4)  # Example blocks
result <- plspm_permutation_test_multi(data, path_matrix, blocks, "Species", n_perm = 100)
print(result)

## End(Not run)
```

`plspm_prod` *Create a multiply variable to create a moderator variable*

Description

Create a multiply variable to create a moderator variable

Usage

```
plspm_prod(data, blocks, ..., intname = "int", n_name = 2, id = FALSE)
```

Arguments

data	data
blocks	latent and measurement blocks
...	interation terms
id	discriminat variable

Value

data

Examples

```
## Not run:

# Using example data
data(satisfaction)
sat_blocks1 <- list(
  IMAG = paste0("imag", 1:5),
  EXPE = paste0("expe", 1:5),
  QUAL = paste0("qual", 1:5),
  VAL = paste0("val", 1:4),
  SAT = paste0("sat", 1:4),
  LOY = paste0("loy", 1:4)
)
# Partially select to create an interaction term
satisfaction%>% plspm_prod(sat_blocks1,
                           intIS = interlist(c("IMAG", 1, 2, 3), c("SAT", 1, 2, 3))) %>% str()

# Partially select to create an interaction term : repeated
satisfaction%>% plspm_prod(sat_blocks1,
                           intIS = interlist(c("IMAG", 1, 2, 3), c("SAT", 1, 2, 3))
)%>% plspm_prod(sat_blocks1,
                intIS = interlist(c("IMAG", 1, 2, 3), c("EXPE", 1, 2))
) %>%str()


# Create an interaction variable
plspm_prod(satisfaction, sat_blocks1, intIS = interlist("IMAG", "SAT"))    %>%str()
# Create a variable that interacts with multiple variables
plspm_prod(satisfaction, sat_blocks1, intIS = interlist("IMAG", "SAT"),
           intIE = interlist("IMAG", "EXPE"))    %>%str()

satisfaction%>% plspm_prod(sat_blocks1,
                           interlist(c("IMAG", 1, 2, 3), c("SAT", 1, 2, 3))) %>%str()

satisfaction%>% plspm_prod(sat_blocks1,
                           interlist("IMAG", 1, 2, 3, "SAT", 1, 2, 3) ) %>%str()

# plspm_prod(satisfaction, sat_blocks1, list("IMAG", "SAT"),
#                                        list("IMAG", "EXPE")) %>%str()


#  Use vector
plspm_prod(satisfaction, sat_blocks1,
```

```
            intIS = list(c("IMAG", 1, 2 ), c("SAT", 3,4)),
            intIE = list(c("IMAG", 1, 2, 3), c("EXPE" ,4,5))
) %>%str()

#interlist use use list
plspm_prod(satisfaction,
            sat_blocks1,
            intIS = interlist(list("IMAG", 1, 2 ,"SAT", 3,4)),
            intIE = interlist(list("IMAG", 1, 2, 3, "EXPE" ,4,5)) ) %>%str()


#interlist use now use list
plspm_prod(satisfaction,
            sat_blocks1,
            intIS = interlist("IMAG", 1, 2 ,"SAT", 3,4),
            intIE = interlist("IMAG", 1, 2, 3, "EXPE" ,4,5)) %>%str()

## End(Not run)
```

`plspm_rotate_resid` *plspm_rotate_resid*

Description

plspm_rotate_resid

Usage

```
plspm_rotate_resid(plotdata, resid_list = NULL, plot = TRUE)
```

Arguments

`plotdata`	plotdata
`resid_list`	resid list
`plot`	TRUE

Value

plot data

Examples

```
## Not run:
#'

pathmodel = "
자기효능감 =~ C_S1 +  C_S2 + C_S3 + C_S4 + C_S5
진로태도 =~ B02 + B04 + B10 + B14
진로동기 =~ A_M1 + A_M2 + A_M3
진로준비 =~ D_P1 + D_P2 + D_P3
```

```
진로동기 ~ H1*자기효능감
진로태도 ~  H2*자기효능감
진로준비 ~ H5 *진로태도
진로준비 ~  H4 * 진로동기
진로준비 ~  H3*자기효능감
"

plspm_diagram_model(pathmodel, layout="tree2",
                    rotation = 2,residuals = T, residScale = 6,
                    edge.label.position = 0.6)%>%
  plspm_rotate_resid(c(진로동기= 160, 진로태도= 160))

#
plspm_diagram_model(pathmodel, layout="tree2",
                    rotation = 2,residuals = T, residScale = 6,
                    edge.label.position = 0.6)%>%
  plspm_curve(c("진로준비~~ 진로동기" = -2))%>%
  plspm_rotate_resid(c(진로동기= 160, 진로태도= 30))

## End(Not run)
```

---

plspm_sem *PLS-PM: Partial Least Squares Path Modeling*

Description

PLS-PM: Partial Least Squares Path Modeling

Usage

```
plspm_sem(
  Data,
  path_matrix,
  blocks,
  interactionTerm = NULL,
  indicator_prod = NULL,
  inter_modes = NULL,
  modes = rep("A", ncol(path_matrix)),
  scaling = NULL,
  scheme = "centroid",
  scaled = TRUE,
  tol = 1e-06,
  maxiter = 100,
  plscomp = NULL,
  boot.val = TRUE,
  br = 500,
  seed = NULL,
  dataset = TRUE,
  summary = TRUE
)
```

Arguments

`Data`	matrix or data frame containing the manifest variables.
`path_matrix`	A square (lower triangular) boolean matrix representing the inner model (i.e. the path relationships between latent variables).
`blocks`	list of vectors with column indices or column names from Data indicating the sets of manifest variables forming each block (i.e. which manifest variables correspond to each block).
`interactionTerm`	interaction effect list
`indicator_prod`	generate indicator product data
`inter_modes`	interaction modes
`modes`	optional argument for runing the non-metric approach; it is a list of string vectors indicating the type of measurement scale for each manifest variable specified in blocks. scaling must be specified when working with non-metric variables. Possible values: "num" (linear transformation, suitable for numerical variables), "raw" (no transformation), "nom" (non-monotonic transformation, suitable for nominal variables), and "ord" (monotonic transformation, suitable for ordinal variables).
`scaling`	character vector indicating the type of measurement f or each block. Possible values are: "A", "B", "newA", "PLScore", "PLScow". The length of modes must be equal to the length of blocks.
`scheme`	string indicating the type of inner weighting scheme. Possible values are "centroid", "factorial", or "path".
`scaled`	standardized. Only used when scaling = NULL. When (TRUE, data is scaled to standardized values (mean=0 and variance=1). The variance is calculated dividing by N instead of N-1).
`tol`	decimal value indicating the tolerance criterion for the iterations (tol=0.000001). Can be specified between 0 and 0.001.
`maxiter`	integer indicating the maximum number of iterations (maxiter=100 by default). The minimum value of maxiter is 100.
`plscomp`	optional vector indicating the number of PLS components (for each block) to be used when handling non-metric data (only used if scaling is provided)
`boot.val`	whether bootstrap validation should be performed. (FALSE by default).
`br`	number bootstrap resamples. Used only when boot.val=TRUE. When boot.val=TRUE, the default number of re-samples is 100.
`dataset`	whether the data matrix used in the computations should be retrieved (TRUE by default).
`summary`	TRUE

Value

plspm result

Examples

```
## Not run:
#'
## Not run:
## typical example of PLS-PM in customer satisfaction analysis
## model with six LVs and reflective indicators
# load dataset satisfaction
data(satisfaction)
# path matrix
# IMAG = c(0,0,0,0,0,0)
# EXPE = c(1,0,0,0,0,0)
# QUAL = c(0,1,0,0,0,0)
# VAL = c(0,1,1,0,0,0)
# SAT = c(1,1,1,1,0,0)
# LOY = c(1,0,0,0,1,0)
# sat_path = rbind(IMAG, EXPE, QUAL, VAL, SAT, LOY)

sat_path = plspm_paths(
  row_names = c("IMAG","EXPE","QUAL","VAL","SAT","LOY"),

  relationship = list(
    path(from="IMAG", to=c("EXPE","SAT","LOY")),
    path("EXPE", c("QUAL","VAL","SAT")),
    path("QUAL", c("VAL","SAT")),
    path("VAL",c("SAT")),
    path("SAT","LOY")
  )
)

# plot diagram of path matrix
innerplot(sat_path)
# blocks of outer model
# sat_blocks = list(1:5, 6:10, 11:15, 16:19, 20:23, 24:27)
#' ######################
library(jjstat)
data(satisfaction)
sat_path = plspm_paths(
  row_names = c("IMAG","EXPE","QUAL","VAL","SAT","LOY"),
  relationship = list(
    path(from="IMAG", to=c("EXPE","SAT","LOY")),
    path("EXPE", c("QUAL","VAL","SAT")),
    path("QUAL", c("VAL","SAT")),
    path("VAL",c("SAT")),
    path("SAT","LOY"))
)
sat_path

# blokcs
sat_blocks1 <- plspm_blocks(
  IMAG = item(paste0("imag",1:5)),
  EXPE = item(paste0("expe", 1:5)),
  QUAL = item( paste0("qual", 1:5)),
  VAL = item(paste0("val", 1:4)),
  SAT = item(paste0("sat", 1:4)),
```

```
  LOY = item(paste0("loy", 1:4))
)

sat_blocks1

satpls_boot = plspm_sem(satisfaction, sat_path, sat_blocks1, scaled = FALSE,
                        boot.val =TRUE, br=100)


#'
# interction effect two stage method
resinter_sem = plspm_sem(Data=satisfaction,
                         sat_path2, blocks = sat_blocks1, modes= sat_mod,
                         interactionTerm = list(
                           list(from = "IMAG*EXPE", to = "LOY"),
                           list(from = "IMAG*QUAL", to = "LOY")))
resinter_sem
summary(resinter_sem)
plspm_path_coefs_plot(resinter_sem)
#'
#'



#'
#'
## Two stage approach
pls_ts2 = plspm_sem(satisfaction, sat_path2, sat_blocks1,
                    interactionTerm = list(list(from = "IMAG*EXPE", to = "LOY")) )
pls_ts2 %>%summary()
pls_ts2 %>% plspm_path_coefs_plot()

## indicator arpproach
pls_int2 = plspm_sem(satisfaction, sat_path2, sat_blocks1,
                     interactionTerm = list(list(from = "IMAG*EXPE", to = "LOY")) ,
                     indicator_prod = list(intIE = interlist("IMAG",1,2, "EXPE",3,4)))
pls_int2 %>%summary()
pls_int2 %>% plspm_path_coefs_plot()
pls_int2 %>% plspm_semPaths2()


## End(Not run)
```

`plspm_semPaths` *Drawing Structural Equation Models Using the lavaan Syntax (park JH PhD)*

Description

Drawing Structural Equation Models Using the lavaan Syntax (park JH PhD)

Usage

```
plspm_semPaths(
```

```
  plsres_boot,
  residuals = FALSE,
  whatLabels = "model",
  rotation = 2,
  residScale = 12,
  layout = "tree2",
  edge.label.cex = 0.7,
  edge.label.position = 0.55,
  edge.color = "black",
  sizeMan = 6,
  sizeMan2 = 3,
  sizeLat = 7,
  sizeLat2 = 4,
  style = "lisrel",
  sig = FALSE,
  exoVar = FALSE,
  exoCov = FALSE,
  curve = 1.5,
  asize = 2,
  mar = c(2, 4, 2, 4),
  nDigits = 3,
  shapeLat = "circle",
  shapeMan = "rectangle",
  shapeInt = "triangle",
  sample.nobs = 100,
  border.width = 2,
  edge.width = 1.5,
  Groups = FALSE,
  group_match = "lat",
  growth = FALSE,
  structural = FALSE,
  edgeLabels = NULL,
  nodeLabels = NULL,
  fixedStyle = 1,
  color = list(lat = "white", man = "white"),
  label.cex = 1,
  type = "plot"
)
```

Arguments

`plsres_boot`	plsres_boot to model syntax
`residuals`	residauls TRUE
`whatLabels`	”model
`rotation`	rotation=2
`residScale`	residScale=12
`layout`	layout=”tree”
`edge.label.cex`	1
`edge.label.position`	0.5

edge.color	edge.color= black
sizeMan	8
sizeMan2	4
sizeLat	10
sizeLat2	6
style	lisrel
sig	FALSE
exoVar	FALSE
exoCov	TRUE
curve	1.5
asize	1.5
mar	c(2,8,3,10)
nDigits	2
shapeLat	”circle, ellipse
shapeMan	retangle
shapeInt	triangle ,
sample.nobs	sample.nobs = 100
border.width	border.width=1
edge.width	edge.width=1.5
Groups	FALSE
group_match	”FALSE”lat#’
growth	growth= FALSE, TRUE model.type =”grouwth”
structural	structural = FALSE
edgeLabels	edgeLabels=NULL
nodeLabels	nodeLabels=NULL
fixedStyle	fixedStyle=1 default rbase linetype
color	list(lat=”skyblue”, man=”yellow”)
label.cex	label.cex
type	type =”plot, and res is lavaan simdata result

Value

plot jutpls_boot

`plspm_semPaths2`	*Drawing Structural Equation Models Using the lavaan Syntax (park JH PhD)*

Description

Drawing Structural Equation Models Using the lavaan Syntax (park JH PhD)

Usage

```
plspm_semPaths2(
  plsres_boot,
  residuals = FALSE,
  whatLabels = "model",
  rotation = 2,
  residScale = 12,
  layout = "tree2",
  edge.label.cex = 0.7,
  edge.label.position = 0.55,
  edge.color = "black",
  sizeMan = 6,
  sizeMan2 = 3,
  sizeLat = 7,
  sizeLat2 = 4,
  style = "lisrel",
  sig = FALSE,
  exoVar = FALSE,
  exoCov = FALSE,
  curve = 1.5,
  asize = 2,
  mar = c(2, 4, 2, 4),
  nDigits = 3,
  shapeLat = "circle",
  shapeMan = "rectangle",
  shapeInt = "triangle",
  sample.nobs = 100,
  border.width = 2,
  edge.width = 1.5,
  Groups = FALSE,
  group_match = "lat",
  growth = FALSE,
  structural = FALSE,
  edgeLabels = NULL,
  nodeLabels = NULL,
  fixedStyle = 1,
  color = list(lat = "gray10", man = "gray90"),
  label.cex = 1,
  type = "plot"
)
```

Arguments

plsres_boot	plsres_boot to model syntax
residuals	residauls TRUE
whatLabels	”model
rotation	rotation=2
residScale	residScale=12
layout	layout=”tree”
edge.label.cex	1
edge.label.position	0.5
edge.color	edge.color= black
sizeMan	8
sizeMan2	4
sizeLat	10
sizeLat2	6
style	lisrel
sig	FALSE
exoVar	FALSE
exoCov	TRUE
curve	1.5
asize	1.5
mar	c(2,8,3,10)
nDigits	2
shapeLat	”circle, ellipse
shapeMan	retangle
shapeInt	triangle ,
sample.nobs	sample.nobs = 100
border.width	border.width=1
edge.width	edge.width=1.5
Groups	FALSE
group_match	”FALSE”lat#’
growth	growth= FALSE, TRUE model.type =”grouwth”
structural	structural = FALSE
edgeLabels	edgeLabels=NULL
nodeLabels	nodeLabels=NULL
fixedStyle	fixedStyle=1 default rbase linetype
color	list(lat=”skyblue”, man=”yellow”)
label.cex	label.cex
type	type =”plot, and res is lavaan simdata result

Value

plot jutpls_boot

`plspm_semPaths_lav` *Generate lavaan syntax from a model or plspm result*

Description

Generate lavaan syntax from a model or plspm result

Usage

```
plspm_semPaths_lav(plsres_boot, type = "lav", paths = NULL, blocks = NULL)
```

Arguments

`plsres_boot`	plspm result
`type`	lav, ,model, edge, edgeLabels
`paths`	paths matrix
`blocks`	blocks tree

Value

lavaan syntax

Examples

```
## Not run:

library(jjstat)
data(satisfaction)
sat_path = plspm_paths(
  row_names = c("IMAG","EXPE","QUAL","VAL","SAT","LOY"),
  relationship = list(
    path(from="IMAG", to=c("EXPE","SAT","LOY")),
    path("EXPE", c("QUAL","VAL","SAT")),
    path("QUAL", c("VAL","SAT")),
    path("VAL",c("SAT")),
    path("SAT","LOY"))
)
sat_path

# blokcs
sat_blocks1 <- plspm_blocks(
  IMAG = item(paste0("imag",1:5)),
  EXPE = item(paste0("expe", 1:5)),
  QUAL = item( paste0("qual", 1:5)),
  VAL = item(paste0("val", 1:4)),
  SAT = item(paste0("sat", 1:4)),
  LOY = item(paste0("loy", 1:4))
)

sat_blocks1
```

```
###lav
satpls_boot%>%
  plspm_semPaths_lav(paths=sat_path, blocks=sat_blocks1)%>%cat()

## model
satpls_boot%>%plspm_semPaths_lav()

## edge
satpls_boot%>%plspm_semPaths_lav("edge")

## draw model
satpls_boot%>%
  plspm_semPaths_lav(paths=sat_path, blocks=sat_blocks1)%>%
  diagram_model(  edgeLabels= satpls_boot%>%plspm_semPaths_lav("edge"),
                  mar=c(2,2,2,2),
                  sizeLat=4, sizeMan=3, sizeMan2= 2, edge.label.cex=0.5, rotation=1)

###
satpls_boot%>%plspm_semPaths_lav()%>%
  diagram_model(edgeLabels=satpls_boot%>%plspm_semPaths_lav("edge"),
                mar=c(2,2,2,2),edge.label.cex=0.5, edge.label.position= 0.7,
                sizeLat=4, sizeMan=3, sizeMan2= 2, rotation=1)


## End(Not run)
```

`plspm_semptools` *plspm_semptools*

Description

plspm_semptools

Usage

```
plspm_semptools(
  semPaths_plot,
  pls_boot,
  position = NULL,
  point = NULL,
  set_push = 2.5,
  set_spread = 2.5,
  nrow = 3,
  ncol = 3,
  loading_position = 0.5
)
```

Arguments

`semPaths_plot`	plot data
`pls_boot`	boot data

`position`	layout position
`point`	point position
`set_push`	each push
`set_spread`	each spread
`nrow`	default 3
`ncol`	default 3
`loading_position`	0.5

Value

plot

Examples

```
## Not run:
#'

############################################################
pathmodel = "
자기효능감 =~ C_S1 +  C_S2 + C_S3 + C_S4 + C_S5
진로태도 =~ B02 + B04 + B10 + B14
진로동기 =~ A_M1 + A_M2 + A_M3
진로준비 =~ D_P1 + D_P2 + D_P3

진로동기 ~ H1*자기효능감
진로태도 ~  H2*자기효능감
진로준비 ~ H5*진로태도 + H4*진로동기 + H3*자기효능감
"
pathmodel

plspm_semPaths2(jutpls_boot,   curve=1, layout="tree2",rotation=1,
                sizeMan= 10, color=list(lat="gray10",man="gray98"),
                edge.label.cex=0.7, mar=c(5,5,5,5),
                edge.label.position=0.55,
                sizeLat= 8)%>%
  semptools::set_sem_layout(
    indicator_order =  plspm_indicator_order(pathmodel),
    indicator_factor =  plspm_indicator_factor(pathmodel),
    factor_layout=  matrix(
      c(  NA, '진로태도', NA,
          '자기효능감', NA, '진로준비',
          NA, '진로동기', NA),
      nrow = 3, ncol = 3, byrow = TRUE),
    factor_point_to= matrix(
      c(  NA, 'up', NA,
          'left', NA, 'right',
          NA, 'down', NA),
      nrow = 3, ncol = 3, byrow = TRUE),
    indicator_push <- c(진로태도 = 2, 진로동기 = 2, 자기효능감=1.5, 진로준비 =1.5),
    indicator_spread <- c(자기효능감 = 3, 진로태도 = 2, 진로동기 =2, 진로준비 = 2)
  )%>%  semptools::set_curve(c("진로준비~~ 진로태도" = 0.5))%>%
  semptools::set_curve(c("진로준비~~ 진로동기" = 0))%>%
  plspm_change_nodeLabels("B02","진로결정",
```

```
                          "B04", "진로독립",
                          "B10", "진로계획",
                          "B14", "진로성향",
                          "A_M1","진로정체",
                          "A_M2","진로통찰",
                          "A_M3","진로탄력",
                          "C_S1","자기평가",
                          "C_S2","직업정보",
                          "C_S3","계획수립",
                          "C_S4","목표설정",
                          "C_S5","해결방식",
                          "D_P1","진로준비",
                          "D_P2","도구준비",
                          "D_P3","목표달성"
  )
# second ##################################
plspm_indicator_order(pathmodel)
plspm_indicator_factor(pathmodel)


#옵션함수
plspm_boot_ind_order(jutpls_boot)
plspm_boot_ind_factor(jutpls_boot)

plspm_boot_factor_layout(jutpls_boot)
plspm_boot_factor_point_to(jutpls_boot, ncol=3)

plspm_ind_push(jutpls_boot)
plspm_ind_spread(jutpls_boot, set= c(자기효능감=4,진로태도=3, 진로동기=3))

##plot
plspm_semPaths2(jutpls_boot,   curve=1, layout="tree2",rotation=1,
                sizeMan= 10, color=list(lat="gray10",man="gray98"),
                edge.label.cex=0.7, mar=c(5,5,5,5),
                edge.label.position=0.55, edge.color= "tomato4",
                sizeLat= 8)%>%
  semptools::set_sem_layout(
    indicator_order = plspm_boot_ind_order(jutpls_boot),
    indicator_factor = plspm_boot_ind_factor(jutpls_boot),
    factor_layout= plspm_boot_factor_layout(jutpls_boot, ncol=3),
    factor_point_to= plspm_boot_factor_point_to(jutpls_boot, ncol=3),
    indicator_push <- plspm_ind_push(jutpls_boot),
    indicator_spread <-  plspm_ind_spread(jutpls_boot, set= c(자기효능감=4,진로태도=3, 진로동기=3)))%>
  plspm_curve(c("진로준비~ 진로동기" =0))%>%
  plspm_change_nodeLabels("B02","진로결정",
                          "B04", "진로독립",
                          "B10", "진로계획",
                          "B14", "진로성향",
                          "A_M1","진로정체",
                          "A_M2","진로통찰",
                          "A_M3","진로탄력",
                          "C_S1","자기평가",
                          "C_S2","직업정보",
                          "C_S3","계획수립",
                          "C_S4","목표설정",
                          "C_S5","해결방식",
                          "D_P1","진로준비",
```

```
                          "D_P2","도구준비",
                          "D_P3","목표달성"
  )

#'
#' #we want ########################################
plspm_semPaths2(jutpls_boot,   curve=1, layout="tree2",rotation=1,
                sizeMan= 10, color=list(lat="gray10",man="gray98"),
                edge.label.cex=0.7, mar=c(5,5,5,5),
                edge.label.position=0.55, edge.color= "tomato4",
                sizeLat= 8) %>%
  plspm_semptools(jutpls_boot)%>%
  plspm_curve(c("진로준비~ 진로동기" =0))%>%
  plspm_change_nodeLabels("B02","진로결정",
                          "B04", "진로독립",
                          "B10", "진로계획",
                          "B14", "진로성향",
                          "A_M1","진로정체",
                          "A_M2","진로통찰",
                          "A_M3","진로탄력",
                          "C_S1","자기평가",
                          "C_S2","직업정보",
                          "C_S3","계획수립",
                          "C_S4","목표설정",
                          "C_S5","해결방식",
                          "D_P1","진로준비",
                          "D_P2","도구준비",
                          "D_P3","목표달성"
  )
#'



## End(Not run)
```

`pls_modelfit_apa` *pls_modelfit_apa*

Description

pls_modelfit_apa

Usage

```
pls_modelfit_apa(model_fit_data)
```

Arguments

`model_fit_data`
: modelfit

Value

data

Examples

```
## Not run:
#'
rest= plspm_modelfit(edt_pls, edblock, bootstrap = 1000, ci = 0.95, trans=FALSE)
rest
rest%>%t()%>%rowdata2col(1)%>%data.frame()

pls_modelfit_apa(rest)


## End(Not run)
```

---

poolcor_comment *metasem primary analysis analysis results by JH, Park*

Description

metasem primary analysis analysis results by JH, Park

Usage

```
poolcor_comment(res)
```

Arguments

res	metaSEM result

Examples

```
## Not run:
## making correlation matrix ##
cormat1 = metacorMat(rd2, startcol= 3, var=c("po","ne","se","jp"))
 ##variable : "po_ne" "po_se" "po_jp" "ne_se" "ne_jp" "se_jp"
 varnames <- c("po","ne","se","jp")
rd2<- data.frame(
  stringsAsFactors = FALSE,
  study = c(1L,
            2L,3L,4L,5L,6L,7L,8L,9L,10L,11L,12L,
            13L,14L,15L,16L,17L,18L,19L,20L,
            21L,22L,23L,24L,25L,26L,27L,28L,29L,
            30L,31L,32L,33L,34L,35L,36L),
  n = c(671L,
        92L,73L,321L,214L,947L,321L,231L,
        531L,567L,213L,928L,269L,593L,78L,983L,
        1504L,247L,79L,875L,68L,201L,891L,
        197L,431L,98L,179L,69L,301L,679L,273L,
        254L,316L,781L,273L,268L),
  po_ne = c(NA,
            NA,-0.33,NA,NA,NA,NA,-0.32,NA,NA,NA,
            NA,NA,NA,-0.21,-0.49,NA,-0.14,-0.29,
            -0.27,NA,-0.37,NA,NA,-0.35,-0.17,NA,
            -0.11,NA,NA,-0.41,NA,NA,NA,NA,-0.39),
  po_se = c(NA,
```

```
            0.37,0.29,0.16,0.41,NA,NA,0.21,0.59,
            0.53,0.31,0.38,NA,NA,0.19,NA,NA,NA,
            NA,0.21,0.07,0.57,0.45,NA,0.18,NA,
            0.45,NA,NA,0.09,NA,NA,0.37,0.19,0.23,
            NA),
  po_jp = c(0.16,
            0.39,0.09,0.07,NA,0.04,NA,NA,0.33,
            0.11,NA,0.04,NA,NA,0.21,0.23,0.09,
            0.11,0.31,0.11,-0.07,0.14,NA,0.19,0.13,
            0.25,0.35,0.29,0.35,NA,0.27,0.19,NA,
            NA,0.13,0.19),
  ne_se = c(NA,
            -0.31,NA,NA,NA,NA,NA,-0.19,NA,NA,NA,
            NA,-0.49,NA,-0.45,NA,NA,NA,NA,-0.38,
            NA,-0.39,NA,NA,-0.21,NA,NA,NA,NA,
            -0.23,NA,NA,NA,NA,-0.44,NA),
  ne_jp = c(NA,
            -0.41,NA,NA,NA,-0.18,-0.11,NA,NA,NA,
            NA,NA,-0.21,-0.15,-0.36,-0.29,NA,
            -0.27,-0.29,-0.23,NA,-0.27,NA,NA,-0.19,
            -0.07,NA,-0.31,NA,NA,0.07,NA,NA,NA,
            -0.27,-0.05),
  se_jp = c(NA,
            NA,NA,0.07,NA,NA,NA,NA,0.27,0.07,NA,
            NA,0.48,0.26,0.41,NA,NA,NA,NA,0.31,
            0.19,0.34,NA,NA,0.19,NA,0.39,NA,
            0.57,NA,NA,NA,NA,NA,NA,NA),
  jobyear = c("f",
              "f","s","s","s","s","f","f","s","s",
              "s","s","f","f","f","f","f","f","s",
              "s","s","s","s","f","s","f","f",
              "f","f","f","f","s","s","s","f","f"),
  wage = c(9L,
           1L,1L,1L,4L,9L,2L,3L,3L,3L,2L,7L,
           1L,8L,6L,3L,5L,7L,8L,4L,7L,1L,4L,
           6L,7L,1L,9L,6L,7L,7L,7L,9L,2L,5L,
           7L,3L)
)

cormat1


## [[1]]
## po ne se   jp
## po 1.00 NA NA 0.16
## ne   NA NA NA   NA
## se   NA NA NA   NA
## jp 0.16 NA NA 1.00
##
## [[2]]
## po    ne    se    jp
## po   NA    NA  0.37  0.39
## ne   NA  1.00 -0.31 -0.41
## se 0.37 -0.31    NA    NA
## jp 0.39 -0.41    NA  1.00
## 출처 meta-analaysis 경로분석모델 1단계 - 상관행렬추정 correlation using metaSEM|자유자재Paper
```

```
### 1. How to obtain the correlation matrix using the fixed effects model
library(metaSEM)
fixed1 <- tssem1(Cov=cormat1, n=rd2$n, method="FEM")
summary(fixed1)
coef(fixed1)

## generate comment
 poolcor_comment(fixed1)

## End(Not run)
```

---

posthoc_duncan *posthoc_duncan*

Description

posthoc_duncan

Usage

```
posthoc_duncan(aov_res, group_var)
```

Arguments

aov_res	aov result
group_var	group

Value

df

Examples

```
## Not run:
#'
aov_post_eng <- pwj3_long %>%
  filter(time == "Post" & factor == "eng") %>%
  aov(formula = score ~ Eng_Class)

# Duncan 사후분석 수행
posthoc_duncan(aov_post_eng, "Eng_Class")

## End(Not run)
```

posthoc_plot *Post-hoc results and plots*

Description

Post-hoc results and plots

Usage

```
posthoc_plot(
  aov_data,
  type = "all",
  caption = "Table. Multiple comparison",
  console = FALSE,
  group = TRUE,
  adjust = "none"
)
```

Arguments

aov_data	aov data
type	type is all, res, g, post hoc
adjust	node fdr bonferroni

Value

plot and analysis result

Examples

```
## Not run:

#Need to replace cyl with factor first. Change to as_trt() first
aov(mpg ~ cyl, data=mtcars %>%as_trt("cyl"))%>%posthoc_plot()

##Error in emmeans::emmeans(., iv) :
aov(mpg ~ factor(cyl), data=mtcars)%>%posthoc_plot()


## End(Not run)
```

PRE *PRE(proportional reduction in error)*

Description

PRE(proportional reduction in error)

Usage

```
PRE(model1, model2)
```

Arguments

model1	model 1
model2	model q mixed model

Value

random slope, inetcept

Examples

```
## Not run:
 PRE(gmlm.dich.m1b, gmlm.dich.m2c)

## End(Not run)
```

Pull *Pull select data.frame col to vector*

Description

Pull select data.frame col to vector

Usage

```
Pull(df, col = "Freq", name = "term", sep = "")
```

Arguments

df	df
col	select col, ”Freq”
name	using names from rownames or col data”name”
sep	paste sep = ”_”

Value

vector

Examples

```
## Not run:
#'
# 예시 데이터
df <- tibble::tibble(
  term = c("학교 수업에서", "혼자서 공부할 때", "활용 안함"),
  Freq = c(2058, 1248, 646)
)

# 함수 사용 예시
Pull(df, col="Freq", name="term")

## End(Not run)
```

`Pull_to_text` *Convert Data Frame Row to Text with Rounded Digits*

Description

Convert Data Frame Row to Text with Rounded Digits

Usage

```
Pull_to_text(df, digits = 3)
```

Arguments

`df`	A data frame or tibble with a single row.
`digits`	Integer, number of decimal places to round numeric values (default = 3).

Value

A character string representing the key-value pairs from the data frame.

Examples

```
df <- tibble::tibble(R = 0.205678, adj.R = 0.204123, F = 195.456, p = 3.67e-222, dof = 6, AIC = 10028
Pull_to_text(df, digits = 3)
```

p_mark_sig *p_mark_sig is Function that displays significance in data*

Description

p_mark_sig is Function that displays significance in data

Usage

```
p_mark_sig(
  data,
  col = "p.value",
  unite = FALSE,
  unite_col = NULL,
  digits = 3,
  ns = "ns",
  rownames_to_column = FALSE,
  exclude = NULL,
  remove = TRUE
)
```

Arguments

data	input data.frame or matrix
col	'col' specifies the name of the column representing the p value in the data. By default, it is set to p.value. For example, if it is written as p, pvalue, p_value, etc., you can change the name.
unite	TRUE combine pvalue and sig
unite_col	unite_col is the variable we want to merge with sig
digits	default 3
ns	ns="ns"
rownames_to_column	drownames_to_column if you need
exclude	Round exclude varible
remove	remove var TRUE

Examples

```
## Not run:
## data(ToothGrowth)
library(dplyr)
library(rstatix)

##method 1
 ToothGrowth %>%
  group_by(dose) %>%
  rstatix::t_test(data =., len ~ supp) %>%
  p_mark_sig("p")
```

```
  ### NEW
  TukeyHSD(aov(weight ~ feed, chickwts))$feed %>% as.data.frame() %>%
  mutate(sig = add_sig(`p adj`))

  TukeyHSD(aov(weight ~ feed, chickwts))$feed %>% p_mark_sig("p adj")


##method 2
 ToothGrowth %>%
 group_by(dose) %>%
 rstatix::t_test(data =., len ~ supp) %>%
  p_mark_sig("p", unite=TRUE)


 ToothGrowth %>%
     group_by(dose) %>%
    rstatix::t_test(data =., len ~ supp) %>%
    p_mark_sig("p", unite = TRUE, unite_col = "statistic" ,ns="")

 ToothGrowth %>%
   group_by(dose) %>%
   rstatix::t_test(data =., len ~ supp) %>%
   p_mark_sig("p", unite = TRUE, unite_col = "statistic" )

## End(Not run)
```

`p_sig_cal` *observed expect table with calculate_chi_sig*

Description

observed expect table with calculate_chi_sig

Usage

```
p_sig_cal(obs_data)
```

Arguments

`obs_data`	matrix

Value

matrix

p_to_001 *p value to <.001*

Description

p value to <.001

Usage

```
p_to_001(df, ..., digits = 3, remove = TRUE)
```

Arguments

df	df data
...	col names and number
digits	3
remove	default TRUE, col replace

Value

df

Examples

```
## Not run:
#'

# 예제 데이터
library(dplyr)

df <- tibble(
  pairwise = c("30분이상_20-30분", "30분이상_10-20분", "30분이상_5-10분", "30분이상_5분이내", "20-30분
               "20-30분_5-10분", "20-30분_5분이내", "10-20분_5-10분", "10-20분_5분이내", "5-10분_5분이
  chisq = c(198, 275, 349, 441, 9.27, 34.2, 87.2, 8.4, 44.6, 16.0),
  df = c(1, 1, 1, 1, 1, 1, 1, 1, 1, 1),
  p.value = c(5.83e-45, 1.09e-61, 8.37e-78, 7.50e-98, 2.33e-3, 4.85e-9, 9.66e-21, 3.76e-3, 2.44e-11, 6
  adj.p = c(1.46e-44, 3


            # 함수 실행 예시
            # 열 번호로 처리
            result <- p_to_001(df, 4, 6, remove = TRUE)

            # 열 이름으로 처리
            result2 <- p_to_001(df, "p.value", "adj.p", remove = TRUE)

            print(result)  # 열 번호로 처리한 결과
            print(result2) # 열 이름으로 처리한 결과
#'

## End(Not run)
```

p_value_cal — *Observation Expectation Table with calculate_chi_sig*

Description

Observation Expectation Table with calculate_chi_sig

Usage

```
p_value_cal(obs_data)
```

Arguments

obs_data	matrix

Value

matrix

read_text — *read_text: Read a Text File with Flexible Handling Using read.delim*

Description

read_text: Read a Text File with Flexible Handling Using read.delim

Usage

```
read_text(file_path, header = TRUE, stringsAsFactors = FALSE, sep = "\t")
```

Arguments

file_path	The path of the file to read.
header	Logical. Does the file contain a header? Defaults to TRUE.
stringsAsFactors	Logical. Should character columns be converted to factors? Defaults to FALSE.

Value

A data frame containing the file's contents.

Examples

```
## Not run:
# Save and read a file
teacher_question <- c("Example question 1", "Example question 2")
write_text(teacher_question, file_path = "output.txt")
result <- read_text("output.txt")

## End(Not run)
```

recode_sep_vec *recode_sep_vec* 벡터데이터를리코딩

Description

recode_sep_vec 벡터데이터를리코딩

Usage

```
recode_sep_vec(vector, ranges_list)
```

Arguments

vector	vector
ranges_list	변환리스트 list("0~5 년" = 0:5)

Value

vector

Examples

```
## Not run:
#'
# 예시 사용
vec <- c("53", "45", "38", "28", "41", "12", "0", "10", "30", "35")

# 함수를 사용하여 벡터 재코딩
new_vec <- recode_sep_vec(vec, list("0~5년" = 0:5,
                                    "5~10년" = 6:10,
                                    "11~15년" = 11:15,
                                    "16~20년" = 16:20,
                                    "21~25년" = 21:25,
                                    "26~30년" = 26:30,
                                    "30년이상" = 31:60))

# 결과 확인
print(new_vec)

# # # 예시 사용
# vec <- c(53, 45, 45, 38, 36, 28, 38, 28, 41, 55, 39, 29, 31, 35, 37, 34, 34, 35, 58, 46)

# # 함수를 사용하여 벡터 재코딩
new_vec <- recode_sep_vec(vec, list("20대" = 20:29, "30대" = 30:39, "40대" = 40:49, "50대" = 50:59))

# # 결과 확인
# print(new_vec)

## End(Not run)
```

reg_apa *regression APA report*

Description

regression APA report

Usage

```
reg_apa(model, digits = 3, md = FALSE)
```

Arguments

model	lm_data
digits	3
md	markdown table

Value

report

Examples

```
## Not run:
lm(mpg ~ hp + wt + disp + qsec, data = mtcars) %>% reg_apa()

## End(Not run)
reg1
```

remove_all_attr *Remove all ATTRs present in the data*

Description

Remove all ATTRs present in the data

Usage

```
remove_all_attr(df)
```

Arguments

df	data

Value

data

`remove_attr` *Remove all ATTRs present in the data, deselect*

Description

Remove all ATTRs present in the data, deselect

Usage

```
remove_attr(df, range = 1:ncol(df))
```

Arguments

`df`	data
`range`	col range

Value

data.frame

Examples

```
## Not run:
#'
# Example usage with your data frame jut3
# Remove attributes from all columns
# jut3_cleaned_all <- remove_attr(jut3)

# Remove attributes from columns 1 to 5
# jut3_cleaned_partial <- remove_attr(jut3, range = 1:5)

# Check the structure
# str(jut3_cleaned_all)
# str(jut3_cleaned_partial)

## End(Not run)
```

`remove_blank` 데이터내에서빈칸제거

Description

데이터내에서빈칸제거

Usage

```
remove_blank(
  data,
  col = NULL,
  data_type = c("data.frame", "vector", "character")
)
```

Arguments

data	df
col	col 숫자와열이름으로지정
data_type	data.frame, vector

Value

df

Examples

```
## Not run:
#'
# 예제 데이터
vector_data <- c("익산 어양 초", "익산옥 야 초")
df_data <- data.frame(
  ID = c(259, 260),
  Name = c("익산 어양 초", "익산옥 야 초"),
  Value = c(1, 1),
  Score = c(0.1237624, 0.1237624)
)

# 벡터에서 공백 제거
clean_vector <- remove_blank(vector_data, data_type = "vector")
print(clean_vector)

# 데이터프레임에서 공백 제거 (열 이름으로)
clean_df <- remove_blank(df_data, col = "Name")
print(clean_df)

# 데이터프레임에서 공백 제거 (열 번호로)
clean_df <- remove_blank(df_data, col = 2)
print(clean_df)
#'


## End(Not run)
```

---

remove_cols *Delete cols in the data*

Description

Delete cols in the data

Usage

```
remove_cols(data, rm_col = NULL, ...)
```

Arguments

data	data
rm_col	first column name or number
...	add columns

Value

data

Examples

```
## Not run:
#'
# 예시 사용법
col_remove(mtcars, 1, "am", 2, "vs", "wt")

# 예시 사용법
col_remove(mtcars, 1, "am", 2, "vs", "wt",4)


# 예시 사용법
col_remove(mtcars, 1, "am", 2, "vs", "wt",3)


# 예시 사용법
col_remove(mtcars, 1, "am", 2, "vs", "wt",3,4)

# 예시 사용법
col_remove(mtcars, rm_col=1:3)
col_remove(mtcars, c(1, 3))
col_remove(mtcars, 1,2, 3)
col_remove(mtcars, 1, "am",2,"vs","wt")
col_remove(mtcars, rm_col="mpg")
col_remove(mtcars, "mpg", "am","vs")


## End(Not run)
```

---

remove_filter *Functions to remove selected rows*

Description

Functions to remove selected rows

Usage

```
remove_filter(data, ...)
```

Arguments

data	data
...	condition filter

Value

data

Examples

```
## Not run:
#'
mtcars %>% remove_filter(cyl == 6 )
mtcars %>% remove_filter(vs == 1)
mtcars %>% remove_filter(am==0)
mtcars %>% remove_filter(vs == 1)
mtcars %>% remove_filter(cyl == 6 | vs == 1)
mtcars %>% remove_filter(cyl == 6 & vs == 1)
mtcars %>% remove_filter(cyl == 6 | cyl == 4 | am == 0)

## End(Not run)
```

---

remove_rows *remove rows*

Description

remove rows

Usage

```
remove_rows(data, ..., filter = FALSE)
```

Arguments

data	data
...	rows
filter	TRUE possible filter

Value

data

Examples

```
## Not run:
#'
# 1~3행 제거
remove_rows(mtcars,  1:30)
remove_rows(mtcars, 1,2,3,6,5,7,8,9,30)
remove_rows(mtcars, rm = c(1,3,4,5,6,7,11:30))
remove_rows(mtcars, cyl == 4, filter=T)

# 1행과 3행 제거
print(remove_rows(mtcars, rm_row = c(1, 3)))

# 필터 조건 적용 (cyl == 6)
remove_rows(mtcars,rm=NULL, cyl == 6) #error
remove_rows(mtcars, cyl == 6 |cyl ==4, filter=T)

# 필터 조건 적용 (cyl == 6 또는 vs == 1)
remove_rows(mtcars, cyl == 6 | vs == 1, filter=T)

# 필터 조건 적용 (cyl == 6 그리고 vs == 1)
remove_rows(mtcars, cyl == 6 | vs == 1, filter=T)

# 필터 조건 적용 후 특정 행 제거 (필터가 우선 적용)
remove_rows(mtcars, 1:3, cyl == 6 | vs == 1)

#'

## End(Not run)
```

`rename_by_dict` *rename_by_dict: Rename both column names and cell values using a dictionary*

Description

This function renames column names and character cell values in a data frame using a named character vector as dictionary. Especially useful in pipes.

Usage

```
rename_by_dict(df, dict)
```

Arguments

`df`	A data.frame or tibble.
`dict`	A character vector of key-value pairs (e.g., c("eng1" = "kor1", ...)) or unnamed like c("eng1", "kor1", ...)

Value

A data.frame with renamed column names and character cell values.

Examples

```
df <- data.frame(latent = c("access", "barrier"), access = 1:2, barrier = 3:4)
var_named <- c("access", "연구접근성", "barrier", "진로장벽")
rename_by_dict(df, var_named)
```

`reorder_row` *Reorder rows in a dataframe with a given row number*

Description

Reorder rows in a dataframe with a given row number

Usage

```
reorder_row(data, ...)
```

Arguments

`data`	data.
`...`	row order

Value

Reordered data

Examples

```
## Not run:
Create a dataframe
df <- data.frame(
  response = c("그렇다", "그렇지않다", "매우그렇다", "보통이다", "전혀그렇지않다"),
  Freq = c(5, 13, 4, 14, 4)
)
df
# Call a function to reorder rows
reorder_row(df, 5, 2, 4, 1,3)  # Put the row numbers in order.

#data process
data_long <- tribble(
  ~response, ~freq,
  "전혀그렇지않다", 1,
  "그렇지않다", 2,
  "보통이다", 3,
  "그렇다", 2,
  "매우그렇다", 0,
  "전혀그렇지않다", 0,
  "그렇지않다", 4,
  "보통이다", 4,
  "그렇다", 0,
  "매우그렇다", 0,
  "전혀그렇지않다", 1,
  "그렇지않다", 3,
  "보통이다", 4,
  "그렇다", 0,
```

```
  "매우그렇다", 0,
  "전혀그렇지않다", 1,
  "그렇지않다", 4,
  "보통이다", 3,
  "그렇다", 0,
  "매우그렇다", 0
)
data_long <- data_long %>%as_trt("response")
data_long

data_long %>%add_rows_freq() %>% rowid_to_column("person")%>%
  select(response)%>%table()%>%dall()%>%
  reorder_row(5,2,4,1,3)


## End(Not run)
```

`Replace` *Create a function to replace NA with an empty string in an R vector data*

Description

Create a function to replace NA with an empty string in an R vector data

Usage

```
Replace(vector, pattern = NA, imp = "")
```

Arguments

`vector`	input data
`pattern`	dectect pattern
`imp`	imputation

Value

imputation data

Examples

```
## Not run:
# Replace(c("A", NA, "B", "", "C", "C"), "C", imp="T")

## End(Not run)
```

replace_dataframe *replace_dataframe*

Description

replace_dataframe

Usage

```
replace_dataframe(data, pattern, imp)
```

Arguments

data	data
pattern	patern
imp	imputation

Value

data

replace_df *Search for and fill in empty data*

Description

Search for and fill in empty data

Usage

```
replace_df(df, pattern = NA, imp = "", col = NULL)
```

Arguments

df	data.frame
pattern	default ""
imp	iimputation , defalut NA

Value

data.frame

Examples

```
## Not run:
#'
New = data.frame(term = c("(Intercept)", "age30대",
 "age40대", "gendermale", "manufacturergoogle",
 "manufacturersamsung",
 "age30대:gendermale", 'age40대:gendermale',
  'age30대:manufacturergoogle', "age40대:manufacturergoogle",
  "age30대:manufacturersamsung", 'age40대:manufacturersamsung'),
estimate = c(6.72127226828228, 0.222516673400011, 0.613427717767302,
 -0.451934840902044, -0.0405765656320795, 0.380612934433137,
  0.00538723055632342, 0.52238521880372, 0.0022025527504637,
  0.0243934423439851, 0.509454602341916, 0.00845576255419509),
std.error = c(0.029551066353128, 0.0393330112847198,
0.0358559955535251, 0.0300846458760622, 0.0387717417205405,
 0.0352086354160786, 0.0383047023682364, 0.0356296478558013,
 0.0519877886919915, 0.0462375166064463,
 0.0454909790784573, 0.0420442735967295),
statistic = c(227.446014568974, 5.65724988074979,
 17.1080933131975, -15.0221093764524, -1.04654998283409,
 10.8102154467288, 0.140641493687488, 14.6615319050554,
  0.0423667327632075, 0.527568176976533,
   11.1990247882612, 0.201115677138321),
p.value = c(0, 1.53817738615173e-08, 1.29158703632758e-65,
5.26066233107499e-51, 0.295307180741282, 3.0794597517271e-27,
 0.888153166786026, 1.13678932467476e-48,
 0.966206347935429, 0.59779908524448,
 4.12246475726497e-29, 0.840608123856931)
)%>%tibble()

New

dfs= New%>%p_mark_sig(ns="", unite=TRUE, unite_col="estimate")

dfs%>%
  replace_df("0.000","< .001", col="p.value")

dfs%>%
  replace_df("0.000","< .001", col=c("p.value", "std.error"))



## End(Not run)
```

`replace_df2` *Search for and fill in NA data*

Description

Search for and fill in NA data

Usage

```
replace_df2(df, pattern = NA, imp = "", cat = TRUE)
```

Arguments

df	data.frame
pattern	default ""
imp	iimputation , defalut NA

Value

data.frame

---

replace_df_rep *replace_df_repeated adation*

Description

replace_df_repeated adation

Usage

```
replace_df_rep(data, ..., detect = TRUE, chr = TRUE)
```

Arguments

data	data
...	pattern and imputation
detect	detect TRUE in data, FALSE data
chr	transformation fac to chr

Value

data

Examples

```
## Not run:

# Use case 1: Passing pattern-replacement string
# pairs directly as variable arguments
data <- data.frame(
  col1 = c("aa", "bb", "cc"),
  col2 = c("aa", "bb", "dd")
)

print("Before Replacement:")
print(data)

result1 <- replace_df_rep(data, "aa", "a1", "bb", "b1")
print("After Replacement (direct arguments):")
```

```
print(result1)

# Use case 2: Passing as a single vector
changes <- c("aa", "a1", "bb", "b1")
result2 <- replace_df_rep(data, changes)
print("After Replacement (vector):")
print(result2)
#'

#'
# 예시 데이터
data <- data.frame(
  Question = c("나는 나의 세 가지 가장 큰 약점을 열거할 수 있다.",
   "나의 행동은 나의 핵심 가치를 반영한다.",
   "나는 스스로 결정하기 전에 다른 사람들의 의견을 구한다."),
  factor = c("자기", "자기", "자기"),
  stringsAsFactors = FALSE
)

data
replace_df_rep(data, "약점", "강점","행동","수행",
"스스로","다른 사람이",  detect = TRUE)
replace_df_rep(data, "약점",
"강점","행동","수행",  detect = FALSE)



## End(Not run)
```

`replace_in_dataframe` *replace_in_dataframe*

Description

replace_in_dataframe

Usage

```
replace_in_dataframe(data, changes)
```

Arguments

`data`	data
`changes`	change

Value

data

`replace_in_df` *replace_in_df*

Description

replace_in_df

Usage

```
replace_in_df(data, pattern, imp)
```

Arguments

`data`	data
`pattern`	pattern
`imp`	""

Value

data

Examples

```
## Not run:
#'
data <- data.frame(
  Question = c("나는 나의 세 가지 가장 큰
  약점을 열거할 수 있다.", "나의 행동은 나의
  핵심 가치를 반영한다.", "나는 스스로 결정하기
  전에 다른 사람들의 의견을 구한다."),
  factor = c("자기", "자기", "자기"),
  stringsAsFactors = FALSE
)

data
data %>% replace_in_df("약점", "강점")

## End(Not run)
```

`Replace_na` *Create a function to replace NA with an empty string in an R vector data*

Description

Create a function to replace NA with an empty string in an R vector data

Usage

```
Replace_na(vector, pattern = NA, imp = "")
```

Arguments

vector	input data
pattern	dectect pattern
imp	imputation

Value

imputation data

Examples

```
## Not run:
# Replace(c("A", NA, "B", "", "C", "C"), "C", imp="T")

## End(Not run)
```

---

report_aov *anova report ,report_aov*

Description

anova report ,report_aov

Usage

```
report_aov(data, digits = 2, trans = FALSE)
```

Arguments

data	aov data
digits	default 3
trans	TRUE english to korea

Value

report

Examples

```
## Not run:
aov(len ~ supp*factor(dose), data= ToothGrowth) %>% report_aov()
aov(len ~ supp*factor(dose), data= ToothGrowth) %>% report_aov(trans=TRUE)



## End(Not run)
```

report_aov_ko *repot anova*

Description

repot anova

Usage

```
report_aov_ko(data, digits = 2, print = TRUE)
```

Arguments

data	aov data
digits	3
print	table show

Value

table

Examples

```
## Not run:
aov(mpg ~ cyl* gear +carb, data=mtcars) %>%report_aov_ko()

aov(mpg ~ cyl* gear +carb, data=mtcars) %>%report_aov_ko(print=T)
aov(mpg ~ cyl* gear +carb, data=mtcars) %>%report_aov_ko(print=F)


## End(Not run)
```

report_lme *linear mixed model report*

Description

Mixed model summary

Usage

```
report_lme(
  lmedata,
  type = "basic",
  form = "lmer",
  apa = FALSE,
  fit_more = FALSE,
  ranef_sig = FALSE,
  show.effect = FALSE,
  show.ci = FALSE
)
```

Arguments

lmedata	lmedata is lmer() function result
type	= 'all' is res, 'Fixed_effect','Random_effect','ICC', 'ConfidenceInterval_95','Satterthwaite_r
form	'lmer','glmer' = 'logit'='dich', 'poisson' ='pois'
apa	= default FALSE, if you REML is FALSE
fit_more	= default FALSE, TRUE detailed report
ranef_sig	ranef_sig =TRUE random effect test
show.effect	if you show.effect = TRUE -> show ranef, fixef, coef
show.ci	TRUE calculation 95percent CI

Examples

```
## Not run:
data(sleepstudy)

sleepstudy |> str()

lmer(Reaction ~ Days + (Days | Subject), sleepstudy) |> lme_report()

lmer(Reaction ~ Days + (Days | Subject), sleepstudy) |> lme_report(apa=TRUE)


book.url <- "https://stat.ethz.ch/~meier/teaching/book-anova"
quality <- readRDS(url(file.path(book.url, "data/quality.rds")))
str(quality)
fit.quality <- lmer(score ~ (1 | employee) + (1 | batch) +
                      (1 | employee:batch), data = quality)
summary(fit.quality)
fit.quality|> lme_report()
fit.quality|> lme_report(apa= TRUE) #error the app is FALSE
fit.quality|> lme_report(apa=FALSE) #error the app is FALSE
fit.quality|> ICC()

lmer(Reaction ~ Days + (Days | Subject), sleepstudy) |>
SEM212::lme_report(apa=TRUE, fit_more = T)
lmer(Reaction ~ Days + (Days | Subject), sleepstudy) |>
SEM212::lme_report(apa=TRUE, fit_more = F)  #default FALSE, it is show that AIC and BIC


## End(Not run)
```

---

report_manova *Functions to report MANOVA results*

Description

Functions to report MANOVA results

Usage

```
report_manova(model_manova, type = "res", digits = 2)
```

Arguments

`model_manova`	manova result
`type`	default res, and all, summary_aov, Pillai, Wilks, Hotelling_Lawley, Roy, Multivariate_Tests
`digits`	default 2

Value

report MANOVA table

Examples

```
## Not run:

Skulls = heplots::data(Skulls)

# execute
manova(cbind(mb,bh,bl,nh)~ epoch, data=Skulls) %>%report_manova()

manova(cbind(mb,bh,bl,nh)~ epoch, data=Skulls) %>%report_manova(type="all")

manova(cbind(mb,bh,bl,nh)~ epoch, data=Skulls) %>%report_manova(type="summmary_aov")

manova(cbind(mb,bh,bl,nh)~ epoch, data=Skulls) %>%report_manova(type="Pillai")

manova(cbind(mb,bh,bl,nh)~ epoch, data=Skulls) %>%report_manova(type="Wilks")

manova(cbind(mb,bh,bl,nh)~ epoch, data=Skulls) %>%report_manova(type="Hotelling_Lawley")

manova(cbind(mb,bh,bl,nh)~ epoch, data=Skulls) %>%report_manova(type="Roy")

manova(cbind(mb,bh,bl,nh)~ epoch, data=Skulls) %>%report_manova(type="Multivariate_Tests")



Y <- cbind.data.frame(tear=c(6.5, 6.2, 5.8, 6.5, 6.5, 6.9, 7.2, 6.9, 6.1, 6.3,
                             6.7, 6.6, 7.2, 7.1, 6.8, 7.1, 7.0, 7.2, 7.5, 7.6),
                      gloss=c(9.5, 9.9, 9.6, 9.6, 9.2, 9.1, 10.0, 9.9, 9.5, 9.4,
          9.1, 9.3, 8.3, 8.4, 8.5, 9.2, 8.8, 9.7, 10.1, 9.2),
          opacity=c(4.4, 6.4, 3.0, 4.1, 0.8, 5.7, 2.0, 3.9, 1.9, 5.7,
                    2.8, 4.1, 3.8, 1.6, 3.4, 8.4, 5.2, 6.9, 2.7, 1.9),
          rate = gl(2,10, labels = c("Low", "High")),
          additive = gl(2, 5, length = 20, labels = c("Low", "High")) )

manova(cbind(tear, gloss, opacity) ~ rate * additive, data=Y)%>%report_manova()

manova(cbind(tear, gloss, opacity) ~ rate * additive, data=Y)%>%report_manova(type="all")
manova(cbind(tear, gloss, opacity) ~ rate * additive, data=Y)%>%
                           report_manova(type="summmary_aov")
manova(cbind(tear, gloss, opacity) ~ rate * additive, data=Y) %>%
  report_manova("Pillai")
manova(cbind(tear, gloss, opacity) ~ rate * additive, data=Y)%>%
  report_manova("Wilks")
manova(cbind(tear, gloss, opacity) ~ rate * additive, data=Y)%>%
  report_manova("Hotelling_Lawley")
manova(cbind(tear, gloss, opacity) ~ rate * additive, data=Y)%>%
```

```
  report_manova("Roy")
manova(cbind(tear, gloss, opacity) ~ rate * additive, data=Y)%>%
  report_manova("Multivariate_Tests")


## End(Not run)
```

`report_stat` *report nomality , t.test , var.test*

Description

report nomality , t.test , var.test

Usage

```
report_stat(data, type = "normality", digits = 3, trans = FALSE, show = "data")
```

Arguments

`data`	test result
`type`	normality, t.test, var.test(leven)
`digits`	default 3
`trans`	translate TRUE korean language
`show`	normal is source and output, data is output

Value

report data

Examples

```
## Not run:

shapiro.test(final) %>% report_stat()

var.test(mpg ~ vs, data = mtcars)%>%
report_stat("var.test")

car::leveneTest(mpg~ factor(vs), mtcars)%>%
 report_stat("leven.test")

car::leveneTest(mpg~ factor(vs), mtcars)%>%
report_stat("leven.test", trans=TRUE)


fligner.test(mpg ~ vs,data= mtcars) %>%
report_stat("t.test")

t.test(mpg ~ vs, data= mtcars, var.equal = FALSE)%>%
report_stat("t.test")
```

```
t.test(len ~ supp, data = ToothGrowth, var.equal=TRUE) %>%
report_stat("t.test")

shapiro.test(final) %>% report_stat()%>% jjstat::k("en","ko")
shapiro.test(final) %>% report_stat(trans =TRUE)

car::leveneTest(mpg~ factor(vs), mtcars)%>%
report_stat("leven.test", trans=TRUE)



## End(Not run)
```

`report_ttest` *report_ttest*

Description

report_ttest

Usage

```
report_ttest(ttest_result, trans = FALSE, type = "report")
```

Arguments

`ttest_result`	t.test result
`trans`	true english to korean
`type`	default 'report', 'all' is t.table and APA

Value

APA report

Examples

```
## Not run:
## english report
t.test(mpg ~ vs, mtcars, var.equal=TRUE) %>%
        report_ttest()

## translation
t.test(mpg ~ vs, mtcars, var.equal=TRUE) %>%
report_ttest(trans=T)

t.test(mpg ~ vs, mtcars) %>% report_ttest("all")

## End(Not run)
```

`report_vartest` *var.test report*

Description

var.test report

Usage

```
report_vartest(var_test, trans = FALSE)
```

Arguments

`var_test`	var.test result
`trans`	tans true english to korean

Value

explain result apa

Examples

```
## Not run:
var.test(mpg ~ vs, data = mtcars)%>%
     report_vartest()

var.test(mpg ~ vs, data = mtcars)%>%
     report_vartest() %>%
     jjstat::kakaoi()

var.test(mpg ~ vs, data = mtcars)%>%
report_vartest(trans=TRUE)


## End(Not run)
```

`revcode` *Custom function to perform reverse coding for multiple variables with custom n values*

Description

Custom function to perform reverse coding for multiple variables with custom n values

Usage

```
revcode(data, n = 8, ..., postfix = "_r", replace = FALSE)
```

Arguments

data	Data frame containing the variables
n	Vector of maximum scale values (one for each va c("v1","v2",...)
...	Variable names to be reverse coded
postfix	postfix is reverse variable name default '_r'
replace	How to enter and replace the reverse coding results

Value

add reverse data in data.frame

Examples

```
## Not run:

### How to decode all variables by applying only one n
revcode(stat_onl, 8, "upgrade", "satisfy", "fatigue" )|>
select(upgrade, satisfy, fatigue, upgrade_r, satisfy_r, fatigue_r)

### the method for the reverse coding variable being applied separately
### to each variable and decoding
revcode(stat_onl, n= c(8,8,8), "upgrade", "satisfy", "fatigue" )|>
select(upgrade, satisfy, fatigue, upgrade_r, satisfy_r, fatigue_r)

## mtcars data
## Since the inverse coding value of the cyl variable is 4,6,8,
## n enters 4 + 8 = 12; the minimum + maximum = n value.
revcode(mtcars[,c("cyl", "gear","carb")], 12, "cyl")


revcode(mtcars[,c("cyl", "gear","carb")],
            n = c(12, 6, 9),"cyl","gear","carb")
#'
df <- data.frame(
  A = c(1, 2, 3, 4, 5),
  B = c(5, 4, 3, 2, 1)
)

# Passing variable names as symbols
df_new1 <- revcode(df, n = 6, A, B)
print(df_new1)
str(df_new1)
print(attr(df_new1$A_r, "reverse")) # Output properties for verification
print(attr(df_new1$B_r, "reverse"))

# pass the variable name as a string
df_new2 <- revcode(df, n = 6, "A", "B")
print(df_new2)
str(df_new2)
print(attr(df_new2$A_r, "reverse"))  # Output properties for verification
print(attr(df_new2$B_r, "reverse"))

# Reverse-code to replace original column
df_replaced <- revcode(df, n = 6, A, B, replace = TRUE)
```

```
print(df_replaced)
str(df_replaced)
print(attr(df_replaced$A, "reverse"))  # Output properties for verification
print(attr(df_replaced$B, "reverse"))

#'


## End(Not run)
```

`rev_coding` *Reverse Coding for Likert Scale Items*

Description

This function performs reverse coding on a numeric vector, commonly used in Likert scale data. The function assumes the Likert scale ranges from 1 to (n - 1), where ‘n‘ is typically 6 for a 5-point scale.

Usage

```
rev_coding(vec, n = 6, check = FALSE)
```

Arguments

`vec`	Numeric vector to be reverse-coded.
`n`	Integer value indicating the maximum Likert scale value plus 1 (e.g., use ‘n = 6‘ for 1 to 5 scale).
`check`	Logical; if TRUE, returns a data frame showing the original and reverse-coded values side-by-side.

Value

A reverse-coded vector or a data frame if ‘check = TRUE‘. Returns an error message if input is not a vector or values exceed the expected maximum.

Examples

```
# Example 1: Basic reverse coding (Likert 1~5, n=6)
x <- c(1, 2, 3, 4, 5)
rev_coding(x)

# Example 2: View original and reverse-coded values
rev_coding(x, check = TRUE)

# Example 3: 7-point Likert scale (1~7), use n = 8
x7 <- c(1, 3, 5, 7)
rev_coding(x7, n = 8)

# Example 4: Using data from mtcars - reverse code 'gear' variable
rev_coding(mtcars$gear, n = 6)

# Example 5: Check = TRUE for inspection
```

```
rev_coding(mtcars$gear, check = TRUE)

# Example 6: Using 'cyl' column from mtcars (values 4, 6, 8), Likert scale mismatch
rev_coding(mtcars$cyl, n = 6)

# Example 7: Using 'Sepal.Length' from iris (continuous values), inappropriate for Likert
rev_coding(iris$Sepal.Length, n = 6)

# Example 8: Using ToothGrowth dataset - 'dose' column
rev_coding(ToothGrowth$dose, n = 6)

# Example 9: Inputting a data frame (error case)
rev_coding(mtcars)

# Example 10: Reverse coding with missing values
x_miss <- c(1, 2, NA, 4, 5)
rev_coding(x_miss, n = 6)

# Example 11: Reverse coding with missing values and check = TRUE
rev_coding(x_miss, n = 6, check = TRUE)
```

---

rev_coding_extract_only

Extract only reverse-coded variables from a data.frame

Description

Extract only reverse-coded variables from a data.frame

Usage

```
rev_coding_extract_only(data, drop_attr = FALSE)
```

Arguments

`data`	A data.frame of class 'revcoded_df'
`drop_attr`	Logical. If TRUE, removes attributes and classes (default = FALSE)

Value

A data.frame containing only reverse-coded variables

Examples

```
df <- data.frame(A = 1:5, B = 5:1)
res <- rev_coding_rep(df, 6, A, B)
rev_coding_extract_only(res)
```

rev_coding_map *Create a mapping table between original and reverse-coded variables*

Description

Create a mapping table between original and reverse-coded variables

Usage

```
rev_coding_map(data)
```

Arguments

data	A data.frame of class 'revcoded_df'

Value

A data.frame with original and reverse-coded variable name pairs

Examples

```
df <- data.frame(A = 1:5, B = 5:1)
res <- rev_coding_rep(df, 6, A, B)
rev_coding_map(res)
```

rev_coding_plot_var *Plot reverse-coded variable against original values*

Description

Displays a comparison between the original and reverse-coded values. Works only when original variable is still available in parent environment or in the given data.

Usage

```
rev_coding_plot_var(x, data = NULL, ...)
```

Arguments

x	A reverse-coded vector with class 'reverse_var'
data	Optional: original data.frame containing the original variable
...	Additional graphical arguments passed to plot

Value

A scatter plot of original vs. reverse-coded values

Examples

```
## Not run:
df <- data.frame(A = 1:5)
res <- rev_coding_rep(df, 6, A)
rev_coding_plot_var(res$A_r, data = df)

## End(Not run)
```

`rev_coding_rep` *Reverse Coding for Multiple Variables with S3 Methods*

Description

Reverse-codes specified variables in a data frame by subtracting values from a given maximum scale value. Supports symbol and character input for variable names. Adds S3 classes for customized print and summary behavior.

Usage

```
rev_coding_rep(data, n = 8, ..., postfix = "_r", replace = FALSE)
```

Arguments

`data`	A data.frame containing variables to be reverse coded.
`n`	A numeric scalar or numeric vector specifying maximum scale values. If length 1, the value is recycled for all variables.
`...`	Variables to be reverse coded. Can be symbols or character strings.
`postfix`	Character string appended to new variable names. Default is "_r".
`replace`	Logical. If TRUE, replaces original variables.

Value

A data.frame of class 'revcoded_df'. Reverse-coded variables have class 'reverse_var'.

Examples

```
## Not run:
##############################################
# 0. 기본 데이터 생성
##############################################
df <- data.frame(
  A = c(1, 2, 3, 4, 5),
  B = c(5, 4, 3, 2, 1),
  C = c(2, 3, 3, 4, 5)
)

##############################################
# 1. 기본 사용: 하나의 n 값으로 여러 변수 역코딩
##############################################
res1 <- rev_coding_rep(df, n = 6, A, B)
print(res1)
summary(res1$A_r)
```

```
##############################################
# 2. 문자열로 변수 이름 전달
##############################################
res2 <- rev_coding_rep(df, n = 6, "A", "B")
print(res2)

##############################################
# 3. replace = TRUE : 기존 변수 덮어쓰기
##############################################
res3 <- rev_coding_rep(df, n = 6, A, B, replace = TRUE)
print(res3)
summary(res3$A)

##############################################
# 4. 변수별로 다른 n 값 적용
##############################################
res4 <- rev_coding_rep(df, n = c(6, 5, 7), A, B, C)
print(res4)

##############################################
# 5. postfix 변경
##############################################
res5 <- rev_coding_rep(df, n = 6, A, B, postfix = "_inv")
names(res5)

##############################################
# 6. NA 값이 포함된 데이터
##############################################
df_na <- data.frame(
  Q1 = c(1, NA, 3, 5),
  Q2 = c(5, 4, NA, 2)
)
res6 <- rev_coding_rep(df_na, n = 6, Q1, Q2)
print(res6)
summary(res6$Q1_r)

##############################################
# 7. mtcars 예제 (범주형처럼 쓰이는 수치형 변수)
##############################################
# cyl = {4,6,8} → min + max = 12
res7 <- rev_coding_rep(
  mtcars[, c("cyl", "gear", "carb")],
  n = c(12, 6, 9),
  "cyl", "gear", "carb"
)
head(res7)

##############################################
# 8. tidyverse 파이프라인에서 사용
##############################################
library(dplyr)
df %>%
  rev_coding_rep(n = 6, A, B) %>%
  select(A_r, B_r) %>%
  summary()
```

```
##########################################
# 9. reverse-coded 변수만 자동 추출
##########################################
get_reverse_vars <- function(x) {
  names(x)[sapply(x, function(col) inherits(col, "reverse_var"))]
}
res9 <- rev_coding_rep(df, n = 6, A, B)
get_reverse_vars(res9)

##########################################
# 10. 심리척도 예제 (Likert scale)
##########################################
likert <- data.frame(
  stress1 = c(1,2,3,4,5),
  stress2 = c(2,3,4,4,5),
  stress3 = c(5,4,3,2,1)
)
res10 <- rev_coding_rep(likert, n = 6, stress3)
print(res10)

##########################################
# 11. 실수 방지 예제 (factor 주의)
##########################################
df_factor <- data.frame(Q1 = factor(c(1,2,3)))
# rev_coding_rep(df_factor, n = 6, Q1)  # 권장하지 않음

## End(Not run)
```

`rev_coding_restore_original` *Remove reverse-coded variables, keeping only original variables*

Description

Remove reverse-coded variables, keeping only original variables

Usage

```
rev_coding_restore_original(data)
```

Arguments

`data`	A data.frame of class 'revcoded_df'

Value

A data.frame without reverse-coded variables

Examples

```
df <- data.frame(A = 1:5, B = 5:1)
res <- rev_coding_rep(df, 6, A, B)
rev_coding_restore_original(res)
```

rev_coding_validate_scale

Validate reverse coding scale (n): scale 유효성검사

Description

Checks if the provided max scale (n) is valid for each variable. A valid n is usually: min(x) + max(x)

Usage

```
rev_coding_validate_scale(data, vars, n, strict = FALSE)
```

Arguments

data	A data.frame
vars	Character vector or symbols: variable names
n	Vector of expected scale maxima (same length as vars or length 1)
strict	Logical. If TRUE, gives warning when mismatch found

Value

A data.frame with columns: var, min, max, expected_n, provided_n, match

Examples

```
df <- data.frame(A = 1:5, B = 5:1)
rev_coding_validate_scale(df, c("A", "B"), n = 6)
rev_coding_validate_scale(df, c("A", "B"), n = c(6, 6))
```

Rload

Rload

Description

Rload

Usage

```
Rload(file, verbose = TRUE)
```

Arguments

file	position
verbose	default TRUR

Value

datafile

Examples

```
## Not run:
#'
#'
#'
Rload("pwj2.RData", verbose = TRUE)
pwj2 %>%head()
Rload(file="F:/Rwork/02_OutAna/Parkwonjun_yonsei_2024/pwj2.RData")

## End(Not run)
```

`Round` *Find and round numeric variables only*

Description

Find and round numeric variables only

Usage

```
Round(data, digits = 3, type = "tibble", exclude = NULL)
```

Arguments

`data`	data.frame
`digits`	default 2
`type`	data type tibble or data.frame , matrix
`exclude`	exclude variable not adapted

Value

rounding data

Examples

```
## Not run:

# vector
Round(12.345654)

##data.frame
mtcars%>% Round(1)

 ##
dff = data.frame(A = c(1.23456, 2.34567, 3.45678),
                 B = c(4.56789, 5.67890, 6.78901),
                C = c("a", "b", "c"))
 ## rounding 1
 dff%>%Round(1)
```

```
#exclue variable B
 dff%>%Round(1, exclude ="B")


## End(Not run)
```

---

round2 *numeric round*

Description

numeric round

Usage

```
round2(data, digits = 2)
```

Arguments

data	data
digits	2

Value

data.frame

Examples

```
## Not run:

dataset = data.frame(
var = c("mpg", "cyl", "disp", "hp",
"drat", "wt", "qsec", "vs", "am",
"gear", "carb"),
N = c(32, 32, 32, 32, 32, 32, 32,
 32, 32, 32, 32),
MEAN = c(20.090625, 6.1875, 230.721875,
 146.6875, 3.5965625, 3.21725, 17.84875,
 0.4375, 0.40625, 3.6875, 2.8125),
SD = c(6.0269480520891, 1.78592164694654,
 123.938693831382, 68.5628684893206,
  0.534678736070971, 0.978457442989697,
  1.78694323609684, 0.504016128774185,
   0.498990917235846, 0.737804065256947,
    1.61519997763185),
MIN = c(10.4, 4, 71.1, 52, 2.76,
1.513, 14.5, 0, 0, 3, 1),
MAX = c(33.9, 8, 472, 335, 4.93,
5.424, 22.9, 1, 1, 5, 8),
Skew = c(0.610655017573288, -0.174611913930169,
 0.381657034147599, 0.726023656361273,
 0.265903904603492, 0.423146464177225,
 0.369045278509436, 0.240257688369044,
```

```
  0.364015894329692, 0.52885446231085, 1.05087376449851),
Kurt = c(-0.372766029820891, -1.76211977398177,
 -1.2072119466699, -0.135551121097421,
 -0.714700615734489, -0.0227107528393127,
 0.335114219571393, -2.00193762400794,
 -1.92474142839069, -1.06975067515432,
  1.25704307347743))
dataset
dataset%>% round2()



## End(Not run)
```

`round_col` *round_col,* 한개의열만 *round* 처리

Description

round_col, 한개의열만 round 처리

Usage

```
round_col(df, col = ncol(df), digits = 3, scientific = FALSE)
```

Arguments

`df`	df
`col`	select col , default last column
`digits`	default 3

Value

data

`row2col` *rownames to column*

Description

rownames to column

Usage

```
row2col(data, colname = "rownames")
```

Arguments

`data`	data.fram
`colname`	new column default rownames

Value

data

Examples

```
## Not run:
mtcars %>%row2col("car")

## End(Not run)
```

---

row2cols *rownames to columns*

Description

rownames to columns

Usage

```
row2cols(data, colname = "rows")
```

Arguments

data	data.fram
colname	new column default rownames

Value

data

Examples

```
## Not run:
mtcars %>%row2cols("car")

## End(Not run)
```

`rowdata2col` *Select from ROW to make it a column name*

Description

Select from ROW to make it a column name

Usage

```
rowdata2col(data, row)
```

Arguments

`data`	data
`row`	select row

Value

data

Examples

```
## Not run:

#'
# 예제 데이터 생성
data <- data.frame(
  X1 = c(1.000000, 13.437500, 5.464659, 16.000000),
  X2 = c(2.000000, 11.800000, 4.813671, 15.000000),
  X3 = c(3.000000, 12.473684, 4.032761, 19.000000)
)
rownames(data) <- c("var", "Mean", "SD", "N")
# 함수 사용 예시
result <- rowdata2col(data, "var")
print(result)



## End(Not run)
```

`rownames2colnames` *rownames2colnames*

Description

rownames2colnames

Usage

```
rownames2colnames(df, reverse = FALSE)
```

Arguments

df	df
reverse	reverse = FALSE

Value

df

Examples

```
## Not run:
#'

# 예시 데이터프레임
df <- data.frame(a = 1:3, b = 4:6)
rownames(df) <- c("row1", "row2", "row3")

# rownames를 colnames로 변환
new_df <- rownames2colnames(df)
print(new_df)

# colnames를 rownames로 변환
df_with_colnames_as_rownames <- rownames2colnames(new_df, reverse = TRUE)
print(df_with_colnames_as_rownames)

## End(Not run)
```

---

rows_to_colnames *Convert a Row into Column Names of a Data Frame*

Description

This function sets the values of a specified row in a data frame as the new column names. The row used for column names is then removed from the data.

Usage

```
rows_to_colnames(df, row = 1)
```

Arguments

df	A data frame. The input data where one row will be promoted to column names.
row	An integer indicating the row number to use as column names. Defaults to 1.

Value

A data frame with updated column names and the specified row removed.

Examples

```
# Create a simple data frame
df <- data.frame(
  V1 = c("Name", "Alice", "Bob"),
  V2 = c("Age", 25, 30),
  V3 = c("Gender", "F", "M"),
  stringsAsFactors = FALSE
)

# Convert the first row to column names
new_df <- rows_to_colnames(df)
print(new_df)

# Use the second row instead
df2 <- data.frame(
  A = c("Header1", "X", "Y"),
  B = c("Header2", 1, 2),
  stringsAsFactors = FALSE
)
rows_to_colnames(df2, row = 2)
```

---

row_filter *select rows filter function*

Description

select rows filter function

Usage

```
row_filter(data, ...)
```

Arguments

data	data
...	row condition

Value

data

Examples

```
## Not run:
#'
mtcars %>% row_filter(cyl == 6 )
mtcars %>% row_filter(vs == 1)
mtcars %>% row_filter(cyl == 6 | vs == 1)
mtcars %>% row_filter(cyl == 6 & vs == 1)

## End(Not run)
```

Rsave *RData fast save*

Description

RData fast save

Usage

```
Rsave(df, file = "")
```

Arguments

df	data to save
file	filename

Value

data

Examples

```
## Not run:

Rsave(Kge_jbs)           # 기본 파일 이름: Kge_jbs.RData
Rsave(Kge_jbs, file = "my_custom_name")  # 사용자 지정 파일 이름: my_custom_name.RData

## End(Not run)
```

Rsave_xlsx *Save Data to Excel File*

Description

This function saves a given data frame to an Excel file in the current working directory. The file is named after the variable name of the input data.

Usage

```
Rsave_xlsx(data)
```

Arguments

data	A data frame to be saved as an Excel file.

Value

None. The function saves the data to a file and prints the save path and a reload command.

Examples

```
## Not run:
# Example: Save a data frame to an Excel file
df <- data.frame(
  Name = c("Alice", "Bob", "Charlie"),
  Age = c(25, 30, 35)
)
Rsave_xlsx(df)

# The file "df.xlsx" will be saved in the current working directory.
# Reload the saved file with:
# df <- openxlsx::read.xlsx(file = "df.xlsx")

## End(Not run)
```

---

save_xlsx *Save Data to Excel File*

Description

This function saves a given data frame to an Excel file in the current working directory. The file is named after the variable name of the input data.

Usage

```
save_xlsx(data)
```

Arguments

data	A data frame to be saved as an Excel file.

Value

None. The function saves the data to a file and prints the save path and a reload command.

Examples

```
## Not run:
# Example: Save a data frame to an Excel file
df <- data.frame(
  Name = c("Alice", "Bob", "Charlie"),
  Age = c(25, 30, 35)
)
Rsave_xlsx(df)

# The file "df.xlsx" will be saved in the current working directory.
# Reload the saved file with:
# df <- openxlsx::read.xlsx(file = "df.xlsx")

## End(Not run)
```

`select_format` *select_format*

Description

select_format

Usage

```
select_format(data, ..., nsmall = 3, digits = 3)
```

Arguments

`data`	data
`...`	col selection
`nsmall`	default 3
`digits`	round 3

Value

data

Examples

```
## Not run:


# Usage example
data2 <- data.frame(a = rnorm(5), b = rnorm(5), c = rnorm(5), d = rnorm(5), e = rnorm(5))
select_format(data2, 1, 2)
select_format(data2, 1:5)
select_format(data2, a)
select_format(data2, a:e)
select_format(data2, "a")

## End(Not run)
```

`SEM` *Title*

Description

Title

Usage

```
SEM(
  model = NULL,
  data = NULL,
  type = "res",
  ind = TRUE,
  cat = FALSE,
  interact = TRUE,
  auto = TRUE,
  n_name = 1,
  prefix = "a",
  paths_name = TRUE,
  add_model = NULL,
  ordered = NULL,
  sampling.weights = NULL,
  sample.cov = NULL,
  sample.mean = NULL,
  sample.th = NULL,
  sample.nobs = NULL,
  group = NULL,
  cluster = NULL,
  constraints = "",
  WLS.V = NULL,
  NACOV = NULL,
  ov.order = "model",
  ...
)
```

Arguments

`model`	A description of the user-specified model. Typically, the model is described using the lavaan model syntax. See model.syntax for more information. Alternatively, a parameter table (eg. the output of the lavaanify() function) is also accepted.
`data`	An optional data frame containing the observed variables used in the model. If some variables are declared as ordered factors, lavaan will treat them as ordinal variables.
`type`	res, fit, New_model, New_data, imlist
`ind`	Automatically Adding Indirect Effects
`cat`	output syntax default FALSE
`interact`	interaction term extract possinle TRUE
`auto`	auto genrate indirect path
`n_name`	indirect name number n_name =1
`prefix`	prefix "a" –"H" change possible
`paths_name`	inde path name default TRUE
`add_model`	add model syntax
`ordered`	Character vector. Only used if the data is in a data.frame. Treat these variables as ordered (ordinal) variables, if they are endogenous in the model. Importantly, all other variables will be treated as numeric (unless

they are declared as ordered in the data.frame.) Since 0.6-4, ordered can also be logical. If TRUE, all observed endogenous variables are #' treated as ordered (ordinal). If FALSE, all observed endogenous variables are considered to be numeric (again, unless they are declared as ordered in the data.frame.)

`sampling.weights`
: A variable name in the data frame containing sampling weight information. Currently only available for non-clustered data. Depending on the sampling.weights.normalization option, these weights may be rescaled (or not) so that their sum equals the number of observations (total or per group).

`sample.cov` Numeric matrix. A sample variance-covariance matrix. The rownames and/or colnames must contain the observed variable names. For a multiple group analysis, a list with a variance-covariance matrix for each group.

`sample.mean` A sample mean vector. For a multiple group analysis, a list with a mean vector for each group.

`sample.th` Vector of sample-based thresholds. For a multiple group analysis, a list with a vector of thresholds for each group.

`sample.nobs` Number of observations if the full data frame is missing and only sample moments are given. For a multiple group analysis, a list or a vector with the number of observations for each group.

`group` Character. A variable name in the data frame defining the groups in a multiple group analysis.

`cluster` Character. A (single) variable name in the data frame defining the clusters in a two-level dataset.

`constraints` Additional (in)equality constraints not yet included in the model syntax. See model.syntax for more information.

`WLS.V` A user provided weight matrix to be used by estimator "WLS"; if the estimator is "DWLS", only the diagonal of this matrix will be used. For a multiple group analysis, a list with a weight matrix for each group. The elements of the weight matrix should be in the following order (if all data is continuous): first the means (if a meanstructure is involved), then the lower triangular elements of the covariance matrix including the diagonal, ordered column by column. In the categorical case: first the thresholds (including the means for continuous variables), then the slopes (if any), the variances of continuous variables (if any), and finally the lower triangular elements of the correlation/covariance matrix excluding the diagonal, ordered column by column.

`NACOV` A user provided matrix containing the elements of (N times) the asymptotic variance-covariance matrix of the sample statistics. For a multiple group analysis, a list with an asymptotic variance-covariance matrix for each group. See the WLS.V argument for information about the order of the elements.

`ov.order` Character. If "model" (the default), the order of the observed variable names (as reflected for example in the output of lavNames()) is determined by the model syntax. If "data", the order is determined by the data (either the full data.frame or the sample (co)variance matrix). If the WLS.V and/or NACOV matrices are provided, this argument is currently set to "data".

... Many more additional options can be defined, using 'name = value'. See lavOptions for a complete list.

Value

data

Examples

```
## Not run:
#'
## The industrialization and Political Democracy Example
## Bollen (1989), page 332
model <- '
  # latent variable definitions
     ind60 =~ x1 + x2 + x3
     dem60 =~ y1 + a*y2 + b*y3 + c*y4
     dem65 =~ y5 + a*y6 + b*y7 + c*y8

  # regressions
    dem60 ~ ind60
    dem65 ~ ind60 + dem60

  # residual correlations
    y1 ~~ y5
    y2 ~~ y4 + y6
    y3 ~~ y7
    y4 ~~ y8
    y6 ~~ y8
'

fit <- sem(model, data = PoliticalDemocracy)
summary(fit, fit.measures = TRUE)
#'
#'

# # Example usage:
data(MarshWenHau)
str(MarshWenHau)

mwh = MarshWenHau

model2 <- "
f1 =~ x1 + x2 + x3
f1Xf2 =~ f1:f2
f2 =~ x4 + x5 + x6
f3 =~ y1 + y2 + y3

f3 ~ f1 + f2
f3 ~ f1Xf2

f1Xf2 ~~ 0*f1
f1Xf2 ~~ 0*f2
"

# interaction model
```

```
intsem =  SEM(model2, data= MarshWenHau)
summary(intsem)
intsem %>%lav_semPaths2(color=list(lat="darkred",man="gold"))


##2nd---------
#base model
lav_extract_mm(model2) %>%cat()
#interaction item list
lav_extract_imlist(model3)
#data
lav_latentProd(mwh, model3)%>%head()

#extracted interaction term
lav_extract_int(model2, lav_extract_imlist(model3))

# or interaction items ; imlist
imlist <- list(
  .f1Xf2 = c("f1", "f2"),
  .i_f1_f4 = c("f1", "f4")
)
# names(imlist)
lav_extract_int(model2, imlist)
#direct
lav_extract_int(model2, imlist= list(.f1Xf2 = c("f1", "f2"),
                                     .i_f1_f4 = c("f1", "f4")))
#estimates model
intsem =  SEM(lav_new_model(model2, lav_extract_imlist(model2)),
              data= lav_latentProd(mwh, model2))

#'

## End(Not run)
```

`semPaths2` *semPlot::semPaths wrapper with customization for grouped or non-grouped path diagrams*

Description

A convenient wrapper around 'semPlot::semPaths' with additional options for layout, labels, grouping, and significance marking using 'semptools::mark_sig'.

Usage

```
semPaths2(
  lav_obj,
  whatLabels = "est",
  rotation = 2,
  nDigits = 2,
  residScale = 18,
  mar = c(2, 8, 3, 8),
```

```
  edge.label.cex = 1,
  edge.label.position = 0.5,
  sizeMan = 6,
  sizeMan2 = 4,
  sizeLat = 8,
  sizeLat2 = 6,
  border.width = 2,
  edge.width = 1.5,
  style = "lisrel",
  sig = TRUE,
  asize = 1.5,
  shapeLat = "circle",
  residuals = FALSE,
  curve = 1,
  layout = "tree",
  exoVar = TRUE,
  exoCov = TRUE,
  structual = FALSE,
  nodeLabels = NULL,
  groups = FALSE
)
```

Arguments

`lav_obj`	A fitted lavaan object.
`whatLabels`	Type of labels to display on edges (e.g., '"est"' for estimates).
`rotation`	Rotation angle for layout.
`nDigits`	Number of digits to round estimates.
`residScale`	Scaling factor for residuals.
`mar`	Margins of the plot.
`edge.label.cex`	Character expansion factor for edge labels.
`edge.label.position`	Position of edge labels along the edge.
`sizeMan`	Size of manifest variables.
`sizeMan2`	Size of exogenous manifest variables.
`sizeLat`	Size of latent variables.
`sizeLat2`	Size of exogenous latent variables.
`border.width`	Border width of nodes.
`edge.width`	Line width of edges.
`style`	Style of path diagram (e.g., '"lisrel"').
`sig`	Logical. If TRUE, significant paths are marked.
`asize`	Arrow size.
`shapeLat`	Shape of latent variables (e.g., '"circle"').
`residuals`	Logical. If TRUE, show residuals.
`curve`	Curvature of edges.
`layout`	Layout of the graph (e.g., '"tree"', '"circle"').

exoVar	Logical. If TRUE, include exogenous variables.
exoCov	Logical. If TRUE, include covariances of exogenous variables.
structual	Logical. If TRUE, plot structural model only.
nodeLabels	Optional custom node labels.
groups	Logical. If TRUE, color nodes by latent groupings.

Value

A path diagram object, optionally plotted if 'sig = TRUE'.

Examples

```
## Not run:
library(lavaan)
library(semPlot)
library(semptools)

model <- '
  visual  =~ x1 + x2 + x3
  textual =~ x4 + x5 + x6
  speed   =~ x7 + x8 + x9
'
fit <- cfa(model, data = HolzingerSwineford1939)

semPaths2(fit)
semPaths2(fit, sig = FALSE)

## End(Not run)
```

---

sem_apa *SEM direct effect interpretation*

Description

SEM direct effect interpretation

Usage

```
sem_apa(data, md = FALSE, caption = "Table: ", print = TRUE)
```

Arguments

data	DE_effect obj
md	markdown_table output
caption	caption= title
print	print TRUE

Value

report apa

Examples

```
## Not run:

 #library(lavaan) # only needed once per session
 models <- '
   # measurement model
     ind60 =~ x1 + x2 + x3
     dem60 =~ y1 + y2 + y3 + y4
     dem65 =~ y5 + y6 + y7 + y8
 # regressions
     dem60 ~ ind60
     dem65 ~ ind60 + dem60
   # residual correlations
     y1 ~~ y5
     y2 ~~ y4 + y6
     y3 ~~ y7
     y4 ~~ y8
    y6 ~~ y8
 '
 fits <- sem(models, data=PoliticalDemocracy)
 summary(fits, standardized=TRUE)

 ##Direct effect
 DE_effect(fits)
 DE_effect(fits)%>% sem_apa()

example(sem)
DE_effect(fit)
DE_effect(fit) %>% sem_apa()


## End(Not run)
```

`sem_apa_ie` *sem_apa_ie*

Description

sem_apa_ie

Usage

```
sem_apa_ie(data, msg = "IE", md = FALSE, est = 3, std = 4, sig = 7, p = 6)
```

Arguments

`data`	data
`msg`	msg=”IE”
`md`	md = FALSE
`est`	est=3

std	std=4
sig	sig = 7
p	p=6

Value

ie report

Examples

```
## Not run:
example(sem)
IE_effect(fit)
IE_effect(fit) %>% sem_apa_ie()
sem_effect_ci(satis_sem1, ci=FALSE)
sem_effect_ci(satis_sem1, ci=FALSE)%>%sem_apa_ie()



## End(Not run)
```

---

sem_apa_ie_CI *sem_apa_ie_CI*

Description

sem_apa_ie_CI

Usage

```
sem_apa_ie_CI(
  data,
  caption = "매개효과 분석 결과 ",
  md = FALSE,
  est = 2,
  std = 3,
  sig = 6,
  CI = 7,
  Z = FALSE,
  tot_filter = "총",
  ind_filter = "IE"
)
```

Arguments

data	lavaan object
caption	” 매개효과분석결과”,
md	markdown
est	est sel 2
std	std sel 3

`sig`	sig sel 6
`CI`	CI seledct 7
`Z`	Z=FALSE
`tot_filter`	tot_filter=" 총",
`ind_filter`	ind_filter="IE"

Value

table

Examples

```
## Not run:

sem_effect_ci(satis_sem1)%>%sem_apa_ie_CI()


## End(Not run)
```

`sem_APA_path_table` *Create an APA-style path coefficient table from a lavaan SEM object*

Description

This function extracts structural path coefficients from a fitted `lavaan` SEM object and returns an APA-style summary table including unstandardized estimates, standard errors, standardized coefficients, p-values, and significance symbols.

Usage

```
sem_APA_path_table(fit, only_sig = FALSE)
```

Arguments

`fit`	A fitted `lavaan` SEM object (e.g., from `sem()` or `cfa()`).
`only_sig`	Logical. If `TRUE`, only statistically significant paths ($p < .05$) are returned. Default is `FALSE`.

Details

Only regression paths (operator `"~"`) are included. Covariances, variances, and intercepts are excluded.

Value

A data.frame containing:

Path Structural path in the form "Outcome <- Predictor"

B Unstandardized regression coefficient

SE Standard error of the coefficient

Beta Standardized regression coefficient (std.all)

p Formatted p-value

sig Significance symbol (* p < .05, ** p < .01, *** p < .001)

Examples

```
## Not run:
#-------------------------------------------------
# Example 1: Simple mediation model
#-------------------------------------------------
library(lavaan)

model_med <- '
  M ~ a*X
  Y ~ b*M + c*X
'

fit_med <- sem(model_med, data = mydata)

# All structural paths
sem_APA_path_table(fit_med)

# Only significant paths
sem_APA_path_table(fit_med, only_sig = TRUE)

#-------------------------------------------------
# Example 2: Multiple predictors
#-------------------------------------------------
model_multi <- '
  Y ~ X1 + X2 + X3
'

fit_multi <- sem(model_multi, data = mydata)

sem_APA_path_table(fit_multi)

#-------------------------------------------------
# Example 3: Full SEM with latent variables
#-------------------------------------------------
model_sem <- '
  F1 =~ x1 + x2 + x3
  F2 =~ y1 + y2 + y3

  F2 ~ F1
'

fit_sem_full <- sem(model_sem, data = mydata)

sem_APA_path_table(fit_sem_full)
```

```
sem_APA_path_table(fit_sem_full, only_sig = TRUE)

## End(Not run)
```

`sem_count_df` *Functions for calculating degrees of freedom in SEM*

Description

Functions for calculating degrees of freedom in SEM

Usage

```
sem_count_df(
  model,
  type = "fit",
  growth = FALSE,
  sample.nobs = 100,
  whatLabels = "model",
  sig = FALSE,
  plot = FALSE,
  rotation = 2
)
```

Arguments

`model`	lavaan syntax
`type`	fit, all
`growth`	grouwth =FALSE
`sample.nobs`	100(default)
`whatLabels`	model, If it is an actual measurement, est
`sig`	FALSE not outpur significant
`plot`	plot FALSE
`rotation`	rotation 4

Value

table and plot

Examples

```
## Not run:
#'
m1 <- "
y1 =~ x1 + x2 +x5
y2 =~ x3 + x4 +x6
y1 ~~ y2"
sem_count_df(m1)
sem_count_df(m1,"fit")
```

```
#그래프를 보고 싶을 때
diagram_model(m1)
#이함수에서 보려면
sem_count_df(m1, plot=TRUE)
#전체를 보여주는 함수
sem_count_df(m1,"all")
sem_count_df(m1,"all", plot=TRUE )
#'
#'
m1a <- "
y1 =~ x1 + x2 +x5
y2 =~ x3 + x4 +x6
y1 ~~ 0*y2"
sem_count_df(m1a)
sem_count_df(m1a,"all")

#이함수에서 보려면
sem_count_df(m1, plot=TRUE)

#simulation data lavaan
sem_count_df(m1,"lav")


## End(Not run)
```

`sem_effect` *SEM direct or indirect effect result*

Description

SEM direct or indirect effect result

Usage

```
sem_effect(
  model_sem,
  effect = "~",
  effect2 = NULL,
  ci = FALSE,
  lower = 8,
  upper = 9,
  unite = FALSE,
  unite_col = "z"
)
```

Arguments

`model_sem`	lavaan object
`effect`	select op
`effect2`	select op 2nd option
`ci`	FALSE
`lower`	8th col select

`upper`	9th col select
`unite`	TRUE unite p +sig
`unite_col`	select "z"

Value

repot

Examples

```
## Not run:

 #library(lavaan) # only needed once per session

models1 <- '
  # measurement model
    ind60 =~ x1 + x2 + x3
    dem60 =~ y1 + y2 + y3 + y4
    dem65 =~ y5 + y6 + y7 + y8
    # regressions
    dem60 ~ ind60
    dem65 ~ ind60 + dem60
      # residual correlations
    y1 ~~ y5
    y2 ~~ y4 + y6
    y3 ~~ y7
    y4 ~~ y8
    y6 ~~ y8
'
models2 ='
  # measurement model
    ind60 =~ x1 + x2 + x3
    dem60 =~ y1 + y2 + y3 + y4
    dem65 =~ y5 + y6 + y7 + y8
     # regressions
    dem60 ~ a1*ind60
    dem65 ~ a2*ind60 + a3*dem60
 #NEw
   ie := a1*a3
   te := a2 +a1*a3
'
fits1 <- sem(models1, data=PoliticalDemocracy)
fits2 <- sem(models2, data=PoliticalDemocracy)
summary(fits1, standardized=TRUE)
summary(fits2, standardized=TRUE)

parameterEstimates(fits1, ci=F, stand=T)
parameterEstimates(fits2, ci=F, stand=T)
sem_effect(fits1)
sem_effect(fits1, effect="=~")
sem_effect(fits1, effect="=~", effect2="~")
sem_effect(fits1, effect="~~")
sem_effect(fits1, effect=":=")

sem_effect(fits2)
sem_effect(fits2, effect="=~")
```

```
sem_effect(fits2, effect="~~")
sem_effect(fits2, effect=":=")
sem_effect(fits2, effect=":=", effect2="~")
 sem_effect(fits)%>% interpretation_de()
 # same method
 sem_effect(fits)%>% report_sem_de()


## End(Not run)
```

`sem_effect_ci` *SEM indirect effect*

Description

SEM indirect effect

Usage

```
sem_effect_ci(
  model_sem,
  effect1 = "DE",
  effect2 = "IE",
  effect3 = "TE",
  label = FALSE,
  ci = TRUE
)
```

Arguments

`model_sem`	lavaan objgect
`effect1`	”DE”
`effect2`	”IE”
`effect3`	”TE”
`label`	label
`ci`	confidence interval

Value

parameterEstimates()

Examples

```
## Not run:
#library(lavaan) # only needed once per session

models1 <- '
  # measurement model
    ind60 =~ x1 + x2 + x3
```

```
    dem60 =~ y1 + y2 + y3 + y4
    dem65 =~ y5 + y6 + y7 + y8
    # regressions
    dem60 ~ ind60
    dem65 ~ ind60 + dem60
      # residual correlations
    y1 ~~ y5
    y2 ~~ y4 + y6
    y3 ~~ y7
    y4 ~~ y8
    y6 ~~ y8

fits1 <- sem(models1, data=PoliticalDemocracy)


models2 ='
  # measurement model
    ind60 =~ x1 + x2 + x3
    dem60 =~ y1 + y2 + y3 + y4
    dem65 =~ y5 + y6 + y7 + y8
     # regressions
    dem60 ~ a1*ind60
    dem65 ~ a2*ind60 + a3*dem60
 #NEw
   ie := a1*a3
   te := a2 +a1*a3
   diff := a1 - a3

fits2 <- sem(models2, data=PoliticalDemocracy)

summary(fits1, standardized=TRUE)
summary(fits2, standardized=TRUE)
fits2 %>% sem_effect_ie()

## End(Not run)
```

`sem_effect_diff` *Diff effect*

Description

Diff effect

Usage

```
sem_effect_diff(
  model_sem,
  effect1 = "Diff",
  effect2 = "diff",
  label = FALSE,
  ci = TRUE
)
```

Arguments

model_sem	lavaan objgect
effect1	”Diff”
effect2	”diff”
label	label
ci	confidence interval

Value

parameterEstimates()

Examples

```
## Not run:
#library(lavaan) # only needed once per session

models1 <- '
  # measurement model
    ind60 =~ x1 + x2 + x3
    dem60 =~ y1 + y2 + y3 + y4
    dem65 =~ y5 + y6 + y7 + y8
    # regressions
    dem60 ~ ind60
    dem65 ~ ind60 + dem60
      # residual correlations
    y1 ~~ y5
    y2 ~~ y4 + y6
    y3 ~~ y7
    y4 ~~ y8
    y6 ~~ y8

models2 ='
  # measurement model
    ind60 =~ x1 + x2 + x3
    dem60 =~ y1 + y2 + y3 + y4
    dem65 =~ y5 + y6 + y7 + y8
     # regressions
    dem60 ~ a1*ind60
    dem65 ~ a2*ind60 + a3*dem60
 #NEw
   ie := a1*a3
   te := a2 +a1*a3
   Diff := a1 - a3




fits1 <- sem(models1, data=PoliticalDemocracy)
fits2 <- sem(models2, data=PoliticalDemocracy)
summary(fits1, standardized=TRUE)
summary(fits2, standardized=TRUE)
fits2 %>% sem_effect_diff()

## End(Not run)
```

`sem_effect_ie` *SEM indirect effect*

Description

SEM indirect effect

Usage

```
sem_effect_ie(
  model_sem,
  effect1 = "IE",
  effect2 = "IE",
  label = FALSE,
  ci = TRUE
)
```

Arguments

`model_sem`	lavaan objgect
`effect1`	”IE”
`effect2`	”IE”
`label`	label
`ci`	confidence interval

Value

parameterEstimates()

Examples

```
## Not run:
#library(lavaan) # only needed once per session

models1 <- '
  # measurement model
    ind60 =~ x1 + x2 + x3
    dem60 =~ y1 + y2 + y3 + y4
    dem65 =~ y5 + y6 + y7 + y8
    # regressions
    dem60 ~ ind60
    dem65 ~ ind60 + dem60
      # residual correlations
    y1 ~~ y5
    y2 ~~ y4 + y6
    y3 ~~ y7
    y4 ~~ y8
    y6 ~~ y8

models2 ='
  # measurement model
```

```
    ind60 =~ x1 + x2 + x3
    dem60 =~ y1 + y2 + y3 + y4
    dem65 =~ y5 + y6 + y7 + y8
     # regressions
    dem60 ~ a1*ind60
    dem65 ~ a2*ind60 + a3*dem60
 #NEw
   ie := a1*a3
   te := a2 +a1*a3
   diff := a1 - a3



fits1 <- sem(models1, data=PoliticalDemocracy)
fits2 <- sem(models2, data=PoliticalDemocracy)
summary(fits1, standardized=TRUE)
summary(fits2, standardized=TRUE)
fits2 %>% sem_effect_ie()

## End(Not run)
```

`sem_effect_ind` *SEM indirect effect*

Description

SEM indirect effect

Usage

```
sem_effect_ind(
  model_sem,
  effect1 = "DE",
  effect2 = "IE",
  digits = 3,
  label = FALSE,
  ci = TRUE
)
```

Arguments

`model_sem`	lavaan objgect
`effect1`	”IE”
`effect2`	”IE”
`digits`	digits=3
`label`	label
`ci`	confidence interval

Value

parameterEstimates()

Examples

```
## Not run:
#library(lavaan) # only needed once per session

models1 <- '
  # measurement model
    ind60 =~ x1 + x2 + x3
    dem60 =~ y1 + y2 + y3 + y4
    dem65 =~ y5 + y6 + y7 + y8
    # regressions
    dem60 ~ ind60
    dem65 ~ ind60 + dem60
      # residual correlations
    y1 ~~ y5
    y2 ~~ y4 + y6
    y3 ~~ y7
    y4 ~~ y8
    y6 ~~ y8

fits1 <- sem(models1, data=PoliticalDemocracy)


models2 ='
  # measurement model
    ind60 =~ x1 + x2 + x3
    dem60 =~ y1 + y2 + y3 + y4
    dem65 =~ y5 + y6 + y7 + y8
     # regressions
    dem60 ~ a1*ind60
    dem65 ~ a2*ind60 + a3*dem60
 #NEw
   ie := a1*a3
   te := a2 +a1*a3
   diff := a1 - a3

fits2 <- sem(models2, data=PoliticalDemocracy)

summary(fits1, standardized=TRUE)
summary(fits2, standardized=TRUE)
fits2 %>% sem_effect_ie()

## End(Not run)
```

---

sem_effect_te *Total effect*

Description

Total effect

Usage

```
sem_effect_te(
  model_sem,
  effect1 = "TE",
  effect2 = "te",
  label = FALSE,
  ci = TRUE
)
```

Arguments

model_sem	lavaan objgect
effect1	”IE”
effect2	”IE”
label	label
ci	confidence interval

Value

parameterEstimates()

Examples

```
## Not run:
#library(lavaan) # only needed once per session

models1 <- '
  # measurement model
    ind60 =~ x1 + x2 + x3
    dem60 =~ y1 + y2 + y3 + y4
    dem65 =~ y5 + y6 + y7 + y8
    # regressions
    dem60 ~ ind60
    dem65 ~ ind60 + dem60
      # residual correlations
    y1 ~~ y5
    y2 ~~ y4 + y6
    y3 ~~ y7
    y4 ~~ y8
    y6 ~~ y8

models2 ='
  # measurement model
    ind60 =~ x1 + x2 + x3
    dem60 =~ y1 + y2 + y3 + y4
    dem65 =~ y5 + y6 + y7 + y8
     # regressions
    dem60 ~ a1*ind60
    dem65 ~ a2*ind60 + a3*dem60
 #NEw
   ie := a1*a3
   te := a2 +a1*a3
   diff := a1 - a3
```

```
fits1 <- sem(models1, data=PoliticalDemocracy)
fits2 <- sem(models2, data=PoliticalDemocracy)
summary(fits1, standardized=TRUE)
summary(fits2, standardized=TRUE)
fits2 %>% sem_effect_te()

## End(Not run)
```

`sem_fit` *Structural Equation Model fit*

Description

Structural Equation Model fit

Usage

```
sem_fit(..., nice_table = FALSE)
```

Arguments

`...`	lavaan model
`nice_table`	markdown table TRUE

Value

table

Examples

```
## Not run:
#'
model_test1 <- '
  # latent variable definitions
     ind60 =~ x1 + x2 + x3
     dem60 =~ y1 + a*y2 + b*y3 + c*y4
     dem65 =~ y5 + a*y6 + b*y7 + c*y8

  # regressions
    dem60 ~ ind60
    dem65 ~ ind60 + dem60

  # residual correlations
    y1 ~~ y5
    y2 ~~ y4 + y6
    y3 ~~ y7
    y4 ~~ y8
    y6 ~~ y8
'
```

```
model_test2 <- '
  # latent variable definitions
     ind60 =~ x1 + x2 + x3
     dem60 =~ y1 + a*y2 + b*y3 + c*y4
     dem65 =~ y5 + a*y6 + b*y7 + c*y8

  # regressions
    dem60 ~ ind60
    dem65 ~ ind60 + dem60

'

fit1 <- sem(model_test1, data = PoliticalDemocracy)
fit2 <- sem(model_test2, data = PoliticalDemocracy)

sem_fit(fit1, fit2)
sem_fit("model-1" = fit1,"model-2" = fit2)
#'

## End(Not run)
```

`sem_lav_hypo` *Hypothesis generation functions*

Description

Hypothesis generation functions

Usage

```
sem_lav_hypo(model)
```

Arguments

`model`	lavaan syntax

Value

text

Examples

```
## Not run:
#'
# 예제 모델
model1a <- "
EXPE ~ H1*IMAG
QUAL ~ H2*EXPE
VAL ~  H3*QUAL + H4*EXPE
SAT ~ H5*IMAG + H6*EXPE + H7*QUAL + H8*VAL
LOY ~ H9*SAT + H10*IMAG
"
```

```
# 함수 실행
sem_lav_hypo(model1a)
#'

# Example usage with the provided model
model10 <- "
IMAG =~ imag1 + imag2 + imag3 + imag4 + imag5
EXPE =~ expe1 + expe2 + expe3 + expe4 + expe5
QUAL =~ qual1 + qual2 + qual3 + qual4 + qual5
VAL =~ val1 + val2 + val3 + val4
SAT =~ sat1 + sat2 + sat3 + sat4
LOY =~ loy1 + loy2 + loy3 + loy4

EXPE ~ IMAG
QUAL ~ EXPE
VAL ~  QUAL+ EXPE
SAT ~ IMAG + EXPE + QUAL + VAL
LOY ~ SAT + IMAG
"
lav_extract_sm(model10)%>%sem_lav_hypo()
lav_extract_sm(model10, prefix="H")%>%sem_lav_hypo()



#'

## End(Not run)
```

`sem_lav_ind_hypo` *Function that takes path data as input and outputs an indirect effect hypothesis.*

Description

Function that takes path data as input and outputs an indirect effect hypothesis.

Usage

```
sem_lav_ind_hypo(paths)
```

Arguments

`paths`	indpath

Value

text

Examples

```
## Not run:
#'
```

```
# 모델 문자열
model1a <- "
EXPE ~ H1*IMAG
QUAL ~ H2*EXPE
VAL ~ H3*QUAL + H4*EXPE
SAT ~ H5*IMAG + H6*EXPE + H7*QUAL + H8*VAL
LOY ~ H9*SAT + H10*IMAG
"

model1 <- "
EXPE ~ IMAG
QUAL ~ EXPE
VAL ~ QUAL + EXPE
SAT ~ IMAG + EXPE + QUAL + VAL
LOY ~ SAT + IMAG
"
lav_extract_path(model1a)%>%
  lav_generate_ind_paths() %>%
  lav_ind_hypo()


## End(Not run)
```

`sem_model` *Drawing Structural Equation Models Using the lavaan Syntax (park JH PhD)*

Description

Drawing Structural Equation Models Using the lavaan Syntax (park JH PhD)

Usage

```
sem_model(
  model,
  residuals = FALSE,
  whatLabels = "model",
  rotation = 2,
  residScale = 12,
  layout = "tree2",
  edge.label.cex = 1,
  edge.label.position = 0.5,
  edge.color = "black",
  sizeMan = 6,
  sizeMan2 = 3,
  sizeLat = 8,
  sizeLat2 = 4,
  style = "lisrel",
  sig = FALSE,
  exoVar = FALSE,
  exoCov = TRUE,
  curve = 1.5,
```

```
  asize = 2,
  mar = c(3, 3, 3, 3),
  nDigits = 3,
  shapeLat = "circle",
  shapeMan = "rectangle",
  shapeInt = "triangle",
  sample.nobs = 100,
  border.width = 2,
  edge.width = 1.5,
  Groups = FALSE,
  group_match = "lat",
  growth = FALSE,
  structural = FALSE,
  edgeLabels = NULL,
  nodeLabels = NULL,
  fixedStyle = 1,
  color = list(lat = "gray10", man = "gold"),
  type = "plot"
)
```

Arguments

`model`	lavaan syntax
`residuals`	residauls TRUE
`whatLabels`	”model
`rotation`	rotation=2
`residScale`	residScale=12
`layout`	layout=”tree”
`edge.label.cex`	1
`edge.label.position`	0.5
`edge.color`	edge.color= black
`sizeMan`	8
`sizeMan2`	4
`sizeLat`	10
`sizeLat2`	6
`style`	lisrel
`sig`	FALSE
`exoVar`	FALSE
`exoCov`	TRUE
`curve`	1.5
`asize`	1.5
`mar`	c(2,8,3,10)
`nDigits`	2
`shapeLat`	”circle, ellipse

shapeMan	retangle
shapeInt	triangle ,
sample.nobs	sample.nobs = 100
border.width	border.width=1
edge.width	edge.width=1.5
Groups	FALSE
group_match	group_match lat
growth	growth= FALSE, TRUE model.type =”grouwth”
structural	structural = FALSE
edgeLabels	edgeLabels=NULL
nodeLabels	nodeLabels=NULL
fixedStyle	fixedStyle=1 default rbase linetype
type	type =”plot, and res is lavaan simdata result

Value

plot

Examples

```
## Not run:
#'
model_test <- '
  # latent variable definitions
     ind60 =~ x1 + x2 + x3
     dem60 =~ y1 + a*y2 + b*y3 + c*y4
     dem65 =~ y5 + a*y6 + b*y7 + c*y8

  # regressions
    dem60 ~ ind60
    dem65 ~ ind60 + dem60

  # residual correlations
    # y1 ~~ y5
    # y2 ~~ y4 + y6
    # y3 ~~ y7
    # y4 ~~ y8
    # y6 ~~ y8
'

HSmodel <- ' visual  =~ x1 + x2 + x3
             textual =~ x4 + x5 + x6
             speed   =~ x7 + x8 + x9 '


m1a <- "
y1 =~ x1 + x2 +x5
y2 =~ x3 + x4 +x6
y1 ~~ y2"

sem_model(model= HSmodel, groups=F,  residuals=F)
```

```
sem_model(model= model_test, style="ram", groups=F)
sem_model(model= model_test, residuals=T)
sem_model(model= m1a, residuals=T)

#'

## End(Not run)
```

`sem_plot` *Fast sem plot draw*

Description

Fast sem plot draw

Usage

```
sem_plot(
  data,
  whatLabels = "std",
  type = "res1",
  opt = 1,
  layout = "tree3",
  curve = 2,
  rotation = 2,
  sizeLat = 10,
  sizeLat2 = 6,
  sizeMan = 8,
  sizeMan2 = 4,
  edge.label.cex = 0.9,
  edge.label.position = 0.5,
  mar = c(1, 5, 1, 5),
  groups = "lat",
  pastel = TRUE,
  fade = FALSE,
  se = FALSE,
  exoVar = FALSE,
  residuals = FALSE,
  residScale = 12,
  intercepts = FALSE,
  semptools = TRUE
)
```

Arguments

`data`	lavaaan object
`whatLabels`	ste
`type`	type res 1
`opt`	1, 2, 3,
`layout`	layY(opt =2, you make layout), or tree3

curve	-2
rotation	rotation = 2
sizeLat	8
sizeLat2	6
sizeMan	6
sizeMan2	3
edge.label.cex	0.7
edge.label.position	0.6
mar	c(1,5,1,5)
groups	groups
pastel	pastel
fade	default FALSE when opt =3
se	se false
exoVar	exoVar=FLASE
residuals	residuals F
residScale	12
intercepts	intercepts =FALSE becouse semptools
semptools	using semptools TRUE

Value

plot

Examples

```
## Not run:

# library(lavaan)
#import data
example(sem)

sem_plot(fit)

example(cfa)
sem_plot(fit)
#'
HS.model <- '
    visual  =~ x1 + b1*x2 + x3
    textual =~ x4 + b2*x5 + x6
    speed   =~ x7 + b3*x8 + x9
speed ~ visual + textual

'
fit1 <- sem(HS.model, data = HolzingerSwineford1939)

fit1 %>%sem_plot(residuals=T, curve=2)

## End(Not run)
```

`shapiro_test_all` *Custom function to perform Shapiro-Wilk test on all columns of a data frame*

Description

Custom function to perform Shapiro-Wilk test on all columns of a data frame

Usage

```
shapiro_test_all(data, digits = 3)
```

Arguments

`data`	data.frame
`digits`	default 3

Value

nomalitytest result

Examples

```
## Not run:
data <- mtcars[,c("mpg","disp","hp","drat","wt","qsec")]

## Shapiro-Wilke Test Results for All Columns
shapiro_test_all(data)


## End(Not run)
```

`shapiro_test_df` *Calculate Shapiro-Wilke test results for all columns*

Description

Calculate Shapiro-Wilke test results for all columns

Usage

```
shapiro_test_df(data, digits = 3)
```

Arguments

`data`	data.frame
`digits`	default 3

Value

nomalitytest result

Examples

```
## Not run:
data <- mtcars[,c("mpg","disp","hp","drat","wt","qsec")]

## Shapiro-Wilke Test Results for All Columns
shapiro_test_df(data)


## End(Not run)
```

`simdata_cov` *Regression with covariance, generate structural equation data restoration*

Description

Regression with covariance, generate structural equation data restoration

Usage

```
simdata_cov(
  covmat = NULL,
  n = 300,
  dv_mean = 0,
  iv_means = rep(0, ncol(covmat) - 1),
  means = NULL,
  cormat = NULL,
  sds = rep(1, ncol(cormat)),
  seed = NULL
)
```

Arguments

`covmat`	cov matrix
`n`	sample size default = 300
`dv_mean`	dv mean
`iv_means`	iv means
`means`	total means default null
`cormat`	correlation matrix
`sds`	sds is need cor to cov
`seed`	set seed today Sys.Data

Value

sim data

Examples

```
## Not run:
#'
#simdata mean setting 0
#set.seed(todaty)
cov(mtcars[,c("mpg","hp","wt")]) %>%cov_simdata(n=32)
cov(mtcars[,c("mpg","hp","wt")]) %>%cov_simdata(n=32, means=c(0,0,0))

cov(mtcars[,c("mpg","hp","wt")]) %>%
  cov_simdata(n=32, means=c(20.09, 146.69 ,3.22))

cov(mtcars[,c("mpg","hp","wt")]) %>%
  cov_simdata(n=32, dv_mean=20.09, iv_means=c(146.69 ,3.22))

#simdata
cov(mtcars[,c("mpg","hp","wt")]) %>%
  cov_simdata(n=32) %>%
  lm(formula = mpg ~ hp + wt)%>%summary()

#revised mean data Closer to real-world data
cov(mtcars[,c("mpg","hp","wt")]) %>%
  cov_simdata(n=32, dv_mean=20.09, iv_means=c(146.69 ,3.22)) %>%
  lm(formula = mpg ~ hp + wt)%>%summary()


#real data
lm(formula = mpg ~ hp +wt,data=mtcars)%>%summary()



# define sample statistics
x <- MASS::Cars93[,c("EngineSize", "Horsepower", "RPM")]
covx <- cov(x)
covx <- matrix(c(1.076122, 39.776999, -339.163745,
                 39.776999, 2743.079, 1146.634,
                 -339.163745, 1146.634, 356088.7097), nrow = 3,
               dimnames = list(c("EngineSize", "Horsepower", "RPM"),
                               c("EngineSize", "Horsepower", "RPM")))
covx
# 함수 실행
sim_data <- simdata_cov(covmat = covx, n = 100)
head(sim_data)
# check
sim_data <- simdata_cov(covmat = covx, n = 100)
lm( EngineSize~ Horsepower+RPM, data = sim_data) %>% summary()
#real check
lm( EngineSize ~ Horsepower +RPM , data=x)%>% summary()



#'

## End(Not run)
```

`simdata_sem` *Structural equation data generation using the lavaan syntax*

Description

Structural equation data generation using the lavaan syntax

Usage

```
simdata_sem(model, N = 100, datatype = "data.frame", empirical = TRUE)
```

Arguments

`model`	lavaan model
`N`	sample size N
`datatype`	”data.frame”, ”cor”, ”matrix”
`empirical`	empirical

Value

data

Examples

```
## Not run:
#'
# devtools::install_github("M-E-Rademaker/cSEM")
# devtools::install_github("M-E-Rademaker/cSEM.DGP")
#
# library(cSEM.DGP)
# library(cSEM)

# sample data
HS.model <- ' visual  =~ x1 + x2 + x3
              textual =~ x4 + x5 + x6
              speed   =~ x7 + x8 + x9
              '

fit <- cfa(HS.model, data = HolzingerSwineford1939)
summary(fit)

#lavaan
HS_model <- ' visual  =~ 0.77*x1 + 0.42*x2 + 0.58*x3
              textual =~ 0.85*x4 + 0.86*x5 + 0.84*x6
              speed   =~ 0.57*x7 + 0.72*x8 + 0.67*x9
              visual ~~  0.46*textual
              visual ~~  0.47*speed
              textual ~~  0.28*speed
          '
#cSEM, cSEM.DGP
# HS_simdata = generateData(.model = HS_model,
#                           .return_type = "data.frame",
```

```
#                            .empirical = TRUE, .N= 301)

HS_simdata1 = simdata_sem(model = HS_model,
                          datatype = "data.frame",
                          empirical = TRUE, N= 301)
fit1 <- cfa(HS.model, data = HS_simdata1)
summary(fit1)
fit1 %>% diagram2("std",rotation =2, mar=c(3,12,3,8), groups=T)
#'
#'
#'


## End(Not run)
```

`simple_bar` *Tsimple_bar*

Description

Tsimple_bar

Usage

```
simple_bar(
  data,
  add_y = 10,
  color = 2:17,
  vjust = 1,
  cex = 1,
  line = FALSE,
  main = NULL,
  xlab = "x",
  ylab = "y"
)
```

Arguments

`data`	table
`add_y`	max 30
`color`	color=1:16
`vjust`	vjust =10
`cex`	cex=1
`line`	T/F
`main`	main sistitle
`xlab`	xlab
`ylab`	ylab

Value

bar graph

Examples

```
## Not run:
table(mtcars$cyl) %>% simple_bar(cex=1.5)

data(bioChemists, package = "pscl")
bioChemists <- bioChemists %>%
rename(
  Num_Article = art, #articles in last 3 years of PhD
   Sex = fem, #coded 1 if female
   Married = mar, #coded 1 if married
   Num_Kid5 = kid5, #number of childeren under age 6
   PhD_Quality = phd, #prestige of PhD program
   Num_MentArticle = ment #articles by mentor in last 3 years
   )
  table(bioChemists$Num_Article) %>%
      simple_bar(cex=1.5, add_y= 30, vjust=10)


## End(Not run)
```

SKEW *skew 계산함수*

Description

skew 계산함수

Usage

```
SKEW(x)
```

Arguments

x	data vector

Value

skewness

Sobel_test *sobel test*

Description

sobel test

Usage

```
Sobel_test(
  a,
  b,
  sa,
  sb,
  roburst = FALSE,
  n = NULL,
  k1 = 2,
  k2 = 2,
  digits = 5,
  conf_level = 0.95
)
```

Arguments

a	est1
b	est2
sa	se1
sb	se2
roburst	roburst T whide correction
n	sample size
k1	number of regressin coefs, default 2
k2	number of regressin coefs, default 2
digits	5
type	Sobel, Aroian, Goodman

Value

test result

Examples

```
## Not run:
#'
# Example usage
a <- 0.5  # Example coefficient a
b <- 0.3  # Example coefficient b
sa <- 0.1 # Standard error of a
sb <- 0.05 # Standard error of b

Sobel_test(a, b, sa, sb)
Sobel_test(a, b, sa, sb, "all")
Sobel_test(a, b, sa, sb, "Sobel")

Sobel_test(a, b, sa, sb,"Aroian")

Sobel_test(a, b, sa, sb,"Goodman")


# roburst se correction : White(1980)
```

```
Sobel_test(0.2, 0.3,0.05, 0.04, TRUE, 100)


## End(Not run)
```

`sobel_test_extend` *sobel_test_extend*

Description

sobel_test_extend

Usage

```
sobel_test_extend(
  coefficients,
  se_values,
  digits = 5,
  show = TRUE,
  sobel = "aroian"
)
```

Arguments

`coefficients`	coefficients c()
`se_values`	se
`digits`	5
`show`	TRUE
`sobel`	aroian(default), sobel, goodman

Value

seobel resutl

Examples

```
## Not run:
#'

# Example usage
sdata <- data.frame(
  paths = c("IMAG -> EXPE", "IMAG -> SAT",
              "IMAG -> LOY", "EXPE -> QUAL", "EXPE -> VAL", "EXPE -> SAT",
            "QUAL -> VAL", "QUAL -> SAT", "VAL -> SAT", "SAT -> LOY"),
  Original = c(0.578959128, 0.200724200, 0.275149576,
               0.848344408, 0.105477650, -0.002753995,
                0.676655529, 0.122144566, 0.589330650, 0.495479322),
  Mean.Boot = c(0.58232765, 0.21209094, 0.27486564,
                 0.84793899, 0.10992051, -0.01008222,
                 0.67780527, 0.15132543, 0.56312717, 0.50757755),
```

```
  Std.Error = c(0.04358293, 0.06056988, 0.07729000,
                0.01864573, 0.06968826, 0.06507903,
               0.07760527, 0.08440236, 0.08723293, 0.07856694),
  perc.025 = c(0.497644719, 0.107561558, 0.140641502,
               0.815794494, -0.025566471, -0.147788188,
              0.519959683, -0.008629516, 0.392664443, 0.351774130),
  perc.975 = c(0.6646198, 0.3275312, 0.4161131,
               0.8828875, 0.2545785, 0.1086856,
              0.8166486, 0.2991045, 0.7012494, 0.6556333)
)



sobel_test_extend(coefficients= c("IMAG -> EXPE" = 0.578959128,
                                  "EXPE -> SAT" = -0.002753995,
                                  "SAT -> LOY" = 0.495479322),
                  se_values = c(se1 = 0.04358293,
                                se2 = 0.06507903,
                                se3 = 0.0785669))
# Example usage
coefficients1 <- c("IMAG -> EXPE" = 0.578959128,
                   "EXPE -> SAT" = -0.002753995, "SAT -> LOY" = 0.495479322)
se_values1 <- c(se1 = 0.04358293, se2 = 0.06507903, se3 = 0.0785669)

sobel_test_extend(coefficients1, se_values1)


# Test with unnamed values
coefficients2 <- c(0.578959128, -0.002753995, 0.495479322)
se_values2 <- c(0.04358293, 0.06507903, 0.0785669)

sobel_test_extend(coefficients2, se_values2)


# Test with est and se named values
coefficients3 <- c(imag2expe = 0.578959128,
                   expe2sat = -0.002753995, sat2loy = 0.495479322)
se_values3 <- c(se1 = 0.04358293, se2 = 0.06507903, se3 = 0.0785669)

sobel_test_extend(coefficients3, se_values3)




## End(Not run)
```

`sobel_test_general` *Generalized Sobel Test function*

Description

Generalized Sobel Test function

Usage

```
sobel_test_general(
  a,
  SE,
  roburst = FALSE,
  n = NULL,
  k = 2,
  digits = 5,
  conf = 0.95
)
```

Arguments

a	est
SE	se
roburst	roburst T whide correction
n	sample size
k	number of regressin coefs, default 2
digits	5
conf	0.90, 0.95, 0.99, 0.999

Value

result

Examples

```
## Not run:
#'

general_Sobel_test(a = c(0.5, 0.3), SE = c(0.1, 0.05))

Sobel_test_2(a=0.5, b=0.3, sa=0.1, sb =0.05)

# Input data
a <- c(0.2, 0.3, 0.4, 0.5)
SE <- c(0.05, 0.04, 0.03, 0.02)

# Calculate results
sobel_test_general(a = c(0.2, 0.3, 0.4, 0.5), SE = c(0.05, 0.04, 0.03, 0.02))
aroian_sobel_test_general(a = c(0.2, 0.3, 0.4, 0.5), SE = c(0.05, 0.04, 0.03, 0.02))
Goodman_sobel_test_general(a = c(0.2, 0.3, 0.4, 0.5), SE = c(0.05, 0.04, 0.03, 0.02))
general_Sobel_test(a = c(0.2, 0.3, 0.4, 0.5), SE = c(0.05, 0.04, 0.03, 0.02))
#'
# 예제 데이터
a <- c(0.5, 0.3, 0.4)
SE <- c(0.1, 0.1, 0.1)

# 함수 실행 (기본 95% 신뢰구간)
sobel_test_general(a, SE)

# 함수 실행 (90% 신뢰구간)
```

```
result_90 <- sobel_test_general(a, SE, conf=0.90)
print(result_90)

# 함수 실행 (99% 신뢰구간)
result_99 <- sobel_test_general(a, SE, conf=0.99)
print(result_99)

# 함수 실행 (99.9% 신뢰구간)
result_999 <- sobel_test_general(a, SE, conf=0.999)
print(result_999)


## End(Not run)
```

`sobel_test_serial` *sobel_test_extend*

Description

sobel_test_extend

Usage

```
sobel_test_serial(
  coefficients,
  se_values,
  digits = 5,
  show = TRUE,
  sobel = "aroian"
)
```

Arguments

`coefficients`	coefficients c()
`se_values`	se
`digits`	5
`show`	TRUE
`sobel`	aroina, sobel, goodman

Value

seobel resutl

Examples

```
## Not run:
#'

# Example usage
```

```
sdata <- data.frame(
  paths = c("IMAG -> EXPE", "IMAG -> SAT",
              "IMAG -> LOY", "EXPE -> QUAL", "EXPE -> VAL", "EXPE -> SAT",
            "QUAL -> VAL", "QUAL -> SAT", "VAL -> SAT", "SAT -> LOY"),
  Original = c(0.578959128, 0.200724200, 0.275149576,
               0.848344408, 0.105477650, -0.002753995,
                0.676655529, 0.122144566, 0.589330650, 0.495479322),
  Mean.Boot = c(0.58232765, 0.21209094, 0.27486564,
                 0.84793899, 0.10992051, -0.01008222,
                 0.67780527, 0.15132543, 0.56312717, 0.50757755),
  Std.Error = c(0.04358293, 0.06056988, 0.07729000,
                  0.01864573, 0.06968826, 0.06507903,
                 0.07760527, 0.08440236, 0.08723293, 0.07856694),
  perc.025 = c(0.497644719, 0.107561558, 0.140641502,
                 0.815794494, -0.025566471, -0.147788188,
                0.519959683, -0.008629516, 0.392664443, 0.351774130),
  perc.975 = c(0.6646198, 0.3275312, 0.4161131,
                 0.8828875, 0.2545785, 0.1086856,
                0.8166486, 0.2991045, 0.7012494, 0.6556333)
)



sobel_test_serial(coefficients= c("IMAG -> EXPE" = 0.578959128,
                                  "EXPE -> SAT" = -0.002753995,
                                  "SAT -> LOY" = 0.495479322),
                  se_values = c(se1 = 0.04358293,
                                se2 = 0.06507903,
                                se3 = 0.0785669))
# Example usage
coefficients1 <- c("IMAG -> EXPE" = 0.578959128,
                   "EXPE -> SAT" = -0.002753995, "SAT -> LOY" = 0.495479322)
se_values1 <- c(se1 = 0.04358293, se2 = 0.06507903, se3 = 0.0785669)

sobel_test_serial(coefficients1, se_values1)


# Test with unnamed values
coefficients2 <- c(0.578959128, -0.002753995, 0.495479322)
se_values2 <- c(0.04358293, 0.06507903, 0.0785669)

sobel_test_serial(coefficients2, se_values2)


# Test with est and se named values
coefficients3 <- c(imag2expe = 0.578959128,
                   expe2sat = -0.002753995, sat2loy = 0.495479322)
se_values3 <- c(se1 = 0.04358293, se2 = 0.06507903, se3 = 0.0785669)

sobel_test_serial(coefficients3, se_values3)




## End(Not run)
```

split_korean_word *split_korean_word- Separating vowels and consonants in Korean*

Description

split_korean_word- Separating vowels and consonants in Korean

Usage

```
split_korean_word(word, paste = TRUE)
```

Arguments

word	input word, one or more
paste	Whether you want the outputs to be separate or combined into one

Value

text data

Examples

```
## Not run:
split_korean_word("책")
## [1] "ㅊ,ㅐ,ㄱ"
split_korean_word("채")
## [1] "ㅊ,ㅐ,"
split_korean_word("고양이")
#' ##[1] "ㄱ,ㅗ,,ㅇ,ㅑ,ㅇ,ㅇ,ㅣ,"
split_korean_word("곱창밥")
## [1] "ㄱ,ㅗ,ㅂ,ㅊ,ㅏ,ㅇ,ㅂ,ㅏ,ㅂ"

## End(Not run)
```

split_korean_word_check *Tsplit_korean_word_check is datacheck*

Description

Tsplit_korean_word_check is datacheck

Usage

```
split_korean_word_check(worddata, type = "all")
```

Arguments

worddata	worddata
type	type defaul all, text, df , all is input and output, text is result , df is data.frame

Value

split word

Examples

```
## Not run:
split_korean_word_check("호랑이")

## End(Not run)
```

`split_kw_df` *split_kw_df Create a data column by separating the initial, neutral, and final Korean characters.*

Description

split_kw_df Create a data column by separating the initial, neutral, and final Korean characters.

Usage

```
split_kw_df(df, sel_col = "word", remove = TRUE)
```

Arguments

`df`	data.frame
`sel_col`	select word column
`remove`	remove False is overlab, TRUE is generate new data

Value

data.frame

Examples

```
## Not run:
wd <- data.frame(
id =  paste0("BY", 1:5),
word = c("토끼", "휘파람", "뻐스", "우리냥","우리낙"),
tone = c("HH","LHH","HL","LHH","LHL" ),
gender =c("F","F","M","M","M"),
age = c(20,60,20,60, 30),
area= c("n","s","n","s","s"),
mix= c(FALSE,FALSE,FALSE,TRUE,TRUE),
stringsAsFactors = FALSE)

wd%>%split_kw_df("word")
wd%>%split_kw_df()


## End(Not run)
```

`split_kw_df_match` *split_kw_df_match, using Goeun Kim, PhD (Seoul National University 2024)*

Description

split_kw_df_match, using Goeun Kim, PhD (Seoul National University 2024)

Usage

```
split_kw_df_match(df, sel_col = "word", remove = TRUE, type = "res")
```

Arguments

`df`	data.frame
`sel_col`	word column
`remove`	TRUE is newdata add
`type`	type is res and all

Value

data.frame

Examples

```
## Not run:

#' wd <- data.frame(
id =  paste0("BY", 1:5),
word = c("토끼", "휘파람", "뻐스", "우리냥","우리낙"),
tone = c("HH","LHH","HL","LHH","LHL" ),
gender =c("F","F","M","M","M"),
age = c(20,60,20,60, 30),
area= c("n","s","n","s","s"),
mix= c(FALSE,FALSE,FALSE,TRUE,TRUE),
stringsAsFactors = FALSE)

# Create new data
wd%>%split_kw_df_match("word", type="res")

wd%>%split_kw_df_match("word", type="all")

wd2 <- data.frame(id =  paste0("BY", 1:5),
word = c("토끼", "휘파람", "뻐스", "우리냥","우리낙망"))

wd2%>%split_kw_df_match("word", type="res")

wd2%>%split_kw_df_match("word", type="all")



## End(Not run)
```

`split_word` *Separate words into syllables*

Description

Separate words into syllables

Usage

```
split_word(word)
```

Arguments

`word`	Words being typed

Value

Syllable-separated data

Examples

```
## Not run:
split_word("HLL")
split_word("호랑이")


## End(Not run)
```

`str_cut` *str_cut function*

Description

A function that modifies the values of a specified column in a data frame by cutting the string based on a specific separator (sep).

Usage

```
str_cut(df, col = "term", sep = "\\(")
```

Arguments

`df`	A data frame.
`col`	The name of the column to modify in the data frame. Default is "term".
`sep`	The separator used to cut the string. Default is "\(".

Value

A modified data frame.

Examples

```
## Not run:
# Example usage
data <- data.frame(
  term = c("AI 학습 진단 및 문제은행 (예: AI 펭톡, EBS 단추)",
           "기타(직접입력)",
           "메타버스/게임화 도구 (예: Zep, 띵커벨, 퀴즈앤, Kahoot! 등)")
)

# Using the function
result <- str_cut(data, "term")
print(result)

## End(Not run)
```

`summary.jjstat_sem` *Summary Method for jjstat_sem Objects*

Description

lavaan 스타일의요약출력을제공하는`jjstat_sem` 클래스의 summary 메서드이다.

Usage

```
## S3 method for class 'jjstat_sem'
summary(object, fit.measures = TRUE, standardized = TRUE, digits = 3, ...)
```

Arguments

`object`	object of class `jjstat_sem`.
`fit.measures`	logical. 모형적합도지수출력여부.
`standardized`	logical. 표준화계수 (std.lv, std.all) 출력여부.
`digits`	integer. 출력시반올림자릿수.
`...`	additional arguments (unused).

Details

본함수는 lavaan 의`summary()` 출력형식을참고하여, 교육용 SEM 엔진의결과를사람이읽기쉬운형태로요약한다. 표준오차는 bootstrap 기반으로계산된다.

Value

Invisibly returns the original `jjstat_sem` object.

Examples

```
## Not run:
res <- jjstat_SEM(
  data = HolzingerSwineford1939[,1:6],
  latent_list = list(
    F1 = c("x1","x2","x3"),
    F2 = c("x4","x5","x6")
  ),
  structural = list(path("F1","F2")),
  nboot = 100
)

summary(res)

## End(Not run)
```

---

summary_bartlett *Factorization Bartlett test results*

Description

Factorization Bartlett test results

Usage

```
summary_bartlett(data, test = "df", cat = TRUE)
```

Arguments

data	data. cor data. result data
test	perform test df, cor, none
cat	show method

Value

result

Examples

```
## Not run:
#'
# Henry Kaiser (1970) introduced an Measure of Sampling
# Adequacy  (MSA) of factor analytic data matrices.
# Kaiser and Rice (1974) then modified it.
#'

New = data.frame(
  A01 = c(4, 4, 5, 4, 5, 5, 5, 4, 4, 2, 4, 4, 5, 4, 4, 4, 4, 5, 1, 3),
  A02 = c(2, 3, 3, 5, 5, 4, 5, 5, 4, 4, 4, 4, 4, 3, 3, 5, 4, 4, 3, 1),
  A03 = c(3, 5, 4, 5, 4, 4, 4, 4, 4, 4, 5, 4, 5, 4, 4, 4, 4, 5, 3, 3),
  A04 = c(4, 4, 4, 4, 5, 4, 5, 4, 4, 3, 5, 4, 5, 5, 5, 4, 5, 5, 1, 5),
  A05 = c(3, 2, 3, 5, 4, 4, 4, 4, 4, 4, 4, 3, 5, 2, 3, 4, 5, 5, 4, 2)
```

```
)
New %>%summary_bartlett("df")
New %>%cor()%>% summary_bartlett("cor")
New %>%cor()%>%psych::cortest.bartlett()%>% summary_bartlett()


## End(Not run)
```

`summary_fa` *FA,PCA result summary*

Description

FA,PCA result summary

Usage

```
summary_fa(fa_data, digits = 3)
```

Arguments

`fa_data`	fa, factanal, pricipla result
`digits`	roundign defaylt 4

Value

table

Examples

```
library(MVT)

#  Mardia, K.V., Kent, J.T., and Bibby, J.M. (1979).  _Multivariate
#  Analysis_. Academic Press, London.
data(examScor)
factanal(scale(examScor), factors = 2, rotation = "varimax")%>% summary_fa()

#
psych::principal(cor(examScor), nfactor = 2) %>%summary_fa()

psych::prcomp(cor(examScor), nfactor = 2) %>%summary_fa()

factanal(scale(examScor), factors = 2) %>%summary_fa()
```

summary_kmo *Kaiser-Meyer-Olkin factor adequacy (KMO)*

Description

Kaiser-Meyer-Olkin factor adequacy (KMO)

Usage

```
summary_kmo(data, test = "df", cat = TRUE)
```

Arguments

data	data, cor data, result data
test	none, cor, df
cat	TRUE

Value

result

Examples

```
## Not run:
#'
## Example usage
New = data.frame(
  A01 = c(4, 4, 5, 4, 5, 5, 5, 4, 4, 2, 4, 4, 5, 4, 4, 4, 4, 5, 1, 3),
  A02 = c(2, 3, 3, 5, 5, 4, 5, 5, 4, 4, 4, 4, 4, 3, 3, 5, 4, 4, 3, 1),
  A03 = c(3, 5, 4, 5, 4, 4, 4, 4, 4, 4, 5, 4, 5, 4, 4, 4, 4, 5, 3, 3),
  A04 = c(4, 4, 4, 4, 5, 4, 5, 4, 4, 3, 5, 4, 5, 5, 5, 4, 5, 5, 1, 5),
  A05 = c(3, 2, 3, 5, 4, 4, 4, 4, 4, 4, 4, 3, 5, 2, 3, 4, 5, 5, 4, 2)
)
New %>%summary_kmo("df")
New %>%summary_kmo("df", cat=FALSE)
New %>%cor()%>% summary_kmo("cor")
New %>%cor()%>%psych::KMO()%>% summary_kmo()

## End(Not run)
```

summary_sem *summary sem result and apa report*

Description

summary sem result and apa report

Usage

```
summary_sem(
  lav_obj,
  whatLabels = "est",
  caption = "Result of path Coefficient",
  effect = "~",
  effect2 = NULL,
  md = TRUE,
  type = "all",
  opt = 1,
  sizeMan = 8,
  sizeMan2 = 5,
  layout = "tree2",
  mar = c(1, 5, 1, 5),
  intercepts = FALSE
)
```

Arguments

lav_obj	lavaan object
whatLabels	whatLabels est
caption	caption possible
effect	select ”~”
effect2	select 2nd
md	markdown default TRUEz
type	all, effect, apa, plot
opt	opt=1~4
sizeMan	sizeMan = 10
sizeMan2	sizeMan2 =6
layout	layout =”tree2”
mar	mar = c(1,5,1,5)
intercepts	intercept=FALSE semptools

Value

result and md

Examples

```
## Not run:
example(sem)
fit %>% summary_sem()


## End(Not run)
```

table_apa *Report frequency analysis result*

Description

Report frequency analysis result

Usage

```
table_apa(data, print = TRUE, md = FALSE, unit = "명")
```

Arguments

data	table_df objecte
print	consolt outpur
md	markdown_table output
unit	unit =" 명" if you want chage unit

Value

report data apa

Examples

```
## Not run:

sats %>%table_df(digits=3) %>% table_apa()
sats %>%table_df(digits=3) %>% table_apa(print=F)
sats %>%table_df(digits=3) %>% table_apa(print=T, md= T)

## End(Not run)
```

table_df *Frequency Analysis Functions*

Description

Frequency Analysis Functions

Usage

```
table_df(data, exclude = NULL, digits = 4, cat = TRUE)
```

Arguments

data	data.frame
exclude	Select variables to exclude from analysis
digits	round
cat	TRUE Data frequency analysis results message output

Value

Publication frequency table

Examples

```
## Not run:
## Identifying variables
 mtcars %>% as_trt("cyl","am","vs","gear")%>% tibble()

 mtcars %>% as_trt("cyl","am","vs","gear")%>% table_df()

 iris %>% as_trt("Species")%>% table_df()
 iris %>% as_trt("Species")%>% table_df() %>%table_apa()


 mtcars %>% as_trt("cyl","am","vs","gear")%>% table_df(exclude=c("cyl", "am"))
 sats %>% table_df()



## End(Not run)
```

---

table_df_bar *Generate a Bar Plot with Automatic Column Renaming*

Description

Generate a Bar Plot with Automatic Column Renaming

Usage

```
table_df_bar(
  df,
  size.x = 20,
  size.text = 6,
  add_range = 100,
  x.title = "Items",
  flip = TRUE,
  hjust = -0.1,
  vjust = -0.5,
  x11_width = 8,
  x11_height = 5,
  linetype = "dashed",
  linecolor = "gray50",
  show_line = FALSE,
  levels = c("전혀 아니다", "아니다", "중간이다", "그렇다",
    "매우 그렇다"),
  sort = FALSE
)
```

Arguments

df	A data frame with at least three columns: the first column for terms, the second for frequencies, and the third for proportions.
size.x	Numeric. Font size for axis text. Default is 20.
size.text	Numeric. Font size for text labels on bars. Default is 6.
add_range	Numeric. Additional range added to the maximum frequency for y-axis limits. Default is 100.
x.title	Character. Title for the x-axis. Default is "Items".
flip	Logical. Whether to flip the plot to horizontal orientation. Default is TRUE.
hjust	Numeric. Horizontal adjustment for text labels on bars. Default is -0.1.
vjust	Numeric. Vertical adjustment for text labels on bars. Default is -0.5.
x11_width	Numeric. Width of the x11 window. Default is 8.
x11_height	Numeric. Height of the x11 window. Default is 5.
linetype	Character. Line type for the optional line. Default is "dashed".
linecolor	Character. Color of the optional line. Default is "gray50".
show_line	Logical. Whether to display a line connecting the bars. Default is FALSE.
levels	Character vector. Custom order for categories on the x-axis. Default is c(" 전혀아니다", " 아니다", " 중간이다", " 그렇다", " 매우그렇다").
sort	Logical. Whether to sort the bars by frequency in descending order. Default is FALSE.

Value

A bar plot displayed in an x11 window.

Examples

```
## Not run:
data <- data.frame(term = c("1", "2", "3", "4"), Freq = c(250, 66, 215, 364),
                   `prop(%)` = c(27.9, 7.37, 24.0, 40.7))
table_df_bar(data)

## End(Not run)
```

---

table_df_bar2 *Bar and Line Plot for Frequency Data*

Description

This function creates a bar and line plot using term frequency and proportion data.

Usage

```
table_df_bar2(
  df,
  size.x = 16,
  size.text = 4,
  add_range = 100,
  x.title = "Items",
  hjust = -0.1,
  vjust = -0.5,
  x11_width = 8,
  x11_height = 5,
  linetype = "dashed",
  linecolor = "gray50",
  show_line = TRUE,
  remove_bar = FALSE,
  sort = FALSE,
  text_angle = 45
)
```

Arguments

df	A data frame with three columns: term (character), Freq (numeric)
size.x	Numeric. Font size for x-axis labels. Default is 16.
size.text	Numeric. Font size for text labels inside the plot. Default is 4.
add_range	Numeric. Extra space added to the top of the y-axis. Default is 100.
x.title	Character. Title for the x-axis. Default is "Items".
hjust	Numeric. Horizontal justification for text labels. Default is -0.1.
vjust	Numeric. Vertical justification for text labels. Default is -0.5.
x11_width	Numeric. Width of the X11 plot window. Default is 8.
x11_height	Numeric. Height of the X11 plot window. Default is 5.
linetype	Character. Line type used for connecting points. Default is "dashed".
linecolor	Character. Line color used for connecting points. Default is "gray50".
show_line	Logical. Whether to show the connecting line. Default is TRUE.
remove_bar	Logical. Whether to remove the bar plot. Default is FALSE.
sort	Logical. Whether to sort the data in descending order of frequency. Default is FALSE.
text_angle	Numeric. Angle for the x-axis text. Default is 45.

Value

None. This function opens a new graphics window and displays a plot.

Examples

```
## Not run:
df <- data.frame(term = c("A", "B", "C"),
                 Freq = c(50, 30, 40),
                 `prop(%)` = c(50, 30, 40))
table_df_bar2(df, sort = TRUE)
```

```
## End(Not run)
```

`table_df_colnames` *table_df_colnames*

Description

table_df_colnames

Usage

```
table_df_colnames(df, input = "구분")
```

Arguments

`df`	df
`input`	" 구분

Value

df

`table_df_multi` *Table and Summary Function for Data Frames*

Description

This function processes a data frame to clean missing rows, calculates the frequency and proportion of terms, and provides different outputs based on the selected type.

Usage

```
table_df_multi(df, type = "res", digits = 4, msg = TRUE)
```

Arguments

`df`	A data frame to process.
`type`	A character string indicating the type of result to return. Options are: • `"res"`: Returns the frequency table (default). • `"freq_table"`: Returns the frequency table. • `"missing"`: Returns the list of missing rows. • `"missing_list"`: Returns the list of missing rows. • `"summary"`: Returns a summary of the frequency table. • `"data"`: Returns the cleaned data frame.
`digits`	An integer specifying the number of decimal places to round proportions. Default is 4.
`msg`	A logical value. If `TRUE`, displays messages about the process. Default is `TRUE`.

Value

Depending on the `type` parameter:

- A frequency table (`"res"`, `"freq_table"`, or `"summary"`).
- A list of indices of missing rows (`"missing"` or `"missing_list"`).
- The cleaned data frame (`"data"`).

Examples

```
## Not run:
# Example: Survey Data with Repeated Questions
# Survey data includes repeated questions with different phrasing
survey_data <- data.frame(
  question1 = c("Yes", "No", "Yes", "", "No"),
  question2 = c("No", "Yes", "Yes", "", ""),
  question3 = c("Yes", "", "No", "Yes", "No")
)

# Process the survey data to identify duplicate responses
result <- table_df_multi(survey_data, type = "res", digits = 2, msg = TRUE)

# Output the frequency table
print(result)

# Additional example to find missing rows
missing_info <- table_df_multi(survey_data, type = "missing", msg = FALSE)
print(missing_info)

## End(Not run)
```

`table_freq` *Frequency analysis table*

Description

Frequency analysis table

Usage

```
table_freq(
  data,
  ...,
  prop = TRUE,
  plot = FALSE,
  angle = 0,
  size_text = 4,
  size_axis = 12,
  legend.position = "top",
  reorder = FALSE,
  type = "res"
)
```

Arguments

data	data.frame
...	vars If you input varabale names ”var1”, ”var2” ,...
prop	proportion value, default TRUE
plot	plot
angle	axis text angle
size_text	default 3, bargraph text
size_axis	default 12, axis test
legend.position	legend.position=”” none. top, bottom, left, right
reorder	TRUE is plot x-variable reoder
type	all, res=data, g=plot=graph

Value

Freqency table

Examples

```
## Not run:
Mtcars = mtcars
Mtcars$am = factor(Mtcars$am, levels=c(0,1), labels= c("automatic","manual" ))
Mtcars$vs  = factor(Mtcars$vs, levels=c(0,1), labels= c("V-shaped","straight" ))
Mtcars$cyl  = factor(Mtcars$cyl, levels=c(4,6,8), labels= c("cyl-4","cyl-6","cyl-8"))

Mtcars %>% table_freq("am")
Mtcars %>% table_freq("am", prop=F)
Mtcars %>% table_freq("vs", plot=FALSE)

Mtcars %>% table_freq("vs")
Mtcars %>% table_freq("vs","am")

Mtcars %>% table_freq("vs","am","cyl")

Mtcars %>% table_freq("vs","am","cyl", angle=90)
Mtcars %>% table_freq("vs","am","cyl", angle=90, reorder = TRUE)

Mtcars %>% table_freq("vs","am","cyl", angle=90)

##error we need option  type = "g"
Mtcars %>% table_freq("vs","am","cyl", angle=90, reorder = TRUE)+ylim(0,14)
Mtcars %>% table_freq("vs","am","cyl", angle=90, reorder = TRUE, type="g")+ylim(0,14)

Mtcars %>% table_freq("vs","am") %>% arrange(am)
Mtcars %>% table_freq("vs","am") %>% arrange(vs)
Mtcars %>% table_freq("vs","am") %>% arrange(Freq)

#ONly data : $result  $g -> NULL
 mtcars %>% table_freq("vs","am", type = "all", plot = F)

# this NULL
mtcars %>% table_freq("vs","am", plot = F, type="g")
```

```
## End(Not run)
```

`table_reorder` *Reorder rows of data*

Description

Reorder rows of data

Usage

```
table_reorder(data, reorder = 1:nrow(data))
```

Arguments

`data`	data.fram
`reorder`	reorder=c(select row order)

Value

data

Examples

```
## Not run:
#'
# Example data
data0 <- data.frame(
  source = c("A05", "A08", "A04", "A06",
  "A09", "A11", "A13", "A02", "A03", "A01",
  "A14", "A15", "A07", "A12", "A10",
  "eigen_value", "Proportion_Var", "Cumulative_Var"),
  MR1 = c("", "", "", "", "", "", "", "0.362",
  "0.449", "0.457", "0.47", "0.538", "0.62",
  "0.792", "0.96", "3.363", "0.224", "0.224"),
  MR2 = c("", "0.382", "", "", "0.445", "0.724",
  "0.906", "0.392", "", "", "0.487", "", "", "",
   "", "2.095", "0.14", "0.364"),
  MR3 = c("", "", "0.402", "0.963", "", "", "",
  "", "", "", "", "", "", "", "1.691",
  "0.113", "0.477")
)

# 함수 테스트
new_order <- c(2, 3, 1, 4:16)
# 1번 columns backwards, and the rest in order
table_reorder(data0, new_order)

## End(Not run)
```

`table_sum` *Functions to sum parts of a table*

Description

Functions to sum parts of a table

Usage

```
table_sum(data, ..., trans = FALSE, sep = "_")
```

Arguments

`data`	table,data.frame
`...`	contrast excon= c(1, 1, -1, -1)
`trans`	data transpose, con must be entered to match the row in the data
`sep`	combine sep =”/’

Value

table

Examples

```
## Not run:
#'
#'
data <- data.frame(T = c(30, 40, 60, 120),
                   S = c(50, 70, 60, 80))
rownames(data) <- LETTERS[1:4]

data = as.matrix(data)


table_sum(data, con= c(1, 1, -1, -1))
table_sum(data, con= c(1, 1, 2, -1))
table_sum(data, con= c(1, 1, 2, 3))
table_sum(data, con= c(0, 1, 2, 1))
table_sum(data, con= c(0, 1, 0, 2))

New = data.frame(
  row.names= c('No_Nicotine_Dependence', 'Nicotine_Dependence'),

  "A" = c(130, 119),
  "B" = c(210, 267),
  "C" = c(43, 91),
  "D" = c(114, 254),
  "E" = c(20, 67)
);New

t(New)%>%table_sum(c(1,1,3,2,2))
```

```
New %>%table_sum(c(1,1))
t(New)
t(New)%>%table_sum(c(1,1,1,2,2))

New%>%table_sum(c(1,1,3,2,2), trans=TRUE)
New%>%table_sum(1,1,3,2,2, trans=TRUE)
New%>%table_sum(c(1,1), trans=FALSE)


## End(Not run)
```

`text2df` *Text to data.frame /It is characterized by the use of blank spaces to separate content.*

Description

Text to data.frame /It is characterized by the use of blank spaces to separate content.

Usage

```
text2df(text, header = TRUE, type = "res", add_vars = NULL)
```

Arguments

`text`	text(Row variable names must not contain spaces.)
`header`	header = TRUE
`type`	res, check
`add_vars`	additinal factor

Value

table data.frame

Examples

```
## Not run:
#'
text <- "변인 평균 표준편차 왜도 첨도
성취감 4.24 0.73 -0.82 0.39
인정감 4.19 0.60 -0.36 -0.42
책임감 4.36 0.56 -0.61 -0.29
발전성 3.65 0.89 -0.50 -0.04
감독 3.59 0.88 -0.46 0.13
보수 3.56 0.90 -0.41 -0.18
대인관계 3.75 0.81 -0.14 -0.54
작업환경 3.83 0.75 -0.21 -0.44
안정성 3.28 1.08 -0.28 -0.68
감정이입 3.96 0.72 -0.40 -0.33
직무적합 4.00 0.75 -0.35 -0.78
```

```
일체감 4.02 0.75 -0.59 -0.13
현학원재직 3.97 0.79 -0.77 0.37
타학원이직 2.99 1.06 -0.41 -0.77
타직종이직 2.76 1.07 -0.13 -0.70"

text2df(text)

text2df(text, vars=c(rep("동기요인", 4), rep("위생요인",5), rep("개인요인",6)  ))

text2df(text, T)%>% make_df_text()

#direct output
"var B se b t
성취감 -.03 .09 -.03 -.33
인정감 .24 .09 .20 2.50
책임감 .30 .08 .23 3.56***
발전성 -.08 .06 -.10 -1.29
감독 .06 .05 .08 1.35
보수 -.14 .07 -.18 -2.01*
대인관계 .06 .06 .06 .89
작업환경 .33 .09 .34 3.82***
안정성 .17 .05 .26 3.42**
성별 -.05 .09 -.03 -.52
연령 -.20 .07 -.22 -3.01**
학력 .05 .07 .04 .70
소득 .00 .04 .01 .12
종사기간 .17 .06 .23 2.71**
기관규모 -.10 .03 -.20 -3.36**" %>%text2df()

reg_text = "var B se b t
성취감 -.03 .09 -.03 -.33
인정감 .24 .09 .20 2.50
책임감 .30 .08 .23 3.56***
발전성 -.08 .06 -.10 -1.29
감독 .06 .05 .08 1.35
보수 -.14 .07 -.18 -2.01*
대인관계 .06 .06 .06 .89
작업환경 .33 .09 .34 3.82***
안정성 .17 .05 .26 3.42**
성별 -.05 .09 -.03 -.52
연령 -.20 .07 -.22 -3.01**
학력 .05 .07 .04 .70
소득 .00 .04 .01 .12
종사기간 .17 .06 .23 2.71**
기관규모 -.10 .03 -.20 -3.36**"

text2df(reg_text)
#'
text2df(reg_text) %>% tibble()

#text to numeric
text2df(reg_text)%>% Char2num() %>% tibble()

text2df(reg_text)%>% Char2num(iter=2:4) %>% tibble()
#'

text ="d           Estimate  Std.Err  z-value  P(>|z|)
```

```
  read~ppsych           -0.275    0.037   -7.385    0.000
    read~motiv             0.461    0.037   12.404    0.000
arith~motiv             0.600    0.036   16.771    0.000
"
text2df(text)
text2df(text) %>% tibble::tibble()

text ="             Estimate Std.Error tvalue Pr(>|t|)
(Intercept) -1.336e-07  3.608e-01   0.000        1
ppsych      -2.747e-01  3.730e-02  -7.363  7.51e-13***
motiv        4.613e-01  3.730e-02  12.367  <2e-16***
"
text2df(text) %>%tibble()
text2df(text) %>%make_df_text()


text ="                   Estimate  Std.Err  z-value  P(>|z|)

    read~ppsych           -0.275    0.037   -7.385    0.000
    read~motiv             0.461    0.037   12.404    0.000

    arith~ppsych           -0.096    0.037   -2.616    0.009
     arith~motiv             0.576    0.037   15.695    0.000
"
text2df(text)



text =" 과장 대리 부장 사원 주임 차장 SUM
남자 30(39.5%) 21(38.9%) 34(79.1%) 30(49.2%)  10(37%)     28(71.8%) 153
여자 46(60.5%) 33(61.1%) 9(20.9%) 31(50.8%) 17(63%)    11(28.2%) 147
SUM 76 54 43 61 27 39 300
"
text2df(text)
#'
## End(Not run)
```

`TE_effect` *Total effect*

Description

Total effect

Usage

```
TE_effect(model_sem, effect1 = "TE", effect2 = "te", label = FALSE, ci = F)
```

Arguments

`model_sem`	lavaan objgect
`effect1`	”IE”
`effect2`	”IE”

label	label
ci	confidence interval

Value

parameterEstimates()

Examples

```
## Not run:
#library(lavaan) # only needed once per session

models1 <- '
  # measurement model
    ind60 =~ x1 + x2 + x3
    dem60 =~ y1 + y2 + y3 + y4
    dem65 =~ y5 + y6 + y7 + y8
    # regressions
    dem60 ~ ind60
    dem65 ~ ind60 + dem60
      # residual correlations
    y1 ~~ y5
    y2 ~~ y4 + y6
    y3 ~~ y7
    y4 ~~ y8
    y6 ~~ y8

models2 ='
  # measurement model
    ind60 =~ x1 + x2 + x3
    dem60 =~ y1 + y2 + y3 + y4
    dem65 =~ y5 + y6 + y7 + y8
     # regressions
    dem60 ~ a1*ind60
    dem65 ~ a2*ind60 + a3*dem60
 #NEw
   ie := a1*a3
   te := a2 +a1*a3
   diff := a1 - a3




fits1 <- sem(models1, data=PoliticalDemocracy)
fits2 <- sem(models2, data=PoliticalDemocracy)
summary(fits1, standardized=TRUE)
summary(fits2, standardized=TRUE)
fits2 %>% TE_effect()

## End(Not run)
```

`to_long` *longdata transfomation*

Description

longdata transfomation

Usage

```
to_long(
  data,
  names_to = "names",
  values_to = "values",
  cols = NULL,
  fix = NULL,
  rownames_to_column = FALSE,
  rowname = "rows"
)
```

Arguments

`data`	widedata
`names_to`	name
`values_to`	value
`cols`	colrange
`fix`	column fixed (thereofore not using cols)
`rownames_to_column`	rownames_to_column=FALSE
`rowname`	defai;t acce

Value

longdata

Examples

```
## Not run:

mtcars[,1:4] %>%to_long("car", "value")
mtcars[,1:4] %>%to_long("car", "value", fix="cyl")
mtcars[,1:4] %>%to_long("car", "value", fix= 1:2)
mtcars[,1:4] %>%to_long("car", "value", fix= c(1))
mtcars[,1:4] %>%to_long("car", "value", fix= c(2))
mtcars[,1:4] %>%to_long("car", "value", fix=c("cyl","mpg"))
mtcars[,1:6] %>%long_df("car", "value", rownames_to_column = F, cols = c(1,3:6))









## End(Not run)
```

`to_table` *data.frame to table*

Description

data.frame to table

Usage

```
to_table(data, sel = ncol(data) - 1, value = ncol(data), type = "mat")
```

Arguments

`data`	data.frame
`sel`	select column (names_to)
`value`	values_to
`type`	mat(matrix), df(data.frame)

Value

table

Examples

```
## Not run:
data <- data.frame(am = c(0, 1, 0, 1),
                   vs = c(0, 0, 1, 1),
                   Freq = c(12, 6, 7, 7))
data
#make  table
to_table(data)

mtcars %>%select(am, vs) %>% table()

mtcars %>%select(am, vs) %>% table() %>%data.frame()

mtcars %>%select(am, vs) %>% table() %>%data.frame() %>%to_table()

mtcars %>%select(am, vs) %>% table() %>%data.frame() %>%to_table()%>%addmargins()

mtcars %>%select(am, vs) %>% table() %>%data.frame() %>%to_table()%>%add_ratio()

mtcars %>%select(am, vs) %>% table() %>%data.frame() %>%
to_table()%>%add_ratio()%>%as.matrix()%>%addmargins()

mtcars %>%select(am, vs) %>% table() %>%data.frame() %>%to_table()%>%add_ratio_df()

mtcars %>%select(am, vs) %>% table() %>%data.frame() %>%to_table()%>%add_sum()

mtcars %>%select(am, vs) %>% table() %>%data.frame() %>%to_table(sel="vs")

mtcars %>%select(am, cyl) %>% table()
```

```
mtcars %>%select(am, cyl) %>% table() %>%data.frame()

mtcars %>%select(am, cyl) %>% table() %>%data.frame() %>%to_table()

mtcars %>%select(am, cyl) %>% table() %>%data.frame() %>%to_table()%>%add_ratio()

mtcars %>%select(am, cyl) %>% table() %>%data.frame() %>%to_table()%>%add_ratio_df()

mtcars %>%select(cyl, am) %>% table() %>%data.frame() %>%to_table() %>%add_sum()

mat = matrix(c(10, 50, 10, 28, 22, 16, 68,12, 14, 124, 116, 120),
ncol = 4,
dimnames = list(c("20s","30s","40s"),
                c("A","B","C","D")))
mat

mat  %>% long_df("age","brand","count") %>%to_table()
# mean data to table
aggregate(len ~ ., data = ToothGrowth, mean) %>% xtabs(formula = len ~.)
#this is jjstat method
aggregate(len ~ ., data = ToothGrowth, mean) %>% to_table()

aggregate(len ~ ., data = ToothGrowth, mean) %>% to_table("dose","len")

#same method
aggregate(len ~ ., data = ToothGrowth, mean) %>% xtabs(len ~ supp + dose, data =.)


## End(Not run)
```

`to_wide` *pivot_wider*

Description

pivot_wider

Usage

```
to_wide(
  data,
  names_from = "names",
  values_from = "values",
  names_prefix = "",
  names_sep = "_",
  names_sort = FALSE,
  rm_cols = NULL,
  add_id = NULL
)
```

Arguments

`data`	long format data

names_from	names_from
values_from	values_from
names_prefix	names_prefix +variable
names_sep	sep
names_sort	colnames sort
rm_cols	Remove variables that interfere with wide format by creating individual IDs
add_id	Select Repeating Groups to create IDs, line in column names or column numbers

Value

wide data

Examples

```
## Not run:
mtcars[1:3, 1:7]
#this is error
mtcars[1:4, 1:5] %>% to_long() %>% wide_df()

mtcars[1:3, 1:7] %>% add_id()%>% to_wide()
mtcars[1:3, 1:7] %>% to_wide(add_id=TRUE)
mtcars[1:3, 1:7] %>% to_wide(cols= 3:7)
mtcars[1:3, 1:7] %>% to_wide(fix=1:2)
mtcars[1:3, 1:7] %>% to_wide(fix= 1:3, rownames_to_column=F)

mtcars[1:3, 1:7] %>% to_wide() %>% mutate(ik = "bc", io = "a")
mtcars[1:3, 1:7] %>% to_wide() %>% mutate(ik = "bc", io = "a")%>% to_wide()

mtcars[1:3, 1:7] %>% to_wide() %>%
  mutate(ik = "bc", io = "a")%>%
  to_wide(names_from = c("names","ik"))

mtcars[1:3, 1:7] %>% to_wide() %>%
  mutate(ik = rownames, io = "a")%>%
  to_wide(names_from = c("ik"))%>%pall()

mtcars[1:3, 1:7] %>% to_wide() %>%
  mutate(ik = "bc", io = "a")%>%
  to_wide(names_from = 2)



#일반적인 데이터에서  롱포맷으로 바꾸기
data <- data.frame(
  weight = c(4.17, 5.58, 5.18, 6.11, 4.50, 4.61, 5.17, 4.53, 5.33, 5.14,
             4.81, 4.17, 4.41, 3.59, 5.87, 3.83, 6.03, 4.89, 4.32, 4.69,
             6.31, 5.12, 5.54, 5.50, 5.37, 5.29, 4.92, 6.15, 5.80, 5.26),
  group = c(rep("ctrl", 10), rep("trt1", 10), rep("trt2", 10))
)
data
data%>%add_rep_id("group")
```

```
data%>%add_rep_id("group")%>%jjstat::as_trt("id") %>%wide_df("group", "weight")

data  %>%wide_df("group", "weight", add_id="group")


#개별적인 그룹을 만들어진 경우
data2 <- data.frame(
  weight = c(4.17, 5.58, 5.18, 6.11, 4.50, 4.61, 5.17, 4.53, 5.33, 5.14,
             4.81, 4.17, 4.41, 3.59, 5.87, 3.83, 6.03, 4.89, 4.32, 4.69,
             6.31, 5.12, 5.54, 5.50, 5.37, 5.29, 4.92, 6.15, 5.80, 5.26),
  ids = c(paste0("C", 1:10), paste0("T",1:10), paste0("S", 1:10))
)

data2

data2 %>%jjstat::mutate_col(group= c(rep("ctrl", 10),
                 rep("trt1", 10), rep("trt2", 10)), col=2)

# 그룹을 나누어주는 아이디 생성
data2 %>%
jjstat::mutate_col(group= c(rep("ctrl", 10),
        rep("trt1", 10), rep("trt2", 10)), col=2)%>%
        add_rep_id("group")

# 개별적인 그룹을 만드는 열이 존재하는 경우 --> 문제발생
data2 %>%
    jjstat::mutate_col(group= c(rep("ctrl", 10),
          rep("trt1", 10), rep("trt2", 10)), col=2)%>%
          wide_df("group", "weight", add_id="group")

#개별적인 그룹을 만드는 열을 제거하는 경우 --> 문제해결
data2 %>%
  jjstat::mutate_col(group= c(rep("ctrl", 10),
  rep("trt1", 10), rep("trt2", 10)),  col=2)%>%
  wide_df("group", "weight", add_id="group", rm_cols="ids") %>%print(n=Inf)
#'
data3= data.frame(
  weight=c(1:9),
  group=c(rep("a",3),rep("b",3),rep("c",3))
)
data3
data3 %>% to_wide("group", "weight", add_id=2)


## End(Not run)
```

`ttest_df` *Repeat t.test to create a table*

Description

Repeat t.test to create a table

Usage

```
ttest_df(
  data,
  dv_var = NULL,
  iv_vars = NULL,
  grp_mean = TRUE,
  unite_t = FALSE,
  unite_p = FALSE,
  digits = 2,
  var.equal = TRUE,
  paired = FALSE,
  sig = FALSE
)
```

Arguments

data	data. fram
dv_var	ddpendent variable
iv_vars	Independent variables
grp_mean	mean each level
unite_t	tvalue+sig
unite_p	p_vakye+sig
digits	digits 2
var.equal	variance equal -student ttest
paired	paired t.test (TRUE)
sig	star

Value

t.test table

Examples

```
## Not run:

ttest_df(data = mtcars, dv_var = "mpg", iv_vars = c("am", "vs"))

ttest_df(data = mtcars, dv_var = "mpg", iv_vars = c("am", "vs"), sig=T)


ttest_df(data = mtcars, dv_var = "mpg", iv_vars = c("am", "vs"), sig=T, unite_t=TRUE)

ttest_df(data = mtcars, dv_var = "mpg", iv_vars = c("am", "vs"), sig=T, unite_p=TRUE)

ttest_df(data = mtcars, dv_var = "mpg", iv_vars = c("am", "vs"), grp_mean=FALSE)



## End(Not run)
```

t_sep *t_sep separate t value and star*

Description

t_sep separate t value and star

Usage

```
t_sep(data)
```

Arguments

data	data

Value

add column

Examples

```
## Not run:
#'
reg_text = "var B se b t
성취감 -.03 .09 -.03 -.33
인정감 .24 .09 .20 2.50
책임감 .30 .08 .23 3.56***
발전성 -.08 .06 -.10 -1.29
감독 .06 .05 .08 1.35
보수 -.14 .07 -.18 -2.01*
대인관계 .06 .06 .06 .89
작업환경 .33 .09 .34 3.82***
안정성 .17 .05 .26 3.42**
성별 -.05 .09 -.03 -.52
연령 -.20 .07 -.22 -3.01**
학력 .05 .07 .04 .70
소득 .00 .04 .01 .12
종사기간 .17 .06 .23 2.71**
기관규모 -.10 .03 -.20 -3.36**"


text2df(reg_text)

text2df(reg_text) %>%t_sep()


## End(Not run)
```

`t_test_report` *Advanceed t test report function*

Description

Advanceed t test report function

Usage

```
t_test_report(
  data,
  iv = NULL,
  dv = NULL,
  type = "all",
  xlab = "independent Variable",
  ylab = "dependent Variable",
  translate = FALSE,
  mu = 0,
  paired = FALSE,
  conf.level = 0.95
)
```

Arguments

`data`	data.frame
`iv`	independent variable, if you want to change the name, you can change the setting
`dv`	dependent variable, if you want to change the name, you can change the setting
`type`	type option default is 'all'. and 'var.test', 'var.test.report', 't.test','t.test.report', 'boxplot', 'var.test.full', 't.test.full', 'descriptive' output each result
`xlab`	graph name dependent variable
`ylab`	graph name dependent variable
`translate`	report translate defualt FALSE, TRUE is korean
`mu`	a number indicating the true value of the mean (or difference in means if you are performing a two sample test).
`paired`	a logical indicating whether you want a paired t-test. var.equal a logical variable indicating whether to treat the two variances as being equal. If TRUE then the pooled variance is used to estimate the variance otherwise the Welch (or Satterthwaite) approximation to the degrees of freedom is used.
`conf.level`	confidence level of the interval.

Examples

```
# mtcars data
## Not run:
##default analysis
t_test_report(mtcars, iv="vs",dv="mpg")
```

```
##$var_test_report

##
##$t_test_result_tibble
## A tibble: 1 × 7
##IV    DV       df t.value   p.value sig   test_type
##<chr> <chr> <dbl>   <dbl>     <dbl> <chr> <chr>
##  1 vs    mpg      30   -4.86 0.0000342 ***   student t.test
##
##$t_test_report
## New namaing xlab and ylab
t_test_report(mtcars, iv="vs",dv="mpg",xlab="transmission", ylab= "males per galon")

## End(Not run)
```

`unCombine` *Functions to uncombind combind_data again*

Description

Functions to uncombind combind_data again

Usage

```
unCombine(
  data,
  into = c("value", "sig"),
  sep = " ",
  type = "all",
  pattern = NULL,
  imp = ""
)
```

Arguments

`data`	conbind data
`into`	column name
`sep`	separate sep= ” ”
`type`	”all”, res1 = m1, res2=m2
`pattern`	remove tail
`imp`	imputation tail

Value

results two data

Examples

```
## Not run:

Generate example data2
combined_data2 <- data.frame(
  "H" = c("1.0587162 ***", "1.1269248 ***", "0.9163776 ***", "0.9530237 ***"),
  "H(H)" = c("1.1477347 ***", "0.5743712 ***", "1.4060606 ***", "0.8624314 ***"),
  "L" = c("0.50877193 ***", "1.94957983 *", "0.03515152 ***", "1.44531611 **")
)
# Generate example data2
combined_data <- data.frame(
  "H" = c("1.0587162(***)", "1.1269248(***)", "0.9163776(***)", "0.9530237(***)"),
  "H(H)" = c("1.1477347(***)", "0.5743712(***)", "1.4060606(***)", "0.8624314(***)"),
  "L" = c("0.50877193(***)", "1.94957983(*)", "0.03515152(***)", "1.44531611(**)")
)

combined_data2
unCombind(combined_data2,sep=" ")

combined_data
unCombind(combined_data, sep="\\(", pattern=")")

#iris cor
iris_cor = iris %>% select(-Species) %>%psych::corr.test()
iris_cor$r
iris_cor$p

combine_data(iris_cor$r%>%round(2),
             iris_cor$p%>%round(5),
             left="(", right=")")
 sig = add_significance_symbols(iris_cor$p)
 sig

iris_cor_combine =combine_data( round(iris_cor$r, 3) , sig)
iris_cor_combine

 combine_data( round(iris_cor$r, 3) , sig) %>%
 lowerMat(diag = 1, fill="")







## End(Not run)
```

`uncombine_data` *Functions to uncombind combind_data again*

Description

Functions to uncombind combind_data again

Usage

```
uncombine_data(
  data,
  into = c("value", "sig"),
  sep = " ",
  type = "res",
  pattern = NULL,
  imp = ""
)
```

Arguments

data	conbind data
into	column name
sep	separate sep= ” ”
type	”all”, res1 = m1, res2=m2
pattern	remove tail
imp	imputation tail

Value

results two data

Examples

```
## Not run:

Generate example data2
combined_data2 <- data.frame(
  "H" = c("1.0587162 ***", "1.1269248 ***", "0.9163776 ***", "0.9530237 ***"),
  "H(H)" = c("1.1477347 ***", "0.5743712 ***", "1.4060606 ***", "0.8624314 ***"),
  "L" = c("0.50877193 ***", "1.94957983 *", "0.03515152 ***", "1.44531611 **")
)
# Generate example data2
combined_data <- data.frame(
  "H" = c("1.0587162(***)", "1.1269248(***)", "0.9163776(***)", "0.9530237(***)"),
  "H(H)" = c("1.1477347(***)", "0.5743712(***)", "1.4060606(***)", "0.8624314(***)"),
  "L" = c("0.50877193(***)", "1.94957983(*)", "0.03515152(***)", "1.44531611(**)")
)

combined_data2
unCombind(combined_data2,sep=" ")

combined_data
unCombind(combined_data, sep="\\(", pattern=")")



## End(Not run)
```

unCount *Functions that generate rows with a frequency*

Description

Functions that generate rows with a frequency

Usage

```
unCount(data, type = "real", sel = ncol(data), fix = NULL)
```

Arguments

data	data
type	res, all
sel	frequency column
fix	final select column

Value

data

Examples

```
## Not run:
dff = data.frame(
  trt = c("r","s","z"),
  con = c("f","m","l"),
  num = c(10,0,2),
  fre = c(4,0,5) )
dff
dff %>%unCount()
dff %>%unCount()
dff %>%unCount(fix=1)
dff %>%unCount(type= "res", fix=1:2)
dff %>%unCount(type= "res", fix="con")
dff %>%unCount(type= "all")
dff %>%unCount(type= "real")
dff %>%unCount(type= "zero")
dff %>%unCount(type= "zero", fix="con")

# Making frequency data into real data
dff %>%unCount()%>% add_rows_freq()
 dff_real =  dff %>%unCount()%>% add_rows_freq()
 dff_real

 freq_data =  mtcars %>% select(am, vs) %>% table()
 freq_data%>% data.frame()
 freq_data%>% data.frame() %>% unCount(type ="zero")
 freq_data%>% data.frame() %>% unCount(type ="zero") %>%table()
rowid_to_column
```

```
## End(Not run)
```

`Unite` *UInte data*

Description

UInte data

Usage

```
Unite(data, col1, col2, Col = NULL, sep = "", remove = TRUE)
```

Arguments

`data`	data
`col1`	select col1
`col2`	select col2
`Col`	col names
`sep`	""
`remove`	true
`left`	" or "["
`right`	"" or "]"

Value

data

Examples

```
## Not run:
#'
# 예시 데이터 생성
data <- data.frame(
  block = c("IMAG", "IMAG", "IMAG", "IMAG"),
  name = c("imag1", "imag2", "imag3", "imag4"),
  Original = c("0.7093265 ***", "0.8773078 ***", "0.8417485 ***", "0.5691934 ***"),
  sig = c("***", "***", "***", "***"),
  Mean.Boot = c(0.7088215, 0.8750787, 0.8401757, 0.5647333),
  Std.Error = c(0.05949724, 0.02402834, 0.02970117, 0.07686419),
  perc.025 = c(0.5774529, 0.8231642, 0.7784983, 0.3876417),
  perc.975 = c(0.8105249, 0.9146114, 0.8863100, 0.6961572)
)

# 예시 호출

# Unite using column numbers
result1 <- Unite(data, 3, 4)
print(result1)

# Unite using column numbers with a new column name
```

```
result2 <- Unite(data, 3, 4, "New")
print(result2)

# Unite using column names
result3 <- Unite(data, "Original", "sig")
print(result3)

# Unite using column names with a new column name
result4 <- Unite(data, "Original", "sig", "New")
print(result4)



## End(Not run)
#
```

`unite_ci` *unite_ci if you add ci*

Description

unite_ci if you add ci

Usage

```
unite_ci(
  data,
  col1 = ncol(data) - 1,
  col2 = ncol(data),
  left = "[",
  right = "]",
  digits = 3,
  colname = "95%CI",
  remove = TRUE
)
```

Arguments

`data`	A data frame containing the data
`col1`	lower
`col2`	upper
`left`	lefe (
`right`	right)
`digits`	round3
`colname`	”95CI”
`remove`	orinal column

Value

data

Examples

```
## Not run:


# Sample data to simulate the scenario
data <- data.frame(
  paths = c('자기효능감-C_S1', '자기효능감-C_S2', '자기효능감-C_S3'),
  Original = c(0.883, 0.826, 0.794),
  Mean.Boot = c(0.885, 0.833, 0.8),
  Std.Error = c(0.022, 0.041, 0.039),
  t = c('40.771***', '20.22***', '20.612***'),
  perc.025 = c(0.837, 0.733, 0.725),
  perc.975 = c(0.923, 0.893, 0.872)
)

# Apply the function to the dataframe with remove = TRUE
unite_ci(data, col1 = 6, col2 = 7, remove = TRUE)

# Apply the function to the dataframe with remove = FALSE
unite_ci(data, col1 = 'perc.025', col2 = 'perc.975', remove = FALSE)


## End(Not run)
```

`Unite_col` *Unite Columns in a Data Frame*

Description

Unite Columns in a Data Frame

Usage

```
Unite_col(
  data,
  col1,
  col2,
  Col = NULL,
  sep = ",",
  remove = TRUE,
  left = "[",
  right = "]"
)
```

Arguments

`data`	A data frame.
`col1`	The first column to unite. Can be numeric (column index) or character (column name).
`col2`	The second column to unite. Can be numeric (column index) or character (column name).

`Col`	Optional, name for the new column (default = NULL).
`sep`	Separator string to use between the combined values (default = ",").
`remove`	Logical, whether to remove the original columns after uniting (default = TRUE).
`left`	String to add to the left side of the combined values (default = "[").
`right`	String to add to the right side of the combined values (default = "]").

Value

A data frame with the combined column.

Examples

```
## Not run:
# Example data
data <- data.frame(
  block = c("IMAG", "IMAG", "IMAG", "IMAG"),
  name = c("imag1", "imag2", "imag3", "imag4"),
  Original = c("0.7093265 ***", "0.8773078 ***", "0.8417485 ***", "0.5691934 ***"),
  sig = c("***", "***", "***", "***"),
  Mean.Boot = c(0.7088215, 0.8750787, 0.8401757, 0.5647333),
  Std.Error = c(0.05949724, 0.02402834, 0.02970117, 0.07686419),
  perc.025 = c(0.5774529, 0.8231642, 0.7784983, 0.3876417),
  perc.975 = c(0.8105249, 0.9146114, 0.8863100, 0.6961572)
)

# Unite using column numbers
result1 <- Unite_col(data, 3, 4)
print(result1)

# Unite using column numbers with a new column name
result2 <- Unite_col(data, 3, 4, "New")
print(result2)

# Unite using column names
result3 <- Unite_col(data, "Original", "sig")
print(result3)

# Unite using column names with a new column name
result4 <- Unite_col(data, "Original", "sig", "New")
print(result4)

## End(Not run)
```

`Unite_confint` *Unite Columns Confint in a Data Frame*

Description

Unite Columns Confint in a Data Frame

Usage

```
Unite_confint(
  data,
  col1,
  col2,
  Col = NULL,
  sep = ", ",
  remove = TRUE,
  left = "[",
  right = "]"
)
```

Arguments

`data`	A data frame.
`col1`	The first column to unite. Can be numeric (column index) or character (column name).
`col2`	The second column to unite. Can be numeric (column index) or character (column name).
`Col`	Optional, name for the new column (default = NULL).
`sep`	Separator string to use between the combined values (default = ",").
`remove`	Logical, whether to remove the original columns after uniting (default = TRUE).
`left`	String to add to the left side of the combined values (default = "[").
`right`	String to add to the right side of the combined values (default = "]").

Value

A data frame with the combined column.

Examples

```
## Not run:
# Example data
data <- data.frame(
  block = c("IMAG", "IMAG", "IMAG", "IMAG"),
  name = c("imag1", "imag2", "imag3", "imag4"),
  Original = c("0.7093265 ***", "0.8773078 ***", "0.8417485 ***", "0.5691934 ***"),
  sig = c("***", "***", "***", "***"),
  Mean.Boot = c(0.7088215, 0.8750787, 0.8401757, 0.5647333),
  Std.Error = c(0.05949724, 0.02402834, 0.02970117, 0.07686419),
  perc.025 = c(0.5774529, 0.8231642, 0.7784983, 0.3876417),
  perc.975 = c(0.8105249, 0.9146114, 0.8863100, 0.6961572)
)

# Unite using column numbers
result1 <- Unite_confint(data, 3, 4)
print(result1)

# Unite using column numbers with a new column name
result2 <- Unite_confint(data, 3, 4, "New")
print(result2)
```

```
# Unite using column names
result3 <- Unite_confint(data, "Original", "sig")
print(result3)

# Unite using column names with a new column name
result4 <- Unite_confint(data, "Original", "sig", "New")
print(result4)

## End(Not run)
```

---

unite_rows 행을결합해주는함수

Description

행을결합해주는함수

Usage

```
unite_rows(data, row1, row2, sep = ", ", left = "[", right = "]")
```

Arguments

data	data
row1	row1 합쳐질곳
row2	row2 가져올곳
sep	sep "," default
left	left "["
right	right"]"

Value

data

Examples

```
## Not run:
#'
data17 <- data.frame(
  criterian = c("Chisq", "df", "p >0.05", "RMSEA <0.05",
                "90%CI.lower", "90%CI.upper", "p <= 0.05",
                "SRMR <0.08", "GFI >0.95", "CFI >0.95", "TLI >0.90", "lower", "lower"),
  Value = c(40.179, 38.000, 0.374, 0.028, 0.000, 0.087,
            0.665, 0.056, 0.920, 0.997, 0.995, 3153.636, 3218.526)
)
data17
# 함수 실행 예시
unite_rows(data17, row1 = 5, row2 = 6)
unite_rows(data17, row1 = 5, row2 = 6)%>%move_row(6,5)
```

```
mtcars[1:5,]
mtcars[1:5,] %>%unite_rows(1,3)

## End(Not run)
```

`validate_residual_cov` *Validate Residual Covariance Specification*

Description

관측변수간잔차공분산설정의유효성을검사한다.

Usage

```
validate_residual_cov(residual_cov, obs_vars)
```

Arguments

`residual_cov`	list. c("y1","y2") 또는 c("y1","y2","0").
`obs_vars`	character. 관측변수이름벡터.

Value

list with elements `pairs`, `fixed`.

`validate_variance` *Validate Variance Constraints*

Description

관측·잠재변수분산고정또는자유설정을검증한다.

Usage

```
validate_variance(variance, obs_vars, latent_vars)
```

Arguments

`variance`	list. c("x","x") 또는 c("f","f","1").
`obs_vars`	character. 관측변수이름.
`latent_vars`	character. 잠재변수이름.

Value

list with elements `free`, `fixed`.

wald_test *wald_test*

Description

wald_test

Usage

```
wald_test(
  b1,
  se1,
  b2,
  se2,
  est1 = NULL,
  est2 = NULL,
  digits = 3,
  type = "res2"
)
```

Arguments

b1	fires coef
se1	firtse se
b2	secoond coef
se2	second se
est1	est1 name
est2	est2 name
digits	round
type	res1 res2,

Value

test result

Examples

```
## Not run:
##Clogg, C. C., Petkova, E., & Haritou, A. (1995).
Statistical methods for comparing regression coefficients between models.
American Journal of Sociology, 100(5), 1261-1293.
##https://blog.naver.com/shoutjoy/223294372137

 #' wald_test(-0.795,-0.795/-1.138, -0.795+0.149,  (-0.795+0.149)/1.344)

wald_test(-0.795,-0.795/-1.138,-0.795+0.149,
          (-0.795+0.149)/1.344,
           est1 = "남성",
           est2 = "여성")
```

```
## End(Not run)


wald_test <- function(b1, se1, b2, se2, est1=NULL, est2=NULL){
  library(broom)
  t1 = b1/se1
  t2 = b2/se2

  z = (b1 - b2)/ sqrt(se1^2 + se2^2)
  p = 2*(1-pnorm(abs(z)))

  var1 = paste0(est1,"_est1 = ", b1,", se = ", round(se1, 3), ", t = ", t1,"." )
  var2 = paste0(est2,"_est2 = ", b2,", se = ", round(se2, 3), ", t = ", t2,"." )

  res = broom::tidy(c(z.value = z,  pvalue = p))
  colnames(res)=c("statistics","value")

  res = res%>% dplry::mutate(coef=c(var1, var2))
  res
  # list(res, var1, var2)
}
```

`wald_test2` *wald_test2 input t value*

Description

wald_test2 input t value

Usage

```
wald_test2(
  b1,
  t1,
  b2,
  t2,
  est1 = NULL,
  est2 = NULL,
  gender = FALSE,
  digits = 3,
  type = "res1"
)
```

Arguments

b1	fires coef
t1	firtse t
b2	secoond coef
t2	second t
est1	est1 name

est2	est2 name
gender	For dichotomous variables with intercepts, the Gender
digits	round
type	res=res1. res2 = all

Value

ztest

Examples

```
## Not run:

wald_test2(-0.795,8.245, -0.795+0.149,  (-0.795+0.149)/1.344)

wald_test2(-0.795,-0.795/-1.138,-0.795+0.149,
           (-0.795+0.149)/1.344,est1="남성",est2="여성")


##gender variable
##first method
wald_test2(1.311, 1.628, 1.311-0.366,  -2.732)
##second method
wald_test2(1.311, 1.628, -0.366,  -2.732, gender=T)
wald_test2(1.311, 1.628, -0.366,  -2.732,"male","female", gender=T)


## End(Not run)
```

---

wald_test_df — *wald_test_df: Wald test for linear model coefficients using a data frame or tibble*

Description

This function performs a Wald test to compare two coefficients from a linear model. It calculates the difference, z-statistic, and p-value between the selected coefficients.

Usage

```
wald_test_df(data, row1 = 2, row2 = 3, rname1 = "", rname2 = "", sel = "row")
```

Arguments

data	An object of class ‘lm‘, representing a fitted linear model.
row1	Integer. The row number of the first coefficient to compare (default = 2).
row2	Integer. The row number of the second coefficient to compare (default = 3).

`rname1`	Character. Name of the first term to select when 'sel = "term"'. Defaults to "".
`rname2`	Character. Name of the second term to select when 'sel = "term"'. Defaults to "".
`sel`	Character. Method for selecting coefficients: "row" (default) for row numbers, or "term" for term names.

Value

A tibble containing the estimated coefficients, standard errors, difference, z-statistic, and p-value.

Examples

```
## Not run:
# Using default row selection
lm(mpg ~ hp + wt + drat, mtcars) %>% wald_test_df()

# Selecting specific rows by number
lm(mpg ~ hp + wt + drat, mtcars) %>% wald_test_df(row1 = 2, row2 = 4)

# Selecting specific terms by name
lm(mpg ~ hp + wt + drat, mtcars) %>% wald_test_df(sel = "term", rname1 = "hp", rname2 = "wt")

## End(Not run)
```

`wald_test_gender` *wald_test2 input t value gender*

Description

wald_test2 input t value gender

Usage

```
wald_test_gender(
  b1,
  t1,
  b2,
  t2,
  est1 = NULL,
  est2 = NULL,
  gender = TRUE,
  digits = 3
)
```

Arguments

`b1`	fires coef
`t1`	firtse t
`b2`	secoond coef

t2	second t
est1	est1 name
est2	est2 name
gender	For dichotomous variables with intercepts, the Gender
digits	round

Value

ztest

Examples

```
## Not run:

wald_test2(-0.795,8.245, -0.795+0.149,  (-0.795+0.149)/1.344)

#Regression: gender variable should consider intercept.
wald_test2(-0.795,-0.795/-1.138,-0.795+0.149,
           (-0.795+0.149)/1.344,est1="남성",est2="여성")


##gender variable
##first method
wald_test_gender(1.311, 1.628, 1.311-0.366,  -2.732)
##second method
wald_test_gender(1.311, 1.628, -0.366,  -2.732, gender=T)
wald_test_gender(1.311, 1.628, -0.366,  -2.732,"male","female", gender=T)


## End(Not run)
```

`wald_test_general` *General Wald Test for Linear Hypotheses*

Description

This function performs a Wald test for general linear hypotheses on a fitted model. The hypothesis can be specified as a string (e.g., '"x1 - x2 + x3 = 0"') or as a numeric vector/matrix (e.g., 'c(1, -1, 1)'). The function provides an option for detailed output using the 'full' parameter.

Usage

```
wald_test_general(
  model,
  hypothesis_matrix,
  rhs = 0,
  test = c("F", "Chisq"),
  vcov_func = NULL,
  full = FALSE
)
```

Arguments

`model`	A fitted model object, such as an object from ‘lm‘ or ‘glm‘.
`hypothesis_matrix`	A hypothesis specified as either a character string (e.g., ‘”x1 - x2 + x3 = 0”‘) or a numeric vector/matrix (e.g., ‘c(1, -1, 1)‘).
`rhs`	Right-hand side of the hypothesis equation. Default is ‘0‘.
`test`	A character string specifying the type of test to perform. Options are ‘”F”‘ (default) or ‘”Chisq”‘.
`vcov_func`	An optional function for specifying a custom covariance matrix. Default is ‘NULL‘.
`full`	Logical. If ‘TRUE‘, returns a tibble with full results. If ‘FALSE‘, prints summary information and returns a simplified tibble. Default is ‘FALSE‘.

Value

A tibble with results. If ‘full = TRUE‘, the tibble includes: - ‘var_comparison‘: The hypothesis expressed as a standardized string. - ‘W‘: The Wald statistic. - ‘df‘: Degrees of freedom for the test. - ‘p_value‘: The p-value associated with the test. - ‘method‘: The type of test performed (‘”F”‘ or ‘”Chisq”‘). - ‘mat‘: A vectorized representation of the hypothesis matrix. If ‘full = FALSE‘, the tibble includes only: - ‘var_comparison‘: The hypothesis expressed as a standardized string. - ‘W‘: The Wald statistic. - ‘df‘: Degrees of freedom for the test. - ‘p_value‘: The p-value associated with the test.

Examples

```
# Example: Linear model with Wald test
set.seed(123)
data <- data.frame(x1 = rnorm(100), x2 = rnorm(100), x3 = rnorm(100), y = rnorm(100))
model <- lm(y ~ x1 + x2 + x3, data = data)

# Perform Wald test with text-based hypothesis
result1 <- wald_test_general(model, "x1 - x2 + x3 = 0", test = "F", full = TRUE)
print(result1)

# Perform Wald test with matrix-based hypothesis
result2 <- wald_test_general(model, c(1, -1, 1), test = "F", full = TRUE)
print(result2)

# Perform Wald test with custom covariance matrix
custom_vcov <- function(model) {
  vcov_matrix <- vcov(model)
  # Example: Inflate variance for demonstration
  vcov_matrix * 1.5
}
result_custom <- wald_test_general(model, c(1, -1, 1), test = "F", vcov_func = custom_vcov, full = TRU
print(result_custom)
```

`wald_test_general_df` *Perform Wald Tests for Multiple Hypotheses*

Description

This function allows multiple hypotheses to be tested on a fitted model. It internally uses ‘wald_test_general‘ to perform each test and combines the results into a single tibble.

Usage

```
wald_test_general_df(
  model,
  ...,
  rhs = 0,
  test = c("F", "Chisq"),
  vcov_func = NULL,
  full = FALSE
)
```

Arguments

`model`	A fitted model object, such as an object from ‘lm‘ or ‘glm‘.
`...`	Hypotheses to test. Each hypothesis can be specified as a character string (e.g., ‘"x1 - x2 + x3 = 0"‘) or a numeric vector/matrix (e.g., ‘c(1, -1, 1)‘).
`rhs`	Right-hand side of the hypothesis equations. Default is ‘0‘.
`test`	A character string specifying the type of test to perform. Options are ‘"F"‘ (default) or ‘"Chisq"‘.
`vcov_func`	An optional function for specifying a custom covariance matrix. Default is ‘NULL‘.
`full`	Logical. If ‘TRUE‘, returns a tibble with full results. Default is ‘FALSE‘.

Value

A tibble with results for all tested hypotheses. Each row corresponds to one hypothesis.

Examples

```
# Example: Multiple hypotheses with a linear model
set.seed(123)
data <- data.frame(x1 = rnorm(100), x2 = rnorm(100), x3 = rnorm(100), y = rnorm(100))
model <- lm(y ~ x1 + x2 + x3, data = data)

# Perform multiple Wald tests
result <- wald_test_general_df(model, "x1 - x2 + x3 = 0", "x1 - x2 = 0", c(1, -1, 1), test = "F", full
print(result)
```

`wald_test_general_mat` *Wald Test for Hypotheses Specified as Matrices*

Description

This function performs a Wald test for general linear hypotheses on a fitted model. The hypothesis is specified as a numeric vector or matrix. The function computes the Wald statistic and associated p-value based on the input hypothesis matrix.

Usage

```
wald_test_general_mat(
  model,
  hypothesis_matrix,
  rhs = 0,
  test = c("F", "Chisq"),
  vcov_func = NULL
)
```

Arguments

`model`	A fitted model object, such as an object from ‘lm‘ or ‘glm‘.
`hypothesis_matrix`	A numeric vector or matrix representing the hypothesis to test. For example, ‘c(1, -1, 1)‘ represents the hypothesis ‘x1 - x2 + x3 = 0‘.
`rhs`	A numeric value for the right-hand side of the hypothesis equation. Default is ‘0‘.
`test`	A character string specifying the type of test to perform. Options are ‘"F"‘ (default) or ‘"Chisq"‘.
`vcov_func`	An optional function for specifying a custom covariance matrix. Default is ‘NULL‘.

Value

A tibble with the following columns: - ‘var_comparison‘: A standardized version of the hypothesis in text form. - ‘W‘: The Wald statistic. - ‘df‘: Degrees of freedom for the test. - ‘p_value‘: The p-value associated with the test. - ‘method‘: The type of test performed (‘"F"‘ or ‘"Chisq"‘). - ‘mat‘: A vectorized representation of the hypothesis matrix.

Examples

```
# Example: Linear model with Wald test
set.seed(123)
data <- data.frame(x1 = rnorm(100), x2 = rnorm(100), x3 = rnorm(100), y = rnorm(100))
model <- lm(y ~ x1 + x2 + x3, data = data)

# Perform Wald test with matrix-based hypothesis
result <- wald_test_general_mat(model, c(1, -1, 1), test = "F")
print(result)
```

```
# Perform Wald test with a custom covariance matrix
custom_vcov <- function(model) {
  vcov_matrix <- vcov(model)
  # Example: Inflate variance for demonstration
  vcov_matrix * 1.5
}
result_custom <- wald_test_general_mat(model, c(1, -1, 1), test = "F", vcov_func = custom_vcov)
print(result_custom)
```

`wald_test_general_str`

Wald Test for Hypotheses Specified as Strings

Description

This function performs a Wald test for general linear hypotheses on a fitted model. The hypothesis is specified as a string (e.g., '"x1 - x2 + x3 = 0"') and is converted into a matrix internally for computation. This allows for flexible specification of hypotheses in an intuitive text-based format.

Usage

```
wald_test_general_str(
  model,
  hypothesis,
  rhs = 0,
  test = c("F", "Chisq"),
  vcov_func = NULL
)
```

Arguments

`model`	A fitted model object, such as an object from 'lm' or 'glm'.
`hypothesis`	A character string representing the hypothesis to test. For example, '"x1 - x2 + x3 = 0"'. The right-hand side ('rhs') is automatically incorporated into the hypothesis.
`rhs`	A numeric value for the right-hand side of the hypothesis equation. Default is '0'.
`test`	A character string specifying the type of test to perform. Options are '"F"' (default) or '"Chisq"'.
`vcov_func`	An optional function for specifying a custom covariance matrix. Default is 'NULL', in which case the model's default covariance matrix is used.

Value

A tibble with the following columns: - 'var_comparison': A standardized version of the hypothesis in text form. - 'W': The Wald statistic. - 'df': Degrees of freedom for the test. - 'p_value': The p-value associated with the test. - 'method': The type of test performed ('"F"' or '"Chisq"'). - 'mat': A vectorized representation of the hypothesis matrix.

Examples

```
# Example: Linear model with Wald test
set.seed(123)
data <- data.frame(x1 = rnorm(100), x2 = rnorm(100), x3 = rnorm(100), y = rnorm(100))
model <- lm(y ~ x1 + x2 + x3, data = data)

# Perform Wald test with text-based hypothesis
result <- wald_test_general_str(model, "x1 - x2 + x3 = 0", test = "F")
print(result)
```

---

wald_test_lm *Pairwise Wald Test for Linear Model Coefficients*

Description

This function performs pairwise Wald tests for coefficients of a linear model.

Usage

```
wald_test_lm(input_data, raw = FALSE)
```

Arguments

input_data	An object of type 'lm', 'summary' of an 'lm' object, or a tidy dataframe containing 'term', 'estimate', and 'std.error' columns.
raw	Logical. If 'TRUE', returns raw results. If 'FALSE', returns formatted results with significance markings.

Value

A dataframe containing pairwise Wald test results. The output includes: - 'var_comparison': Pairwise variable comparison. - 'est1', 'se1', 'est2', 'se2': Estimates and standard errors of the variables being compared. - 'Z': Wald statistic. - 'p_value': p-value for the Wald test.

Examples

```
## Not run:
# Example with an lm object
model <- lm(mpg ~ hp + wt, data = mtcars)
wald_test_lm(model, raw = TRUE)
wald_test_lm(model, raw = FALSE)

# Example with a summary of lm object
model_summary <- summary(model)
wald_test_lm(model_summary, raw = TRUE)

# Example with a tidy dataframe
tidy_data <- broom::tidy(model)
wald_test_lm(tidy_data, raw = FALSE)

## End(Not run)
```

`web` *Functions that output to a web browser*

Description

Functions that output to a web browser

Usage

```
web(
  data,
  out = TRUE,
  tibble = TRUE,
  digits = 3,
  font_size = 14,
  file = "output.html"
)
```

Arguments

`data`	data
`out`	TRUE browser
`tibble`	show data
`digits`	default 3
`font_size`	default 14
`file`	file name default ”output.html

Value

web output

Examples

```
## Not run:
mtcars%>%web()


## End(Not run)
```

webr *data to web*

Description

data to web

Usage

```
webr(
  data,
  show = TRUE,
  digits = 3,
  font_size = 18,
  border_width = 1,
  border_color = "#ccc",
  file = "output.html"
)
```

Arguments

data	data.frame
show	TRUE: web FALSE" console
digits	round 3
font_size	14
border_width	1
border_color	"gray"
file	file = "output.html"

Value

web

Examples

```
## Not run:
#'
# Example usage
webr(mtcars)
webr(mtcars, font_size=20)

## End(Not run)
```

white_se *White* 의수정된표준오차를계산하는함수정의

Description

White 의수정된표준오차를계산하는함수정의

Usage

```
white_se(original_se, n = NULL, k = 2, type = "white")
```

Arguments

original_se	original_se
n	sample size
k	number of regression coefs
type	all, white, roburst

Value

se data

Examples

```
## Not run:
#'
# 함수 사용 예시
original_se <- 0.009619  # 예시 표준오차
n <- 100  # 예시 표본 크기
k <- 2  # 예시 회귀 계수 개수 (절편 포함)

# 함수 호출 및 결과 출력
white_se(original_se= 0.009619, n= 100, k=2)

## End(Not run)
```

wide_df *pivot_wider*

Description

pivot_wider

Usage

```
wide_df(
  data,
  names_from = "names",
  values_from = "values",
  names_prefix = "",
  names_sep = "_",
  names_sort = FALSE,
  rm_cols = NULL,
  add_id = NULL
)
```

Arguments

data	long format data
names_from	names_from
values_from	values_from
names_prefix	names_prefix +variable
names_sep	sep
names_sort	colnames sort
rm_cols	Remove variables that interfere with wide format by creating individual IDs
add_id	Select Repeating Groups to create IDs, line in column names or column numbers

Value

wide data

Examples

```
## Not run:
mtcars[1:3, 1:7]
mtcars[1:3, 1:7] %>% long_df()
mtcars[1:3, 1:7] %>% long_df(cols= 3:7)
mtcars[1:3, 1:7] %>% long_df(fix=1:2)
mtcars[1:3, 1:7] %>% long_df(fix= 1:3, rownames_to_column=F)

mtcars[1:3, 1:7] %>% long_df() %>% mutate(ik = "bc", io = "a")

#wide_df
mtcars[1:3, 1:7] %>% long_df() %>% mutate(ik = "bc", io = "a")%>% wide_df()

mtcars[1:3, 1:7] %>% long_df() %>%
  mutate(ik = "bc", io = "a")%>%
  wide_df(names_from = c("names","ik"))


mtcars[1:3, 1:7] %>% long_df() %>%
  mutate(ik = rownames, io = "a")%>%
  wide_df(names_from = c("ik"))%>%pall()
```

```
dfa = mtcars[1:3, 1:7] %>% long_df()

dfa%>%
  mutate(ik = "bc", io = "a")%>%
#'   wide_df(names_from = 2)


#일반적인 데이터에서  롱포맷으로 바꾸기
data <- data.frame(
  weight = c(4.17, 5.58, 5.18, 6.11, 4.50, 4.61, 5.17, 4.53, 5.33, 5.14,
             4.81, 4.17, 4.41, 3.59, 5.87, 3.83, 6.03, 4.89, 4.32, 4.69,
             6.31, 5.12, 5.54, 5.50, 5.37, 5.29, 4.92, 6.15, 5.80, 5.26),
  group = c(rep("ctrl", 10), rep("trt1", 10), rep("trt2", 10))
)
data
data%>%add_rep_id("group")

data%>%add_rep_id("group")%>%jjstat::as_trt("id") %>%wide_df("group", "weight")

data  %>%wide_df("group", "weight", add_id="group")


#개별적인 그룹을 만들어진 경우
data2 <- data.frame(
  weight = c(4.17, 5.58, 5.18, 6.11, 4.50, 4.61, 5.17, 4.53, 5.33, 5.14,
             4.81, 4.17, 4.41, 3.59, 5.87, 3.83, 6.03, 4.89, 4.32, 4.69,
             6.31, 5.12, 5.54, 5.50, 5.37, 5.29, 4.92, 6.15, 5.80, 5.26),
  ids = c(paste0("C", 1:10), paste0("T",1:10), paste0("S", 1:10))
)

data2

data2 %>%jjstat::mutate_col(group= c(rep("ctrl", 10),
                 rep("trt1", 10), rep("trt2", 10)), col=2)

# 그룹을 나누어주는 아이디 생성
data2 %>%
jjstat::mutate_col(group= c(rep("ctrl", 10),
        rep("trt1", 10), rep("trt2", 10)), col=2)%>%
        add_rep_id("group")

# 개별적인 그룹을 만드는 열이 존재하는 경우 --> 문제발생
data2 %>%
    jjstat::mutate_col(group= c(rep("ctrl", 10),
          rep("trt1", 10), rep("trt2", 10)), col=2)%>%
          wide_df("group", "weight", add_id="group")

#개별적인 그룹을 만드는 열을 제거하는 경우 --> 문제해결
data2 %>%
  jjstat::mutate_col(group= c(rep("ctrl", 10),
  rep("trt1", 10), rep("trt2", 10)),  col=2)%>%
  wide_df("group", "weight", add_id="group", rm_cols="ids") %>%print(n=Inf)
#'
data3= data.frame(
  weight=c(1:9),
  group=c(rep("a",3),rep("b",3),rep("c",3))
)
```

```
data3
data3 %>% wide_df("group", "weight", add_id=2)

## End(Not run)
```

`write_text` *write_text: Save R Object as Text File and Provide Read Instructions*

Description

write_text: Save R Object as Text File and Provide Read Instructions

Usage

```
write_text(data, file_path = NULL, folder_open = TRUE)
```

Arguments

`data`	Any R object to be saved as a text file.
`file_path`	Optional. A string specifying the file path where the text file will be saved. If not provided, the file name is derived from the object's name with ".text" extension.
`folder_open`	Logical indicating whether to open the folder containing the saved file. Defaults to TRUE.

Value

NULL. The function saves a text file and optionally opens the folder where it is saved.

Examples

```
## Not run:
# Example: Save a summary of a linear model
lmres <- lm(mpg ~ hp + wt, data = mtcars) %>% summary()
write_text(lmres)
#'
#'
lmres = lm(mpg~ hp+wt, data=mtcars) %>% summary()
write_text(lmres)

# resultlm <- read_text("F:/Rwork/07 futureDuck/2024FEI/lmres.text")
# resultlm
resultlm <- readLines("F:/Rwork/07 futureDuck/2024FEI/lmres.text")



## End(Not run)
```

ztest *Z test compare coefficient*

Description

Z test compare coefficient

Usage

```
ztest(b1, se1, b2, se2, type = "res", digits = 3)
```

Arguments

b1	b1 coeff
se1	se1 standard error
b2	b2 coeff
se2	se2 se
type	result type res1, res2
digits	rounding defualt 3

Value

z value and p value

Examples

```
## Not run:
# hp, wt
ztest(b1= -0.032230,se1= 0.00892,   b2=-3.23 , se2= 0.796 )

## End(Not run)
```

ztest_df *Z-test using data.frame(or tibble)*

Description

Z-test using data.frame(or tibble)

Usage

```
ztest_df(data, row1 = 2, row2 = 3, rname1 = "", rname2 = "", sel = "row")
```

Arguments

`data`	lm() and data.frame
`row1`	row1 = 2
`row2`	row2 = 3
`rname1`	select first row name
`rname2`	select second row name
`sel`	row is number, term rowname

Value

diff, z, p value

Examples

```
## Not run:
 lm(mpg ~ hp + wt + drat, mtcars) %>%
 ztest_df()

# #selecting variable
lm(mpg ~ hp + wt + drat, mtcars) %>%
   ztest_df(row1=2, row2=3)
#
# # When other variables are selected and analyzed
 lm(mpg ~ hp + wt + drat, mtcars) %>%
   ztest_df(row1=2, row2=4)
#
# # When variable-names are selected
 lm(mpg ~ hp + wt + drat, mtcars) %>%
   ztest_df(sel="term", rname1="hp", rname2="wt")
#
 lm(mpg ~ hp + wt + drat, mtcars) %>%
   ztest_df(sel="term", rname1="hp", rname2="drat")

## End(Not run)
```